

Vector Databases and AI Applications

pgvector, Pinecone, Weaviate and the New Database Layer

Rajesh Guntupalli

Contents

pgvector, Pinecone, Weaviate and the New Database Layer	1
Chapter 1: What Are Embeddings and Why Databases Need Them	3
From Keywords to Coordinates: How Machines Represent Meaning	3
Anatomy of an Embedding: What the Numbers Actually Represent	4
Why Traditional Indexes Fail for Vector Search	5
Storing Vectors in PostgreSQL with pgvector	6
The Ecosystem Map: pgvector, Pinecone, Weaviate, and When to Use Each	8
Key Takeaways	9
Chapter 2: How Vector Search Works: ANN vs Exact Search	11
The Geometry of Similarity: What Vector Search Is Actually Doing	11
Exact Search (KNN): When Brute Force Is the Right Answer	12
Approximate Nearest Neighbor Search: The Algorithmic Landscape	13
HNSW in pgvector and Weaviate: Configuration That Actually Matters	15
IVFFlat for Batch Workloads and the Recall-Speed Tradeoff in Practice	17
Recall, Latency, and Throughput: Benchmarking Your Search Configuration	18
Filtering During Vector Search: The Pre-Filter vs Post-Filter Distinction	19
Distance Operators in SQL: A Reference for pgvector Queries	21
Key Takeaways	22
Chapter 3: Distance Metrics: Cosine, Euclidean, Dot Product	25
3.1 What Distance Metrics Actually Measure (And Why It's Not Just Geometry)	25
3.2 Cosine Similarity in Practice: When Angle Is All That Matters	27
3.3 Euclidean Distance: When Magnitude Carries Meaning	28
3.4 Dot Product: Maximum Throughput for Recommendation and Ranking Systems	30
3.5 Choosing the Right Metric for Your Embedding Model and Use Case	32
3.6 Metric Configuration Across Vector Databases: A Practical Reference	35
3.7 Distance Metrics and HNSW Index Quality: The Hidden Interaction	38
3.8 Normalization Strategies: Write-Time vs Query-Time vs None	39
3.9 Distance Metrics in Multi-Vector and Sparse+Dense Hybrid Systems	41
Key Takeaways	43
Chapter 4: The Vector Database Landscape: pgvector, Pinecone, Weaviate, Qdrant, Chroma	45
The Core Decision Framework: Where Vector Search Lives in Your Stack	45
pgvector: Vector Search Inside PostgreSQL	46

Pinecone: Managed Vector Search as a Service	48
Weaviate: Hybrid Search and the Knowledge Graph Model	50
Qdrant and Chroma: Performance-First and Developer-First	52
Side-by-Side: Connection Patterns and Query Structures	55
Operational Comparison: Deployment, Scaling, and Durability	56
Choosing Between Systems: A Decision Tree	57
Key Takeaways	59
Chapter 5: When to Use pgvector vs a Dedicated Vector Database	61
5.1 Understanding What You Are Actually Choosing Between	61
5.2 pgvector in Practice: Setup, Indexing, and Performance Baselines	62
5.3 When pgvector Is the Right Answer	65
5.4 When Dedicated Vector Databases Win	66
5.5 Hybrid Approaches and Migration Paths	68
5.6 Making the Decision: A Practical Framework	69
Key Takeaways	70
Chapter 6: Data Types in Vector Storage: Text, Images, Audio, Code	71
6.1 Text Embeddings: The Dominant Workload	71
6.2 Image Embeddings: Visual Similarity at Database Scale	74
6.3 Audio Embeddings: Time-Series Signals as Vectors	76
6.4 Code Embeddings: Semantic Search Across Source Files	78
6.5 Multi-Modal Pipelines: Combining Modalities in One Index	81
6.6 Distance Metrics by Modality	83
6.7 Storage Sizing and Cost Modeling	84
Key Takeaways	85
Chapter 7: Installing and Configuring pgvector	87
Getting pgvector Installed	87
The vector Data Type and Basic Schema Design	88
Distance Operators and Query Patterns	89
Building and Tuning Indexes	90
postgresql.conf Tuning for Vector Workloads	92
Populating Embeddings from Python	94
Hybrid Search: Combining Vector and Full-Text	96
Partitioning Large Embedding Tables	97
Monitoring Vector Query Performance	98
Security and Multi-Tenancy Patterns	99
Upgrading pgvector	100
Key Takeaways	100
Chapter 8: Creating and Storing Embeddings in PostgreSQL	103
8.1 How Embeddings Fit Into a PostgreSQL Schema	103
8.2 Generating Embeddings with the OpenAI API	104
8.3 Using Open-Source Embedding Models	107
8.4 Building the Vector Index	109
8.5 Querying Embeddings in Production	111
8.6 Schema Evolution and Model Migration	116

8.7 Operational Monitoring and Maintenance	117
Key Takeaways	119
Chapter 9: HNSW vs IVFFlat Indexing: Performance Tradeoffs	121
How Vector Indexes Work: The ANN Problem	121
IVFFlat: The Inverted Index Approach	122
HNSW: The Graph Navigation Approach	123
Head-to-Head Comparison: When to Choose Which	125
Managed Vector Databases: How They Expose These Tradeoffs	127
Production Tuning Playbook	129
Filtering and the ANN Problem	132
Choosing Between Indexes: A Decision Framework	133
SQL Server Context: What the Ecosystem Looks Like	133
Key Takeaways	134
Chapter 10: Tuning pgvector for High-Throughput Workloads	137
Understanding How HNSW and IVFFlat Trade Recall for Speed	137
PostgreSQL Server Configuration for Vector Workloads	138
Scaling Ingestion: Bulk Loading Without Choking the Index	140
Hybrid Search Architecture: Combining Vector and Scalar Filters	142
Connection Pooling and Concurrency Patterns for AI Applications	144
Recall Measurement and Regression Testing	146
Observability: Exposing pgvector Metrics to Prometheus	147
Key Takeaways	149
Chapter 11: Hybrid Search: Combining Full-Text and Vector Search	151
11.1 Why Neither Approach Alone Is Sufficient	151
11.2 Scoring Fusion: RRF, Linear Combination, and When to Use Each	152
11.3 Hybrid Search in pgvector + PostgreSQL Full-Text Search	153
11.4 Hybrid Search in Pinecone	156
11.5 Hybrid Search in Weaviate	159
11.6 Filtering in Hybrid Search	161
11.7 Evaluating Hybrid Search Quality	163
11.8 Operational Considerations: Latency, Cost, and Freshness	165
11.9 A Complete End-to-End Example	166
Key Takeaways	169
Chapter 12: Multi-Tenant Vector Storage in PostgreSQL	171
12.1 The Three Models of Multi-Tenancy (and Where Each Breaks Down)	171
12.2 Implementing Row-Level Vector Isolation	173
12.3 Schema-Per-Tenant Architecture	175
12.4 Index Management at Tenant Scale	177
12.5 Cross-Tenant Query Routing	181
12.6 Tenant-Aware Connection Pooling	182
12.7 Scaling Considerations and Performance Cliffs	184
12.8 A Complete Working Example	185
Key Takeaways	188

Chapter 13: pgvector on RDS, Aurora, AlloyDB, and Supabase	189
13.1 Platform Landscape: What You’re Actually Getting	189
13.2 Enabling pgvector and Schema Design on Managed Services	190
13.3 Index Strategy: HNSW vs. IVFFlat on Managed Infrastructure	192
13.4 Ingestion Pipelines: Embedding Generation and Bulk Loading	193
13.5 Query Patterns, Connection Pooling, and Performance Tuning	195
13.6 Aurora Serverless v2 and Cost Architecture	198
13.7 AlloyDB ML Integration: Embedding Without Leaving SQL	199
13.8 Monitoring and Observability	201
13.9 Migration Path: Moving Between Platforms	202
13.10 Practical Checklist Before Going to Production	203
Key Takeaways	204
Chapter 14: Monitoring and Diagnosing pgvector Query Performance	207
14.1 Understanding How pgvector Executes Similarity Queries	207
14.2 Core Query Diagnostics: EXPLAIN, ANALYZE, and Execution Statistics	208
14.3 Index Health and Build Quality	209
14.4 Diagnosing the Scalar-Filter Interaction Problem	211
14.5 Production Observability: Metrics, Alerting, and Continuous Profiling	212
14.6 Recall Measurement in Production	216
14.7 Index Maintenance Scheduling	218
14.8 Putting It Together: A Monitoring Runbook	219
Key Takeaways	222
Chapter 15: Pinecone: Architecture, Indexing, and Namespaces	223
15.1 Pinecone’s Architecture Through a Database Lens	223
15.2 Indexing Algorithms: What HNSW Means at the Pinecone Layer	225
15.3 Upserting Vectors and Working with Metadata	226
15.4 Namespaces: Multi-Tenancy Without Schema Separation	229
15.5 Production Operations: Monitoring, Scaling, and Failure Patterns	231
15.6 Advanced Patterns: Hybrid Search and Re-Ranking	235
15.7 Integrating Pinecone with Your Data Stack	238
Key Takeaways	242
Chapter 16: Weaviate: Schema Design and Hybrid Search	245
16.1 Weaviate’s Data Model: Classes, Properties, and the Object Graph	245
16.2 Schema Design Patterns for Production RAG Systems	247
16.3 Hybrid Search: BM25 + Vector Fusion	248
16.4 Multi-Tenancy: One Schema, Many Isolated Datasets	250
16.5 Batch Ingestion, HNSW Tuning, and Query Performance	252
16.6 Filtering Strategies: Pre-Filter vs. Post-Filter	253
16.7 Generative Search and RAG Modules	255
16.8 Backup, Restore, and Operational Concerns	256
16.9 Weaviate vs. pgvector: Decision Framework	257
Key Takeaways	258
Chapter 17: Qdrant: Payload Filtering and Precision Tuning	259
17.1 Qdrant’s Data Model: Collections, Points, and Payloads	259

17.2 Payload Filtering: From Simple Conditions to Complex Predicates	262
17.3 Filtered HNSW: How Qdrant Avoids the Post-Filter Penalty	264
17.4 Precision Tuning: ef, Quantization, and Segment Configuration	266
17.5 Named Vectors and Multivector Collections	269
17.6 Scroll, Recommendation, and Discovery APIs	271
17.7 Production Deployment Patterns	272
17.8 Practical Integration Pattern: Tenant-Isolated Search	274
Key Takeaways	277
Chapter 18: Chroma: Local-First Vector DB for Development	279
18.1 Chroma’s Architecture: An Embedded Database Perspective	279
18.2 Getting Started: Collections, Embeddings, and Basic Queries	280
18.3 Embedding Functions: Wiring Chroma to Your Model Layer	282
18.4 Client/Server Mode and LangChain Integration	284
18.5 HNSW Tuning, Scaling Limits, and When to Graduate	286
18.6 Real RAG Pipeline with Chroma: End-to-End Example	289
18.7 Migrating from Chroma to a Production Vector Database	293
18.8 Chroma in CI/CD: Testing Vector Features Without a Server	296
Key Takeaways	299
Chapter 19: Milvus: Open-Source Vector Search at Enterprise Scale	301
19.1 Milvus Architecture: Disaggregated Storage and Compute	301
19.2 Deployment: From Docker Compose to Kubernetes	302
19.3 Collection Design, Indexing, and Schema Modeling	305
19.4 Querying: ANN Search, Filtered Search, and Hybrid Search	308
19.5 Ingestion Pipelines and Data Management	311
19.6 Multi-Tenancy Patterns	314
19.7 Operational Concerns: Monitoring, Backup, and Upgrades	316
19.8 Milvus vs. Alternatives: When to Choose What	318
Key Takeaways	319
Chapter 20: Benchmarking Vector Databases: Throughput, Latency, Recall	321
The Three Metrics That Define Vector Search Performance	321
Constructing a Realistic Benchmark Dataset	322
Building a Multi-Database Benchmark Harness	325
pgvector: Index Tuning for Production Throughput	331
Weaviate and Milvus: Distributed Vector Search Benchmarking	333
Filtered Search Benchmarking: The Production Reality Check	334
Interpreting Benchmark Results: Common Traps	336
Continuous Benchmarking in CI/CD	337
Choosing Between Databases: A Decision Framework	339
Key Takeaways	339
Chapter 21: What is RAG and Why It Needs a Vector Database	341
21.1 The Problem RAG Solves	341
21.2 RAG Architecture: The Three-Stage Pipeline	342
21.3 Embeddings: The Translation Layer Between Text and Geometry	344
21.4 Vector Search Mechanics and Why It Beats Full-Text for Retrieval	345

21.5 Building a Production RAG Pipeline with Pinecone and LlamaIndex	347
21.6 RAG Failure Modes and What the Database Can Fix	349
21.7 Choosing Your Vector Database for RAG	352
21.8 Chunking Strategies and Their Database Implications	354
Key Takeaways	356
Chapter 22: Chunking Strategies: Size, Overlap, Semantic Splitting	359
22.1 Why Chunking Is a Database Design Problem	359
22.2 Fixed-Size Chunking: Mechanics and Limits	360
22.3 Overlap Design: What Gets Repeated and Why	361
22.4 Structural and Semantic Splitting	363
22.5 Ingesting Chunks into pgvector and Pinecone	365
22.6 Querying Across Chunk Strategies	368
22.7 Choosing a Chunking Strategy by Document Type	370
22.8 Measuring Chunking Quality	371
22.9 Operational Considerations: Re-Chunking and Schema Evolution	373
Key Takeaways	374
Chapter 23: Embedding Model Selection: OpenAI, Cohere, Sentence Transformers	377
What an Embedding Model Actually Does (and Why It Matters to a DBA)	377
OpenAI Embeddings: Production Workhorse	378
Cohere Embeddings: Retrieval-Optimized with Input Type Awareness	380
Sentence Transformers: Local Embeddings for Privacy and Cost Control	382
Comparing Models: Quality, Cost, and Latency at Production Scale	385
Model Versioning and Migration Strategy	387
Choosing the Right Model: A Decision Framework	388
End-to-End Example: Evaluating Two Models Side by Side	389
Key Takeaways	390
Chapter 24: Building a RAG Pipeline on PostgreSQL and pgvector	393
24.1 RAG Architecture Through a Database Lens	393
24.2 Schema Design for RAG Documents	394
24.3 The Ingestion Pipeline	395
24.4 The Retrieval Layer	398
24.5 LLM Integration and Prompt Assembly	401
24.6 Multi-Tenancy and Row-Level Security	403
24.7 Evaluation: Knowing Whether Your RAG Pipeline Works	405
24.8 Production Patterns and Operational Checklist	406
Key Takeaways	408
Chapter 25: Re-ranking Results for Accuracy	411
25.1 Why Vector Search Alone Is Not Enough	411
25.2 Cross-Encoder Architecture and Why It Works Better	412
25.3 Re-ranking with Cohere and Production API Integration	414
25.4 Implementing Re-ranking in pgvector Pipelines	416
25.5 LLM-Based Re-ranking and Reciprocal Rank Fusion	418
25.6 Production Patterns: Latency, Caching, and Model Selection	420
25.7 Evaluating Re-ranking Quality	422

25.8 End-to-End Pipeline: pgvector + Cross-Encoder + RAG Generation	424
Key Takeaways	426
Chapter 26: Caching Embeddings: Avoiding Redundant API Calls	429
Why Embeddings Are Unusually Cacheable	429
Cache Architecture: Where Embeddings Live	430
Implementing a Redis Embedding Cache	431
PostgreSQL as a Persistent Embedding Cache	434
Cache Invalidation: The Hard Part	437
LangChain and LlamaIndex Cache Integration	438
Semantic Cache: A Step Beyond Exact Match	440
Monitoring Cache Performance	442
Cache Warming Strategies	444
Key Takeaways	445
Chapter 27: Evaluating RAG Quality: Metrics That Matter	447
The RAG Quality Problem: Why “It Feels Right” Doesn’t Scale	447
Retrieval Metrics: Measuring What the Vector Search Returns	448
Generation Metrics: Faithfulness, Relevance, and Hallucination Detection	450
End-to-End Correctness: Answer Quality at Scale	452
Storing and Trending Evaluation Results in PostgreSQL	453
Production Monitoring: Online Metrics and Feedback Loops	455
Building an Evaluation-Driven Development Loop	458
A Note on Evaluation Dataset Maintenance	459
Key Takeaways	459
Chapter 28: Indexing Pipelines: Keeping Vectors Fresh	461
The Vector Indexing Lifecycle	461
Building the Embedding Generation Pipeline	462
Change Data Capture for Continuous Indexing	464
Index Optimization and Segment Management	467
Handling Embedding Model Changes	468
Recall Degradation Detection	472
Backfill Strategies for Large Datasets	474
Key Takeaways	476
Chapter 29: Real-Time vs Batch Embedding Generation	479
29.1 Understanding the Embedding Generation Pipeline	479
29.2 Real-Time Embedding Generation	480
29.3 Batch Embedding Generation	483
29.4 Hybrid Architectures: Combining Both Strategies	486
29.5 Cost Optimization and Model Selection	489
29.6 Monitoring and Observability	491
29.7 Re-Embedding and Index Migration	493
29.8 Choosing Your Strategy: A Practical Framework	495
Key Takeaways	496
Chapter 30: Handling Embedding Model Version Changes	499

Why Model Changes Break Vector Search	499
Detecting Degradation Before and After Migration	500
Migration Strategies: Zero-Downtime Approaches	503
Backfill Job Design: Resumability and Rate Limiting	509
Handling Dimension Changes: The Hard Case	512
Version Pinning and Model Deprecation Tracking	516
Key Takeaways	518
Chapter 31: Multi-Modal Search: Text and Image Together	519
The Embedding Bridge: How Text and Images Share a Vector Space	519
Storage Architecture for Mixed-Modal Data	520
Ingestion Pipelines: Embedding at Scale	523
Query Patterns: Fusing Text and Image Signals	526
Multi-Modal Search in pgvector: The SQL Perspective	529
Score Normalization Before Fusion	531
Handling Queries Without an Image	533
Pinecone: Multi-Modal with Namespaces and Metadata Filtering	534
Production Considerations: Latency, Cost, and Quality Tradeoffs	537
A Complete End-to-End Example: Fashion Visual Search	537
Key Takeaways	539
Chapter 32: Graph and Vector: Combining Relationship and Semantic Search	541
Why Graph Structure and Vector Similarity Serve Different Retrieval Needs	541
Native Graph-Vector Databases: Weaviate, Neo4j Vector, and Amazon Neptune Analytics	542
pgvector Meets Recursive CTEs: Graph-Vector Hybrid in PostgreSQL	544
Graph-Aware RAG: Using Relationship Context in LLM Pipelines	546
Choosing Between Graph-Vector Integration Strategies	549
Implementing the Application-Layer Orchestration Pattern	550
SQL Server Compatibility: Graph Tables and Vector Search	554
Evaluating Hybrid Retrieval Quality	556
Key Takeaways	558
Chapter 33: Semantic Caching for LLM Applications	561
33.1 How Semantic Caching Differs from Conventional Caching	561
33.2 Building a Semantic Cache with pgvector	563
33.3 Semantic Caching with Redis and a Dedicated Vector Store	567
33.4 LangChain and LlamaIndex Integration	570
33.5 Cache Hit Rate Optimization	573
33.6 Production Operations: Monitoring, Eviction, and Invalidation	577
33.7 Semantic Caching for Agentic and Multi-Turn Workloads	580
33.8 Failure Modes and Mitigations	582
Key Takeaways	583
Chapter 34: Access Control on Vector Data	585
Why Vector Access Control Is Different	585
Row-Level Security in pgvector	586
Namespace Isolation in Pinecone and Weaviate	589
Metadata Filtering as a Security Predicate	592

API Key Management and Least-Privilege Design	594
Audit Logging for Vector Queries	596
Handling Sensitive Data in Embedding Pipelines	598
A Unified Authorization Layer	600
Key Takeaways	603
Chapter 35: Disaster Recovery for Vector Stores	605
The Unique DR Challenges of Vector Stores	605
Backup Strategies by Vector Store Platform	606
Replication and High Availability Topologies	613
Cross-Region Failover Runbook	617
Recovery Point Objective and Recovery Time Objective Planning	622
The Re-Embedding Recovery Path	623
Testing Your DR Plan	627
Synchronizing Vector Stores with Source Systems After Recovery	632
DR Runbook Checklist	634
Key Takeaways	634
Chapter 36: Backup Strategies for Embedding Data	637
The Three-Artifact Problem	637
Backup Strategies for pgvector	638
Exporting from Managed Vector Databases	641
Model Versioning as a Backup Dependency	644
Recovery Procedures and RTO Planning	646
Backup Schedules and Retention Policies	649
Disaster Recovery Runbook	650
Key Takeaways	651
Chapter 37: Cost Optimization: Dimensionality Reduction and Quantization	653
The Cost Structure of Vector Storage	653
Dimensionality Reduction: PCA, Matryoshka, and Native Truncation	654
Scalar Quantization: float32 to int8	656
Product Quantization and Vector Database Native Support	658
Measuring the Tradeoffs: A Recall Benchmarking Framework	660
Combining Both Techniques: Dimension Reduction + Quantization	662
pgvector Optimization Workflow	664
Operational Checklist: Before You Compress	666
Key Takeaways	666
Chapter 38: Edge Deployment of Vector Search	669
What “Edge” Means for Vector Search	669
Choosing a Vector Engine for Constrained Environments	670
Index Synchronization: Keeping Edge in Sync with Central	674
Offline-First Application Architecture	677
WebAssembly: Bringing Vector Search to the Browser	681
Security Considerations at the Edge	684
Benchmarking Edge Hardware	684
Putting It Together: A Retail Edge Deployment	686

Key Takeaways	687
Chapter 39: The Future: Hybrid Relational-Vector Databases	689
The Convergence Is Already Happening	689
Hybrid Query Architectures: SQL Meets Semantic Search	690
Vector-Native Extensions to the SQL Standard	693
Dedicated Vector Databases: Where They Remain Superior	696
The Next Architectural Patterns	699
The Operational Convergence: One Platform, Many Workloads	703
The Five-Year Horizon: Predictions and Preparation	704
Choosing Your Architecture Today	706
Key Takeaways	707
Chapter 40: Building a Production Vector DB Monitoring Dashboard	709
The Three-Layer Monitoring Model	709
Instrumenting pgvector with Prometheus and Custom Metrics	710
Monitoring Pinecone and Weaviate via Their Native APIs	714
Embedding Quality and Recall Probes	717
Building the Golden Dataset	722
Alerting Rules and Escalation Thresholds	723
Grafana Dashboard Layout	726
Handling Embedding Model Rotation	727
Putting It All Together: The Monitoring Stack Deployment	729
Key Takeaways	731

pgvector, Pinecone, Weaviate and the New Database Layer

By Rajesh Guntupalli

Last updated: July 30, 2026

Chapter 1: What Are Embeddings and Why Databases Need Them

If you have spent years designing schemas, tuning indexes, and optimizing query plans, the idea of storing lists of floating-point numbers as a first-class database primitive probably sounds strange. This chapter explains why that instinct is worth revisiting. Embeddings are the data structure at the center of modern AI applications — from semantic search and recommendation engines to retrieval-augmented generation (RAG) pipelines — and understanding them at the infrastructure level is what separates engineers who bolt AI onto existing systems from those who build systems that actually perform at scale.

From Keywords to Coordinates: How Machines Represent Meaning

Traditional databases store meaning in structure. A product with `category = 'running shoes'` and `brand = 'Nike'` communicates meaning through schema design, foreign keys, and the queries you write against them. The database itself has no concept of what “running shoes” means — it only knows that two rows share the same string value in a column.

Natural language does not work this way. When a user searches for “comfortable sneakers for long runs,” no keyword match will reliably surface products described as “cushioned athletic footwear for distance training.” The words are different. The meaning is the same.

Embeddings solve this by converting text, images, audio, or any structured content into points in a high-dimensional vector space, where proximity encodes semantic similarity. Instead of asking “do these two strings share characters?”, you ask “how close are these two points?”

A vector embedding is a fixed-length array of floating-point numbers — typically 384 to 3,072 dimensions depending on the model. Each dimension captures some latent feature learned during model training. No individual dimension has a human-readable meaning; the meaning emerges from the geometry of the entire space.

For a database professional, the most useful mental model is this: **an embedding is a computed column that captures semantic meaning, generated by a neural network rather than a SQL expression.** Just as a computed column like `LOWER(email)` transforms a raw value into a normalized form optimized for a specific access pattern, an embedding transforms raw content into a form optimized for similarity-based retrieval.

The critical difference is that you cannot reconstruct the original content from an embedding (it is a lossy, one-way transformation), and the “similarity” it enables is not byte-level equality — it is semantic relatedness, measured geometrically.

Anatomy of an Embedding: What the Numbers Actually Represent

To reason about embeddings as infrastructure, you need a concrete picture of what they look like and where they come from.

An embedding model takes an input — a sentence, a product description, a code snippet, an image — and produces a dense vector. “Dense” means nearly all dimensions have non-zero values, which distinguishes embeddings from the sparse TF-IDF or bag-of-words vectors that predated deep learning. Here is what generating an embedding looks like in practice using OpenAI’s API:

```
# Python example - generating an embedding with OpenAI
from openai import OpenAI

client = OpenAI() # uses OPENAI_API_KEY from environment

text = "Cushioned athletic footwear for long-distance running"

response = client.embeddings.create(
    input=text,
    model="text-embedding-3-small" # 1536 dimensions, ~$0.02 per 1M tokens
)

vector = response.data[0].embedding
print(f"Dimensions: {len(vector)}") # 1536
print(f"First 5 values: {vector[:5]}") # [-0.012, 0.043, -0.071, 0.019, 0.055]
```

That list of 1,536 floats is the embedding. Store it alongside the original text, and you have the foundation of a semantic search system.

For local or self-hosted workloads — important when data cannot leave your environment — the `sentence-transformers` library gives you access to hundreds of open-source models:

```
# Python example - generating embeddings locally with sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2") # 384 dimensions, fast, runs on CPU

sentences = [
    "Cushioned athletic footwear for long-distance running",
    "Comfortable sneakers for long runs",
    "Men's formal leather dress shoes",
]

embeddings = model.encode(sentences)

print(f"Shape: {embeddings.shape}") # (3, 384)

# Semantic similarity - closer to 1.0 means more similar
from sklearn.metrics.pairwise import cosine_similarity
sim_matrix = cosine_similarity(embeddings)

print(f"Sentence 0 vs 1: {sim_matrix[0][1]:.4f}") # high - very similar
print(f"Sentence 0 vs 2: {sim_matrix[0][2]:.4f}") # low - dissimilar
```

The output is telling. Sentences 0 and 1 use completely different words but score high on similarity. Sentence 2, which shares some nouns with sentence 0 (“footwear,” “shoes”), scores much lower

because the semantic context is entirely different. (Exact scores depend on model version and are illustrative here, not reproduced from a real run.) No traditional index — B-tree, full-text, or otherwise — can produce this result.

A few properties of embedding vectors that matter operationally:

Dimensionality determines storage and compute cost. A 1,536-dimension float32 vector occupies 6,144 bytes — about 6 KB per row. One million vectors requires roughly 6 GB of raw storage before indexes. A 3,072-dimension model (like `text-embedding-3-large`) doubles that cost for modest accuracy gains. Choosing the right model is an infrastructure decision as much as an accuracy decision.

The embedding space is model-specific. Vectors from `text-embedding-3-small` and `all-MiniLM-L6-v2` live in completely different coordinate spaces and cannot be compared. All vectors in a collection must come from the same model. This has significant operational implications: upgrading your embedding model means re-embedding your entire dataset.

Normalization affects which distance metric to use. Many models produce L2-normalized vectors (unit vectors), for which cosine similarity and dot product are equivalent. Some do not. Know your model’s output characteristics before choosing a distance metric — the wrong choice degrades search quality silently.

Why Traditional Indexes Fail for Vector Search

At this point a reasonable question is: why not just store vectors in a standard `FLOAT[]` column and query them with a computed similarity? PostgreSQL has supported array types for decades.

You can do this. For small datasets it even works. The problem is that similarity search over vectors does not have the algebraic properties that make B-tree and hash indexes effective.

A B-tree index works because values have a total ordering: for any two keys, one is definitively greater than, equal to, or less than the other. Queries like `WHERE price < 100` carve off a predictable subtree. Vector similarity has no total ordering. Asking “which of these 10 million vectors is closest to my query vector?” requires computing distance to every candidate — a full table scan — unless you use specialized data structures.

Consider what a naive implementation costs. At 10 million vectors of 1,536 dimensions each:

- Full scan: $10\text{M} \times 1,536$ multiplications + additions per query
- At ~10 billion FLOPs per second on a single CPU core, that is roughly 1.5 seconds per query
- That is before any I/O overhead

This is why approximate nearest neighbor (ANN) algorithms exist, and why vector databases exist as a category. The two dominant index types you will encounter throughout this book are:

HNSW (Hierarchical Navigable Small World): Builds a layered graph structure where each node connects to its nearest neighbors. Search traverses from coarse upper layers to precise lower layers. Query time is $O(\log n)$ with excellent recall (typically 95–99%) at the cost of significant build time and memory. HNSW is the right default for most production workloads.

IVFFlat (Inverted File with Flat quantization): Partitions the vector space into k clusters (Voronoi cells) at index build time. A query searches only the `nprobe` nearest clusters rather than the full dataset. Much faster to build and lighter on memory than HNSW, but recall depends heavily on the `nprobe` parameter. IVFFlat works well for large, relatively static datasets where you can afford an offline index rebuild.

The tradeoff table that matters in practice:

	HNSW	IVFFlat
Build time	Slow (minutes–hours for 10M vectors)	Fast (seconds–minutes)
Memory	High (overhead scales with graph connectivity, i.e. <code>m</code>)	Low
Query speed	Very fast	Moderate (tunable)
Recall at default settings	~97%	~85–95%
Suitable for	Real-time, high-QPS	Batch, large static corpora

Neither algorithm guarantees finding the exact nearest neighbor — that is the “approximate” in ANN. For most production use cases, 95% recall at 10ms latency is far more valuable than 100% recall at 1,500ms latency. This is a tradeoff database professionals sometimes resist initially because exact results are the norm in relational systems. In vector search, approximation is an engineering choice, not a deficiency.

Storing Vectors in PostgreSQL with pgvector

PostgreSQL is where most database professionals will encounter vectors first, and `pgvector` is the extension that makes it possible. Rather than immediately reaching for a specialized vector database, understanding what `pgvector` can do — and where it hits limits — gives you the foundation to make informed infrastructure decisions throughout this book.

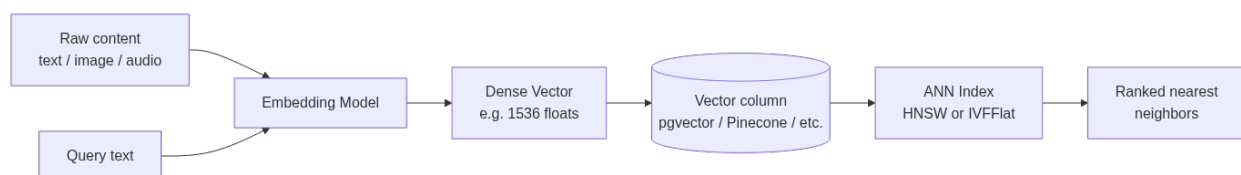


Figure 1: The same embedding model encodes both stored content and incoming queries into one shared vector space — the ANN index only has to search that space, not the original content.

Installation is straightforward:

```
# Install pgvector on Ubuntu/Debian (PostgreSQL 15+)
sudo apt install postgresql-15-pgvector

# Or compile from source
git clone https://github.com/pgvector/pgvector.git
cd pgvector
make && sudo make install
```

With the extension available, enabling it in your database and creating a table with a vector column takes seconds:

```
-- PostgreSQL / pgvector
-- Enable the extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Create a product table with an embedding column
CREATE TABLE products (
  id          BIGSERIAL PRIMARY KEY,
  sku         TEXT NOT NULL UNIQUE,
  name        TEXT NOT NULL,
  description TEXT,
  category    TEXT,
  embedding   vector(1536), -- OpenAI text-embedding-3-small dimensions
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Insert a row with a pre-computed embedding (abbreviated here)
-- In production, embeddings are populated by an application-layer pipeline
INSERT INTO products (sku, name, description, category, embedding)
VALUES (
  'NK-RUN-001',
  'Nike Air Zoom Pegasus 40',
  'Cushioned daily trainer for long-distance running',
  'running',
  '[0.012, -0.043, 0.071, ...]::vector -- 1536 values in practice
);

-- Create an HNSW index for approximate nearest neighbor search
-- m = max connections per layer, ef_construction = build quality
CREATE INDEX ON products USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Semantic similarity search: find 10 most similar products to a query vector
-- $1 is the query embedding generated at application layer
SELECT
  id,
  sku,
  name,
  category,
  1 - (embedding <=> $1) AS cosine_similarity
FROM products
WHERE category = 'running' -- pre-filter on scalar columns
ORDER BY embedding <=> $1 -- <=> is cosine distance operator
LIMIT 10;
```

The `<=>` operator computes cosine distance. `pgvector` also provides `<->` for L2 (Euclidean) distance and `<#>` for negative inner product (dot product). The choice depends on your embedding model's normalization properties.

A few things to notice in this schema. The pre-filter `WHERE category = 'running'` runs before the vector search. This is important for query performance and result relevance, but it interacts with the HNSW index in a way that is not obvious: `pgvector` performs the filter after retrieving HNSW candidates, not before. For high-selectivity filters (e.g., `WHERE user_id = 12345`), this can degrade recall because the ANN search may return candidates that are then filtered away. This is a known limitation of `pgvector`'s current architecture and a common reason teams graduate to dedicated vector databases for complex filtered searches.

When `pgvector` is enough: Up to approximately 1–2 million vectors, single-server deployments, workloads where vector search is one feature among many in a relational application, and teams that want to stay in the PostgreSQL operational model.

When to consider a dedicated vector database: Beyond 5 million vectors, high-QPS re-

quirements (>1,000 queries/second), complex metadata filtering, multi-tenant isolation, or hybrid dense+sparse search requirements.

The Ecosystem Map: pgvector, Pinecone, Weaviate, and When to Use Each

The vector database market has grown rapidly, and the naming is not always consistent. Some are databases built specifically for vectors. Others are extensions to existing systems. Still others are cloud services that abstract away the infrastructure entirely. A practical map of the landscape will ground the rest of this book.

pgvector — PostgreSQL extension. Already discussed. Best for: teams already on PostgreSQL who want to add semantic search without new infrastructure. Managed through AWS RDS, Azure Database for PostgreSQL, Supabase, and Neon.

Pinecone — Fully managed, cloud-native vector database. No infrastructure to operate. Supports hybrid search (dense + sparse), namespace-based multi-tenancy, and serverless tiers. Best for: teams that want production-grade vector search without operational overhead, and can accept SaaS pricing and data residency constraints.

Weaviate — Open-source vector database with GraphQL and REST APIs. Runs on-premises or in the cloud. Has built-in support for multiple modalities (text, image, audio) and an integrated inference API. Best for: teams needing a full object store with built-in vectorization, not just an index alongside another database.

Qdrant — Open-source, written in Rust. Excellent performance-per-dollar profile. Strong support for payload filtering with native scalar indexes that interact correctly with HNSW (solving the pre-filter problem mentioned above). Best for: self-hosted deployments where performance and operational simplicity matter.

Chroma — Lightweight, embedded-first vector store. Runs in-process with Python applications. Best for: prototyping, local development, and applications that do not need persistence beyond a single process.

Milvus — Open-source, cloud-native vector database designed for horizontal scaling. Kubernetes-native architecture with separate storage, query, and index layers. Best for: large-scale deployments requiring distributed operation and fine-grained resource management.

Here is a minimal example showing the same semantic search operation across two systems — pgvector and Pinecone — so you can see how the abstractions differ:

```
# Python example - semantic search with Pinecone
from pinecone import Pinecone
from openai import OpenAI

openai_client = OpenAI()
pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")

index = pc.Index("products-index") # must already exist

# Generate query embedding
query_text = "comfortable sneakers for long runs"
query_response = openai_client.embeddings.create(
    input=query_text,
```

```
model="text-embedding-3-small"
)
query_vector = query_response.data[0].embedding

# Search with metadata filter - Pinecone handles pre-filtering correctly
results = index.query(
    vector=query_vector,
    top_k=10,
    filter={"category": {"$eq": "running"}}, # metadata pre-filter
    include_metadata=True
)

for match in results["matches"]:
    print(f"Score: {match['score']:.4f} | SKU: {match['metadata']['sku']} | "
          f"↪ {match['metadata']['name']}")
```

The Pinecone `filter` parameter is processed as a true pre-filter against the index, not a post-filter on ANN results. This is one of the concrete operational differences between pgvector and a purpose-built system. At low vector counts the distinction is academic. At millions of vectors with selective filters it is the difference between 95% recall and 60% recall.

The right tool for your workload depends on scale, operational constraints, data residency requirements, and where vectors fit in your overall data architecture. This book covers all of them — the goal is informed selection, not advocacy for any single system.

Key Takeaways

- An embedding is a fixed-length array of floating-point numbers that represents the semantic content of text, images, or other data as a point in high-dimensional space. Two semantically similar inputs produce vectors that are geometrically close, regardless of surface-level similarity in the original content.
 - Traditional B-tree and hash indexes cannot support similarity search because vector similarity has no total ordering. Approximate nearest neighbor (ANN) algorithms — primarily HNSW and IVFFlat — solve this problem by trading exact results for dramatically lower query latency.
 - pgvector brings vector storage and ANN search into PostgreSQL, making it the natural starting point for database teams. It is well-suited for datasets under 1–2 million vectors but has known limitations around pre-filtered searches and horizontal scaling that matter at production scale.
 - The embedding model you choose determines storage cost, query performance, and which distance metric is appropriate. All vectors in a collection must come from the same model, making embedding model upgrades a significant data migration event that must be planned for from the start.
 - The vector database ecosystem — pgvector, Pinecone, Weaviate, Qdrant, Chroma, Milvus — spans a wide range of operational models from embedded libraries to fully managed cloud services. Selection criteria include dataset size, query-per-second requirements, metadata filtering complexity, and whether you need to own the infrastructure or prefer a managed service.
-

Chapter 2: How Vector Search Works: ANN vs Exact Search

Vector search is the engine underneath every modern AI application — the mechanism that turns a user’s question into a ranked list of relevant documents, products, or records. If you have spent your career building indexes on B-trees and tuning sequential scan thresholds, vector search will feel both familiar and disorienting. The data structures are different, the query semantics are different, and the performance tradeoffs are inverted in ways that will surprise you. This chapter walks through exactly how vector search works at the algorithmic level, why approximate nearest neighbor search dominates production systems, and how to make informed decisions about which algorithm and which tool to deploy for a given workload.

The Geometry of Similarity: What Vector Search Is Actually Doing

Before touching any index structure, it helps to understand what the database is computing when it runs a vector query. A vector is simply an array of floating-point numbers — typically 384 to 3072 dimensions depending on the embedding model. Each number represents some learned feature of the original content. You do not interpret the individual numbers the way you would a column value. The meaning lives in the geometric relationships between vectors.

When a user asks “what are the symptoms of iron deficiency?”, an embedding model converts that string into a vector. The database then finds vectors in storage that are geometrically close to that query vector. Closeness is measured by a distance or similarity function. Three are in common use:

Cosine similarity measures the angle between two vectors, ignoring magnitude. It is the default choice for text embeddings because the same concept expressed in a long document versus a short sentence should still score as similar.

Euclidean distance (L2) measures straight-line distance in the high-dimensional space. It works well for image embeddings and some multimodal models, but is sensitive to vector magnitude.

Dot product is the raw inner product without normalization. When vectors are unit-normalized, dot product and cosine similarity produce identical rankings. Many approximate index structures optimize for dot product because it is cheaper to compute at scale.

The choice of distance function is not just a configuration option — it changes your results. A system tuned with cosine similarity and then switched to L2 will return a different ranked list

for the same query. This matters when you are integrating vector search with an existing scoring pipeline.

```
# Python example - computing distances manually to build intuition
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def euclidean_distance(a, b):
    return np.linalg.norm(np.array(a) - np.array(b))

def dot_product(a, b):
    return np.dot(a, b)

# Simulating two embedding vectors (normally 384-3072 dims; using 8 here for clarity)
query_vec = np.array([0.12, 0.87, 0.34, 0.56, 0.91, 0.23, 0.67, 0.45])
doc_vec_a = np.array([0.11, 0.85, 0.36, 0.54, 0.89, 0.25, 0.65, 0.43])
doc_vec_b = np.array([0.90, 0.10, 0.80, 0.05, 0.12, 0.95, 0.07, 0.88])

print("=== Cosine Similarity ===")
print(f"Query vs Doc A: {cosine_similarity(query_vec, doc_vec_a):.4f}") # ~0.9998
print(f"Query vs Doc B: {cosine_similarity(query_vec, doc_vec_b):.4f}") # ~0.45

print("\n=== Euclidean Distance (lower = closer) ===")
print(f"Query vs Doc A: {euclidean_distance(query_vec, doc_vec_a):.4f}") # ~0.057
print(f"Query vs Doc B: {euclidean_distance(query_vec, doc_vec_b):.4f}") # ~1.83

print("\n=== Dot Product ===")
print(f"Query vs Doc A: {dot_product(query_vec, doc_vec_a):.4f}")
print(f"Query vs Doc B: {dot_product(query_vec, doc_vec_b):.4f}")
```

One concept that trips up database engineers is the **curse of dimensionality**. In low-dimensional spaces (2D, 3D), nearest neighbor search is intuitive and B-tree-style partitioning works well. As dimensions grow past roughly 20–30, the distance between any two random points converges toward the same value. By the time you reach 1536 or 3072 dimensions — the sizes OpenAI’s `text-embedding-3-small` and `text-embedding-3-large` produce, respectively — traditional spatial indexes like R-trees become nearly useless. Every point looks roughly equidistant from every other point when you measure raw L2, which is why embedding models are designed so that semantically similar content clusters in specific regions of the space despite the high dimensionality.

Exact Search (KNN): When Brute Force Is the Right Answer

Exact k-nearest neighbor search is conceptually simple: compute the distance from the query vector to every vector in the dataset and return the k smallest distances. This guarantees 100% recall — you will never miss a closer neighbor. In database terms, it is a full table scan with a computed distance column, sorted and limited.

pgvector supports exact search natively. If you create a table with a `vector` column and do not create an HNSW or IVFFlat index, every query runs exact KNN against the full table.

```
-- PostgreSQL / pgvector
-- Exact KNN search - no index, guaranteed 100% recall
-- Assumes: table documents(id, content, embedding vector(1536))

SELECT
    id,
    content,
    1 - (embedding <=> query.vec) AS cosine_similarity
FROM
```



```
documents,
(SELECT '[0.12, 0.87, 0.34, ...]':vector(1536) AS vec) AS query
ORDER BY
  embedding <=> query.vec    -- <=> is cosine distance in pgvector
LIMIT 10;

-- For L2 distance use <-> operator
-- SELECT id, embedding <-> '[...]'::vector AS l2_dist FROM documents ORDER BY l2_dist LIMIT 10;

-- For inner product use <#> operator (returns negative dot product; negate for similarity)
-- SELECT id, -(embedding <#> '[...]'::vector) AS dot_sim FROM documents ORDER BY dot_sim DESC LIMIT 10;
```

When is exact search the right answer? More often than you might think:

- **Small datasets:** Under 100,000 vectors with 1536 dimensions, exact search on modern NVMe storage with pgvector typically completes in under 100ms. You do not need an ANN index.
- **High-recall requirements:** Compliance, medical, and legal workloads sometimes cannot accept the recall tradeoff that ANN introduces. Exact search is the only way to guarantee you found the actual nearest neighbors.
- **Development and evaluation:** When benchmarking your embedding model or tuning your chunking strategy, exact search gives you ground truth. Use it to measure recall of your ANN configuration.
- **Rarely queried, frequently updated datasets:** ANN indexes must be rebuilt or updated as data changes. If your dataset changes constantly and is queried infrequently, maintaining an HNSW index costs more than the query savings justify.

The performance ceiling of exact search is real. A table of 1 million 1536-dimensional vectors requires computing approximately 1.536 billion floating-point operations per query. On a modern CPU with AVX-512 SIMD instructions (which pgvector uses when available), that runs in roughly 3–8 seconds for a cold query on a single node — too slow for a user-facing application. At 10 million vectors, exact search becomes impractical regardless of hardware. This is where approximate search earns its place.

Approximate Nearest Neighbor Search: The Algorithmic Landscape

Approximate nearest neighbor (ANN) search accepts a controlled recall loss in exchange for query latency that is orders of magnitude lower than exact search. The term “approximate” means the index might miss some true nearest neighbors — it returns results that are close, but not guaranteed to be the absolute closest. In practice, a well-tuned ANN index returns 95–99% of the true nearest neighbors at 5–50ms latency on datasets of tens of millions of vectors.

Three families of algorithms dominate production systems. Understanding how each works lets you read vendor documentation and benchmark results critically rather than accepting default configurations.

IVFFlat (Inverted File Index with Flat Quantization)

IVFFlat divides the vector space into `nlist` Voronoi cells (clusters) using k-means clustering during index build. Each vector is assigned to the nearest cluster centroid. At query time, the algorithm identifies the `nprobe` closest centroids to the query vector and performs exact search only within those cells.

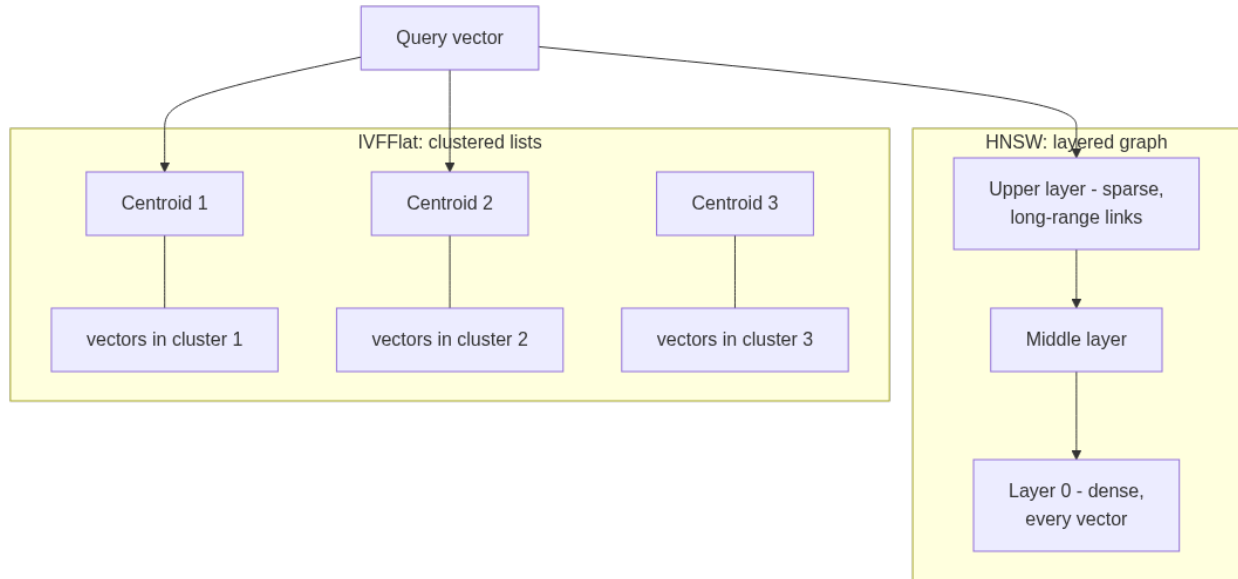


Figure 2: HNSW descends from sparse upper layers toward the query; IVFFlat jumps straight to the nearest cluster centroids and scans only those lists.

Think of it as a hash partition scheme, but the partitioning function is “geometric proximity” instead of a hash function. If your data has 1 million vectors and you build with `nlist=1000`, each cell contains roughly 1000 vectors. Setting `nprobe=10` means the query searches approximately 10,000 vectors instead of 1,000,000 — a 100x reduction in distance computations.

The tradeoff: k-means clustering happens at index build time. If your data distribution shifts significantly after the index is built (new product categories, new languages, domain shift in embeddings), recall degrades. IVFFlat requires periodic index rebuilds to stay healthy.

HNSW (Hierarchical Navigable Small World)

HNSW is the algorithm behind most production vector databases today, including pgvector (since version 0.5.0), Weaviate, Qdrant, and Milvus. It constructs a layered graph where each vector is a node connected to its approximate nearest neighbors. The graph is hierarchical — upper layers have fewer nodes with longer-range connections, lower layers are dense with short-range connections. Queries start at the top layer and greedily descend toward the query vector.

For database engineers, HNSW is analogous to a B-tree in one important way: it supports efficient incremental inserts without full rebuilds. Unlike IVFFlat, you can add vectors to an HNSW index and they are integrated into the graph structure at insert time. The `ef_construction` parameter controls how aggressively the graph is built (higher values = better recall but slower inserts), and `m` controls the number of bidirectional links per node (higher values = better recall and faster queries, but larger index size).

Product Quantization (PQ) and Scalar Quantization (SQ)

Quantization is a compression layer applied on top of IVFFlat or HNSW. Rather than storing full 32-bit floats for each dimension, vectors are compressed to smaller representations — sometimes 8x to 32x smaller. This dramatically reduces memory requirements and can improve query throughput because more vectors fit in CPU cache.

The tradeoff is recall loss from quantization error plus additional complexity in tuning. Pinecone and Weaviate apply quantization transparently. In pgvector 0.7+, halfvec support (16-bit floats) gives you a 2x storage reduction with minimal recall impact.

The table below summarizes the practical characteristics:

Property	Exact KNN	IVFFlat	HNSW
Recall	100%	85–98% (tunable)	90–99% (tunable)
Query latency (1M vecs)	3–8s	5–50ms	1–20ms
Build time	None	Fast (k-means)	Slow (graph construction)
Incremental inserts	Instant	Requires rebuild	Supported natively
Memory overhead	Low	Moderate	High (graph links)
Best for	<100K vectors	Batch workloads, stable datasets	Production real-time search

HNSW in pgvector and Weaviate: Configuration That Actually Matters

HNSW is where most production teams spend their tuning time. The two parameters that dominate behavior are `m` and `ef_construction` at build time, and `ef` (also called `ef_search` or `hnsw.ef_search`) at query time.

```
-- PostgreSQL / pgvector
-- Creating an HNSW index with production-appropriate parameters

-- First, set memory for index construction (run before CREATE INDEX)
SET maintenance_work_mem = '4GB';

-- Create the HNSW index
-- m: number of connections per node (default 16; range 2-100)
-- ef_construction: search depth during build (default 64; range 4-1000)
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 24, ef_construction = 128);

-- At query time, control recall vs speed tradeoff
-- Higher ef_search = better recall, slower queries (default 40)
SET hnsw.ef_search = 100;

-- Now run the approximate nearest neighbor query
SELECT
    d.id,
    d.content,
    d.metadata->>'source' AS source,
    1 - (d.embedding <=> $1::vector) AS similarity
FROM documents d
WHERE similarity > 0.75           -- pre-filter: only high-similarity results
ORDER BY d.embedding <=> $1::vector
LIMIT 5;

-- Check if your index is being used
EXPLAIN (ANALYZE, BUFFERS)
SELECT id FROM documents
```

```
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector
LIMIT 5;
-- Look for "Index Scan using documents_embedding_idx" in output
-- If you see "Seq Scan", increase enable_seqscan=off or check vector dimensions match
```

The interaction between `m`, `ef_construction`, and `ef_search` is the core tuning loop. A reasonable starting point for a production RAG application with 1–5 million vectors:

- `m = 16` (default): acceptable for most workloads
- `m = 24–32`: better recall, ~20% larger index, measurably faster queries on high-traffic systems because the graph navigation converges faster
- `ef_construction = 128`: good balance of build time and recall; doubling to 256 costs roughly 2x build time for marginal recall improvement beyond 99%
- `ef_search = 64–200` at query time: tune this per query type based on latency SLA

Weaviate exposes the same HNSW parameters through its schema configuration:

```
# Python example - configuring HNSW in Weaviate with the v4 Python client

import weaviate
import weaviate.classes as wvc

client = weaviate.connect_to_local() # or connect_to_weaviate_cloud(cluster_url=...)

# Define a collection with explicit HNSW configuration
client.collections.create(
    name="DocumentChunks",
    vectorizer_config=wvc.config.Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-small",
        dimensions=1536
    ),
    vector_index_config=wvc.config.Configure.VectorIndex.hnsw(
        distance_metric=wvc.config.VectorDistances.COSINE,
        ef_construction=128,          # build-time recall vs speed
        max_connections=24,          # equivalent to pgvector's m parameter
        ef=-1,                       # -1 = dynamic: Weaviate auto-tunes ef at query time
        dynamic_ef_min=100,
        dynamic_ef_max=500,
        dynamic_ef_factor=8,
    ),
    properties=[
        wvc.config.Property(name="content", data_type=wvc.config.DataType.TEXT),
        wvc.config.Property(name="source", data_type=wvc.config.DataType.TEXT),
        wvc.config.Property(name="chunk_index", data_type=wvc.config.DataType.INT),
    ]
)

print("Collection created with HNSW index")
client.close()
```

One production reality that surprises engineers coming from relational databases: HNSW index memory requirements are not optional. `pgvector` loads the HNSW graph into `shared_buffers` or the OS page cache during queries. For a 1-million-vector, 1536-dimension dataset with `m=16`, expect approximately 3–6GB of index size on disk and similar working memory requirements for sustained query performance. On systems where the index does not fit in memory, query latency degrades sharply as the OS pages in graph nodes from disk. This is why managed vector database services like Pinecone and Qdrant Cloud advertise memory-optimized infrastructure — they are solving a real problem, not just upselling.

IVFFlat for Batch Workloads and the Recall-Speed Tradeoff in Practice

IVFFlat gets less attention than HNSW in the AI application ecosystem, but it has a legitimate place in workloads where you control data loading patterns and can tolerate periodic index rebuilds. The build process is faster than HNSW, and for very large datasets (50M+ vectors) the memory footprint is more predictable.

The pgvector IVFFlat configuration follows a well-established rule of thumb: set `nlist` to `rows / 1000` for up to 1 million rows, or `sqrt(rows)` beyond that, and start `nprobe` around `sqrt(nlist)` before tuning further against your own recall measurements.

```
-- PostgreSQL / pgvector
-- IVFFlat index configuration and recall tuning

-- For a dataset of 1 million vectors:
-- sqrt(1,000,000) = 1000 → nlist = 1000
-- Starting nprobe sqrt(1000) 32, then tune up or down from there

-- IMPORTANT: Index should be created AFTER data is loaded
-- IVFFlat builds centroids from existing data - an empty-table index is useless
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 1000);

-- At query time, tune nprobe for your latency vs recall SLA
-- More nprobe = higher recall, more latency
SET ivfflat.probes = 100; -- searches 10% of clusters

-- Benchmark different nprobe values against exact search results.
-- `SET` cannot appear inside a CTE, so set the session parameter first,
-- then run the comparison query as a separate statement.

-- Step 1: capture ground truth once (no ivfflat index used for this query,
-- or run against a table with no ANN index)
-- CREATE TEMP TABLE exact_results AS
--   SELECT id, embedding <=> (SELECT embedding FROM documents WHERE id = 12345) AS dist
--   FROM documents
--   ORDER BY dist LIMIT 10;

-- Step 2: for each probes value you want to test, set it, then query and compare
SET ivfflat.probes = 50; -- test with probes=50

SELECT
  COUNT(*) AS matching,
  COUNT(*) / 10.0 AS recall_at_10
FROM exact_results e
JOIN (
  SELECT id FROM documents
  ORDER BY embedding <=> (SELECT embedding FROM documents WHERE id = 12345)
  LIMIT 10
) a USING (id);

-- Repeat for each probes value you want to compare, and plot recall@10 against
-- observed latency for your own data - do not assume published numbers transfer;
-- higher probes always trades latency for recall, but the exact curve is
-- data- and hardware-specific and must be measured, not assumed.
```

The choice between IVFFlat and HNSW in pgvector comes down to update patterns and memory budget. If you are loading a fixed dataset of embeddings (say, pre-computed embeddings for a product catalog that refreshes weekly), IVFFlat is a reasonable choice because it is built once and queries are efficient. If your dataset grows continuously — new documents indexed in real time, new user content, streaming embeddings — HNSW’s incremental insert support makes it operationally

far simpler.

When you move beyond pgvector to dedicated vector databases, the algorithm choice is often abstracted away. Pinecone uses HNSW-family algorithms internally. Qdrant defaults to HNSW but exposes quantization configuration through its collection API:

```
# Python example - Qdrant collection creation with HNSW and scalar quantization

from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, HnswConfigDiff,
    ScalarQuantizationConfig, ScalarType, QuantizationConfig
)

client = QdrantClient(url="http://localhost:6333")

client.create_collection(
    collection_name="knowledge_base",
    vectors_config=VectorParams(
        size=1536,                                # dimensions must match embedding model output
        distance=Distance.COSINE,
        hnsw_config=HnswConfigDiff(
            m=16,
            ef_construct=128,
            full_scan_threshold=10_000, # vectors below this count use exact search
            on_disk=False,              # keep graph in RAM for latency-sensitive queries
        ),
    ),
    quantization_config=QuantizationConfig(
        scalar=ScalarQuantizationConfig(
            type=ScalarType.INT8,          # compress float32 to int8: 4x size reduction
            quantile=0.99,                 # 99th percentile used for range calibration
            always_ram=True,               # keep quantized vectors in memory
        )
    )
)

print("Qdrant collection created with HNSW + INT8 quantization")
print("Expected memory reduction: ~4x vs raw float32 vectors")
print("Expected recall impact: ~1-3% depending on data distribution")
```

The quantization numbers matter for cost estimation. A collection of 10 million 1536-dimensional float32 vectors occupies roughly 61GB of raw storage ($10\text{M} \times 1536 \times 4$ bytes). With INT8 scalar quantization, that drops to approximately 15GB. On managed cloud services billing by memory allocation, quantization is not a theoretical optimization — it is a budget decision.

Recall, Latency, and Throughput: Benchmarking Your Search Configuration

Reading vendor benchmarks is necessary but insufficient. The ANN Benchmarks project (ann-benchmarks.com) provides standardized recall-vs-throughput curves across algorithms and datasets, but the results only hold for the specific dimensionality and data distribution in the benchmark. Your production embeddings, generated by a specific model on a specific domain, will have a different recall curve.

Every production vector search deployment should include a recall evaluation pipeline. The process is straightforward: run your production queries against both exact search and your ANN index, compute recall@k (what fraction of the true top-k results appear in the ANN results), and set

`ef_search` or `nprobe` to meet your target.

For most RAG applications, `recall@10` above 0.95 is sufficient because the LLM synthesizes answers across retrieved chunks — missing one of ten relevant chunks rarely degrades the final answer quality. For semantic deduplication or recommendation systems where the exact ranking matters more, you may need `recall@10` above 0.99, which pushes toward higher `ef_search` values and larger `m` configurations.

Throughput is the metric that catches teams off guard at scale. A single pgvector query returning results in 15ms is easy to achieve. Sustaining 500 queries per second across concurrent connections is a different problem. At that scale, you are contending for CPU across parallel graph traversals, competing for shared buffer cache with simultaneous page faults, and hitting connection pool limits. This is the threshold where the architecture question shifts from “is HNSW fast enough?” to “should I be running Pinecone or Qdrant with horizontal sharding instead of pgvector on a single RDS instance?”

These are rough, order-of-magnitude planning heuristics, not guarantees — always validate against your own hardware, data, and query mix before committing to a design:

- **pgvector on a single node** can reasonably be expected to handle on the order of hundreds of queries per second at sub-50ms latency for a 1–5 million vector HNSW index, depending heavily on CPU cores, memory, and `ef_search`. Supabase and Neon offer managed pgvector and can vertically scale to handle moderate traffic.
- **Pinecone** in pod-based deployments is designed for thousands of queries per second across distributed indexes. The serverless tier optimizes for cost over raw throughput.
- **Qdrant** with multiple nodes and sharding is designed to scale into the thousands of queries per second with appropriately provisioned hardware and sharding, though exact throughput and p99 latency depend on your dataset and configuration and should be benchmarked directly.
- **Weaviate** with its built-in HNSW implementation and optional productquantization can achieve similar throughput to Qdrant when horizontally scaled, with the additional advantage of built-in vectorization that reduces network round-trips for embedding generation at query time.

The practical implication: start with pgvector if your workload is below roughly 100–200 queries per second on a well-provisioned instance, and plan for migration or a dedicated vector database service as your usage grows. The migration is not painless — schemas differ, client libraries differ — but the algorithmic concepts in this chapter transfer directly. You are always configuring `m`, `ef_construction`, `ef_search`, and a distance metric, regardless of which system you run.

Filtering During Vector Search: The Pre-Filter vs Post-Filter Distinction

Production applications almost never search across all vectors. A legal document search restricts to a specific client’s corpus. An e-commerce recommendation system restricts to in-stock items. A customer support bot restricts to the current user’s subscription tier. This combination of metadata filtering with vector similarity search is where many systems hit unexpected performance problems.

There are two fundamentally different approaches:

Post-filtering performs the ANN search first, returning the top-k approximate nearest neighbors from the full index, then applies the metadata filter to that result set. This is fast but dangerous: if the filter is highly selective (say, only 0.1% of your vectors match the filter), the ANN search might return 100 vectors, the filter removes 99 of them, and you return 1 result when the user wanted 10. The solution — returning a much larger initial ANN set before filtering — defeats the performance advantage.

Pre-filtering applies the metadata filter first to identify the matching vector subset, then runs ANN search or exact search over only those vectors. This guarantees correct results for selective filters but requires the database to either maintain separate indexes per filter value (expensive for high-cardinality filters) or fall back to exact search over the filtered subset.

pgvector's behavior is worth understanding precisely: as of version 0.7, pgvector uses the HNSW index with post-filtering. When a `WHERE` clause accompanies a vector query, pgvector applies the filter after retrieving candidates from the graph. For highly selective filters, this can cause the effective recall to drop dramatically. The workaround is partitioning data into separate tables by filter value, or increasing `hnsw.ef_search` to retrieve more candidates before filtering:

```
-- PostgreSQL / pgvector
-- Demonstrating the pre-filter vs post-filter behavior

-- This query uses post-filtering: HNSW is traversed first, then the WHERE clause
-- filters the results. If 'tenant_id = 42' matches only 1% of rows,
-- you may get fewer than 5 results even though relevant vectors exist.
SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> $1::vector
LIMIT 5;

-- Workaround 1: Increase ef_search so more candidates pass through before filtering
SET hnsw.ef_search = 400; -- retrieve 400 candidates, then filter to 5

SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> $1::vector
LIMIT 5;

-- Workaround 2: Partition by tenant into child tables (best for high-selectivity filters)
-- Each child table has its own HNSW index, so the search scope is already restricted
CREATE TABLE documents_tenant_42 (
    LIKE documents INCLUDING ALL
) INHERITS (documents);

CREATE INDEX ON documents_tenant_42
USING hnsw (embedding vector_cosine_ops)
WITH (m = 24, ef_construction = 128);

-- Queries against documents_tenant_42 use the tenant-specific HNSW index
-- and never traverse vectors from other tenants
SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM documents_tenant_42
ORDER BY embedding <=> $1::vector
LIMIT 5;
```

Dedicated vector databases handle this problem differently. Pinecone uses **namespaces** as the primary isolation boundary — each namespace is a separate vector partition, and search is always scoped to a namespace. Weaviate uses **tenant isolation** within a collection, which creates per-tenant HNSW segments. Qdrant implements **payload-based filtering** through a dedicated payload index that runs before vector search when the filter is selective enough:


```
# Python example - Qdrant filtered vector search with pre-filtering behavior

from qdrant_client import QdrantClient
from qdrant_client.models import Filter, FieldCondition, MatchValue, SearchRequest

client = QdrantClient(url="http://localhost:6333")

# Qdrant automatically chooses pre-filter or post-filter based on
# the estimated selectivity of the filter condition.
# For high-selectivity filters (few matching points), it falls back
# to exact search over the filtered subset.

results = client.search(
    collection_name="knowledge_base",
    query_vector=query_embedding,          # list of floats, 1536 dimensions
    query_filter=Filter(
        must=[
            FieldCondition(
                key="tenant_id",
                match=MatchValue(value=42)
            ),
            FieldCondition(
                key="document_type",
                match=MatchValue(value="contract")
            )
        ]
    ),
    limit=10,
    with_payload=True,                    # include metadata in results
    score_threshold=0.75                  # only return vectors above this similarity
)

for result in results:
    print(f"ID: {result.id}, Score: {result.score:.4f}")
    print(f"Content: {result.payload['content'][:100]}...")
    print()
```

The key insight for production systems: **your filter selectivity determines your architecture**. If most queries apply a filter that matches more than 10–20% of your vectors, post-filtering with a high `ef_search` works fine. If you have tenant isolation, document-type restrictions, or date range filters that reduce the candidate set to a small fraction of the total, you need either pre-filtering support (Qdrant, Weaviate) or explicit data partitioning (pgvector table-per-tenant, Pinecone namespaces).

Distance Operators in SQL: A Reference for pgvector Queries

Because pgvector is the most common path for teams already running PostgreSQL, having a clear reference for its operator syntax avoids subtle bugs. Each operator corresponds to a different distance function, and each has a corresponding index operator class:

```
-- PostgreSQL / pgvector
-- Complete reference for vector operators and their corresponding index ops

-- Table setup reference
CREATE TABLE vectors (
    id BIGSERIAL PRIMARY KEY,
    embedding vector(1536) NOT NULL
);

-- Operator: <=> - Cosine distance (1 - cosine_similarity)
-- Range: 0 (identical direction) to 2 (opposite direction)
-- Index operator class: vector_cosine_ops
```

```
CREATE INDEX ON vectors USING hnsw (embedding vector_cosine_ops);
SELECT id, embedding <=> '[...]'::vector AS cosine_dist FROM vectors ORDER BY cosine_dist LIMIT 5;

-- Operator: <-> - L2 (Euclidean) distance
-- Range: 0 (identical) to unbounded
-- Index operator class: vector_l2_ops
CREATE INDEX ON vectors USING hnsw (embedding vector_l2_ops);
SELECT id, embedding <-> '[...]'::vector AS l2_dist FROM vectors ORDER BY l2_dist LIMIT 5;

-- Operator: <#> - Negative inner product (returns -dot_product)
-- Negate the result to get similarity; ORDER BY ascending gives highest dot product first
-- Index operator class: vector_ip_ops
CREATE INDEX ON vectors USING hnsw (embedding vector_ip_ops);
SELECT id, -(embedding <#> '[...]'::vector) AS dot_product FROM vectors ORDER BY embedding <#>
    <- '[...]'::vector LIMIT 5;

-- Only one index can cover each operator class per column.
-- If you need to query by both cosine and L2, create two separate indexes,
-- but be aware of the storage and maintenance cost.

-- Verify which operator is used at query time - mismatched operators silently skip the index
EXPLAIN SELECT id FROM vectors ORDER BY embedding <=> '[...]'::vector LIMIT 5;
-- Output should contain: "Index Scan using vectors_embedding_idx"
-- If it shows "Seq Scan", the operator does not match the index operator class
```

A common production bug: a team builds an HNSW index with `vector_cosine_ops` but queries use `<->` (L2 distance). PostgreSQL silently falls back to a sequential scan, and query latency spikes from milliseconds to seconds. The EXPLAIN output catches this immediately, but it often goes unnoticed until load testing.

Key Takeaways

- **Exact KNN guarantees 100% recall** but scales poorly beyond roughly 100,000 vectors for latency-sensitive applications. Use it during development for ground-truth measurements and in production only when recall requirements are non-negotiable and dataset size permits.
- **HNSW is the dominant ANN algorithm for production vector search:** it supports incremental inserts, delivers sub-millisecond to low-tens-of-milliseconds latency at millions of vectors, and is available in pgvector, Weaviate, Qdrant, and Milvus with directly comparable tuning parameters (`m`, `ef_construction`, `ef_search`).
- **IVFFlat remains valuable for stable, batch-loaded datasets** where build simplicity and predictable memory usage matter more than incremental insert support. Set `nlist` based on dataset size, start `nprobe` at `sqrt(nlist)`, and always measure recall against your own data rather than relying on published benchmark curves.
- **The distance function is part of the schema, not just a query option:** mixing cosine and L2 operators within the same pipeline produces inconsistent rankings. Choose one distance metric when you design your embedding pipeline, create your index with the matching operator class, and enforce it throughout your application layer.
- **Filter selectivity determines architecture:** post-filtering works when metadata filters are broad; highly selective filters (tenant isolation, document-type restrictions) require either native pre-filtering support (Qdrant, Weaviate), explicit data partitioning (pgvector table-per-tenant), or namespace-level separation (Pinecone) to avoid recall collapse in the filtered result set.

Chapter 3: Distance Metrics: Cosine, Euclidean, Dot Product

Every vector search query asks the same fundamental question: how similar is this vector to that one? The answer depends entirely on how you measure similarity — and that choice is not arbitrary. The three dominant distance metrics in production vector systems — cosine similarity, Euclidean distance, and dot product — each make different mathematical assumptions about what “closeness” means, and each is optimized for different embedding types, storage architectures, and query patterns. Getting this wrong means your semantic search returns irrelevant results, your recommendation engine surfaces stale content, and your RAG pipeline retrieves the wrong context — all while your vector index silently operates on numbers that look reasonable but mean nothing useful. This chapter gives you a precise, production-grounded understanding of all three metrics: the math behind them, how they behave with real embeddings, and how to configure them correctly in pgvector, Pinecone, Weaviate, Qdrant, and other systems.

3.1 What Distance Metrics Actually Measure (And Why It’s Not Just Geometry)

Before writing a single line of SQL or configuration, you need to understand what these metrics are computing and why the choice matters at the embedding layer.

An embedding is a dense numerical vector — typically 768 to 3072 dimensions for modern language models — that encodes semantic meaning as a position in high-dimensional space. The embedding model learns to position semantically similar inputs near each other and dissimilar inputs far apart. “Near” and “far” are defined by the distance metric used during training. This is the point most database engineers miss: **the metric you use at query time must match or be compatible with the metric the model was optimized for during training.** Using the wrong metric against an embedding trained with a different objective function degrades search quality measurably.

Think of it in database terms. An embedding is like a computed column that encodes a complex function of the source data. The distance metric is the comparison operator used in the WHERE clause. Just as you would not use a string inequality comparison on a numeric column and expect meaningful results, you should not use Euclidean distance on embeddings trained with cosine objectives and expect meaningful ranking.

The three metrics differ in what they ignore and what they emphasize:

- **Cosine similarity** measures the angle between two vectors, ignoring their magnitudes. Two vectors pointing in the same direction score 1.0 regardless of how long they are.
- **Euclidean distance** (L2) measures the straight-line distance between two vector endpoints in space. It is sensitive to both direction and magnitude.
- **Dot product** (inner product) measures both angle and magnitude together. It rewards vectors that are both directionally aligned and large.

This means cosine similarity treats a document with 10 mentions of “database” identically to a document with 100 mentions, as long as the relative proportions of all terms are the same. Euclidean distance would treat those documents differently. Dot product would actively prefer the document with more signal — which is exactly the behavior you want in some recommendation scenarios.

Here is a concrete Python example showing all three computed on the same pair of vectors:

```
# Python example - computing all three distance metrics from scratch
import numpy as np

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    """
    Measures angle between vectors. Range: -1.0 to 1.0.
    1.0 = identical direction, 0.0 = orthogonal, -1.0 = opposite.
    """
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def euclidean_distance(a: np.ndarray, b: np.ndarray) -> float:
    """
    Measures straight-line distance. Range: 0.0 to infinity.
    Lower = more similar.
    """
    return np.linalg.norm(a - b)

def dot_product(a: np.ndarray, b: np.ndarray) -> float:
    """
    Measures both direction and magnitude. Range: -infinity to infinity.
    Higher = more similar (for normalized vectors, equals cosine similarity).
    """
    return np.dot(a, b)

# Two 4-dimensional vectors for illustration
# In production these would be 768-3072 dimensional embeddings
vec_a = np.array([0.2, 0.5, 0.1, 0.8])
vec_b = np.array([0.4, 1.0, 0.2, 1.6]) # vec_b = 2 * vec_a - same direction, 2x magnitude

print(f"Cosine similarity: {cosine_similarity(vec_a, vec_b):.6f}") # 1.000000 - identical angle
print(f"Euclidean distance: {euclidean_distance(vec_a, vec_b):.6f}") # 0.969536 - sees the difference
print(f"Dot product: {dot_product(vec_a, vec_b):.6f}") # 1.880000 - vec_b amplifies the
    ↳ signal

# Now normalize both to unit length and compare again
vec_a_norm = vec_a / np.linalg.norm(vec_a)
vec_b_norm = vec_b / np.linalg.norm(vec_b)

print("\n--- After L2 normalization ---")
print(f"Cosine similarity: {cosine_similarity(vec_a_norm, vec_b_norm):.6f}") # 1.000000
print(f"Euclidean distance: {euclidean_distance(vec_a_norm, vec_b_norm):.6f}") # 0.000000 - identical
    ↳ after norm
print(f"Dot product: {dot_product(vec_a_norm, vec_b_norm):.6f}") # 1.000000 - equals
    ↳ cosine
```

This last block is the key mathematical insight you need to internalize: **when all vectors are L2-normalized to unit length, cosine similarity and dot product are mathematically equivalent, and the Euclidean distance between them is a monotonic transformation of the cosine distance.** Many embedding models — including OpenAI’s text-embedding-3-small and all-MiniLM-L6-v2 — output normalized or near-normalized vectors. In those cases, all three

metrics produce the same ranking order, and the choice between them becomes a question of computational efficiency rather than mathematical correctness.

3.2 Cosine Similarity in Practice: When Angle Is All That Matters

Cosine similarity is the dominant metric for text embeddings and the default choice in most RAG pipelines. Its strength is scale invariance: the length of the vector does not affect the similarity score, only the direction. This property naturally handles documents of different lengths, embedding outputs from different batch sizes, and representations that might vary in norm due to quantization or compression.

The cosine distance ($1 - \text{cosine similarity}$) is used when you need a proper distance metric that satisfies the triangle inequality, which is required by some index structures. Most vector databases accept configuration for both.

In pgvector, cosine distance is expressed with the `<=>` operator:

```
-- PostgreSQL / pgvector
-- Create a table for document embeddings (e.g., from OpenAI text-embedding-3-small, 1536 dims)
CREATE TABLE documents (
    id          BIGSERIAL PRIMARY KEY,
    content     TEXT NOT NULL,
    source_url  TEXT,
    embedding   VECTOR(1536)
);

-- Create an HNSW index optimized for cosine distance
-- HNSW is preferred over IVFFlat for cosine when dataset < 5M rows
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Semantic search: find the 5 most similar documents to a query embedding
-- The :query_embedding parameter is your 1536-dimensional query vector
SELECT
    id,
    content,
    source_url,
    1 - (embedding <=> :query_embedding) AS cosine_similarity
FROM documents
ORDER BY embedding <=> :query_embedding
LIMIT 5;

-- Note: pgvector uses cosine DISTANCE (<=>), not similarity
-- cosine_distance = 1 - cosine_similarity
-- Range of cosine distance: 0.0 (identical) to 2.0 (opposite)
-- For normalized vectors it ranges 0.0 to 2.0, practical range is usually 0.0 to 1.0
```

The same operation in Pinecone uses cosine as the index-level setting — you cannot change the metric after index creation:

```
# Python example - creating a Pinecone index with cosine metric and querying it
from pinecone import Pinecone, ServerlessSpec
import openai

pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")

# Metric is set at index creation and cannot be changed afterward
# This is the key operational difference from a traditional database - plan ahead
pc.create_index(
    name="documents-cosine",
    dimension=1536,
    # matches OpenAI text-embedding-3-small
```

```

metric="cosine",                # options: cosine, euclidean, dotproduct
spec=ServerlessSpec(cloud="aws", region="us-east-1")
)

index = pc.Index("documents-cosine")

# Generate a query embedding using OpenAI
client = openai.OpenAI(api_key="YOUR_OPENAI_API_KEY")

query_text = "how do I optimize PostgreSQL for write-heavy workloads?"
response = client.embeddings.create(
    input=query_text,
    model="text-embedding-3-small"
)
query_embedding = response.data[0].embedding # list of 1536 floats

# Query the index - Pinecone returns cosine similarity scores (higher = more similar)
# Note: Pinecone inverts the cosine distance so 1.0 = identical, 0.0 = unrelated
results = index.query(
    vector=query_embedding,
    top_k=5,
    include_metadata=True
)

for match in results.matches:
    print(f"ID: {match.id:30s} Score: {match.score:.4f} Title: {match.metadata.get('title', 'N/A')}")

```

One production pattern worth establishing early: normalize your embeddings before storing them if your embedding model does not guarantee unit-length output. The `sentence-transformers` library normalizes by default when you use `encode(normalize_embeddings=True)`. OpenAI’s `text-embedding-3` models output vectors that are already close to unit norm but not exactly 1.0 due to floating-point precision. For strict cosine implementations, explicit normalization costs little at write time and eliminates a class of subtle ranking errors.

3.3 Euclidean Distance: When Magnitude Carries Meaning

Euclidean distance (L2 distance) is the default metric for many image embedding models, multi-modal systems, and any domain where the magnitude of an embedding encodes meaningful information. In text applications it is less common, but in computer vision, audio, and scientific data contexts it often outperforms cosine.

The intuition: if your embedding model is trained to place embeddings in a space where both direction and length encode distinct properties — for example, where longer vectors represent more “intense” or “prominent” concepts — then cosine similarity throws away half the information. Euclidean distance preserves it.

In `pgvector`, Euclidean distance uses the `<->` operator:

```

-- PostgreSQL / pgvector
-- Image similarity search using Euclidean distance
-- e.g., embeddings from CLIP (512 dims) or ResNet features (2048 dims)
CREATE TABLE product_images (
    id          BIGSERIAL PRIMARY KEY,
    product_id  UUID NOT NULL,
    image_url   TEXT NOT NULL,
    embedding    VECTOR(512)          -- CLIP ViT-B/32 output dimension
);

-- HNSW with L2 operator class
CREATE INDEX ON product_images USING hnsw (embedding vector_l2_ops)

```



```

WITH (m = 16, ef_construction = 64);

-- Find visually similar products to a given query image
-- Lower L2 distance = more similar
SELECT
  p.id,
  p.product_id,
  p.image_url,
  p.embedding <-> :query_image_embedding AS l2_distance
FROM product_images p
ORDER BY p.embedding <-> :query_image_embedding
LIMIT 10;

-- IVFFlat alternative for larger datasets (> 1M rows)
-- Requires ANALYZE after load; less accurate but faster at scale
CREATE INDEX ON product_images USING ivfflat (embedding vector_l2_ops)
  WITH (lists = 100);
-- Rule of thumb: lists = sqrt(total_rows) for balanced speed/recall
-- After loading data: ANALYZE product_images;

```

The critical operational difference between IVFFlat and HNSW becomes concrete with Euclidean distance. IVFFlat clusters the vector space using k-means centroids, and L2 distance is the natural clustering objective. HNSW builds a hierarchical graph and works well with any metric. For datasets under roughly 1 million vectors on PostgreSQL, HNSW with `vector_l2_ops` gives better recall with less tuning. Above that scale — or on machines with limited RAM for the HNSW graph — IVFFlat is more memory-efficient.

Here is a comparison that shows the recall and latency tradeoffs you should test against your actual data before committing to an index type:

```

# Python example - benchmarking IVFFlat vs HNSW recall for L2 distance using pgvector
import psycopg2
import numpy as np
import time

conn = psycopg2.connect("host=localhost dbname=vectordb user=postgres password=secret")
cur = conn.cursor()

def run_ann_query(query_vector: list, index_type: str, k: int = 10) -> tuple[list, float]:
    """Run an ANN query and return (result_ids, latency_ms)."""
    vec_str = "[" + ",".join(str(v) for v in query_vector) + "]"

    # Set ef_search for HNSW (higher = better recall, slower)
    if index_type == "hnsw":
        cur.execute("SET hnsw.ef_search = 100")
    elif index_type == "ivfflat":
        cur.execute("SET ivfflat.probes = 10") # probes sqrt(lists) is a common starting point

    start = time.perf_counter()
    cur.execute(
        f"SELECT id FROM product_images ORDER BY embedding <-> %s::vector LIMIT %s",
        (vec_str, k)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000
    result_ids = [row[0] for row in cur.fetchall()]
    return result_ids, elapsed_ms

# Load 100 random query vectors for benchmarking
np.random.seed(42)
query_vectors = [np.random.rand(512).tolist() for _ in range(100)]

# Run benchmark
latencies = []
for qvec in query_vectors:
    _, latency = run_ann_query(qvec, index_type="hnsw", k=10)
    latencies.append(latency)

```

```

print(f"HNSW L2 - p50: {np.percentile(latencies, 50):.1f}ms "
      f"p95: {np.percentile(latencies, 95):.1f}ms "
      f"p99: {np.percentile(latencies, 99):.1f}ms")

cur.close()
conn.close()

```

In Weaviate, Euclidean distance is configured at the collection level under `vectorizer` settings and requires the `l2-squared` distance metric parameter. Weaviate calls it `l2-squared` because it omits the square root for computational efficiency — the square root is a monotonic function, so it does not change the ranking order. You will see this optimization in several production vector databases: they compute L2-squared internally but report the API documentation in terms of standard L2.

3.4 Dot Product: Maximum Throughput for Recommendation and Ranking Systems

The dot product (also called inner product) is the most computationally efficient of the three metrics because it avoids the normalization step required by cosine similarity. On modern hardware with SIMD instructions, this matters at scale: skipping normalization measurably reduces per-query CPU cost under high query volume, though the exact savings depend on hardware and vector dimensionality — benchmark it on your own workload rather than assuming a fixed percentage.

Beyond performance, dot product is the correct metric for specific problem categories:

1. **Recommendation systems with popularity signals:** If your embedding model is trained with a two-tower architecture (common in large-scale recommendation at companies like Google, Meta, and Spotify), the item embedding magnitudes are deliberately trained to encode relevance intensity. Using cosine would strip that signal away.
2. **Dense retrieval models trained with in-batch negatives:** Models like DPR (Dense Passage Retrieval) and BGE-M3 with specific training configurations expect inner product similarity at inference time to match their training objective.
3. **Maximum Inner Product Search (MIPS) problems:** A well-studied class of problems in recommendation, where the goal is to find the item with the highest dot product against a query embedding.

In pgvector, dot product similarity uses the `<#>` operator. Note the sign convention: pgvector returns the **negative** inner product because all of its operators follow the “lower is more similar” convention:

```

-- PostgreSQL / pgvector
-- Recommendation system using dot product (inner product)
-- Assumes item embeddings are NOT normalized - magnitude encodes popularity/quality signal
CREATE TABLE item_embeddings (
    item_id      BIGINT PRIMARY KEY,
    item_type    TEXT NOT NULL,      -- 'article', 'product', 'video'
    embedding    VECTOR(256)         -- two-tower model, 256-dimensional item tower
);

-- HNSW with inner product operator class
CREATE INDEX ON item_embeddings USING hnsw (embedding vector_ip_ops)
    WITH (m = 32, ef_construction = 128);
-- Higher m (32 vs default 16) improves recall for dot product
-- because the HNSW graph needs more connections to navigate non-Euclidean topology

-- Get top-10 recommendations for a user embedding

```

```

-- <#> returns NEGATIVE dot product - ORDER BY ascending to get highest dot product first
SELECT
  item_id,
  item_type,
  (embedding <#> :user_embedding) * -1 AS dot_product_score
FROM item_embeddings
WHERE item_type = 'article'
ORDER BY embedding <#> :user_embedding -- ascending negative = descending positive
LIMIT 10;

```

The sign inversion is a common source of bugs when engineers first switch from cosine to dot product in pgvector. Cosine distance is inherently non-negative (0.0 to 2.0), so “ORDER BY embedding <=> query ASC” feels natural. With <#>, which returns a negative value, you must either remember to sort ascending (which retrieves the most negative negated dot product, i.e., the highest actual dot product) or multiply by -1 in a subquery. Both approaches work; consistency in your codebase matters more than which pattern you choose.

In Qdrant, dot product is a first-class metric with clean semantics — no sign inversion:

```

# Python example - Qdrant collection with dot product metric for a recommendation system
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct, Filter, FieldCondition, MatchValue
)
import numpy as np

client = QdrantClient(host="localhost", port=6333)

# Create collection with dot product distance
# Qdrant uses "Dot" - returns higher scores for more similar vectors (no sign inversion)
client.create_collection(
    collection_name="item_recommendations",
    vectors_config=VectorParams(
        size=256,
        distance=Distance.DOT # also available: COSINE, EUCLID, MANHATTAN
    )
)

# Upsert item embeddings (simulating a two-tower model output)
np.random.seed(42)
items = [
    PointStruct(
        id=i,
        vector=np.random.randn(256).tolist(), # NOT normalized - magnitude matters
        payload={
            "title": f"Article {i}",
            "category": "technology" if i % 2 == 0 else "science",
            "engagement_score": float(np.random.uniform(0.1, 5.0))
        }
    )
    for i in range(1, 10001) # 10,000 items
]
client.upsert(collection_name="item_recommendations", points=items)

# Query: find top items for a user embedding, filtered by category
user_embedding = np.random.randn(256).tolist()

results = client.search(
    collection_name="item_recommendations",
    query_vector=user_embedding,
    query_filter=Filter(
        must=[FieldCondition(key="category", match=MatchValue(value="technology"))]
    ),
    limit=10,
    with_payload=True
)

```

```

for result in results:
    print(f"ID: {result.id:5d} Score: {result.score:.4f} "
          f"Title: {result.payload['title']:12s} "
          f"Engagement: {result.payload['engagement_score']:.2f}")

```

One warning about dot product at scale: HNSW index quality degrades more noticeably with dot product than with cosine when vectors have highly variable magnitudes. The HNSW graph is built using the dot product as the proximity function during construction, and if your training data has very large magnitude variance, the graph can develop poor connectivity in regions of the embedding space. Monitor your recall metrics in production and consider normalizing embeddings if recall drops below acceptable thresholds.

3.5 Choosing the Right Metric for Your Embedding Model and Use Case

The theoretical distinctions between metrics only matter if they translate into measurable differences in your system. Here is a framework for making the decision systematically rather than guessing.

Step 1: Check the embedding model documentation. This is non-negotiable. Every major embedding model specifies its recommended similarity function:

Embedding Model	Recommended Metric	Normalized Output
OpenAI text-embedding-3-small/large	cosine	Near-unit (use cosine or dot product)
OpenAI text-embedding-ada-002	cosine	Near-unit
Cohere embed-english-v3.0	cosine	Yes (unit norm)
sentence-transformers/all-MiniLM-L6-v2	cosine	Yes (with normalize=True)
CLIP (OpenAI)	cosine	Yes
Google textembedding-gecko	dot product	No — magnitude is meaningful
BGE-M3 (BAAI)	cosine or dot product (depends on mode)	Depends on usage
DPR (Facebook)	dot product	No

Step 2: Test whether normalization changes your results. If an embedding model outputs nearly-normalized vectors, cosine and dot product will produce nearly identical rankings. You can verify this quickly:

```

# Python example - checking embedding magnitude distribution and metric correlation
from sentence_transformers import SentenceTransformer
import numpy as np
from scipy.stats import spearmanr

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

sentences = [
    "How do I create an index in PostgreSQL?",
    "What is the difference between HNSW and IVFFlat?",

```

```

"Python machine learning frameworks comparison",
"Vector database performance benchmarks",
"Database normalization and denormalization tradeoffs",
"Kubernetes pod scheduling and resource limits",
"Natural language processing with transformer models",
"B-tree vs hash index performance",
]

# encode with normalization (default for sentence-transformers)
embeddings_norm = model.encode(sentences, normalize_embeddings=True)
# encode without normalization
embeddings_raw = model.encode(sentences, normalize_embeddings=False)

# Check magnitude distribution
norms = np.linalg.norm(embeddings_raw, axis=1)
print(f"Raw embedding norms - min: {norms.min():.4f} max: {norms.max():.4f} mean: {norms.mean():.4f}")
# For all-MiniLM-L6-v2, these are very close to 1.0, confirming near-unit output

# Compute cosine similarity and dot product rankings for the first query against all others
query_idx = 0
query_norm = embeddings_norm[query_idx]
query_raw = embeddings_raw[query_idx]

cosine_scores = [
    np.dot(query_norm, embeddings_norm[i]) for i in range(len(sentences))
]
dot_scores = [
    np.dot(query_raw, embeddings_raw[i]) for i in range(len(sentences))
]
l2_distances = [
    np.linalg.norm(query_raw - embeddings_raw[i]) for i in range(len(sentences))
]

# Rank correlation between cosine and dot product - should be ~1.0 for near-normalized models
rho_cosine_dot, _ = spearmanr(cosine_scores, dot_scores)
# Rank correlation between cosine similarity and negative L2 distance
rho_cosine_l2, _ = spearmanr(cosine_scores, [-d for d in l2_distances])

print(f"Spearman rank correlation - cosine vs dot product: {rho_cosine_dot:.4f}")
print(f"Spearman rank correlation - cosine vs -L2: {rho_cosine_l2:.4f}")
# Values near 1.0 confirm the metrics produce equivalent rankings for this model

```

If the Spearman correlation between metrics is above 0.99, you can choose whichever metric your infrastructure makes cheapest without sacrificing search quality. If the correlation is lower — especially between cosine and L2 — it signals that magnitudes vary meaningfully and the choice of metric genuinely changes which documents you retrieve.

Step 3: Match the metric to the task type.

Task	Recommended Metric	Reason
Semantic text search / RAG	cosine	Scale-invariant; works with any document length
Question answering retrieval (DPR-style)	dot product	Training objective uses inner product
Image/audio similarity	euclidean	Magnitude-sensitive models are common
Recommendation (two-tower)	dot product	Item magnitude encodes quality signal
Clustering / anomaly detection	euclidean	Centroid-based algorithms assume L2 geometry

Task	Recommended Metric	Reason
Deduplication / near-duplicate detection	cosine	Direction-only comparison handles length variation
Cross-modal retrieval (text→image)	cosine	CLIP and similar models output normalized vectors

Step 4: Measure, don't assume. Build a small evaluation harness with labeled query-document pairs and compute recall@K for each metric against your actual data. The framework below works with any embedding model and vector store:

```
# Python example - evaluation harness for comparing distance metrics
# Requires: a set of queries, a document corpus, and ground-truth relevant document IDs
import numpy as np
from sentence_transformers import SentenceTransformer
from typing import Callable

def compute_recall_at_k(
    query_embeddings: np.ndarray,
    doc_embeddings: np.ndarray,
    ground_truth: list[set[int]],  # ground_truth[i] = set of relevant doc indices for query i
    similarity_fn: Callable[[np.ndarray, np.ndarray], np.ndarray],
    k: int = 10
) -> float:
    """
    Computes Recall@K: fraction of relevant documents found in the top-K results.
    similarity_fn takes (query_embedding, all_doc_embeddings) and returns similarity scores.
    """
    total_recall = 0.0
    for i, query_emb in enumerate(query_embeddings):
        scores = similarity_fn(query_emb, doc_embeddings)
        top_k_indices = set(np.argsort(scores)[::-1][:k])  # descending - higher score = more similar
        relevant = ground_truth[i]
        if len(relevant) > 0:
            recall = len(top_k_indices & relevant) / len(relevant)
            total_recall += recall
    return total_recall / len(query_embeddings)

# Define similarity functions for each metric (all return higher = more similar)
def cosine_sim_fn(query: np.ndarray, docs: np.ndarray) -> np.ndarray:
    query_norm = query / np.linalg.norm(query)
    doc_norms = docs / np.linalg.norm(docs, axis=1, keepdims=True)
    return doc_norms @ query_norm

def dot_product_fn(query: np.ndarray, docs: np.ndarray) -> np.ndarray:
    return docs @ query

def neg_l2_fn(query: np.ndarray, docs: np.ndarray) -> np.ndarray:
    # Negate so that lower distance = higher score (consistent with other fns)
    return -np.linalg.norm(docs - query, axis=1)

# Example usage - plug in your own queries, docs, and ground_truth labels
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# In a real evaluation, these would be your actual queries and corpus
sample_queries = ["PostgreSQL index types", "vector search algorithms", "recommendation systems"]
sample_docs = [
    "B-tree indexes in PostgreSQL are used for ordered data.",
    "HNSW is a graph-based approximate nearest neighbor algorithm.",
    "IVFFlat partitions the vector space into Voronoi cells.",
    "Collaborative filtering is a common recommendation technique.",
    "Two-tower models learn user and item representations separately.",
    "Cosine similarity measures the angle between two vectors.",
]

# ground_truth[i] = indices of docs relevant to query i
```

```

ground_truth = [
    {0, 1, 2},      # docs relevant to "PostgreSQL index types"
    {1, 2, 5},      # docs relevant to "vector search algorithms"
    {3, 4},         # docs relevant to "recommendation systems"
]

query_embs = model.encode(sample_queries, normalize_embeddings=False)
doc_embs   = model.encode(sample_docs,   normalize_embeddings=False)

for metric_name, fn in [("cosine", cosine_sim_fn), ("dot_product", dot_product_fn), ("neg_l2",
↪ neg_l2_fn)]:
    recall = compute_recall_at_k(query_embs, doc_embs, ground_truth, fn, k=3)
    print(f"Recall@3 - {metric_name:15s}: {recall:.4f}")

```

Run this against a representative slice of your production query distribution — at least a few hundred labeled pairs — before committing to an index configuration. The cost of rebuilding a Pinecone index or recreating a pgvector index with a different operator class is non-trivial once you have millions of vectors stored.

3.6 Metric Configuration Across Vector Databases: A Practical Reference

Each vector database exposes the distance metric configuration differently, and the naming conventions are not consistent across systems. This section documents the exact configuration syntax for each major platform so you can translate your metric choice into working infrastructure.

pgvector (PostgreSQL extension)

pgvector exposes three operator classes and three corresponding query operators:

```

-- PostgreSQL / pgvector - complete operator class reference

-- 1. COSINE DISTANCE: vector_cosine_ops / <=>
-- Returns cosine distance (0.0 = identical, 2.0 = opposite)
-- Use for: text embeddings, CLIP, most sentence-transformer models
CREATE INDEX idx_docs_cosine ON documents USING hnsw (embedding vector_cosine_ops);
SELECT id, 1 - (embedding <=> '[0.1,0.2,0.3]') AS cosine_similarity FROM documents
ORDER BY embedding <=> '[0.1,0.2,0.3]' LIMIT 5;

-- 2. EUCLIDEAN (L2) DISTANCE: vector_l2_ops / <->
-- Returns L2 distance (0.0 = identical, unbounded above)
-- Use for: image embeddings, audio, scientific / sensor data
CREATE INDEX idx_images_l2 ON product_images USING hnsw (embedding vector_l2_ops);
SELECT id, embedding <-> '[0.1,0.2,0.3]' AS l2_distance FROM product_images
ORDER BY embedding <-> '[0.1,0.2,0.3]' LIMIT 5;

-- 3. INNER PRODUCT (DOT PRODUCT): vector_ip_ops / <#>
-- Returns NEGATIVE inner product - ORDER BY ASC to retrieve highest dot product
-- Use for: recommendation systems, DPR, two-tower models
CREATE INDEX idx_items_ip ON item_embeddings USING hnsw (embedding vector_ip_ops);
SELECT id, (embedding <#> '[0.1,0.2,0.3]') * -1 AS dot_product FROM item_embeddings
ORDER BY embedding <#> '[0.1,0.2,0.3]' LIMIT 5;

-- Additional operator classes added in pgvector 0.7+:
-- vector_l1_ops / <+> (L1 / Manhattan distance)
-- bit_hamming_ops / <~> (Hamming distance for binary vectors)
-- bit_jaccard_ops / <%> (Jaccard distance for binary vectors)
-- sparsevec_l2_ops (L2 for sparse vectors stored in sparsevec type)

```

Pinecone

Pinecone's metric is set once at index creation and is immutable. The API naming maps directly to the three primary metrics:

```
# Python example - Pinecone metric configuration reference
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key="YOUR_API_KEY")

# COSINE - default for most text embedding use cases
pc.create_index(
    name="my-cosine-index",
    dimension=1536,
    metric="cosine",
    spec=ServerlessSpec(cloud="aws", region="us-east-1")
)

# EUCLIDEAN - for image embeddings and magnitude-sensitive models
pc.create_index(
    name="my-euclidean-index",
    dimension=512,
    metric="euclidean",
    spec=ServerlessSpec(cloud="aws", region="us-east-1")
)

# DOT PRODUCT - for two-tower recommendation models
pc.create_index(
    name="my-dotproduct-index",
    dimension=256,
    metric="dotproduct",
    spec=ServerlessSpec(cloud="aws", region="us-east-1")
)

# IMPORTANT: changing the metric requires creating a new index and migrating data
# Plan this decision before populating the index with production data
```

Weaviate

Weaviate configures the distance metric per collection in the schema definition. Starting with Weaviate v4, the Python client uses a fluent builder pattern:

```
# Python example - Weaviate v4 client metric configuration
import weaviate
import weaviate.classes as wvc

client = weaviate.connect_to_local() # or connect_to_weaviate_cloud(cluster_url=...)

# COSINE - for text/NLP collections
client.collections.create(
    name="Documents",
    vectorizer_config=wvc.config.Configure.Vectorizer.none(), # bring your own vectors
    vector_index_config=wvc.config.Configure.VectorIndex.hnsw(
        distance_metric=wvc.config.VectorDistances.COSINE
    ),
    properties=[
        wvc.config.Property(name="content", data_type=wvc.config.DataType.TEXT),
        wvc.config.Property(name="source_url", data_type=wvc.config.DataType.TEXT),
    ]
)

# L2 (EUCLIDEAN SQUARED) - for image/multimodal collections
client.collections.create(
    name="ProductImages",
    vectorizer_config=wvc.config.Configure.Vectorizer.none(),
    vector_index_config=wvc.config.Configure.VectorIndex.hnsw(
        distance_metric=wvc.config.VectorDistances.L2_SQUARED # note: squared L2, not L2
    ),
)
```



```

        properties=[
            wvc.config.Property(name="product_id", data_type=wvc.config.DataType.TEXT),
            wvc.config.Property(name="image_url", data_type=wvc.config.DataType.TEXT),
        ]
    )

    # DOT PRODUCT - for recommendation models
    client.collections.create(
        name="ItemRecommendations",
        vectorizer_config=wvc.config.Configure.Vectorizer.none(),
        vector_index_config=wvc.config.Configure.VectorIndex.hnsw(
            distance_metric=wvc.config.VectorDistances.DOT
        ),
        properties=[
            wvc.config.Property(name="title", data_type=wvc.config.DataType.TEXT),
            wvc.config.Property(name="category", data_type=wvc.config.DataType.TEXT),
        ]
    )

    # Weaviate also supports: MANHATTAN, HAMMING
    # All metrics other than COSINE are returned as raw distances (lower=more similar)
    # COSINE in Weaviate returns cosine DISTANCE (0=identical), not cosine similarity

    client.close()

```

Qdrant

Qdrant uses a clean enum-based API with consistent sign conventions — higher score always means more similar, regardless of the underlying distance function:

```

# Python example - Qdrant distance metric configuration reference
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams

client = QdrantClient(host="localhost", port=6333)

# All four standard metrics are available
# Qdrant normalizes scores so that higher always means more similar

# COSINE
client.create_collection(
    collection_name="text_search",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
)

# EUCLIDEAN (L2)
client.create_collection(
    collection_name="image_search",
    vectors_config=VectorParams(size=512, distance=Distance.EUCLID)
)

# DOT PRODUCT (inner product)
client.create_collection(
    collection_name="recommendations",
    vectors_config=VectorParams(size=256, distance=Distance.DOT)
)

# MANHATTAN (L1) - available in Qdrant, less common in other databases
client.create_collection(
    collection_name="sparse_features",
    vectors_config=VectorParams(size=1024, distance=Distance.MANHATTAN)
)

```

Chroma

Chroma uses a string parameter called `space` in the HNSW configuration:

```
# Python example - Chroma metric configuration
import chromadb

client = chromadb.PersistentClient(path="/path/to/chroma/data")

# COSINE - default and most common for text embeddings
collection = client.create_collection(
    name="documents_cosine",
    metadata={"hnsw:space": "cosine"} # options: "cosine", "l2", "ip"
)

# L2 (Euclidean)
collection_l2 = client.create_collection(
    name="images_l2",
    metadata={"hnsw:space": "l2"}
)

# Inner product / dot product
collection_ip = client.create_collection(
    name="recommendations_ip",
    metadata={"hnsw:space": "ip"}
)
```

3.7 Distance Metrics and HNSW Index Quality: The Hidden Interaction

An aspect of metric selection that rarely appears in documentation but consistently affects production systems is how the distance metric interacts with HNSW graph construction quality.

HNSW (Hierarchical Navigable Small World) builds a layered proximity graph during index construction. At each layer, nodes are connected to their nearest neighbors according to the index's distance function. When traversing the graph at query time, the algorithm greedily follows edges toward the query in the same metric space. The underlying mathematical guarantee — that the graph is navigable — relies on the metric satisfying the triangle inequality.

Cosine distance and Euclidean distance both satisfy the triangle inequality. Dot product does **not**. This means HNSW with dot product is operating outside its formal convergence guarantees. In practice this still works well, but it requires higher values of `m` (the number of neighbors per node) and `ef_construction` (the candidate queue size during graph construction) to achieve the same recall as cosine or Euclidean configurations on the same data:

```
-- PostgreSQL / pgvector
-- HNSW parameter tuning for dot product vs cosine
-- For the same recall target, dot product typically needs higher m and ef_construction

-- Cosine - standard parameters work well
CREATE INDEX idx_docs_cosine ON documents USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Dot product - increase m and ef_construction to compensate for non-metric space
CREATE INDEX idx_items_ip ON item_embeddings USING hnsw (embedding vector_ip_ops)
    WITH (m = 32, ef_construction = 128);

-- At query time, also increase ef_search for dot product if recall is low
-- Default ef_search is 40 in pgvector; for dot product, try 80-200
SET hnsw.ef_search = 100;

-- Monitor recall with pg_stat_statements or application-level logging
-- A recall drop of more than 5-10% relative to brute-force is worth investigating
```

The exact parameter values needed depend on your data distribution. Embeddings from models with high magnitude variance tend to require more aggressive HNSW parameters under dot product. If you find that recall is degrading despite tuning, consider whether normalizing your embeddings and switching to cosine would be semantically equivalent for your model — and if so, that switch is almost always worth making for index reliability.

Another hidden interaction: **IVFFlat and L2 distance**. IVFFlat works by clustering the vector space with k-means at index build time. K-means minimizes within-cluster variance, which is defined in terms of Euclidean distance. When you create an IVFFlat index with `vector_cosine_ops`, pgvector internally normalizes all vectors before clustering, so the clusters are built in the unit-norm subspace — this is correct behavior but it means that if your input vectors are not normalized, the cluster assignment at query time uses the normalized version of your query. The practical implication is that IVFFlat with cosine ops works correctly without any special handling on your part, but understanding this behavior matters when debugging recall issues.

3.8 Normalization Strategies: Write-Time vs Query-Time vs None

The normalization decision interacts with every other choice in this chapter. There are three practical strategies, each with different tradeoffs:

Strategy 1: Normalize at write time (recommended for most applications)

Store normalized vectors in the database. The embedding is divided by its L2 norm before insertion:

```
# Python example - write-time normalization pipeline
import numpy as np
import psycopg2

def normalize_l2(vec: np.ndarray) -> np.ndarray:
    norm = np.linalg.norm(vec)
    if norm == 0:
        return vec # handle zero vectors gracefully - avoid division by zero
    return vec / norm

def insert_document_with_normalized_embedding(
    conn: psycopg2.extensions.connection,
    content: str,
    raw_embedding: np.ndarray
) -> int:
    """Normalize the embedding at write time before storing."""
    normalized = normalize_l2(raw_embedding)
    vec_str = "[" + ",".join(f"{v:.8f}" for v in normalized) + "]"

    cur = conn.cursor()
    cur.execute(
        "INSERT INTO documents (content, embedding) VALUES (%s, %s::vector) RETURNING id",
        (content, vec_str)
    )
    doc_id = cur.fetchone()[0]
    conn.commit()
    cur.close()
    return doc_id

# Benefits of write-time normalization:
# 1. Dot product becomes equivalent to cosine - no sign inversion needed in queries
# 2. IVFFlat and HNSW behave predictably - the metric space is well-defined
# 3. You can switch between cosine and dot product operators in queries without rebuilding the index
# 4. Simplifies query code - no special handling needed at read time
```

Strategy 2: Normalize at query time only

Store raw vectors. Normalize only the query vector before issuing the similarity search. This is implicit when using `vector_cosine_ops` — pgvector normalizes both the stored vectors and the query internally. The stored vectors on disk are still raw:

```
-- PostgreSQL / pgvector
-- Query-time normalization is implicit in the cosine operator
-- The stored embedding is raw; pgvector computes cosine using the normalized form internally
-- This means switching to vector_ip_ops later will NOT give cosine-equivalent results
-- unless you rebuild the index with pre-normalized vectors

-- To normalize a query vector explicitly in SQL (useful for debugging):
SELECT
  id,
  content,
  1 - (embedding <=> (
    -- Normalize the query vector in-place using pgvector's l2_normalize function
    l2_normalize('[0.3, 0.8, 0.5, 0.2, ...]':vector)
  )) AS cosine_similarity
FROM documents
ORDER BY embedding <=> l2_normalize('[0.3, 0.8, 0.5, 0.2, ...]':vector)
LIMIT 5;

-- l2_normalize() was added in pgvector 0.5.0
-- For older versions: (v / sqrt(v <#> v)) is equivalent but verbose
```

Strategy 3: No normalization (dot product or Euclidean with raw embeddings)

Keep vectors raw when magnitude is semantically meaningful and your metric is explicitly dot product or Euclidean. This is appropriate for two-tower recommendation models and image embedding systems. There is no special infrastructure required — just insert vectors as-is and configure the index with the appropriate operator class.

The risk: if a code change somewhere in your pipeline (an updated embedding model, a library version bump that changes normalization behavior, a preprocessing step added upstream) silently changes the magnitude distribution of incoming vectors, your similarity scores shift without any database error. Build magnitude monitoring into your data pipeline:

```
# Python example - embedding magnitude monitoring in a data pipeline
import numpy as np
from dataclasses import dataclass

@dataclass
class EmbeddingHealthMetrics:
    count: int
    mean_norm: float
    std_norm: float
    min_norm: float
    max_norm: float
    zero_vector_count: int

def check_embedding_batch_health(embeddings: np.ndarray) -> EmbeddingHealthMetrics:
    """
    Run before inserting a batch of embeddings into the vector database.
    Alert if norms deviate significantly from historical baseline.
    """
    norms = np.linalg.norm(embeddings, axis=1)
    zero_count = int(np.sum(norms == 0))

    metrics = EmbeddingHealthMetrics(
        count=len(embeddings),
        mean_norm=float(np.mean(norms)),
        std_norm=float(np.std(norms)),
        min_norm=float(np.min(norms)),
        max_norm=float(np.max(norms)),
    )
```

```

        zero_vector_count=zero_count
    )

    # Alert conditions - tune thresholds to your baseline
    if zero_count > 0:
        print(f"WARNING: {zero_count} zero vectors detected - these will cause NaN in cosine
              ↳ similarity")
    if metrics.std_norm > 0.5 * metrics.mean_norm:
        print(f"WARNING: High norm variance (std/mean = {metrics.std_norm/metrics.mean_norm:.2f}) "
              f"↳ check for mixed embedding model outputs or preprocessing changes")
    if abs(metrics.mean_norm - 1.0) > 0.1:
        print(f"INFO: Mean norm {metrics.mean_norm:.4f} deviates from unit norm "
              f"↳ cosine and dot product may give different rankings")

    return metrics

# Usage in a batch insert pipeline:
# raw_embeddings = model.encode(batch_texts, normalize_embeddings=False)
# health = check_embedding_batch_health(raw_embeddings)
# log_to_monitoring_system(health)
# if health.zero_vector_count == 0 and health.std_norm < threshold:
#     insert_to_vector_db(raw_embeddings)

```

3.9 Distance Metrics in Multi-Vector and Sparse+Dense Hybrid Systems

Production retrieval systems increasingly use multiple representations per document: a dense embedding for semantic similarity, a sparse BM25-style vector for lexical matching, and sometimes additional dense vectors for different semantic aspects (e.g., one for the document title, one for the body). Each representation can use a different distance metric, and the final score is a weighted combination.

Qdrant natively supports multiple named vectors per point, each with its own distance metric:

```

# Python example - Qdrant multi-vector collection with per-vector distance metrics
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, SparseVectorParams, SparseIndexParams,
    NamedVector, NamedSparseVector, PointStruct, SparseVector
)
import numpy as np

client = QdrantClient(host="localhost", port=6333)

# Collection with three vector types:
# 1. Dense title embedding (cosine) - short, context-specific
# 2. Dense body embedding (cosine) - full document semantics
# 3. Sparse BM25 vector (dot product) - lexical matching
client.create_collection(
    collection_name="hybrid_documents",
    vectors_config={
        "title_dense": VectorParams(size=384, distance=Distance.COSINE),
        "body_dense": VectorParams(size=1536, distance=Distance.COSINE),
    },
    sparse_vectors_config={
        "body_sparse": SparseVectorParams(
            index=SparseIndexParams(on_disk=False)
        )
    }
)

# Insert a document with all three representations

```

```

client.upsert(
    collection_name="hybrid_documents",
    points=[
        PointStruct(
            id=1,
            vector={
                "title_dense": np.random.rand(384).tolist(),
                "body_dense": np.random.rand(1536).tolist(),
                "body_sparse": SparseVector(
                    indices=[102, 445, 891, 2034, 5511], # BM25 term IDs
                    values=[0.82, 0.61, 0.44, 0.38, 0.21] # TF-IDF weights
                )
            },
            payload={"title": "Understanding HNSW Index Construction", "category": "database"}
        )
    ]
)

# Hybrid query: combine dense body semantic search with sparse lexical matching
# Qdrant supports RRF (Reciprocal Rank Fusion) and linear combination for hybrid fusion
from qdrant_client.models import Query, FusionQuery, Fusion, Prefetch

results = client.query_points(
    collection_name="hybrid_documents",
    prefetch=[
        Prefetch(
            query=np.random.rand(1536).tolist(),
            using="body_dense",
            limit=20
        ),
        Prefetch(
            query=SparseVector(
                indices=[102, 891, 5511],
                values=[0.75, 0.42, 0.18]
            ),
            using="body_sparse",
            limit=20
        )
    ],
    query=FusionQuery(fusion=Fusion.RRF), # Reciprocal Rank Fusion combines the two result lists
    limit=5,
    with_payload=True
)

```

In pgvector, multi-vector hybrid search requires a different approach — combining multiple distance computations in a single SQL query with explicit weight parameters:

```

-- PostgreSQL / pgvector
-- Hybrid search: combining dense cosine similarity with a BM25-style score stored as a column
-- For true sparse+dense, consider pg_sparse (a separate extension) or pre-compute BM25 scores

-- Schema supporting hybrid scoring
CREATE TABLE hybrid_documents (
    id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    title_embedding VECTOR(384), -- dense, cosine
    body_embedding VECTOR(1536), -- dense, cosine
    bm25_score_col REAL -- pre-computed BM25 score for specific query context
                           -- (in practice, computed dynamically using ts_rank)
);

CREATE INDEX ON hybrid_documents USING hnsw (title_embedding vector_cosine_ops);
CREATE INDEX ON hybrid_documents USING hnsw (body_embedding vector_cosine_ops);

-- Hybrid query combining body semantic similarity and full-text relevance
-- Alpha controls the weight: 1.0 = pure semantic, 0.0 = pure BM25
SELECT
    id,

```

```
title,
:alpha * (1 - (body_embedding <=> :query_dense_embedding)) +
(1 - :alpha) * ts_rank(to_tsvector('english', body), plainto_tsquery('english', :query_text))
AS hybrid_score
FROM hybrid_documents
WHERE to_tsvector('english', body) @@ plainto_tsquery('english', :query_text)
OR (body_embedding <=> :query_dense_embedding) < :cosine_distance_threshold
ORDER BY hybrid_score DESC
LIMIT 10;

-- Note: this query does NOT use the vector index efficiently if the WHERE clause is too broad.
-- For production hybrid search in PostgreSQL, partition the problem:
-- 1. Retrieve top-N by dense similarity using the HNSW index
-- 2. Retrieve top-N by BM25 using the GIN full-text index
-- 3. Merge and re-rank in application code or a CTE
```

Key Takeaways

- **Match the metric to the model’s training objective:** using cosine similarity against a model trained with dot product objectives — or vice versa — degrades search quality silently. Always consult the embedding model documentation before configuring your vector index.
 - **When vectors are L2-normalized to unit length, cosine similarity, dot product, and Euclidean distance all produce identical ranking order:** many popular text embedding models (including sentence-transformers and OpenAI’s text-embedding-3 series) output near-unit-norm vectors, making the metric choice primarily a question of computational cost rather than result quality for those models.
 - **Dot product is the most computationally efficient metric but the most fragile at index construction time:** because dot product does not satisfy the triangle inequality, HNSW graphs built on dot product require higher `m` and `ef_construction` parameters to achieve equivalent recall compared to cosine or Euclidean configurations. Monitor recall in production when using dot product with HNSW.
 - **Normalize embeddings at write time rather than relying on query-time operator behavior:** write-time normalization makes cosine and dot product equivalent, simplifies query code, reduces the risk of silent accuracy regressions when embedding models are updated, and enables monitoring of the magnitude distribution as a pipeline health signal.
 - **Every major vector database uses a different naming convention and sign convention for the same underlying metric:** pgvector’s `<#>` operator returns negative dot product; Weaviate uses `L2_SQUARED` rather than `L2`; Pinecone calls the metric `dotproduct` while Qdrant calls it `DOT`; Chroma uses `ip` for inner product. Build a reference table for your team and encode the metric configuration in infrastructure-as-code to prevent mismatches between the embedding model, the stored vectors, and the index operator class.
-

Chapter 4: The Vector Database Landscape: pgvector, Pinecone, Weaviate, Qdrant, Chroma

The vector database market matured rapidly between 2022 and 2024, producing a crowded field of purpose-built systems, managed cloud services, and extensions bolted onto existing relational databases. For database professionals, navigating this landscape requires the same analytical framework you apply to any storage engine selection: understand the data model, the indexing strategy, the operational model, and the cost curve. This chapter maps the five most production-relevant systems — pgvector, Pinecone, Weaviate, Qdrant, and Chroma — against those dimensions, with real connection code, query patterns, and the benchmarks that separate marketing claims from production reality.

The Core Decision Framework: Where Vector Search Lives in Your Stack

Before comparing systems, establish what problem you are actually solving. Vector databases are not a monolithic category — they span a spectrum from pure-vector stores to hybrid search engines to vector-capable relational databases. The position on that spectrum drives every downstream architectural decision.

The Spectrum

At one end sits pgvector: a PostgreSQL extension that adds vector similarity operators and indexes to the relational engine you already run. You get ACID transactions, foreign keys, joins, and pg_dump — at the cost of ANN performance that tops out around 100M vectors before you need to shard aggressively.

At the other end sit purpose-built vector databases like Pinecone: managed services with no DDL, no transactions, and no SQL, designed exclusively to return approximate nearest neighbors at sub-50ms latency on billion-scale corpora.

Between those poles sit systems like Weaviate, Qdrant, and Chroma, each making different tradeoffs between operational simplicity, hybrid search capability, and raw performance.

The Three Selection Questions

Before writing any connection code, answer these three questions:

1. **Vector count and growth rate.** Under 1M vectors: any system works. 1M–50M: all options remain viable with tuning. 50M–500M: purpose-built systems start pulling ahead. 500M+: Pinecone, Weaviate, or Qdrant with horizontal scaling.
2. **Query pattern mix.** Pure semantic search: any purpose-built system. Semantic search combined with relational filtering (user_id, tenant_id, date ranges, price bands): pgvector's join capability or Weaviate/Qdrant's metadata filters become significant.
3. **Operational footprint.** If your organization already runs managed PostgreSQL (RDS, Cloud SQL, Supabase, Neon), pgvector adds a line to your migration script. If you are building a new service with no existing database infrastructure, a managed vector service removes operational overhead that would otherwise require dedicated DBA time.

Latency Benchmarks as a Reality Check

Independent, reproducible cross-system benchmarks are harder to find than vendor marketing pages suggest — dataset, dimensionality, hardware, and index parameters all shift the numbers enough that a table copied from one source rarely transfers to your workload. The directional pattern reported across most public benchmarking efforts (including the community-run ann-benchmarks project, which benchmarks ANN libraries rather than full database systems) is that purpose-built vector engines tend to out-throughput pgvector's HNSW/IVFFlat implementation at a given recall target, sometimes substantially. Treat that as a prior to test, not a number to design around — run the comparison yourself against your own data, dimensionality, and hardware before committing to an architecture.

pgvector: Vector Search Inside PostgreSQL

pgvector is the most operationally conservative choice and, for many teams, the right one. It extends PostgreSQL with a `vector` data type, two index types (ivfflat and HNSW), and three distance operators (L2, cosine, inner product). The extension first appeared in 2021 and reached production-ready HNSW support in version 0.5.0 (late 2023).

Installation and Schema Setup

```
# shell
# Install pgvector on Ubuntu/Debian
sudo apt install postgresql-16-pgvector

# Or build from source
git clone https://github.com/pgvector/pgvector.git
cd pgvector
make
sudo make install

-- PostgreSQL / pgvector
-- Enable the extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Documents table with embeddings (1536 dims = OpenAI text-embedding-ada-002)
CREATE TABLE documents (
  id          BIGSERIAL PRIMARY KEY,
  tenant_id   UUID NOT NULL,
  content     TEXT NOT NULL,
  metadata    JSONB,
```

```

embedding    VECTOR(1536),
created_at   TIMESTAMPTZ DEFAULT NOW()
);

-- HNSW index - better recall and query latency than ivfflat
-- m: connections per layer (16 is default, 32 improves recall at memory cost)
-- ef_construction: search width during build (64 default, 128 for higher recall)
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Partial index on a specific tenant - useful for multi-tenant SaaS
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64)
    WHERE tenant_id = '3f2a1b00-0000-0000-0000-000000000001';

```

Querying with Metadata Filters

pgvector's join capability is where it earns its place for relational workloads. You can combine ANN search with arbitrary SQL predicates in a single query planner pass.

```

-- PostgreSQL / pgvector
-- Set HNSW search parameter at session level
-- ef_search > ef_construction improves recall at query time
SET hnsw.ef_search = 100;

-- Semantic search scoped to a tenant, filtered by metadata, ranked by recency
-- The query embedding comes from your application layer as a literal
SELECT
    d.id,
    d.content,
    d.metadata->>'source_url'      AS source_url,
    d.metadata->>'doc_type'        AS doc_type,
    1 - (d.embedding <=> $1::vector) AS cosine_similarity,
    d.created_at
FROM documents d
WHERE
    d.tenant_id = $2
    AND d.metadata->>'doc_type' IN ('manual', 'faq')
    AND d.created_at >= NOW() - INTERVAL '90 days'
ORDER BY d.embedding <=> $1::vector -- cosine distance, ascending
LIMIT 10;

-- Operators reference:
-- <=> cosine distance
-- <-> L2 (Euclidean) distance
-- <#> negative inner product (use for maximum inner product search)

```

Python Integration with pgvector

```

# Python
import asyncpg
import numpy as np
from openai import OpenAI
from pgvector.asyncpg import register_vector

openai_client = OpenAI()

async def embed_text(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", # 1536 dims, cheaper than ada-002
        input=text
    )
    return response.data[0].embedding

async def upsert_document(pool: asyncpg.Pool, doc_id: int, tenant_id: str,
                          content: str, metadata: dict) -> None:
    embedding = await embed_text(content)
    async with pool.acquire() as conn:
        await register_vector(conn)

```

```
await conn.execute(
    """
    INSERT INTO documents (id, tenant_id, content, metadata, embedding)
    VALUES ($1, $2, $3, $4, $5)
    ON CONFLICT (id) DO UPDATE
        SET content = EXCLUDED.content,
            metadata = EXCLUDED.metadata,
            embedding = EXCLUDED.embedding
    """,
    doc_id, tenant_id, content, metadata, embedding
)

async def semantic_search(pool: asyncpg.Pool, query: str,
                          tenant_id: str, k: int = 10) -> list[dict]:
    query_embedding = await embed_text(query)
    async with pool.acquire() as conn:
        await register_vector(conn)
        # ef_search tunable per-connection
        await conn.execute("SET hnsw.ef_search = 100")
        rows = await conn.fetch(
            """
            SELECT id, content, metadata,
                   1 - (embedding <=> $1::vector) AS similarity
            FROM documents
            WHERE tenant_id = $2
            ORDER BY embedding <=> $1::vector
            LIMIT $3
            """,
            query_embedding, tenant_id, k
        )
    return [dict(r) for r in rows]
```

When pgvector is the Right Call

pgvector wins when: your vector count stays under 10M, you need transactional consistency between document metadata and embeddings, your team operates PostgreSQL already, or you need complex JOIN queries that would require application-side orchestration in a purpose-built system. It loses when: you need sub-20ms P99 latency at 50M+ vectors, your indexing throughput requirements exceed what a single PostgreSQL primary can sustain, or you need native sparse+dense hybrid scoring.

Pinecone: Managed Vector Search as a Service

Pinecone occupies the opposite corner of the design space. You get no SQL, no ACID, and no self-hosting option — but you also get no infrastructure management, automatic horizontal scaling, and consistently low latency at billion-scale. The operational model resembles an S3 bucket: provision an index, write vectors through an API, query through the same API.

Pinecone organizes data into **indexes** (equivalent to a table) containing **namespaces** (logical partitions, roughly equivalent to schema-level separation). Each vector record carries an ID, a dense embedding, an optional sparse vector for hybrid search, and a metadata dict of filterable key-value pairs.

Pinecone Index Setup and Upsert

```
# Python
from pinecone import Pinecone, ServerlessSpec
from openai import OpenAI
import time
```

```

pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")
openai_client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

INDEX_NAME = "product-catalog"
DIMENSION = 1536 # text-embedding-3-small output dimension

# Create a serverless index (AWS us-east-1)
# Serverless charges per query/write RU, not per pod-hour
if INDEX_NAME not in [i.name for i in pc.list_indexes()]:
    pc.create_index(
        name = INDEX_NAME,
        dimension = DIMENSION,
        metric = "cosine",
        spec = ServerlessSpec(cloud="aws", region="us-east-1")
    )
    # Wait for index to be ready
    while not pc.describe_index(INDEX_NAME).status["ready"]:
        time.sleep(1)

index = pc.Index(INDEX_NAME)

def embed_batch(texts: list[str]) -> list[list[float]]:
    """Embed a batch of texts. OpenAI allows up to 2048 inputs per call."""
    response = openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=texts
    )
    return [item.embedding for item in response.data]

def upsert_products(products: list[dict], namespace: str = "default") -> None:
    """
    products: list of dicts with keys: id, name, description, category, price
    Pinecone upsert is idempotent - safe to re-run on updates.
    """
    BATCH_SIZE = 100 # Pinecone recommends 100-200 vectors per upsert call
    for i in range(0, len(products), BATCH_SIZE):
        batch = products[i : i + BATCH_SIZE]
        texts = [{"p['name']} {p['description']}" for p in batch]
        embeddings = embed_batch(texts)
        vectors = [
            {
                "id": str(p["id"]),
                "values": emb,
                "metadata": {
                    "name": p["name"],
                    "category": p["category"],
                    "price": float(p["price"]) # metadata values: str, int, float, bool, list
                }
            }
            for p, emb in zip(batch, embeddings)
        ]
        index.upsert(vectors=vectors, namespace=namespace)
        print(f"Upserted batch {i // BATCH_SIZE + 1}, {len(vectors)} vectors")

def query_products(query_text: str, category: str = None,
                   max_price: float = None, top_k: int = 10,
                   namespace: str = "default") -> list[dict]:
    query_embedding = embed_batch([query_text])[0]

    # Metadata filter uses MongoDB-style operators
    filter_expr = {}
    if category:
        filter_expr["category"] = {"$eq": category}
    if max_price is not None:
        filter_expr["price"] = {"$lte": max_price}

    results = index.query(
        vector = query_embedding,
        top_k = top_k,

```

```
        filter          = filter_expr if filter_expr else None,
        include_metadata = True,
        namespace       = namespace
    )
    return [
        {
            "id":      m.id,
            "score":    m.score,
            "name":     m.metadata.get("name"),
            "category": m.metadata.get("category"),
            "price":    m.metadata.get("price")
        }
        for m in results.matches
    ]
```

Pinecone's Operational Model

The serverless tier (launched 2024) removes the pod-sizing decision that made early Pinecone capacity planning painful. In the pod-based model, undersizing a p1.x1 pod meant reindexing into a larger pod with downtime. Serverless autoscales within the cloud region, billed per read unit (RU) and write unit (WU). A single query costs roughly 1 RU per 1K dimensions per result returned — budget accordingly for high-throughput applications.

Pinecone does not support cross-record joins or aggregation. If your application needs “find similar products, then look up inventory levels,” the inventory lookup happens in a separate database call. This is not a limitation unique to Pinecone — it is the general tradeoff of purpose-built vector systems, and it drives many teams back toward pgvector when their query patterns are primarily relational.

Weaviate: Hybrid Search and the Knowledge Graph Model

Weaviate presents a different abstraction than either Pinecone or pgvector. It organizes data as **collections** of **objects**, where each object carries properties (typed fields), an optional vector, and cross-references to other objects. The cross-reference mechanism gives Weaviate a graph traversal capability that neither Pinecone nor pgvector offers natively.

Weaviate's distinguishing production feature is its native **hybrid search**: a single query path that combines dense vector similarity (via HNSW) with sparse BM25 keyword scoring, fused using Reciprocal Rank Fusion (RRF) or a weighted linear combination. For document retrieval workloads where users submit keyword queries alongside semantic intent, hybrid mode consistently outperforms pure vector search in NDCG metrics.

Weaviate Connection and Schema

```
# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import Configure, Property, DataType, VectorDistances
from openai import OpenAI

# Weaviate Cloud (WCD) connection
client = weaviate.connect_to_weaviate_cloud(
    cluster_url = "https://your-cluster.weaviate.network",
    auth_credentials = weaviate.auth.AuthApiKey("YOUR_WEAVIATE_API_KEY"),
    headers = {"X-OpenAI-API-Key": "YOUR_OPENAI_API_KEY"} # for module vectorization
)
```

```

COLLECTION_NAME = "TechArticle"

# Create collection - Weaviate handles vectorization via modules
# text2vec-openai calls the OpenAI API at insert time automatically
if not client.collections.exists(COLLECTION_NAME):
    client.collections.create(
        name = COLLECTION_NAME,
        vectorizer_config = Configure.Vectorizer.text2vec_openai(
            model = "text-embedding-3-small"
        ),
        vector_index_config = Configure.VectorIndex.hnsw(
            distance_metric = VectorDistances.COSINE,
            ef_construction = 128,
            max_connections = 16,
            ef = 64 # query-time search width
        ),
        properties = [
            Property(name="title", data_type=DataType.TEXT),
            Property(name="body", data_type=DataType.TEXT),
            Property(name="author", data_type=DataType.TEXT),
            Property(name="category", data_type=DataType.TEXT),
            Property(name="published_at", data_type=DataType.DATE),
            Property(name="view_count", data_type=DataType.INT),
        ]
    )
    print(f"Collection '{COLLECTION_NAME}' created.")

collection = client.collections.get(COLLECTION_NAME)

# Batch insert - Weaviate auto-vectorizes via the configured module
with collection.batch.dynamic() as batch:
    for article in articles: # articles: list[dict]
        batch.add_object(properties={
            "title": article["title"],
            "body": article["body"],
            "author": article["author"],
            "category": article["category"],
            "published_at": article["published_at"], # ISO 8601 string
            "view_count": article["view_count"]
        })

# Hybrid search: combines BM25 + vector, fused via RRF
results = collection.query.hybrid(
    query = "optimizing PostgreSQL index performance",
    alpha = 0.75, # 0.0 = pure BM25, 1.0 = pure vector, 0.75 = vector-weighted
    limit = 10,
    filters = wvc.query.Filter.by_property("category").equal("database") &
        wvc.query.Filter.by_property("view_count").greater_than(100),
    return_properties = ["title", "author", "category", "published_at"],
    return_metadata = wvc.query.MetadataQuery(score=True, explain_score=True)
)

for obj in results.objects:
    print(f"Score: {obj.metadata.score:.4f} | Title: {obj.properties['title']}")
    # explain_score shows BM25 and vector sub-scores for debugging recall issues

client.close()

```

Weaviate's Generative Module

Weaviate ships a `generate` module that pipes retrieved objects directly into a configured LLM. This is RAG-in-one-call: vector retrieval and LLM generation in a single API round-trip.

```

# Python
# Generative search - retrieve + synthesize in one Weaviate call
response = collection.generate.hybrid(
    query = "What are the best practices for PostgreSQL vacuuming?",
    alpha = 0.7,
    limit = 5,

```

```
grouped_task      = "Summarize the key PostgreSQL vacuuming best practices "  
                  "from these articles in three concise bullet points.",  
grouped_properties = ["title", "body"]  
)  
  
print(response.generated)  # LLM-synthesized answer from retrieved context
```

This pattern works well for internal tooling and prototypes but has important production caveats: you are coupling your retrieval latency to your LLM call latency, and a slow generation response blocks the entire query. In high-throughput RAG pipelines, the retrieval and generation steps are better decoupled into separate service boundaries.

Qdrant and Chroma: Performance-First and Developer-First

Qdrant

Qdrant (developed in Rust) is the performance-oriented choice among the open-source purpose-built systems. Its HNSW implementation with quantization support (scalar, product, and binary quantization) enables significantly higher vector density per node compared to Weaviate's pure float32 storage. The payload filter engine evaluates metadata conditions during the HNSW graph traversal rather than as a post-filter, which prevents the precision degradation that affects post-filter approaches at high selectivity ratios.

```
# Python  
from qdrant_client import QdrantClient  
from qdrant_client.models import (  
    Distance, VectorParams, PointStruct,  
    Filter, FieldCondition, Range, MatchValue,  
    ScalarQuantization, ScalarQuantizationConfig, ScalarType,  
    SearchRequest  
)  
from openai import OpenAI  
  
qdrant = QdrantClient(url="http://localhost:6333")  
openai_client = OpenAI()  
  
COLLECTION = "support_tickets"  
DIM         = 1536  
  
# Create collection with scalar quantization (int8)  
# Reduces memory 4x with ~2-5% recall drop at ef=128  
qdrant.recreate_collection(  
    collection_name = COLLECTION,  
    vectors_config  = VectorParams(size=DIM, distance=Distance.COSINE),  
    quantization_config = ScalarQuantization(  
        scalar = ScalarQuantizationConfig(  
            type      = ScalarType.INT8,  
            quantile  = 0.99, # clip outliers beyond 99th percentile  
            always_ram = True  # keep quantized index in RAM, raw vectors on disk  
        )  
    )  
)  
  
# Create payload index for fast metadata filtering  
qdrant.create_payload_index(  
    collection_name = COLLECTION,  
    field_name      = "priority",  
    field_schema    = "keyword"  
)  
qdrant.create_payload_index(  
    collection_name = COLLECTION,
```



```

        field_name      = "created_ts",
        field_schema    = "integer"
    )

def upsert_tickets(tickets: list[dict]) -> None:
    BATCH = 50
    for i in range(0, len(tickets), BATCH):
        batch = tickets[i : i + BATCH]
        texts = [t["subject"] + " " + t["description"] for t in batch]
        vectors = openai_client.embeddings.create(
            model = "text-embedding-3-small",
            input = texts
        ).data
        points = [
            PointStruct(
                id      = t["id"], # integer or UUID string
                vector  = vectors[j].embedding,
                payload = {
                    "subject": t["subject"],
                    "priority": t["priority"], # "high", "medium", "low"
                    "created_ts": int(t["created_ts"]), # Unix timestamp
                    "customer_id": t["customer_id"]
                }
            )
            for j, t in enumerate(batch)
        ]
        qdrant.upsert(collection_name=COLLECTION, points=points)

def search_tickets(query: str, priority: str = None,
                  days_back: int = 30, top_k: int = 10) -> list[dict]:
    import time
    query_vec = openai_client.embeddings.create(
        model="text-embedding-3-small", input=[query]
    ).data[0].embedding

    # Build filter with payload conditions
    conditions = []
    if priority:
        conditions.append(FieldCondition(key="priority", match=MatchValue(value=priority)))
    if days_back:
        cutoff_ts = int(time.time()) - days_back * 86400
        conditions.append(FieldCondition(key="created_ts", range=Range(gte=cutoff_ts)))

    query_filter = Filter(must=conditions) if conditions else None

    results = qdrant.search(
        collection_name = COLLECTION,
        query_vector     = query_vec,
        query_filter     = query_filter,
        limit            = top_k,
        with_payload     = True,
        search_params    = {"ef": 128, "exact": False} # ef controls recall/speed tradeoff
    )
    return [
        {
            "id":      r.id,
            "score":    r.score,
            "subject":  r.payload.get("subject"),
            "priority": r.payload.get("priority"),
            "customer_id": r.payload.get("customer_id")
        }
        for r in results
    ]

```

Qdrant's payload filter pre-filtering model is worth understanding in detail. When you specify a `Filter` at query time, Qdrant evaluates whether to walk the HNSW graph with the filter applied inline (pre-filter) or to retrieve a candidate set and then apply the filter afterward (post-filter). It makes this decision automatically based on the estimated selectivity of the filter: high-selectivity

filters (few matching points) trigger a different traversal strategy than low-selectivity ones. This adaptive behavior prevents the recall collapse that pure post-filter implementations exhibit when filter selectivity is very high. For workloads with aggressive metadata filtering — think per-user or per-tenant scoping — Qdrant’s approach to this tradeoff is an important production consideration.

Qdrant’s distributed mode uses a shard-per-collection model with configurable replication factor. You specify the number of shards at collection creation time. Unlike Pinecone serverless (which manages sharding invisibly), Qdrant requires explicit capacity planning: too few shards limits write throughput, too many increases cross-node coordination overhead at query time. The general guidance from Qdrant’s documentation is to size shards at roughly 200M vectors each for most hardware configurations.

Chroma

Chroma sits at the opposite end of the operational spectrum from Qdrant. It is explicitly developer-first: single-file embedded mode, no infrastructure to run, a Python-native API, and direct LangChain and LlamaIndex integration out of the box. Chroma is the fastest path from “I need vector storage in my prototype” to working code, measured in minutes rather than hours.

```
# Python
import chromadb
from chromadb.utils import embedding_functions

# Embedded mode - persistent local storage, no server process required
chroma = chromadb.PersistentClient(path="./chroma_data")

# Chroma has built-in embedding functions for common providers
oai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key = "YOUR_OPENAI_API_KEY",
    model_name = "text-embedding-3-small"
)

collection = chroma.get_or_create_collection(
    name = "research_papers",
    embedding_function = oai_ef,
    metadata = {"hnsw:space": "cosine"} # distance metric config
)

# Add documents - Chroma handles embedding automatically via the configured function
collection.add(
    ids = ["paper_001", "paper_002", "paper_003"],
    documents = [
        "Attention is All You Need: transformer architecture for sequence modeling",
        "BERT: Pre-training of Deep Bidirectional Transformers",
        "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"
    ],
    metadatas = [
        {"year": 2017, "venue": "NeurIPS", "citations": 80000},
        {"year": 2018, "venue": "NAACL", "citations": 75000},
        {"year": 2020, "venue": "NeurIPS", "citations": 12000}
    ]
)

# Query - returns distances, documents, and metadata in one call
results = collection.query(
    query_texts = ["how transformers use attention mechanisms"],
    n_results = 2,
    where = {"year": {"$gte": 2018}}, # metadata filter
    include = ["documents", "metadatas", "distances"]
)

for doc, meta, dist in zip(
    results["documents"][0],
    results["metadatas"][0],
```

```
    results["distances"][0]
):
    print(f"Distance: {dist:.4f} | Venue: {meta['venue']} | {doc[:80]}")
```

Chroma's production limitations become visible quickly. Its default HNSW index is not distributed and has no native horizontal scaling. The client-server mode (connecting to a `chromadb.HttpClient`) provides some separation of concerns but does not partition data across nodes. For workloads exceeding a few million vectors or requiring multi-instance write concurrency, Chroma's architecture will not support the load, and migration to a more capable system should be planned from the start.

Where Chroma is genuinely strong: LangChain integration, rapid iteration on embedding strategies, evaluation harnesses, and offline batch processing pipelines. Many production RAG systems started as Chroma prototypes and migrated to Qdrant or Weaviate when scale requirements became clearer — the Chroma API's simplicity speeds up the early feedback loop without requiring operational investment.

Side-by-Side: Connection Patterns and Query Structures

The code differences across systems reflect their underlying design philosophies. Here is the same semantic search operation — retrieve the top-10 most similar documents to a query, filtered by category and a date range — expressed in all five systems.

pgvector (PostgreSQL)

```
-- PostgreSQL / pgvector
SET hnswef_search = 100;

SELECT id, title, category, published_at,
       1 - (embedding <=> $1::vector) AS similarity
FROM articles
WHERE category = $2
  AND published_at >= $3
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

Pinecone

```
# Python / Pinecone
results = index.query(
    vector      = query_embedding,
    top_k       = 10,
    filter      = {
        "category": {"$eq": category},
        "published_ts": {"$gte": int(since_date.timestamp())}
    },
    include_metadata = True,
    namespace        = "articles"
)
```

Weaviate

```
# Python / Weaviate v4 client
results = collection.query.hybrid(
    query      = query_text,
    alpha      = 0.75,
    limit      = 10,
    filters    = wvc.query.Filter.by_property("category").equal(category) &
                 wvc.query.Filter.by_property("published_at").greater_than(since_date_iso)
)
```

Qdrant

```
# Python / Qdrant
results = qdrant.search(
    collection_name = "articles",
    query_vector     = query_embedding,
    query_filter     = Filter(must=[
        FieldCondition(key="category", match=MatchValue(value=category)),
        FieldCondition(key="published_ts", range=Range(gte=int(since_date.timestamp()))
    ]),
    limit            = 10,
    with_payload     = True
)
```

Chroma

```
# Python / Chroma
results = collection.query(
    query_texts = [query_text],
    n_results   = 10,
    where       = {
        "$and": [
            {"category": {"$eq": category}},
            {"published_ts": {"$gte": int(since_date.timestamp())}}
        ]
    },
    include = ["documents", "metadatas", "distances"]
)
```

The SQL form (pgvector) is the most readable for anyone familiar with relational query patterns. The Python forms converge toward a similar structure — vector, filter, limit — but each system’s filter syntax differs enough that switching systems requires rewriting query construction logic, which argues for an abstraction layer in any application that might migrate between backends.

Operational Comparison: Deployment, Scaling, and Durability

Understanding how each system is deployed, scaled, and backed up is as important as query API surface when evaluating production suitability.

Dimension	pgvector	Pinecone	Weaviate	Qdrant	Chroma
Deployment model	PostgreSQL extension	Managed cloud only	Self-host or WCD cloud	Self-host or Qdrant Cloud	Embedded or self-host
Horizontal scaling	PostgreSQL shard-ing/Citus	Automatic (serverless)	Distributed nodes	Distributed shards	None (single-node)
Durability model	PostgreSQL WAL + PITR	Fully managed	WAL + snapshots	WAL + snapshots	Filesystem (no WAL)
Backup/restore	pg_dump, PITR, logical replication	Fully managed	Collection snapshots	Collection snapshots	Directory copy
Multi-tenancy	Row-level security + schemas	Namespaces per index	Multi-tenancy via filters	Payload-based isolation	Collection per tenant

Dimension	pgvector	Pinecone	Weaviate	Qdrant	Chroma
Native hybrid search	External (tsvector + vector)	No (sparse+dense requires BYO sparse)	Yes (BM25 + HNSW, RRF)	Yes (sparse + dense)	No
Transactions	Full ACID	None	None	None	None
Cost model	Infrastructure cost	Per-RU/WU consumption	Infrastructure or WCD subscription	Infrastructure or Qdrant Cloud	Infrastructure (free tier)

A Note on Multi-Tenancy

Multi-tenant isolation deserves more attention than single-tenant benchmarks suggest. At moderate tenant counts (tens to hundreds), all five systems can implement per-tenant isolation through metadata filters or separate collections. At high tenant counts (thousands to millions), the strategies diverge:

- **pgvector**: PostgreSQL row-level security with `tenant_id` column scales to any tenant count without per-tenant infrastructure, though partial indexes per tenant only make sense for high-volume tenants.
- **Pinecone**: Namespaces are the natural multi-tenant primitive. High namespace counts are supported but each namespace shares the index's total capacity.
- **Weaviate**: Separate collections per tenant is the recommended pattern for strong isolation, but collection count at very high tenant scale becomes a management concern. The `multi-tenancy` module (added in later versions) provides first-class support.
- **Qdrant**: Payload-based isolation (filter by `customer_id`) is operationally simple. For strong isolation, separate collections per tenant work but carry per-collection overhead.
- **Chroma**: Separate collections per tenant is the only practical isolation strategy; at high tenant counts this becomes unmanageable.

Choosing Between Systems: A Decision Tree

Given the framework, data, and operational comparisons above, the following decision path covers the majority of production scenarios:

Start

Already running PostgreSQL and vectors < 10M?

YES → pgvector. Add HNSW index, deploy as PostgreSQL extension.

Need ACID consistency between vector and non-vector data?

YES → pgvector. Transactions across embeddings and relational metadata require a relational engine.

Vectors > 50M OR need sub-20ms P99 OR pure vector-only workload?

YES → Go to purpose-built

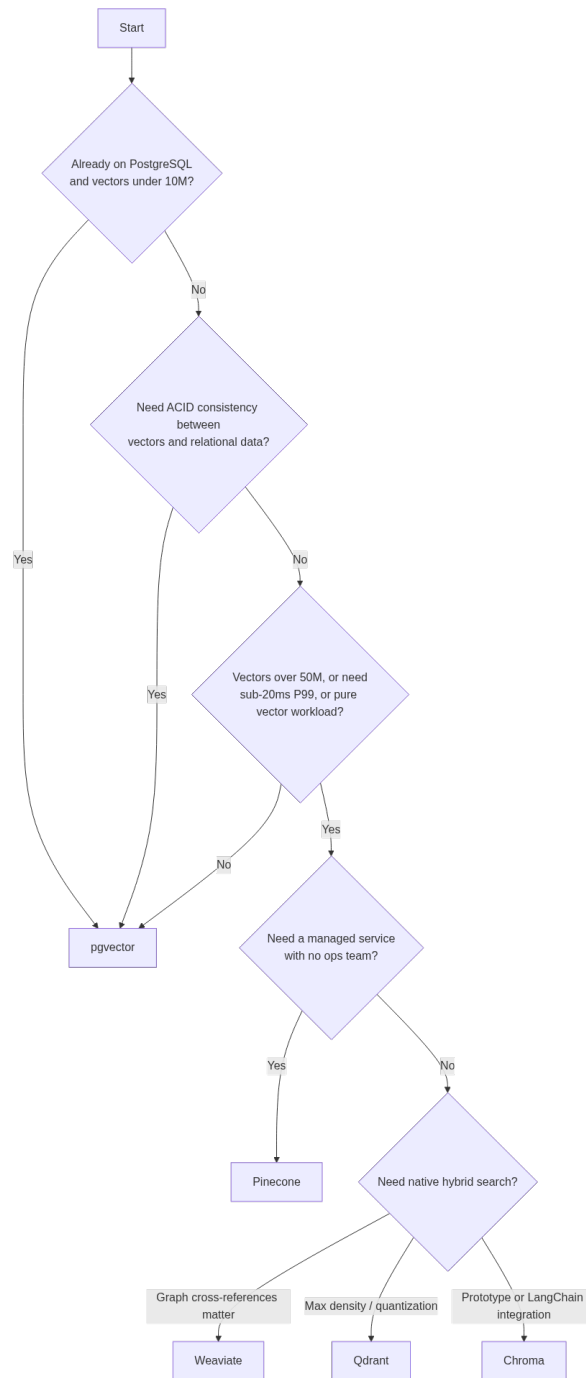


Figure 3: The decision tree in prose form below — the diagram is a visual shortcut to the same logic.

Need managed service, no ops team for vector infrastructure?

YES → Pinecone (serverless for variable load, pod for steady state)

Need native hybrid search (keyword + semantic in one query)?

YES → Weaviate or Qdrant

Graph cross-references between objects matter?

YES → Weaviate

Maximum vector density / quantization / Rust performance?

YES → Qdrant

Prototype, evaluation, offline pipeline, or LangChain integration?

YES → Chroma (with migration plan when volume grows)

None of the above? Start with pgvector, migrate when constraints appear.

The last branch deserves emphasis. The operational cost of migrating from pgvector to a purpose-built system — re-indexing vectors, migrating embeddings, rewriting query code — is real but manageable if planned. Starting with pgvector and migrating when your workload demonstrates a genuine need for purpose-built performance is often lower total cost than provisioning Pinecone infrastructure for a workload that never grows past 2M vectors.

Key Takeaways

- **pgvector is the pragmatic default for teams already on PostgreSQL:** it adds vector similarity search without introducing a new operational system, supports ACID transactions, and handles workloads up to tens of millions of vectors on well-tuned hardware before requiring architectural changes.
- **Pinecone eliminates infrastructure management at the cost of SQL and ACID:** its serverless model is the fastest path to production for teams without dedicated database operations capability, but every relational query pattern (joins, aggregations, cross-record logic) must be implemented in application code around the vector API.
- **Weaviate's hybrid search is its primary production differentiator:** the combination of BM25 and HNSW scoring in a single query path, fused via RRF, consistently outperforms pure vector search on real-world document retrieval benchmarks, making it the strongest choice for knowledge base and document retrieval applications.
- **Qdrant's pre-filter architecture and quantization support make it the performance choice for metadata-heavy workloads:** in-traversal filter evaluation prevents the recall degradation that post-filter systems exhibit under high selectivity, and scalar/binary quantization reduces memory requirements substantially with controlled recall tradeoffs.
- **Chroma accelerates the prototype-to-feedback loop but is not a production-scale system:** its embedded mode and LangChain-native integration make early iteration fast, but the absence of horizontal scaling and WAL-based durability mean a planned migration to a more capable system should be part of any architecture that expects to grow beyond a few million vectors.

Chapter 5: When to Use pgvector vs a Dedicated Vector Database

The question practitioners ask most often when entering the vector database space is not how to build embeddings—it is where to store them. `pgvector` brings vector search into PostgreSQL, the database most teams already operate. Dedicated vector databases like Pinecone, Weaviate, Qdrant, and Milvus were purpose-built for embedding workloads from the ground up. Both camps have vocal advocates, and the honest answer is that both are correct under different conditions. This chapter gives you a framework for making that decision with precision—based on scale thresholds, operational posture, query patterns, and what your application actually needs at production load—not based on hype or vendor preference.

5.1 Understanding What You Are Actually Choosing Between

Before comparing tools, clarify what the choice involves architecturally. `pgvector` is a PostgreSQL extension. It adds a `vector` data type, two index types (IVFFlat and HNSW), and a family of distance operators to a relational engine that has been in production for over thirty years. The value proposition is transactional consistency, JOINS, ACID guarantees, and a single operational footprint. You already have monitoring, backups, replication, connection pooling, and team expertise around PostgreSQL. Adding `pgvector` means adding one extension, not one more managed service.

Dedicated vector databases are purpose-built systems where the entire storage engine, indexing layer, and query planner are optimized around one problem: finding approximate nearest neighbors at speed across large, high-dimensional vector collections. They make different engineering tradeoffs—often sacrificing ACID transactions, complex relational queries, and familiar tooling in exchange for throughput, recall at scale, and operational features like automatic shard rebalancing, real-time index updates without degradation, and native multi-tenancy patterns.

Think of it the way a DBA thinks about choosing between a relational database and a columnar analytics store. Both store data. Both answer queries. The choice depends on access patterns, scale, and what you are willing to give up on each side.

The relevant comparison axes are:

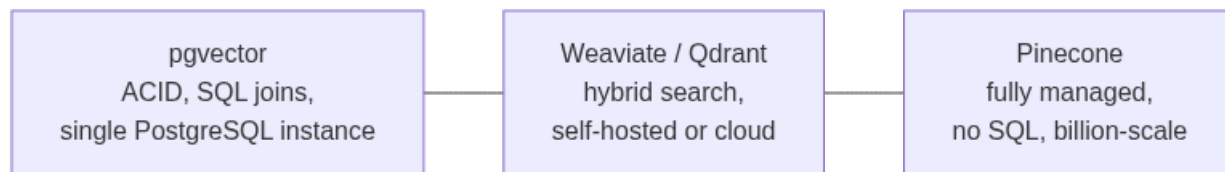


Figure 4: The vector database spectrum: pgvector trades scale ceiling for relational integration; Pinecone trades relational features and SQL for zero-ops horizontal scale.

Dimension	pgvector	Dedicated Vector DB
Vector index types	IVFFlat, HNSW	HNSW, DiskANN, IVF, quantization variants
Max practical vector count	~5–50M (hardware-dependent)	Billions (Pinecone, Milvus)
ANN recall at scale	Good up to ~10M	Tunable at 100M+
Transactional consistency	Full ACID	Eventual / limited
Hybrid queries (vector + relational)	Native SQL JOINS	Limited or workaround-based
Operational complexity	Low (already running PG)	Medium to high (new service)
Managed cloud options	Supabase, Neon, RDS, AlloyDB	Pinecone, Weaviate Cloud, Zilliz

5.2 pgvector in Practice: Setup, Indexing, and Performance Baselines

Let us establish pgvector’s operational baseline before making comparisons. The following setup reflects a realistic production schema for a document retrieval application storing OpenAI `text-embedding-3-small` embeddings (1536 dimensions).

```

-- PostgreSQL / pgvector
-- Install extension (requires PostgreSQL 14+ and pgvector 0.5+)
CREATE EXTENSION IF NOT EXISTS vector;

-- Document chunks table with embeddings
CREATE TABLE document_chunks (
  id          BIGSERIAL PRIMARY KEY,
  document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  chunk_index INTEGER NOT NULL,
  content     TEXT NOT NULL,
  metadata    JSONB,
  embedding   vector(1536),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Scalar filtering index for common metadata queries
CREATE INDEX idx_chunks_document_id ON document_chunks(document_id);
CREATE INDEX idx_chunks_metadata_source
  ON document_chunks USING gin(metadata jsonb_path_ops);

-- HNSW index for approximate nearest neighbor search
-- m=16, ef_construction=64 is a reasonable starting point
-- Increase m to 32-64 for higher recall at cost of memory and build time
CREATE INDEX idx_chunks_embedding_hnsw

```

```

ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- At query time, set ef_search for recall/speed tradeoff
-- ef_search=40 is pgvector default; raise to 100-200 for higher recall
SET hnsw.ef_search = 100;

-- Semantic search query with pre-filter and JOIN
-- This is where pgvector shines: native SQL composition
SELECT
    dc.id,
    dc.content,
    dc.metadata->>'source' AS source,
    d.title,
    1 - (dc.embedding <=> $1::vector) AS cosine_similarity
FROM document_chunks dc
JOIN documents d ON d.id = dc.document_id
WHERE
    dc.metadata->>'tenant_id' = $2
    AND dc.created_at > NOW() - INTERVAL '90 days'
ORDER BY dc.embedding <=> $1::vector
LIMIT 10;

```

The `<=>` operator computes cosine distance. `pgvector` also supports `<->` (L2/Euclidean) and `<#>` (negative inner product for dot product similarity). Choose based on how your embedding model was trained—OpenAI recommends cosine similarity for their models.

Now connect an embedding pipeline in Python:

```

# Python
import os
import psycopg2
from psycopg2.extras import execute_values
from openai import OpenAI

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

DB_DSN = os.environ["PG_DSN"]
# e.g. "postgresql://user:pass@localhost:5432/mydb"

def get_embedding(text: str, model: str = "text-embedding-3-small") -> list[float]:
    response = openai_client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

def upsert_chunks(chunks: list[dict], conn) -> None:
    """
    chunks: list of dicts with keys: document_id, chunk_index, content, metadata
    """
    embeddings = [
        get_embedding(chunk["content"]) for chunk in chunks
    ]
    rows = [
        (
            chunk["document_id"],
            chunk["chunk_index"],
            chunk["content"],
            psycopg2.extras.Json(chunk["metadata"]),
            embedding,
        )
        for chunk, embedding in zip(chunks, embeddings)
    ]
    with conn.cursor() as cur:
        execute_values(
            cur,
            """
            INSERT INTO document_chunks
            (document_id, chunk_index, content, metadata, embedding)
            VALUES %s

```

```

        ON CONFLICT (document_id, chunk_index)
        DO UPDATE SET
            content = EXCLUDED.content,
            metadata = EXCLUDED.metadata,
            embedding = EXCLUDED.embedding
    """
    rows,
    template="%s, %s, %s, %s, %s::vector)",
)
conn.commit()

def semantic_search(
    query: str,
    tenant_id: str,
    top_k: int = 10,
    ef_search: int = 100,
) -> list[dict]:
    query_embedding = get_embedding(query)

    with psycopg2.connect(DB_DSN) as conn:
        with conn.cursor() as cur:
            cur.execute(f"SET hnswef_search = {ef_search}")
            cur.execute(
                """
                SELECT
                    dc.id,
                    dc.content,
                    dc.metadata->>'source' AS source,
                    d.title,
                    1 - (dc.embedding <=> %s::vector) AS similarity
                FROM document_chunks dc
                JOIN documents d ON d.id = dc.document_id
                WHERE dc.metadata->>'tenant_id' = %s
                ORDER BY dc.embedding <=> %s::vector
                LIMIT %s
                """
                (query_embedding, tenant_id, query_embedding, top_k),
            )
            rows = cur.fetchall()

    return [
        {
            "id": row[0],
            "content": row[1],
            "source": row[2],
            "title": row[3],
            "similarity": float(row[4]),
        }
        for row in rows
    ]

```

Performance baselines you should expect from this setup:

- 1M vectors, 1536 dimensions, HNSW (m=16), ef_search=100: ~5–15ms per query on a 16-core machine with sufficient RAM to hold the index (approximately 6–8 GB for 1M × 1536-dim float32 vectors).
- At 5M vectors: query latency climbs to 30–80ms depending on hardware and index parameters, which is acceptable for many applications.
- At 20M vectors: HNSW index size exceeds 120 GB for 1536 dimensions. You need SSDs with high IOPS, careful `shared_buffers` and `effective_cache_size` tuning, and you will start hitting the ceiling of what pgvector can comfortably handle on a single node.

The memory pressure is the primary scaling constraint for pgvector. HNSW requires the graph structure to fit in memory for fast traversal. When it does not, performance degrades significantly as it falls back to disk reads.

5.3 When pgvector Is the Right Answer

pgvector wins decisively in several well-defined scenarios. Recognize these patterns in your own application before reaching for a new service.

You already have relational data that must be joined with vector results. This is the most important pgvector advantage. If your application needs to filter by user permissions, tenant, subscription tier, date ranges, or any attribute stored in a relational schema, pgvector can perform pre-filter queries that combine B-tree index scans with ANN search in a single query plan. Dedicated vector databases have metadata filtering, but it is bolt-on—you express filters as key-value pairs against a limited metadata payload, not as SQL predicates across a normalized schema. Weaviate and Qdrant have improved their filtering significantly, but nothing matches the expressiveness of SQL when your data model is already relational.

Your vector count is below 5 million. Below this threshold, pgvector with HNSW delivers sub-20ms queries reliably on commodity cloud instances. For most enterprise internal tools, document search, knowledge base applications, and recommendation features serving thousands to hundreds of thousands of users, 5M vectors is a ceiling you will not approach for years. There is no reason to add operational complexity for scale you do not have.

You need transactional consistency between vector data and metadata. Consider an e-commerce catalog where a product's embedding and its price, inventory status, and description must be updated atomically. With pgvector, this is a single `UPDATE` in a transaction. With a dedicated vector database, you are managing two separate systems—your relational database for structured attributes and your vector database for embeddings—and you must handle the consistency window between them in application code. That consistency gap is a real operational burden.

Your team runs PostgreSQL and nothing else. Operational expertise is a legitimate engineering resource. If your DBAs know PostgreSQL deeply—replication, failover, vacuuming, query plan analysis, connection pooling—adding pgvector costs almost nothing in operational overhead. Adding Pinecone or Weaviate adds a new observability surface, new failure modes, new cost centers, and new on-call burden. That cost is not zero.

Managed services reduce pgvector's operational cost further. Supabase and Neon both offer pgvector as a first-class managed feature. Neon's serverless PostgreSQL with autoscaling is particularly useful for applications with variable embedding workloads.

```
# shell
# Supabase CLI: create a new project with pgvector enabled by default
npx supabase init
npx supabase start

# Neon: provision a serverless PostgreSQL instance
# Install Neon CLI
npm install -g neonctl

# Create project
neonctl projects create --name "vector-app" --region-id "aws-us-east-2"

# Get connection string and connect
neonctl connection-string --project-id <project-id>

# pgvector is available on both Neon and Supabase without any manual installation
# Just run: CREATE EXTENSION vector;
```

5.4 When Dedicated Vector Databases Win

The case for dedicated vector databases is not about features checklist marketing. It is about specific engineering constraints that emerge at scale.

Vector counts above 20–50 million. At this scale, a single pgvector node cannot hold the HNSW graph in memory. You can tune around this with IVFFlat (which is more memory-efficient but has lower recall) or accept higher latency from disk-based HNSW traversal, but you are fighting the architecture. Pinecone, Milvus, and Weaviate are built around distributed indexing. They shard vector collections across nodes, rebalance automatically as data grows, and maintain query latency SLAs at hundreds of millions of vectors. This is what they were built to do.

High write throughput with real-time freshness requirements. pgvector’s HNSW index is built and updated in-place, but index maintenance under heavy concurrent write load degrades query performance. If your application ingests tens of thousands of new vectors per minute while simultaneously serving ANN queries, you will see query latency spikes during index updates on pgvector. Qdrant and Weaviate use background indexing with HNSW segment merging that separates write and query paths more cleanly. Pinecone’s managed index handles this transparently.

Native multi-tenancy at scale. If you are building a SaaS product where each tenant has their own isolated vector namespace with potentially millions of vectors, managing this in pgvector requires row-level security, careful index design, and connection pooling per tenant—manageable but complex. Pinecone’s namespaces and Weaviate’s multi-tenancy features (tenant-isolated shards in v1.20+) handle this as a first-class concern.

Sub-millisecond ANN queries at high QPS. Pinecone in particular is engineered for query latency at high throughput. If your application serves 1,000+ queries per second against a 50M vector collection with p99 latency requirements under 10ms, you are in territory where a managed, purpose-built system has a structural advantage. This is the regime of large-scale recommendation systems and real-time personalization engines, not typical enterprise document retrieval.

Here is a production-grade Pinecone example demonstrating the operational pattern:

```
# Python
import os
from pinecone import Pinecone, ServerlessSpec
from openai import OpenAI

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

INDEX_NAME = "document-chunks-prod"
DIMENSION = 1536 # text-embedding-3-small

# Create index (one-time setup)
# Serverless on AWS us-east-1 - no infrastructure to manage
if INDEX_NAME not in [idx.name for idx in pc.list_indexes()]:
    pc.create_index(
        name=INDEX_NAME,
        dimension=DIMENSION,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1"),
    )

index = pc.Index(INDEX_NAME)
```

```

def upsert_to_pinecone(chunks: list[dict]) -> None:
    """
    chunks: list of dicts with id, content, metadata
    Pinecone expects: id (str), values (list[float]), metadata (dict)
    """
    texts = [chunk["content"] for chunk in chunks]

    # Batch embedding - OpenAI supports up to 2048 inputs per request
    response = openai_client.embeddings.create(
        input=texts,
        model="text-embedding-3-small",
    )
    embeddings = [item.embedding for item in response.data]

    vectors = [
        {
            "id": chunk["id"],
            "values": embedding,
            "metadata": {
                "content": chunk["content"],
                "source": chunk["metadata"].get("source", ""),
                "tenant_id": chunk["metadata"].get("tenant_id", ""),
                "created_at": chunk["metadata"].get("created_at", ""),
            },
        }
        for chunk, embedding in zip(chunks, embeddings)
    ]

    # Pinecone upsert - batch size 100 is recommended
    batch_size = 100
    for i in range(0, len(vectors), batch_size):
        index.upsert(
            vectors=vectors[i : i + batch_size],
            namespace=chunks[i]["metadata"].get("tenant_id", "default"),
        )

def search_pinecone(
    query: str,
    tenant_id: str,
    top_k: int = 10,
    source_filter: str | None = None,
) -> list[dict]:
    query_embedding = openai_client.embeddings.create(
        input=query,
        model="text-embedding-3-small",
    ).data[0].embedding

    # Metadata filter - Pinecone filter syntax is MongoDB-style
    filter_expr = {}
    if source_filter:
        filter_expr["source"] = {"$eq": source_filter}

    results = index.query(
        vector=query_embedding,
        top_k=top_k,
        namespace=tenant_id,
        filter=filter_expr if filter_expr else None,
        include_metadata=True,
    )

    return [
        {
            "id": match.id,
            "score": match.score,
            "content": match.metadata.get("content", ""),
            "source": match.metadata.get("source", ""),
        }
        for match in results.matches
    ]

```

Pinecone’s serverless tier eliminates cluster management entirely. You pay per query and storage unit, not for idle compute—a meaningful operational and cost advantage for workloads with bursty or unpredictable traffic patterns.

5.5 Hybrid Approaches and Migration Paths

The binary framing of “pgvector vs dedicated” obscures the most common production pattern: starting with pgvector and migrating when you cross a scale threshold. Design for this from the beginning.

The migration trigger is usually one of three signals: HNSW index size exceeds available RAM on your largest affordable instance, query latency p95 exceeds your SLA at peak load, or write throughput causes measurable index maintenance latency spikes. Instrument these metrics from day one.

The abstraction layer that makes migration tractable is a thin retrieval interface in your application code. If your RAG pipeline calls `retriever.search(query, filters)` rather than speaking directly to PostgreSQL, swapping the backend is a configuration change, not a refactor.

LangChain and LlamaIndex both provide this abstraction:

```
# Python
# LangChain vectorstore abstraction - swap backends without changing RAG logic
import os
from langchain_openai import OpenAIEmbeddings
from langchain_postgres import PGVector # pgvector backend
from langchain_pinecone import PineconeVectorStore # Pinecone backend
from langchain_weaviate import WeaviateVectorStore # Weaviate backend

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# --- pgvector backend (development / early production) ---
PG_DSN = os.environ["PG_DSN"]

pgvector_store = PGVector(
    embeddings=embeddings,
    collection_name="document_chunks",
    connection=PG_DSN,
    use_jsonb=True,
)

# --- Pinecone backend (scale-out path) ---
from pinecone import Pinecone
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])

pinecone_store = PineconeVectorStore(
    index=pc.Index("document-chunks-prod"),
    embedding=embeddings,
    text_key="content",
)

# --- Weaviate backend (alternative: self-hosted, richer filtering) ---
import weaviate
wv_client = weaviate.connect_to_local() # or connect_to_weaviate_cloud()

weaviate_store = WeaviateVectorStore(
    client=wv_client,
    index_name="DocumentChunk",
    text_key="content",
    embedding=embeddings,
)
```



```
# Your RAG retriever is backend-agnostic
def build_retriever(backend: str = "pgvector"):
    store_map = {
        "pgvector": pgvector_store,
        "pinecone": pinecone_store,
        "weaviate": weaviate_store,
    }
    store = store_map[backend]
    return store.as_retriever(
        search_type="similarity",
        search_kwargs={"k": 10},
    )

# Usage - switch VECTOR_BACKEND env var to migrate
retriever = build_retriever(backend=os.environ.get("VECTOR_BACKEND", "pgvector"))
```

For teams running Weaviate or Qdrant rather than Pinecone, note that Weaviate Cloud and Qdrant Cloud both offer managed hosting, while Qdrant also supports Docker and Kubernetes deployment for teams who prefer to operate their own infrastructure and want the flexibility of DiskANN-based indexing at scale.

When migrating, do not do a hard cutover. Run both systems in parallel, shadow-write to the new backend, and validate recall quality against your evaluation dataset before shifting query traffic. This is the same dual-write pattern used in database migrations—the principles transfer directly.

5.6 Making the Decision: A Practical Framework

Reduce the decision to a checklist you can run through in thirty minutes for any new project or existing system approaching a scale threshold.

Start with pgvector if all of the following are true: - Your estimated vector count at 18-month growth is below 10 million - Your team already operates PostgreSQL - Your application logic requires JOINS or transactional consistency between vector and relational data - Query latency requirements are relaxed enough to tolerate 10–50ms p95

Evaluate dedicated vector databases if any of the following are true: - You are projecting above 20 million vectors within 12 months - You need sub-5ms p99 at 500+ QPS - Your write throughput exceeds 10,000 vectors per minute continuously - Multi-tenancy isolation is a hard product requirement at large per-tenant scale - Your vectors are the primary data, not a secondary index on relational data

Use Pinecone when you want zero infrastructure management and are comfortable with a fully managed SaaS cost model. Its serverless tier is the fastest path to production with no operational burden.

Use Weaviate when you need richer hybrid search (BM25 + vector combined natively), a GraphQL query interface, or want self-hosted control over your data. Weaviate’s **hybrid** search combining sparse and dense retrieval is production-ready and significantly better than pgvector’s full-text search combination.

Use Qdrant when you want Rust-based performance, on-premises control, and need advanced quantization (scalar, product, binary) to reduce index memory footprint at large scale.

Use **Milvus** when you are operating at the very large end of the scale spectrum (hundreds of millions to billions of vectors) and have the engineering capacity to operate a distributed system with its full Kubernetes deployment model.

The decision is not permanent. **pgvector** is a valid production choice for a system serving millions of users if the vectors stay within its sweet spot. The teams that get into trouble are the ones who either over-engineer toward a dedicated vector database before they have the scale to justify it, or who stay on **pgvector** past the point where its limitations are causing real user-facing degradation. Know your thresholds, instrument your metrics, and make the switch when the data tells you to—not when a vendor’s benchmark slides tell you to.

Key Takeaways

- **pgvector is not a compromise—it is the right default.** For applications with fewer than roughly 10 million vectors, existing PostgreSQL infrastructure, and relational data that must be joined with search results, **pgvector** delivers production-quality ANN search without adding a new operational surface. The HNSW index with tuned `ef_search` consistently achieves low double-digit millisecond latency on commodity cloud hardware within that envelope.
- **The primary scaling constraint for pgvector is memory, not CPU.** HNSW graph traversal requires the index to reside in RAM to maintain low latency. At 1536 dimensions, a 20M-vector HNSW index consumes over 120 GB of memory. When the index no longer fits in RAM, query latency degrades non-linearly. This—not some abstract notion of architectural purity—is the practical trigger for evaluating a dedicated vector database.
- **Dedicated vector databases solve distributed scale and high write throughput, not basic ANN search.** Pinecone, Weaviate, Qdrant, and Milvus earn their place in the stack when vector counts exceed 20–50 million, write ingestion runs continuously at high volume, or p99 latency requirements at several hundred QPS demand infrastructure purpose-built for that workload. Adopting them before those thresholds are reached trades operational simplicity for no measurable user benefit.
- **Build a retrieval abstraction layer from the start.** Whether using LangChain’s `VectorStore` interface, LlamaIndex’s retriever model, or a thin internal adapter, keeping your RAG pipeline decoupled from the specific vector backend reduces migration to a configuration change rather than a codebase refactor. Shadow-write to the new backend and validate recall against a held-out evaluation dataset before shifting query traffic—the same dual-write discipline used in relational database migrations applies directly here.
- **The four dedicated databases occupy distinct positions.** Pinecone optimizes for zero infrastructure management and low-latency queries on a fully managed SaaS model. Weaviate leads on native hybrid search combining BM25 and dense retrieval with self-hosted or cloud deployment. Qdrant provides Rust-native performance with aggressive quantization options for memory reduction on-premises. Milvus targets the extreme end of the scale spectrum with a fully distributed Kubernetes architecture. Matching the tool to the actual operational and scale requirements—rather than to vendor marketing—is what separates systems that run well in production from those that are perpetually over-engineered or under-resourced.

Chapter 6: Data Types in Vector Storage: Text, Images, Audio, Code

Vector databases are, at their core, indifferent to the original form of your data. Whether you hand them a paragraph from a legal document, a spectrogram of a customer service call, a JPEG thumbnail, or a Python function, the storage layer sees only a sequence of floating-point numbers. But the path from raw data to those numbers — the embedding pipeline — varies dramatically by modality, and the choices you make there determine search quality, index behavior, and infrastructure cost more than almost anything else. This chapter walks through the four modalities that drive the majority of production vector workloads: text, images, audio, and code. For each, you will see how embeddings are generated, what dimensionality and distance metrics apply, and how to wire it into real storage systems with runnable examples.

6.1 Text Embeddings: The Dominant Workload

Text is where most teams start, and for good reason. The tooling is mature, the models are well-benchmarked, and the use cases — semantic search, RAG pipelines, document classification, duplicate detection — map directly onto problems DBAs already understand. If you have maintained a full-text search index in PostgreSQL or Elasticsearch, you already understand the tension text embeddings are meant to resolve: keyword search finds documents that share vocabulary, but not necessarily meaning. A query for “myocardial infarction” will miss documents that only use “heart attack.” Embedding-based search collapses that gap.

How text embeddings work

A text embedding model takes a string and outputs a dense vector. The vector encodes semantic meaning in a high-dimensional space: documents about similar topics land near each other under cosine similarity or dot product distance. The dimensionality ranges widely — 384 dimensions for fast models like `all-MiniLM-L6-v2`, 1536 for OpenAI’s `text-embedding-3-small`, 3072 for `text-embedding-3-large`. Higher dimension is not always better; it increases storage cost and slows index construction, and for many retrieval tasks a 768-dimensional model is indistinguishable in quality from a 3072-dimensional one.

Chunking is a database partitioning problem

Before you embed a document, you must decide how to cut it up. Embedding models have token limits: `text-embedding-3-small` supports 8,191 tokens, sentence-transformers models typically support 512 tokens. Feeding a 200-page PDF as a single string is not an option. Chunking

strategies include fixed-size windows, sentence boundaries, paragraph boundaries, and semantic chunking. The right choice depends on your retrieval pattern — the same tradeoff you make when deciding between row-level and block-level access in a traditional DBMS.

A practical baseline: 512-token chunks with 50-token overlap. The overlap prevents semantic content from being split across chunk boundaries the way a page boundary in a B-tree splits a row.

```
# Python
# Text embedding pipeline using OpenAI + pgvector
# Requirements: openai>=1.0, psycpg2-binary, tiktoken

import openai
import psycpg2
import tiktoken
from typing import List, Tuple

client = openai.OpenAI() # reads OPENAI_API_KEY from environment

def chunk_text(text: str, model: str = "text-embedding-3-small",
               chunk_tokens: int = 512, overlap_tokens: int = 50) -> List[str]:
    """Split text into overlapping token-bounded chunks."""
    enc = tiktoken.encoding_for_model(model)
    tokens = enc.encode(text)
    chunks = []
    start = 0
    while start < len(tokens):
        end = min(start + chunk_tokens, len(tokens))
        chunk_tokens_slice = tokens[start:end]
        chunks.append(enc.decode(chunk_tokens_slice))
        if end == len(tokens):
            break
        start += chunk_tokens - overlap_tokens
    return chunks

def embed_texts(texts: List[str], model: str = "text-embedding-3-small") -> List[List[float]]:
    """Batch embed up to 2048 strings per API call."""
    response = client.embeddings.create(input=texts, model=model)
    return [item.embedding for item in response.data]

def ingest_document(conn, doc_id: str, title: str, body: str) -> int:
    """Chunk, embed, and store a document. Returns number of chunks inserted."""
    chunks = chunk_text(body)
    embeddings = embed_texts(chunks)

    with conn.cursor() as cur:
        cur.executemany(
            """
            INSERT INTO document_chunks (doc_id, title, chunk_index, chunk_text, embedding)
            VALUES (%s, %s, %s, %s, %s::vector)
            """,
            [
                (doc_id, title, i, chunk, str(emb))
                for i, (chunk, emb) in enumerate(zip(chunks, embeddings))
            ]
        )
    conn.commit()
    return len(chunks)

# Schema (run once)
SCHEMA_SQL = """
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS document_chunks (
    id          BIGSERIAL PRIMARY KEY,
    doc_id      TEXT          NOT NULL,
    title       TEXT,
    chunk_index INTEGER       NOT NULL,
    chunk_text  TEXT          NOT NULL,
```

```

embedding    vector(1536) NOT NULL,
created_at   TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX IF NOT EXISTS document_chunks_embedding_idx
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
"""

# Query
def semantic_search(conn, query: str, top_k: int = 5) -> List[Tuple]:
    query_embedding = embed_texts([query])[0]
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT doc_id, title, chunk_index, chunk_text,
                   1 - (embedding <=> %s::vector) AS cosine_similarity
            FROM    document_chunks
            ORDER   BY embedding <=> %s::vector
            LIMIT   %s
            """,
            (str(query_embedding), str(query_embedding), top_k)
        )
    return cur.fetchall()

```

The `<=>` operator is pgvector’s cosine distance. One minus that value gives you cosine similarity, which is more intuitive when you need to filter by a quality threshold (e.g., `WHERE 1 - (embedding <=> query) > 0.75`).

Choosing a text embedding model

Model	Dimensions	Max Tokens	Cost / 1M tokens	Notes
text-embedding-3-small	1536	8,191	\$0.020	Best price/quality for RAG
text-embedding-3-large	3072	8,191	\$0.130	Marginal gains, 6.5× cost
all-MiniLM-L6-v2	384	512	Free (local)	Fast, low memory, good for high-volume
BAAI/bge-large-en-v1.5	1024	512	Free (local)	MTEB top-10, strong retrieval
embed-english-v3.0 (Cohere)	1024	512	\$0.100	Strong multilingual variants

For offline or high-volume workloads, BAAI/bge-large-en-v1.5 via sentence-transformers is the practical default. For OpenAI-native stacks, text-embedding-3-small covers most production cases without the cost penalty of the large variant.

6.2 Image Embeddings: Visual Similarity at Database Scale

Image search appears exotic, but the database framing is familiar: you are building an index over a feature space, and the features happen to come from a convolutional or transformer-based vision model rather than a B-tree key. The use cases span e-commerce (find visually similar products), content moderation (detect near-duplicate or policy-violating images), medical imaging (retrieve histology slides similar to a reference), and manufacturing (surface defect matching).

How image embeddings are generated

Vision models map an image to a dense vector by passing it through a neural network and extracting the activations at a specific layer — typically the penultimate layer before the classification head. CLIP (Contrastive Language-Image Pretraining) models are especially useful because they embed images and text into the same vector space, which allows cross-modal queries: you can retrieve images using a text string as the query.

Common models and their output dimensions:

- **OpenAI CLIP (ViT-B/32):** 512 dimensions
- **OpenAI CLIP (ViT-L/14):** 768 dimensions
- **Google’s ViT-G via Vertex AI:** 1408 dimensions
- **ResNet-50 (ImageNet features):** 2048 dimensions (older, rarely used for search)

CLIP at ViT-L/14 is the standard starting point for production image search. It handles both similarity search within images and text-to-image retrieval, which reduces the number of specialized indexes you need to maintain.

```
# Python
# Image embedding pipeline using CLIP + Weaviate
# Requirements: weaviate-client>=4.0, open-clip-torch, Pillow, torch

import weaviate
import open_clip
import torch
from PIL import Image
from pathlib import Path
import base64, io, json

# Load CLIP model (CPU or GPU depending on hardware)
model, _, preprocess = open_clip.create_model_and_transforms(
    "ViT-L-14", pretrained="openai"
)
model.eval()
tokenizer = open_clip.get_tokenizer("ViT-L-14")

def embed_image(image_path: str) -> list[float]:
    """Return a 768-dim CLIP vector for an image file."""
    img = preprocess(Image.open(image_path).convert("RGB")).unsqueeze(0)
    with torch.no_grad():
        features = model.encode_image(img)
        features /= features.norm(dim=-1, keepdim=True) # L2 normalize
    return features.squeeze().tolist()

def embed_text_for_image_search(text: str) -> list[float]:
    """Embed a text query into the same CLIP space as images."""
    tokens = tokenizer([text])
    with torch.no_grad():
        features = model.encode_text(tokens)
        features /= features.norm(dim=-1, keepdim=True)
    return features.squeeze().tolist()

# Weaviate schema
```

```

client = weaviate.connect_to_local() # or weaviate.connect_to_weaviate_cloud(...)

from weaviate.classes.config import Configure, Property, DataType, VectorDistances

if not client.collections.exists("ProductImage"):
    client.collections.create(
        name="ProductImage",
        vectorizer_config=Configure.Vectorizer.none(), # we supply our own vectors
        vector_index_config=Configure.VectorIndex.hnsw(
            distance_metric=VectorDistances.COSINE,
            ef_construction=128,
            max_connections=32,
        ),
        properties=[
            Property(name="product_id", data_type=DataType.TEXT),
            Property(name="file_path", data_type=DataType.TEXT),
            Property(name="description", data_type=DataType.TEXT),
        ],
    )

# Ingest images
def ingest_product_images(image_dir: str):
    collection = client.collections.get("ProductImage")
    with collection.batch.dynamic() as batch:
        for path in Path(image_dir).glob("*.jpg"):
            product_id = path.stem
            vector = embed_image(str(path))
            batch.add_object(
                properties={
                    "product_id": product_id,
                    "file_path": str(path),
                    "description": f"Product image {product_id}",
                },
                vector=vector,
            )

# Text-to-image search
def search_images_by_text(query: str, top_k: int = 10):
    query_vector = embed_text_for_image_search(query)
    collection = client.collections.get("ProductImage")
    results = collection.query.near_vector(
        near_vector=query_vector,
        limit=top_k,
        return_metadata=["distance"],
    )
    for obj in results.objects:
        print(f"{obj.properties['product_id']} | "
              f"distance={obj.metadata.distance:.4f} | "
              f"path={obj.properties['file_path']}")

# Example: search_images_by_text("red running shoes with white sole")

```

Storage considerations for images

A 768-dimensional float32 vector consumes 3,072 bytes — about 3 KB per image. At one million images, that is 3 GB of vector data before indexing overhead. The HNSW index typically adds 1.5–2× the raw vector size, putting you at roughly 6–9 GB for the index alone. That fits comfortably in the memory of a cloud instance sized for this workload. The images themselves should not go into the vector database; store them in object storage (S3, GCS) and keep only the path or a content-addressed hash in the metadata fields.

Preprocessing matters

Unlike text, images require explicit preprocessing before embedding: resize to the model’s expected input resolution (224×224 for ViT-B/32, 336×336 for ViT-L/14@336), normalize pixel values, and handle color mode. Feeding a grayscale medical image directly to a model expecting three-channel

RGB input without conversion will produce silently incorrect embeddings — the query will return results, they just will not be meaningful. Always validate your preprocessing pipeline with a small labeled set before bulk ingestion.

6.3 Audio Embeddings: Time-Series Signals as Vectors

Audio is the least common modality in general-purpose vector workloads, but it is heavily used in specific verticals: call center analytics, music recommendation, speaker verification, podcast search, and environmental sound classification. The embedding pipeline is more involved than text or images because audio must be converted from a raw waveform to a fixed-length vector in a way that captures the relevant signal while discarding noise.

Representations and models

There are two main approaches to audio embedding:

1. **Spectrogram-based:** Convert the waveform to a mel-spectrogram (a 2D time-frequency representation), then pass it through a CNN or transformer. Models: OpenL3, PANNs (Pretrained Audio Neural Networks), CLAP (Contrastive Language-Audio Pretraining).
2. **Waveform-based:** Feed raw audio samples directly to a transformer. Models: wav2vec 2.0, Whisper encoder features, EnCodec.

For semantic audio search — finding recordings similar in content or speaker — CLAP is the current standard because it supports cross-modal text-to-audio queries, the same pattern that makes CLIP useful for images.

For speaker identification, models like `speechbrain/spkrec-ecapa-voxceleb` produce 192-dimensional speaker embeddings optimized for cosine similarity between the same speaker across recordings.

```
# Python
# Audio embedding pipeline using CLAP + Qdrant
# Requirements: qdrant-client, msclap, librosa, numpy

import librosa
import numpy as np
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct, SearchRequest
)
from msclap import CLAP # Microsoft CLAP implementation

# Initialize CLAP and Qdrant
clap_model = CLAP(version="2023", use_cuda=False)
qdrant = QdrantClient(url="http://localhost:6333")

COLLECTION_NAME = "audio_clips"
VECTOR_DIM = 512 # CLAP 2023 audio embedding dimension

# Create collection
qdrant.recreate_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(size=VECTOR_DIM, distance=Distance.COSINE),
)

def embed_audio_file(file_path: str) -> np.ndarray:
    """
    Load an audio file and return a CLAP embedding.
```



```

CLAP internally resamples to 44.1 kHz and expects mono or stereo.
"""
embeddings = clap_model.get_audio_embeddings(
    [file_path], resample=True
)
vec = embeddings[0]
# L2 normalize for cosine similarity
return (vec / np.linalg.norm(vec)).tolist()

def embed_audio_text_query(text: str) -> list[float]:
    """Embed a text query into the CLAP audio space."""
    embeddings = clap_model.get_text_embeddings([text])
    vec = embeddings[0]
    return (vec / np.linalg.norm(vec)).tolist()

def ingest_audio_clips(clips: list[dict]):
    """
    clips: list of {"id": int, "file_path": str, "label": str, "duration_s": float}
    """
    points = []
    for clip in clips:
        vector = embed_audio_file(clip["file_path"])
        points.append(
            PointStruct(
                id=clip["id"],
                vector=vector,
                payload={
                    "file_path": clip["file_path"],
                    "label": clip["label"],
                    "duration_s": clip["duration_s"],
                },
            )
        )
    qdrant.upsert(collection_name=COLLECTION_NAME, points=points)

def search_audio_by_text(query: str, top_k: int = 5):
    """Find audio clips matching a natural language description."""
    query_vector = embed_audio_text_query(query)
    results = qdrant.search(
        collection_name=COLLECTION_NAME,
        query_vector=query_vector,
        limit=top_k,
        with_payload=True,
    )
    for hit in results:
        print(f"Score: {hit.score:.4f} | "
              f"Label: {hit.payload['label']} | "
              f"File: {hit.payload['file_path']}")

# Example: search_audio_by_text("dogs barking outdoors")
# Example: search_audio_by_text("angry customer complaining about billing")

```

Segmentation and temporal alignment

Long audio recordings — interviews, conference calls, podcasts — require segmentation before embedding, similar to text chunking. A common strategy is to use voice activity detection (VAD) to find speech segments, then embed each segment of 5–30 seconds independently. This gives finer retrieval granularity: instead of finding “the episode that discusses refund policies,” you can surface the 20-second clip where refund policies are explicitly mentioned.

The metadata schema for audio should carry temporal anchors: `start_time_s` and `end_time_s` for each embedded segment. These allow the retrieval result to play back or link to the exact moment in a recording, which is the audio equivalent of returning a document chunk rather than a full document.

Sampling rate and codec normalization

Audio from different sources arrives at different sample rates (8 kHz telephony, 44.1 kHz music, 16 kHz podcast) and codecs (MP3, AAC, FLAC, OGG). Most embedding models expect a specific input: wav2vec 2.0 requires 16 kHz mono PCM. Feeding 8 kHz telephone audio without upsampling, or stereo audio without downmixing, degrades embedding quality. Build a normalization step (librosa's `resample`, ffmpeg via subprocess) at the front of your ingestion pipeline and enforce it — treat it the same way you would enforce data type coercion in an ETL pipeline.

6.4 Code Embeddings: Semantic Search Across Source Files

Code is a modality that most vector database introductions skip, but it is one of the most commercially valuable. Developers search for how a function was used across a codebase, how a specific API was called, or what code handles a particular edge case. Traditional regex-based code search (`grep`, `ripgrep`) is syntactically exact; embedding-based search finds code that does the same thing expressed differently.

How code embeddings differ from text embeddings

Code has structure that plain text does not: function signatures, indentation scope, docstrings, variable names, and control flow. Models trained specifically on code — `microsoft/codebert-base`, `Salesforce/codet5-base`, OpenAI's `text-embedding-3-small` with code content, and `voyage-code-2` from Voyage AI — outperform generic text models on code retrieval tasks because they understand that `def add(a, b): return a + b` and `func sum(x int, y int) int { return x + y }` are semantically equivalent even across languages.

`voyage-code-2` from Voyage AI currently leads benchmarks for code retrieval, producing 1536-dimensional embeddings. For teams already on the OpenAI stack, `text-embedding-3-small` performs well enough on code and avoids introducing a second embedding provider.

Chunking strategies for code

File-level embedding loses too much precision; function-level or class-level chunking is the practical standard. Parse the AST of each source file and extract functions, methods, and classes as individual embedding units. Include the docstring, signature, and body as the text passed to the embedding model. For Python, the `ast` module handles this natively; for multi-language codebases, `tree-sitter` provides a consistent AST interface across languages.

```
# Python
# Code embedding pipeline using tree-sitter + Pinecone + Voyage AI
# Requirements: pinecone-client>=3.0, voyageai, tree-sitter, tree-sitter-python

import ast
import textwrap
from pathlib import Path
import voyageai
from pinecone import Pinecone, ServerlessSpec

voyage = voyageai.Client() # reads VOYAGE_API_KEY from environment
pc = Pinecone()           # reads PINECONE_API_KEY from environment

INDEX_NAME = "codebase-search"
DIMENSION = 1536           # voyage-code-2 output dimension

# Create Pinecone index (serverless, cosine)
if INDEX_NAME not in [idx.name for idx in pc.list_indexes()]:
    pc.create_index(
```

```

        name=INDEX_NAME,
        dimension=DIMENSION,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1"),
    )

index = pc.Index(INDEX_NAME)

# Extract Python functions from source
def extract_python_functions(source_path: str) -> list[dict]:
    """
    Parse a Python file and return one dict per function/method,
    containing the name, source text, and file path.
    """
    source = Path(source_path).read_text(encoding="utf-8")
    tree = ast.parse(source)
    units = []

    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            func_lines = source.splitlines()[
                node.lineno - 1 : node.end_lineno
            ]
            func_text = textwrap.dedent("\n".join(func_lines))
            units.append({
                "name": node.name,
                "file_path": source_path,
                "start_line": node.lineno,
                "end_line": node.end_lineno,
                "source": func_text,
            })

    return units

def embed_code_units(units: list[dict]) -> list[list[float]]:
    """Embed a list of code units using voyage-code-2."""
    texts = [u["source"] for u in units]
    response = voyage.embed(texts, model="voyage-code-2", input_type="document")
    return response.embeddings

def ingest_python_file(file_path: str, namespace: str = ""):
    """Parse, embed, and upsert all functions from a Python file."""
    units = extract_python_functions(file_path)
    if not units:
        return

    embeddings = embed_code_units(units)

    vectors = []
    for unit, embedding in zip(units, embeddings):
        # Pinecone IDs must be strings; use file+line as a stable key
        vec_id = f"{unit['file_path']}::{unit['name']}::{unit['start_line']}"
        vectors.append({
            "id": vec_id,
            "values": embedding,
            "metadata": {
                "name": unit["name"],
                "file_path": unit["file_path"],
                "start_line": unit["start_line"],
                "end_line": unit["end_line"],
                "source": unit["source"][:1000], # Pinecone metadata is size-limited
            },
        })

    # Pinecone recommends upsert batches of 100 or fewer
    batch_size = 100
    for i in range(0, len(vectors), batch_size):
        index.upsert(vectors=vectors[i : i + batch_size], namespace=namespace)

def ingest_directory(root_dir: str, namespace: str = ""):

```

```

"""Walk a directory tree and ingest all .py files."""
for path in Path(root_dir).rglob("*.py"):
    print(f"Ingesting {path}")
    ingest_python_file(str(path), namespace=namespace)

# Search
def search_code(query: str, top_k: int = 5, namespace: str = "") -> list[dict]:
    """
    Embed a natural language or code query and retrieve matching functions.
    """

    response = voyage.embed([query], model="voyage-code-2", input_type="query")
    query_vector = response.embeddings[0]

    results = index.query(
        vector=query_vector,
        top_k=top_k,
        include_metadata=True,
        namespace=namespace,
    )

    matches = []
    for match in results.matches:
        matches.append({
            "score": match.score,
            "name": match.metadata["name"],
            "file_path": match.metadata["file_path"],
            "start_line": match.metadata["start_line"],
            "source": match.metadata["source"],
        })
    return matches

# Usage
# ingest_directory("/home/user/myproject/src")
# results = search_code("function that validates JWT tokens")
# for r in results:
#     print(f"{r['score']:.4f} | {r['file_path']}:{r['start_line']} | {r['name']}")

```

Multi-language codebases

When your codebase spans multiple languages — Go services, Python scripts, TypeScript frontends, SQL stored procedures — you have two practical options:

1. **Single shared index:** Embed all code regardless of language into the same Pinecone or pgvector index. This works well when queries are natural language (“find the function that handles payment retries”), because the model maps both the query and code across languages into a shared semantic space. Include `language` as a metadata field and filter when needed.
2. **Per-language indexes:** Maintain separate indexes or namespaces per language. This is useful when queries are themselves code (“find Go functions similar to this Python function”) because it lets you tune chunking, model selection, and distance thresholds per language without cross-contamination.

For most teams, the single shared index is simpler to operate and surprisingly effective.

SQL and database code

SQL stored procedures, views, and migration scripts are a special case worth calling out. Developers routinely need to find queries that JOIN a particular table, update a specific column, or implement a pattern like a slowly changing dimension. Embedding SQL is straightforward — feed the SQL text to the embedding model — but chunking differs: a SQL file may contain dozens of independent statements. Split on statement boundaries (semicolons, GO delimiters in SQL Server, \$\$ delimiters in PostgreSQL) and embed each statement independently.

The metadata fields that pay off for SQL code search: `database_name`, `schema_name`, `object_type` (table, view, procedure, trigger), `tables_referenced` (extracted by parsing), and `last_modified`. This turns your vector database into a semantic catalog layer on top of your existing database assets.

6.5 Multi-Modal Pipelines: Combining Modalities in One Index

Real applications rarely deal with a single modality in isolation. A product catalog has a name (text), a description (text), images, and potentially a promotional audio clip. A customer support system has transcribed calls (text derived from audio), agent notes (text), and screenshots of error messages (images). Building a single search experience across all of these requires a decision: do you use a single unified embedding space, or do you maintain separate indexes and fuse results at query time?

Unified embedding space with CLIP/CLAP

CLIP and CLAP models are trained to project images and text (or audio and text) into a shared vector space using contrastive learning. A text query lands near matching images, and image queries land near matching captions. This makes it possible to maintain a single index and query it with any modality.

The limitation is that these shared spaces are trained for specific modality pairs. CLIP handles image text; CLAP handles audio text; there is no single off-the-shelf model that embeds text, images, audio, and code into one coherent space. Unified multi-modal models exist (Google's Gemini, Meta's ImageBind) but they are not yet standardized in the same way that CLIP is for image-text tasks.

Late fusion with per-modality indexes

The alternative is to maintain separate indexes per modality and fuse results at query time. A search query is embedded by each relevant model, each index returns its top-K results, and a fusion layer combines the ranked lists. Reciprocal Rank Fusion (RRF) is the standard algorithm for this:

```
# Python
# Reciprocal Rank Fusion for multi-modal result merging
from collections import defaultdict
from typing import Any

def reciprocal_rank_fusion(
    ranked_lists: list[list[Any]],
    id_key: str = "id",
    k: int = 60,
) -> list[dict]:
    """
    Merge multiple ranked result lists using RRF.
    Each item in a ranked_list must be a dict with a unique id_key.
    k=60 is the standard constant that dampens the influence of high ranks.
    """
    scores: dict[Any, float] = defaultdict(float)
    items: dict[Any, dict] = {}

    for ranked_list in ranked_lists:
        for rank, item in enumerate(ranked_list, start=1):
            item_id = item[id_key]
            scores[item_id] += 1.0 / (k + rank)
            items[item_id] = item # keep the last-seen metadata
```

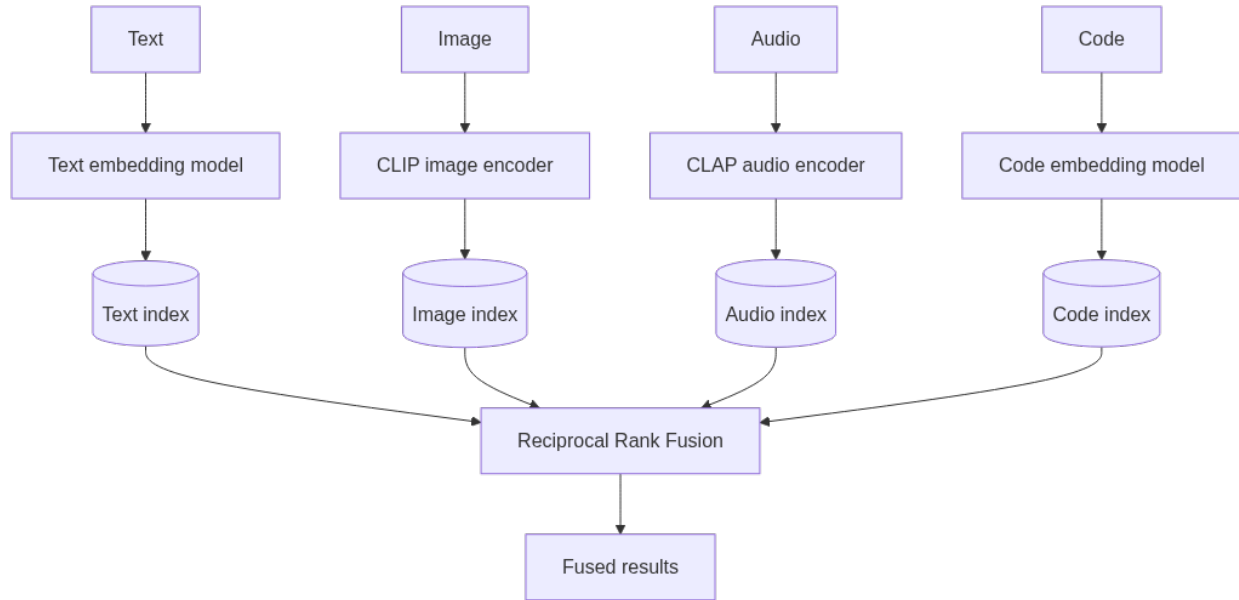


Figure 5: Each modality gets its own embedding model and index; results are merged at query time via Reciprocal Rank Fusion rather than forced into one shared vector space.

```

fused = sorted(
    [{"rrf_score": score, **items[item_id]} for item_id, score in scores.items()],
    key=lambda x: x["rrf_score"],
    reverse=True,
)
return fused

# Example: fuse text search and image search results for a product catalog
# text_results = [{"id": "prod_001", "name": "Red Sneaker", ...}, ...]
# image_results = [{"id": "prod_003", "name": "Trail Runner", ...}, ...]
# fused = reciprocal_rank_fusion([text_results, image_results])

```

RRF does not require score normalization across modalities, which is a significant practical advantage: cosine similarity scores from a text index are not directly comparable to distance scores from an image index, but rank positions are comparable.

Metadata alignment across modalities

When operating a multi-modal pipeline, the metadata schema becomes the spine that connects records across indexes. A `product_id` field that exists in both the text chunk table and the image vector collection is what makes post-retrieval joining possible. Design your metadata schema before you build the ingestion pipelines, or you will find yourself with three indexes that cannot be joined without a full table scan.

A practical metadata pattern for a product catalog:

```

-- PostgreSQL: unified metadata table that all modality indexes reference
CREATE TABLE product_assets (
    asset_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id    TEXT NOT NULL,
    modality      TEXT NOT NULL CHECK (modality IN ('text', 'image', 'audio', 'code')),
    source_ref    TEXT NOT NULL,    -- chunk text, S3 path, git SHA, etc.
    embedding_dim INTEGER NOT NULL,
    index_name    TEXT NOT NULL,    -- which external index holds the vector

```

```

    external_id TEXT NOT NULL,    -- the vector's ID in that index
    created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON product_assets (product_id);
CREATE INDEX ON product_assets (modality, index_name);

```

This table acts as a reverse-lookup registry: given a vector ID returned by Pinecone or Weaviate, you can join back to the canonical `product_id` and fetch any other modality’s vectors or metadata.

6.6 Distance Metrics by Modality

Not all vector spaces are created equal, and the choice of distance metric affects both retrieval quality and index performance. The table below summarizes the practical defaults by modality:

Modality	Typical Model Family	Recommended Metric	Notes
Text	Sentence-transformers, OpenAI	Cosine similarity	Vectors are L2-normalized by most models
Text (dot product tuned)	Matryoshka / OpenAI v3	Dot product	Only if model is trained with dot product objective
Images	CLIP variants	Cosine similarity	L2-normalize outputs before storage
Audio (semantic)	CLAP	Cosine similarity	Same contrastive training as CLIP
Audio (speaker ID)	ECAPA-TDNN, x-vectors	Cosine similarity	Speaker embeddings respond well to cosine
Code	voyage-code-2, CodeBERT	Cosine similarity	Normalize outputs
Binary / hash	SimHash, MinHash	Hamming distance	Use <code>bit</code> type in pgvector 0.7+

The near-universal answer is cosine similarity, which is why most production systems normalize vectors to unit length at ingestion time and then use dot product as a fast approximation (dot product on unit vectors equals cosine similarity). The pgvector operators `<=>` (cosine distance) and `<#>` (negative dot product) let you switch without changing the schema.

Euclidean (L2) distance is appropriate when the magnitude of the vector carries semantic information, which is rare in embedding models designed for retrieval — those models are typically trained with normalization objectives. Hamming distance is useful for binary fingerprinting and near-duplicate detection in image or document deduplication pipelines.

6.7 Storage Sizing and Cost Modeling

Before committing to a database architecture, teams need to estimate how much vector data they are actually dealing with. The arithmetic is simple but frequently underestimated.

Per-vector storage

A single float32 (4-byte) vector of dimension d consumes $4d$ bytes on disk. The HNSW index — the dominant index type in production vector stores — adds roughly 1.5–2× the raw vector size in graph structure overhead. A 1536-dimensional OpenAI embedding therefore consumes:

- Raw vector: $4 \times 1536 = 6,144$ bytes (6 KB)
- With HNSW overhead: 12–15 KB per vector

At one million documents with five chunks each (five million vectors):

- Raw data: $5M \times 6 \text{ KB} = 30 \text{ GB}$
- Index: $5M \times 15 \text{ KB} = 75 \text{ GB}$ total on-disk

That is a meaningful infrastructure commitment. Dimensionality reduction strategies — Matryoshka representation learning (MRL), PCA post-processing, or switching to a smaller model — can cut that by 50–75% at a modest quality cost.

Quantization

Scalar quantization (SQ8) and binary quantization (BQ) reduce vector storage at the cost of some retrieval precision. pgvector supports `halfvec` (float16, half the storage of float32) and `bit` (binary, 32× reduction from float32). Pinecone offers built-in int8 quantization. Weaviate supports both scalar and product quantization natively.

A common production pattern:

1. Store the full-precision vectors on disk for re-ranking.
2. Use a quantized version for the HNSW index to reduce memory footprint.
3. After ANN retrieval returns top-100 candidates, re-rank with exact dot product on the full-precision vectors.

This two-stage pattern maintains high recall while reducing the memory needed to serve the HNSW index, which must fit in RAM for low-latency queries.

Cost across platforms

Managed vector databases price on stored vectors, queries per second, and sometimes on the hosting tier (pod-based vs. serverless). Self-hosted pgvector running on a compute instance with adequate RAM is often the most cost-effective option at moderate scale — tens of millions of vectors — because you pay only for compute and storage, not a per-vector premium. Beyond a few hundred million vectors, the operational overhead of running a purpose-built system like Qdrant or Weaviate at scale often justifies the higher unit cost.

Key Takeaways

- **Modality determines the embedding pipeline, not the storage layer.** pgvector, Pinecone, Weaviate, and Qdrant are all modality-agnostic; what changes is the model you call before writing a vector to the index. Design your ingestion pipeline first, then select the database that fits your operational constraints.
 - **Chunking and segmentation are first-class engineering problems.** Whether you are splitting a legal document into 512-token windows, extracting Python functions from an AST, or segmenting a call recording by voice activity, how you divide data before embedding determines retrieval granularity more than model choice does.
 - **CLIP and CLAP enable cross-modal text-to-X queries without a separate query path.** By embedding images or audio into the same vector space as text, you avoid maintaining a specialized query interface for each modality — a single near-vector query handles both image similarity and text-to-image retrieval.
 - **Cosine similarity with L2-normalized vectors is the safe default across all modalities.** Euclidean distance is rarely appropriate for embedding-based retrieval; Hamming distance applies only to binary fingerprinting. When in doubt, normalize at ingestion and use cosine.
 - **Size your infrastructure before you build.** A 1536-dimensional embedding model applied to five million vectors produces roughly 75 GB of indexed data. Factor in HNSW overhead, quantization trade-offs, and the distinction between disk storage and RAM residency before selecting a hosting tier — vector indexes must fit in memory to serve low-latency queries.
-

Chapter 7: Installing and Configuring pgvector

pgvector brings vector similarity search directly into PostgreSQL, letting you store embeddings as a native column type and query them with familiar SQL semantics. For database professionals already invested in PostgreSQL infrastructure, this is the path of least resistance into production AI applications — no new operational surface, no separate replication topology, no additional monitoring stack. This chapter walks through every stage of a pgvector deployment: from binary installation through index configuration, connection pooling, and the operational habits that keep vector workloads from interfering with the transactional workloads sharing the same cluster.

Getting pgvector Installed

pgvector is a PostgreSQL extension, so the installation path depends on how PostgreSQL itself was installed and where you're running it.

On Debian/Ubuntu with the PGDG repository:

```
# shell
# Ensure you have the PostgreSQL contrib packages and build tools
sudo apt-get update
sudo apt-get install -y postgresql-server-dev-16 build-essential git

# Clone and build pgvector
cd /tmp
git clone --branch v0.7.0 https://github.com/pgvector/pgvector.git
cd pgvector
make
sudo make install

# Confirm the shared library landed in the right place
ls $(pg_config --pkglibdir)/vector.so
ls $(pg_config --sharedir)/extension/vector*
```

On RHEL/Rocky Linux 9:

```
# shell
# Install via the PostgreSQL YUM repository (preferred for production)
sudo dnf install -y pgvector_16

# If building from source on RHEL:
sudo dnf install -y postgresql16-devel gcc git make
cd /tmp && git clone --branch v0.7.0 https://github.com/pgvector/pgvector.git
cd pgvector && make && sudo make install
```

On macOS with Homebrew (development only):

```
# shell
brew install pgvector

# If PostgreSQL was installed via Homebrew separately:
brew install postgresql@16
export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"
cd /tmp && git clone --branch v0.7.0 https://github.com/pgvector/pgvector.git
cd pgvector && make && make install
```

Once the shared library is in place, enable the extension per database:

```
-- PostgreSQL / pgvector
-- Run as a superuser in the target database
CREATE EXTENSION IF NOT EXISTS vector;

-- Verify
SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';
-- extname | extversion
-- -----+-----
-- vector  | 0.7.0
```

No `postgresql.conf` changes are required to load the extension — unlike some extensions, `pgvector` does not appear in `shared_preload_libraries`. The `CREATE EXTENSION` call is sufficient.

Managed services: On AWS RDS for PostgreSQL 15+, Aurora PostgreSQL 15.2+, Google Cloud SQL for PostgreSQL 14+, Azure Database for PostgreSQL Flexible Server 13+, Supabase, and Neon, `pgvector` is available as a first-class extension requiring only the `CREATE EXTENSION` command. No binary installation is needed. Check the managed service’s documentation for the maximum supported `pgvector` version, as it typically lags the upstream release by one or two minor versions.

The vector Data Type and Basic Schema Design

`pgvector` introduces three types: `vector` (exact 32-bit floats), `halfvec` (16-bit floats, introduced in 0.7.0), and `sparsevec` (sparse vectors for models like SPLADE). The `vector` type is what you will use for the vast majority of embedding workloads.

A vector column is defined with a fixed dimensionality:

```
-- PostgreSQL / pgvector
-- Standard embedding table for a document store
CREATE TABLE documents (
    id          BIGSERIAL PRIMARY KEY,
    content     TEXT NOT NULL,
    metadata    JSONB,
    embedding   vector(1536),           -- OpenAI text-embedding-3-small dimensions
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- For models with different dimensionalities:
-- text-embedding-3-large: vector(3072)
-- text-embedding-ada-002: vector(1536)
-- Cohere embed-english-v3.0: vector(1024)
-- nomic-embed-text-v1.5: vector(768)
-- all-MiniLM-L6-v2: vector(384)

-- Insert a row with an embedding (literal syntax for testing)
INSERT INTO documents (content, metadata, embedding)
VALUES (
    'PostgreSQL is a powerful open source relational database.',
    '{"source": "manual", "category": "database"}',
    ''
);
```

```
    '[0.021, -0.014, 0.033, ...]':::vector(1536)
);
```

The dimensionality is enforced at the column level, not the type level — you cannot insert a 768-dimensional vector into a `vector(1536)` column. This is the equivalent of a type mismatch; PostgreSQL will throw an error, which is exactly the behavior you want when different embedding models produce incompatible vector spaces.

Null handling: `embedding` columns can be NULL, and NULL embeddings are excluded from similarity searches automatically. This is useful when you ingest document metadata before the embedding pipeline has processed the content.

The `halfvec` type: For production deployments where storage is a constraint, `halfvec(1536)` cuts storage per vector from 6,144 bytes to 3,072 bytes at the cost of slight precision loss. For most embedding search workloads, the precision difference is negligible — retrieval quality benchmarks on typical RAG workloads show less than 0.5% change in recall@10 when switching from `vector` to `halfvec` with OpenAI embeddings.

```
-- PostgreSQL / pgvector
-- Comparing storage footprints
SELECT
  pg_size_pretty(1536 * 4) AS vector_bytes_per_row,
  pg_size_pretty(1536 * 2) AS halfvec_bytes_per_row,
  pg_size_pretty(1536 * 4 * 1000000) AS vector_1m_rows_raw,
  pg_size_pretty(1536 * 2 * 1000000) AS halfvec_1m_rows_raw;
-- vector_bytes_per_row | halfvec_bytes_per_row | vector_1m_rows_raw | halfvec_1m_rows_raw
-----
6144 bytes             | 3072 bytes             | 5859 MB             | 2930 MB
```

Distance Operators and Query Patterns

pgvector exposes similarity search through three distance operators. Choosing the right one is not a matter of preference — it depends on how the embedding model was trained and what the model’s documentation specifies.

Operator	Name	Use case
<code><-></code>	L2 (Euclidean) distance	Default; models trained with L2 loss
<code><=></code>	Cosine distance	Most sentence transformers; text embedding models
<code><#></code>	Negative inner product	Models normalized to unit sphere; dot product ranking

All three operators return a *distance* (lower is more similar for `<->` and `<=>`; more negative is more similar for `<#>`). They do not return a similarity score directly — convert if needed: `1 - (embedding <=> query_vec)` gives cosine similarity in `[-1, 1]`.

```
-- PostgreSQL / pgvector
-- Basic nearest-neighbor query using cosine distance
-- The query embedding would typically come from application code
SELECT
  id,
```

```
content,
metadata,
1 - (embedding <=> '[0.021, -0.014, 0.033, ...]':vector) AS cosine_similarity
FROM documents
ORDER BY embedding <=> '[0.021, -0.014, 0.033, ...]':vector
LIMIT 10;

-- Pre-filtering with metadata before vector search
-- This pushes a WHERE clause before the ORDER BY/LIMIT
SELECT
id,
content,
1 - (embedding <=> $1::vector) AS cosine_similarity
FROM documents
WHERE metadata->>'category' = 'database'
AND created_at > NOW() - INTERVAL '90 days'
ORDER BY embedding <=> $1::vector
LIMIT 10;

-- Finding documents above a similarity threshold
-- Note: cosine distance threshold of 0.3 = cosine similarity >= 0.7
SELECT id, content, embedding <=> $1::vector AS distance
FROM documents
WHERE embedding <=> $1::vector < 0.3
ORDER BY embedding <=> $1::vector
LIMIT 50;
```

Without an index, these queries perform exact nearest-neighbor search via sequential scan — every row is compared to the query vector. For tables under approximately 100,000 rows, sequential scan is often fast enough and always produces exact results. Above that threshold, indexing becomes necessary to maintain acceptable query latency.

Building and Tuning Indexes

pgvector supports two index types: IVFFlat and HNSW. They represent different tradeoffs between build time, memory, query speed, and recall accuracy. Understanding this tradeoff in database terms: IVFFlat is roughly analogous to a clustered index that buckets rows into partitions, while HNSW builds a multi-layer navigable graph structure more comparable to a B-tree in that it maintains structure on insert rather than requiring a full rebuild.

IVFFlat:

```
-- PostgreSQL / pgvector
-- IVFFlat index - requires data to exist before building
-- Rule of thumb: lists = rows / 1000, capped at sqrt(rows)
-- For 1M rows: lists between 1000 and 1000 (sqrt(1M) = 1000)

CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- For a 1M-row table:
CREATE INDEX documents_embedding_ivfflat_idx ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 1000);

-- At query time, probes controls the accuracy/speed tradeoff
-- More probes = higher recall, slower query
-- Default is 1 (fast, lower recall); production typically 10-50
SET ivfflat.probes = 20;

SELECT id, content
```

```
FROM documents
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

IVFFlat requires the table to be populated before index creation — you cannot build the index on an empty table and expect good performance because the k-means clustering that determines list centroids needs representative data. This is a meaningful operational constraint: you must load your data before indexing, not the other way around.

HNSW:

```
-- PostgreSQL / pgvector
-- HNSW index - can be built incrementally, supports INSERT without rebuild
-- m: number of connections per layer (default 16, range 2-100)
-- ef_construction: size of dynamic candidate list during build (default 64)
-- Higher values = better recall, longer build time, more memory

CREATE INDEX documents_embedding_hnsw_idx ON documents
USING hnsw (embedding_vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- For higher recall requirements:
CREATE INDEX documents_embedding_hnsw_idx ON documents
USING hnsw (embedding_vector_cosine_ops)
WITH (m = 32, ef_construction = 128);

-- At query time, ef controls search quality
-- Default 40; increase for higher recall at cost of latency
SET hnsw.ef_search = 100;

SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM documents
ORDER BY embedding <=> $1::vector
LIMIT 10;

-- Check index size and build progress
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size
FROM pg_indexes
WHERE tablename = 'documents';
```

Choosing between IVFFlat and HNSW: For most production workloads starting fresh, HNSW is the better default. It supports incremental inserts without performance degradation, delivers higher recall at equivalent query times, and does not require probe tuning at query time. IVFFlat is appropriate when build time and memory during index construction are the binding constraints — HNSW index builds consume significantly more memory (roughly 10–20 bytes per dimension per row during construction).

Recall benchmarking before going to production:

```
# Python
# Measure recall@10 for your specific dataset and index configuration
# Compare ANN (approximate) results against exact sequential scan results

import psycopg2
import numpy as np
from typing import List, Tuple

def measure_recall(
    conn_string: str,
    query_vectors: List[np.ndarray],
    k: int = 10
) -> float:
    """
    Compare ANN index results against exact results to measure recall.
    """
```

```
recall@k = |ANN results  exact results| / k
"""
conn = psycopg2.connect(conn_string)

recalls = []

for qvec in query_vectors:
    vec_literal = "[" + ",".join(str(x) for x in qvec.tolist()) + "]"

    with conn.cursor() as cur:
        # Exact results via sequential scan
        cur.execute("SET enable_indexscan = OFF; SET enable_bitmapscan = OFF;")
        cur.execute(
            f"""
            SELECT id FROM documents
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """,
            (vec_literal, k)
        )
        exact_ids = {row[0] for row in cur.fetchall()}

        # ANN results via index
        cur.execute("SET enable_indexscan = ON; SET enable_bitmapscan = ON;")
        cur.execute("SET hnswef_search = 100;")
        cur.execute(
            f"""
            SELECT id FROM documents
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """,
            (vec_literal, k)
        )
        ann_ids = {row[0] for row in cur.fetchall()}

    recall = len(exact_ids & ann_ids) / k
    recalls.append(recall)

conn.close()
avg_recall = np.mean(recalls)
print(f"Recall@{k} over {len(query_vectors)} queries: {avg_recall:.4f}")
return avg_recall

# Target: recall@10 >= 0.95 for most RAG applications
# If below target, increase ef_search (HNSW) or probes (IVFFlat)
```

A recall@10 of 0.95 or above is a reasonable target for RAG applications — this means that for 95% of queries, the 10 documents most relevant to the query are present in the results. Dropping below 0.90 will be noticeable in end-user answer quality.

postgresql.conf Tuning for Vector Workloads

Vector workloads stress PostgreSQL in ways that differ from OLTP workloads. HNSW index builds are memory-intensive. Embedding columns are wide, making row sizes larger than typical. Query plans involving vector ORDER BY/LIMIT patterns behave differently from standard B-tree index scans. Several configuration parameters need attention.

```
# shell
# /etc/postgresql/16/main/postgresql.conf
# Entries relevant to vector workloads - adjust to your hardware

# Memory
shared_buffers = 8GB                                # 25% of RAM; vector index pages benefit from buffer cache
```



```

work_mem = 256MB                # HNSW build and sort operations use work_mem per operation
maintenance_work_mem = 4GB      # Controls memory available during CREATE INDEX
effective_cache_size = 24GB      # Optimizer hint for available OS + buffer cache

# Parallelism for index builds (pgvector 0.6.0+)
max_parallel_maintenance_workers = 4 # Parallel HNSW/IVFFlat index builds
max_parallel_workers = 8
max_parallel_workers_per_gather = 4

# WAL and checkpoints - vector inserts during embedding pipelines can be write-heavy
wal_buffers = 64MB
checkpoint_completion_target = 0.9
max_wal_size = 4GB

# Connection and timeout settings
statement_timeout = 60000        # 60s; vector index builds may need higher per-session override
lock_timeout = 10000

# For high-throughput embedding insert workloads, consider:
synchronous_commit = off         # Improves insert throughput; loses last ~wal_writer_delay ms on
    ↪ crash

```

Parallel index builds were added in pgvector 0.6.0 and make a substantial difference. Building an HNSW index on 1 million 1536-dimensional vectors drops from approximately 45 minutes single-threaded to around 12 minutes with 4 workers on a machine with 8 cores and 32GB RAM. Set `max_parallel_maintenance_workers` in `postgresql.conf` and optionally override per session:

```

-- PostgreSQL / pgvector
-- Override for a specific index build session
SET max_parallel_maintenance_workers = 7;

CREATE INDEX documents_embedding_hnsw_idx ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Monitor build progress (PostgreSQL 12+)
SELECT
    phase,
    blocks_done,
    blocks_total,
    tuples_done,
    tuples_total,
    ROUND(100.0 * tuples_done / NULLIF(tuples_total, 0), 1) AS pct_complete
FROM pg_stat_progress_create_index
WHERE relid = 'documents'::regclass;

```

Connection pooling: PgBouncer in transaction mode works with pgvector queries but there is an important caveat. The `SET ivfflat.probes` and `SET hnsw.ef_search` commands set session-level parameters. In transaction pooling mode, these settings do not persist across transactions — they are lost when the connection is returned to the pool. The correct pattern is to include the `SET` in every transaction or query batch, or to use `SET LOCAL` within a transaction block, or to configure `pgbouncer.ini` with `server_reset_query` to reset state.

```

-- PostgreSQL / pgvector
-- Safe pattern for transaction-mode PgBouncer: SET LOCAL within explicit transaction
BEGIN;
SET LOCAL hnsw.ef_search = 100;
SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM documents
ORDER BY embedding <=> $1::vector
LIMIT 10;
COMMIT;

```

Vacuuming and bloat: Embedding tables with high update or delete rates accumulate dead tuples just like any other table, and dead tuples inflate HNSW graph traversal costs. Ensure auto-

vacuum is not suppressed on embedding tables and monitor `pg_stat_user_tables.n_dead_tup`. For workloads where embeddings are frequently replaced (re-indexing documents with a new embedding model, for example), consider a delete-and-reinsert pattern with periodic `VACUUM ANALYZE` rather than in-place `UPDATE`.

Populating Embeddings from Python

The typical production pattern is a Python pipeline that reads document content from PostgreSQL, calls an embedding model, and writes vectors back to the same database. Here is a complete, runnable example using OpenAI's embedding API and `psycopg3`:

```
# Python
import os
import psycopg
import numpy as np
from openai import OpenAI
from typing import List, Tuple

OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
DATABASE_URL = os.environ["DATABASE_URL"] # postgresql://user:pass@host:5432/dbname

client = OpenAI(api_key=OPENAI_API_KEY)

def get_embeddings(texts: List[str], model: str = "text-embedding-3-small") -> List[List[float]]:
    """
    Batch embedding call. OpenAI allows up to 2048 inputs per request.
    text-embedding-3-small: 1536 dimensions, cost-effective for most use cases.
    text-embedding-3-large: 3072 dimensions, higher quality at higher cost.
    """
    response = client.embeddings.create(
        input=texts,
        model=model,
        encoding_format="float"
    )
    return [item.embedding for item in response.data]

def embed_documents_batch(batch_size: int = 100) -> None:
    """
    Process documents without embeddings in batches.
    Uses SKIP LOCKED to support parallel workers without conflicts.
    """
    with psycopg.connect(DATABASE_URL) as conn:
        # Count pending work
        with conn.cursor() as cur:
            cur.execute("SELECT COUNT(*) FROM documents WHERE embedding IS NULL")
            total_pending = cur.fetchone()[0]
            print(f"Documents pending embedding: {total_pending}")

        processed = 0
        while True:
            with conn.cursor() as cur:
                # Fetch a batch; SKIP LOCKED enables safe parallel processing
                cur.execute(
                    """
                    SELECT id, content FROM documents
                    WHERE embedding IS NULL
                    ORDER BY id
                    LIMIT %s
                    FOR UPDATE SKIP LOCKED
                    """,
                    (batch_size,)
                )
```

```

        rows = cur.fetchall()

    if not rows:
        break

    ids = [row[0] for row in rows]
    texts = [row[1] for row in rows]

    # Call embedding API
    embeddings = get_embeddings(texts)

    # Bulk update using executemany
    with conn.cursor() as cur:
        cur.executemany(
            "UPDATE documents SET embedding = %s::vector WHERE id = %s",
            [
                ("[" + ",".join(str(x) for x in emb) + "]", doc_id)
                for emb, doc_id in zip(embeddings, ids)
            ]
        )
    conn.commit()

    processed += len(ids)
    print(f"Processed {processed}/{total_pending} documents")

print("Embedding pipeline complete.")

def similarity_search(
    query: str,
    top_k: int = 5,
    category_filter: str = None
) -> List[Tuple]:
    """
    End-to-end similarity search: embed query, retrieve nearest neighbors.
    """
    query_embedding = get_embeddings([query])[0]
    vec_literal = "[" + ",".join(str(x) for x in query_embedding) + "]"

    filter_clause = ""
    params = [vec_literal, top_k]
    if category_filter:
        filter_clause = "WHERE metadata->'category' = %s"
        params = [vec_literal, category_filter, top_k]

    sql = f"""
    SELECT
        id,
        content,
        metadata,
        1 - (embedding <=> %s::vector) AS cosine_similarity
    FROM documents
    {filter_clause}
    ORDER BY embedding <=> %s::vector
    LIMIT %s
    """

    # Rebuild params correctly for the query structure
    if category_filter:
        params = [vec_literal, category_filter, vec_literal, top_k]
    else:
        params = [vec_literal, vec_literal, top_k]

    with psycopg.connect(DATABASE_URL) as conn:
        conn.execute("SET hnsw.ef_search = 100")
        with conn.cursor() as cur:
            cur.execute(sql, params)
            results = cur.fetchall()

    return results

```

```
if __name__ == "__main__":
    # Run the embedding pipeline
    embed_documents_batch(batch_size=100)

    # Test search
    results = similarity_search(
        query="How do I configure connection pooling in PostgreSQL?",
        top_k=5
    )
    for doc_id, content, metadata, similarity in results:
        print(f"[{similarity:.4f}] {content[:80]}...")
```

The FOR UPDATE SKIP LOCKED pattern deserves emphasis. Without it, running multiple embedding workers in parallel causes the same rows to be selected by multiple workers simultaneously, resulting in redundant API calls and update conflicts. SKIP LOCKED makes each worker atomically claim a batch that no other worker is currently processing.

Hybrid Search: Combining Vector and Full-Text

Pure vector search excels at semantic similarity but can miss exact keyword matches that a user expects. A document mentioning “PgBouncer” by name may not surface at the top of a cosine search for “PgBouncer” if the embedding space for the query overlaps more strongly with conceptually related content. Full-text search, conversely, finds exact matches but cannot generalize across paraphrases or synonyms. Combining both in a hybrid search query gives you the best of both worlds.

PostgreSQL’s `ts_vector` / `ts_query` full-text infrastructure coexists naturally with pgvector on the same table.

```
-- PostgreSQL / pgvector
-- Add a full-text search column alongside the embedding column
ALTER TABLE documents ADD COLUMN fts_vector TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', content)) STORED;

CREATE INDEX documents_fts_idx ON documents USING GIN (fts_vector);

-- Hybrid search: combine BM25-style full-text rank with cosine similarity
-- Reciprocal Rank Fusion (RRF) is a principled way to merge two ranked lists
-- RRF score = 1/(k + rank_vector) + 1/(k + rank_fts), k=60 is a common default

WITH vector_ranked AS (
    SELECT
        id,
        ROW_NUMBER() OVER (ORDER BY embedding <=> $1::vector) AS rank_v
    FROM documents
    WHERE embedding IS NOT NULL
    ORDER BY embedding <=> $1::vector
    LIMIT 100 -- oversample before fusion
),
fts_ranked AS (
    SELECT
        id,
        ROW_NUMBER() OVER (ORDER BY ts_rank_cd(fts_vector, query) DESC) AS rank_f
    FROM documents,
        plainto_tsquery('english', $2) AS query
    WHERE fts_vector @@ query
    ORDER BY ts_rank_cd(fts_vector, query) DESC
    LIMIT 100
),
```

```
fused AS (  
  SELECT  
    COALESCE(v.id, f.id) AS id,  
    COALESCE(1.0 / (60 + v.rank_v), 0) +  
    COALESCE(1.0 / (60 + f.rank_f), 0) AS rrf_score  
  FROM vector_ranked v  
  FULL OUTER JOIN fts_ranked f ON v.id = f.id  
)  
SELECT  
  d.id,  
  d.content,  
  d.metadata,  
  fused.rrf_score  
FROM fused  
JOIN documents d ON d.id = fused.id  
ORDER BY rrf_score DESC  
LIMIT 10;
```

The Reciprocal Rank Fusion formula ($1 / (k + \text{rank})$) down-weights lower-ranked results from each signal and combines them without requiring score normalization — cosine distances and BM25 scores live in different numerical ranges and cannot be directly added without calibration. RRF sidesteps this by operating purely on ranks.

For production use, parameterize both \$1 (the query vector, pre-computed in application code) and \$2 (the raw query string for full-text). A single round-trip to the database handles the entire hybrid retrieval.

Partitioning Large Embedding Tables

For tables expected to grow into tens of millions of rows, PostgreSQL’s declarative partitioning can manage index size and enable partition pruning. Common strategies are range partitioning by ingestion date and hash partitioning by a stable identifier.

```
-- PostgreSQL / pgvector  
-- Range-partitioned documents table (partition by month)  
CREATE TABLE documents_partitioned (  
  id          BIGSERIAL,  
  content     TEXT NOT NULL,  
  metadata    JSONB,  
  embedding   vector(1536),  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  PRIMARY KEY (id, created_at)  
) PARTITION BY RANGE (created_at);  
  
-- Create partitions  
CREATE TABLE documents_2024_q1  
  PARTITION OF documents_partitioned  
  FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');  
  
CREATE TABLE documents_2024_q2  
  PARTITION OF documents_partitioned  
  FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');  
  
-- HNSW indexes must be created per partition  
-- pgvector does not support global partitioned indexes  
CREATE INDEX ON documents_2024_q1  
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);  
  
CREATE INDEX ON documents_2024_q2  
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);  
  
-- Queries against the parent table benefit from partition pruning
```

```
-- when a WHERE clause on created_at eliminates irrelevant partitions
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, content
FROM documents_partitioned
WHERE created_at >= '2024-04-01' AND created_at < '2024-07-01'
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

The critical limitation to understand: each partition has its own HNSW or IVFFlat index covering only the rows in that partition. A query against the parent table that cannot be pruned to a single partition will perform parallel index scans across all partitions and merge-sort the results. For exact nearest-neighbor search over the full dataset, this works correctly. For approximate search, recall is computed independently per partition, which can degrade overall recall — the true global top-10 may be distributed across partitions, but each partition only surfaces its local approximate top-10.

Strategies to mitigate this:

1. **Oversample per partition.** Query each partition for top-100, then rerank the merged 100×N candidates in application code.
2. **Partition by a filter dimension.** If your application always filters by metadata->>'tenant_id', partition by tenant hash so each query touches exactly one partition.
3. Use `pg_partman` for automatic partition creation and management, especially for time-range partitions.

Monitoring Vector Query Performance

Standard PostgreSQL monitoring tools apply to pgvector workloads with a few additions.

```
-- PostgreSQL / pgvector
-- Check whether queries are using the vector index or falling back to seqscan
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, content
FROM documents
ORDER BY embedding <=> $1::vector
LIMIT 10;

-- What to look for in the plan:
-- "Index Scan using documents_embedding_hnsw_idx" - index is being used
-- "Seq Scan" - index is NOT being used (check enable_indexscan, table size, or index existence)

-- pgvector disables index scan when the planner estimates it is cheaper to seqscan
-- Force index usage for benchmarking:
SET enable_seqscan = OFF;
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, content FROM documents
ORDER BY embedding <=> $1::vector
LIMIT 10;

-- Track slow vector queries via pg_stat_statements
-- Requires pg_stat_statements in shared_preload_libraries
SELECT
    LEFT(query, 100) AS query_snippet,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_ms
FROM pg_stat_statements
```

```
WHERE query ILIKE '%<=>%'
      OR query ILIKE '%<->%'
      OR query ILIKE '%<#>%'
ORDER BY mean_exec_time DESC
LIMIT 20;

-- Monitor HNSW index bloat from deletes
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
WHERE indexname LIKE '%hnsw%' OR indexname LIKE '%ivfflat%';

-- Table-level stats for vacuum health
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
    last_autovacuum,
    last_autoanalyze
FROM pg_stat_user_tables
WHERE relname LIKE '%document%';
```

The planner and the `enable_indexscan` flag: pgvector’s approximate index scan is treated as an index scan by the planner. The planner may choose a sequential scan when the `LIMIT` is high (because fetching many rows from an index scan has higher overhead than a single seqscan), when statistics are stale, or when the query has a `WHERE` clause that filters more rows than the planner expects. Running `ANALYZE` documents after bulk inserts keeps statistics current and prevents the planner from making poor choices.

Setting `vector.hnsw_ef_search` at the database level is useful as a baseline:

```
-- PostgreSQL / pgvector
-- Set a default ef_search for all connections without application-level control
ALTER DATABASE mydb SET hnsw.ef_search = 100;
-- Individual sessions can still override with SET hnsw.ef_search = N
```

Security and Multi-Tenancy Patterns

When multiple tenants share a single pgvector database, ensuring that similarity searches cannot leak vectors or content across tenant boundaries requires standard PostgreSQL row-level security — the same mechanisms that govern any shared-schema multi-tenancy.

```
-- PostgreSQL / pgvector
-- Multi-tenant embedding table with RLS
CREATE TABLE tenant_documents (
    id          BIGSERIAL PRIMARY KEY,
    tenant_id   UUID NOT NULL,
    content     TEXT NOT NULL,
    embedding   vector(1536),
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX ON tenant_documents
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

```
-- Row-level security
ALTER TABLE tenant_documents ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON tenant_documents
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Application sets the tenant context per connection/transaction
-- With PgBouncer transaction mode, SET LOCAL is required:
BEGIN;
SET LOCAL app.tenant_id = '550e8400-e29b-41d4-a716-446655440000';
SELECT id, content, 1 - (embedding <=> $1::vector) AS similarity
FROM tenant_documents
ORDER BY embedding <=> $1::vector
LIMIT 10;
COMMIT;
-- RLS automatically filters to the current tenant's rows
-- The HNSW index is shared but the RLS predicate is applied after graph traversal
```

One nuance specific to vector workloads: RLS predicates are applied *after* the HNSW graph traversal returns its approximate neighbor set. This means the index may traverse graph nodes belonging to other tenants during search, even though those rows are filtered from the final result. For the vast majority of applications this is acceptable — the content of other tenants’ vectors is not exposed, only the topology of the HNSW graph is traversed. For workloads where even graph topology leakage is a concern (highly sensitive multi-tenant environments), the alternative is separate tables per tenant or separate schemas, each with their own HNSW index.

Upgrading pgvector

pgvector minor-version upgrades (e.g., 0.6.x → 0.7.0) are handled via `ALTER EXTENSION`:

```
-- PostgreSQL / pgvector
-- After installing the new pgvector binaries on the OS:
ALTER EXTENSION vector UPDATE TO '0.7.0';

-- Check the result
SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';
```

HNSW indexes built with earlier versions do not need to be rebuilt for minor-version upgrades unless the release notes explicitly state otherwise. IVFFlat indexes are more sensitive to version differences — check the pgvector changelog before assuming cross-version compatibility.

For major PostgreSQL version upgrades (e.g., PostgreSQL 15 → 16), `pg_upgrade` works with pgvector provided the pgvector binaries for the target PostgreSQL version are installed on the new cluster before running `pg_upgrade`. The indexes are preserved.

Key Takeaways

- **pgvector installs as a standard PostgreSQL extension.** No changes to `shared_preload_libraries` are needed; `CREATE EXTENSION vector` is the only required step after placing the binaries.
- **Choose the distance operator based on the embedding model’s training, not preference.** Cosine (`<=>`) is correct for most sentence transformer and OpenAI embedding models;

verify in the model's documentation before choosing L2 or inner product.

- **HNSW is the better default index for most production workloads** because it supports incremental inserts, delivers higher recall at comparable query speeds, and does not require pre-populated data for index construction. IVFFlat remains appropriate when build-time memory is the binding constraint.
 - **Session-level parameters (`hnsw.ef_search`, `ivfflat.probes`) must be re-applied per transaction** when using PgBouncer in transaction pooling mode; use `SET LOCAL` inside explicit transaction blocks to ensure consistent search quality without parameter bleed-across connections.
 - **Hybrid search via Reciprocal Rank Fusion** combines vector similarity and full-text search without score normalization, handling the case where exact keyword matches and semantic similarity each contribute different signals to retrieval quality.
-

Chapter 8: Creating and Storing Embeddings in PostgreSQL

Embeddings are the bridge between unstructured text and the mathematical operations that make semantic search possible. If you’ve spent years designing schemas for structured data, the shift to storing 1,536-dimensional floating-point vectors alongside your rows can feel disorienting—but the underlying database engineering is familiar territory. This chapter walks through the complete workflow: generating embeddings from text using the OpenAI API and open-source alternatives, storing them efficiently in PostgreSQL using the pgvector extension, and structuring your schema for the query patterns you’ll actually run in production. By the end, you’ll have a working embedding pipeline that integrates with existing PostgreSQL infrastructure and handles the edge cases that break naive implementations.

8.1 How Embeddings Fit Into a PostgreSQL Schema

Before writing any code, it helps to understand what you’re actually storing. An embedding is a fixed-length array of 32-bit floats. The OpenAI `text-embedding-3-small` model produces 1,536 floats per input string. A single embedding consumes $1,536 \times 4$ bytes = 6,144 bytes, or roughly 6 KB. At 1 million rows, that’s approximately 6 GB of raw vector data before indexes.

From a PostgreSQL internals perspective, pgvector introduces a new column type called `vector(n)` where `n` is the number of dimensions. This type stores the float array in a binary format with a header, and it participates in standard PostgreSQL mechanics: TOAST for oversized tuples, MVCC, WAL, logical replication, and point-in-time recovery all work as expected. The extension adds three distance operators that map directly to similarity functions:

Operator	Function	Use Case
<code><-></code>	L2 (Euclidean) distance	When absolute magnitude matters
<code><#></code>	Negative inner product	Max inner product search, dot product similarity
<code><=></code>	Cosine distance	Normalized text embeddings (most common)

For text embeddings from OpenAI, Cohere, or sentence-transformers, cosine distance (`<=>`) is almost

always correct. These models output normalized or near-normalized vectors, and cosine distance measures the angle between them regardless of magnitude. L2 distance works better for embeddings where magnitude carries meaning, such as certain vision or audio models.

A minimal schema that stores document embeddings alongside their source text looks like this:

```
-- PostgreSQL / pgvector

-- Enable the extension (requires superuser or pg_extension_owner membership)
CREATE EXTENSION IF NOT EXISTS vector;

-- Core documents table
CREATE TABLE documents (
    id          BIGSERIAL PRIMARY KEY,
    source_url  TEXT,
    title       TEXT NOT NULL,
    body        TEXT NOT NULL,
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Chunks table - documents are split before embedding
-- Typical chunk size: 256-512 tokens for retrieval tasks
CREATE TABLE document_chunks (
    id          BIGSERIAL PRIMARY KEY,
    document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,           -- position within parent document
    chunk_text   TEXT NOT NULL,         -- the actual text that was embedded
    token_count  INT,                  -- tokens used for this chunk
    embedding    vector(1536),          -- OpenAI text-embedding-3-small dimensions
    model_name    TEXT NOT NULL DEFAULT 'text-embedding-3-small',
    embedded_at  TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE (document_id, chunk_index)
);

-- Standard B-tree indexes for filtered queries
CREATE INDEX ON document_chunks (document_id);
CREATE INDEX ON document_chunks (model_name);
CREATE INDEX ON document_chunks (embedded_at);

-- Partial index: only index rows where embedding is populated
-- This avoids wasting index space on unprocessed chunks
CREATE INDEX ON document_chunks (id) WHERE embedding IS NULL;
```

The separation between `documents` and `document_chunks` is not optional for production systems. Embedding models have token limits—OpenAI’s embedding models cap at 8,191 tokens. More importantly, shorter chunks produce more precise retrieval results. A chunk of 256–512 tokens typically outperforms a full 2,000-token document on recall benchmarks because the embedding captures a specific concept rather than a diffuse mixture of topics. The `chunk_index` column preserves ordering so you can reconstruct context windows around matching chunks during retrieval.

8.2 Generating Embeddings with the OpenAI API

The OpenAI embeddings endpoint is the most common starting point. The API is stateless and synchronous: you send text, you receive a vector. The key operational concerns are batching, error handling, token limits, and cost.

OpenAI’s `text-embedding-3-small` costs \$0.02 per million tokens as of mid-2024, compared to \$0.13 per million tokens for `text-embedding-3-large`. For most retrieval applications,

text-embedding-3-small at 1,536 dimensions outperforms the older text-embedding-ada-002 at the same cost point, making it the default choice for new projects.

The following Python script handles the full embedding generation workflow: chunking a document, calling the API in batches, and inserting results into PostgreSQL via psycopg2. Batching up to 2,048 inputs per API call is supported by OpenAI; staying at 100–500 per batch is a practical compromise between throughput and memory pressure.

```
# Python
import os
import time
import psycopg2
import psycopg2.extras
from openai import OpenAI
from typing import Generator

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

DB_DSN = os.environ.get(
    "DATABASE_URL",
    "postgresql://postgres:password@localhost:5432/vectordb"
)

EMBEDDING_MODEL = "text-embedding-3-small"
EMBEDDING_DIMENSIONS = 1536
BATCH_SIZE = 100          # inputs per API call
MAX_TOKENS_PER_CHUNK = 500 # approximate; 1 token 4 characters

def chunk_text(text: str, max_chars: int = 2000) -> Generator[str, None, None]:
    """
    Simple paragraph-aware chunker.
    Production systems should use tiktoken for precise token counting.
    """
    paragraphs = text.split("\n\n")
    current_chunk = []
    current_len = 0

    for para in paragraphs:
        para = para.strip()
        if not para:
            continue
        if current_len + len(para) > max_chars and current_chunk:
            yield "\n\n".join(current_chunk)
            current_chunk = [para]
            current_len = len(para)
        else:
            current_chunk.append(para)
            current_len += len(para)

    if current_chunk:
        yield "\n\n".join(current_chunk)

def get_embeddings(texts: list[str], retries: int = 3) -> list[list[float]]:
    """
    Call OpenAI embeddings API with retry logic for rate limits.
    Returns a list of embedding vectors in the same order as input texts.
    """
    for attempt in range(retries):
        try:
            response = client.embeddings.create(
                model=EMBEDDING_MODEL,
                input=texts,
            )
            # The API guarantees same-order response as input
            return [item.embedding for item in response.data]
        except Exception as e:
```

```

    if "rate_limit" in str(e).lower() and attempt < retries - 1:
        wait = 2 ** attempt # exponential backoff: 1s, 2s, 4s
        print(f"Rate limited. Waiting {wait}s before retry {attempt + 1}")
        time.sleep(wait)
    else:
        raise

def embed_document(document_id: int, title: str, body: str) -> int:
    """
    Chunk and embed a document, storing results in document_chunks.
    Returns the number of chunks created.
    """
    chunks = list(chunk_text(body))
    if not chunks:
        return 0

    conn = psycopg2.connect(DB_DSN)
    inserted = 0

    try:
        # Process in batches to respect API limits
        for batch_start in range(0, len(chunks), BATCH_SIZE):
            batch_texts = chunks[batch_start : batch_start + BATCH_SIZE]
            embeddings = get_embeddings(batch_texts)

            rows = []
            for i, (text, embedding) in enumerate(zip(batch_texts, embeddings)):
                chunk_index = batch_start + i
                token_estimate = len(text) // 4 # rough estimate
                rows.append({
                    "document_id": document_id,
                    "chunk_index": chunk_index,
                    "chunk_text": text,
                    "token_count": token_estimate,
                    "embedding": embedding,
                    "model_name": EMBEDDING_MODEL,
                })

            with conn.cursor() as cur:
                psycopg2.extras.execute_values(
                    cur,
                    """
                    INSERT INTO document_chunks
                        (document_id, chunk_index, chunk_text, token_count, embedding, model_name)
                    VALUES %s
                    ON CONFLICT (document_id, chunk_index) DO UPDATE
                        SET chunk_text = EXCLUDED.chunk_text,
                            token_count = EXCLUDED.token_count,
                            embedding = EXCLUDED.embedding,
                            model_name = EXCLUDED.model_name,
                            embedded_at = NOW()
                    """,
                    [
                        (
                            r["document_id"],
                            r["chunk_index"],
                            r["chunk_text"],
                            r["token_count"],
                            r["embedding"],
                            r["model_name"],
                        )
                        for r in rows
                    ],
                    template="(%s, %s, %s, %s, %s::vector, %s)",
                    page_size=BATCH_SIZE,
                )
            conn.commit()
            inserted += len(rows)
    
```

```
finally:
    conn.close()

return inserted

if __name__ == "__main__":
    # Example: insert a document and embed it
    conn = psycopg2.connect(DB_DSN)
    with conn.cursor() as cur:
        cur.execute(
            "INSERT INTO documents (title, body) VALUES (%s, %s) RETURNING id",
            ("PostgreSQL Architecture", "PostgreSQL uses a process-per-connection model...\n\n"
            "The shared buffer pool is the primary caching mechanism...\n\n"
            "Write-ahead logging ensures durability across crashes..."),
        )
        doc_id = cur.fetchone()[0]
    conn.commit()
    conn.close()

    count = embed_document(doc_id, "PostgreSQL Architecture", "sample body text")
    print(f"Created {count} embedded chunks for document {doc_id}")
```

Three implementation details here are worth explaining. First, the `ON CONFLICT ... DO UPDATE` clause handles re-embedding gracefully. When you upgrade models or fix chunking logic, you re-run the pipeline without managing deletes. Second, the explicit `::vector` cast in the `execute_values` template is required—`psycopg2` doesn't know how to serialize Python lists as the `pgvector` type natively. Third, token estimation using character count divided by 4 is intentionally rough; production systems should use the `tiktoken` library for precise counts.

8.3 Using Open-Source Embedding Models

The OpenAI API introduces a network call, a cost, and a dependency on an external service. For on-premises deployments, regulated industries, or high-throughput scenarios where per-token costs accumulate quickly, running an embedding model locally is the correct tradeoff.

The `sentence-transformers` library provides access to hundreds of pretrained models that run on CPU or GPU. The `all-MiniLM-L6-v2` model produces 384-dimensional embeddings and runs at roughly 14,000 sentences per second on an 8-core CPU. The `BAAI/bge-large-en-v1.5` model at 1,024 dimensions scores near OpenAI's `ada-002` on the MTEB benchmark while running entirely locally. For most RAG applications, one of these two models is the right starting point.

```
# Python
import os
import numpy as np
import psycopg2
import psycopg2.extras
from sentence_transformers import SentenceTransformer

# Model selection:
# all-MiniLM-L6-v2 → 384 dims, fast, good for general retrieval
# BAAI/bge-large-en-v1.5 → 1024 dims, higher quality, 2x slower
# intfloat/e5-large-v2 → 1024 dims, strong on asymmetric retrieval tasks
MODEL_NAME = "BAAI/bge-large-en-v1.5"
EMBEDDING_DIMENSIONS = 1024

# Load once at module level - model loading takes 2-5 seconds
model = SentenceTransformer(MODEL_NAME)

DB_DSN = os.environ.get(
```

```

    "DATABASE_URL",
    "postgres://postgres:password@localhost:5432/vectordb"
)

def embed_texts_local(texts: list[str], batch_size: int = 64) -> np.ndarray:
    """
    Generate embeddings using a local sentence-transformers model.
    Returns a numpy array of shape (len(texts), EMBEDDING_DIMENSIONS).

    BGE models expect a query prefix for retrieval tasks:
    - Documents: embed as-is
    - Queries: prefix with "Represent this sentence: "
    """
    embeddings = model.encode(
        texts,
        batch_size=batch_size,
        normalize_embeddings=True, # ensures cosine similarity = dot product
        show_progress_bar=len(texts) > 100,
        convert_to_numpy=True,
    )
    return embeddings

def embed_query(query_text: str) -> list[float]:
    """
    Embed a user query. BGE models use a different prefix for queries vs. documents.
    This asymmetric encoding improves retrieval quality on BGE models.
    """
    prefixed = f"Represent this sentence for searching relevant passages: {query_text}"
    embedding = model.encode([prefixed], normalize_embeddings=True)
    return embedding[0].tolist()

def bulk_embed_pending_chunks(batch_size: int = 500) -> int:
    """
    Find chunks without embeddings and fill them using the local model.
    Designed to run as a background job or one-time migration.
    """
    conn = psycopg2.connect(DB_DSN)
    total_processed = 0

    try:
        while True:
            with conn.cursor() as cur:
                # Use the partial index we created on (id) WHERE embedding IS NULL
                cur.execute(
                    """
                    SELECT id, chunk_text
                    FROM document_chunks
                    WHERE embedding IS NULL
                    ORDER BY id
                    LIMIT %s
                    """
                    ,
                    (batch_size,),
                )
                rows = cur.fetchall()

            if not rows:
                break

            ids = [r[0] for r in rows]
            texts = [r[1] for r in rows]

            embeddings = embed_texts_local(texts)

            update_data = [
                (embedding.tolist(), model_id)
                for embedding, model_id in zip(embeddings, ids)
            ]

```



```
with conn.cursor() as cur:
    psycopg2.extras.execute_values(
        cur,
        """
        UPDATE document_chunks AS dc
        SET embedding = data.embedding::vector,
            model_name = %s,
            embedded_at = NOW()
        FROM (VALUES %s) AS data(embedding, id)
        WHERE dc.id = data.id::bigint
        """,
        update_data,
        template="(%s, %s)",
    )

    conn.commit()
    total_processed += len(rows)
    print(f"Embedded {total_processed} chunks so far...")

finally:
    conn.close()

return total_processed


if __name__ == "__main__":
    # Smoke test: embed a single query and print its first 5 dimensions
    vec = embed_query("How does PostgreSQL handle concurrent writes?")
    print(f"Query embedding: {len(vec)} dimensions, first 5: {vec[:5]}")

    # Run the backfill job
    count = bulk_embed_pending_chunks()
    print(f"Total chunks embedded: {count}")
```

One tradeoff to understand: **sentence-transformers** models are session-local. On a multi-process application server, each worker loads the model separately, consuming 500 MB–2 GB of RAM per worker depending on the model. The standard mitigation is to move embedding generation out of the web application entirely—into a dedicated embedding service or a background queue that workers dispatch to asynchronously.

The dimension mismatch between models is a schema problem that must be handled explicitly. If you embed documents with **all-MiniLM-L6-v2** (384 dimensions) and later migrate to **BAAI/bge-large-en-v1.5** (1,024 dimensions), the column type **vector(384)** will reject new inserts. The **model_name** column in the schema above makes this tractable: you can run both models simultaneously during a migration window by using separate columns or a second table, verify retrieval quality, then cut over.

8.4 Building the Vector Index

Storing vectors without an index is viable only up to approximately 100,000 rows. Beyond that threshold, every similarity query becomes a full sequential scan—PostgreSQL reads every row, computes the distance, and sorts. At 1 million rows and 1,536 dimensions, a sequential scan typically takes 3–8 seconds depending on hardware. With an HNSW index, the same query returns in 5–50 milliseconds.

pgvector supports two index types: **IVFFlat** (inverted file with flat quantization) and **HNSW** (Hierarchical Navigable Small World graphs). The performance characteristics follow directly from

their algorithms:

- **IVFFlat** partitions the vector space into **lists** (Voronoi cells) during index build. At query time, it searches only the nearest **probes** cells. Build is fast, memory footprint is low, but recall is lower than HNSW at equivalent speed. Requires data to be pre-loaded before building (`CREATE INDEX AFTER` populating, not before).
- **HNSW** builds a multi-layer proximity graph. Query traversal is faster and recall is higher, but the index takes longer to build and consumes more memory (approximately 6–10 bytes per vector per edge, plus the vectors themselves). New rows are inserted directly into the graph, making it better for append-heavy workloads.

For most production deployments, HNSW is the right default. The following SQL creates and tunes it:

```
-- PostgreSQL / pgvector

-- HNSW index using cosine distance
-- m: number of connections per layer (default 16, range 4-64)
--   Higher m → better recall, larger index, slower build
-- ef_construction: size of the dynamic candidate list during build (default 64)
--   Higher ef_construction → better recall, much slower build
--   Recommended range: 64-200 for production

CREATE INDEX CONCURRENTLY idx_chunks_embedding_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- For filtered queries (e.g., "find similar chunks within document X"),
-- you CANNOT rely on the HNSW index alone - PostgreSQL may choose a seq scan
-- when the WHERE clause is selective. Partial indexes help:

-- Index only chunks from a specific model version
CREATE INDEX CONCURRENTLY idx_chunks_embedding_v2
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
WHERE model_name = 'text-embedding-3-small';

-- Control query-time recall vs. speed tradeoff
-- ef_search=40 is the default; higher values improve recall at cost of latency
-- Set this per-session for queries that require higher accuracy
SET hnsw.ef_search = 100;

-- Verify the index is being used
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  dc.id,
  dc.chunk_text,
  dc.embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM document_chunks dc
WHERE dc.model_name = 'text-embedding-3-small'
ORDER BY distance
LIMIT 10;

-- Check index size and usage statistics
SELECT
  indexname,
  pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size,
  idx_scan,
  idx_tup_read,
  idx_tup_fetch
FROM pg_stat_user_indexes
WHERE tablename = 'document_chunks';
```

One operational gotcha: `CREATE INDEX CONCURRENTLY` on an HNSW index with 1 million rows and

`ef_construction = 64` takes 20–90 minutes depending on hardware and dimensionality. Plan for this in maintenance windows. The index build is CPU-bound; parallelism is controlled by `max_parallel_maintenance_workers`. Setting this to 4–8 for the index build session reduces wall-clock time significantly:

```
-- PostgreSQL / pgvector

-- Increase parallel workers for this session before running the index build
SET max_parallel_maintenance_workers = 4;

-- For IVFFlat, the rule of thumb for lists is sqrt(row_count)
-- At 1M rows: lists = 1000
-- probes at query time: typically lists / 10 = 100 (higher → better recall, slower)
CREATE INDEX CONCURRENTLY idx_chunks_embedding_ivfflat
ON document_chunks
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 1000);

-- Set probes at query time
SET ivfflat.probes = 100;
```

The recall-latency tradeoff is a parameter you tune against your data, not a fixed property of the extension. For RAG applications where top-10 recall at 95% is the target, HNSW with `m=16`, `ef_construction=64`, and `ef_search=80` is a reasonable starting configuration. Run the MTEB retrieval benchmarks against a sample of your own data to validate.

8.5 Querying Embeddings in Production

The query pattern for semantic search in a RAG pipeline has three stages: embed the user query, find nearest neighbors, join to retrieve context. Each stage has production considerations that aren't visible in tutorial examples.

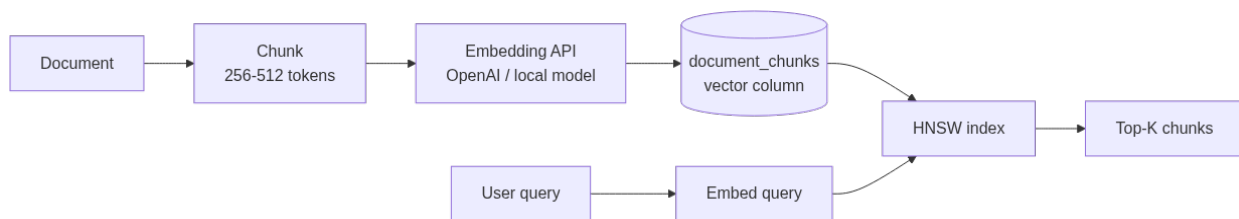


Figure 6: The write path (chunk, embed, store, index) and the read path (embed query, search index) share the same embedding model — that’s what makes the vectors comparable.

The query below represents a full production pattern: embedding generation, nearest neighbor retrieval with metadata filters, and result hydration in a single round-trip using a parameterized query:

```
# Python
import os
import psycopg2
import psycopg2.extras
from openai import OpenAI

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
DB_DSN = os.environ.get("DATABASE_URL", "postgres://postgres:password@localhost:5432/vectordb")
```

```

EMBEDDING_MODEL = "text-embedding-3-small"
RETRIEVAL_LIMIT = 10      # retrieve top-10, re-rank to top-5 before sending to LLM
SIMILARITY_THRESHOLD = 0.3 # cosine distance; 0 = identical, 2 = opposite

def embed_query_openai(query: str) -> list[float]:
    response = client.embeddings.create(model=EMBEDDING_MODEL, input=[query])
    return response.data[0].embedding

def retrieve_chunks(
    query: str,
    document_ids: list[int] | None = None,
    limit: int = RETRIEVAL_LIMIT,
) -> list[dict]:
    """
    Retrieve the most semantically similar chunks for a query, optionally filtered by document IDs.
    Returns results ordered by cosine similarity (ascending distance = descending similarity).
    """
    query_embedding = embed_query_openai(query)

    conn = psycopg2.connect(DB_DSN)
    try:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
            # Build the filter clause dynamically
            filter_clause = "WHERE dc.embedding IS NOT NULL"
            params: list = [query_embedding, SIMILARITY_THRESHOLD]

            if document_ids:
                filter_clause += " AND dc.document_id = ANY(%s)"
                params.append(document_ids)

            params.append(limit)

            cur.execute(
                f"""
                SELECT
                    dc.id                AS chunk_id,
                    dc.document_id,
                    dc.chunk_index,
                    dc.chunk_text,
                    dc.token_count,
                    d.title              AS document_title,
                    d.source_url,
                    dc.embedding <=> %s::vector AS cosine_distance,
                    1 - (dc.embedding <=> %s::vector) AS cosine_similarity
                FROM document_chunks dc
                JOIN documents d ON d.id = dc.document_id
                {filter_clause}
                ORDER BY dc.embedding <=> %s::vector
                LIMIT %s
                """,
                [query_embedding, query_embedding, *params[2:]],
            )
            results = cur.fetchall()
    finally:
        conn.close()

    # Post-filter: discard results below similarity threshold
    return [
        dict(r) for r in results
        if r["cosine_distance"] <= SIMILARITY_THRESHOLD * 2
    ]

```

There is a subtle parameter duplication in that query: the embedding vector appears twice in the SELECT list (once for the `cosine_distance` alias, once for `cosine_similarity`) and again in the ORDER BY. PostgreSQL does not evaluate the expression three times against the index; the planner recognizes the repeated expression and the ORDER BY drives index access. However,

passing the embedding as a bind parameter three times is unavoidable in parameterized queries—you cannot reference a positional parameter by name in standard PostgreSQL parameter syntax. A cleaner approach uses a CTE to bind the embedding once:

```
# Python
def retrieve_chunks_cte(
    query: str,
    document_ids: list[int] | None = None,
    limit: int = RETRIEVAL_LIMIT,
) -> list[dict]:
    """
    Same retrieval logic, but binds the query embedding exactly once via a CTE.
    Cleaner parameter management for complex queries.
    """
    query_embedding = embed_query_openai(query)

    conn = psycopg2.connect(DB_DSN)
    try:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
            filter_clause = "WHERE dc.embedding IS NOT NULL"
            params: list = [query_embedding]

            if document_ids:
                filter_clause += " AND dc.document_id = ANY(%s)"
                params.append(document_ids)

            params.append(limit)

            cur.execute(
                f"""
                WITH query_vec AS (
                    SELECT %s::vector AS vec
                )
                SELECT
                    dc.id                               AS chunk_id,
                    dc.document_id,
                    dc.chunk_index,
                    dc.chunk_text,
                    dc.token_count,
                    d.title                               AS document_title,
                    d.source_url,
                    dc.embedding <=> qv.vec              AS cosine_distance,
                    1 - (dc.embedding <=> qv.vec)         AS cosine_similarity
                FROM document_chunks dc
                JOIN documents d ON d.id = dc.document_id
                CROSS JOIN query_vec qv
                {filter_clause}
                ORDER BY dc.embedding <=> qv.vec
                LIMIT %s
                """
                ,
                params,
            )
            results = cur.fetchall()
    finally:
        conn.close()

    return [dict(r) for r in results]
```

The CTE version passes the embedding once. The `CROSS JOIN query_vec qv` adds zero overhead—the CTE materializes a single row, and PostgreSQL folds it into the scan. This pattern becomes especially valuable when the vector participates in multiple distance expressions within the same query, such as hybrid scoring that weights cosine distance against a BM25-style text rank.

Filtered Vector Search and the Pre-Filter Problem

The most common performance trap in production pgvector deployments is combining WHERE clause filters with ORDER BY distance. Consider this query:

```
-- PostgreSQL / pgvector

-- Naive: filter by document_id, then find nearest neighbors
-- The HNSW index searches the full graph first, then filters - this is POST-filtering
-- If document_id = 42 contains only 0.5% of rows, recall collapses

SELECT dc.chunk_text, dc.embedding <=> '[...]'::vector AS dist
FROM document_chunks dc
WHERE dc.document_id = 42
ORDER BY dist
LIMIT 10;
```

When PostgreSQL evaluates this query, the planner has two options. If the HNSW index is used, it performs approximate nearest neighbor search across the entire table and then applies the `document_id = 42` filter to the candidates returned from the index. If the filtered set is a small fraction of the total table, the index may return fewer than 10 usable results even with a large candidate pool. This is called the **post-filter problem**.

The alternative is a sequential scan: PostgreSQL ignores the HNSW index, scans only the rows matching `document_id = 42` (using the B-tree index on that column), computes exact distances, and sorts. For highly selective filters—when the filtered set contains thousands to tens of thousands of rows—the sequential scan often produces better results with lower latency than the HNSW post-filter approach.

The practical strategy is to force this tradeoff explicitly by understanding your data distribution:

```
-- PostgreSQL / pgvector

-- Check how many chunks exist per document to understand filter selectivity
SELECT
    document_id,
    COUNT(*) AS chunk_count,
    COUNT(*) * 100.0 / SUM(COUNT(*)) OVER () AS pct_of_total
FROM document_chunks
GROUP BY document_id
ORDER BY chunk_count DESC
LIMIT 20;

-- For small filtered sets (< ~10,000 rows), disable HNSW and use exact search
-- This forces the planner to scan the filtered rows exactly
SET enable_indexscan = off; -- disables index scans including HNSW
SELECT dc.chunk_text, dc.embedding <=> '[...]'::vector AS dist
FROM document_chunks dc
WHERE dc.document_id = 42
ORDER BY dist
LIMIT 10;
RESET enable_indexscan;

-- Better long-term solution: partition the table by a high-cardinality filter column
-- or use a partial HNSW index per logical partition (e.g., per tenant in SaaS)

-- Partial index for a specific document (useful if one document is queried very frequently)
CREATE INDEX CONCURRENTLY idx_chunks_doc42_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
WHERE document_id = 42;
```

For multi-tenant SaaS applications where queries are always scoped to a single tenant, building one

partial HNSW index per tenant is the cleanest solution when tenant data sizes are large enough to warrant it. For smaller tenants, exact search over their filtered rows is faster than any approximate index.

Hybrid Search: Combining Vector Similarity with Full-Text Search

Pure vector search misses exact keyword matches. A query for “PostgreSQL 16 logical replication slot” may semantically match a document about replication in general while missing a document that uses those exact terms in a different context. Hybrid search combines cosine similarity with PostgreSQL’s built-in full-text search to capture both signals.

```
-- PostgreSQL / pgvector

-- Add a full-text search column to the chunks table
ALTER TABLE document_chunks
ADD COLUMN ts_chunk TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', chunk_text)) STORED;

CREATE INDEX ON document_chunks USING GIN (ts_chunk);

-- Hybrid search: combine BM25 rank with cosine similarity
-- Reciprocal Rank Fusion (RRF) is a parameter-free fusion method
-- rrf_score = 1 / (k + rank), with k = 60 being the standard value

WITH query_vec AS (
    SELECT '[...]'::vector AS vec -- substitute actual query embedding
),
query_text AS (
    SELECT to_tsquery('english', 'postgresql & replication & slot') AS tsq
),
vector_ranked AS (
    SELECT
        dc.id,
        dc.chunk_text,
        dc.document_id,
        ROW_NUMBER() OVER (ORDER BY dc.embedding <=> qv.vec) AS vector_rank
    FROM document_chunks dc
    CROSS JOIN query_vec qv
    WHERE dc.embedding IS NOT NULL
    ORDER BY dc.embedding <=> qv.vec
    LIMIT 60 -- retrieve top-60 from each signal, fuse to top-10
),
text_ranked AS (
    SELECT
        dc.id,
        ts_rank_cd(dc.ts_chunk, qt.tsq) AS text_score,
        ROW_NUMBER() OVER (ORDER BY ts_rank_cd(dc.ts_chunk, qt.tsq) DESC) AS text_rank
    FROM document_chunks dc
    CROSS JOIN query_text qt
    WHERE dc.ts_chunk @@ qt.tsq
    ORDER BY text_score DESC
    LIMIT 60
),
fused AS (
    SELECT
        COALESCE(vr.id, tr.id) AS chunk_id,
        COALESCE(1.0 / (60 + vr.vector_rank), 0)
        + COALESCE(1.0 / (60 + tr.text_rank), 0) AS rrf_score
    FROM vector_ranked vr
    FULL OUTER JOIN text_ranked tr ON tr.id = vr.id
)
SELECT
    f.chunk_id,
    dc.chunk_text,
    d.title AS document_title,
    f.rrf_score
```

```
FROM fused f
JOIN document_chunks dc ON dc.id = f.chunk_id
JOIN documents d ON d.id = dc.document_id
ORDER BY f.rrf_score DESC
LIMIT 10;
```

Reciprocal Rank Fusion is favored in production because it requires no tuning: the constant $k = 60$ dampens the impact of extreme ranks and is robust to score scale differences between the two signals. Alternatives like linear combination ($\alpha * \text{cosine_score} + (1 - \alpha) * \text{bm25_score}$) require careful per-dataset calibration.

8.6 Schema Evolution and Model Migration

Embedding models improve over time. When OpenAI released `text-embedding-3-small` as a replacement for `text-embedding-ada-002`, the new model outperformed its predecessor while costing less. Any production system that embeds documents must have a migration path for re-embedding with a new model.

The naive approach—dropping the `embedding` column and re-running—has zero downtime resilience. Applications querying during the migration return no results or errors. The correct approach uses a multi-column shadow migration:

```
-- PostgreSQL / pgvector

-- Step 1: Add a new column for the new model without touching the old one
ALTER TABLE document_chunks
ADD COLUMN embedding_v2 vector(1536); -- adjust dimensions if model differs

-- Step 2: Add a migration-tracking index
CREATE INDEX ON document_chunks (id) WHERE embedding_v2 IS NULL;

-- Step 3: Run the backfill in the background (Python job iterates this query)
-- Application continues using embedding column during this window

-- Step 4: After backfill completes, verify coverage
SELECT
  COUNT(*) FILTER (WHERE embedding IS NOT NULL) AS old_embedded,
  COUNT(*) FILTER (WHERE embedding_v2 IS NOT NULL) AS new_embedded,
  COUNT(*) AS total
FROM document_chunks;

-- Step 5: Switch application queries to use embedding_v2
-- This is a code deploy, not a schema change

-- Step 6: Build the production HNSW index on embedding_v2
CREATE INDEX CONCURRENTLY idx_chunks_embedding_v2_hnsw
ON document_chunks
USING hnsw (embedding_v2 vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Step 7: After the index is live and application is using embedding_v2,
-- drop the old column and its index
DROP INDEX CONCURRENTLY idx_chunks_embedding_hnsw;
ALTER TABLE document_chunks DROP COLUMN embedding;

-- Step 8: Rename for clarity
ALTER TABLE document_chunks RENAME COLUMN embedding_v2 TO embedding;
ALTER INDEX idx_chunks_embedding_v2_hnsw RENAME TO idx_chunks_embedding_hnsw;
```

This approach is explicit and reversible. At any point during the backfill, the old index is live and queries work. The application code switch at step 5 is instantaneous and rollback-safe—you simply

revert the code deploy.

For systems with strict schema change policies, a separate migration table avoids column proliferation when migrating through multiple model versions:

```
-- PostgreSQL / pgvector

-- Separate table to hold embeddings by model version
-- Allows multiple embedding versions to coexist without schema changes
CREATE TABLE chunk_embeddings (
    chunk_id    BIGINT NOT NULL REFERENCES document_chunks(id) ON DELETE CASCADE,
    model_name  TEXT NOT NULL,
    dimensions  INT NOT NULL,
    embedding   vector(1536), -- must match maximum expected dimensions
    embedded_at TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (chunk_id, model_name)
);

CREATE INDEX CONCURRENTLY idx_chunk_embeddings_hnsw
ON chunk_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
WHERE model_name = 'text-embedding-3-small';
```

The separate-table design has a cost: every similarity query requires a JOIN. For high-QPS applications, the join overhead is measurable. The tradeoff is flexibility during migration versus query performance during steady state. Most teams start with the single-column approach and add the separation only when managing three or more model versions simultaneously.

8.7 Operational Monitoring and Maintenance

Running vectors in production adds two monitoring concerns that don't exist in purely relational schemas: index health and embedding pipeline throughput.

Index Bloat and VACUUM

HNSW indexes in pgvector do not support in-place deletion of dead vectors from the graph. When rows are updated or deleted, the corresponding nodes in the HNSW graph become logically deleted but remain in the structure until a VACUUM operation cleans them. High update rates on the `embedding` column—typical during initial backfills or model migrations—can create significant bloat. Monitor with:

```
-- PostgreSQL

-- Check for dead tuples and bloat in the chunks table
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    ROUND(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
    last_vacuum,
    last_autovacuum,
    last_analyze
FROM pg_stat_user_tables
WHERE relname = 'document_chunks';

-- If dead_pct is high after a migration, run a manual VACUUM ANALYZE
VACUUM ANALYZE document_chunks;

-- For autovacuum tuning on embedding-heavy tables:
```

```

-- Lower the threshold so autovacuum triggers more frequently
ALTER TABLE document_chunks SET (
    autovacuum_vacuum_scale_factor = 0.01, -- trigger at 1% dead tuples
    autovacuum_analyze_scale_factor = 0.01
);

```

Embedding Pipeline Monitoring

Track these metrics in your application observability stack:

```

# Python - instrumentation pattern for embedding pipelines

import time
import logging
from dataclasses import dataclass, field
from datetime import datetime

logger = logging.getLogger(__name__)

@dataclass
class EmbeddingPipelineMetrics:
    chunks_processed: int = 0
    chunks_failed: int = 0
    api_calls: int = 0
    total_tokens: int = 0
    total_latency_ms: float = 0.0
    start_time: datetime = field(default_factory=datetime.utcnow)

    def record_batch(
        self,
        batch_size: int,
        token_count: int,
        latency_ms: float,
        success: bool = True,
    ) -> None:
        self.api_calls += 1
        self.total_latency_ms += latency_ms
        if success:
            self.chunks_processed += batch_size
            self.total_tokens += token_count
        else:
            self.chunks_failed += batch_size

    def report(self) -> dict:
        elapsed = (datetime.utcnow() - self.start_time).total_seconds()
        return {
            "chunks_processed": self.chunks_processed,
            "chunks_failed": self.chunks_failed,
            "api_calls": self.api_calls,
            "total_tokens": self.total_tokens,
            "avg_latency_ms": self.total_latency_ms / max(self.api_calls, 1),
            "throughput_chunks_per_sec": self.chunks_processed / max(elapsed, 1),
            "elapsed_seconds": elapsed,
        }

def get_embedding_pipeline_lag(conn) -> dict:
    """
    Returns metrics on how many chunks are waiting to be embedded.
    Run this as a scheduled job and emit to your metrics system.
    """
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT
                COUNT(*) FILTER (WHERE embedding IS NULL)      AS pending,
                COUNT(*) FILTER (WHERE embedding IS NOT NULL) AS embedded,
                COUNT(*)                                         AS total,
            """

```

```
        MIN(created_at) FILTER (WHERE embedding IS NULL) AS oldest_pending
    FROM document_chunks
    """
)
row = cur.fetchone()
return {
    "pending_chunks": row[0],
    "embedded_chunks": row[1],
    "total_chunks": row[2],
    "oldest_pending_at": row[3].isoformat() if row[3] else None,
}
```

Tracking `oldest_pending_at` is more operationally useful than raw count. A backlog of 50,000 chunks is acceptable if they arrived in the last hour; the same number is alarming if the oldest chunk is 12 hours old, indicating a pipeline stall.

Query Latency Baselines

Establish query latency baselines before going to production and re-measure after each schema change or index rebuild:

```
-- PostgreSQL

-- Baseline: latency for a nearest-neighbor search at various table sizes
-- Run EXPLAIN ANALYZE multiple times and take median

-- Warm the cache first
SELECT COUNT(*) FROM document_chunks;

-- Measure query with HNSW at default ef_search
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, chunk_text, embedding <=> '[...]'::vector AS dist
FROM document_chunks
ORDER BY dist
LIMIT 10;

-- Verify index usage: look for "Index Scan using idx_chunks_embedding_hnsw"
-- in the EXPLAIN output. If you see "Seq Scan", the planner chose sequential scan
-- because the table is small or statistics are stale - run ANALYZE first.
```

Key Takeaways

- **Embeddings are fixed-size float arrays stored in `vector(n)` columns.** For OpenAI `text-embedding-3-small`, that is 1,536 dimensions consuming 6 KB per row. Separate the documents table from a chunks table to handle token limits and improve retrieval precision.
- **Batch your API calls and handle rate limits with exponential backoff.** The OpenAI embeddings endpoint supports up to 2,048 inputs per request; 100–500 per batch is a practical production target. Use `ON CONFLICT DO UPDATE` to make the pipeline idempotent.
- **Choose the HNSW index for append-heavy workloads and IVFFlat when you need lower memory overhead with acceptable recall.** Tune `ef_search` at query time to balance recall against latency without rebuilding the index.
- **Filtered vector search requires explicit strategy.** The post-filter problem means HNSW recall collapses for highly selective `WHERE` clauses. Use partial indexes for frequently filtered subsets, or allow the planner to fall back to exact sequential scan for small filtered sets.
- **Model migration follows a shadow-column pattern.** Add a new embedding column, backfill in the background, switch application code to use the new column, build the new

index, then drop the old column. This keeps queries functional at every step and makes the cutover instantaneous and rollback-safe.

Chapter 9: HNSW vs IVFFlat

Indexing: Performance Tradeoffs

Vector search at scale lives or dies by its index structure. When you move beyond brute-force exact search — the vector equivalent of a full table scan — you enter the world of Approximate Nearest Neighbor (ANN) algorithms, and two of them dominate production deployments: HNSW (Hierarchical Navigable Small World) and IVFFlat (Inverted File Index with Flat quantization). Both solve the same fundamental problem — find the k most similar vectors to a query without scanning every row — but they make dramatically different engineering tradeoffs around memory, build time, query latency, and recall accuracy. This chapter unpacks how each algorithm works, where the numbers actually land in practice, and how to make the right choice for your workload. Database engineers will find familiar analogies throughout: if B-tree vs. hash index decisions feel natural to you, this chapter will feel like home.

How Vector Indexes Work: The ANN Problem

Exact nearest neighbor search requires computing the distance from a query vector to every stored vector. At 1 million vectors with 1536 dimensions (OpenAI’s `text-embedding-ada-002` output size), that’s 1.5 billion floating-point operations per query. Even with SIMD parallelism, you’re looking at hundreds of milliseconds — unacceptable for interactive applications.

ANN algorithms trade a small amount of recall accuracy for dramatic speed improvements. The core insight is identical to why you don’t read every page of a book to find a topic — you use an index that organizes content spatially, letting you skip most of the data.

Recall is the ANN equivalent of query correctness. A recall of 0.95 means that for any query, your ANN search returns 95% of the same results that a brute-force exact search would return. Production systems typically target 0.90–0.99 recall depending on application sensitivity. A recommendation engine might tolerate 0.90. A medical record retrieval system needs 0.99+.

Think of it this way: recall in vector search maps roughly to what DBAs know as index selectivity, except you control the tradeoff dynamically through search parameters rather than waiting for a statistics update.

IVFFlat: The Inverted Index Approach

IVFFlat follows a partitioning strategy that any DBA who has worked with table partitioning will recognize immediately.

How it works:

During index build, IVFFlat runs k-means clustering on your vector dataset, dividing all vectors into **lists** number of clusters. Each cluster has a centroid — a representative vector at its geometric center. At query time, the algorithm:

1. Computes distance from the query vector to all centroids
2. Selects the **probes** closest centroids (you control this number)
3. Scans every vector in those selected clusters for exact distance comparisons
4. Returns the top-k results

This is structurally identical to a partitioned table with a partition pruning step. The centroids are your partition keys, and **probes** is how many partitions you're willing to scan.

Build-time parameters:

- **lists**: Number of clusters. Rule of thumb: $\sqrt{n}$ for datasets up to 1 million rows; $n/1000$ for larger datasets. More lists = faster queries but slower build and higher minimum **probes** needed for good recall.
- Build time scales with dataset size roughly linearly after the k-means phase.

Query-time parameters:

- **probes**: Number of clusters to scan. Higher values increase recall but increase query latency proportionally. Default is 1 (fast, low recall). Setting **probes** = **lists** degrades to exact search.

```
-- PostgreSQL / pgvector
-- Create a table and populate it
CREATE TABLE documents (
    id          BIGSERIAL PRIMARY KEY,
    content     TEXT,
    embedding   vector(1536)
);

-- Build an IVFFlat index
-- lists = sqrt(1,000,000) 1000 for a 1M row dataset
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 1000);

-- At query time, set probes in the session or transaction
-- probes = 10 gives ~0.90 recall; probes = 50 gives ~0.98 recall
SET ivfflat.probes = 10;

SELECT id, content, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM documents
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;

-- For higher-stakes queries, bump probes in the same session
SET ivfflat.probes = 50;

SELECT id, content, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM documents
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;
```

IVFFlat performance characteristics (illustrative, not measured):

The directional pattern is consistent across IVFFlat deployments: raising **probes** relative to **lists** trades query throughput for recall, and both build time and query cost scale with dataset size. The rough shape looks like this —

Dataset Size	lists	probes (low → high)	Recall (low → high)	Relative QPS (low → high)
100K vectors	100	low	lower	higher
100K vectors	100	high	higher	lower
1M vectors	1000	low	lower	higher
1M vectors	1000	high	higher	lower

Do not treat these as target numbers. Actual recall, QPS, and build time depend heavily on your embedding model, data distribution, CPU, and PostgreSQL configuration — measure them on your own dataset using the benchmarking scripts later in this chapter before setting production parameters.

Critical limitation: IVFFlat requires the index to be built on a fully populated (or near-final) dataset. The k-means centroids are computed once at build time. If your data distribution shifts significantly — because you’ve added a new category of documents that clusters differently — you’ll need to rebuild the index to restore recall. This is not unlike rebuilding a hash-partitioned table after a distribution skew event.

HNSW: The Graph Navigation Approach

HNSW works on a fundamentally different principle. Rather than partitioning the vector space, it builds a multi-layer graph where nodes are your vectors and edges connect nearby vectors. The “hierarchical” part means the graph has multiple layers: upper layers are sparse long-range connections, lower layers are dense short-range connections. This mirrors skip list data structures — upper lanes for coarse navigation, lower lanes for precise landing.

How it works:

During index build, each vector is inserted into the graph. HNSW assigns each vector a random maximum layer (following an exponential distribution so most vectors appear only in layer 0). Edges are drawn greedily to the M nearest neighbors at each layer.

At query time: 1. Start at the entry point in the top layer 2. Greedily traverse edges toward the query vector, descending layers 3. At layer 0 (the base layer with all vectors), run a beam search with width `ef_search` 4. Return the top-k candidates

Build-time parameters:

- **m:** Number of bidirectional edges per node (typically 8–64). Higher values increase recall and memory usage. Memory per vector scales as $O(m)$.
- **ef_construction:** Size of the dynamic candidate list during build. Higher values improve index quality but slow build time. Typically 64–512.

Query-time parameters:

- **ef_search** (called **hnsw.ef_search** in pgvector): Beam width during search. Higher values increase recall but increase latency. Never set lower than **k** (your **LIMIT** value).

```
-- PostgreSQL / pgvector
-- HNSW index with tuned parameters
-- m=16 is the default; ef_construction=64 is the default
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Set ef_search at query time
-- ef_search = 40 gives ~0.95 recall
-- ef_search = 100 gives ~0.99 recall
SET hnsw.ef_search = 64;

SELECT id, content, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM documents
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;

-- Check index size
SELECT pg_size_pretty(pg_relation_size('documents_embedding_idx'));

-- Inspect index build progress (pgvector 0.6+)
SELECT phase, blocks_done, blocks_total,
       tuples_done, tuples_total
FROM pg_stat_progress_create_index
WHERE relid = 'documents'::regclass;
```

HNSW performance characteristics (illustrative, not measured):

The same directional pattern applies to HNSW: raising **ef_search** improves recall at the cost of query throughput, and larger **m/ef_construction** improve recall further but push build time up substantially. The rough shape —

Dataset Size	m	ef_construction	ef_search (low → high)	Recall (low → high)	Relative QPS (low → high)
100K vectors	16	64	low	lower	higher
100K vectors	16	64	high	higher	lower
1M vectors	16	64	moderate	high	moderate
1M vectors	32	128	high	highest	lowest

Treat these as directional only — do not use them as capacity-planning numbers. The build time gap between the two algorithms is real and gets worse at scale: HNSW build at the million-vector range commonly takes an order of magnitude longer than IVFFlat at a comparable recall target, but the exact multiplier depends on your hardware and parameters and should be measured on your own dataset using the benchmarking scripts later in this chapter.

Memory footprint: HNSW stores the graph structure in memory. At **m=16** with 1M float32 vectors at 1536 dimensions: - Raw vector data: ~6 GB - HNSW graph overhead: ~500 MB–1 GB - Total working set: ~7 GB

This is the primary reason HNSW can saturate RAM on large datasets before you hit query performance limits.

Head-to-Head Comparison: When to Choose Which

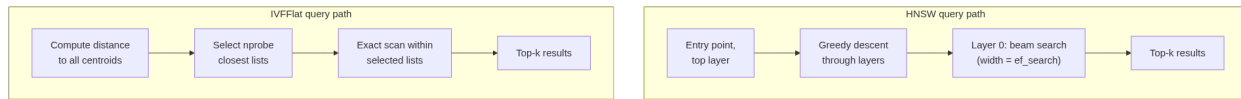


Figure 7: HNSW narrows in by descending graph layers; IVFFlat narrows in by selecting clusters, then scans exactly within them.

The choice between HNSW and IVFFlat comes down to five operational axes. Every production decision should be evaluated against all five simultaneously.

1. Query Latency

HNSW wins at low concurrency. Its graph traversal is CPU cache-friendly and predictable, and at a comparable recall target it typically shows lower and more consistent p99 latency than IVFFlat on the same dataset, because IVFFlat performs brute-force scans within each probed cluster, which has worse cache behavior at high dimensionality. The actual gap between the two varies with hardware, dimensionality, and parameter choice — measure it on your own data rather than assuming a fixed ratio.

At high concurrency (50+ simultaneous queries), the memory bandwidth requirements of HNSW’s graph traversal become a bottleneck, and the gap narrows.

2. Index Build Time

IVFFlat wins decisively. For a 1M vector dataset, IVFFlat builds in minutes; HNSW takes hours. For 10M+ vectors, HNSW build time can extend into days without parallelism. If your pipeline requires frequent full index rebuilds — for example, after bulk data loads or schema migrations — IVFFlat is the pragmatic choice.

3. Incremental Insert Behavior

HNSW wins. Inserting new vectors into an HNSW index is a graph operation: find neighbors, add edges, done. No rebalancing required. Query recall degrades gracefully as vectors are inserted.

IVFFlat cannot efficiently accommodate new vectors that fall outside the distribution of the original k-means clusters. New vectors get assigned to their nearest centroid, but if the new data represents a genuinely new cluster (say, you started indexing documents in a new domain), recall drops because the probed clusters won’t contain the relevant neighbors. You’re rebuilding.

```

# Python
# Demonstrating incremental insert behavior difference with pgvector via psycopg2
import psycopg2
import numpy as np
from pgvector.psycopg2 import register_vector

conn = psycopg2.connect("dbname=vectordb user=postgres")
register_vector(conn)
cur = conn.cursor()

def insert_batch(embeddings: list[np.ndarray], contents: list[str]):
    """
    HNSW: insert works seamlessly; graph expands with each new node.
    IVFFlat: insert works but may degrade recall if new vectors
            fall outside existing cluster centroids.
    """
    for embedding, content in zip(embeddings, contents):
  
```

```

        cur.execute(
            "INSERT INTO documents (content, embedding) VALUES (%s, %s)",
            (content, embedding)
        )
    conn.commit()
    print(f"Inserted {len(embeddings)} vectors.")

def measure_recall(query_vector: np.ndarray, k: int = 10) -> float:
    """
    Compare ANN results against exact search to estimate recall.
    """
    # Exact search (sequential scan, bypasses index)
    cur.execute("SET enable_indexscan = off")
    cur.execute(
        "SELECT id FROM documents ORDER BY embedding <=> %s LIMIT %s",
        (query_vector, k)
    )
    exact_ids = set(row[0] for row in cur.fetchall())

    # ANN search (uses index)
    cur.execute("SET enable_indexscan = on")
    cur.execute("SET hnsw.ef_search = 64")
    cur.execute(
        "SELECT id FROM documents ORDER BY embedding <=> %s LIMIT %s",
        (query_vector, k)
    )
    ann_ids = set(row[0] for row in cur.fetchall())

    recall = len(exact_ids & ann_ids) / k
    return recall

# Simulate incremental inserts and recall measurement
rng = np.random.default_rng(42)
batch_size = 10_000
num_batches = 5

for batch_num in range(num_batches):
    # Simulate new embeddings arriving (e.g., from OpenAI API)
    new_embeddings = [rng.standard_normal(1536).astype(np.float32)
                      for _ in range(batch_size)]
    new_contents = [f"doc_batch{batch_num}_{i}" for i in range(batch_size)]

    insert_batch(new_embeddings, new_contents)

    # Sample query to measure recall after each batch
    query = rng.standard_normal(1536).astype(np.float32)
    recall = measure_recall(query)

    cur.execute("SELECT COUNT(*) FROM documents")
    total = cur.fetchone()[0]
    print(f"After batch {batch_num + 1}: {total:,} total vectors, recall = {recall:.3f}")

cur.close()
conn.close()

```

4. Memory Usage

IVFFlat wins on memory efficiency. The inverted lists are compact structures; only the centroid vectors and the flat vector lists need to stay in memory for good performance. HNSW's graph layer — the bidirectional edge lists — consumes memory proportional to $m * n$, where n is the number of vectors. At $m=16$ and 10M vectors, the graph alone occupies ~2.5 GB before the vector data itself.

For memory-constrained deployments (small cloud instances, embedded applications), IVFFlat often fits where HNSW does not.

5. Recall Consistency

HNSW produces more consistent recall across diverse query distributions. IVFFlat recall is uneven: queries whose nearest neighbors span multiple clusters get worse recall than queries whose neighborhood is tightly contained within one cluster. This is the partitioning equivalent of a query that crosses partition boundaries — you’ll miss results unless you increase **probes**.

HNSW’s graph traversal is more distribution-agnostic. It navigates toward the answer regardless of how the vector space is partitioned, which is why it tends to be the default recommendation for general-purpose vector search.

Managed Vector Databases: How They Expose These Tradeoffs

The same fundamental tradeoffs appear in every managed vector database, even when the interface abstracts away the implementation details.

```
# Python
# Weaviate: HNSW configuration (Weaviate default index type)
import weaviate
from weaviate.classes.config import Configure, VectorDistances

client = weaviate.connect_to_local()

# Create a collection with explicit HNSW tuning
client.collections.create(
    name="Documents",
    vectorizer_config=Configure.Vectorizer.none(), # bring-your-own vectors
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        ef_construction=128, # higher = better index quality, slower build
        max_connections=32, # equivalent to m in HNSW
        ef=64, # ef_search equivalent; can be overridden per query
        dynamic_ef_min=100, # auto-tune ef_search based on load
        dynamic_ef_max=500,
        dynamic_ef_factor=8,
    )
)

# Create a collection with flat (exact) index for small datasets
client.collections.create(
    name="SmallDocuments",
    vectorizer_config=Configure.Vectorizer.none(),
    vector_index_config=Configure.VectorIndex.flat(
        distance_metric=VectorDistances.COSINE,
        # Flat index = exact search; appropriate for < 10K vectors
        # No recall tradeoff, but latency scales linearly with dataset size
    )
)

# Query with per-request ef override
collection = client.collections.get("Documents")
results = collection.query.near_vector(
    near_vector=[0.1, 0.2, ...], # 1536-dim query vector
    limit=10,
    return_metadata=["distance"],
)

for obj in results.objects:
    print(f"ID: {obj.uuid}, Distance: {obj.metadata.distance:.4f}")

client.close()
```

```

# Python
# Qdrant: comparing HNSW configuration options
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, HnswConfigDiff,
    OptimizersConfigDiff, SearchParams
)

client = QdrantClient(host="localhost", port=6333)

# Create collection with tuned HNSW
client.recreate_collection(
    collection_name="production_docs",
    vectors_config=VectorParams(
        size=1536,
        distance=Distance.COSINE,
    ),
    hnsw_config=HnswConfigDiff(
        m=16, # edges per node; default 16
        ef_construct=100, # build-time beam width; default 100
        full_scan_threshold=10_000, # fall back to exact search below this count
        # This is a key Qdrant feature: automatic exact search for small collections
    ),
    optimizers_config=OptimizersConfigDiff(
        indexing_threshold=20_000, # don't build HNSW until 20K vectors loaded
        # Prevents expensive partial index builds during bulk ingestion
    )
)

# Query with per-request ef override - critical for latency/recall tuning
search_results = client.search(
    collection_name="production_docs",
    query_vector=[0.1, 0.2, ...], # your query embedding
    limit=10,
    search_params=SearchParams(
        hnsw_ef=128, # override collection default for this query
        exact=False # True forces exact search regardless of index
    )
)

for result in search_results:
    print(f"ID: {result.id}, Score: {result.score:.4f}")

# Qdrant's quantization support (not available in pgvector IVFFlat):
# Scalar quantization reduces memory 4x at ~1% recall cost
from qdrant_client.models import ScalarQuantization, ScalarQuantizationConfig, ScalarType

client.recreate_collection(
    collection_name="memory_optimized_docs",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
    quantization_config=ScalarQuantization(
        scalar=ScalarQuantizationConfig(
            type=ScalarType.INT8,
            quantile=0.99, # clip outliers beyond 99th percentile
            always_ram=True, # keep quantized vectors in RAM, original on disk
        )
    )
)

```

Pinecone abstracts index type selection entirely — you choose a serverless or pod-based deployment, and the platform manages the underlying algorithm. Pod-based deployments expose `metric` (cosine, euclidean, dotproduct) and pod type/size but not raw HNSW parameters. The tradeoff is operational simplicity at the cost of fine-grained tuning. For teams without a dedicated ML infrastructure engineer, this is often the right call.

Milvus exposes both HNSW and IVFFlat as first-class options along with more exotic indexes like DISKANN (for data that exceeds RAM), IVF_SQ8 (scalar quantization of IVFFlat), and

HNSW_SQ. For very large datasets (>100M vectors) that cannot fit in RAM, DISKANN becomes the relevant comparison point — but that’s beyond the scope of this chapter.

Production Tuning Playbook

Understanding the algorithms is necessary but not sufficient. Here is how experienced practitioners actually set parameters in production.

Step 1: Establish your recall floor

Before touching index parameters, decide what recall is acceptable for your application. Run exact search on a representative query sample and benchmark baseline latency. Then set a target: “We need 0.95 recall at under 20ms p99 at 100 QPS.”

Step 2: Size your index

For IVFFlat: `lists = sqrt(row_count)` is a reliable starting point up to 1M rows. For HNSW: start with `m=16`, `ef_construction=64` — these are conservative defaults that work well across most workloads.

Step 3: Tune probes or ef_search against recall

Run the recall measurement loop from the earlier Python example against your actual query distribution. Plot recall vs. latency as you sweep the parameter. The curve will be steep initially (large recall gains per unit of latency) and then flatten (diminishing returns). Pick the knee of the curve.

```
# Python
# Recall-latency sweep for IVFFlat probes tuning
import psycpg2
import numpy as np
import time
from pgvector.psycpg2 import register_vector

conn = psycpg2.connect("dbname=vectordb user=postgres")
register_vector(conn)
cur = conn.cursor()

def benchmark_ivfflat(query_vectors, k=10, probes_range=None):
    if probes_range is None:
        probes_range = [1, 2, 5, 10, 20, 50, 100]

    results = []

    # Precompute exact results for recall calculation
    cur.execute("SET enable_indexscan = off")
    exact_results = {}
    for i, qv in enumerate(query_vectors):
        cur.execute(
            "SELECT id FROM documents ORDER BY embedding <=> %s LIMIT %s",
            (qv, k)
        )
        exact_results[i] = set(r[0] for r in cur.fetchall())

    cur.execute("SET enable_indexscan = on")

    for probes in probes_range:
        cur.execute(f"SET ivfflat.probes = {probes}")

        latencies = []
        recalls = []
```

```

for i, qv in enumerate(query_vectors):
    t0 = time.perf_counter()
    cur.execute(
        "SELECT id FROM documents ORDER BY embedding <=> %s LIMIT %s",
        (qv, k)
    )
    ann_ids = set(r[0] for r in cur.fetchall())
    latencies.append((time.perf_counter() - t0) * 1000) # ms
    recalls.append(len(exact_results[i] & ann_ids) / k)

results.append({
    "probes": probes,
    "mean_recall": np.mean(recalls),
    "p99_latency_ms": np.percentile(latencies, 99),
    "mean_lat_ms": np.mean(latencies),
})
print(f"probes={probes:4d} | recall={np.mean(recalls):.3f} | "
      f"p99={np.percentile(latencies, 99):.1f}ms | "
      f"mean={np.mean(latencies):.1f}ms")

return results

def benchmark_hnsw(query_vectors, k=10, ef_range=None):
    if ef_range is None:
        ef_range = [10, 20, 40, 64, 100, 200, 400]

    results = []

    cur.execute("SET enable_indexscan = off")
    exact_results = {}
    for i, qv in enumerate(query_vectors):
        cur.execute(
            "SELECT id FROM documents ORDER BY embedding <=> %s LIMIT %s",
            (qv, k)
        )
        exact_results[i] = set(r[0] for r in cur.fetchall())

    cur.execute("SET enable_indexscan = on")

    for ef in ef_range:
        cur.execute(f"SET hnsw.ef_search = {ef}")

        latencies = []
        recalls = []

        for i, qv in enumerate(query_vectors):
            t0 = time.perf_counter()
            cur.execute(
                "SELECT id FROM documents ORDER BY embedding <=> %s LIMIT %s",
                (qv, k)
            )
            ann_ids = set(r[0] for r in cur.fetchall())
            latencies.append((time.perf_counter() - t0) * 1000)
            recalls.append(len(exact_results[i] & ann_ids) / k)

        results.append({
            "ef_search": ef,
            "mean_recall": np.mean(recalls),
            "p99_latency_ms": np.percentile(latencies, 99),
            "mean_lat_ms": np.mean(latencies),
        })
        print(f"ef_search={ef:4d} | recall={np.mean(recalls):.3f} | "
              f"p99={np.percentile(latencies, 99):.1f}ms | "
              f"mean={np.mean(latencies):.1f}ms")

    return results

# Generate representative query sample - use real queries from your workload when possible
rng = np.random.default_rng(0)

```

```

query_sample = [rng.standard_normal(1536).astype(np.float32) for _ in range(200)]

print("=== IVFFlat Sweep ===")
ivf_results = benchmark_ivfflat(query_sample)

print("\n=== HNSW Sweep ===")
hnsw_results = benchmark_hnsw(query_sample)

cur.close()
conn.close()

```

Step 4: Validate on your actual query distribution

Synthetic random vectors have a different nearest-neighbor structure than real embeddings from text, images, or tabular data. Real embeddings cluster around semantic topics. This means recall numbers measured on random vectors are often pessimistic for IVFFlat (your real queries are more likely to stay within a cluster) and can be either optimistic or pessimistic for HNSW depending on cluster density. Always validate with a sample of production queries before finalizing parameters.

Step 5: Monitor recall in production

Recall is not static. It drifts as your data grows, as your embedding model changes, and as query patterns evolve. The cheapest ongoing measurement strategy is shadow exact search: periodically route a small fraction of queries (e.g., 0.1%) through a sequential scan and compare results against the index-served response. When measured recall drops below your floor, rebuild the index.

```

-- PostgreSQL
-- Simple recall monitoring using a dedicated test query table
-- Store known exact results, then re-run through the index periodically

CREATE TABLE recall_probes (
    probe_id    BIGSERIAL PRIMARY KEY,
    query_vec   vector(1536),
    exact_ids   BIGINT[],          -- precomputed exact top-10 ids
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- Stored procedure to measure current recall on stored probes
CREATE OR REPLACE FUNCTION measure_index_recall(k INT DEFAULT 10)
RETURNS TABLE(probe_id BIGINT, recall FLOAT)
LANGUAGE plpgsql AS $$
DECLARE
    rec RECORD;
    ann_ids BIGINT[];
    overlap INT;
BEGIN
    FOR rec IN SELECT * FROM recall_probes LOOP
        -- ANN search using the current index
        SELECT ARRAY(
            SELECT id FROM documents
            ORDER BY embedding <=> rec.query_vec
            LIMIT k
        ) INTO ann_ids;

        -- Compute intersection size
        SELECT COUNT(*) INTO overlap
        FROM UNNEST(rec.exact_ids) e
        JOIN UNNEST(ann_ids) a ON e = a;

        probe_id := rec.probe_id;
        recall   := overlap::FLOAT / k;
        RETURN NEXT;
    END LOOP;
END;
$$;

```

```
-- Run recall check
SELECT
  AVG(recall)      AS mean_recall,
  MIN(recall)      AS min_recall,
  PERCENTILE_CONT(0.05) WITHIN GROUP (ORDER BY recall) AS p5_recall
FROM measure_index_recall(10);
```

Filtering and the ANN Problem

A dimension of performance that benchmark tables frequently omit: what happens when you combine vector search with scalar filters?

Consider a query like “find the 10 most similar documents to this query vector, but only among documents owned by `tenant_id = 42`.” This is pre-filtering in database terms — restricting the candidate set before ranking.

IVFFlat with pre-filtering: pgvector applies the WHERE clause first via a sequential scan or regular index, then computes vector distances only for qualifying rows. This is accurate — exact search on the filtered subset — but slow when the filter is selective enough to defeat the vector index.

HNSW with pre-filtering: pgvector uses a similar approach. For very selective filters (tiny candidate sets), both indexes abandon ANN and fall back to exact search, which is actually the right behavior.

The failure mode is a moderately selective filter — say, WHERE `tenant_id = 42` returns 50,000 of your 1,000,000 rows. Now you want ANN search on those 50,000, but your index was built on all 1,000,000. The planner must choose between:

- Sequential scan of all 50,000 qualifying rows (exact but potentially slow)
- ANN search with probes/ef_search high enough to cover the filtered subset

```
-- PostgreSQL / pgvector
-- Pre-filtered vector search
-- Add a btree index on the filter column for efficient candidate selection
CREATE INDEX ON documents (tenant_id);

-- For HNSW: pgvector will use the HNSW index and apply the filter post-scan
-- If the filter is very selective, the planner may choose a sequential scan
SET hnsw.ef_search = 200; -- raise ef_search to compensate for filtering

SELECT id, content, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;

-- Examine the plan
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, content, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;
```

Managed vector databases handle this differently. Weaviate and Qdrant implement **post-filtering** (ANN search then apply filter, may return fewer than k results) and **pre-filtering** (filter candidates then ANN search within the subset, accurate but requires a different index structure). Qdrant’s

approach uses a filterable HNSW where payload indexes interact with the graph traversal — you get accurate filtered results without a full sequential scan.

For multi-tenant workloads with strict per-tenant isolation, the practical solution in pgvector is per-tenant tables or partitioning, so each tenant’s vectors get their own index. This is standard database partitioning logic applied to the vector dimension.

Choosing Between Indexes: A Decision Framework

Rather than a simple rule, apply this checklist:

Choose HNSW when: - Query latency at low-to-moderate concurrency is the primary constraint
- Your dataset grows continuously via inserts (not bulk rebuilds) - You need consistent recall across diverse query types - Memory is available (budget roughly 1.5–2x raw vector size for the full working set) - Build time is acceptable (plan for hours at multi-million-row scale)

Choose IVFFlat when: - Build time is constrained (nightly rebuilds, CI pipelines, frequent schema changes) - Memory is limited and you cannot afford the HNSW graph overhead - Your dataset is relatively static (infrequent inserts, bulk-loaded) - You need to tune recall precisely per query by varying probes at runtime - Dataset size is under 500K vectors, where the build time gap shrinks substantially

Choose exact search (no ANN index) when: - Dataset is under roughly 50K vectors — brute force at this size is fast enough for interactive applications - Recall must be 1.0 (e.g., compliance or audit contexts where missing a result is unacceptable) - Index maintenance cost is not justified for the query volume

```
-- PostgreSQL / pgvector
-- Decision aid: estimate whether your dataset needs an ANN index at all
-- If this query runs under your latency budget, you may not need an index

EXPLAIN (ANALYZE, BUFFERS)
SELECT id, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM documents
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;

-- If "Execution Time" is acceptable, defer index creation.
-- Sequential scan avoids all recall tradeoffs and has zero build cost.
-- Re-evaluate at 50K, 200K, and 500K rows.
```

SQL Server Context: What the Ecosystem Looks Like

SQL Server does not ship with native vector index support as of this writing. Microsoft’s vector search path runs through:

1. **Azure Cognitive Search** — managed service that handles vector indexing separately from SQL Server, using HNSW internally
2. **Azure SQL / SQL Server with JSON + CLR** — storing vectors as JSON or binary, computing distances via T-SQL or CLR extensions (brute force, no ANN)

3. SQL Server 2022 + external integrations — offloading vector search to a sidecar service (Qdrant, Weaviate, or Pinecone) and joining results back via application code

The implication for SQL Server shops is architectural: vector search currently lives outside the database engine, which means you’re managing data consistency between SQL Server (your source of truth) and a separate vector store. The dual-write pattern discussed in earlier chapters applies directly here.

```
-- SQL Server
-- Brute-force vector similarity using T-SQL (no ANN index available)
-- Vectors stored as comma-separated strings, decoded in application layer
-- This approach does not scale beyond ~100K rows at interactive latency

-- Alternatively, store as VARBINARY and decode in CLR
-- The pattern below uses a precomputed similarity cache for known query vectors

CREATE TABLE vector_similarity_cache (
    query_hash      CHAR(64)      NOT NULL,  -- SHA256 of query vector bytes
    document_id     BIGINT        NOT NULL,
    similarity       FLOAT        NOT NULL,
    computed_at     DATETIME2     NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_sim_cache PRIMARY KEY (query_hash, document_id)
);

CREATE INDEX IX_sim_cache_query_score
    ON vector_similarity_cache (query_hash, similarity DESC)
    INCLUDE (document_id);

-- Application pattern:
-- 1. Hash the incoming query vector
-- 2. Check cache (valid for N minutes)
-- 3. On cache miss: call external vector search service, populate cache
-- 4. JOIN cache results with SQL Server document table for metadata enrichment

-- This pattern allows SQL Server to remain authoritative for business data
-- while delegating vector search to a purpose-built system
```

The SQL Server ecosystem is moving quickly in this area; future versions and Azure SQL updates are likely to close this gap. Teams building on SQL Server today should design their vector search layer as a separable concern so that native support can be adopted when available.

Key Takeaways

- **HNSW and IVFFlat solve the same ANN problem through opposite strategies:** HNSW builds a navigable graph for efficient traversal; IVFFlat partitions the vector space into clusters for scoped brute-force search. Neither is universally superior — workload characteristics determine the right choice.
- **The dominant tradeoff is build time vs. query performance:** HNSW delivers lower latency and better recall consistency but requires hours to build at million-row scale. IVFFlat builds in minutes but degrades recall when data distribution shifts and cannot recover without a rebuild.
- **Both algorithms expose a single runtime dial that controls the recall-latency tradeoff** — probes for IVFFlat, `ef_search` for HNSW. Benchmark this dial against your actual query distribution before finalizing, and monitor recall drift in production over time.

- **Pre-filtering (WHERE clause + vector search) requires extra care:** ANN indexes are built on the full dataset, not filtered subsets. Highly selective filters can defeat the index and force exact search; moderately selective filters can silently reduce recall. Raise `ef_search/probes` or restructure data (per-tenant tables, partitioning) when filtering is a core access pattern.
 - **SQL Server teams must treat vector search as an external service today:** Without native ANN index support in the engine, the pragmatic path is a sidecar vector database with a dual-write consistency pattern, designed so that future native support can be adopted by replacing the sidecar rather than rearchitecting the application.
-

Chapter 10: Tuning pgvector for High-Throughput Workloads

pgvector ships with sensible defaults, but production workloads expose its limits quickly. A single-table similarity search that performs well at 100K vectors starts to degrade noticeably past 1M, and write-heavy embedding ingestion pipelines can turn a well-tuned PostgreSQL instance into a bottleneck that blocks your entire AI application. This chapter covers the mechanics of pgvector’s indexing algorithms, how to map familiar PostgreSQL tuning levers to vector-specific behavior, and the patterns that let you sustain thousands of queries per second without sacrificing recall accuracy.

Understanding How HNSW and IVFFlat Trade Recall for Speed

Before reaching for configuration knobs, you need a clear mental model of what the two index types are actually doing—because the tradeoffs are fundamentally different from B-tree tuning.

IVFFlat (Inverted File with Flat lists) works like a k-means partition. During `CREATE INDEX`, it assigns every vector to one of `lists` centroids. At query time, it probes `probes` lists, computes exact distances within those lists, and returns the top-k results. Think of it as a clustered index where the “key” is a cluster center, and you’re controlling how many clusters you scan. More lists = better recall but slower build; more probes = better recall but slower query.

HNSW (Hierarchical Navigable Small World) builds a multi-layer graph where each vector is a node connected to its nearest neighbors. Queries traverse from a sparse top layer down to a dense bottom layer, pruning the search space at each hop. The key build parameters are `m` (edges per node, default 16) and `ef_construction` (beam width during build, default 64). The query parameter `hnsw.ef_search` controls how many candidates are evaluated during traversal.

The practical difference for a DBA: IVFFlat has a faster, more predictable build time and lower memory footprint, but recall degrades sharply if your data distribution shifts after the index is built. HNSW maintains higher recall across distribution changes, but build time scales superlinearly with `m` and `ef_construction`, and the index itself can consume 2–3× the memory of an equivalent IVFFlat index.

Here’s a benchmark-driven comparison you can run yourself:

```
-- PostgreSQL / pgvector
-- Create a test table with 1M 1536-dim vectors (OpenAI ada-002 shape)
CREATE TABLE embeddings (
  id          BIGSERIAL PRIMARY KEY,
  content     TEXT,
```

```

embedding VECTOR(1536)
);

-- Populate with random data for benchmarking
INSERT INTO embeddings (content, embedding)
SELECT
  'doc_' || g,
  (
    SELECT array_agg(random()::float4 - 0.5)::vector(1536)
    FROM generate_series(1, 1536)
  )
FROM generate_series(1, 1000000) g;

-- Baseline: sequential scan recall is 100% by definition
-- Time a brute-force query first
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM embeddings
ORDER BY distance
LIMIT 10;

-- Build IVFFlat: rule of thumb is sqrt(row_count) to 4*sqrt(row_count) for lists
-- For 1M rows: lists between 1000 and 4000
CREATE INDEX idx_ivfflat ON embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 2000);

-- Build HNSW with production-grade parameters
CREATE INDEX idx_hnsw ON embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 24, ef_construction = 128);

-- Set probes for IVFFlat at query time (session or per-transaction)
SET ivfflat.probes = 40;

-- Set ef_search for HNSW at query time
SET hnsw.ef_search = 100;

-- Measure approximate query with HNSW
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM embeddings
ORDER BY distance
LIMIT 10;

```

A working rule: for IVFFlat, `probes / lists` gives you a rough recall proxy. At `probes = 1`, recall is roughly 60–70% for typical text embeddings. At `probes = 40` out of `lists = 2000` (2%), recall is typically 95–97%. At `probes = 200` (10%), recall approaches 99%. For HNSW, `ef_search = 40` usually delivers ~95% recall; `ef_search = 100` delivers ~98–99%. These numbers vary significantly with embedding model and data distribution—always measure recall on your actual data.

PostgreSQL Server Configuration for Vector Workloads

Vector workloads stress PostgreSQL differently than OLTP. The dominant costs are memory bandwidth, CPU cache pressure from high-dimensional distance calculations, and index build I/O. The standard `postgresql.conf` knobs matter, but the relevant thresholds shift.

Shared Buffers and Effective Cache Size

HNSW index traversal is heavily random-access across the graph structure. If the index fits in `shared_buffers`, traversal costs drop dramatically. For a table with 1M 1536-dim float32 vectors,

the raw vector data is ~6GB. The HNSW index with `m=16` adds roughly 1–1.5GB on top of that. Size your `shared_buffers` to hold the hot subset of both.

```
# shell - postgresql.conf tuning for a vector workload host
# Assume: 64GB RAM, dedicated PostgreSQL instance

# Memory
shared_buffers = 16GB           # ~25% of RAM; cover hot HNSW index + working set
effective_cache_size = 48GB      # planner hint: what's available OS + PG cache
work_mem = 256MB                # each sort/hash node; vector ORDER BY sorts are large
maintenance_work_mem = 4GB      # critical for HNSW build performance

# Parallelism
max_parallel_workers_per_gather = 4
max_parallel_workers = 16
max_worker_processes = 24

# WAL and I/O (for write-heavy ingestion)
wal_buffers = 64MB
checkpoint_completion_target = 0.9
max_wal_size = 4GB
min_wal_size = 1GB

# For NVMe storage, reduce random_page_cost
random_page_cost = 1.1
effective_io_concurrency = 200

# Autovacuum tuning for high-write vector tables
autovacuum_vacuum_cost_delay = 2ms
autovacuum_vacuum_scale_factor = 0.01 # vacuum more aggressively on large tables
autovacuum_analyze_scale_factor = 0.005
```

Parallel Index Builds

pgvector 0.6.0 added parallel HNSW index builds. This is one of the highest-impact settings for tables with millions of vectors—a serial build of a 5M-vector HNSW index at `m=24`, `ef_construction=128` can take 2–4 hours. With 8 parallel workers it typically drops to 25–40 minutes.

```
-- PostgreSQL / pgvector
-- Enable parallel workers for the build session
SET max_parallel_maintenance_workers = 7; -- uses up to 7 workers + 1 leader = 8 total

-- Then build the index
CREATE INDEX idx_hnsw_parallel ON embeddings
USING hnsw (embedding_vector_cosine_ops)
WITH (m = 24, ef_construction = 128);

-- Monitor build progress in a separate session
SELECT
    phase,
    blocks_done,
    blocks_total,
    tuples_done,
    tuples_total,
    round(100.0 * tuples_done / NULLIF(tuples_total, 0), 1) AS pct_complete
FROM pg_stat_progress_create_index
WHERE relid = 'embeddings'::regclass;
```

work_mem and Vector Sorts

When PostgreSQL executes `ORDER BY embedding <=> $1 LIMIT k` without an index (or when the planner falls back to seqscan), it materializes candidates in a sort node. With 1M rows at 1536 dims (float32), that's a 6GB sort. If `work_mem` is too low, it spills to disk and query latency spikes into seconds. Set `work_mem` high enough to keep that sort in memory, or ensure your index is always being used.

You can verify index usage:

```
-- PostgreSQL / pgvector
-- Confirm the planner is using the HNSW index, not a seqscan
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, content, embedding <=> '[0.1, 0.2, ...]':vector AS score
FROM embeddings
ORDER BY score
LIMIT 20;

-- If you see "Seq Scan" instead of "Index Scan using idx_hnsw_parallel",
-- the planner may be choosing seqscan because the table is small enough
-- or because enable_indexscan is off. Force it during testing:
SET enable_seqscan = OFF;

-- Also check: is your query vector the right dimension?
-- Dimension mismatch silently falls back to seqscan in older pgvector versions
SELECT typmod FROM pg_attribute
WHERE attrelid = 'embeddings':regclass
AND attname = 'embedding';
```

Scaling Ingestion: Bulk Loading Without Choking the Index

Most production AI applications have two phases: an initial bulk load of millions of historical embeddings, and an ongoing stream of new embeddings from user activity or document indexing. Both require different strategies.

Bulk Loading Strategy

The worst approach is inserting rows one at a time while the HNSW index is live. Each insert triggers an index entry insertion, which requires graph traversal to find the insertion point— $O(\log n)$ per insert, with high cache miss rates on a large graph. At 1M rows, individual inserts are ~10–15ms each through pgvector’s HNSW; that’s roughly 70–100 inserts/second.

The right approach: load data without the vector index, then build the index in one pass.

```
# Python - bulk loading embeddings using COPY for maximum throughput

import psycopg2
import psycopg2.extras
import numpy as np
from io import StringIO

def bulk_load_embeddings(conn_string: str, records: list[dict]) -> None:
    """
    Load embeddings using COPY for 10-50x throughput vs INSERT.
    records: list of {"id": int, "content": str, "embedding": np.ndarray}
    """
    conn = psycopg2.connect(conn_string)
    conn.autocommit = False
    cur = conn.cursor()

    # Step 1: Drop the vector index if it exists
    cur.execute("""
        DROP INDEX IF EXISTS idx_hnsw_parallel;
    """)

    # Step 2: Disable autovacuum on this table during bulk load
    cur.execute("""
        ALTER TABLE embeddings SET (autovacuum_enabled = false);
    """)

    # Step 3: Use COPY with a StringIO buffer
```



```

# pgvector accepts vectors in '[f1,f2,...,fn]' format
buffer = StringIO()
for rec in records:
    vec_str = '[' + ','.join(f'{v:.6f}' for v in rec['embedding']) + ']'
    # Escape tab and newline in content
    content = rec['content'].replace('\t', ' ').replace('\n', ' ')
    buffer.write(f"{rec['id']}\t{content}\t{vec_str}\n")

buffer.seek(0)
cur.copy_from(buffer, 'embeddings', columns=('id', 'content', 'embedding'))

conn.commit()

# Step 4: Rebuild the index after load
cur.execute("SET maintenance_work_mem = '4GB'")
cur.execute("SET max_parallel_maintenance_workers = 7")
cur.execute("""
    CREATE INDEX idx_hnsw_parallel ON embeddings
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 24, ef_construction = 128);
""")

# Step 5: Re-enable autovacuum and run analyze
cur.execute("""
    ALTER TABLE embeddings SET (autovacuum_enabled = true);
""")
cur.execute("ANALYZE embeddings;")

conn.commit()
cur.close()
conn.close()
print(f"Loaded {len(records)} records and rebuilt index.")

# For ongoing streaming inserts, use batched inserts with execute_values
def stream_insert_embeddings(conn_string: str, batch: list[dict]) -> None:
    """
    For ongoing inserts: batch size of 500-2000 rows hits the
    sweet spot between commit overhead and lock contention.
    """
    conn = psycopg2.connect(conn_string)
    conn.autocommit = False
    cur = conn.cursor()

    records = [
        (rec['id'], rec['content'],
         '[' + ','.join(f'{v:.6f}' for v in rec['embedding']) + ']')
        for rec in batch
    ]

    psycopg2.extras.execute_values(
        cur,
        "INSERT INTO embeddings (id, content, embedding) VALUES %s "
        "ON CONFLICT (id) DO UPDATE SET embedding = EXCLUDED.embedding",
        records,
        template="(%s, %s, %s::vector)",
        page_size=500
    )

    conn.commit()
    cur.close()
    conn.close()

```

Filling Factor and Index Bloat

HNSW indexes grow as you insert rows, but they don't rebalance—unlike B-trees, there's no analog to a page split. However, deletes and updates leave dead tuples that inflate the index. Run `VACUUM ANALYZE` regularly on embedding tables, and monitor `pg_stat_user_indexes` for index size growth

anomalies:

```
-- PostgreSQL / pgvector
-- Monitor index health and bloat indicators
SELECT
    i.relname                AS index_name,
    pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
    pg_size_pretty(pg_relation_size(t.oid)) AS table_size,
    s.idx_scan,
    s.idx_tup_read,
    s.idx_tup_fetch,
    round(100.0 * pg_relation_size(i.oid) /
          NULLIF(pg_relation_size(t.oid), 0), 1) AS index_to_table_pct
FROM pg_index x
JOIN pg_class i ON i.oid = x.indexrelid
JOIN pg_class t ON t.oid = x.indrelid
JOIN pg_stat_user_indexes s ON s.indexrelid = i.oid
WHERE t.relname = 'embeddings'
ORDER BY pg_relation_size(i.oid) DESC;

-- If the HNSW index exceeds ~2x the raw vector data size,
-- consider a REINDEX CONCURRENTLY to reclaim space
REINDEX INDEX CONCURRENTLY idx_hnsw_parallel;
```

Hybrid Search Architecture: Combining Vector and Scalar Filters

Production RAG and semantic search systems rarely do pure vector search. The typical pattern is: filter by tenant, date range, document type, or access control, then rank by similarity. How you structure this filter interaction has a massive impact on both performance and recall.

The Pre-filter vs. Post-filter Problem

PostgreSQL’s query planner applies scalar filters either before or after the vector index scan, depending on statistics and cost estimates. If it applies the filter *after* fetching the top-k from the index (post-filter), and the filter eliminates 90% of results, you get fewer than k results. If it applies the filter *before* (pre-filter via seqscan), you get correct results but scan every row.

pgvector doesn’t natively support pre-filtered HNSW the way specialized vector databases do. Your architectural options:

```
-- PostgreSQL / pgvector

-- Option 1: Partial indexes - build one HNSW index per tenant
-- Effective when tenant count is small and per-tenant volume is large
CREATE INDEX idx_hnsw_tenant_42 ON embeddings
USING hnsw (embedding_vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
WHERE tenant_id = 42;

-- The planner will use this index when the query includes the filter
SELECT id, content, embedding <=> $1 AS score
FROM embeddings
WHERE tenant_id = 42
ORDER BY score
LIMIT 10;

-- Option 2: Partition the table by tenant, build one index per partition
-- Better for large tenant counts with predictable partitioning keys
CREATE TABLE embeddings_partitioned (
    id          BIGSERIAL,
    tenant_id   INT NOT NULL,
    content     TEXT,
```

```

        embedding VECTOR(1536),
        created_at TIMESTAMPTZ DEFAULT now()
    ) PARTITION BY LIST (tenant_id);

CREATE TABLE embeddings_t1 PARTITION OF embeddings_partitioned
    FOR VALUES IN (1);
CREATE TABLE embeddings_t2 PARTITION OF embeddings_partitioned
    FOR VALUES IN (2);

-- Build per-partition HNSW indexes
CREATE INDEX ON embeddings_t1 USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
CREATE INDEX ON embeddings_t2 USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Option 3: Two-pass query - oversample from the index, then filter in app
-- Use this when filters are unpredictable or cardinality is unknown
-- Fetch 10x more candidates than needed, filter in application layer
SELECT id, content, metadata, embedding <=> $1 AS score
FROM embeddings
ORDER BY score
LIMIT 200; -- fetch 200, apply filter in app, return top 20

-- Option 4: Use a composite score with WHERE clause on indexed scalar columns
-- Add a GIN or BRIN index on metadata columns alongside HNSW
CREATE INDEX idx_embeddings_created ON embeddings USING BRIN (created_at);
CREATE INDEX idx_embeddings_tenant ON embeddings (tenant_id);

-- This query lets the planner choose between bitmap index scan on tenant_id
-- and HNSW scan; with good statistics it usually picks correctly
SELECT id, content, embedding <=> $1 AS score
FROM embeddings
WHERE tenant_id = $2
AND created_at > now() - INTERVAL '30 days'
ORDER BY score
LIMIT 10;

```

Quantization and Dimensionality Reduction

For very large tables (10M+ vectors), you can reduce the memory footprint of the vector data itself. pgvector 0.7.0 introduced binary quantization and scalar quantization, which trade a small recall penalty for substantial memory savings:

```

-- PostgreSQL / pgvector 0.7+
-- Halfvec: store as float16 instead of float32 (2x memory reduction)
-- Useful for OpenAI text-embedding-3-small (1536 dims)
CREATE TABLE embeddings_hv (
    id BIGSERIAL PRIMARY KEY,
    content TEXT,
    embedding HALFVEC(1536) -- 3KB per row instead of 6KB
);

CREATE INDEX idx_hnsw_halfvec ON embeddings_hv
USING hnsw (embedding halfvec_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Binary quantization: extreme compression, ~5-10% recall loss for most models
-- Good for pre-filtering candidates before a reranking pass
CREATE TABLE embeddings_bq (
    id BIGSERIAL PRIMARY KEY,
    content TEXT,
    embedding BIT(1536) -- 192 bytes per row instead of 6KB
);

CREATE INDEX idx_hnsw_bit ON embeddings_bq
USING hnsw (embedding bit_hamming_ops)
WITH (m = 16, ef_construction = 64);

-- Two-phase search: binary quantization for candidate retrieval,

```

```
-- exact float32 distance for reranking
WITH candidates AS (
    SELECT id
    FROM embeddings_bq
    ORDER BY embedding <~> $1::bit(1536) -- Hamming distance, fast
    LIMIT 100
)
SELECT e.id, e.content, e.embedding <=> $2::vector AS score
FROM embeddings e
JOIN candidates c ON c.id = e.id
ORDER BY score
LIMIT 10;
```

Connection Pooling and Concurrency Patterns for AI Applications

AI applications have a distinct connection pattern: many short, parallel requests during inference, each needing a fast vector lookup. This matches OLTP patterns superficially but diverges in two ways—queries are CPU-bound (distance calculations) rather than I/O-bound, and session-level GUC settings like `hnsw.ef_search` complicate connection pooling.

PgBouncer Configuration for Vector Workloads

PgBouncer in transaction-mode pooling resets session state between transactions, which breaks session-level `SET hnsw.ef_search`. Use statement-mode only if your queries are single-statement, or use transaction-mode with `SET LOCAL` instead of `SET`:

```
# shell - pgbouncer.ini for a vector search service

[databases]
vectordb = host=127.0.0.1 port=5432 dbname=vectordb

[pgbouncer]
pool_mode = transaction           # transaction-mode for AI services
max_client_conn = 1000           # total clients (LLM app + workers)
default_pool_size = 20           # backend connections per db/user pair
reserve_pool_size = 5            # emergency reserve
reserve_pool_timeout = 3
max_db_connections = 50          # cap on total backend connections

# Keep connections alive - vector index traversal benefits from warm caches
server_idle_timeout = 600
client_idle_timeout = 0

# Authentication
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt

# Logging - monitor slow vector queries
log_pooler_errors = 1
stats_period = 60

# Python - using SET LOCAL to safely set ef_search per transaction
# Works correctly with PgBouncer transaction-mode pooling

import asyncpg
import asyncio

async def vector_search(
    pool: asyncpg.Pool,
    query_vector: list[float],
    tenant_id: int,
    ef_search: int = 100,
    limit: int = 10
```

```

) -> list[dict]:
    """
    Thread-safe vector search that sets ef_search per transaction.
    SET LOCAL is scoped to the transaction and safe with connection pooling.
    """
    async with pool.acquire() as conn:
        async with conn.transaction():
            # SET LOCAL: reverts automatically when transaction ends
            await conn.execute(f"SET LOCAL hnsw.ef_search = {ef_search}")

            rows = await conn.fetch(
                """
                SELECT
                    id,
                    content,
                    embedding <=> $1::vector AS score
                FROM embeddings
                WHERE tenant_id = $2
                ORDER BY score
                LIMIT $3
                """,
                str(query_vector),
                tenant_id,
                limit
            )

            return [dict(r) for r in rows]

async def main():
    pool = await asyncpg.create_pool(
        dsn="postgresql://app_user:password@pgbouncer_host:6432/vectordb",
        min_size=5,
        max_size=20,
        command_timeout=10.0 # fail fast on slow vector queries
    )

    # Concurrent search simulation
    query_vec = [0.1] * 1536 # replace with real embedding
    tasks = [
        vector_search(pool, query_vec, tenant_id=1, ef_search=100)
        for _ in range(50)
    ]
    results = await asyncio.gather(*tasks)
    print(f"Completed {len(results)} parallel searches")
    await pool.close()

asyncio.run(main())

```

Monitoring Query Latency Under Load

Under concurrent load, vector query latency has a different distribution than OLTP queries. The p99 can be 5–10× the median because HNSW traversal depth varies with the query vector’s position in the graph. Monitor percentiles, not averages:

```

-- PostgreSQL / pgvector
-- Requires pg_stat_statements extension
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Find slow vector queries
SELECT
    left(query, 80)                AS query_preview,
    calls,
    round(mean_exec_time::numeric, 2) AS avg_ms,
    round(min_exec_time::numeric, 2)  AS min_ms,
    round(max_exec_time::numeric, 2)  AS max_ms,
    round(stddev_exec_time::numeric, 2) AS stddev_ms,
    round(total_exec_time::numeric, 2) AS total_ms,
    round(100.0 * total_exec_time /

```

```

        sum(total_exec_time) OVER (), 1) AS pct_total_time
FROM pg_stat_statements
WHERE query ILIKE '%<=>%'      -- vector distance operator
      OR query ILIKE '%<#>%'   -- negative inner product
      OR query ILIKE '%<+>%'   -- L1 distance
ORDER BY total_exec_time DESC
LIMIT 20;

-- Real-time active vector queries
SELECT
    pid,
    now() - pg_stat_activity.query_start AS duration,
    state,
    wait_event_type,
    wait_event,
    left(query, 120) AS query
FROM pg_stat_activity
WHERE query ILIKE '%<=>%'
      AND state != 'idle'
ORDER BY duration DESC;

```

Recall Measurement and Regression Testing

Recall—the fraction of true nearest neighbors returned by the approximate index—is the metric that distinguishes a production-grade pgvector deployment from one that silently returns wrong answers. Measure it continuously as your data grows and your embedding model changes.

```

# Python - recall measurement harness for pgvector HNSW

import psycopg2
import numpy as np
import time
from typing import Any

def measure_recall(
    conn_string: str,
    test_vectors: list[np.ndarray],
    k: int = 10,
    ef_search: int = 100
) -> dict[str, Any]:
    """
    Measures recall@k by comparing HNSW results to exact brute-force results.
    test_vectors: list of query vectors
    Returns: recall@k, avg latency (ms), p99 latency (ms)
    """
    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()

    recalls = []
    latencies = []

    for qvec in test_vectors:
        vec_str = '[' + ', '.join(f'{v:.6f}' for v in qvec) + ']'

        # Ground truth: exact brute-force (seqscan)
        cur.execute("SET enable_indexscan = OFF")
        cur.execute(
            f"SELECT id FROM embeddings ORDER BY embedding <=> %s::vector LIMIT %s",
            (vec_str, k)
        )
        exact_ids = set(row[0] for row in cur.fetchall())
        cur.execute("SET enable_indexscan = ON")

        # Approximate: HNSW with ef_search

```

```

cur.execute(f"SET hnsw.ef_search = {ef_search}")
start = time.perf_counter()
cur.execute(
    f"SELECT id FROM embeddings ORDER BY embedding <=> %s::vector LIMIT %s",
    (vec_str, k)
)
elapsed_ms = (time.perf_counter() - start) * 1000
approx_ids = set(row[0] for row in cur.fetchall())

recall = len(exact_ids & approx_ids) / k
recalls.append(recall)
latencies.append(elapsed_ms)

cur.close()
conn.close()

latencies_arr = np.array(latencies)
return {
    "recall_at_k": round(np.mean(recalls), 4),
    "k": k,
    "ef_search": ef_search,
    "n_queries": len(test_vectors),
    "avg_latency_ms": round(np.mean(latencies_arr), 2),
    "p50_latency_ms": round(np.percentile(latencies_arr, 50), 2),
    "p99_latency_ms": round(np.percentile(latencies_arr, 99), 2),
}

# Sweep ef_search values to find the recall/latency Pareto frontier
def recall_latency_sweep(
    conn_string: str,
    test_vectors: list[np.ndarray],
    ef_search_values: list[int] = [20, 40, 60, 100, 150, 200],
    k: int = 10
) -> list[dict]:
    results = []
    for ef in ef_search_values:
        result = measure_recall(conn_string, test_vectors, k=k, ef_search=ef)
        results.append(result)
        print(
            f"ef_search={ef:4d} | recall={result['recall_at_k']:.4f} "
            f"| avg={result['avg_latency_ms']:.6f}ms "
            f"| p99={result['p99_latency_ms']:.6f}ms "
        )
    return results

```

Run this sweep after every schema migration, embedding model change, or significant data volume increase. Plot recall vs. latency and choose the `ef_search` value that sits at the knee of the curve—typically where doubling `ef_search` yields less than 1% additional recall.

Observability: Exposing pgvector Metrics to Prometheus

Production deployments need vector-specific metrics beyond what `pg_stat_statements` provides. The following shows how to instrument a pgvector deployment with a lightweight Prometheus exporter:

```

# Python - custom Prometheus metrics for pgvector
# Extend prometheus_postgres_exporter or run as a sidecar

from prometheus_client import Gauge, Histogram, start_http_server
import psycopg2
import time

```

```

# Define metrics
INDEX_SIZE_BYTES = Gauge(
    'pgvector_index_size_bytes',
    'HNSW or IVFFlat index size in bytes',
    ['index_name', 'table_name']
)
TABLE_ROW_COUNT = Gauge(
    'pgvector_table_row_count',
    'Approximate row count in embedding table',
    ['table_name']
)
QUERY_RECALL_ESTIMATE = Gauge(
    'pgvector_recall_estimate',
    'Estimated recall from last recall measurement',
    ['table_name', 'ef_search']
)
VECTOR_QUERY_LATENCY = Histogram(
    'pgvector_query_latency_seconds',
    'Latency of vector similarity queries',
    ['table_name'],
    buckets=[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0]
)

def collect_pgvector_metrics(conn_string: str) -> None:
    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()

    # Index sizes
    cur.execute("""
        SELECT
            i.relname AS index_name,
            t.relname AS table_name,
            pg_relation_size(i.oid) AS index_bytes
        FROM pg_index x
        JOIN pg_class i ON i.oid = x.indexrelid
        JOIN pg_class t ON t.oid = x.indrelid
        JOIN pg_am a ON a.oid = i.relam
        WHERE a.amname IN ('hns', 'ivfflat')
    """)
    for row in cur.fetchall():
        INDEX_SIZE_BYTES.labels(
            index_name=row[0], table_name=row[1]
        ).set(row[2])

    # Row counts (use reltuples for speed on large tables)
    cur.execute("""
        SELECT relname, reltuples
        FROM pg_class
        WHERE relname IN ('embeddings', 'embeddings_hv')
        AND relkind = 'r'
    """)
    for row in cur.fetchall():
        TABLE_ROW_COUNT.labels(table_name=row[0]).set(row[1])

    cur.close()
    conn.close()

def metrics_loop(conn_string: str, interval_seconds: int = 30) -> None:
    start_http_server(9187) # standard postgres_exporter port
    while True:
        try:
            collect_pgvector_metrics(conn_string)
        except Exception as e:
            print(f"Metrics collection error: {e}")
        time.sleep(interval_seconds)

```

Pair these custom metrics with standard PostgreSQL dashboards in Grafana. Set alerts on `pgvector_index_size_bytes` growing more than 20% per day (indicates write amplification is

sues) and on `pgvector_recall_estimate` dropping below your SLA threshold (typically 0.95 for RAG applications).

Key Takeaways

- **IVFFlat vs. HNSW is a build-vs-query tradeoff:** IVFFlat builds faster and uses less memory but degrades in recall as data distribution drifts; HNSW maintains recall more reliably at the cost of a larger, slower-to-build index. Choose based on whether your embedding distribution is stable and how often you can afford to rebuild.
 - **Parallel HNSW builds are a force multiplier:** Setting `max_parallel_maintenance_workers` before `CREATE INDEX` can reduce build time by 4–8× on multi-core hardware. This is the single highest-ROI configuration change for large-scale deployments.
 - **Bulk load outside the index:** Dropping the HNSW index before mass insertion and rebuilding afterward is dramatically faster than inserting through a live index—often 10–50× faster for batches over 100K rows. Use `COPY` instead of `INSERT` for the data load itself.
 - **Hybrid filtering requires architectural choices:** PostgreSQL’s planner cannot pre-filter inside HNSW traversal. Use partial indexes for small tenant counts, table partitioning for larger ones, or application-layer oversampling with post-filter when filter predicates are dynamic and unpredictable.
 - **Recall degrades silently without measurement:** Index parameter changes, data volume growth, and embedding model updates all affect recall without raising errors. Build a continuous recall measurement harness and treat recall regression the same way you treat a test suite failure—as a deployment blocker until investigated.
-

Chapter 11: Hybrid Search: Combining Full-Text and Vector Search

Full-text search and vector search solve different problems, and in production AI applications you almost always need both. Full-text search excels at exact keyword matching, proper nouns, SKU codes, and domain-specific terminology that embedding models may not have seen during training. Vector search excels at semantic similarity, handling paraphrase, typos, and intent that keywords miss entirely. This chapter covers how to combine these two retrieval mechanisms into a single pipeline — the architecture decisions, scoring fusion strategies, and the practical implementation details that determine whether hybrid search actually outperforms either approach alone. You will come away with working code for pgvector, Pinecone, and Weaviate, and a mental model for tuning the balance between lexical and semantic signals in your own applications.

11.1 Why Neither Approach Alone Is Sufficient

To understand why hybrid search exists, it helps to be precise about where each approach fails.

Where full-text search fails

Traditional full-text search — whether PostgreSQL `tsvector`, Elasticsearch BM25, or SQL Server Full-Text — operates on term frequency and inverse document frequency (TF-IDF) with some normalization. The retrieval model rewards documents that share exact terms with the query. This creates predictable failure modes:

- *Vocabulary mismatch*: A user queries “car insurance” but the document says “automobile coverage.” No overlap, no result.
- *Intent without keywords*: “I need to fix my credit score” doesn’t share terms with an article titled “Improving Your FICO Rating.”
- *Multilingual queries*: A Spanish query over English documents retrieves nothing unless you’ve built language-specific synonym rings, which is expensive to maintain.

Where vector search fails

Embedding-based retrieval encodes queries and documents into a shared semantic space and retrieves by distance. The failure modes are different but equally real:

- *Rare proper nouns*: “NVDA earnings Q3 2024” — a model trained on general text may embed NVDA close to many tech companies, diluting precision on this specific ticker.
- *Exact strings*: Product codes, error codes, UUIDs. `ERR_CONN_REFUSED` is semantically close to other error strings, but a user searching for exactly that code wants exactly that code.
- *Recency and domain shift*: New terminology post-training cutoff is either out-of-vocabulary or embedded based on morphological similarity alone, which is unreliable.

The complementary hypothesis

The practical observation, borne out repeatedly in production retrieval systems, is that full-text and vector search retrieve different sets of relevant documents, with modest overlap. A hybrid retrieval system that can combine both sets tends to outperform either alone on NDCG and recall@k metrics. The BEIR benchmark suite — a widely used collection of retrieval datasets spanning multiple domains — is the standard academic reference point for this comparison, and hybrid BM25 + dense retrieval configurations frequently outperform either component alone across a majority of its datasets, though the exact margin and dataset count vary by model pairing and fusion method, so treat any single number here as illustrative rather than a benchmark to cite directly.

For database engineers, this is analogous to query plan diversity in parallel query execution: two access paths that each capture a partial result can be combined through a merge operation with a ranking function to produce a result that is better than either path alone.

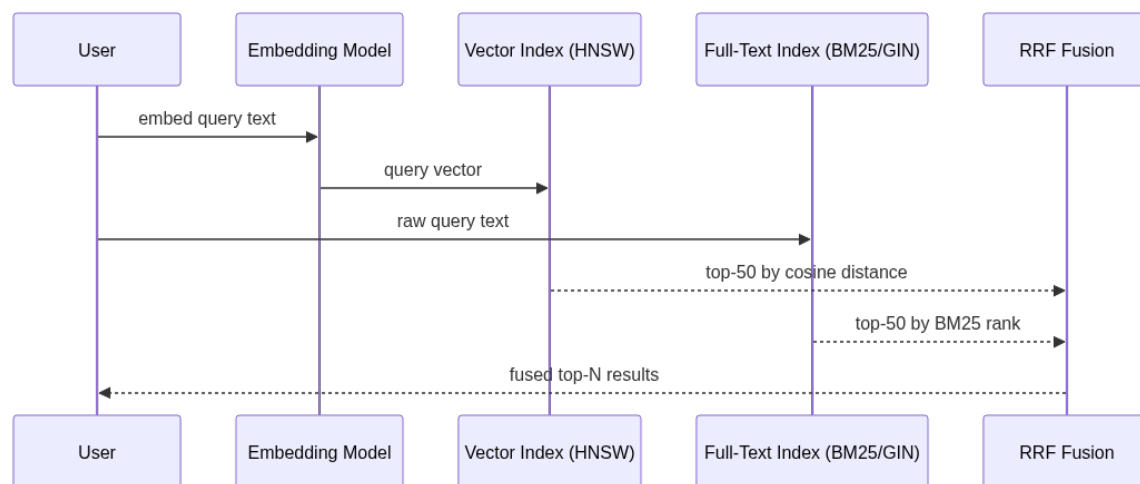


Figure 8: Vector and full-text retrieval run independently; Reciprocal Rank Fusion merges the two ranked lists using rank position alone, so no score normalization is needed.

11.2 Scoring Fusion: RRF, Linear Combination, and When to Use Each

The core architectural question in hybrid search is not how to retrieve from two sources — that part is straightforward. The hard part is how to merge two ranked lists with incomparable scores into a single ranked result.

Reciprocal Rank Fusion (RRF)

RRF is the dominant approach in production systems because it is score-agnostic. It only uses rank position, not the raw score. The formula:

$$\text{RRF_score}(d) = \sum 1 / (k + \text{rank_i}(d))$$

Where k is a smoothing constant (typically 60, from the original Cormack et al. 2009 paper), and $\text{rank_i}(d)$ is the position of document d in result list i . A document that appears at rank 1 in both lists scores $1/61 + 1/61 = 0.0328$. A document that appears at rank 1 in only one list but rank 100 in the other scores $1/61 + 1/160 = 0.0226$.

The strength of RRF is that you don't need to normalize scores from BM25 and cosine similarity, which live on completely different scales. The weakness is that it treats all rank positions equally in terms of *relative* weight, which means you cannot explicitly up-weight the semantic signal over the lexical signal or vice versa.

Linear Score Combination

When you need tunable control, linear combination gives you a single weight that governs the balance:

$$\text{hybrid_score}(d) = \alpha * \text{normalize}(\text{vector_score}(d)) + (1 - \alpha) * \text{normalize}(\text{bm25_score}(d))$$

This requires score normalization — min-max normalization per result set is typical. The challenge is that BM25 score distributions vary with corpus size and query length, so your normalization bounds shift with every query. This makes α a hyperparameter that needs continuous calibration as your corpus changes.

In practice: $\alpha = 0.75$ (heavy semantic) works well for open-domain question answering where intent matters more than keywords. $\alpha = 0.25$ (heavy lexical) works well for enterprise search over technical documentation where product names and codes are common. $\alpha = 0.5$ is a reasonable starting point for unknown workloads.

Which to choose

Criterion	RRF	Linear Combination
Score normalization required	No	Yes
Tunable weighting	No	Yes
Stable across corpus changes	High	Medium
Implementation complexity	Low	Medium
Calibration overhead	None	Requires labeled queries

For most new deployments, start with RRF. Move to linear combination only if you have labeled evaluation data to calibrate α and a system for monitoring retrieval quality over time.

11.3 Hybrid Search in pgvector + PostgreSQL Full-Text Search

PostgreSQL has native full-text search via `tsvector`/`tsquery` and the `ts_rank` function. Combined with pgvector's `<=>` operator, you can implement a complete hybrid search pipeline inside a single database.

Schema design

```
-- PostgreSQL / pgvector
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
  id          BIGSERIAL PRIMARY KEY,
  title       TEXT NOT NULL,
  body       TEXT NOT NULL,
  source      TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  -- Vector embedding (text-embedding-3-small = 1536 dims)
  embedding   VECTOR(1536),
  -- Full-text search vector (precomputed for performance)
  fts_vector  TSVECTOR GENERATED ALWAYS AS (
    setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
    setweight(to_tsvector('english', coalesce(body, '')), 'B')
  ) STORED
);

-- GIN index for full-text search
CREATE INDEX idx_documents_fts
  ON documents USING GIN (fts_vector);

-- HNSW index for vector search
CREATE INDEX idx_documents_embedding
  ON documents USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

RRF hybrid query

```
-- PostgreSQL / pgvector
-- Hybrid search using Reciprocal Rank Fusion
-- :query_text = raw text query from user
-- :query_embed = pgvector literal of the embedding, e.g. '[0.1, 0.2, ...]'
-- :k           = RRF smoothing constant (default 60)
-- :top_n       = number of final results to return

WITH
-- Semantic search: top 50 candidates by cosine similarity
vector_results AS (
  SELECT
    id,
    1 - (embedding <=> :query_embed::vector) AS score,
    ROW_NUMBER() OVER (ORDER BY embedding <=> :query_embed::vector) AS rank
  FROM documents
  WHERE embedding IS NOT NULL
  ORDER BY embedding <=> :query_embed::vector
  LIMIT 50
),
-- Full-text search: top 50 candidates by BM25-equivalent ts_rank
fts_results AS (
  SELECT
    id,
    ts_rank_cd(fts_vector, query) AS score,
    ROW_NUMBER() OVER (ORDER BY ts_rank_cd(fts_vector, query) DESC) AS rank
  FROM documents,
    to_tsquery('english', :query_text) query
  WHERE fts_vector @@ query
  ORDER BY ts_rank_cd(fts_vector, query) DESC
  LIMIT 50
),
-- RRF merge
rrf_scores AS (
  SELECT
    COALESCE(v.id, f.id) AS id,
    COALESCE(1.0 / (60 + v.rank), 0) +
    COALESCE(1.0 / (60 + f.rank), 0) AS rrf_score
  FROM vector_results v
  FULL OUTER JOIN fts_results f ON v.id = f.id
```

```

)
SELECT
    d.id,
    d.title,
    d.body,
    r.rrf_score
FROM rrf_scores r
JOIN documents d ON d.id = r.id
ORDER BY r.rrf_score DESC
LIMIT :top_n;

```

Performance notes

The query above runs two independent index scans and then merges via a hash join on the combined candidate set (at most 100 rows). At 10M documents, expect: - Vector HNSW scan: 5–15ms with `ef_search = 40` - GIN full-text scan: 2–8ms depending on term selectivity - Merge and sort: sub-millisecond

Total latency under 20ms for the retrieval phase at this scale is achievable on a properly sized instance (8+ vCPU, fast NVMe SSD). The main tuning lever is reducing the `LIMIT 50` in each subquery — smaller candidate sets mean faster merges at the cost of potentially missing relevant documents.

Python wrapper with automatic embedding

```

# Python
import psycopg2
import numpy as np
from openai import OpenAI

openai_client = OpenAI() # uses OPENAI_API_KEY env var

def get_embedding(text: str, model: str = "text-embedding-3-small") -> list[float]:
    response = openai_client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

def prepare_fts_query(text: str) -> str:
    """Convert raw text to a PostgreSQL tsquery-compatible string."""
    # Strip punctuation and join with & for AND semantics
    # For production, consider using websearch_to_tsquery instead
    tokens = [t.strip() for t in text.split() if len(t.strip()) > 2]
    return " & ".join(tokens)

def hybrid_search(
    conn,
    query_text: str,
    top_n: int = 10,
    k_rrf: int = 60,
) -> list[dict]:
    embedding = get_embedding(query_text)
    fts_query = prepare_fts_query(query_text)

    # Format vector as PostgreSQL literal
    vec_literal = "[" + ",".join(f"{x:.6f}" for x in embedding) + "]"

    sql = """
WITH
vector_results AS (
    SELECT id,
           ROW_NUMBER() OVER (ORDER BY embedding <=> %s::vector) AS rank
    FROM documents
    WHERE embedding IS NOT NULL
    ORDER BY embedding <=> %s::vector
    LIMIT 50
),
fts_results AS (

```

```
SELECT id,
       ROW_NUMBER() OVER (
         ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', %s)) DESC
       ) AS rank
FROM documents
WHERE fts_vector @@ to_tsquery('english', %s)
ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', %s)) DESC
LIMIT 50
),
rrf_scores AS (
  SELECT
    COALESCE(v.id, f.id) AS id,
    COALESCE(1.0 / (%s + v.rank), 0) +
    COALESCE(1.0 / (%s + f.rank), 0) AS rrf_score
  FROM vector_results v
  FULL OUTER JOIN fts_results f ON v.id = f.id
)
SELECT d.id, d.title, LEFT(d.body, 300) AS snippet, r.rrf_score
FROM rrf_scores r
JOIN documents d ON d.id = r.id
ORDER BY r.rrf_score DESC
LIMIT %s;
"""

params = (
    vec_literal, vec_literal,      # vector search (used twice for ROW_NUMBER)
    fts_query, fts_query,          # FTS (used three times)
    k_rrf, k_rrf,                  # RRF constant
    top_n,
)

with conn.cursor() as cur:
    cur.execute(sql, params)
    rows = cur.fetchall()

return [
    {"id": r[0], "title": r[1], "snippet": r[2], "rrf_score": float(r[3])}
    for r in rows
]

# Usage
conn = psycopg2.connect("postgresql://user:pass@localhost:5432/mydb")
results = hybrid_search(conn, "how to improve credit score quickly", top_n=5)
for r in results:
    print(f"{{r['rrf_score']:.4f}} {{r['title']}}")
```

One practical note on `prepare_fts_query`: for production workloads, replace the manual tokenization with PostgreSQL's `websearch_to_tsquery`, which handles phrase queries and negation correctly. The simple `&` join is shown here for transparency, but it fails on multi-word phrases.

11.4 Hybrid Search in Pinecone

Pinecone's hybrid search model relies on sparse vectors (representing BM25 term weights) alongside dense vectors. Both are stored in the same index, and the query combines them using a weighted sum controlled by the `alpha` parameter — `alpha=1.0` is pure dense, `alpha=0.0` is pure sparse.

Pinecone uses their own sparse encoding scheme, but the inputs are standard: you compute sparse vectors using a BM25 encoder (Pinecone provides one via `pinecone-text`) and dense vectors using any embedding model.

```
# Python
from pinecone import Pinecone
```



```
from pinecone_text.sparse import BM25Encoder
from openai import OpenAI

# Initialize clients
pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")
openai_client = OpenAI()

# Connect to a dotproduct-metric index (required for hybrid search)
# Index must be created with metric="dotproduct" and sparse vectors enabled
index = pc.Index("hybrid-search-demo")

# Initialize BM25 encoder - fit on your corpus first
bm25 = BM25Encoder()

def fit_bm25_on_corpus(corpus: list[str], save_path: str = "bm25_params.json"):
    """Fit BM25 encoder on corpus and persist params."""
    bm25.fit(corpus)
    bm25.dump(save_path)
    return bm25

def load_bm25(path: str = "bm25_params.json") -> BM25Encoder:
    return BM25Encoder.load(path)

def get_dense_embedding(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

def upsert_documents(documents: list[dict], bm25_encoder: BM25Encoder):
    """
    documents: list of {"id": str, "text": str, "metadata": dict}
    """
    vectors = []
    texts = [d["text"] for d in documents]

    # Batch embed (dense)
    embed_response = openai_client.embeddings.create(
        input=texts,
        model="text-embedding-3-small"
    )
    dense_vecs = [e.embedding for e in embed_response.data]

    # Sparse encode
    sparse_vecs = bm25_encoder.encode_documents(texts)

    for doc, dense, sparse in zip(documents, dense_vecs, sparse_vecs):
        vectors.append({
            "id": doc["id"],
            "values": dense, # dense vector
            "sparse_values": sparse, # sparse BM25 vector
            "metadata": doc.get("metadata", {}),
        })

    # Upsert in batches of 100 (Pinecone limit)
    batch_size = 100
    for i in range(0, len(vectors), batch_size):
        index.upsert(vectors=vectors[i:i + batch_size])

def hybrid_query(
    query_text: str,
    bm25_encoder: BM25Encoder,
    top_k: int = 10,
    alpha: float = 0.7, # 0 = pure BM25, 1 = pure dense
    filter: dict = None,
) -> list[dict]:
    """
    alpha=0.7 gives 70% weight to semantic similarity, 30% to BM25.
    """
```

```

dense_vec = get_dense_embedding(query_text)
sparse_vec = bm25_encoder.encode_queries(query_text)

# Scale vectors according to alpha for weighted combination
# Pinecone combines: score = alpha * dense_dot + (1-alpha) * sparse_dot
scaled_dense = [v * alpha for v in dense_vec]
scaled_sparse = {
    "indices": sparse_vec["indices"],
    "values": [v * (1 - alpha) for v in sparse_vec["values"]],
}

query_params = {
    "vector": scaled_dense,
    "sparse_vector": scaled_sparse,
    "top_k": top_k,
    "include_metadata": True,
}
if filter:
    query_params["filter"] = filter

result = index.query(**query_params)

return [
    {
        "id": m["id"],
        "score": m["score"],
        "metadata": m["metadata"],
    }
    for m in result["matches"]
]

# Example usage
bm25_encoder = load_bm25("bm25_params.json")

results = hybrid_query(
    query_text="vehicle insurance coverage requirements",
    bm25_encoder=bm25_encoder,
    top_k=5,
    alpha=0.7,
)

for r in results:
    print(f"[{r['score']:.4f}] ID={r['id']} {r['metadata'].get('title','')}")

```

Index creation requirement

Pinecone's hybrid search requires a `dotproduct` metric index. `cosine` and `euclidean` indexes do not support sparse vectors. This is because the combination of dense and sparse scores is implemented as a dotproduct internally. If you are creating a new index for hybrid use, specify this from the start — you cannot convert an existing index.

```

# Python
# Create a new hybrid-capable Pinecone index
pc.create_index(
    name="hybrid-search-demo",
    dimension=1536,  # must match your embedding model
    metric="dotproduct",  # required for hybrid search
    spec=ServerlessSpec(
        cloud="aws",
        region="us-east-1",
    ),
)

```

Tradeoff to understand: Pinecone's BM25 sparse encoding is computed at indexing time against the full corpus term statistics. If your corpus grows significantly (by more than ~20%), the IDF weights in your BM25 encoder become stale and retrieval quality degrades. Plan for periodic BM25

re-fitting, followed by a full re-upsert if sparse vectors change materially.

11.5 Hybrid Search in Weaviate

Weaviate has first-class hybrid search support through its `hybrid` query parameter, which combines BM25 (their inverted index) and vector search using a configurable fusion strategy — either `rankedFusion` (RRF) or `relativeScoreFusion` (score normalization + linear combination).

```
# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.query import HybridFusion
from openai import OpenAI

# Connect to Weaviate (local or cloud)
client = weaviate.connect_to_local() # or weaviate.connect_to_wcs(...)

openai_client = OpenAI()

# --- Schema setup (run once) ---
def create_documents_collection(client):
    """
    Weaviate manages embeddings automatically if you configure a vectorizer.
    Here we use manual vectors (bring-your-own-embeddings) for full control.
    """
    if client.collections.exists("Document"):
        return

    client.collections.create(
        name="Document",
        vectorizer_config=wvc.config.Configure.Vectorizer.none(), # manual embeddings
        properties=[
            wvc.config.Property(name="title", data_type=wvc.config.DataType.TEXT),
            wvc.config.Property(name="body", data_type=wvc.config.DataType.TEXT),
            wvc.config.Property(name="source", data_type=wvc.config.DataType.TEXT),
        ],
        vector_index_config=wvc.config.Configure.VectorIndex.hnsw(
            distance_metric=wvc.config.VectorDistances.COSINE,
            ef_construction=128,
            max_connections=16,
        ),
        inverted_index_config=wvc.config.Configure.inverted_index(
            bm25_b=0.75, # length normalization factor
            bm25_k1=1.2, # term frequency saturation
        ),
    )

def embed_text(text: str) -> list[float]:
    resp = openai_client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return resp.data[0].embedding

# --- Insert documents ---
def insert_documents(client, docs: list[dict]):
    collection = client.collections.get("Document")
    with collection.batch.dynamic() as batch:
        for doc in docs:
            embedding = embed_text(doc["title"] + " " + doc["body"])
            batch.add_object(
                properties={
                    "title": doc["title"],
                    "body": doc["body"],
```

```

        "source": doc.get("source", ""),
    },
    vector=embedding,
)

# --- Hybrid query using Weaviate's built-in fusion ---
def hybrid_search_weaviate(
    client,
    query_text: str,
    query_embedding: list[float],
    top_k: int = 10,
    alpha: float = 0.7,
    fusion_type: HybridFusion = HybridFusion.RANKED, # RRF
) -> list[dict]:
    """
    alpha=1.0  → pure vector search
    alpha=0.0  → pure BM25 search
    alpha=0.5  → balanced hybrid

    fusion_type:
        HybridFusion.RANKED          → Reciprocal Rank Fusion
        HybridFusion.RELATIVE_SCORE → Normalized score combination
    """
    collection = client.collections.get("Document")

    results = collection.query.hybrid(
        query=query_text,          # used for BM25 scoring
        vector=query_embedding,    # used for vector scoring
        alpha=alpha,
        fusion_type=fusion_type,
        limit=top_k,
        return_properties=["title", "body", "source"],
        return_metadata=wvc.query.MetadataQuery(score=True, explain_score=True),
    )

    return [
        {
            "id": str(obj.uuid),
            "title": obj.properties["title"],
            "snippet": obj.properties["body"][:300],
            "score": obj.metadata.score,
        }
        for obj in results.objects
    ]

# --- Usage ---
create_documents_collection(client)

query = "how to repair damaged credit"
q_embedding = embed_text(query)

results = hybrid_search_weaviate(
    client,
    query_text=query,
    query_embedding=q_embedding,
    top_k=5,
    alpha=0.6,
    fusion_type=HybridFusion.RANKED,
)

for r in results:
    print(f"{r['score']:.4f} {r['title']}")

client.close()

```

Choosing between RANKED and RELATIVE_SCORE in Weaviate

HybridFusion.RANKED applies RRF — the same formula discussed in Section 11.2. It is stable, requires no score normalization, and handles corpus changes gracefully.

`HybridFusion.RELATIVE_SCORE` normalizes both score distributions to $[0, 1]$ within each result set before combining. This preserves information about how much better rank-1 is than rank-10 (which RRF ignores), but it is sensitive to the shape of the score distribution. If your BM25 scores for a particular query are clustered tightly (e.g., many documents with similar term frequency), the relative score fusion will treat them as nearly equivalent even when there is a meaningful difference in relevance. For most workloads, start with `RANKED`.

GraphQL alternative for Weaviate

If you are working with Weaviate's HTTP interface directly rather than the Python client, the equivalent hybrid query via GraphQL:

```
{
  Get {
    Document(
      hybrid: {
        query: "how to repair damaged credit"
        vector: [0.012, -0.034, ...] # truncated
        alpha: 0.6
        fusionType: rankedFusion
      }
      limit: 5
    ) {
      title
      body
      _additional {
        score
        explainScore
      }
    }
  }
}
```

11.6 Filtering in Hybrid Search

Production hybrid search almost always involves metadata filters — user-specific document sets, date ranges, product categories, access control lists. Applying filters correctly is harder than it looks, because the interaction between filters and vector indexes can silently degrade recall.

Pre-filter vs. Post-filter

Post-filtering is the naive approach: retrieve top-k from each component, merge, then discard results that don't match the filter. It is easy to implement but broken by design when the filter is selective. If 95% of your corpus is filtered out, top-k vector retrieval returns mostly documents that will be discarded, and you end up with fewer than k results despite abundant matching documents.

Pre-filtering applies the filter before or during the approximate nearest neighbor search. This is the correct approach for selective filters, but it requires index support. PostgreSQL with pgvector supports this naturally through `WHERE` clause conditions that combine with the vector index scan (though with very selective filters, the query planner may fall back to a sequential scan — always `EXPLAIN ANALYZE`). Pinecone and Weaviate both implement pre-filtering natively for their managed indexes.

Implementing pre-filtering in pgvector

```
-- PostgreSQL / pgvector
-- Hybrid search with pre-filter on source and date range
```

```

WITH
vector_results AS (
  SELECT
    id,
    ROW_NUMBER() OVER (ORDER BY embedding <=> :query_embed::vector) AS rank
  FROM documents
  WHERE embedding IS NOT NULL
    AND source = :source_filter          -- pre-filter applied before ranking
    AND created_at >= :date_from
  ORDER BY embedding <=> :query_embed::vector
  LIMIT 50
),
fts_results AS (
  SELECT
    id,
    ROW_NUMBER() OVER (
      ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', :query_text)) DESC
    ) AS rank
  FROM documents
  WHERE fts_vector @@ to_tsquery('english', :query_text)
    AND source = :source_filter          -- same pre-filter
    AND created_at >= :date_from
  ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', :query_text)) DESC
  LIMIT 50
),
rrf_scores AS (
  SELECT
    COALESCE(v.id, f.id) AS id,
    COALESCE(1.0 / (60 + v.rank), 0) +
    COALESCE(1.0 / (60 + f.rank), 0) AS rrf_score
  FROM vector_results v
  FULL OUTER JOIN fts_results f ON v.id = f.id
)
SELECT d.id, d.title, LEFT(d.body, 300) AS snippet, r.rrf_score
FROM rrf_scores r
JOIN documents d ON d.id = r.id
ORDER BY r.rrf_score DESC
LIMIT :top_n;

```

When the filter column has low cardinality and high selectivity (e.g., `source` narrows results to fewer than 1% of the corpus), the HNSW index may be bypassed entirely. In that case, consider a partial index:

```

-- PostgreSQL
-- Partial HNSW index for a specific source
CREATE INDEX idx_documents_embedding_internal
  ON documents USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64)
  WHERE source = 'internal';

```

Partial indexes require one index per filter predicate combination, so this approach scales only when you have a small, stable set of filter values that appear frequently.

Filtering in Pinecone

```

# Python
# Pinecone pre-filtering using metadata filters
results = hybrid_query(
  query_text="vehicle insurance requirements",
  bm25_encoder=bm25_encoder,
  top_k=10,
  alpha=0.7,
  filter={
    "$and": [
      {"source": {"$eq": "internal"}},
      {"created_year": {"$gte": 2022}},
    ]
  }
)

```

)

Pinecone applies metadata filters during the ANN search, not after. This means the filter is always pre-filter semantically, though Pinecone’s internal implementation uses a combination of brute-force scan and index pruning depending on filter selectivity. For highly selective filters over large indexes (hundreds of millions of vectors), Pinecone recommends namespacing (partitioning by filter value) rather than relying on metadata filters alone.

Filtering in Weaviate

```
# Python
import weaviate.classes as wvc

results = collection.query.hybrid(
    query=query_text,
    vector=query_embedding,
    alpha=0.6,
    limit=10,
    filters=wvc.query.Filter.by_property("source").equal("internal")
    & wvc.query.Filter.by_property("created_year").greater_or_equal(2022),
    return_properties=["title", "body", "source"],
    return_metadata=wvc.query.MetadataQuery(score=True),
)
```

Weaviate implements filtering through an ACORN-style approach that integrates the filter into the HNSW graph traversal, substantially reducing the post-filter recall loss that affects naïve implementations.

11.7 Evaluating Hybrid Search Quality

You cannot tune `alpha` or decide between RRF and linear combination without a way to measure whether your changes help. This section covers the minimum evaluation infrastructure you need.

The offline evaluation loop

1. **Collect a query set with relevance judgments.** The minimum useful set is 50–100 queries, each with at least 3–5 relevant document IDs manually labeled by domain experts or collected from user click logs. For production systems, you want several hundred labeled queries to get stable metric estimates.
2. **Compute retrieval metrics.** The two most useful for hybrid search tuning are:
 - *Recall@k*: Of all relevant documents, what fraction appear in the top-k results? Use $k=10$ or $k=20$ depending on your application’s display window.
 - *NDCG@k* (Normalized Discounted Cumulative Gain): Accounts for rank position. A relevant document at rank 1 is worth more than one at rank 10.
3. **Run A/B comparisons across configurations.** Test BM25-only, vector-only, RRF hybrid, and linear-combination hybrid. The hybrid configurations should consistently match or exceed the best single-mode configuration.

Minimal Python evaluation harness

```
# Python
import numpy as np

def recall_at_k(retrieved_ids: list, relevant_ids: set, k: int) -> float:
```

```

    """Fraction of relevant documents appearing in top-k retrieved."""
    if not relevant_ids:
        return 0.0
    retrieved_at_k = set(retrieved_ids[:k])
    return len(retrieved_at_k & relevant_ids) / len(relevant_ids)

def dcg_at_k(retrieved_ids: list, relevant_ids: set, k: int) -> float:
    """Discounted Cumulative Gain at k."""
    dcg = 0.0
    for i, doc_id in enumerate(retrieved_ids[:k]):
        if doc_id in relevant_ids:
            dcg += 1.0 / np.log2(i + 2)  # i+2 because log2(1) = 0
    return dcg

def ndcg_at_k(retrieved_ids: list, relevant_ids: set, k: int) -> float:
    """Normalized DCG: DCG / ideal DCG."""
    ideal_hits = min(len(relevant_ids), k)
    ideal_dcg = sum(1.0 / np.log2(i + 2) for i in range(ideal_hits))
    if ideal_dcg == 0:
        return 0.0
    return dcg_at_k(retrieved_ids, relevant_ids, k) / ideal_dcg

def evaluate_retrieval(
    queries: list[dict],          # [{"query": str, "relevant_ids": list[str]}]
    retrieval_fn,                # callable(query_text) -> list[str] of doc IDs
    k: int = 10,
) -> dict:
    recalls, ndcgs = [], []
    for item in queries:
        retrieved = retrieval_fn(item["query"])
        relevant = set(item["relevant_ids"])
        recalls.append(recall_at_k(retrieved, relevant, k))
        ndcgs.append(ndcg_at_k(retrieved, relevant, k))
    return {
        f"recall@{k}": np.mean(recalls),
        f"ndcg@{k}": np.mean(ndcgs),
        "n_queries": len(queries),
    }

# Example: compare BM25-only vs hybrid
labeled_queries = [
    {"query": "fix credit score fast", "relevant_ids": ["doc_42", "doc_17", "doc_88"]},
    # ... more labeled queries
]

bm25_metrics = evaluate_retrieval(labeled_queries, bm25_only_search, k=10)
vector_metrics = evaluate_retrieval(labeled_queries, vector_only_search, k=10)
hybrid_metrics = evaluate_retrieval(labeled_queries, hybrid_search_fn, k=10)

print(f"BM25-only: {bm25_metrics}")
print(f"Vector-only: {vector_metrics}")
print(f"Hybrid: {hybrid_metrics}")

```

Online signals

Offline metrics require labeled data, which is expensive to collect. Online signals from user behavior can supplement them:

- *Click-through rate on top-3 results*: If users consistently click results at position 4 or lower, your ranking is off.
- *Zero-result rate*: The fraction of queries where the retrieval system returns nothing useful. This is a strong signal that either your FTS query construction is too strict or your embeddings don't cover the domain.
- *Query reformulation rate*: Users who immediately rephrase a query signal dissatisfaction with the first retrieval attempt. This is detectable from session logs.

These signals don't replace labeled evaluation, but they allow continuous monitoring in production without ongoing manual labeling effort.

11.8 Operational Considerations: Latency, Cost, and Freshness

Latency budget

Hybrid search is inherently more expensive than single-mode retrieval: you are running two retrieval paths plus a merge. At small corpus sizes (< 1M documents), this is rarely a problem — both paths return in single-digit milliseconds. At larger scales, you have two strategies:

1. *Parallel execution*: Run the vector and full-text searches concurrently in separate coroutines/threads, then merge the results. This reduces wall-clock latency to $\max(t_{\text{vector}}, t_{\text{fts}})$ rather than the sum.
2. *Candidate set reduction*: Reduce the LIMIT in each subquery. Retrieving top-20 candidates from each path instead of top-50 cuts merge overhead and typically loses minimal quality — the probability that a genuinely relevant document appears between rank 20 and rank 50 in only one of the two retrievers is low.

Parallel PostgreSQL hybrid search in Python

```
# Python
import asyncio
import asyncpg

async def vector_search_async(pool, query_embed: str, limit: int = 20) -> list[tuple]:
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            """
            SELECT id, ROW_NUMBER() OVER (ORDER BY embedding <=> $1::vector) AS rank
            FROM documents
            WHERE embedding IS NOT NULL
            ORDER BY embedding <=> $1::vector
            LIMIT $2
            """,
            query_embed, limit,
        )
    return [(r["id"], r["rank"]) for r in rows]

async def fts_search_async(pool, fts_query: str, limit: int = 20) -> list[tuple]:
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            """
            SELECT id,
            ROW_NUMBER() OVER (
            ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', $1)) DESC
            ) AS rank
            FROM documents
            WHERE fts_vector @@ to_tsquery('english', $1)
            ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', $1)) DESC
            LIMIT $2
            """,
            fts_query, limit,
        )
    return [(r["id"], r["rank"]) for r in rows]

def rrf_merge(
    vector_results: list[tuple],
    fts_results: list[tuple],
    k: int = 60,

```

```
top_n: int = 10,
) -> list[tuple]:
    scores: dict[int, float] = {}
    for doc_id, rank in vector_results:
        scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    for doc_id, rank in fts_results:
        scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
    return ranked[:top_n]

async def parallel_hybrid_search(pool, query_text: str, top_n: int = 10) -> list[tuple]:
    embedding = get_embedding(query_text)
    vec_literal = "[" + ", ".join(f"{x:.6f}" for x in embedding) + "]"
    fts_query = prepare_fts_query(query_text)

    vector_task = asyncio.create_task(vector_search_async(pool, vec_literal, limit=20))
    fts_task = asyncio.create_task(fts_search_async(pool, fts_query, limit=20))

    vector_results, fts_results = await asyncio.gather(vector_task, fts_task)
    return rrf_merge(vector_results, fts_results, top_n=top_n)
```

Embedding cost and caching

Every hybrid query requires an embedding call (for the vector component) and a text query construction (for the BM25 component). The BM25 text query is free — it is just string manipulation. The embedding call is where cost concentrates. At high query volumes, cache embeddings for repeated or near-duplicate queries using an LRU cache keyed on normalized query text. A hit rate of 20–30% is common in production search applications due to popular queries.

```
# Python
from functools import lru_cache

@lru_cache(maxsize=4096)
def get_embedding_cached(text: str) -> tuple[float, ...]:
    """Cache embeddings for repeated queries. Returns tuple (hashable)."""
    return tuple(get_embedding(text))
```

Index freshness

Hybrid search has two freshness concerns: the vector index and the BM25 statistics.

For pgvector, the HNSW index is updated incrementally on every INSERT — no rebuild required. The GIN index for full-text is also maintained incrementally. Freshness is not a concern.

For Pinecone, new vectors appear in the index within seconds of upsert. However, if you re-fit your BM25 encoder because corpus term statistics have shifted significantly, you need to re-encode and re-upsert the sparse vectors for all existing documents. This is a batch operation — plan for it in your data engineering pipeline.

For Weaviate, both the HNSW index and the inverted index are updated in real-time on object insertion. The BM25 IDF weights are computed dynamically at query time from the live inverted index, so they stay current without any batch refresh step. This is a meaningful operational advantage over solutions that pre-compute and freeze BM25 statistics.

11.9 A Complete End-to-End Example

This section assembles a minimal but complete hybrid search service backed by PostgreSQL and pgvector, exposing a FastAPI endpoint. It is intended as a starting template, not a production-

ready service — error handling, authentication, and connection pool sizing are left to the reader.

```
# Python
# hybrid_search_service.py
from contextlib import asynccontextmanager
import asyncio
import asyncpg
from fastapi import FastAPI, Query
from pydantic import BaseModel
from openai import AsyncOpenAI

EMBEDDING_MODEL = "text-embedding-3-small"
DB_DSN = "postgresql://user:pass@localhost:5432/mydb"
POOL_MIN_SIZE = 5
POOL_MAX_SIZE = 20

openai_client = AsyncOpenAI()
pool: asyncpg.Pool = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    global pool
    pool = await asyncpg.create_pool(
        DB_DSN,
        min_size=POOL_MIN_SIZE,
        max_size=POOL_MAX_SIZE,
    )
    yield
    await pool.close()

app = FastAPI(title="Hybrid Search Service", lifespan=lifespan)

class SearchResult(BaseModel):
    id: int
    title: str
    snippet: str
    rrf_score: float

class SearchResponse(BaseModel):
    query: str
    results: list[SearchResult]

async def embed(text: str) -> list[float]:
    resp = await openai_client.embeddings.create(
        input=text,
        model=EMBEDDING_MODEL,
    )
    return resp.data[0].embedding

def to_fts_query(text: str) -> str:
    tokens = [t.strip() for t in text.split() if len(t.strip()) > 2]
    return " & ".join(tokens) if tokens else "null"

@app.get("/search", response_model=SearchResponse)
async def search(
    q: str = Query(..., min_length=2, description="Search query"),
    top_n: int = Query(10, ge=1, le=50),
    k_rrf: int = Query(60, ge=1),
):
    embedding = await embed(q)
    vec_literal = "[" + ", ".join(f"{x:.6f}" for x in embedding) + "]"
    fts_query = to_fts_query(q)

    # Run both searches in parallel
    async with pool.acquire() as conn:
        vector_task = conn.fetch(
            """
            SELECT id,
                ROW_NUMBER() OVER (ORDER BY embedding <=> $1::vector) AS rank
            FROM documents
            """
        )
```

```

        WHERE embedding IS NOT NULL
        ORDER BY embedding <=> $1::vector
        LIMIT 50
        """
        vec_literal,
    )
    fts_task = conn.fetch(
        """
        SELECT id,
               ROW_NUMBER() OVER (
                   ORDER BY ts_rank_cd(fts_vector,
                                     to_tsquery('english', $1)) DESC
               ) AS rank
        FROM documents
        WHERE fts_vector @@ to_tsquery('english', $1)
        ORDER BY ts_rank_cd(fts_vector, to_tsquery('english', $1)) DESC
        LIMIT 50
        """,
        fts_query,
    )
    # asyncpg doesn't support concurrent queries on the same connection;
    # use separate connections for true parallelism
    vector_rows = await vector_task
    fts_rows     = await fts_task

    # RRF merge in Python
    scores: dict[int, float] = {}
    for row in vector_rows:
        scores[row["id"]] = scores.get(row["id"], 0.0) + 1.0 / (k_rrf + row["rank"])
    for row in fts_rows:
        scores[row["id"]] = scores.get(row["id"], 0.0) + 1.0 / (k_rrf + row["rank"])

    ranked_ids = sorted(scores, key=lambda x: scores[x], reverse=True)[:top_n]

    if not ranked_ids:
        return SearchResponse(query=q, results=[])

    # Fetch document details for top results
    async with pool.acquire() as conn:
        docs = await conn.fetch(
            "SELECT id, title, LEFT(body, 300) AS snippet FROM documents WHERE id = ANY($1)",
            ranked_ids,
        )
    doc_map = {d["id"]: d for d in docs}

    results = [
        SearchResult(
            id=doc_id,
            title=doc_map[doc_id]["title"],
            snippet=doc_map[doc_id]["snippet"],
            rrf_score=scores[doc_id],
        )
        for doc_id in ranked_ids
        if doc_id in doc_map
    ]

    return SearchResponse(query=q, results=results)

```

To run:

```

pip install fastapi uvicorn asyncpg openai
uvicorn hybrid_search_service:app --host 0.0.0.0 --port 8080

```

Test with:

```

curl "http://localhost:8080/search?q=how+to+improve+credit+score&top_n=5"

```

Note the comment in the code about asyncpg connection sharing: asyncpg connections are not safe for concurrent queries on the same connection object. For true parallel execution, acquire two

connections from the pool, or use the `asyncio.gather` pattern shown in Section 11.8 with separate `pool.acquire()` calls.

Key Takeaways

- **Full-text search and vector search are complementary, not competing.** Full-text retrieval excels at exact terms, product codes, and proper nouns; vector retrieval excels at semantic intent and vocabulary mismatch. Combining both consistently improves retrieval quality over either approach alone.
 - **Reciprocal Rank Fusion (RRF) is the right default fusion strategy.** It requires no score normalization, is robust to corpus changes, and needs no labeled calibration data. Move to linear score combination only when you have sufficient labeled queries to tune and monitor the `alpha` weight reliably.
 - **Pre-filtering is not optional for selective metadata filters.** Post-filtering hybrid search degrades silently when filters are selective, returning fewer than `k` results even when many matching documents exist. Use database-native pre-filtering in all three systems covered here.
 - **Candidate set size is your primary latency lever.** Reducing the per-path candidate limit from 50 to 20 cuts merge overhead and typically loses minimal recall, because the probability of a relevant document appearing only between rank 20 and rank 50 in a single retriever is small. Measure the tradeoff on your own workload before committing to a limit.
 - **BM25 statistics have a freshness lifecycle in managed systems.** In Pinecone, sparse vectors are computed at indexing time against frozen corpus statistics and must be re-encoded when the corpus grows substantially. In Weaviate, IDF weights are recomputed from the live inverted index at query time, eliminating this concern. In PostgreSQL, the GIN index is updated incrementally on every write. Match your re-indexing strategy to the freshness model of your chosen system.
-

Chapter 12: Multi-Tenant Vector Storage in PostgreSQL

Multi-tenancy is one of the oldest problems in database engineering, and it doesn't get simpler when you add vectors to the mix. If you're building a SaaS product where every customer gets their own AI-powered search, a platform where multiple teams share infrastructure for semantic retrieval, or an enterprise system where business units need data isolation, you need a coherent strategy for partitioning vector data. This chapter walks through the three canonical approaches to multi-tenant vector storage in PostgreSQL with pgvector — row-level isolation, schema-level isolation, and database-level isolation — and gives you the tools to choose between them based on your scale, compliance requirements, and operational constraints. Beyond the data model, you'll also work through index management across tenants, query routing, and the performance cliffs that catch engineers off guard in production.

12.1 The Three Models of Multi-Tenancy (and Where Each Breaks Down)

Before writing a single line of SQL, you need to understand the fundamental tradeoff: shared infrastructure reduces cost and operational overhead, but increases blast radius and isolation risk. Vector workloads amplify this tradeoff because index structures are not row-addressable — an HNSW index built over millions of rows cannot be surgically partitioned the way a B-tree on a primary key can.

Row-Level Isolation puts all tenants in a single table with a `tenant_id` column. It is the simplest model and maps directly to how most DBAs handle multi-tenancy in OLTP systems. The problem is that pgvector's approximate nearest neighbor (ANN) indexes — HNSW and IVFFlat — are built over the entire table. When you query with a `WHERE tenant_id = $1` filter, the planner has two options: scan the index and post-filter, or skip the index and do a sequential scan. For a tenant with 1,000 rows in a table of 10 million, neither option is fast. The index scan produces thousands of candidates before the filter reduces them to the tenant's actual result set. The sequential scan is simply slow. This is the “tenant size disparity” problem, and it is the primary reason row-level isolation fails at scale.

Schema-Level Isolation gives each tenant their own schema within a single PostgreSQL database. Each schema contains its own `embeddings` table and its own HNSW index. This sidesteps the index contamination problem entirely — each tenant's index is built only over their own data. The

cost is operational complexity: adding a tenant means executing DDL, managing schema-scoped connections, and ensuring your migration tooling handles hundreds or thousands of schemas without timing out. At roughly 500–1,000 schemas, operational overhead becomes a real concern; beyond 5,000 schemas, PostgreSQL’s catalog can become a performance bottleneck during connection and query planning.

Database-Level Isolation gives each tenant a separate PostgreSQL database. This provides the strongest isolation — separate connection pools, separate WAL streams, separate backup targets — but multiplies your operational surface area by the number of tenants. It’s appropriate for enterprise SaaS where each customer is a large organization with their own compliance requirements, not for consumer applications with thousands of small tenants.

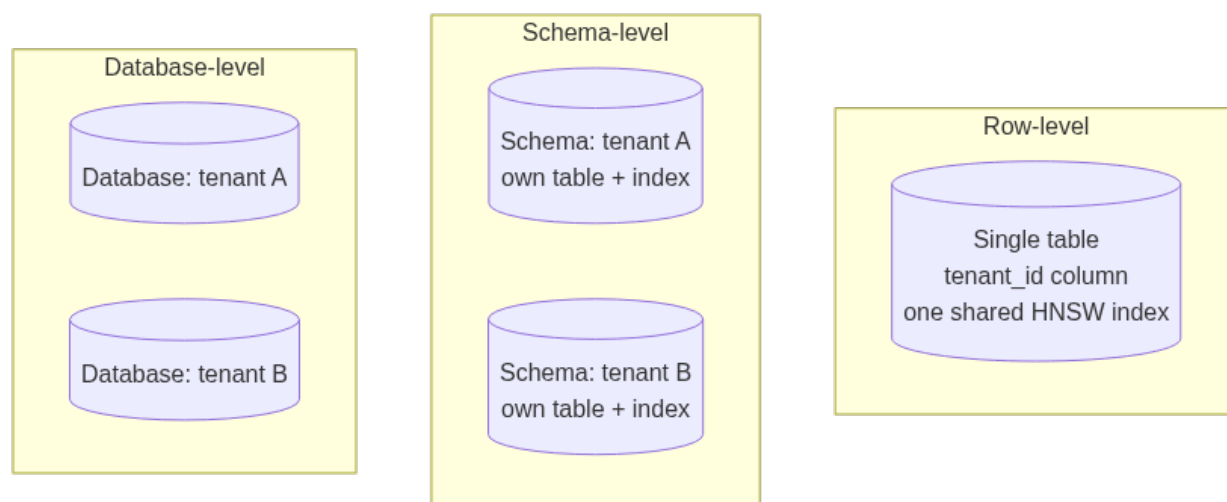


Figure 9: Isolation strength increases left to right; so does per-tenant operational overhead.

The decision matrix looks like this in practice:

Model	Tenants	Tenant Size	Isolation Requirement	Recommended
Row-level	< 100	Roughly equal	Low	Prototyping, small SaaS
Schema-level	100–5,000	Variable	Medium	Most production SaaS
Database-level	< 500	Large	High (SOC 2, HIPAA)	Enterprise, regulated

Most production systems end up in the schema-level model. The rest of this chapter focuses there, with coverage of row-level patterns where they remain applicable.

12.2 Implementing Row-Level Vector Isolation

Row-level isolation is worth understanding thoroughly, even if you'll eventually outgrow it, because it's the right starting point for most systems and the performance problems are instructive.

The baseline schema for a row-level multi-tenant vector store looks like this:

```
-- PostgreSQL / pgvector
CREATE EXTENSION IF NOT EXISTS vector;

-- Core document store with tenant partitioning
CREATE TABLE tenant_embeddings (
    id          BIGSERIAL PRIMARY KEY,
    tenant_id   UUID NOT NULL,
    doc_id      TEXT NOT NULL,
    content     TEXT NOT NULL,
    embedding   vector(1536),           -- OpenAI text-embedding-3-small dimensions
    metadata    JSONB DEFAULT '{}',
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE (tenant_id, doc_id)
);

-- Composite index for filtering: tenant_id first, then vector search
-- Note: this is a regular B-tree index, not a vector index
CREATE INDEX idx_tenant_embeddings_tenant
    ON tenant_embeddings (tenant_id, created_at DESC);

-- HNSW index over the full table
-- m=16, ef_construction=64 are conservative defaults for mixed workloads
CREATE INDEX idx_tenant_embeddings_hnsw
    ON tenant_embeddings
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Row-level security for application-layer enforcement
ALTER TABLE tenant_embeddings ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON tenant_embeddings
    USING (tenant_id = current_setting('app.current_tenant_id')::UUID);
```

The RLS policy requires the application to set `app.current_tenant_id` before executing queries. In Python using `psycopg2`:

```
# Python
import psycopg2
import numpy as np
from openai import OpenAI

openai_client = OpenAI()

def get_connection(dsn: str):
    conn = psycopg2.connect(dsn)
    conn.autocommit = False
    return conn

def set_tenant_context(conn, tenant_id: str):
    """Set the RLS context for the current connection."""
    with conn.cursor() as cur:
        cur.execute(
            "SELECT set_config('app.current_tenant_id', %s, FALSE)",
            (tenant_id,)
        )

def embed_text(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
```

```

return response.data[0].embedding

def insert_document(conn, tenant_id: str, doc_id: str, content: str, metadata: dict):
    embedding = embed_text(content)
    embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"

    with conn.cursor() as cur:
        set_tenant_context(conn, tenant_id)
        cur.execute("""
            INSERT INTO tenant_embeddings (tenant_id, doc_id, content, embedding, metadata)
            VALUES (%s, %s, %s, %s::vector, %s)
            ON CONFLICT (tenant_id, doc_id) DO UPDATE
                SET content = EXCLUDED.content,
                    embedding = EXCLUDED.embedding,
                    metadata = EXCLUDED.metadata
            """, (tenant_id, doc_id, content, embedding_str, psycopg2.extras.Json(metadata)))
        conn.commit()

def semantic_search(conn, tenant_id: str, query: str, top_k: int = 10):
    """
    ANN search scoped to a single tenant.
    The WHERE clause on tenant_id will post-filter HNSW results.
    For small tenants in large tables, set_limit may need adjustment.
    """
    query_embedding = embed_text(query)
    embedding_str = "[" + ",".join(str(x) for x in query_embedding) + "]"

    with conn.cursor() as cur:
        set_tenant_context(conn, tenant_id)
        # hnswef_search controls recall vs speed tradeoff at query time
        cur.execute("SET hnswef_search = 100")
        cur.execute("""
            SELECT
                doc_id,
                content,
                metadata,
                1 - (embedding <=> %s::vector) AS cosine_similarity
            FROM tenant_embeddings
            WHERE tenant_id = %s
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """, (embedding_str, tenant_id, embedding_str, top_k))
        return cur.fetchall()

```

One specific behavior to understand: when pgvector executes an HNSW search with a `WHERE tenant_id = $1` predicate, it scans the HNSW graph and collects `ef_search` candidates, then applies the filter. If your tenant has 500 rows in a table of 2 million, the probability that any given HNSW candidate passes the filter is 0.025%. The engine may return fewer than `top_k` results because so few candidates pass. Increasing `hnswef_search` to 400 or 800 improves recall but increases latency proportionally. This is the wall you hit with row-level isolation.

The mitigation, short of switching to schema-level isolation, is partial indexes. If tenant access patterns are known ahead of time, you can create a dedicated HNSW index per tenant:

```

-- PostgreSQL / pgvector
-- Partial HNSW index for a specific high-volume tenant
-- Only viable for tenants with > ~50,000 rows (index overhead is otherwise not justified)
CREATE INDEX idx_embeddings_tenant_acme
ON tenant_embeddings
USING hnswef (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
WHERE tenant_id = 'a1b2c3d4-e5f6-7890-abcd-ef1234567890';

-- Check that the planner uses the partial index
EXPLAIN (ANALYZE, BUFFERS)
SELECT doc_id, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM tenant_embeddings

```

```
WHERE tenant_id = 'a1b2c3d4-e5f6-7890-abcd-ef1234567890'
ORDER BY embedding <=> '[0.1, 0.2, ...]':::vector
LIMIT 10;
```

Partial indexes work well for your top 10–20 largest tenants. Beyond that, the schema-level model is a cleaner solution.

12.3 Schema-Per-Tenant Architecture

Schema-level isolation is where most production vector SaaS platforms land. Each tenant gets a dedicated schema, and within that schema you have full DDL freedom: indexes tuned to that tenant’s vector count, retention policies, custom metadata columns.

The first engineering problem is provisioning automation. You need to create schemas, tables, and indexes reliably, idempotently, and fast enough that a new tenant can be onboarded within seconds.

```
# Python
import psycopg2
from psycopg2 import sql
import logging

logger = logging.getLogger(__name__)

TENANT_SCHEMA_DDL = """
    CREATE SCHEMA IF NOT EXISTS {schema};

    CREATE TABLE IF NOT EXISTS {schema}.embeddings (
        id          BIGSERIAL PRIMARY KEY,
        doc_id       TEXT NOT NULL UNIQUE,
        content      TEXT NOT NULL,
        embedding     vector(1536),
        metadata     JSONB DEFAULT '{}',
        created_at   TIMESTAMPTZ DEFAULT NOW()
    );

    -- IVFFlat is preferred at smaller scales (< 1M rows per tenant)
    -- because it has lower memory overhead than HNSW and builds faster.
    -- Switch to HNSW when tenant exceeds ~500k rows.
    CREATE INDEX IF NOT EXISTS embeddings_ivfflat_{tenant_slug}
        ON {schema}.embeddings
        USING ivfflat (embedding vector_cosine_ops)
        WITH (lists = 100);

    COMMENT ON SCHEMA {schema} IS 'Tenant: {tenant_id}';
"""

def sanitize_schema_name(tenant_id: str) -> str:
    """
    Convert a tenant UUID to a safe schema name.
    PostgreSQL identifiers must start with a letter or underscore.
    """
    safe = tenant_id.replace("-", "_")
    return f"t_{safe}"

def provision_tenant_schema(conn, tenant_id: str) -> str:
    """
    Create schema and supporting objects for a new tenant.
    Returns the schema name on success.
    Idempotent - safe to call multiple times.
    """
    schema_name = sanitize_schema_name(tenant_id)

    try:
```

```

with conn.cursor() as cur:
    # Use psycopg2.sql for safe identifier interpolation
    # String formatting is acceptable here only for the DDL block
    # where we've already sanitized the identifier above.
    ddl = TENANT_SCHEMA_DDL.format(
        schema=schema_name,
        tenant_id=tenant_id,
        tenant_slug=schema_name
    )
    cur.execute(ddl)
conn.commit()
logger.info(f"Provisioned schema {schema_name} for tenant {tenant_id}")
return schema_name
except Exception as e:
    conn.rollback()
    logger.error(f"Failed to provision schema for tenant {tenant_id}: {e}")
    raise

def deprovision_tenant_schema(conn, tenant_id: str, dry_run: bool = True) -> str:
    """
    Remove all data for a tenant. Irreversible.
    Always run with dry_run=True first in production.
    """
    schema_name = sanitize_schema_name(tenant_id)
    drop_stmt = sql.SQL("DROP SCHEMA IF EXISTS {} CASCADE").format(
        sql.Identifier(schema_name)
    )

    if dry_run:
        logger.info(f"[DRY RUN] Would execute: {drop_stmt.as_string(conn)}")
        return schema_name

    with conn.cursor() as cur:
        cur.execute(drop_stmt)
    conn.commit()
    logger.info(f"Deprovisioned schema {schema_name}")
    return schema_name

```

With schemas provisioned, the query path needs to route each request to the correct schema. Connection pools complicate this: a connection checked out from PgBouncer has no inherent tenant context, so you must set `search_path` explicitly on every query or use session-scoped configuration.

```

# Python
from contextlib import contextmanager
import psycopg2.pool

# Transaction-mode pooling (PgBouncer) requires setting search_path per query.
# Session-mode pooling allows SET search_path once per connection checkout,
# but connections cannot be reused across tenants without resetting.

class TenantVectorStore:
    def __init__(self, pool: psycopg2.pool.ThreadedConnectionPool):
        self.pool = pool

    @contextmanager
    def tenant_connection(self, tenant_id: str):
        """
        Context manager that yields a connection scoped to the tenant schema.
        Resets search_path on release to prevent cross-tenant leakage.
        """
        schema_name = sanitize_schema_name(tenant_id)
        conn = self.pool.getconn()
        try:
            with conn.cursor() as cur:
                # Set search_path for this transaction block.
                # The 'public' fallback gives access to pgvector functions.
                cur.execute(
                    sql.SQL("SET LOCAL search_path TO {}, public").format(
                        sql.Identifier(schema_name)
                    )
                )

```

```
        )
    )
    yield conn
finally:
    conn.reset()      # Clears SET LOCAL changes
    self.pool.putconn(conn)

def search(self, tenant_id: str, query_embedding: list[float], top_k: int = 10):
    embedding_str = "[" + ",".join(str(x) for x in query_embedding) + "]"

    with self.tenant_connection(tenant_id) as conn:
        with conn.cursor() as cur:
            cur.execute("""
                SELECT
                    doc_id,
                    content,
                    metadata,
                    1 - (embedding <=> %s::vector) AS score
                FROM embeddings
                ORDER BY embedding <=> %s::vector
                LIMIT %s
            """, (embedding_str, embedding_str, top_k))
            return cur.fetchall()

def upsert(self, tenant_id: str, doc_id: str, content: str,
            embedding: list[float], metadata: dict):
    embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"

    with self.tenant_connection(tenant_id) as conn:
        with conn.cursor() as cur:
            cur.execute("""
                INSERT INTO embeddings (doc_id, content, embedding, metadata)
                VALUES (%s, %s, %s::vector, %s)
                ON CONFLICT (doc_id) DO UPDATE
                SET content = EXCLUDED.content,
                    embedding = EXCLUDED.embedding,
                    metadata = EXCLUDED.metadata
            """, (doc_id, content, embedding_str,
                  psycopg2.extras.Json(metadata)))
        conn.commit()
```

The `SET LOCAL search_path` approach is important: `SET LOCAL` applies only within the current transaction and is automatically rolled back when the transaction ends or the connection is reset. This prevents a schema from bleeding across connection pool boundaries.

12.4 Index Management at Tenant Scale

The most underestimated operational challenge in schema-per-tenant architectures is index lifecycle management. When you have 1,000 tenants, each with an IVFFlat or HNSW index, you have 1,000 separate index structures that need to be monitored, rebuilt when stale, and upgraded when you change embedding models.

When to switch index types: IVFFlat is appropriate up to approximately 500,000 vectors per tenant. The rule of thumb is `lists = sqrt(row_count)`, so a 100,000-row tenant should have `lists = ~316`. Beyond 500,000 rows, HNSW provides better recall at comparable query latency because IVFFlat's probe-based search degrades when the query vector's nearest list doesn't align well with its actual neighbors.

Rebuilding indexes after bulk loads: Both IVFFlat and HNSW are static structures that don't automatically rebalance. After a bulk import that increases a tenant's row count by more than

20–30%, the index becomes stale and recall drops. You need a process to detect this and schedule rebuilds.

```
-- PostgreSQL / pgvector
-- Monitoring view: index staleness across all tenant schemas
-- Run this periodically to identify schemas needing index maintenance

CREATE OR REPLACE VIEW public.tenant_index_health AS
SELECT
    schemaname,
    tablename,
    n_live_tup                AS live_rows,
    n_dead_tup               AS dead_rows,
    last_vacuum,
    last_analyze,
    ROUND(n_dead_tup::NUMERIC / NULLIF(n_live_tup, 0) * 100, 2) AS dead_pct,
    pg_size_pretty(pg_total_relation_size(
        schemaname || '.' || tablename
    ))                      AS total_size
FROM pg_stat_user_tables
WHERE tablename = 'embeddings'
    AND schemaname LIKE 't_%' -- matches our tenant schema naming convention
ORDER BY live_rows DESC;

-- Find tenants whose index needs rebuilding:
-- Heuristic: more than 25% dead tuples, or live rows have grown > 2x since last analyze
SELECT
    schemaname,
    live_rows,
    dead_pct,
    last_analyze,
    CASE
        WHEN dead_pct > 25                                THEN 'REBUILD_URGENT'
        WHEN last_analyze < NOW() - INTERVAL '7 days'
            AND live_rows > 10000                          THEN 'REBUILD_SCHEDULED'
        ELSE 'OK'
    END AS index_status
FROM public.tenant_index_health
WHERE dead_pct > 10
    OR last_analyze < NOW() - INTERVAL '7 days'
ORDER BY dead_pct DESC, live_rows DESC;
```

When a tenant needs an index rebuild, you want to do it without locking out queries. `REINDEX CONCURRENTLY` is your friend, but it requires PostgreSQL 12+ and cannot run inside a transaction block:

```
# shell
# Rebuild HNSW index concurrently for a specific tenant schema
# Run as the application database user with sufficient privileges
# This takes 2-10 minutes for a 500k-row tenant on typical cloud hardware

TENANT_SCHEMA="t_a1b2c3d4_e5f6_7890_abcd_ef1234567890"

psql "$DATABASE_URL" -c "
    REINDEX INDEX CONCURRENTLY
    ${TENANT_SCHEMA}.embeddings_ivfflat_${TENANT_SCHEMA};
"

# After reindex, update statistics so the planner has accurate cost estimates
psql "$DATABASE_URL" -c "
    ANALYZE ${TENANT_SCHEMA}.embeddings;
"

# Verify the index is healthy
psql "$DATABASE_URL" -c "
    SELECT
        indexname,
        pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size,
        idx_scan,
```

```

        idx_tup_read,
        idx_tup_fetch
    FROM pg_stat_user_indexes
    WHERE schemaname = '${TENANT_SCHEMA}'
        AND tablename = 'embeddings';

```

For automated index lifecycle management at scale, wrap this in a Python job that reads from the `tenant_index_health` view and prioritizes rebuilds:

```

# Python
import psycopg2
from psycopg2 import sql
import logging
from dataclasses import dataclass
from datetime import datetime, timedelta

logger = logging.getLogger(__name__)

@dataclass
class TenantIndexStatus:
    schema_name: str
    live_rows: int
    dead_pct: float
    last_analyze: datetime | None
    status: str

def get_stale_tenant_indexes(conn) -> list[TenantIndexStatus]:
    """Return tenants requiring index maintenance, ordered by urgency."""
    with conn.cursor() as cur:
        cur.execute("""
            SELECT
                schemaname,
                live_rows,
                dead_pct,
                last_analyze,
                CASE
                    WHEN dead_pct > 25 THEN 'REBUILD_URGENT'
                    WHEN last_analyze < NOW() - INTERVAL '7 days'
                        AND live_rows > 10000 THEN 'REBUILD_SCHEDULED'
                    ELSE 'OK'
                END AS status
            FROM public.tenant_index_health
            WHERE dead_pct > 10
                OR last_analyze < NOW() - INTERVAL '7 days'
            ORDER BY
                CASE WHEN dead_pct > 25 THEN 0 ELSE 1 END,
                dead_pct DESC
        """)
        rows = cur.fetchall()

    return [
        TenantIndexStatus(
            schema_name=r[0],
            live_rows=r[1],
            dead_pct=float(r[2] or 0),
            last_analyze=r[3],
            status=r[4]
        )
        for r in rows
        if r[4] != 'OK'
    ]

def rebuild_tenant_index(conn, schema_name: str) -> bool:
    """
    Rebuild the vector index for a tenant schema.
    Returns True on success, False on failure.
    REINDEX CONCURRENTLY cannot run inside a transaction - use autocommit.
    """

```

```

original_autocommit = conn.autocommit
conn.autocommit = True

try:
    index_name = f"embeddings_ivfflat_{schema_name}"
    full_index = f"{schema_name}.{index_name}"

    with conn.cursor() as cur:
        logger.info(f"Starting concurrent reindex for {schema_name}")
        cur.execute(
            sql.SQL("REINDEX INDEXCONCURRENTLY {}").format(
                sql.Identifier(schema_name, index_name)
            )
        )
        cur.execute(
            sql.SQL("ANALYZE {}.embeddings").format(
                sql.Identifier(schema_name)
            )
        )

    logger.info(f"Reindex complete for {schema_name}")
    return True

except Exception as e:
    logger.error(f"Reindex failed for {schema_name}: {e}")
    return False

finally:
    conn.autocommit = original_autocommit

def run_index_maintenance(dsn: str, max_rebuilds_per_run: int = 10):
    """
    Top-level maintenance job. Call from a cron job or scheduled task.
    Processes the most urgent tenants first, capped at max_rebuilds_per_run
    to bound wall-clock time.
    """
    conn = psycopg2.connect(dsn)
    try:
        stale = get_stale_tenant_indexes(conn)
        logger.info(f"Found {len(stale)} tenants needing index maintenance")

        rebuilt = 0
        for tenant in stale[:max_rebuilds_per_run]:
            success = rebuild_tenant_index(conn, tenant.schema_name)
            if success:
                rebuilt += 1
            else:
                logger.warning(
                    f"Skipping remaining tenants after failure at {tenant.schema_name}"
                )
                break

        logger.info(f"Index maintenance complete: {rebuilt}/{len(stale)} tenants rebuilt")
    finally:
        conn.close()

```

Changing embedding models: When you migrate from one embedding model to another (e.g., from text-embedding-ada-002 at 1536 dimensions to a newer model at a different dimensionality), the vector column type changes. This is a disruptive migration. The safest approach is a shadow column strategy:

```

-- PostgreSQL / pgvector
-- Phase 1: Add new embedding column alongside the old one
ALTER TABLE {schema}.embeddings
    ADD COLUMN embedding_v2 vector(3072); -- new model dimensionality

-- Phase 2: Backfill new embeddings in batches (done by your migration job)
-- Phase 3: Create new index on embedding_v2

```



```
CREATE INDEX CONCURRENTLY embeddings_hnsw_v2_{schema}
ON {schema}.embeddings
USING hnsw (embedding_v2 vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Phase 4: Once backfill complete, swap queries to use embedding_v2
-- Phase 5: Drop old column and old index
-- ALTER TABLE {schema}.embeddings DROP COLUMN embedding;
```

The shadow column approach lets you serve queries from the old index while the new one is being built, with zero downtime.

12.5 Cross-Tenant Query Routing

Most SaaS applications need to perform cross-tenant operations at some point: admin dashboards that aggregate stats across all tenants, internal search that spans the entire document corpus, or anomaly detection that compares embedding distributions across customers.

Cross-tenant queries in a schema-per-tenant system require either dynamic SQL that unions across schemas or a separate denormalized table. The dynamic SQL approach is more flexible but has query planning overhead for large tenant counts.

```
# Python
def cross_tenant_search(
    conn,
    query_embedding: list[float],
    tenant_ids: list[str],
    top_k_per_tenant: int = 5,
    final_top_k: int = 20
) -> list[dict]:
    """
    Federated vector search across multiple tenants.
    Executes one query per tenant schema and merges results client-side.

    For admin/internal use only - never expose this to tenant-scoped
    application paths. Performance scales linearly with tenant count.
    """
    embedding_str = "[" + ",".join(str(x) for x in query_embedding) + "]"
    results = []

    with conn.cursor() as cur:
        for tenant_id in tenant_ids:
            schema_name = sanitize_schema_name(tenant_id)
            try:
                cur.execute(
                    sql.SQL("""
                        SELECT
                            %s AS tenant_id,
                            doc_id,
                            content,
                            1 - (embedding <=> %s::vector) AS score
                        FROM {schema}.embeddings
                        ORDER BY embedding <=> %s::vector
                        LIMIT %s
                    """).format(schema=sql.Identifier(schema_name)),
                    (tenant_id, embedding_str, embedding_str, top_k_per_tenant)
                )
                rows = cur.fetchall()
                results.extend({
                    "tenant_id": r[0],
                    "doc_id": r[1],
                    "content": r[2],
                    "score": float(r[3])
                })
```

```
    } for r in rows)
except psycopg2.errors.InvalidSchemaName:
    logger.warning(f"Schema not found for tenant {tenant_id}, skipping")
    continue

# Merge and re-rank by score
results.sort(key=lambda x: x["score"], reverse=True)
return results[:final_top_k]
```

For larger cross-tenant operations, a materialized summary table avoids the per-schema fan-out:

```
-- PostgreSQL / pgvector
-- Materialized cross-tenant view for admin analytics
-- Populated by a background job, not a live view
CREATE TABLE public.cross_tenant_embeddings_summary (
    tenant_id      UUID NOT NULL,
    schema_name    TEXT NOT NULL,
    doc_id         TEXT NOT NULL,
    embedding       vector(1536),
    created_at     TIMESTAMPTZ,
    synced_at      TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (tenant_id, doc_id)
);

CREATE INDEX idx_cross_tenant_hnsw
ON public.cross_tenant_embeddings_summary
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Query across all tenants with a single vector search
SELECT
    tenant_id,
    doc_id,
    1 - (embedding <=> '[0.1, 0.2, ...] '::vector) AS score
FROM public.cross_tenant_embeddings_summary
ORDER BY embedding <=> '[0.1, 0.2, ...] '::vector
LIMIT 20;
```

The summary table trades freshness for query speed. Your sync job needs to handle inserts, updates, and deletes from all tenant schemas, which is non-trivial at scale. Logical replication or a change-data-capture pipeline (e.g., Debezium) is the production-grade approach.

12.6 Tenant-Aware Connection Pooling

Connection management is where schema-per-tenant architectures most commonly fail in production. The core tension: PgBouncer in transaction-mode pooling (the recommended mode for high-concurrency applications) does not support `SET search_path` persisting across transactions because connections are returned to the pool after each transaction. Session-mode pooling preserves settings but reduces effective concurrency because a connection is pinned to one application session.

The practical solution is to use `SET LOCAL` within a transaction, or to explicitly set `search_path` as part of every query batch. The code pattern in Section 12.3 already does this. Here's the infrastructure configuration to pair with it:

```
# pgbouncer.ini
[pgbouncer]
pool_mode = transaction
max_client_conn = 5000
default_pool_size = 25
min_pool_size = 5
reserve_pool_size = 5
```

```
reserve_pool_timeout = 3
server_reset_query = DISCARD ALL
server_check_query = SELECT 1
server_check_delay = 30
ignore_startup_parameters = extra_float_digits

# Application database
[myapp_db]
host = 127.0.0.1
port = 5432
dbname = myapp
pool_size = 25
```

The `server_reset_query = DISCARD ALL` ensures that any session-level settings left by previous transactions are cleared when a server connection is returned to the pool. This is your safety net against schema leakage.

On the application side, always pair your schema-setting code with explicit transaction management:

```
# Python
# Correct pattern: SET LOCAL inside explicit transaction
# SET LOCAL is reverted automatically at transaction end
def safe_tenant_query(pool, tenant_id: str, query_sql: str, params: tuple):
    schema_name = sanitize_schema_name(tenant_id)
    conn = pool.getconn()
    try:
        conn.autocommit = False
        with conn.cursor() as cur:
            cur.execute(
                sql.SQL("SET LOCAL search_path TO {}, public").format(
                    sql.Identifier(schema_name)
                )
            )
            cur.execute(query_sql, params)
            result = cur.fetchall()
            conn.commit()
        return result
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.autocommit = True
        pool.putconn(conn)
```

For async workloads using `asyncpg` or `SQLAlchemy async`:

```
# Python (asyncpg)
import asyncpg

async def async_tenant_search(
    pool: asyncpg.Pool,
    tenant_id: str,
    query_embedding: list[float],
    top_k: int = 10
) -> list[asyncpg.Record]:
    schema_name = sanitize_schema_name(tenant_id)
    embedding_str = "[" + ",".join(str(x) for x in query_embedding) + "]"

    async with pool.acquire() as conn:
        async with conn.transaction():
            await conn.execute(
                f"SET LOCAL search_path TO {asyncpg.utils._quote_ident(schema_name)}, public"
            )
            return await conn.fetch("""
                SELECT
                    doc_id,
                    content,
```

```
        1 - (embedding <=> $1::vector) AS score
FROM embeddings
ORDER BY embedding <=> $1::vector
LIMIT $2
"""', embedding_str, top_k)
```

Note: `asyncpg`'s `_quote_ident` is an internal utility. In production, implement your own identifier quoting or use `f'"{schema_name}"'` after validating the schema name against your known-safe naming scheme.

12.7 Scaling Considerations and Performance Cliffs

As tenant count and per-tenant data volume grow, several performance cliffs appear that aren't obvious from small-scale testing.

PostgreSQL catalog contention: The `pg_class`, `pg_attribute`, and `pg_index` system catalogs are shared across the entire instance. When you have 3,000 tenant schemas each with 10 tables and 5 indexes, you have 15,000 entries in `pg_class` and 75,000 in `pg_index`. Connection startup and query planning read these catalogs, and the catalog can become a hot read path at scale. Monitor with:

```
-- PostgreSQL
-- Catalog table bloat and access frequency
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    seq_scan,
    idx_scan
FROM pg_stat_sys_tables
WHERE relname IN ('pg_class', 'pg_attribute', 'pg_index', 'pg_namespace')
ORDER BY seq_scan DESC;
```

If you see high `seq_scan` counts on `pg_class`, you're hitting catalog contention. The mitigation is reducing schema count (consolidate small tenants to row-level partitioning) or sharding across PostgreSQL instances.

Memory pressure from concurrent index builds: HNSW index construction is memory-intensive. Each concurrent `CREATE INDEX` operation can consume `maintenance_work_mem`. With 50 tenants being provisioned simultaneously, you can exhaust available memory on the database server. Serialize index builds or cap concurrent DDL operations:

```
# Python
import asyncio

async def provision_tenants_with_concurrency_limit(
    tenant_ids: list[str],
    dsn: str,
    max_concurrent: int = 5
):
    """Provision tenant schemas with bounded DDL concurrency."""
    semaphore = asyncio.Semaphore(max_concurrent)

    async def provision_one(tenant_id: str):
        async with semaphore:
            # Each task gets its own connection to avoid transaction interference
            conn = await asyncpg.connect(dsn)
            try:
                await async_provision_tenant(conn, tenant_id)
```

```
        finally:
            await conn.close()

    await asyncio.gather(*[provision_one(tid) for tid in tenant_ids])
```

IVFFlat lists misconfiguration: The `lists` parameter must be set when the index is created and cannot be changed without dropping and recreating the index. A tenant provisioned with `lists = 100` when it had 10,000 rows will have severely degraded recall once it grows to 1,000,000 rows (where `lists = ~1000` would be appropriate). The automated index health check from Section 12.4 should flag this. Add a check:

```
-- PostgreSQL / pgvector
-- Detect tenants whose IVFFlat list count is under-provisioned
-- relative to current row count
SELECT
    h.schemaname,
    h.live_rows,
    (ix.reloptions::text) AS index_options,
    ROUND(SQRT(h.live_rows)) AS recommended_lists,
    -- Extract the actual lists value from index options
    (
        SELECT val::int
        FROM unnest(ix.reloptions) AS opt(val_text)
        CROSS JOIN LATERAL (SELECT val_text) AS t(val)
        WHERE val LIKE 'lists=%'
        LIMIT 1
    ) AS current_lists
FROM public.tenant_index_health h
JOIN pg_indexes pi
    ON pi.schemaname = h.schemaname
   AND pi.tablename = 'embeddings'
   AND pi.indexdef LIKE '%ivfflat%'
JOIN pg_class ix
    ON ix.relname = pi.indexname
WHERE h.live_rows > 0
ORDER BY h.live_rows DESC;
```

Vacuum and autovacuum across many schemas: pgvector tables accumulate dead tuples like any other heap. With 1,000 schemas each doing moderate write activity, autovacuum workers can be spread thin. Increase `autovacuum_max_workers` and tune per-table autovacuum thresholds for high-churn tenant tables:

```
-- PostgreSQL
-- Aggressive autovacuum settings for high-write tenant embedding tables
-- Apply after provisioning for tenants with expected high write volume
ALTER TABLE {schema}.embeddings SET (
    autovacuum_vacuum_scale_factor = 0.01, -- Vacuum after 1% of rows are dead
    autovacuum_analyze_scale_factor = 0.005, -- Analyze after 0.5% of rows change
    autovacuum_vacuum_cost_delay = 2 -- ms; reduce I/O throttling for these tables
);
```

12.8 A Complete Working Example

To tie together the patterns from this chapter, here is a minimal but production-representative multi-tenant vector store with FastAPI, asyncpg, and pgvector:

```
# Python
# main.py - FastAPI multi-tenant vector store
from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel
import asyncpg
import os
```

```

import logging
from contextlib import asynccontextmanager
from openai import AsyncOpenAI

logger = logging.getLogger(__name__)
openai_client = AsyncOpenAI()

# -----
# Application lifecycle
# -----

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.pool = await asyncpg.create_pool(
        dsn=os.environ["DATABASE_URL"],
        min_size=5,
        max_size=20,
        command_timeout=30
    )
    yield
    await app.state.pool.close()

app = FastAPI(lifespan=lifespan)

# -----
# Schema helpers (repeated from earlier sections, included for completeness)
# -----

def sanitize_schema_name(tenant_id: str) -> str:
    safe = tenant_id.replace("-", "_")
    return f"t_{safe}"

def quote_ident(name: str) -> str:
    """Minimal safe quoting for schema names we control."""
    if not all(c.isalnum() or c == '_' for c in name):
        raise ValueError(f"Unsafe identifier: {name}")
    return f'"{name}"'

PROVISION_DDL = """
CREATE SCHEMA IF NOT EXISTS {schema};
CREATE TABLE IF NOT EXISTS {schema}.embeddings (
    id          BIGSERIAL PRIMARY KEY,
    doc_id      TEXT NOT NULL UNIQUE,
    content     TEXT NOT NULL,
    embedding   vector(1536),
    metadata    JSONB DEFAULT '{}',
    created_at  TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS embeddings_hnsw
ON {schema}.embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
"""

async def ensure_tenant_schema(pool: asyncpg.Pool, tenant_id: str):
    schema = sanitize_schema_name(tenant_id)
    ddl = PROVISION_DDL.format(schema=quote_ident(schema))
    async with pool.acquire() as conn:
        await conn.execute(ddl)

# -----
# Embedding helper
# -----

async def get_embedding(text: str) -> list[float]:
    response = await openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return response.data[0].embedding

```

```

# -----
# API models
# -----

class UpsertRequest(BaseModel):
    doc_id: str
    content: str
    metadata: dict = {}

class SearchRequest(BaseModel):
    query: str
    top_k: int = 10

# -----
# Routes
# -----

@app.post("/tenants/{tenant_id}/provision")
async def provision_tenant(tenant_id: str):
    await ensure_tenant_schema(app.state.pool, tenant_id)
    return {"status": "ok", "tenant_id": tenant_id}

@app.post("/tenants/{tenant_id}/documents")
async def upsert_document(tenant_id: str, req: UpsertRequest):
    schema = sanitize_schema_name(tenant_id)
    schema_q = quote_ident(schema)
    embedding = await get_embedding(req.content)
    embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"

    async with app.state.pool.acquire() as conn:
        async with conn.transaction():
            await conn.execute(
                f"SET LOCAL search_path TO {schema_q}, public"
            )
            await conn.execute("""
                INSERT INTO embeddings (doc_id, content, embedding, metadata)
                VALUES ($1, $2, $3::vector, $4)
                ON CONFLICT (doc_id) DO UPDATE
                    SET content = EXCLUDED.content,
                        embedding = EXCLUDED.embedding,
                        metadata = EXCLUDED.metadata
            """, req.doc_id, req.content, embedding_str,
                req.metadata)

    return {"status": "ok", "doc_id": req.doc_id}

@app.post("/tenants/{tenant_id}/search")
async def search_documents(tenant_id: str, req: SearchRequest):
    schema = sanitize_schema_name(tenant_id)
    schema_q = quote_ident(schema)
    query_embedding = await get_embedding(req.query)
    embedding_str = "[" + ",".join(str(x) for x in query_embedding) + "]"

    async with app.state.pool.acquire() as conn:
        async with conn.transaction():
            await conn.execute(
                f"SET LOCAL search_path TO {schema_q}, public"
            )
            rows = await conn.fetch("""
                SELECT
                    doc_id,
                    content,
                    metadata,
                    1 - (embedding <=> $1::vector) AS score
                FROM embeddings
                ORDER BY embedding <=> $1::vector
                LIMIT $2
            """, embedding_str, req.top_k)

```

```
    return [
        {
            "doc_id": r["doc_id"],
            "content": r["content"],
            "metadata": dict(r["metadata"]),
            "score": float(r["score"])
        }
        for r in rows
    ]

@app.delete("/tenants/{tenant_id}")
async def deprovision_tenant(tenant_id: str):
    schema = sanitize_schema_name(tenant_id)
    schema_q = quote_ident(schema)
    async with app.state.pool.acquire() as conn:
        await conn.execute(f"DROP SCHEMA IF EXISTS {schema_q} CASCADE")
    return {"status": "ok", "tenant_id": tenant_id}
```

This example is intentionally minimal — it has no authentication, no rate limiting, and no observability instrumentation. In a real deployment you would add all three. The patterns for schema provisioning, `search_path` scoping, and connection management are production-ready as written.

Key Takeaways

- **Row-level isolation** is simple to implement but breaks down when tenant sizes diverge significantly, because HNSW and IVFFlat indexes scan the full table before applying tenant filters. Partial indexes per tenant extend row-level isolation but are operationally viable only for your largest few tenants.
 - **Schema-per-tenant isolation** is the production default for most SaaS vector workloads. Each tenant gets its own index built only over their data, eliminating the filter post-scan problem. The cost is DDL automation complexity and PostgreSQL catalog pressure above roughly 3,000–5,000 schemas.
 - **Index lifecycle management** requires active monitoring: IVFFlat `lists` parameters become under-provisioned as tenants grow, dead tuple accumulation degrades recall, and embedding model changes require a zero-downtime shadow-column migration strategy.
 - **Connection pool configuration** is as important as the data model. Use `SET LOCAL search_path` within explicit transactions to scope queries to tenant schemas, and configure `DISCARD ALL` as the PgBouncer server reset query to prevent cross-tenant schema leakage.
 - **Cross-tenant operations** — admin search, analytics, anomaly detection — require either a fan-out query that iterates tenant schemas or a separately maintained materialized summary table with its own HNSW index. The fan-out approach scales linearly with tenant count; the summary table trades freshness for constant-time queries.
-

Chapter 13: pgvector on RDS, Aurora, AlloyDB, and Supabase

Running pgvector in production rarely means spinning up a raw EC2 instance and compiling the extension by hand. Most teams arrive at vector search through the managed PostgreSQL services they already operate—Amazon RDS, Amazon Aurora, Google AlloyDB, or Supabase—and they need to know exactly what each platform delivers, what it costs, and where the walls are. This chapter maps pgvector’s behavior across all four environments: how to enable it, how to tune it, where each platform diverges from vanilla PostgreSQL, and how to architect a production embedding pipeline that survives traffic spikes, schema migrations, and the cost pressure that follows any successful AI feature launch.

13.1 Platform Landscape: What You’re Actually Getting

Before touching a single SQL statement, understand what managed pgvector actually means across these four providers.

Amazon RDS for PostgreSQL added pgvector support in October 2023 for PostgreSQL 15.x and later, and backported it to 14.x and 13.x shortly after. You get the standard pgvector extension installed by the engine—no custom builds, no OS-level access. The version you get is whatever RDS has packaged; you cannot upgrade the extension independently of the engine minor version. As of mid-2024, RDS ships pgvector 0.6.x on PostgreSQL 15 and 0.7.x on PostgreSQL 16. This matters because HNSW index support landed in pgvector 0.5.0, and the `halfvec` and `bit` types for memory-efficient storage arrived in 0.7.0.

Amazon Aurora PostgreSQL-Compatible shares the same extension packaging as RDS but adds Aurora-specific behavior: storage auto-scaling, the Aurora parallel query engine, and Global Database replication. Vector data lives in Aurora’s distributed storage layer, which means your HNSW index pages are stored across the distributed volume rather than local disk—a meaningful difference when reasoning about index build I/O and buffer pool pressure.

Google AlloyDB is Google’s PostgreSQL-compatible managed service with a disaggregated storage architecture and a built-in columnar engine. AlloyDB added pgvector support in 2023 and additionally offers `google_ml_integration`, which lets you call Vertex AI embedding models directly from SQL. The columnar engine can accelerate certain vector workload patterns—specifically full-table scans during exact KNN searches on small-to-medium datasets—in ways that standard PostgreSQL storage cannot.

Supabase is a developer-focused platform built on PostgreSQL with pgvector pre-enabled on every project. Supabase wraps the database with a REST API layer (PostgREST), a real-time subscription engine, and a JavaScript/Python SDK that integrates directly with vector operations. This makes Supabase the fastest path from zero to a working vector search endpoint, but the tradeoff is reduced control over low-level PostgreSQL configuration parameters that matter at scale—`work_mem`, `effective_cache_size`, and `maintenance_work_mem` are not directly tunable on the free and Pro tiers.

The following table summarizes the key operational differences:

Feature	RDS PostgreSQL	Aurora PostgreSQL	AlloyDB	Supabase
pgvector version control	Engine-tied	Engine-tied	Engine-tied	Engine-tied
HNSW support	(PG 15+)	(PG 15+)		
halfvec type	(PG 16+)	(PG 16+)		
Max dimensions	16,000	16,000	16,000	16,000
Direct ML integration			Vertex AI	
PostgREST API				
Custom GUC tuning	Full	Full	Partial	Limited
Read replicas for search		(serverless optional)		(paid tiers)

13.2 Enabling pgvector and Schema Design on Managed Services

Enabling the extension follows the same SQL command on all four platforms, but the prerequisites differ.

On **RDS** and **Aurora**, you need to be connected as the master user or a user with the `rds_superuser` role:

```
-- PostgreSQL / pgvector
-- Run as master user (e.g., postgres or your RDS master username)
CREATE EXTENSION IF NOT EXISTS vector;

-- Verify version
SELECT extversion FROM pg_extension WHERE extname = 'vector';
-- Returns: 0.7.0 (or whatever RDS has packaged)
```

On **AlloyDB**, the extension is pre-loaded in the shared library. Enable it the same way, but note that `google_ml_integration` must also be enabled to use the ML embedding functions:

```
-- PostgreSQL / pgvector (AlloyDB)
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS google_ml_integration;

-- Grant the AlloyDB service account access to Vertex AI
-- (Done in GCP IAM, not SQL - the AlloyDB instance's service account
-- needs roles/aiplatform.user on the project)
```

```
-- Test a Vertex AI embedding call directly from SQL
SELECT google_ml.embedding(
  'textembedding-gecko@003',
  'What is the boiling point of water?'
)::vector(768);
```

On **Supabase**, the extension is already enabled. Connect via the Supabase dashboard’s SQL editor or any PostgreSQL client using the connection string from your project settings:

```
-- PostgreSQL / pgvector (Supabase - extension already active)
-- Verify it's available
SELECT extversion FROM pg_extension WHERE extname = 'vector';

-- Create a table for document embeddings
CREATE TABLE documents (
  id          BIGSERIAL PRIMARY KEY,
  content     TEXT NOT NULL,
  metadata    JSONB,
  embedding   VECTOR(1536), -- OpenAI text-embedding-ada-002 dimensions
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Schema design decisions that matter at production scale:

The dimension count is fixed at column definition time. If you switch embedding models—which happens more often than teams expect—you need to add a new column or rebuild the table. Design for this from the start by keeping the embedding model name in metadata:

```
-- PostgreSQL / pgvector
CREATE TABLE document_embeddings (
  id          BIGSERIAL PRIMARY KEY,
  document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  model_name   TEXT NOT NULL,          -- 'text-embedding-ada-002', 'embed-english-v3.0', etc.
  model_version TEXT NOT NULL,         -- track model versions explicitly
  embedding   VECTOR(1536),
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (document_id, model_name, model_version)
);

-- Partial index to serve only the current production model
-- without filtering overhead on every query
CREATE INDEX ON document_embeddings (model_name, model_version)
  WHERE model_name = 'text-embedding-ada-002' AND model_version = '002';
```

For large tables where you want memory efficiency, use **halfvec** (available in pgvector 0.7.0+) to halve the storage footprint at the cost of marginally reduced precision—typically less than 0.5% recall loss on standard benchmarks:

```
-- PostgreSQL / pgvector
-- halfvec uses 2 bytes per dimension vs 4 bytes for vector
ALTER TABLE document_embeddings
  ADD COLUMN embedding_half halfvec(1536);

-- Cast from full precision if migrating
UPDATE document_embeddings
  SET embedding_half = embedding::halfvec(1536)
  WHERE model_name = 'text-embedding-ada-002';
```

13.3 Index Strategy: HNSW vs. IVFFlat on Managed Infrastructure

Managed PostgreSQL adds constraints that change the optimal index choice compared to self-hosted deployments. You cannot tune kernel memory parameters, cannot pin HNSW graphs in OS-level hugepages, and you’re competing with the rest of your workload for shared buffer space.

IVFFlat builds faster and consumes less memory during construction, but requires a training step (clustering) that scans the entire table. On Aurora, this clustering scan goes through the distributed storage layer and generates significant I/O that competes with OLTP traffic. On a 10M-row embeddings table, an IVFFlat build with `lists = 1000` can run 2–4 hours on a db.r6g.2xlarge and consume the majority of I/O throughput during that window.

HNSW builds more slowly in wall-clock time and requires substantial `maintenance_work_mem` during construction (plan for 4–8 GB for a million 1536-dimensional vectors), but it requires no training data and delivers better recall at equivalent query latency. It’s the right default for production on managed services unless you’re constrained on build-time memory.

```
-- PostgreSQL / pgvector
-- HNSW index for cosine similarity (typical for OpenAI and Cohere embeddings)
-- Set maintenance_work_mem high before building
SET maintenance_work_mem = '8GB';

CREATE INDEX CONCURRENTLY idx_embeddings_hnsw
ON document_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 128);

-- m = 16: connections per node in the graph (higher = better recall, more memory)
-- ef_construction = 128: candidate list size during build
-- (higher = better recall, slower build)

-- For query time, set ef_search on the session or transaction:
SET hnsw.ef_search = 100; -- default is 40; increase for better recall at cost of latency

-- Verify index is being used
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, content, 1 - (embedding <=> '[0.1, 0.2, ...]::vector') AS similarity
FROM document_embeddings
WHERE model_name = 'text-embedding-ada-002'
ORDER BY embedding <=> '[0.1, 0.2, ...]::vector'
LIMIT 10;
```

On **RDS Multi-AZ** and **Aurora**, `CREATE INDEX CONCURRENTLY` is strongly preferred over a blocking index build—the index can be built without holding an exclusive lock on the table, which matters when your embeddings table is being written to by an ingestion pipeline. The concurrency trade-off is a longer build time (typically 20–40% more wall clock) and the requirement that no transaction runs for the duration of the build.

AlloyDB columnar engine deserves a specific note here. For exact KNN search (no approximate index, full table scan with `ORDER BY distance LIMIT k`), AlloyDB’s columnar engine can execute the distance computation in vectorized batches that outperform row-by-row execution by 3–10x on tables up to roughly 500K rows. Beyond that threshold, approximate index search with HNSW will consistently beat even the columnar engine, so treat the columnar path as an optimization for smaller “hot” tables, not a replacement for index design:

```
-- PostgreSQL / pgvector (AlloyDB specific)
-- Enable columnar scan hint for a specific query
-- AlloyDB uses its columnar engine automatically for eligible scans,
```

```
-- but you can check with EXPLAIN

EXPLAIN (ANALYZE, FORMAT TEXT)
SELECT id, 1 - (embedding <=> $1::vector) AS similarity
FROM document_embeddings
ORDER BY embedding <=> $1::vector
LIMIT 20;
-- Look for "Columnar Scan" in the plan output - indicates AlloyDB's engine
-- is handling the distance computation in vectorized form
```

13.4 Ingestion Pipelines: Embedding Generation and Bulk Loading

The vector search index is only as good as the embeddings feeding it. On managed services, the ingestion pipeline sits outside the database and must handle embedding API rate limits, model versioning, failure recovery, and bulk insert efficiency.

The following Python pipeline handles document ingestion against a Supabase or RDS/Aurora backend, using the OpenAI Embeddings API and batch-aware upsert logic:

```
# Python
import os
import time
import psycopg2
import psycopg2.extras
from openai import OpenAI
from typing import Generator

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# Connection string from environment - works for RDS, Aurora, AlloyDB, or Supabase
# For Supabase, use the "direct connection" string (port 5432), not the connection pooler,
# for bulk inserts to avoid pgBouncer transaction-mode limitations with COPY
DB_DSN = os.environ["DATABASE_URL"]

def chunk_documents(documents: list[dict], chunk_size: int = 500) -> Generator:
    """Yield successive chunks of documents with basic token-aware splitting."""
    for doc in documents:
        text = doc["content"]
        # Simple word-count chunking - replace with tiktoken-aware chunking in production
        words = text.split()
        for i in range(0, len(words), chunk_size):
            chunk_text = " ".join(words[i:i + chunk_size])
            yield {
                "document_id": doc["id"],
                "content": chunk_text,
                "chunk_index": i // chunk_size,
                "metadata": doc.get("metadata", {}),
            }

def generate_embeddings_batch(
    texts: list[str],
    model: str = "text-embedding-3-small",
    max_retries: int = 3
) -> list[list[float]]:
    """
    Call OpenAI Embeddings API with exponential backoff.
    text-embedding-3-small: 1536 dims, $0.02/1M tokens
    text-embedding-3-large: 3072 dims, $0.13/1M tokens
    text-embedding-ada-002: 1536 dims, $0.10/1M tokens (legacy)
    """
    for attempt in range(max_retries):
        try:
            response = openai_client.embeddings.create(
                input=texts,
```

```

        model=model,
        encoding_format="float"
    )
    return [item.embedding for item in response.data]
except Exception as e:
    if attempt == max_retries - 1:
        raise
    wait = 2 ** attempt
    print(f"Embedding API error (attempt {attempt + 1}): {e}. Retrying in {wait}s.")
    time.sleep(wait)

def ingest_documents(documents: list[dict], batch_size: int = 100) -> int:
    """
    Ingest documents into pgvector with batch embedding and upsert.
    Returns count of rows inserted/updated.
    """
    conn = psycopg2.connect(DB_DSN)
    conn.autocommit = False

    upsert_sql = """
        INSERT INTO document_embeddings
            (document_id, model_name, model_version, embedding, chunk_index, chunk_text)
        VALUES %s
        ON CONFLICT (document_id, model_name, model_version, chunk_index)
        DO UPDATE SET
            embedding = EXCLUDED.embedding,
            chunk_text = EXCLUDED.chunk_text,
            updated_at = NOW()
    """

    total_inserted = 0
    batch: list[dict] = []

    try:
        with conn.cursor() as cur:
            for chunk in chunk_documents(documents):
                batch.append(chunk)

            if len(batch) >= batch_size:
                texts = [c["content"] for c in batch]
                embeddings = generate_embeddings_batch(texts)

                rows = [
                    (
                        c["document_id"],
                        "text-embedding-3-small",
                        "3-small-v1",
                        embedding, # psycopg2 will serialize list to vector literal
                        c["chunk_index"],
                        c["content"],
                    )
                    for c, embedding in zip(batch, embeddings)
                ]

                psycopg2.extras.execute_values(
                    cur, upsert_sql, rows, page_size=batch_size
                )
                total_inserted += len(rows)
                conn.commit()
                print(f"Committed batch of {len(rows)} rows. Total: {total_inserted}")
                batch = []

            # Flush remaining
            if batch:
                texts = [c["content"] for c in batch]
                embeddings = generate_embeddings_batch(texts)
                rows = [
                    (c["document_id"], "text-embedding-3-small", "3-small-v1",
                     embedding, c["chunk_index"], c["content"])
                    for c, embedding in zip(batch, embeddings)

```

```

    ]
    psycogp2.extras.execute_values(cur, upsert_sql, rows)
    total_inserted += len(rows)
    conn.commit()

except Exception:
    conn.rollback()
    raise
finally:
    conn.close()

return total_inserted

```

One pattern worth calling out explicitly: **never embed inside a database transaction that also holds row locks**. If the OpenAI API call takes 800ms and you're holding a lock on a row you just inserted, you've introduced an 800ms lock wait for any concurrent writer touching the same document. Always generate embeddings before opening the transaction, then insert or upsert atomically.

13.5 Query Patterns, Connection Pooling, and Performance Tuning

Vector similarity queries have a shape that differs from typical OLTP queries: they're read-heavy, they frequently run against large tables with selective WHERE clauses, and they benefit significantly from connection routing to read replicas. This section covers the tuning knobs that actually move the needle on managed services.

Query structure for hybrid search — combining vector similarity with metadata filters—is one of the most common production patterns:

```

-- PostgreSQL / pgvector
-- Hybrid search: semantic similarity + metadata filter + recency boost
-- This is the pattern used in most production RAG pipelines

SET hnswef_search = 80; -- tune per-query based on latency/recall requirements

SELECT
    de.id,
    de.chunk_text,
    d.title,
    d.source_url,
    d.published_at,
    1 - (de.embedding <=> $1::vector(1536)) AS semantic_score,
    -- Recency boost: decays over 90 days
    EXP(-EXTRACT(EPOCH FROM (NOW() - d.published_at)) / (90 * 86400)) AS recency_score,
    -- Combined score
    (0.7 * (1 - (de.embedding <=> $1::vector(1536))))
    + (0.3 * EXP(-EXTRACT(EPOCH FROM (NOW() - d.published_at)) / (90 * 86400)))
    AS combined_score
FROM document_embeddings de
JOIN documents d ON d.id = de.document_id
WHERE
    de.model_name = 'text-embedding-3-small'
    AND de.model_version = '3-small-v1'
    AND d.tenant_id = $2 -- partition by tenant for SaaS architectures
    AND d.category = ANY($3) -- optional category filter passed as text[]
ORDER BY combined_score DESC
LIMIT 10;

-- Index to support the metadata filters efficiently
-- Create this before the HNSW index - the planner will use it for filter pushdown
CREATE INDEX idx_docs_tenant_category ON documents (tenant_id, category);

```

```
CREATE INDEX idx_embeddings_model ON document_embeddings (model_name, model_version);
```

Connection pooling considerations on managed services:

RDS and Aurora benefit from RDS Proxy for connection pooling at the infrastructure level. RDS Proxy uses connection multiplexing but switches to session mode when prepared statements are used—pgvector queries typically use parameterized queries with `$1::vector`, which counts as a prepared statement pattern. This can inadvertently increase actual backend connection counts. Monitor `DatabaseConnections` in CloudWatch alongside `DBConnectionsCurrentlyBorrowed` on the proxy to detect this.

Supabase exposes two connection strings: one through Supavisor (their connection pooler, transaction mode) and one direct to PostgreSQL. For vector ingestion pipelines using `COPY` or multi-statement transactions, always use the direct connection string on port 5432, not the pooler on port 6543. Transaction-mode poolers will fail or misbehave on `COPY` and explicit transaction blocks.

Partition strategy for large embedding tables: On Aurora and RDS, if your embeddings table exceeds ~50M rows, partitioning by a hash of `document_id` improves HNSW index build parallelism (each partition builds its own index) and makes individual partition vacuums faster:

```
-- PostgreSQL / pgvector
-- Range partition by creation date for time-series workloads,
-- or hash partition for even distribution across tenants

CREATE TABLE document_embeddings (
  id          BIGSERIAL,
  document_id BIGINT NOT NULL,
  model_name  TEXT NOT NULL,
  model_version TEXT NOT NULL,
  embedding   VECTOR(1536),
  chunk_text  TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

-- Create monthly partitions
CREATE TABLE document_embeddings_2024_01
  PARTITION OF document_embeddings
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE document_embeddings_2024_02
  PARTITION OF document_embeddings
  FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Each partition gets its own HNSW index
CREATE INDEX ON document_embeddings_2024_01
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

CREATE INDEX ON document_embeddings_2024_02
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

Autovacuum and bloat are significant concerns with high-ingestion embedding tables. Each new embedding is an `INSERT`, not an `UPDATE`, which keeps bloat lower than update-heavy workloads—but re-embedding existing documents (when you switch models) generates dead tuples at the rate of your ingestion pipeline. Set aggressive autovacuum parameters on the embeddings table:

```
-- PostgreSQL / pgvector
ALTER TABLE document_embeddings SET (
  autovacuum_vacuum_scale_factor = 0.01, -- vacuum when 1% of rows are dead (default: 20%)
  autovacuum_analyze_scale_factor = 0.005, -- analyze when 0.5% of rows change
```



```
autovacuum_vacuum_cost_delay = 2          -- reduce cost delay for this table
);
```

Supabase-specific query path via PostgREST: For applications using the Supabase client library, vector search runs through a PostgreSQL function exposed via PostgREST:

```
# Python
# Supabase vector search using the Python client
# The match_documents function encapsulates the similarity query
from supabase import create_client, Client
import os

url: str = os.environ["SUPABASE_URL"]
key: str = os.environ["SUPABASE_ANON_KEY"]

supabase: Client = create_client(url, key)

def search_documents(
    query_embedding: list[float],
    match_threshold: float = 0.78,
    match_count: int = 10,
    tenant_id: int = None
) -> list[dict]:
    """
    Calls a Supabase RPC function that performs vector similarity search.
    The function must be created in the database first (see SQL below).
    """
    params = {
        "query_embedding": query_embedding,
        "match_threshold": match_threshold,
        "match_count": match_count,
    }
    if tenant_id is not None:
        params["filter_tenant_id"] = tenant_id

    result = supabase.rpc("match_documents", params).execute()
    return result.data
```

The corresponding PostgreSQL function that PostgREST exposes:

```
-- PostgreSQL / pgvector (Supabase)
CREATE OR REPLACE FUNCTION match_documents(
    query_embedding    VECTOR(1536),
    match_threshold    FLOAT DEFAULT 0.78,
    match_count        INT  DEFAULT 10,
    filter_tenant_id   BIGINT DEFAULT NULL
)
RETURNS TABLE (
    id            BIGINT,
    chunk_text    TEXT,
    similarity     FLOAT,
    metadata      JSONB
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        de.id,
        de.chunk_text,
        1 - (de.embedding <=> query_embedding) AS similarity,
        d.metadata
    FROM document_embeddings de
    JOIN documents d ON d.id = de.document_id
    WHERE
        de.model_name = 'text-embedding-3-small'
        AND de.model_version = '3-small-v1'
        AND (filter_tenant_id IS NULL OR d.tenant_id = filter_tenant_id)
        AND 1 - (de.embedding <=> query_embedding) > match_threshold
```

```

ORDER BY de.embedding <=> query_embedding
LIMIT match_count;
END;
$$;

-- Grant execute permission to the anon and authenticated roles
GRANT EXECUTE ON FUNCTION match_documents TO anon, authenticated;

```

One important detail: the `match_threshold` filter (`1 - (embedding <=> query_embedding) > threshold`) is evaluated *after* the HNSW index scans its candidate list. It does not pre-filter the index scan. If your threshold is very high (e.g., 0.95), you may get fewer than `match_count` results even though matching rows exist in the table—because the index returned only `ef_search` candidates and none of the truly similar rows beyond that count were examined. In these cases, increase `hnsw.ef_search` or lower the threshold and rerank in application code.

13.6 Aurora Serverless v2 and Cost Architecture

Aurora Serverless v2 introduced a different cost model for vector workloads: you pay for Aurora Capacity Units (ACUs) consumed rather than instance hours. Vector similarity search is CPU-intensive, especially during HNSW graph traversal at high `ef_search` values. Understanding how ACU consumption maps to query patterns is necessary to avoid bill shock.

Each ACU provides approximately 2 GB of RAM and proportional CPU. An HNSW query against a 1M-row table with `ef_search = 100` typically requires holding several hundred megabytes of index graph in buffer cache for low-latency execution. If Aurora Serverless v2 scales down between query bursts, the buffer cache is cold and the first queries after scale-up will trigger index page reads from the Aurora storage layer, adding latency.

To manage this:

```

-- PostgreSQL / pgvector (Aurora Serverless v2)
-- Set minimum ACU capacity to a value that keeps the HNSW index warm in shared_buffers
-- Do this at the cluster level via the AWS Console or CLI - not in SQL
-- Example: for a 1M-row, 1536-dim table, the HNSW index occupies roughly
-- m * dimensions * 4 bytes * rows = 16 * 1536 * 4 * 1,000,000 98 GB worst case
-- In practice, pgvector HNSW uses significantly less (roughly 1.5-2x the raw vector size),
-- but plan for 4-8 GB of buffer cache for warm HNSW query performance

-- Check current buffer cache hit rate for the embeddings table
SELECT
    relname,
    heap_blks_read,
    heap_blks_hit,
    idx_blks_read,
    idx_blks_hit,
    ROUND(
        100.0 * idx_blks_hit / NULLIF(idx_blks_hit + idx_blks_read, 0),
        2
    ) AS idx_cache_hit_pct
FROM pg_statio_user_tables
WHERE relname = 'document_embeddings';

-- A healthy index cache hit rate is above 95%; below 80% indicates the HNSW
-- index is not fitting in shared_buffers and you need to increase memory/ACUs

```

Cost optimization patterns for Aurora and RDS:

Route all vector search traffic to read replicas. On Aurora, read replicas share the same storage

layer as the primary, so they serve queries from a warm buffer cache that the primary populated. The replica lag for embeddings tables is low because embedding updates are typically batched, not real-time:

```
# Python
# Connection routing: writes to primary, reads to replica
import psycopg2
from contextlib import contextmanager

PRIMARY_DSN = os.environ["DATABASE_URL_PRIMARY"]
REPLICA_DSN = os.environ["DATABASE_URL_REPLICA"]

@contextmanager
def read_connection():
    """Use read replica for all similarity search queries."""
    conn = psycopg2.connect(REPLICA_DSN)
    try:
        yield conn
    finally:
        conn.close()

@contextmanager
def write_connection():
    """Use primary for ingestion and updates."""
    conn = psycopg2.connect(PRIMARY_DSN)
    try:
        yield conn
    finally:
        conn.close()

def search_similar(query_embedding: list[float], top_k: int = 10) -> list[dict]:
    with read_connection() as conn:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
            cur.execute("SET hnsw.ef_search = 80")
            cur.execute("""
                SELECT id, chunk_text,
                       1 - (embedding <=> %s::vector) AS similarity
                FROM document_embeddings
                WHERE model_name = 'text-embedding-3-small'
                ORDER BY embedding <=> %s::vector
                LIMIT %s
            """, (query_embedding, query_embedding, top_k))
            return [dict(row) for row in cur.fetchall()]
```

13.7 AlloyDB ML Integration: Embedding Without Leaving SQL

AlloyDB's `google_ml_integration` extension closes the gap between embedding model calls and database operations, which matters for pipelines that need low-latency, transactionally consistent embedding generation.

The practical use case: you receive a document, insert it, embed it, and store the embedding in a single database-side function call. No application middleware needed for the embedding step:

```
-- PostgreSQL / pgvector (AlloyDB)
-- End-to-end document insert + embed in a single function
CREATE OR REPLACE FUNCTION insert_document_with_embedding(
    p_content      TEXT,
    p_metadata     JSONB DEFAULT '{}')
)
RETURNS BIGINT
LANGUAGE plpgsql
AS $$
DECLARE
```

```

v_doc_id    BIGINT;
v_embedding VECTOR(768); -- textembedding-gecko@003 produces 768 dims
BEGIN
-- Insert the document
INSERT INTO documents (content, metadata)
VALUES (p_content, p_metadata)
RETURNING id INTO v_doc_id;

-- Generate embedding via Vertex AI (network call from AlloyDB to Vertex AI)
v_embedding := google_ml.embedding(
    'textembedding-gecko@003',
    p_content
)::vector(768);

-- Insert the embedding
INSERT INTO document_embeddings
(document_id, model_name, model_version, embedding)
VALUES
(v_doc_id, 'textembedding-gecko', '003', v_embedding);

RETURN v_doc_id;
END;
$$;

-- Usage
SELECT insert_document_with_embedding(
    'PostgreSQL HNSW indexes support concurrent builds without exclusive table locks.',
    '{"source": "manual", "category": "database"}'::jsonb
);

```

This pattern has a significant limitation to be aware of: the Vertex AI embedding call happens inside the transaction. If the Vertex AI endpoint is slow or unavailable, your database transaction blocks until it times out. For high-throughput ingestion, this approach can create connection pool exhaustion faster than a decoupled pipeline. Use it for low-volume, latency-sensitive use cases (e.g., embedded in a trigger for small tables) or for background enrichment jobs with low concurrency.

AlloyDB also supports **online schema changes** for vector columns via `ALTER TABLE ... ADD COLUMN`, which is non-blocking for additions. Combined with a background job that populates the new embedding column, you can roll out a new embedding model with zero downtime:

```

-- PostgreSQL / pgvector (AlloyDB)
-- Step 1: Add new column for updated model (non-blocking)
ALTER TABLE document_embeddings
ADD COLUMN embedding_v2 VECTOR(768); -- switching to gecko@003 from a custom model

-- Step 2: Background populate (run as a batch job, not inline)
-- Populate in batches to avoid long-running transactions
DO $$
DECLARE
    batch_size INT := 500;
    last_id    BIGINT := 0;
    max_id     BIGINT;
BEGIN
    SELECT MAX(id) INTO max_id FROM document_embeddings WHERE embedding_v2 IS NULL;

    WHILE last_id < max_id LOOP
        UPDATE document_embeddings de
        SET embedding_v2 = google_ml.embedding(
            'textembedding-gecko@003',
            (SELECT content FROM documents d WHERE d.id = de.document_id)
        )::vector(768)
        WHERE de.id > last_id
            AND de.id <= last_id + batch_size
            AND de.embedding_v2 IS NULL;

        last_id := last_id + batch_size;
        PERFORM pg_sleep(0.05); -- brief pause to reduce I/O pressure
    END LOOP;
END;

```

```

    END LOOP;
END;
$$;

-- Step 3: Build HNSW index on new column
CREATE INDEX CONCURRENTLY idx_embeddings_v2_hnsw
    ON document_embeddings
    USING hnsw (embedding_v2 vector_cosine_ops)
    WITH (m = 16, ef_construction = 128);

-- Step 4: Drop old column (non-blocking on AlloyDB, brief lock on standard PG)
-- Only after verifying queries use the new index
ALTER TABLE document_embeddings DROP COLUMN embedding;
ALTER TABLE document_embeddings RENAME COLUMN embedding_v2 TO embedding;

```

13.8 Monitoring and Observability

Production vector search requires a monitoring strategy beyond standard database metrics. The key additional signals are: index build status, index size, cache hit rates for vector index pages, and per-query recall (if approximate indexes are used).

```

-- PostgreSQL / pgvector
-- Index health and size monitoring
SELECT
    i.relname AS index_name,
    t.relname AS table_name,
    pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
    ix.indisvalid AS is_valid,
    ix.indisready AS is_ready
FROM pg_index ix
JOIN pg_class i ON i.oid = ix.indexrelid
JOIN pg_class t ON t.oid = ix.indrelid
WHERE t.relname = 'document_embeddings'
ORDER BY pg_relation_size(i.oid) DESC;

-- Active index builds (useful during CONCURRENTLY builds)
SELECT
    phase,
    blocks_done,
    blocks_total,
    ROUND(100.0 * blocks_done / NULLIF(blocks_total, 0), 1) AS pct_complete,
    tuples_done,
    tuples_total
FROM pg_stat_progress_create_index
WHERE relid = 'document_embeddings'::regclass;

-- Long-running vector queries (queries over 5 seconds)
SELECT
    pid,
    now() - pg_stat_activity.query_start AS duration,
    query,
    state
FROM pg_stat_activity
WHERE (now() - pg_stat_activity.query_start) > INTERVAL '5 seconds'
    AND query ILIKE '%<=>% ' -- vector distance operator
ORDER BY duration DESC;

```

For **Supabase**, the dashboard exposes query performance metrics through the built-in Query Performance analyzer, which is backed by `pg_stat_statements`. Add the extension if it is not already enabled (it is on paid tiers):

```

-- PostgreSQL / pgvector (Supabase)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

```

```
-- Top 10 slowest vector queries
SELECT
  LEFT(query, 120) AS query_preview,
  calls,
  ROUND((mean_exec_time)::numeric, 2) AS avg_ms,
  ROUND((total_exec_time / 1000)::numeric, 2) AS total_sec,
  rows
FROM pg_stat_statements
WHERE query ILIKE '%<=>%'
ORDER BY mean_exec_time DESC
LIMIT 10;
```

For **RDS and Aurora**, publish Performance Insights metrics to CloudWatch and set alarms on: - DBLoad exceeding the vCPU count (indicates vector query CPU saturation) - ReadIOPS sustained above baseline during non-build periods (indicates index cache misses) - FreeableMemory falling below 15% (HNSW index pages are being evicted)

13.9 Migration Path: Moving Between Platforms

Teams frequently start on Supabase for the development velocity and migrate to Aurora or AlloyDB as scale demands grow. The migration is mechanically straightforward—pgvector data is just PostgreSQL data—but the operational gaps require planning.

Exporting from Supabase:

```
# Shell
# pg_dump from Supabase - use the direct connection string (port 5432)
# The connection string is in Project Settings > Database
pg_dump \
  --host=db.abcdefghijklmno.supabase.co \
  --port=5432 \
  --username=postgres \
  --dbname=postgres \
  --no-password \
  --format=custom \
  --compress=9 \
  --table=documents \
  --table=document_embeddings \
  --file=embeddings_export.dump

# This exports the data but NOT the Supabase-specific auth tables (auth.users, etc.)
# For a full application migration, handle auth separately using the Supabase migration tools
```

Restoring to Aurora or RDS:

```
# Shell
# Restore to Aurora PostgreSQL
# Ensure the target DB has the vector extension enabled first
psql \
  --host=your-aurora-cluster.cluster-xxxx.us-east-1.rds.amazonaws.com \
  --port=5432 \
  --username=postgres \
  --dbname=yourdb \
  --command="CREATE EXTENSION IF NOT EXISTS vector;"

pg_restore \
  --host=your-aurora-cluster.cluster-xxxx.us-east-1.rds.amazonaws.com \
  --port=5432 \
  --username=postgres \
  --dbname=yourdb \
  --jobs=4 \
  --no-owner \
  --no-acl \
  # parallel restore - match to vCPU count
```

```
embeddings_export.dump
```

The indexes do not need to be exported and restored—they can be rebuilt on the target after data load. In fact, restoring with `--section=data` (no indexes) and then rebuilding indexes on the target is faster than serializing and deserializing HNSW graph structures:

```
# Shell
# Export data only (no indexes), then rebuild indexes on target
pg_dump \
  --section=pre-data \
  --section=data \
  # --section=post-data is omitted - this skips index creation
  --format=custom \
  --table=document_embeddings \
  --file=embeddings_data_only.dump

# After restore, rebuild HNSW index with target-appropriate parameters
```

13.10 Practical Checklist Before Going to Production

The following covers the decisions that cause the most production incidents on managed pgvector deployments:

1. Confirm pgvector version and feature availability

```
-- PostgreSQL / pgvector
SELECT extversion FROM pg_extension WHERE extname = 'vector';
-- Minimum for HNSW: 0.5.0
-- Minimum for halfvec: 0.7.0
-- Minimum for binary quantization (bit type): 0.7.0
```

2. Set `maintenance_work_mem` before every index build

On RDS and Aurora, this is a session parameter—it resets after the connection closes. Add it to your migration script or index-build runbook:

```
-- PostgreSQL / pgvector
SET maintenance_work_mem = '6GB'; -- Adjust based on instance class
-- db.r6g.xlarge (32 GB RAM): safe to set 8 GB for index builds
-- db.r6g.large (16 GB RAM): use 4 GB; leave headroom for other queries
CREATE INDEX CONCURRENTLY ...
```

3. Verify HNSW index is actually used by the planner

```
-- PostgreSQL / pgvector
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, 1 - (embedding <=> '[...]'::vector) AS similarity
FROM document_embeddings
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;
-- Must show "Index Scan using idx_embeddings_hnsw" - if you see "Seq Scan",
-- the planner has decided sequential scan is cheaper (common on very small tables
-- or when enable_indexscan is off)
```

4. Test recall explicitly

HNSW is approximate. Before deploying, measure recall against an exact KNN baseline on a representative sample:

```
# Python
def measure_recall(
    conn,
    query_embedding: list[float],
```

```
k: int = 10,
ef_search: int = 80
) -> float:
    """Compare HNSW results against exact KNN to measure recall."""
    with conn.cursor() as cur:
        # Exact KNN (seq scan, no index)
        cur.execute("SET enable_indexscan = OFF")
        cur.execute("""
            SELECT id FROM document_embeddings
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """, (query_embedding, k))
        exact_ids = {row[0] for row in cur.fetchall()}

        # HNSW approximate KNN
        cur.execute("SET enable_indexscan = ON")
        cur.execute(f"SET hnsw.ef_search = {ef_search}")
        cur.execute("""
            SELECT id FROM document_embeddings
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """, (query_embedding, k))
        approx_ids = {row[0] for row in cur.fetchall()}

    return len(exact_ids & approx_ids) / k
```

5. Plan for HNSW index rebuild during model transitions

When you switch embedding models, the existing HNSW index is worthless—it was built on vectors from the old model, and new-model vectors will not be nearby in the same space. Create the new column and index before dropping the old one, and serve from both during the transition window.

Key Takeaways

- **Managed pgvector is engine-version-locked:** The pgvector version you get on RDS, Aurora, AlloyDB, or Supabase is determined by the platform's PostgreSQL engine packaging, not independently upgradable. Verify feature availability (HNSW, halfvec, bit) before committing to a schema design.
- **HNSW is the right default for production on managed services,** but requires substantial `maintenance_work_mem` during index construction (plan for 4–8 GB for million-row, high-dimensional tables). Always build with `CREATE INDEX CONCURRENTLY` on live systems to avoid table-level locks.
- **Ingestion pipelines must decouple embedding generation from database transactions:** Calling external embedding APIs (OpenAI, Vertex AI) inside a database transaction holds locks and blocks connections. Generate embeddings first, then commit atomically. AlloyDB's SQL-level Vertex AI integration is the exception, and it comes with its own concurrency trade-offs.
- **Hybrid search (vector + metadata filters) requires both an HNSW index and supporting B-tree indexes** on the filter columns. The planner combines them via bitmap scans or index scans with filter pushdown. Without B-tree indexes on tenant ID, category, or date range columns, metadata filters will cause sequential scans that defeat the purpose of the vector index.

- **Connection routing and buffer cache warmth determine real-world vector query latency:** On Aurora and RDS, route search traffic to read replicas to protect write throughput. On Aurora Serverless v2, set a minimum ACU capacity that keeps your HNSW index warm in shared buffers—cold-start latency after scale-down can be an order of magnitude higher than steady-state latency.
-

Chapter 14: Monitoring and Diagnosing pgvector Query Performance

pgvector brings vector similarity search into PostgreSQL, which means you get the full benefit of a mature relational engine alongside your embedding workloads—but you also inherit PostgreSQL’s particular approach to query planning, index maintenance, and execution statistics. Understanding what to measure, where to look, and how to interpret what you find is the difference between a vector search layer that degrades silently and one that you can tune with confidence. This chapter walks through the full monitoring stack for pgvector: from the query-level diagnostics you should run daily, through index health signals, to production observability patterns that integrate with your existing database monitoring infrastructure.

14.1 Understanding How pgvector Executes Similarity Queries

Before you can diagnose performance problems, you need a mental model of what pgvector is actually doing when it executes a vector similarity query. Seasoned DBAs often apply their intuition from B-tree indexes and sequential scan costs, but pgvector’s behavior is different enough that those instincts can mislead.

The two execution paths

pgvector supports two index types: IVFFlat and HNSW. Both serve approximate nearest neighbor (ANN) searches but traverse entirely different data structures.

IVFFlat works by partitioning vectors into *lists* (Voronoi cells) at index build time. At query time, the **probes** parameter controls how many of those lists are scanned. A low **probes** value is fast but potentially inaccurate; a high value approaches brute-force performance while improving recall. This is directly analogous to a partition pruning parameter—except that unlike partition pruning, which is binary (either a partition is scanned or it isn’t), **probes** is a continuous tradeoff.

HNSW builds a layered graph of approximate neighbors. At query time, **ef_search** controls how many candidates are explored during graph traversal. Higher **ef_search** means better recall and slower queries. HNSW generally outperforms IVFFlat on recall-latency tradeoffs for the same dataset, but it builds slower and consumes more memory.

When no index is present, or when the query planner decides an index scan is more expensive

than a sequential scan, pgvector falls back to an exact brute-force search over the entire table. For tables under roughly 100,000 rows at 1,536 dimensions (a common OpenAI embedding size), this sequential scan is often competitive with or faster than indexed search—a fact that surprises DBAs accustomed to indexes always being beneficial.

What the planner sees

PostgreSQL’s query planner treats vector index scans as any other access path, but it has limited visibility into ANN recall and the cost of graph or cluster traversal. The planner uses generic cost estimates for index scans, which can lead to suboptimal decisions—especially when filters combine vector predicates with scalar predicates (for example, `WHERE tenant_id = $1 ORDER BY embedding <=> $2 LIMIT 10`). This filter interaction is one of the most common sources of performance surprises in production pgvector deployments and is covered in detail in section 14.4.

14.2 Core Query Diagnostics: EXPLAIN, ANALYZE, and Execution Statistics

The standard PostgreSQL diagnostic toolkit works for pgvector queries, but you need to know what to look for in the output.

Reading EXPLAIN ANALYZE for vector queries

```
-- PostgreSQL / pgvector
-- Basic vector query with EXPLAIN ANALYZE
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    id,
    content,
    embedding <=> '[0.021, -0.045, 0.133, ...]':vector AS distance
FROM documents
ORDER BY embedding <=> '[0.021, -0.045, 0.133, ...]':vector
LIMIT 10;
```

Key fields to examine in the output:

- **Node type:** Look for `Index Scan using <index_name>` vs `Seq Scan`. A sequential scan on a large table is a signal that either no index exists, the index was not built on the same operator class, or the planner estimated the sequential scan as cheaper.
- **Actual Rows vs Plan Rows:** A large mismatch here indicates stale table statistics. Run `ANALYZE documents` and recheck.
- **Buffers: shared hit / shared read:** High `shared read` values indicate that the index or table data is not in `shared_buffers`, causing disk I/O. For HNSW indexes, the upper layers of the graph are accessed on nearly every query and should be cache-resident.
- **Actual Time:** The wall-clock time for node execution. For a 10-result ANN query on a properly indexed 1M-row table using HNSW with typical `ef_search=64`, expect actual time in the 5–50ms range depending on vector dimensions and hardware.

Confirming which index is being used

```
-- PostgreSQL / pgvector
-- Verify index usage and operator class
SELECT
    indexname,
    indexdef
```

```

FROM pg_indexes
WHERE tablename = 'documents';

-- Check that the query is using the expected index
EXPLAIN SELECT id
FROM documents
ORDER BY embedding <=> '[0.021, -0.045, 0.133, ...]':vector
LIMIT 10;

-- Force a sequential scan to establish baseline comparison
SET enable_indexscan = off;
EXPLAIN (ANALYZE, BUFFERS)
SELECT id
FROM documents
ORDER BY embedding <=> '[0.021, -0.045, 0.133, ...]':vector
LIMIT 10;
RESET enable_indexscan;

```

Sampling query latency from pg_stat_statements

pg_stat_statements captures aggregate performance data across all executions of a normalized query. For vector queries, this gives you the sustained latency profile rather than a single EXPLAIN run.

```

-- PostgreSQL / pgvector
-- Enable pg_stat_statements if not already active
-- Add to postgresql.conf: shared_preload_libraries = 'pg_stat_statements'

-- Find the slowest vector similarity queries
SELECT
    left(query, 120)                AS query_preview,
    calls,
    round(mean_exec_time::numeric, 2) AS mean_ms,
    round(stddev_exec_time::numeric, 2) AS stddev_ms,
    round(total_exec_time::numeric, 2) AS total_ms,
    round((100 * total_exec_time /
        sum(total_exec_time) OVER ()):numeric, 2) AS pct_total,
    rows / calls                    AS avg_rows_returned
FROM pg_stat_statements
WHERE query ILIKE '%<=>%'
       OR query ILIKE '%<#>%'
       OR query ILIKE '%<+>%'
ORDER BY mean_exec_time DESC
LIMIT 20;

```

The three operator patterns cover the three distance metrics: `<=>` for cosine distance, `<#>` for negative inner product, and `<+>` for L1 distance. A well-tuned ANN query returning 10 results from a 1M-row indexed table should show `mean_ms` under 50ms; values above 200ms warrant investigation.

14.3 Index Health and Build Quality

An IVFFlat or HNSW index that was built under suboptimal conditions—too few rows, wrong `lists` count for IVFFlat, or insufficient `ef_construction` for HNSW—will deliver degraded recall or performance that no amount of query tuning can fix. Index health monitoring is a maintenance discipline, not a one-time setup check.

Inspecting index metadata

```

-- PostgreSQL / pgvector
-- View index build parameters and storage
SELECT

```

```

i.relname          AS index_name,
t.relname          AS table_name,
pg_size_pretty(pg_relation_size(i.oid)) AS index_size,
ix.indisvalid      AS is_valid,
ix.indisready      AS is_ready,
am.amname          AS access_method,
pg_get_indexdef(i.oid) AS index_definition
FROM pg_index ix
JOIN pg_class i  ON i.oid = ix.indexrelid
JOIN pg_class t  ON t.oid = ix.indrelid
JOIN pg_am am    ON am.oid = i.relam
WHERE t.relname = 'documents'
      AND am.amname IN ('hnsu', 'ivfflat');

-- Check for bloat: rows in table vs rows at index build time
-- (IVFFlat does not automatically rebalance after inserts)
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    last_vacuum,
    last_analyze,
    last_autoanalyze
FROM pg_stat_user_tables
WHERE relname = 'documents';

```

IVFFlat: the reindex-after-growth rule

IVFFlat partitions are fixed at index build time. If your table had 100,000 rows when you built the index with `lists = 100` and now has 2,000,000 rows, the list distribution is wildly suboptimal. The recommended formula is `lists = sqrt(row_count)` for tables up to a few million rows, and `lists = row_count / 1000` for larger tables.

```

-- PostgreSQL / pgvector
-- Check current table size and compute recommended lists value
SELECT
    relname,
    n_live_tup,
    round(sqrt(n_live_tup::numeric)) AS live_rows,
    round(n_live_tup / 1000.0)       AS recommended_lists_small,
    pg_size_pretty(pg_relation_size(relid)) AS recommended_lists_large,
    table_size
FROM pg_stat_user_tables
WHERE relname = 'documents';

-- Rebuild the IVFFlat index with corrected parameters
-- Use CONCURRENTLY to avoid locking in production
DROP INDEX CONCURRENTLY IF EXISTS documents_embedding_ivfflat_idx;

CREATE INDEX CONCURRENTLY documents_embedding_ivfflat_idx
ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 1000); -- adjust based on query above

```

HNSW: memory pressure signals

HNSW indexes hold their upper graph layers in memory for performance. On a system where `shared_buffers` is too small or the index is being evicted, query latency spikes. Monitor buffer hit ratios specifically for the index relation:

```

-- PostgreSQL / pgvector
-- Buffer hit ratio for the vector index specifically
SELECT
    c.relname,
    sum(b.heap_blks_hit) AS heap_hits,
    sum(b.heap_blks_read) AS heap_reads,
    round(100.0 * sum(b.heap_blks_hit) /

```

```

        nullif(sum(b.heap_blks_hit) + sum(b.heap_blks_read), 0), 2) AS heap_hit_pct,
        sum(b.idx_blks_hit) AS idx_hits,
        sum(b.idx_blks_read) AS idx_reads,
        round(100.0 * sum(b.idx_blks_hit) /
              nullif(sum(b.idx_blks_hit) + sum(b.idx_blks_read), 0), 2) AS idx_hit_pct
FROM pg_statio_user_tables b
JOIN pg_class c ON c.relname = b.relname
WHERE b.relname = 'documents'
GROUP BY c.relname;

```

Target `idx_hit_pct` above 95% for HNSW indexes. Values below 80% indicate the index is being repeatedly read from disk, which will push your median query latency above 100ms regardless of `ef_search` settings.

14.4 Diagnosing the Scalar-Filter Interaction Problem

The most common production performance issue in pgvector deployments is not slow ANN search—it is the interaction between vector ordering and scalar filter predicates. This problem is well-known in the dedicated vector database community but tends to catch PostgreSQL DBAs off guard because it violates normal index selectivity reasoning.

Why the problem occurs

Consider this query:

```

SELECT id, content
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> $1
LIMIT 10;

```

The planner has two realistic strategies:

1. Apply the `WHERE tenant_id = 42` filter first (using an index on `tenant_id`), then do an exact brute-force vector search over the filtered rows.
2. Use the HNSW or IVFFlat index to find approximate nearest neighbors across all rows, then discard results that don't match `tenant_id = 42`.

Strategy 1 is correct but PostgreSQL's vector indexes don't support index-level filtering—the filter is applied as a post-scan recheck. Strategy 2 can return far fewer than `LIMIT 10` results if most ANN candidates don't pass the scalar filter, causing PostgreSQL to repeatedly widen its search, which dramatically inflates execution time.

Diagnosing with EXPLAIN

```

-- PostgreSQL / pgvector
-- Demonstrate the filter-interaction problem
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, content, embedding <=> $1 AS distance
FROM documents
WHERE tenant_id = 42
      AND metadata->>'language' = 'en'
ORDER BY embedding <=> $1
LIMIT 10;

-- If you see "Rows Removed by Filter" that is large relative to
-- actual rows returned, the index is scanning far more candidates
-- than needed. Example problematic output:
--   Index Scan using documents_embedding_hnsw_idx on documents

```

```
-- (actual rows=10, loops=1)
-- Filter: ((tenant_id = 42) AND (...))
-- Rows Removed by Filter: 3840  <-- this is the signal
```

A Rows Removed by Filter count that is 10x or more the LIMIT value is the key diagnostic signal. For HNSW, this means `ef_search` is being exhausted without satisfying the filter, and PostgreSQL is doing many graph traversal iterations.

Mitigation patterns

```
-- PostgreSQL / pgvector
-- Pattern 1: Partial index per tenant (effective for small tenant counts)
CREATE INDEX documents_tenant42_hnsw_idx
ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
WHERE tenant_id = 42;

-- Pattern 2: Increase ef_search for filtered queries only
-- (session-level, not persistent)
SET hnsw.ef_search = 200; -- default is 40; increase until filter hit rate improves
SELECT id, content
FROM documents
WHERE tenant_id = 42
ORDER BY embedding <=> $1
LIMIT 10;
RESET hnsw.ef_search;

-- Pattern 3: Pre-filter with CTE, then vector-rank within result
-- Effective when the scalar filter is highly selective (< 5% of rows)
WITH tenant_docs AS (
    SELECT id, content, embedding
    FROM documents
    WHERE tenant_id = 42
    AND created_at > now() - interval '30 days'
)
SELECT id, content, embedding <=> $1 AS distance
FROM tenant_docs
ORDER BY embedding <=> $1
LIMIT 10;

-- Pattern 4: Check planner's estimated vs actual rows for the filter
-- to decide which strategy the planner chose
EXPLAIN (ANALYZE)
SELECT id
FROM documents
WHERE tenant_id = 42;
-- If estimated rows ~= actual rows, statistics are current;
-- if estimate is way off, the planner is making poor strategy decisions.
-- Run: ANALYZE documents;
```

The right mitigation depends on filter selectivity and the number of distinct filter values. Partial indexes work well for a small number of high-traffic tenants. CTE pre-filtering works well when scalar filters remove more than 90% of the table. Raising `ef_search` is a stopgap for cases where recall is suffering due to filtered candidate exhaustion but you can't restructure data layout immediately.

14.5 Production Observability: Metrics, Alerting, and Continuous Profiling

Ad-hoc EXPLAIN runs tell you what a query did on a specific execution. Production observability tells you what your vector layer is doing continuously, across all query patterns, over time. For

most PostgreSQL shops, this means wiring pgvector metrics into an existing Prometheus/Grafana stack or a managed monitoring service.

Python-based latency profiling with psycopg2

```
# Python
import time
import statistics
import psycopg2
import numpy as np
from typing import List, Tuple

def profile_vector_query(
    conn_string: str,
    query_vectors: List[np.ndarray],
    limit: int = 10,
    ef_search: int = 64,
    iterations: int = 100,
) -> dict:
    """
    Profile vector search latency over multiple query vectors.
    Returns p50, p95, p99 latencies and throughput.
    """
    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()

    # Set HNSW search parameter for this session
    cur.execute(f"SET hnsw.ef_search = {ef_search}")

    latencies_ms: List[float] = []

    for i in range(iterations):
        vec = query_vectors[i % len(query_vectors)]
        vec_str = "[" + ",".join(str(round(float(v), 6)) for v in vec) + "]"

        sql = f"""
            SELECT id, embedding <=> %s::vector AS distance
            FROM documents
            ORDER BY embedding <=> %s::vector
            LIMIT %s
        """

        start = time.perf_counter()
        cur.execute(sql, (vec_str, vec_str, limit))
        rows = cur.fetchall()
        elapsed_ms = (time.perf_counter() - start) * 1000
        latencies_ms.append(elapsed_ms)

        # Verify we got the expected number of results
        if len(rows) < limit:
            print(f"Warning: iteration {i} returned only {len(rows)} results")

    cur.close()
    conn.close()

    sorted_latencies = sorted(latencies_ms)
    n = len(sorted_latencies)

    return {
        "ef_search": ef_search,
        "iterations": iterations,
        "p50_ms": round(sorted_latencies[int(n * 0.50)], 2),
        "p95_ms": round(sorted_latencies[int(n * 0.95)], 2),
        "p99_ms": round(sorted_latencies[int(n * 0.99)], 2),
        "mean_ms": round(statistics.mean(latencies_ms), 2),
        "max_ms": round(max(latencies_ms), 2),
        "qps": round(1000 / statistics.mean(latencies_ms), 1),
    }
```

```

# Usage: sweep ef_search values to find the latency-recall tradeoff point
if __name__ == "__main__":
    import os

    conn_str = os.environ["PGVECTOR_CONN"]

    # Generate synthetic query vectors (replace with real embedding samples)
    rng = np.random.default_rng(42)
    sample_queries = [
        rng.standard_normal(1536).astype(np.float32)
        for _ in range(20)
    ]
    # Normalize for cosine distance
    sample_queries = [
        v / np.linalg.norm(v) for v in sample_queries
    ]

    for ef in [20, 40, 64, 100, 150, 200]:
        result = profile_vector_query(
            conn_str, sample_queries, ef_search=ef, iterations=50
        )
        print(
            f"ef_search={ef:4d} | "
            f"p50={result['p50_ms']:6.1f}ms | "
            f"p95={result['p95_ms']:6.1f}ms | "
            f"p99={result['p99_ms']:6.1f}ms | "
            f"qps={result['qps']:6.1f}"
        )

```

This sweep produces output like:

```

ef_search= 20 | p50=  8.2ms | p95= 12.1ms | p99= 18.4ms | qps= 112.3
ef_search= 40 | p50= 14.6ms | p95= 19.8ms | p99= 28.7ms | qps=  65.4
ef_search= 64 | p50= 22.3ms | p95= 31.2ms | p99= 44.9ms | qps=  43.1
ef_search= 100 | p50= 33.7ms | p95= 47.6ms | p99= 68.2ms | qps=  28.9
ef_search= 150 | p50= 49.2ms | p95= 69.3ms | p99= 98.1ms | qps=  19.8
ef_search= 200 | p50= 64.8ms | p95= 91.4ms | p99=131.2ms | qps=  15.1

```

The inflection point where latency grows faster than linearly is where you're hitting diminishing returns. For most production workloads, `ef_search=64` to `ef_search=100` is the reasonable operating range.

Continuous monitoring with `pg_stat_activity` and custom metrics

```

-- PostgreSQL / pgvector
-- Active vector queries right now (useful during incident investigation)
SELECT
    pid,
    now() - pg_stat_activity.query_start AS duration,
    state,
    wait_event_type,
    wait_event,
    left(query, 200) AS query_preview
FROM pg_stat_activity
WHERE state != 'idle'
    AND (query ILIKE '%<=>%' OR query ILIKE '%<#>%')
ORDER BY duration DESC;

-- Long-running vector queries (more than 500ms)
SELECT
    pid,
    round(extract(epoch FROM (now() - query_start)) * 1000) AS duration_ms,
    state,
    wait_event_type || ': ' || coalesce(wait_event, '') AS wait_detail,
    left(query, 300) AS query

```

```

FROM pg_stat_activity
WHERE state != 'idle'
  AND query ILIKE '%<=>%'
  AND (now() - query_start) > interval '500 milliseconds'
ORDER BY duration_ms DESC;

-- Aggregate: track query latency trend over time using a scheduled job
-- Store results in a monitoring table for trend analysis
CREATE TABLE IF NOT EXISTS vector_query_stats_snapshots (
  snapshot_time    timestampz DEFAULT now(),
  query_fingerprint text,
  calls            bigint,
  mean_ms          numeric,
  p99_estimate_ms  numeric,
  total_ms         numeric
);

INSERT INTO vector_query_stats_snapshots
(query_fingerprint, calls, mean_ms, total_ms)
SELECT
  md5(query) AS query_fingerprint,
  calls,
  round(mean_exec_time::numeric, 2),
  round(total_exec_time::numeric, 2)
FROM pg_stat_statements
WHERE query ILIKE '%<=>%'
ORDER BY total_exec_time DESC
LIMIT 50;

```

Schedule that INSERT every 5 minutes via `pg_cron` or an external cron job to build a latency trend dataset you can plot in Grafana.

Alerting thresholds for production pgvector

Metric	Warning	Critical	Action
Mean vector query latency	> 100ms	> 500ms	Check <code>ef_search</code> , index health, buffer hit ratio
Index buffer hit ratio	< 95%	< 80%	Increase <code>shared_buffers</code> , investigate eviction
Rows Removed by Filter ratio	> 50x LIMIT	> 200x LIMIT	Raise <code>ef_search</code> , add partial index, or restructure query
IVFFlat probes effectiveness	recall < 0.90	recall < 0.75	Rebuild index with higher <code>lists</code> , increase <code>probes</code>
Dead tuple ratio on documents table	> 10%	> 25%	Run <code>VACUUM ANALYZE documents</code>
Index build age (IVFFlat)	> 2x rows at build	> 5x rows at build	Schedule <code>REINDEX CONCURRENTLY</code>

14.6 Recall Measurement in Production

Latency and throughput are easy to measure with wall-clock time. Recall—the fraction of true nearest neighbors that your ANN query actually returns—is harder to measure but equally important. A query that returns in 5ms but surfaces the wrong documents is worse than a 50ms query that surfaces the right ones.

Offline recall estimation using ground truth

```
# Python
import numpy as np
import psycopg2
from sklearn.metrics.pairwise import cosine_similarity
from typing import List, Tuple, Dict

def compute_exact_neighbors(
    vectors: np.ndarray,
    query_vectors: np.ndarray,
    k: int
) -> List[List[int]]:
    """
    Brute-force exact nearest neighbors using sklearn.
    Returns list of lists of row indices (0-based).
    """
    sims = cosine_similarity(query_vectors, vectors)
    # argsort ascending gives furthest first; flip for nearest
    return [
        list(np.argsort(-row)[:k])
        for row in sims
    ]

def measure_recall(
    conn_string: str,
    row_ids: List[int],
    vectors: np.ndarray,
    query_vectors: np.ndarray,
    k: int = 10,
    ef_search: int = 64,
) -> Dict[str, float]:
    """
    Compare ANN results from pgvector against exact brute-force results.
    row_ids must correspond 1:1 with vectors (row_ids[i] is the DB id for vectors[i]).
    """
    exact_neighbors = compute_exact_neighbors(vectors, query_vectors, k)

    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()
    cur.execute(f"SET hnswef_search = {ef_search}")

    recall_scores: List[float] = []

    for q_idx, qvec in enumerate(query_vectors):
        vec_str = "[" + ",".join(str(round(float(v), 6)) for v in qvec) + "]"
        cur.execute(
            """
            SELECT id
            FROM documents
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """
            ,
            (vec_str, k),
        )
        ann_ids = [row[0] for row in cur.fetchall()]

        # Convert exact neighbor indices to DB ids
        exact_ids = set(row_ids[idx] for idx in exact_neighbors[q_idx])
        ann_ids_set = set(ann_ids)
```

```

        overlap = len(exact_ids & ann_ids_set)
        recall_scores.append(overlap / k)

    cur.close()
    conn.close()

recall_arr = np.array(recall_scores)
return {
    "ef_search": ef_search,
    "k": k,
    "mean_recall": round(float(recall_arr.mean()), 4),
    "min_recall": round(float(recall_arr.min()), 4),
    "p10_recall": round(float(np.percentile(recall_arr, 10)), 4),
    "queries_below_90pct": int((recall_arr < 0.90).sum()),
}

# Usage
if __name__ == "__main__":
    import os

    conn_str = os.environ["PGVECTOR_CONN"]

    # In practice, load these from your actual documents table
    rng = np.random.default_rng(0)
    n_docs = 50_000
    dims = 1536
    doc_vectors = rng.standard_normal((n_docs, dims)).astype(np.float32)
    doc_vectors /= np.linalg.norm(doc_vectors, axis=1, keepdims=True)

    # Simulate row IDs starting at 1
    doc_ids = list(range(1, n_docs + 1))

    n_queries = 100
    query_vecs = rng.standard_normal((n_queries, dims)).astype(np.float32)
    query_vecs /= np.linalg.norm(query_vecs, axis=1, keepdims=True)

    print(f"{'ef_search':>10} | {'mean_recall':>12} | {'min_recall':>10} | {'<90% queries':>12}")
    print("-" * 55)
    for ef in [20, 40, 64, 100, 200]:
        result = measure_recall(
            conn_str, doc_ids, doc_vectors, query_vecs,
            k=10, ef_search=ef
        )
        print(
            f"{result['ef_search']:>10} | "
            f"{result['mean_recall']:>12.4f} | "
            f"{result['min_recall']:>10.4f} | "
            f"{result['queries_below_90pct']:>12}"
        )

```

Run this script on a representative sample of real query vectors—not random ones—to understand actual production recall. Random vectors do not reflect the clustered distribution of real embeddings from a specific domain, and recall numbers derived from random queries will overestimate real-world accuracy.

In-production recall proxy: result count consistency

You cannot easily run brute-force comparison on every production query, but you can measure an indirect recall proxy: the consistency of result set overlap between two consecutive identical queries at different `ef_search` settings.

```

-- PostgreSQL / pgvector
-- Inline recall check: run the same query at two ef_search levels
-- and measure overlap. Automate this with a scheduled probe query.

```

```

-- Low-fidelity run
SET hnswef_search = 40;
SELECT array_agg(id ORDER BY distance) AS low_fidelity_ids
INTO TEMP low_fi
FROM (
    SELECT id, embedding <=> '[0.021, -0.045, ...]':vector AS distance
    FROM documents
    ORDER BY embedding <=> '[0.021, -0.045, ...]':vector
    LIMIT 20
) s;

-- High-fidelity run (treat as ground truth for this check)
SET hnswef_search = 300;
SELECT array_agg(id ORDER BY distance) AS high_fidelity_ids
INTO TEMP high_fi
FROM (
    SELECT id, embedding <=> '[0.021, -0.045, ...]':vector AS distance
    FROM documents
    ORDER BY embedding <=> '[0.021, -0.045, ...]':vector
    LIMIT 20
) s;
RESET hnswef_search;

-- Compute overlap ratio
SELECT
    (
        SELECT count(*)
        FROM unnest((SELECT low_fidelity_ids FROM low_fi)) AS lf(id)
        WHERE id = ANY((SELECT high_fidelity_ids FROM high_fi))
    )::float / 20 AS recall_proxy;

```

Schedule this probe once per minute against a stable set of known query vectors. If `recall_proxy` drops below 0.85 on a representative set, it's a signal that the index has drifted (for IVFFlat) or that `ef_search` is too low relative to current filter patterns.

14.7 Index Maintenance Scheduling

Unlike B-tree indexes, which tolerate incremental inserts gracefully, both IVFFlat and HNSW indexes have specific maintenance considerations.

HNSW: incremental inserts and graph quality

HNSW supports incremental inserts without a full rebuild. Each new vector is linked into the graph at build time using the current `m` and `ef_construction` parameters. This means the graph quality remains consistent as rows are added—a significant operational advantage. However, deleted rows create dead entries in the graph that are only reclaimed by `VACUUM`. Monitor dead tuple counts to schedule vacuums proactively:

```

-- PostgreSQL / pgvector
-- Check dead tuple accumulation on the documents table
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
    last_vacuum,
    last_autovacuum,
    autovacuum_count
FROM pg_stat_user_tables
WHERE relname = 'documents';

```

```
-- If dead_pct > 5% and autovacuum hasn't run recently, trigger manually
VACUUM (ANALYZE, VERBOSE) documents;
```

For workloads with high delete or update rates (common in RAG pipelines where document chunks are re-ingested), consider lowering `autovacuum_vacuum_scale_factor` for the table:

```
-- PostgreSQL / pgvector
-- Make autovacuum more aggressive for the documents table
ALTER TABLE documents SET (
    autovacuum_vacuum_scale_factor = 0.01,    -- vacuum when 1% of rows are dead
    autovacuum_analyze_scale_factor = 0.005   -- analyze when 0.5% of rows change
);
```

IVFFlat: rebuild scheduling

IVFFlat does not update its cluster centroids as new data arrives. Track the ratio of current rows to rows at index build time, and schedule a `REINDEX CONCURRENTLY` when that ratio exceeds 2:

```
-- PostgreSQL / pgvector
-- Store index build row count for IVFFlat drift tracking
CREATE TABLE IF NOT EXISTS ivfflat_index_registry (
    index_name      text PRIMARY KEY,
    table_name      text NOT NULL,
    build_row_count bigint NOT NULL,
    build_lists     int  NOT NULL,
    built_at        timestampz DEFAULT now()
);

-- Populate at index build time (run immediately after CREATE INDEX)
INSERT INTO ivfflat_index_registry
    (index_name, table_name, build_row_count, build_lists)
SELECT
    'documents_embedding_ivfflat_idx',
    'documents',
    n_live_tup,
    1000 -- match the lists parameter used in CREATE INDEX
FROM pg_stat_user_tables
WHERE relname = 'documents'
ON CONFLICT (index_name) DO UPDATE
    SET build_row_count = EXCLUDED.build_row_count,
        built_at = now();

-- Daily drift check query
SELECT
    r.index_name,
    r.build_row_count,
    s.n_live_tup AS current_rows,
    round(s.n_live_tup::numeric / nullif(r.build_row_count, 0), 2) AS growth_ratio,
    CASE
        WHEN s.n_live_tup > r.build_row_count * 2
        THEN 'REINDEX RECOMMENDED'
        WHEN s.n_live_tup > r.build_row_count * 1.5
        THEN 'REINDEX SOON'
        ELSE 'OK'
    END AS status
FROM ivfflat_index_registry r
JOIN pg_stat_user_tables s ON s.relname = r.table_name;
```

14.8 Putting It Together: A Monitoring Runbook

Effective monitoring requires not just the right queries but a clear protocol for when to run them and what to do with the results. The following runbook covers the three most common investigation triggers: sudden latency increase, recall complaints, and index rebuild after bulk ingestion.

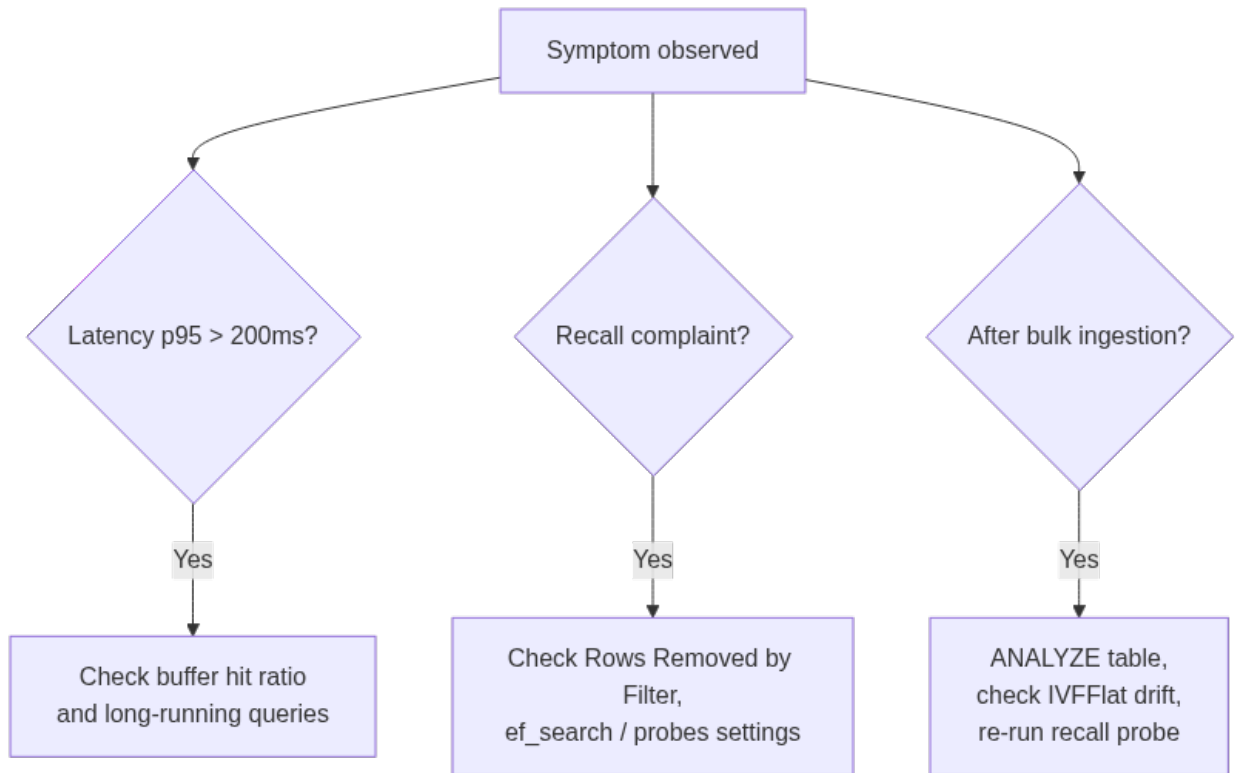


Figure 10: The three most common investigation triggers and their first diagnostic step — detailed queries for each follow below.

Trigger 1: Sudden latency increase (p95 > 200ms)

```

-- Step 1: Confirm the problem is vector queries, not general DB load
SELECT
    wait_event_type,
    wait_event,
    count(*) AS sessions
FROM pg_stat_activity
WHERE state != 'idle'
GROUP BY wait_event_type, wait_event
ORDER BY count(*) DESC;

-- Step 2: Check if buffer hit ratio has dropped
SELECT
    relname,
    round(100.0 * idx_blks_hit /
        nullif(idx_blks_hit + idx_blks_read, 0), 2) AS idx_hit_pct
FROM pg_statio_user_tables
WHERE relname = 'documents';

-- Step 3: Check for long-running vector queries blocking resources
SELECT pid, now() - query_start AS age, state, left(query, 200)
FROM pg_stat_activity
WHERE query ILIKE '%<=>%' AND state != 'idle'
ORDER BY age DESC;

-- Step 4: If idx_hit_pct < 90%, check shared_buffers utilization
SHOW shared_buffers;

-- Step 5: Check for bloated dead tuples causing index scan overhead
SELECT n_dead_tup, n_live_tup FROM pg_stat_user_tables
WHERE relname = 'documents';

```

Trigger 2: Application team reports poor search quality (recall degradation)

```

-- Step 1: Confirm which index is being used
EXPLAIN SELECT id FROM documents
ORDER BY embedding <=> '[0.021, ...]':vector LIMIT 10;

-- Step 2: Check if a filtered query is exhausting ef_search candidates
EXPLAIN (ANALYZE, BUFFERS)
SELECT id FROM documents
WHERE tenant_id = $1
ORDER BY embedding <=> $2::vector LIMIT 10;
-- Look for large "Rows Removed by Filter" count

-- Step 3: Check IVFFlat drift if index type is ivfflat
SELECT * FROM ivfflat_index_registry; -- compare build_row_count to current

-- Step 4: Verify current ef_search / probes settings
SHOW hnsw.ef_search;
SHOW ivfflat.probes;

```

Trigger 3: Post-bulk-ingestion index validation

After loading a large batch of new documents (common in RAG pipeline refresh cycles):

```

-- Step 1: Verify new rows are reflected in statistics
ANALYZE documents;

-- Step 2: Check dead tuple ratio after bulk insert
SELECT n_live_tup, n_dead_tup FROM pg_stat_user_tables
WHERE relname = 'documents';

-- Step 3: For IVFFlat, check growth ratio and reindex if needed
SELECT * FROM ivfflat_index_registry;

-- Step 4: Run the recall probe with current ef_search
-- (use the Python measure_recall() function from section 14.6)

```

```
-- Step 5: If reindex is needed, execute concurrently
REINDEX INDEX CONCURRENTLY documents_embedding_ivfflat_idx;
-- Then update the registry
UPDATE ivfflat_index_registry
SET build_row_count = (
    SELECT n_live_tup FROM pg_stat_user_tables WHERE relname = 'documents'
),
built_at = now()
WHERE index_name = 'documents_embedding_ivfflat_idx';
```

Key Takeaways

- **EXPLAIN ANALYZE is your first tool, but pg_stat_statements is your continuous instrument.** Single-query EXPLAIN captures one execution; aggregate statistics from pg_stat_statements filtered on vector operator patterns (<=>, <#>, <+>) reveal the sustained latency profile and help you identify query variants that are underperforming in production traffic.
 - **The scalar-filter interaction is pgvector’s most common production performance trap.** When a WHERE clause filter is applied as a post-scan recheck on ANN results, a high Rows Removed by Filter count in EXPLAIN output is the diagnostic signal. Mitigation options—partial indexes per filter value, pre-filtering CTEs, or raising ef_search—should be chosen based on filter selectivity and the number of distinct filter values.
 - **IVFFlat indexes drift silently as data grows.** The cluster centroids are fixed at build time, so as the row count grows beyond roughly twice the build-time row count, search quality degrades without any error or warning. Track build-time row counts in a registry table and schedule REINDEX CONCURRENTLY when the growth ratio exceeds 2.
 - **HNSW’s upper graph layers must stay in the buffer cache.** An idx_hit_pct below 90% on the documents table means the HNSW graph is being read from disk on frequent queries, pushing latency well above the single-digit millisecond range that HNSW is capable of. Monitor buffer hit ratios per index, tune shared_buffers, and use the pg_prewarm extension to reload the index into cache after a PostgreSQL restart.
 - **Recall cannot be inferred from latency alone.** A fast query with low ef_search or a drifted IVFFlat index may be returning wrong results without any latency signal. Build an offline recall measurement pipeline using a sample of real production query vectors compared against brute-force ground truth, and complement it with a scheduled in-production recall proxy that compares low- and high-fidelity ANN results on stable probe vectors.
-

Chapter 15: Pinecone: Architecture, Indexing, and Namespaces

Pinecone is the managed vector database that most teams encounter first when building production AI applications. Unlike pgvector, which layers vector search on top of a relational engine you already operate, Pinecone is purpose-built from the ground up for one job: storing high-dimensional vectors and returning approximate nearest neighbors at low latency and high throughput. Understanding how Pinecone works internally—its pod architecture, indexing strategies, namespace model, and operational boundaries—gives you the mental model to make informed decisions about when to use it, how to configure it correctly, and where its constraints will push back against your application requirements. This chapter maps Pinecone’s architecture to concepts you already know from relational database operations, then walks through real implementation patterns for the most common production scenarios.

15.1 Pinecone’s Architecture Through a Database Lens

Before writing a single line of code, it helps to understand what kind of system Pinecone actually is. Database professionals often expect a familiar storage hierarchy: pages, extents, tablespaces, and transaction logs. Pinecone’s architecture looks different, but the underlying concerns—storage, indexing, query routing, and replication—are recognizable once you know where to look.

The Index as the Primary Object

In Pinecone, the fundamental unit of organization is the **index**, not a table or schema. An index holds vectors of a fixed dimension, uses a single distance metric, and has a defined capacity configuration. You cannot change the dimension or metric after creation—this is analogous to choosing a data type for a column, except you cannot alter it later. Plan accordingly.

Each index runs on infrastructure that Pinecone calls **Pods**. Pods are the physical compute and storage units allocated to your index. Think of them loosely like tablespaces with defined IOPS and memory characteristics. The pod type you choose controls query latency, storage capacity, and cost.

Pod Types: The Capacity Model

Pinecone exposes three pod families:

Pod Type	Optimized For	Approximate Capacity (p1)	Approximate QPS
s1	Storage efficiency	5M vectors/pod	~200 QPS
p1	Balanced workloads	1M vectors/pod	~200 QPS
p2	Low latency	1M vectors/pod	~200 QPS with lower p99

Pod types can be suffixed with a size multiplier: `p1.x1`, `p1.x2`, `p1.x4`, `p1.x8`. Each step up doubles memory and storage but also doubles cost. The `s1` pod type uses roughly five times less pod capacity per vector, making it the right choice when you're storing tens of millions of vectors that are not queried at high frequency—essentially a cold tier.

This maps directly to what a DBA knows as capacity planning. You're choosing between storage-optimized vs. compute-optimized instance types, a decision familiar from RDS instance selection. The key difference is that Pinecone makes the tradeoff explicit in the pod type label rather than hiding it behind instance size tiers.

Replicas and High Availability

Replicas in Pinecone behave like read replicas in a traditional RDBMS. Adding replicas increases query throughput and provides redundancy, but does not increase storage capacity. If you need more capacity, you add pods. If you need more QPS, you add replicas. These are orthogonal levers.

Index Capacity Model:

```
Total storage = pods × capacity_per_pod
Total QPS      = replicas × QPS_per_pod
```

A common mistake is adding replicas when you've hit a storage limit. More replicas will not help—you've exhausted pod storage, not query throughput. Pinecone will surface this as `ResourceExhausted` errors on upserts, not on queries.

Serverless Indexes: The Architecture Shift

In 2024, Pinecone introduced **serverless indexes**, which abandon the pod model entirely. Serverless indexes decouple storage from compute: vectors live in object storage (backed by cloud blob stores), and compute resources are provisioned on demand per query. You no longer provision pods, replicas, or capacity multipliers.

The tradeoff is query latency predictability. Pod-based indexes keep vectors in memory or fast NVMe storage, delivering single-digit millisecond p50 latency. Serverless indexes have higher and more variable p99 latency because cold data must be fetched from object storage. For applications that can tolerate 50–200ms query latency and have bursty or unpredictable workloads, serverless significantly reduces operational overhead and cost. For real-time recommendation systems or latency-sensitive retrieval, pod-based indexes remain the right choice.

Treat serverless as you would a provisioned IOPS vs. burst-credit EBS volume decision: great for workloads that don't require sustained low latency, but not a substitute for dedicated resources when latency SLAs are tight.

15.2 Indexing Algorithms: What HNSW Means at the Pinecone Layer

Pinecone uses **HNSW** (Hierarchical Navigable Small World) as its primary indexing algorithm for pod-based indexes, and a proprietary algorithm for serverless indexes. DBAs familiar with B-tree indexes have a strong intuition about how indexes trade write overhead for read performance—HNSW follows the same logic but operates in high-dimensional vector space rather than on sorted keys.

HNSW Recap from a Database Perspective

HNSW builds a multi-layer graph where each node represents a vector. The top layers are sparse long-range connections; the bottom layers are dense short-range connections. A query starts at the top layer, navigates toward the approximate nearest region, then descends through layers for refinement. The analogy is a skip list combined with a proximity graph.

The relevant parameters are:

- **M** — The number of bidirectional connections per node in the graph. Higher M means better recall but more memory. Pinecone sets this internally; you don't tune it directly.
- **ef_construction** — The size of the candidate list during index build. Higher values produce a more accurate graph but slower upsert throughput. Again, managed internally.
- **top_k** — You control this at query time. It's the number of approximate nearest neighbors to return.

What you *do* control in Pinecone is the **top_k** value and metadata filtering, which together govern how aggressively the index traversal is scoped. This is covered in detail in the query patterns section below.

Dimension and Distance Metric Selection

The distance metric is set at index creation and cannot be changed. Pinecone supports:

- **Cosine similarity** — Default for most embedding models. Measures angle between vectors; normalizes for magnitude. Use this unless you have a specific reason not to.
- **Euclidean distance** (`euclidean`) — Measures absolute distance in vector space. Appropriate for spatial data or embeddings where magnitude carries meaning.
- **Dot product** (`dotproduct`) — Measures the inner product. Appropriate for models that output non-normalized embeddings where magnitude is intentional (e.g., some fine-tuned retrieval models).

A common mistake is using dot product with normalized embeddings from OpenAI or Cohere, where cosine similarity is the correct metric. The mathematical result is the same only when vectors are unit-normalized—but relying on this coincidence without understanding it creates subtle bugs when you swap embedding models.

Creating an Index: Pod-Based vs. Serverless

```
# Python
import pinecone
from pinecone import Pinecone, ServerlessSpec, PodSpec

pc = Pinecone(api_key="YOUR_API_KEY")

# Serverless index - recommended for development and moderate production workloads
```

```
pc.create_index(
    name="document-embeddings-serverless",
    dimension=1536,          # Must match your embedding model output dimension
    metric="cosine",
    spec=ServerlessSpec(
        cloud="aws",
        region="us-east-1"
    )
)

# Pod-based index - for production workloads with strict latency requirements
pc.create_index(
    name="document-embeddings-pods",
    dimension=1536,
    metric="cosine",
    spec=PodSpec(
        environment="us-east1-gcp",
        pod_type="p1.x2",    # x2 = 2GB memory per pod, doubles capacity
        pods=2,              # Total storage = 2 × ~1M vectors per p1.x1 × 2 = ~4M vectors
        replicas=2,          # Read replicas for query throughput
        shards=1             # Horizontal partitioning; increase for very large indexes
    )
)

# Verify index status before connecting
import time

while not pc.describe_index("document-embeddings-pods").status["ready"]:
    print("Waiting for index to initialize...")
    time.sleep(5)

print("Index ready.")
```

The `pods`, `replicas`, and `shards` parameters map directly to a familiar infrastructure model: - `pods` = storage capacity units - `replicas` = read scaling factor - `shards` = horizontal partitions for indexes exceeding single-pod capacity

15.3 Upserting Vectors and Working with Metadata

In Pinecone's data model, each record is a **vector** paired with a unique string ID and an optional metadata dictionary. There are no schemas enforced at the database layer—metadata is schemaless JSON, similar to a JSONB column in PostgreSQL where you apply no CHECK constraints. The flexibility is useful but requires application-level discipline.

The Upsert Pattern

Pinecone uses upsert semantics: if a record with the given ID already exists, it is replaced entirely. There is no partial update—if you change one metadata field, you must resubmit the entire record including the full vector. This is analogous to a `REPLACE INTO` in MySQL or a full row replacement via `ON CONFLICT DO UPDATE SET * = EXCLUDED.*`. Plan your data pipelines accordingly, because re-embedding is expensive if done unnecessarily.

```
# Python
import openai
from pinecone import Pinecone

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")

openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")
```

```

def embed_texts(texts: list[str], model: str = "text-embedding-3-small") -> list[list[float]]:
    """
    Batch embed texts using OpenAI.
    text-embedding-3-small outputs 1536 dimensions.
    """
    response = openai_client.embeddings.create(
        input=texts,
        model=model
    )
    return [item.embedding for item in response.data]

# Sample document corpus
documents = [
    {
        "id": "doc-001",
        "text": "PostgreSQL VACUUM reclaims storage from dead tuples created by updates and deletes.",
        "metadata": {
            "source": "postgres-internals",
            "category": "maintenance",
            "doc_type": "technical",
            "created_at": "2024-01-15"
        }
    },
    {
        "id": "doc-002",
        "text": "Index bloat in PostgreSQL accumulates when autovacuum cannot keep pace with write  
↪ load.",
        "metadata": {
            "source": "postgres-internals",
            "category": "performance",
            "doc_type": "technical",
            "created_at": "2024-01-16"
        }
    },
    {
        "id": "doc-003",
        "text": "Pinecone serverless indexes use object storage for vectors with on-demand compute  
↪ scaling.",
        "metadata": {
            "source": "vector-db-guide",
            "category": "architecture",
            "doc_type": "technical",
            "created_at": "2024-02-01"
        }
    }
]

# Generate embeddings in batch (reduces API calls and cost)
texts = [doc["text"] for doc in documents]
embeddings = embed_texts(texts)

# Build upsert batch - Pinecone accepts dicts with id, values, metadata
vectors_to_upsert = [
    {
        "id": doc["id"],
        "values": embedding,
        "metadata": doc["metadata"]
    }
    for doc, embedding in zip(documents, embeddings)
]

# Batch upsert - recommended batch size is 100 vectors per request
# Larger batches risk hitting the 2MB request size limit
upsert_response = index.upsert(
    vectors=vectors_to_upsert,
    namespace="production" # Namespaces are covered in section 15.4
)

print(f"Upserted {upsert_response['upserted_count']} vectors")

```

```
# Verify a vector was stored correctly
fetch_result = index.fetch(ids=["doc-001"], namespace="production")
print(f"Fetch vector ID: {list(fetch_result['vectors'].keys())}")
print(f"Metadata: {fetch_result['vectors']['doc-001']['metadata']}")
```

Metadata Filtering: The Most Important Operational Detail

Metadata filtering in Pinecone behaves differently from what you'd expect coming from SQL WHERE clauses, and misunderstanding this distinction causes serious performance problems in production.

In a SQL database, a filtered query touches only the rows that match the WHERE clause. Pinecone's metadata filter is applied **after** the approximate nearest neighbor search narrows the candidate set. The filter reduces the result set from the ANN results, it does not direct the ANN traversal (except in the specialized case of Pinecone's indexed metadata for serverless, which is handled differently).

This means that if your filter is highly selective—say, filtering to a `tenant_id` that matches only 0.1% of your vectors—the ANN search still traverses the full index, and the post-filter step discards most candidates. You end up retrieving far fewer results than `top_k` requested, and in extreme cases, you get zero results even when matching vectors exist.

The operational rule of thumb: **metadata filters work well when they match at least 10–20% of the index.** Below that threshold, increase `top_k` to compensate, or consider using namespaces for true data isolation (see Section 15.4).

```
# Python
# Querying with metadata filters

query_text = "How does PostgreSQL handle dead tuples and storage reclamation?"
query_embedding = embed_texts([query_text])[0]

# Basic query without filter
results = index.query(
    vector=query_embedding,
    top_k=5,
    namespace="production",
    include_metadata=True
)

print("=== Unfiltered Results ===")
for match in results["matches"]:
    print(f" ID: {match['id']}, Score: {match['score']:.4f}, "
          f"Category: {match['metadata'].get('category', 'N/A')}")

# Filtered query - metadata filter uses MongoDB-style operators
results_filtered = index.query(
    vector=query_embedding,
    top_k=5,
    namespace="production",
    include_metadata=True,
    filter={
        "category": {"$in": ["maintenance", "performance"]},
        "doc_type": {"$eq": "technical"}
    }
)

print("\n=== Filtered Results (maintenance or performance, technical docs) ===")
for match in results_filtered["matches"]:
    print(f" ID: {match['id']}, Score: {match['score']:.4f}, "
          f"Category: {match['metadata'].get('category', 'N/A')}")

# Supported filter operators:
```


# \$eq, \$ne	- equality, inequality
# \$gt, \$gte	- greater than comparisons (numeric fields)
# \$lt, \$lte	- less than comparisons (numeric fields)
# \$in, \$nin	- membership in list
# \$and, \$or	- logical combinators

Metadata Storage Limits

Pinecone enforces a 40KB metadata limit per vector. This is rarely a problem for standard metadata fields (strings, numbers, small arrays), but it matters if you're tempted to store the full document text in metadata for retrieval. The practice of storing full text in Pinecone metadata is common but wrong at scale—you're using a vector index as a key-value store for blob data it was not designed to hold. Store document text in your primary database (PostgreSQL, a document store, or object storage), and use Pinecone's IDs as foreign keys to retrieve it. This pattern is covered in depth in Chapter 22 on RAG pipelines.

15.4 Namespaces: Multi-Tenancy Without Schema Separation

Namespaces are one of Pinecone's most useful features for production applications, and one of the most frequently misused. A namespace is a logical partition within an index. Vectors in different namespaces are completely isolated from each other—a query targeting `namespace="tenant-a"` will never return vectors from `namespace="tenant-b"`, regardless of similarity scores.

The Database Analogy

Namespaces map most closely to **schemas** in PostgreSQL or **databases** within a SQL Server instance—they provide logical separation within a shared storage engine. Unlike schemas, you don't need to define namespaces in advance. They are created implicitly when you first upsert a vector into them and destroyed when they become empty.

The critical distinction from schemas: namespaces share the index's pod capacity. You cannot assign different pod types or capacity budgets to different namespaces. If `tenant-a` stores 900,000 vectors and the index capacity is 1,000,000, there are only 100,000 slots remaining across all other namespaces. This is capacity pooling, not isolation.

Multi-Tenant Application Pattern

The canonical use case for namespaces is multi-tenant applications where each tenant's data must be queryable only in the context of that tenant. This is more reliable than metadata filtering for tenant isolation because it is enforced at the storage layer, not the application query layer.

```
# Python
# Multi-tenant vector upsert and query using namespaces

from pinecone import Pinecone
import openai

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

def get_embedding(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=[text],
        model="text-embedding-3-small"
    )
```

```

    return response.data[0].embedding

# ---- Tenant A: Legal firm document corpus ----
tenant_a_docs = [
    {"id": "legal-001", "text": "Force majeure clauses excuse contract non-performance during
↳ extraordinary events."},
    {"id": "legal-002", "text": "Indemnification provisions transfer liability risk between contracting
↳ parties."},
]

tenant_a_vectors = [
    {"id": doc["id"], "values": get_embedding(doc["text"]), "metadata": {"type": "contract"}}
    for doc in tenant_a_docs
]

index.upsert(vectors=tenant_a_vectors, namespace="tenant-legal-firm-a")

# ---- Tenant B: Medical records system ----
tenant_b_docs = [
    {"id": "med-001", "text": "Systolic blood pressure above 140 mmHg indicates stage 2 hypertension."},
    {"id": "med-002", "text": "HbA1c levels above 6.5% are diagnostic criteria for type 2 diabetes."},
]

tenant_b_vectors = [
    {"id": doc["id"], "values": get_embedding(doc["text"]), "metadata": {"type": "clinical"}}
    for doc in tenant_b_docs
]

index.upsert(vectors=tenant_b_vectors, namespace="tenant-medical-system-b")

# ---- Namespace-isolated queries ----
def search_tenant(tenant_namespace: str, query: str, top_k: int = 3):
    """
    Query is strictly scoped to the tenant's namespace.
    No cross-tenant data leakage regardless of query content.
    """
    query_vector = get_embedding(query)
    results = index.query(
        vector=query_vector,
        top_k=top_k,
        namespace=tenant_namespace,
        include_metadata=True
    )
    return results["matches"]

# This query runs only against tenant A's data
legal_results = search_tenant(
    "tenant-legal-firm-a",
    "What contract clauses address unexpected events beyond control?"
)
print("Legal firm results:", [r["id"] for r in legal_results])

# Same query against tenant B - will return medical data, not legal
medical_results = search_tenant(
    "tenant-medical-system-b",
    "What contract clauses address unexpected events beyond control?"
)
print("Medical system results:", [r["id"] for r in medical_results])
# Output: medical vectors closest to the query in B's namespace

# Namespace statistics - how many vectors per namespace
index_stats = index.describe_index_stats()
print("\nVectors per namespace:")
for ns, stats in index_stats.get("namespaces", {}).items():
    print(f"  {ns}: {stats['vector_count']} vectors")

```

Namespace Deletion and Maintenance

Deleting all vectors from a namespace effectively destroys it. There is no explicit DROP NAMESPACE command—the namespace simply ceases to exist when empty. For tenant offboarding, you can delete by namespace directly:

```
# Python
# Delete all vectors within a namespace - equivalent to DROP SCHEMA CASCADE
index.delete(delete_all=True, namespace="tenant-legal-firm-a")

# Delete specific vectors within a namespace
index.delete(
    ids=["legal-001", "legal-002"],
    namespace="tenant-legal-firm-a"
)

# List all namespaces currently in use
stats = index.describe_index_stats()
active_namespaces = list(stats.get("namespaces", {}).keys())
print(f"Active namespaces: {active_namespaces}")
```

When Namespaces Are Not Enough

Namespaces provide logical isolation but share physical resources. For strict compliance requirements (GDPR right-to-erasure with audit proof, HIPAA physical data isolation), namespaces within a shared index are generally insufficient. In those cases, use **separate indexes per tenant**, accepting the higher cost. A single-tenant index on a p1.x1 pod has a cost per tenant that varies by Pinecone's current pricing model—viable for enterprise SaaS but may be prohibitive for consumer-scale applications. Verify current costs with Pinecone's pricing documentation.

The namespace vs. separate-index decision follows the same logic as shared vs. dedicated database servers for tenant isolation in traditional SaaS architectures.

15.5 Production Operations: Monitoring, Scaling, and Failure Patterns

Running Pinecone in production requires operational discipline that mirrors what you'd apply to any managed database service: understand your capacity headroom, monitor query latency percentiles, and have runbooks for failure modes before they occur.

Index Statistics and Health Monitoring

```
# Python
# Comprehensive index monitoring - suitable for integration into Datadog,
# CloudWatch custom metrics, or a Prometheus exporter

from pinecone import Pinecone
import time

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-pods")

def collect_index_metrics(index_name: str) -> dict:
    """
    Collect key operational metrics from a Pinecone index.
    Call this on a schedule (e.g., every 60 seconds) and push to your monitoring system.
    """
    pc_client = Pinecone(api_key="YOUR_API_KEY")
    idx = pc_client.Index(index_name)

    # describe_index_stats returns vector count and namespace breakdown
```

```
stats = idx.describe_index_stats()

# describe_index returns pod config, status, and replicas
index_config = pc_client.describe_index(index_name)

total_vectors = stats.get("total_vector_count", 0)
dimension = stats.get("dimension", 0)

# Capacity utilization - critical for pod-based indexes
# p1.x1 holds ~1M vectors; x2 = ~2M, x4 = ~4M, x8 = ~8M
pod_type = index_config.spec.pod.pod_type if hasattr(index_config.spec, 'pod') else "serverless"
pods = index_config.spec.pod.pods if hasattr(index_config.spec, 'pod') else None
replicas = index_config.spec.pod.replicas if hasattr(index_config.spec, 'pod') else None

namespace_stats = {
    ns: data["vector_count"]
    for ns, data in stats.get("namespaces", {}).items()
}

return {
    "index_name": index_name,
    "total_vectors": total_vectors,
    "dimension": dimension,
    "pod_type": pod_type,
    "pods": pods,
    "replicas": replicas,
    "namespace_vector_counts": namespace_stats,
    "collected_at": time.time()
}

def estimate_capacity_utilization(metrics: dict) -> float:
    """
    Estimate percentage of pod capacity consumed.
    Useful for alerting before you hit ResourceExhausted errors.
    Capacity estimates are approximate - refer to Pinecone documentation
    for current per-pod limits.
    """
    pod_type = metrics.get("pod_type", "")
    pods = metrics.get("pods", 1) or 1
    total_vectors = metrics.get("total_vectors", 0)

    # Approximate per-pod capacities by pod family
    # These change as Pinecone updates infrastructure - treat as guidelines
    per_pod_capacity = {
        "p1": 1_000_000,
        "p2": 1_000_000,
        "s1": 5_000_000,
    }

    pod_family = pod_type.split(".")[0] if pod_type else "p1"
    base_capacity = per_pod_capacity.get(pod_family, 1_000_000)

    # x2, x4, x8 multipliers double/quadruple capacity
    multiplier = 1
    if "." in pod_type:
        suffix = pod_type.split(".")[1] # e.g., "x2"
        if suffix.startswith("x"):
            try:
                multiplier = int(suffix[1:])
            except ValueError:
                multiplier = 1

    total_capacity = base_capacity * multiplier * pods
    return (total_vectors / total_capacity) * 100 if total_capacity > 0 else 0.0

# Example usage
metrics = collect_index_metrics("document-embeddings-pods")
utilization = estimate_capacity_utilization(metrics)
```

```
print(f"Index: {metrics['index_name']}")
print(f"Vectors: {metrics['total_vectors']:,}")
print(f"Pod type: {metrics['pod_type']}, Pods: {metrics['pods']}, Replicas: {metrics['replicas']}")
print(f"Estimated capacity utilization: {utilization:.1f}%")

if utilization > 80:
    print("WARNING: Index approaching capacity. Plan pod upgrade or horizontal expansion.")
elif utilization > 95:
    print("CRITICAL: Index near capacity. Upserts will fail with ResourceExhausted.")
```

Latency Measurement Patterns

Pinecone does not expose p99 query latency directly through its Python client—that telemetry lives in your own instrumentation. Wrap every query call with a timing decorator and feed the results into your observability stack.

```
# Python
import time
import statistics
from functools import wraps
from collections import deque

# Simple sliding-window latency tracker
class LatencyTracker:
    def __init__(self, window_size: int = 1000):
        self.latencies = deque(maxlen=window_size)

    def record(self, latency_ms: float):
        self.latencies.append(latency_ms)

    def percentile(self, p: float) -> float:
        if not self.latencies:
            return 0.0
        sorted_latencies = sorted(self.latencies)
        idx = int(len(sorted_latencies) * p / 100)
        return sorted_latencies[min(idx, len(sorted_latencies) - 1)]

    def report(self) -> dict:
        if not self.latencies:
            return {}
        return {
            "p50_ms": self.percentile(50),
            "p95_ms": self.percentile(95),
            "p99_ms": self.percentile(99),
            "mean_ms": statistics.mean(self.latencies),
            "sample_count": len(self.latencies),
        }

query_latency = LatencyTracker(window_size=500)

def timed_query(index, vector, top_k, namespace, **kwargs):
    """
    Execute a Pinecone query and record wall-clock latency.
    Use this wrapper everywhere instead of calling index.query() directly.
    """
    start = time.perf_counter()
    result = index.query(
        vector=vector,
        top_k=top_k,
        namespace=namespace,
        **kwargs
    )
    elapsed_ms = (time.perf_counter() - start) * 1000
    query_latency.record(elapsed_ms)
    return result
```

```
# After a load test or production traffic window:
print("Query latency report:", query_latency.report())
```

Common Failure Modes and Mitigations

Failure Mode	Cause	Mitigation
<code>ResourceExhausted</code> on upsert	Index at pod capacity	Upgrade pod size or add pods before hitting 90%
Fewer results than <code>top_k</code>	Metadata filter too selective	Increase <code>top_k</code> ; consider namespace isolation
Stale vectors after upsert	Pinecone indexes asynchronously	Add read-your-writes wait loop; see below
High p99 latency spikes	Replica saturation	Add replicas; or switch to pod type with better cache behavior
400 on dimension mismatch	Embedding model changed	Validate vector dimensions before every upsert batch
Timeout on large batch upsert	Request body exceeds 2MB	Limit batch to 100 vectors; reduce metadata payload size

Read-Your-Writes Consistency

Pinecone is eventually consistent on write paths. An upsert call that returns success does not guarantee the vector is immediately queryable. For workflows that require confirming vector availability after upsert—such as in integration tests or post-ingest verification jobs—poll `describe_index_stats` until the vector count increments, or use a small polling loop on `fetch`.

```
# Python
# Wait for a vector to become queryable after upsert
def wait_for_vector(index, vector_id: str, namespace: str,
                    timeout_seconds: int = 30, poll_interval: float = 0.5):
    """
    Poll until a vector with the given ID is fetchable.
    Useful in tests and post-ingest verification pipelines.
    Not recommended for hot paths - adds latency.
    """
    deadline = time.time() + timeout_seconds
    while time.time() < deadline:
        result = index.fetch(ids=[vector_id], namespace=namespace)
        if vector_id in result.get("vectors", {}):
            return True
        time.sleep(poll_interval)
    raise TimeoutError(
        f"Vector '{vector_id}' not visible in namespace '{namespace}' "
        f"after {timeout_seconds}s"
    )

# Usage: after upserting a critical document, verify before proceeding
index.upsert(
    vectors=[{"id": "critical-doc-001", "values": get_embedding("important text"), "metadata": {}},
    namespace="production"
)
wait_for_vector(index, "critical-doc-001", "production")
print("Vector confirmed queryable.")
```

Scaling Playbook: When to Take Each Action

Database professionals expect a capacity scaling decision matrix. Here is the equivalent for Pinecone pod-based indexes:

Observed Symptom	Diagnosis	Action
ResourceExhausted on upsert	Pod storage full	Scale pod size (x1 → x2 → x4) or add pods
Rising p99 query latency	Replica throughput saturated	Add replicas
top_k results consistently short	Filter selectivity < 10%	Increase top_k or move to namespaces
Upsert throughput degrading	Saturating pod write path	Increase batch concurrency or pod count
Cold start latency spikes (serverless)	Object storage fetch on cold data	Migrate hot data to pod-based index
High cost with low query rate	Over-provisioned pods	Migrate to serverless for cost reduction

15.6 Advanced Patterns: Hybrid Search and Re-Ranking

Pinecone’s core operation is approximate nearest neighbor search over dense vectors. However, most production retrieval systems need more than raw ANN scores—they need the ability to blend keyword relevance with semantic similarity, and to re-rank initial retrieval results with a more expensive model before returning the final set.

Sparse-Dense Hybrid Search

Pinecone supports **sparse-dense hybrid search** on pod-based indexes using the `dotproduct` metric. Sparse vectors represent keyword or BM25-style term-frequency signals; dense vectors represent semantic embeddings. Combining them lets you retrieve results that are both keyword-relevant and semantically similar—the same goal that Elasticsearch’s Reciprocal Rank Fusion (RRF) achieves at the query rewrite layer.

The architecture requires generating both a dense embedding and a sparse vector for each document at index time, then combining them at query time with an `alpha` weighting parameter.

```
# Python
# Sparse-dense hybrid search
# Requires: pod-based index with dotproduct metric and sparse vector support

from pinecone import Pinecone
from pinecone_text.sparse import BM25Encoder
import openai

pc = Pinecone(api_key="YOUR_API_KEY")

# Create an index that supports sparse vectors
# Note: hybrid search requires dotproduct metric and pod-based index
pc.create_index(
    name="hybrid-search-index",
    dimension=1536,
    metric="dotproduct",
    spec=PodSpec(
        environment="us-east1-gcp",
        pod_type="p1.x1",
```

```

        pods=1
    )
)

index = pc.Index("hybrid-search-index")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

# BM25 encoder - fit on your document corpus for accurate IDF weights
# In production, fit once on the full corpus, serialize, and reuse
bm25 = BM25Encoder()

documents = [
    "PostgreSQL VACUUM reclaims storage from dead tuples created by updates and deletes.",
    "Index bloat in PostgreSQL accumulates when autovacuum cannot keep pace with write load.",
    "BM25 is a term frequency ranking function used in full-text search systems.",
]

# Fit BM25 on corpus (computes IDF weights)
bm25.fit(documents)

# Encode sparse vectors for each document
sparse_doc_vectors = bm25.encode_documents(documents)

# Encode dense vectors for each document
dense_response = openai_client.embeddings.create(
    input=documents,
    model="text-embedding-3-small"
)
dense_doc_vectors = [item.embedding for item in dense_response.data]

# Upsert hybrid vectors
hybrid_vectors = []
for i, (dense, sparse) in enumerate(zip(dense_doc_vectors, sparse_doc_vectors)):
    hybrid_vectors.append({
        "id": f"hybrid-doc-{i:03d}",
        "values": dense,
        "sparse_values": {
            "indices": sparse["indices"],
            "values": sparse["values"]
        },
        "metadata": {"text": documents[i]}
    })

index.upsert(vectors=hybrid_vectors, namespace="hybrid")

# Hybrid query with alpha blending
# alpha = 1.0 → pure dense search
# alpha = 0.0 → pure sparse (keyword) search
# alpha = 0.5 → equal blend

def hybrid_query(query_text: str, alpha: float = 0.7, top_k: int = 5):
    """
    Execute a hybrid sparse-dense query.
    alpha controls the balance: higher = more semantic, lower = more keyword.
    """
    # Encode query as both dense and sparse
    dense_query = openai_client.embeddings.create(
        input=[query_text],
        model="text-embedding-3-small"
    ).data[0].embedding

    sparse_query = bm25.encode_queries(query_text)

    # Scale sparse and dense components by alpha
    def scale_sparse(sparse_vec, scale: float):
        return {
            "indices": sparse_vec["indices"],
            "values": [v * scale for v in sparse_vec["values"]]
        }

```



```

def scale_dense(dense_vec, scale: float):
    return [v * scale for v in dense_vec]

results = index.query(
    vector=scale_dense(dense_query, alpha),
    sparse_vector=scale_sparse(sparse_query, 1 - alpha),
    top_k=top_k,
    namespace="hybrid",
    include_metadata=True
)
return results["matches"]

# Compare pure semantic vs. hybrid results
print("=== Pure Semantic Search (alpha=1.0) ===")
semantic_results = hybrid_query("dead rows storage reclamation", alpha=1.0)
for r in semantic_results:
    print(f" {r['id']}: {r['score']:.4f} - {r['metadata']['text'][:60]}...")

print("\n=== Hybrid Search (alpha=0.7, 30% BM25) ===")
hybrid_results = hybrid_query("dead rows storage reclamation", alpha=0.7)
for r in hybrid_results:
    print(f" {r['id']}: {r['score']:.4f} - {r['metadata']['text'][:60]}...")

```

Re-Ranking with a Cross-Encoder

Approximate nearest neighbor retrieval returns a candidate set quickly, but ANN scores are coarse signals. Cross-encoder re-ranking runs a more expensive model over the (query, candidate) pairs to produce precise relevance scores. The pattern is always two-stage:

1. **Retrieval** (fast, approximate): Pinecone returns top-100 candidates.
2. **Re-ranking** (slower, precise): Cross-encoder scores each candidate; return top-10.

```

# Python
# Two-stage retrieval and cross-encoder re-ranking

from sentence_transformers import CrossEncoder
from pinecone import Pinecone
import openai

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

# Cross-encoder from sentence-transformers - runs locally
# ms-marco-MiniLM-L-6-v2 is a well-validated choice for passage reranking
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def two_stage_retrieval(
    query: str,
    namespace: str,
    retrieve_k: int = 50,
    rerank_top_k: int = 5
) -> list[dict]:
    """
    Stage 1: Retrieve retrieve_k candidates from Pinecone using ANN.
    Stage 2: Re-rank with cross-encoder and return rerank_top_k results.

    retrieve_k should be at least 5-10× rerank_top_k to give the
    cross-encoder enough candidates to work with.
    """
    # Stage 1: ANN retrieval
    query_embedding = openai_client.embeddings.create(
        input=[query],
        model="text-embedding-3-small"
    ).data[0].embedding

```

```
ann_results = index.query(
    vector=query_embedding,
    top_k=retrieve_k,
    namespace=namespace,
    include_metadata=True
)

candidates = ann_results["matches"]
if not candidates:
    return []

# Stage 2: Cross-encoder re-ranking
# Cross-encoder expects (query, passage) pairs
# Requires document text to be stored in metadata or fetched from primary DB
pairs = [
    (query, candidate["metadata"].get("text", ""))
    for candidate in candidates
]

# Predict returns a score per pair - higher is more relevant
rerank_scores = cross_encoder.predict(pairs)

# Attach rerank scores and sort descending
for candidate, score in zip(candidates, rerank_scores):
    candidate["rerank_score"] = float(score)

reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
return reranked[:rerank_top_k]

# Example usage
query = "How does PostgreSQL reclaim space from deleted rows?"
final_results = two_stage_retrieval(
    query=query,
    namespace="production",
    retrieve_k=20,
    rerank_top_k=3
)

print(f"Query: {query}\n")
print("Re-ranked results:")
for i, result in enumerate(final_results, 1):
    print(f"  {i}. [{result['rerank_score']:.3f}] {result['id']}: "
          f"{result['metadata'].get('text', '')[:80]}...")
```

The cross-encoder step runs in the application tier—it has no dependency on Pinecone and can be parallelized. For latency-sensitive applications, run the cross-encoder inference on GPU or use a hosted re-ranking API (Cohere’s Rerank API provides this as a managed service) rather than the CPU-bound local model.

15.7 Integrating Pinecone with Your Data Stack

Pinecone does not exist in isolation. In production architectures, it sits alongside a primary relational database, an event streaming layer, and an embedding generation service. Getting the integration topology right determines whether your vector search layer stays consistent with the rest of your data.

The Canonical Integration Topology

The pattern is clear: Pinecone stores vectors and returns IDs. PostgreSQL stores authoritative text and structured data. Your application joins the two at query time using the IDs Pinecone returns.

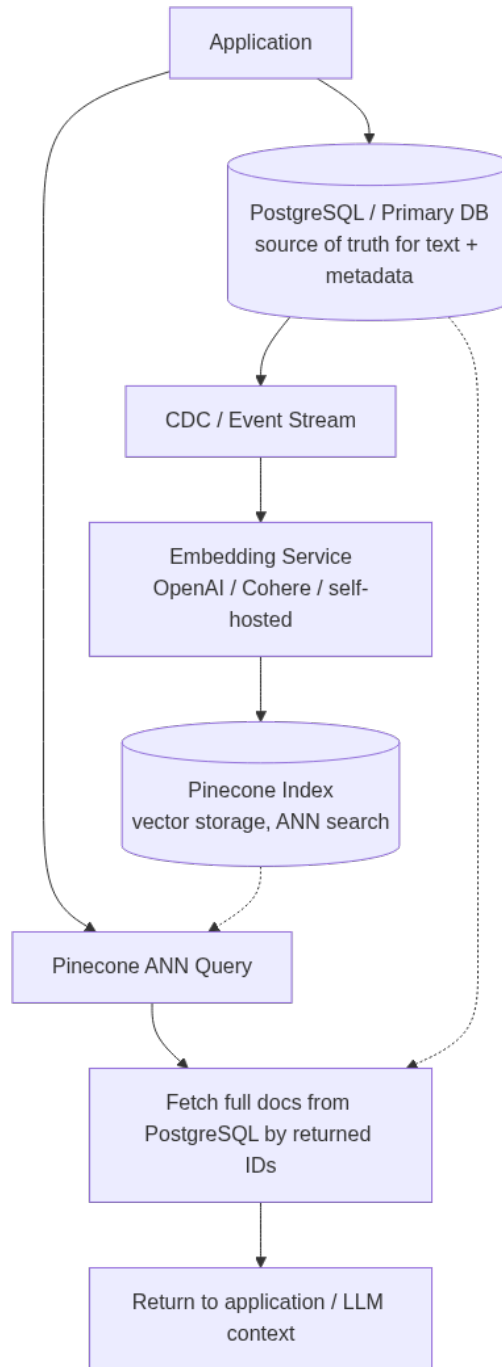


Figure 11: Write path (left): PostgreSQL changes flow through CDC into the embedding service and land in Pinecone. Read path (right): the application queries Pinecone for nearest-neighbor IDs, then fetches authoritative text from PostgreSQL.

Never treat Pinecone’s metadata as the sole copy of document content.

Keeping Pinecone Synchronized with PostgreSQL

The most common production headache with Pinecone is index drift: records updated or deleted in PostgreSQL that still exist (stale) in Pinecone. Address this with one of three patterns, in order of reliability:

Pattern 1: Transactional outbox with CDC

Write changes to an outbox table in PostgreSQL in the same transaction as the business record change. A downstream consumer (Debezium, pglogical, or a custom pg_notify listener) reads the outbox and triggers re-embedding + Pinecone upsert. Deletes in PostgreSQL produce a tombstone event that triggers `index.delete()`.

```
-- PostgreSQL
-- Outbox table for Pinecone sync events
CREATE TABLE pinecone_sync_queue (
    id                BIGSERIAL PRIMARY KEY,
    operation          TEXT NOT NULL,      -- 'upsert' or 'delete'
    document_id        TEXT NOT NULL,
    namespace          TEXT NOT NULL DEFAULT 'default',
    payload            JSONB,              -- document text and metadata for upsert
    created_at         TIMESTAMPTZ DEFAULT NOW(),
    processed_at       TIMESTAMPTZ
);

CREATE INDEX idx_pinecone_sync_unprocessed
ON pinecone_sync_queue (created_at)
WHERE processed_at IS NULL;

-- Trigger function: queue a Pinecone sync event on document change
CREATE OR REPLACE FUNCTION queue_pinecone_sync()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO pinecone_sync_queue (operation, document_id, namespace)
        VALUES ('delete', OLD.id::TEXT, 'production');
        RETURN OLD;
    ELSE
        INSERT INTO pinecone_sync_queue (operation, document_id, namespace, payload)
        VALUES (
            'upsert',
            NEW.id::TEXT,
            'production',
            jsonb_build_object(
                'text', NEW.content,
                'metadata', jsonb_build_object(
                    'category', NEW.category,
                    'created_at', NEW.created_at,
                    'doc_type', NEW.doc_type
                )
            )
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Attach trigger to document table
CREATE TRIGGER trg_queue_pinecone_sync
AFTER INSERT OR UPDATE OR DELETE ON documents
FOR EACH ROW EXECUTE FUNCTION queue_pinecone_sync();

# Python
# Outbox consumer - runs as a background worker or scheduled job
```

```
import time
import psycopg2
from pinecone import Pinecone
import openai

pc = Pinecone(api_key="YOUR_API_KEY")
index = pc.Index("document-embeddings-serverless")
openai_client = openai.OpenAI(api_key="YOUR_OPENAI_KEY")

def process_sync_queue(pg_conn, batch_size: int = 50):
    """
    Fetch unprocessed events from the outbox, execute Pinecone operations,
    then mark events as processed. Runs in a polling loop.
    """
    with pg_conn.cursor() as cur:
        # Fetch a batch of unprocessed events, lock them to avoid double-processing
        cur.execute("""
            SELECT id, operation, document_id, namespace, payload
            FROM pinecone_sync_queue
            WHERE processed_at IS NULL
            ORDER BY created_at
            LIMIT %s
            FOR UPDATE SKIP LOCKED
        """, (batch_size,))

        events = cur.fetchall()
        if not events:
            return 0

        upsert_batch = []
        delete_ids_by_namespace = {}

        for event_id, operation, document_id, namespace, payload in events:
            if operation == 'delete':
                delete_ids_by_namespace.setdefault(namespace, []).append(document_id)

            elif operation == 'upsert' and payload:
                text = payload.get("text", "")
                metadata = payload.get("metadata", {})

                # Generate embedding for the document
                embedding = openai_client.embeddings.create(
                    input=[text],
                    model="text-embedding-3-small"
                ).data[0].embedding

                upsert_batch.append({
                    "id": document_id,
                    "values": embedding,
                    "metadata": {**metadata, "text": text},
                    "_namespace": namespace # Store namespace for grouping
                })

        # Execute upserts grouped by namespace
        upserts_by_ns = {}
        for vec in upsert_batch:
            ns = vec.pop("_namespace")
            upserts_by_ns.setdefault(ns, []).append(vec)

        for ns, vectors in upserts_by_ns.items():
            index.upsert(vectors=vectors, namespace=ns)

        # Execute deletes grouped by namespace
        for ns, ids in delete_ids_by_namespace.items():
            index.delete(ids=ids, namespace=ns)

        # Mark events as processed
        processed_ids = [e[0] for e in events]
        cur.execute("""
            UPDATE pinecone_sync_queue
```

```
        SET processed_at = NOW()
        WHERE id = ANY(%s)
    """ % (processed_ids,)

    pg_conn.commit()
    return len(events)

# Polling loop - in production, deploy as a dedicated service with retry logic
pg_conn = psycopg2.connect("postgresql://user:password@host:5432/dbname")

while True:
    processed = process_sync_queue(pg_conn, batch_size=50)
    if processed == 0:
        time.sleep(2) # Back off when queue is empty
    else:
        print(f"Processed {processed} sync events")
```

Pattern 2: Application-layer dual write

On every create/update/delete operation in your application, write to PostgreSQL first, then synchronously upsert/delete from Pinecone. Simpler than the outbox pattern but introduces a failure mode: if the Pinecone call fails after the PostgreSQL commit, your systems diverge. Acceptable for low-stakes applications; not recommended for any dataset where consistency matters.

Pattern 3: Periodic full re-index

For datasets that change infrequently—large document corpora where new versions are published in batches—a nightly full re-index is often the most operationally straightforward approach. Export all documents from PostgreSQL, batch-embed them, and upsert into Pinecone. Stale data exists for at most one index cycle, which is acceptable for many knowledge base applications.

The three patterns represent the classic reliability vs. complexity tradeoff: outbox CDC is the most reliable but operationally heaviest; full re-index is the simplest but least timely.

Key Takeaways

- **Pinecone’s capacity model maps to familiar infrastructure concepts:** pods correspond to storage units (analogous to tablespace capacity), replicas correspond to read scaling (analogous to read replicas), and shards provide horizontal partitioning. Understanding which lever to pull—pods for storage, replicas for throughput—prevents the common mistake of adding replicas when you’ve hit a storage limit.
- **Namespaces provide logical tenant isolation enforced at the query layer,** not at the storage hardware layer. They are the right choice for multi-tenant applications where data privacy between tenants is required at the application level, but not a substitute for separate indexes when regulatory compliance demands physical data isolation (HIPAA, GDPR right-to-erasure audit trails).
- **Metadata filtering operates post-ANN, not pre-ANN,** which means highly selective filters on large indexes return fewer results than `top_k` requests. The practical threshold is that filters should match at least 10–20% of the index to produce reliable result sets. For cardinality-based isolation below that threshold, namespace separation is more reliable than metadata filtering.

- **Serverless and pod-based indexes represent fundamentally different tradeoff profiles:** serverless eliminates capacity provisioning overhead and is cost-efficient for bursty or moderate workloads, but trades away the predictable low-latency (single-digit millisecond p50) that pod-based indexes deliver. Choose based on your application’s latency SLA, not on index size alone.
 - **Keeping Pinecone synchronized with your primary database requires an explicit consistency strategy.** The transactional outbox pattern—queuing sync events in PostgreSQL within the same transaction as business record changes—is the most robust approach. Application-layer dual writes are simpler but introduce a divergence window on partial failures, and periodic full re-indexing is appropriate only when data freshness requirements allow for batch-cycle latency.
-

Chapter 16: Weaviate: Schema Design and Hybrid Search

Weaviate occupies a distinct position in the vector database landscape: it ships with built-in vectorization modules, a graph-inspired object model, and first-class hybrid search that combines dense vector retrieval with BM25 keyword scoring — all within a single system that requires no external embedding pipeline at insert time. For database engineers, the learning curve is less about grasping vector concepts and more about accepting a schema model that sits somewhere between a document store and a relational system, with a query language (GraphQL) that rewards understanding before it rewards productivity. This chapter walks through Weaviate’s schema design philosophy, its class-and-property model, practical hybrid search configuration, multi-tenancy, and the operational patterns that make Weaviate viable at production scale.

16.1 Weaviate’s Data Model: Classes, Properties, and the Object Graph

Weaviate organizes data into **classes** — the rough equivalent of tables in a relational database, or collections in MongoDB. Each class holds **objects** that carry scalar properties plus an optional vector. Unlike most vector databases where you bring your own vector, Weaviate can generate vectors automatically using a configured vectorizer module (OpenAI, Cohere, sentence-transformers via its `text2vec-transformers` module, and others). The vector becomes a first-class citizen of the object, not an add-on.

The class definition is the schema contract. Every class declaration specifies:

- The vectorizer module (or `none` if you supply vectors yourself)
- Per-property data types (`text`, `int`, `number`, `boolean`, `date`, `uuid`, cross-references)
- Indexing behavior per property (BM25 inverted index, filterable, sortable)
- Vector index configuration (HNSW parameters, quantization)
- Module-specific settings (which model, tokenization strategy)

The **cross-reference** system is Weaviate’s most database-distinct feature. A property can reference objects in another class, creating a directed relationship that can be traversed in GraphQL queries. Think of it as a foreign key with graph traversal semantics — closer to a JOIN than a document embed, but resolved lazily at query time.

Tokenization matters more than it looks. For BM25 indexing, each `text` property has a

tokenization setting: `word`, `lowercase`, `whitespace`, or `field`. The choice affects how terms are indexed and matched. Product codes, URLs, and structured identifiers should use `field` (exact match) or `whitespace`. Natural language content should use `word` (default). Getting this wrong produces silent retrieval failures — your keyword search simply misses matches because the indexed tokens don't align with the query tokens.

Here is a realistic schema definition for a document retrieval system using the Weaviate Python client v4:

```
# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import (
    Configure,
    Property,
    DataType,
    Tokenization,
    VectorDistances,
)

client = weaviate.connect_to_local() # or connect_to_weaviate_cloud(...)

# Define a DocumentChunk class with OpenAI vectorizer
client.collections.create(
    name="DocumentChunk",
    description="Chunked document passages for RAG retrieval",
    vectorizer_config=Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-small",
        dimensions=1536,
    ),
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        ef_construction=128,
        max_connections=16,
        ef=64,
        dynamic_ef_factor=8,
        dynamic_ef_min=100,
        dynamic_ef_max=500,
    ),
    properties=[
        Property(
            name="content",
            data_type=DataType.TEXT,
            description="The chunk text content",
            tokenization=Tokenization.WORD,
            vectorize_property_name=False,
            skip_vectorization=False,
        ),
        Property(
            name="source_document",
            data_type=DataType.TEXT,
            tokenization=Tokenization.FIELD,
            skip_vectorization=True,
        ),
        Property(
            name="chunk_index",
            data_type=DataType.INT,
            skip_vectorization=True,
        ),
        Property(
            name="created_at",
            data_type=DataType.DATE,
            skip_vectorization=True,
        ),
        Property(
            name="metadata_tags",
            data_type=DataType.TEXT_ARRAY,

```

```

        tokenization=Tokenization.WORD,
        skip_vectorization=True,
    ),
],
# Enable inverted index for BM25 hybrid search
inverted_index_config=Configure.inverted_index(
    bm25_b=0.75,
    bm25_k1=1.2,
    index_null_values=False,
    index_property_length=False,
),
)

print("DocumentChunk class created successfully")
client.close()

```

The BM25 parameters **k1** and **b** are tunable. **k1=1.2** (term frequency saturation) and **b=0.75** (document length normalization) are the classic defaults from Elasticsearch and Lucene — reasonable starting points for general English text. For short chunks (128–256 tokens), reduce **b** toward 0.5 because length normalization matters less when documents are uniformly short.

16.2 Schema Design Patterns for Production RAG Systems

Schema design mistakes in Weaviate are more expensive to fix than in a traditional RDBMS because the vector index is built during ingestion. Rebuilding it from scratch on a dataset of tens of millions of objects is an hours-long operation. The following patterns reflect hard-won production decisions.

Pattern 1: Separate classes for separate embedding models. Do not mix objects vectorized with `text-embedding-3-small` and objects vectorized with `text-embedding-3-large` in the same class. The vector space is different even though both are OpenAI models. Weaviate v1.24+ supports named vectors (multiple vectors per object), which solves this cleanly for cross-modal retrieval, but for standard single-model pipelines, one model per class is the rule.

Pattern 2: Push filtering metadata into scalar properties, not cross-references. Cross-references are powerful but expensive to filter on at scale. If you need to filter chunks by `tenant_id`, `document_type`, or `language`, store those as scalar properties with `index_filterable=True`. Filterable scalar indexes in Weaviate use a roaring bitmap structure — sub-millisecond for high-cardinality filters. Cross-reference traversal requires a separate resolution step and doesn’t compose well with pre-filtering.

Pattern 3: Use `skip_vectorization=True` aggressively. By default, Weaviate concatenates all non-skipped text properties before sending them to the vectorizer. A `source_document` path like `s3://my-bucket/reports/Q4-2023-financials.pdf` pollutes the semantic vector. Mark every property that isn’t semantic content as `skip_vectorization=True`.

Pattern 4: Named vectors for multi-representation objects. Weaviate v1.24 introduced named vectors, enabling one object to carry multiple vector representations. This matters for hybrid retrieval strategies where you want to search by question embedding AND summary embedding simultaneously:

```

# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import Configure, Property, DataType, Tokenization

```

```

client = weaviate.connect_to_local()

# Multi-vector schema: one object, two named vector spaces
client.collections.create(
    name="ResearchPaper",
    properties=[
        Property(name="title", data_type=DataType.TEXT, tokenization=Tokenization.WORD),
        Property(name="abstract", data_type=DataType.TEXT, tokenization=Tokenization.WORD),
        Property(name="full_text", data_type=DataType.TEXT, tokenization=Tokenization.WORD),
        Property(name="authors", data_type=DataType.TEXT_ARRAY, skip_vectorization=True),
        Property(name="year", data_type=DataType.INT, skip_vectorization=True),
        Property(name="doi", data_type=DataType.TEXT,
            tokenization=Tokenization.FIELD, skip_vectorization=True),
    ],
    vectorizer_config=[
        # Named vector 1: abstract-level semantic search
        Configure.NamedVectors.text2vec_openai(
            name="abstract_vector",
            source_properties=["title", "abstract"],
            model="text-embedding-3-small",
        ),
        # Named vector 2: full document retrieval
        Configure.NamedVectors.text2vec_openai(
            name="fulltext_vector",
            source_properties=["full_text"],
            model="text-embedding-3-large",
        ),
    ],
)

client.close()

```

At query time, you specify which named vector to search against. The overhead is roughly $2\times$ storage and index memory, but the retrieval flexibility is worth it for heterogeneous content where title/abstract search and full-text search have materially different precision characteristics.

Property index flags reference:

Flag	Type	Purpose	Default
<code>index_filterable</code>	bool	Roaring bitmap for pre-filtering	true
<code>index_searchable</code>	bool	BM25 inverted index	true for text
<code>index_range_filters</code>	bool	Numeric range index	false
<code>skip_vectorization</code>	bool	Exclude from embedding input	false

Turn off `index_searchable` for properties you never BM25-search. It saves disk and speeds up writes. For an INT property like `chunk_index` you'll never keyword-search, both `index_searchable` and `skip_vectorization` should be `true` and `true` respectively.

16.3 Hybrid Search: BM25 + Vector Fusion

Hybrid search is the single most operationally significant feature Weaviate offers over simpler vector stores. Pure vector search fails on proper nouns, product codes, rare terminology, and out-of-vocabulary tokens that the embedding model hasn't seen at training time. Pure BM25 fails on semantic paraphrase and synonym matching. Hybrid search fuses both signals.

Weaviate implements hybrid search via **Reciprocal Rank Fusion (RRF)** or a **relative score fusion** strategy. Both take the ranked lists from BM25 and vector ANN independently, then merge them. The `alpha` parameter controls the blend: `alpha=0.0` is pure BM25, `alpha=1.0` is pure vector, `alpha=0.5` is an equal blend. The default RRF formula is:

$$\text{RRF_score}(d) = \text{sum_over_rankings}(1 / (k + \text{rank}(d)))$$

where `k=60` is a smoothing constant. This is the same formula used in Elasticsearch's hybrid search and in academic IR literature. It is robust to outliers and scale differences between BM25 and cosine similarity scores.

```
# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.query import HybridFusion, MetadataQuery, Filter

client = weaviate.connect_to_local()

collection = client.collections.get("DocumentChunk")

# Basic hybrid search
response = collection.query.hybrid(
    query="transformer architecture self-attention mechanism",
    alpha=0.7,          # 70% weight toward vector, 30% BM25
    limit=10,
    return_metadata=MetadataQuery(score=True, explain_score=True),
)

for obj in response.objects:
    print(f"Score: {obj.metadata.score:.4f}")
    print(f"Explain: {obj.metadata.explain_score}")
    print(f"Content: {obj.properties['content'][:120]}")
    print("---")

# Hybrid search with pre-filtering
response_filtered = collection.query.hybrid(
    query="quarterly revenue forecast",
    alpha=0.5,
    limit=10,
    filters=Filter.by_property("source_document").contains_any(
        ["finance/Q4-2023.pdf", "finance/Q3-2023.pdf"]
    ),
    return_metadata=MetadataQuery(score=True),
)

# Hybrid search with named vectors (multi-vector schema)
research_collection = client.collections.get("ResearchPaper")

response_named = research_collection.query.hybrid(
    query="attention is all you need transformer",
    alpha=0.6,
    target_vector="abstract_vector",  # search in abstract embedding space
    limit=5,
    return_metadata=MetadataQuery(score=True, explain_score=True),
)

client.close()
```

Alpha tuning is empirical, not theoretical. Run your benchmark query set against your actual data, vary `alpha` from 0.3 to 0.9 in 0.1 increments, and measure `recall@10` or `NDCG@10` against human-labeled relevance judgments. Published benchmarks across IR datasets suggest `alpha` between 0.6 and 0.75 is optimal for most English-language knowledge retrieval tasks. For technical queries with precise terminology (SQL error codes, API method names, CVE identifiers), shift toward 0.3–0.4 to give BM25 more weight. For conversational or natural language questions,

0.7–0.8 is typical.

Score fusion strategy choice:

- **HybridFusion.RANKED (RRF)**: rank-based, insensitive to raw score magnitudes. Recommended when BM25 and vector scores have very different distributions (they almost always do).
- **HybridFusion.RELATIVE_SCORE**: normalizes both score lists to [0,1] then linearly combines. More sensitive to score distribution; can produce better results when both signals are calibrated.

RRF is the safer default for production deployments where score distribution is unknown or shifts with data updates.

16.4 Multi-Tenancy: One Schema, Many Isolated Datasets

Multi-tenancy in Weaviate is a first-class schema feature, not an application-layer workaround. This matters for SaaS platforms and enterprise deployments where you need per-customer data isolation without operating separate Weaviate instances.

Weaviate’s multi-tenancy model assigns each tenant a dedicated shard. The implications are significant:

- **Vector index isolation**: each tenant’s HNSW graph is independent. A query for Tenant A never scans Tenant B’s index.
- **Lifecycle control**: tenants can be activated (hot), deactivated (warm, on disk), or offloaded (cold, off disk) independently. This is the key mechanism for cost-optimized multi-tenant deployments where not all customers are active simultaneously.
- **No cross-tenant contamination**: a malformed insert or a slow query on one tenant cannot degrade another tenant’s shard.

The tradeoff is memory. Each active tenant’s HNSW graph must be loaded into memory. At 10,000 objects per tenant with `ef_construction=128` and 1536-dimension float32 vectors, expect roughly 70–90 MB per tenant in memory. At 1,000 active tenants that’s 70–90 GB — plan your node sizing accordingly, or use tenant activation/deactivation to manage the active set.

```
# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import Configure, Property, DataType, Tokenization
from weaviate.classes tenants import Tenant, TenantActivityStatus

client = weaviate.connect_to_local()

# Create a multi-tenant collection
client.collections.create(
    name="CustomerDocuments",
    multi_tenancy_config=Configure.multi_tenancy(
        enabled=True,
        auto_tenant_creation=False,  # explicit tenant management
    ),
    vectorizer_config=Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-small",
    ),
    properties=[
        Property(name="content", data_type=DataType.TEXT, tokenization=Tokenization.WORD),
```

```

        Property(name="doc_type",      data_type=DataType.TEXT, tokenization=Tokenization.FIELD),
        Property(name="uploaded_by",  data_type=DataType.TEXT, skip_vectorization=True),
    ],
)

collection = client.collections.get("CustomerDocuments")

# Create tenants (one per customer)
collection.tenants.create([
    Tenant(name="customer_acme",      activity_status=TenantActivityStatus.ACTIVE),
    Tenant(name="customer_globex",    activity_status=TenantActivityStatus.ACTIVE),
    Tenant(name="customer_initech",    activity_status=TenantActivityStatus.INACTIVE), # warm
])

# Insert objects into a specific tenant
tenant_collection = collection.with_tenant("customer_acme")

with tenant_collection.batch.dynamic() as batch:
    batch.add_object(properties={
        "content":      "Q4 revenue exceeded targets by 12% driven by APAC expansion.",
        "doc_type":      "financial_report",
        "uploaded_by":    "finance_team",
    })
    batch.add_object(properties={
        "content":      "New product roadmap approved by executive committee.",
        "doc_type":      "strategy_doc",
        "uploaded_by":    "product_team",
    })

# Query within a tenant
response = tenant_collection.query.hybrid(
    query="revenue targets quarterly results",
    alpha=0.6,
    limit=5,
)

for obj in response.objects:
    print(obj.properties["content"])

# Deactivate an inactive tenant to free memory (tenant data stays on disk)
collection.tenants.update([
    Tenant(
        name="customer_globex",
        activity_status=TenantActivityStatus.INACTIVE,
    )
])

# Reactivate when needed
collection.tenants.update([
    Tenant(
        name="customer_globex",
        activity_status=TenantActivityStatus.ACTIVE,
    )
])

client.close()

```

Tenant activation patterns in practice:

Scenario	Strategy
Enterprise SaaS, most tenants active	All ACTIVE, scale horizontally
Long-tail SaaS, 20% tenants use 80% of queries	Active set management via usage metrics
Archive/compliance, tenants rarely queried	OFFLOADED by default, activate on-demand
Testing/staging environments	INACTIVE until test run starts

Auto-tenant creation (`auto_tenant_creation=True`) creates a tenant shard the first time an object is inserted for an unknown tenant key. It's convenient for development but dangerous in production — a client bug with an unexpected tenant ID can create thousands of orphaned shards.

16.5 Batch Ingestion, HNSW Tuning, and Query Performance

Weaviate's HNSW implementation is configurable at class definition time. The parameters are identical in semantics to those in FAISS and hnsplib because Weaviate's implementation derives from the same academic specification, but the defaults are conservative — intentionally so, because Weaviate serves latency-sensitive HTTP workloads, not offline batch retrieval.

The three HNSW parameters you will tune:

- `ef_construction` (default: 128): graph quality during build. Higher = better recall, slower ingestion. Range 64–256 covers most use cases. Don't go below 64 — recall degrades sharply. Don't exceed 256 unless you have a recall requirement above 99.5%.
- `max_connections` (M, default: 64): neighbors per node. Higher = better recall on large datasets, higher memory. For datasets under 10M objects, M=16 is enough. For 100M+ objects, M=32–64.
- `ef` (default: 64 or dynamic): query-time beam width. The primary recall vs. latency knob. Set via `dynamic_ef_factor` to auto-scale based on limit.

Memory estimation for HNSW:

Memory (bytes) $\text{num_objects} \times (\text{dimensions} \times 4 + \text{max_connections} \times 8 \times 2)$

For 1M objects, 1536 dimensions, M=16:

$1,000,000 \times (1536 \times 4 + 16 \times 8 \times 2) = 1,000,000 \times (6144 + 256) = \sim 6.4 \text{ GB}$

This is the graph memory floor. Add scalar property storage and inverted index overhead on top.

Batch ingestion with error handling:

```
# Python
import weaviate
import weaviate.classes as wvc
from weaviate.classes.config import ConsistencyLevel
import json
from pathlib import Path
from tqdm import tqdm

client = weaviate.connect_to_local()
collection = client.collections.get("DocumentChunk")

def load_chunks(filepath: str) -> list[dict]:
    """Load pre-chunked documents from JSONL."""
    with open(filepath) as f:
        return [json.loads(line) for line in f]

chunks = load_chunks("chunks.jsonl")

# Dynamic batching: Weaviate auto-tunes batch size based on server capacity
failed_objects = []

with collection.batch.dynamic() as batch:
    for chunk in tqdm(chunks, desc="Ingesting chunks"):
        batch.add_object(
```



```

properties={
    "content":      chunk["text"],
    "source_document": chunk["source"],
    "chunk_index":  chunk["index"],
    "metadata_tags": chunk.get("tags", []),
},
# Optionally supply a deterministic UUID for idempotency
uuid=wvc.util.generate_uuid5(
    f"{chunk['source']}_{chunk['index']}"
),
)

# Check for failed objects after batch completes
if batch.number_errors > 0:
    print(f"Batch completed with {batch.number_errors} errors")
    for failed in collection.batch.failed_objects:
        print(f"Failed: {failed.message} | Object: {failed.object_.properties}")

print(f"Total objects in collection: {collection.aggregate.over_all().total_count}")
client.close()

```

Key batch performance observations from production deployments:

- Dynamic batching (`batch.dynamic()`) outperforms fixed-size batching in most cases because Weaviate’s server-side admission control adapts to memory pressure during HNSW graph construction.
- With external vectorization (OpenAI, Cohere), the embedding API call is the bottleneck — not Weaviate’s insert throughput. Use async batching or parallel chunk pipelines with a semaphore to control concurrent embedding calls.
- With built-in vectorizers (`text2vec-transformers` running locally), GPU availability dominates throughput. A single T4 GPU handles roughly 2,000–4,000 tokens per second for `sentence-transformers/all-MiniLM-L6-v2`.
- Set `WEAVIATE_GRPC_PORT` and use gRPC transport for batch inserts in v1.23+. gRPC batch ingestion is meaningfully faster than the REST API for large payloads because it avoids JSON serialization overhead — benchmark against your own payload shapes rather than assuming a fixed improvement.

Query latency benchmarks (approximate, single-node Weaviate, 1M objects):

Search Type	ef=64	ef=128	ef=256
Pure vector	~8ms	~14ms	~25ms
Pure BM25	~5ms	~5ms	~5ms
Hybrid (RRF)	~12ms	~18ms	~30ms
Hybrid + filter	~15ms	~22ms	~38ms

BM25 latency is largely independent of `ef` because it uses a separate inverted index path. Hybrid latency is dominated by the vector search component, so `ef` tuning applies to hybrid queries the same way it applies to pure vector queries.

16.6 Filtering Strategies: Pre-Filter vs. Post-Filter

Weaviate’s approach to filtered vector search has evolved significantly. Early versions used post-filtering — execute ANN search, then discard results that don’t match the filter predicate. Post-

filtering is simple to implement but statistically problematic: if your filter eliminates 90% of the dataset, you may retrieve only 1–2 results even when you asked for 10, because the ANN graph was built over the full dataset and doesn't know which nodes will survive the filter.

Weaviate v1.17+ implements **flat search fallback** and **ACORN-style filtered ANN**, where the execution strategy adapts based on filter selectivity:

- **High selectivity filter** (small result set, e.g., 0.1% of objects match): Weaviate falls back to flat (brute-force) search over the filtered subset. This is faster than navigating the HNSW graph when the candidate pool is small enough.
- **Low selectivity filter** (large result set, e.g., 80% of objects match): standard HNSW graph traversal with filter evaluation at candidate inspection time.

The threshold for switching strategies is configurable via `flat_search_cutoff` in the collection's vector index configuration. The default threshold is typically around 40,000 objects — if fewer objects match the filter, flat search is used.

```
# Python
from weaviate.classes.config import Configure, VectorDistances

client.collections.create(
    name="ProductCatalog",
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        ef_construction=128,
        max_connections=16,
        ef=96,
        flat_search_cutoff=40_000, # tune based on your filter distribution
    ),
    # ... properties omitted for brevity
)
```

Filter composition rules that matter:

1. Use `Filter.by_property("field").equal()` for equality filters — these hit the roaring bitmap index directly.
2. Avoid filtering on cross-references when a scalar property would suffice. Cross-reference filters require graph traversal before the vector search even begins.
3. Combine multiple filters with `Filter.all_of([...])` (AND) or `Filter.any_of([...])` (OR). The query planner intersects roaring bitmaps, which is fast.
4. Range filters on numeric properties require `index_range_filters=True` at schema definition time. Without this flag, numeric range queries fall back to a full-scan of the property store — expensive at scale.

```
# Python
from weaviate.classes.query import Filter, MetadataQuery

collection = client.collections.get("ProductCatalog")

# Compound filter: category AND price range AND in-stock
response = collection.query.hybrid(
    query="wireless noise-cancelling headphones",
    alpha=0.65,
    limit=20,
    filters=Filter.all_of([
        Filter.by_property("category").equal("electronics"),
        Filter.by_property("price_usd").greater_than(50.0),
        Filter.by_property("price_usd").less_than(500.0),
        Filter.by_property("in_stock").equal(True),
    ]),
    return_metadata=MetadataQuery(score=True),
)
```

```

)

for obj in response.objects:
    print(f"{obj.properties['name']:40s}  score={obj.metadata.score:.4f}")

```

16.7 Generative Search and RAG Modules

Weaviate's generative search module (`generative-openai`, `generative-cohere`, `generative-anthropic`, and others) integrates LLM generation directly into the query pipeline. Instead of retrieving objects and then calling an LLM in application code, you declare the generation step in the query itself.

This is architecturally convenient but not always the right choice. The tradeoff is flexibility vs. simplicity: Weaviate's built-in generative module handles the simplest RAG patterns well, but complex prompt engineering, multi-step retrieval, and retrieval with re-ranking often require application-layer orchestration anyway.

When to use the generative module: - Prototyping RAG systems quickly - Simple question-answering over a well-bounded corpus - Single-turn queries with straightforward summarization requirements

When to move generation to the application layer: - Multi-turn conversations requiring context management - Re-ranking retrieved results before generation - Prompt templates that depend on user-specific data - Cost optimization across multiple providers based on query type

```

# Python
from weaviate.classes.generate import GenerativeConfig
from weaviate.classes.query import MetadataQuery

client = weaviate.connect_to_local() # Weaviate configured with generative-openai module

collection = client.collections.get("DocumentChunk")

# Single-prompt generation: apply one prompt to all retrieved objects
response = collection.generate.hybrid(
    query="climate change mitigation strategies",
    alpha=0.65,
    limit=5,
    single_prompt="Summarize the key point of this passage in one sentence: {content}",
    return_metadata=MetadataQuery(score=True),
)

for obj in response.objects:
    print(f"Source: {obj.properties['source_document']}")
    print(f"Generated: {obj.generated}")
    print()

# Grouped task: synthesize a response across all retrieved objects
response_grouped = collection.generate.hybrid(
    query="what are the main approaches to carbon capture?",
    alpha=0.65,
    limit=8,
    grouped_task=(
        "Based on the provided passages, write a concise technical summary "
        "of the main carbon capture approaches discussed, citing specific methods."
    ),
    grouped_properties=["content", "source_document"],
)

print("Synthesized answer:")

```

```
print(response_grouped.generated)
```

The `single_prompt` mode calls the LLM once per retrieved object — useful for per-passage annotation or extraction. The `grouped_task` mode sends all retrieved passages to the LLM in a single call — the standard RAG synthesis pattern.

Module configuration in the Weaviate instance settings (set at startup, not per-query):

```
# docker-compose.yml excerpt
environment:
  ENABLE_MODULES: "text2vec-openai,generative-openai"
  OPENAI_APIKEY: "${OPENAI_API_KEY}"
  # For Azure OpenAI:
  # AZURE_APIKEY: "${AZURE_OPENAI_API_KEY}"
```

If you operate Weaviate in a zero-trust network where outbound API calls are restricted, the `text2vec-transformers` and `generative-ollama` modules run fully locally using self-hosted models — no external API calls.

16.8 Backup, Restore, and Operational Concerns

Weaviate’s backup system operates at the collection level and supports pluggable backends: filesystem, S3, GCS, and Azure Blob Storage. Backups are **live** — they do not require stopping the database or taking it offline.

```
# Python
import weaviate

client = weaviate.connect_to_local()

# Trigger a backup to S3 (requires backup-s3 module enabled)
result = client.backup.create(
    backup_id="backup-2024-01-15-prod",
    backend="s3",
    include_collections=["DocumentChunk", "ResearchPaper"],
    wait_for_completion=True,
)
print(f"Backup status: {result.status}")

# Restore from backup
restore_result = client.backup.restore(
    backup_id="backup-2024-01-15-prod",
    backend="s3",
    include_collections=["DocumentChunk"],
    wait_for_completion=True,
)
print(f"Restore status: {restore_result.status}")

client.close()
```

Operational checklist for production Weaviate deployments:

Concern	Recommendation
Memory sizing	2× the HNSW memory estimate; leave headroom for batch ingestion spikes
Disk sizing	3–4× raw vector data size (inverted index, WAL, metadata)
Replication	<code>replication_factor=3</code> for production; <code>consistency_level=QUORUM</code> for reads

Concern	Recommendation
Backup cadence	Daily backup to object storage; verify restore quarterly
Module isolation	Run <code>text2vec-transformers</code> as a separate container; GPU contention with Weaviate’s own Go runtime causes latency spikes
Schema versioning	Treat class definitions as code; store in version control alongside migration scripts
Monitoring	Export Prometheus metrics via <code>/v1/metrics</code> ; alert on <code>batch_durations_ms_bucket</code> , <code>query_dimensions_ms_bucket</code> , and heap memory

Replication configuration:

```
# Python
from weaviate.classes.config import Configure, Reconfigure

# At class creation time
client.collections.create(
    name="DocumentChunk",
    replication_config=Configure.replication(factor=3),
    # ... other config
)

# Consistency level at query time
from weaviate.classes.query import MetadataQuery
from weaviate.classes.config import ConsistencyLevel

collection = client.collections.get("DocumentChunk")

# Read with quorum consistency
response = collection.with_consistency_level(
    ConsistencyLevel.QUORUM
).query.hybrid(
    query="distributed systems consensus",
    alpha=0.6,
    limit=10,
)
```

Weaviate uses a leaderless replication model similar to Cassandra’s — all replica nodes are equal, and consistency is controlled per-operation via ONE, QUORUM, or ALL. For RAG workloads, QUORUM reads with QUORUM writes provides the right balance between consistency and availability.

16.9 Weaviate vs. pgvector: Decision Framework

After the preceding chapters on pgvector and this chapter on Weaviate, the natural question is when to choose one over the other. This is not a “best database wins” comparison — it is a context-dependent architectural decision.

Choose pgvector when:

- Your application data already lives in PostgreSQL and you want to avoid a second operational system
- Your team’s operational expertise is SQL and relational databases
- Your vector dataset is under ~10M objects and hybrid search requirements are modest
- Strong ACID transactions matter (financial records, inventory, anything with coordinated writes)
- You need arbitrary JOIN semantics between vector results and relational data

Choose Weaviate when: - You need production-grade hybrid search (BM25 + vector) with minimal configuration - Multi-tenancy with per-tenant shard isolation is a hard requirement - Built-in vectorization modules remove embedding pipeline complexity from your architecture - Your dataset grows to tens or hundreds of millions of objects with demanding recall/latency SLAs - Named vectors (multi-representation objects) fit your retrieval model - You want generative search integrated into the query layer for prototyping

The operational cost of adding Weaviate to a PostgreSQL shop is real. You now operate two stateful systems, two backup procedures, two monitoring stacks. Many teams start with pgvector, hit a concrete limitation (multi-tenancy isolation, hybrid search quality, recall at scale), and then migrate to Weaviate for the specific workload that needs it while keeping PostgreSQL for transactional data.

The reverse migration — from Weaviate back to pgvector — is rare but happens when teams underestimate PostgreSQL’s capacity or overestimate the complexity of their vector workload.

Key Takeaways

- **Weaviate’s class-and-property schema is a design contract:** tokenization choices, `skip_vectorization` flags, and index configuration flags (`index_filterable`, `index_searchable`, `index_range_filters`) have permanent performance consequences that cannot be changed without rebuilding the index, so schema design deserves the same rigor as DDL in a relational database.
 - **Hybrid search with Reciprocal Rank Fusion combines BM25 and vector ANN into a single ranked result:** the `alpha` parameter blends the two signals, and empirical tuning against labeled relevance judgments — rather than theoretical defaults — consistently produces the best retrieval quality for a specific domain and query type.
 - **Multi-tenancy is a first-class Weaviate feature backed by shard isolation:** each tenant gets an independent HNSW graph, enabling per-tenant lifecycle management (ACTIVE, INACTIVE, OFFLOADED) that controls memory consumption in large-scale SaaS deployments, but active-tenant memory budgeting must be part of node sizing from day one.
 - **Named vectors (Weaviate v1.24+) allow one object to carry multiple independent vector representations:** this supports cross-modal and multi-granularity retrieval strategies — for example, searching by abstract embedding and full-text embedding independently — at the cost of roughly proportional additional storage and index memory.
 - **Filter selectivity drives Weaviate’s execution strategy:** high-selectivity filters trigger a flat brute-force search over the matching subset, which is faster than HNSW graph traversal when the candidate pool is small; schema-time flags like `index_range_filters=True` on numeric properties are prerequisites for efficient range filtering, and omitting them causes silent full-scan fallbacks at query time.
-

Chapter 17: Qdrant: Payload Filtering and Precision Tuning

Qdrant occupies a distinctive position in the vector database landscape: it was built from the ground up in Rust with a clear architectural opinion that vector search and structured metadata filtering should be first-class equals, not an afterthought. Where other systems treat payload filters as a post-processing step applied after ANN retrieval, Qdrant’s filtered HNSW implementation weaves payload conditions directly into the graph traversal itself. The result is a system where adding filters often costs less than you expect, and where the gap between “give me the nearest vectors” and “give me the nearest vectors matching these exact business conditions” nearly disappears. This chapter walks through Qdrant’s data model, its payload indexing system, the mechanics of filtered HNSW, and the production configuration knobs that separate a tuned deployment from one that simply runs.

17.1 Qdrant’s Data Model: Collections, Points, and Payloads

Before touching query syntax, you need to understand how Qdrant organizes data. The mental model maps cleanly to relational concepts, which makes it approachable for DBAs.

A **collection** is Qdrant’s equivalent of a table. It holds a set of **points**, where each point contains:

- A **unique ID** — either a 64-bit unsigned integer or a UUID string
- One or more **vectors** — dense, sparse, or multivector depending on configuration
- A **payload** — an arbitrary JSON document, roughly analogous to a JSONB column in PostgreSQL

Unlike traditional databases where schema is enforced at write time, Qdrant’s payload schema is flexible by default. You can insert points with completely different payload shapes into the same collection. However, the analogy to PostgreSQL’s JSONB goes further than you might expect: just as PostgreSQL requires explicit index creation on JSONB paths before those paths become filter-efficient, Qdrant requires you to create **payload indexes** before payload fields become efficient filter predicates.

This distinction matters. Without a payload index, Qdrant performs a full scan of all payload documents to evaluate a filter condition — effectively a sequential scan. With a payload index, filter evaluation happens in a purpose-built data structure before or during vector traversal.

Supported payload types and their indexed forms:

Payload Type	Qdrant Index Type	Analogy
String/keyword	Keyword index	B-tree on VARCHAR
Integer	Integer index	B-tree on BIGINT
Float	Float index	B-tree on DOUBLE
Boolean	— (inline, fast)	BIT column
Geo coordinates	Geo index	PostGIS spatial index
Full text	Text index (tokenized)	GIN index on tsvector
UUID	UUID index	B-tree on UUID

The collection-level configuration also controls vector parameters, distance metrics, and the HNSW graph parameters that affect both search quality and resource consumption. Let's stand up a collection and populate it with something realistic.

```
# Python
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance,
    VectorParams,
    PointStruct,
    CreateCollection,
)
from sentence_transformers import SentenceTransformer
import uuid

# Connect to a local Qdrant instance (default port 6333)
client = QdrantClient(host="localhost", port=6333)

COLLECTION_NAME = "product_catalog"
VECTOR_DIM = 384 # all-MiniLM-L6-v2 output dimension

# Create collection with explicit HNSW configuration
client.create_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(
        size=VECTOR_DIM,
        distance=Distance.COSINE,
        # HNSW index params - tunable per collection
        hnsw_config={
            "m": 16, # edges per node; higher = better recall, more RAM
            "ef_construct": 100, # build-time beam width
            "full_scan_threshold": 10_000, # switch to brute force below this count
        },
    ),
)

# Create payload indexes BEFORE bulk insert (more efficient)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="category",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="price",
    field_schema="float",
)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
    field_name="in_stock",
    field_schema="bool",
)
client.create_payload_index(
    collection_name=COLLECTION_NAME,
```



```

        field_name="description",
        field_schema={
            "type": "text",
            "tokenizer": "word",
            "min_token_len": 2,
            "max_token_len": 20,
            "lowercase": True,
        },
    )

    # Encode some product descriptions
    encoder = SentenceTransformer("all-MiniLM-L6-v2")

    products = [
        {"id": str(uuid.uuid4()), "text": "Wireless noise-canceling over-ear headphones",
         "category": "electronics", "price": 149.99, "in_stock": True, "brand": "SoundPro"},
        {"id": str(uuid.uuid4()), "text": "Ergonomic mechanical keyboard with RGB backlight",
         "category": "electronics", "price": 89.99, "in_stock": True, "brand": "TypeMaster"},
        {"id": str(uuid.uuid4()), "text": "Standing desk converter, 36-inch wide surface",
         "category": "furniture", "price": 229.00, "in_stock": False, "brand": "DeskRise"},
        {"id": str(uuid.uuid4()), "text": "Portable USB-C hub with 4K HDMI output",
         "category": "electronics", "price": 59.99, "in_stock": True, "brand": "HubTech"},
        {"id": str(uuid.uuid4()), "text": "Adjustable lumbar support office chair",
         "category": "furniture", "price": 399.00, "in_stock": True, "brand": "ErgoSeat"},
    ]

    vectors = encoder.encode([p["text"] for p in products]).tolist()

    points = [
        PointStruct(
            id=p["id"],
            vector=v,
            payload={
                "category": p["category"],
                "price": p["price"],
                "in_stock": p["in_stock"],
                "brand": p["brand"],
                "description": p["text"],
            },
        )
        for p, v in zip(products, vectors)
    ]

    # Upsert in a single batch
    operation_info = client.upsert(
        collection_name=COLLECTION_NAME,
        wait=True, # synchronous - waits for indexing to complete
        points=points,
    )
    print(f"Upsert status: {operation_info.status}")

    # Verify collection info
    info = client.get_collection(COLLECTION_NAME)
    print(f"Points count: {info.points_count}")
    print(f"Indexed vectors: {info.indexed_vectors_count}")

```

The `wait=True` parameter deserves mention. Qdrant is eventually consistent by default — vectors are accepted into a write-ahead buffer before being incorporated into the HNSW graph. Setting `wait=True` blocks until the operation is confirmed durable, which is the correct choice for transactional writes. For bulk loading, omitting it (or setting `wait=False`) allows pipelining and can improve throughput by an order of magnitude, provided you validate completeness afterward.

17.2 Payload Filtering: From Simple Conditions to Complex Predicates

Qdrant's filter language uses a nested boolean structure that maps directly to SQL WHERE clauses. Once you see the correspondence, the API becomes predictable.

The top-level filter is always one of: `must` (AND), `should` (OR), `must_not` (NOT), or their nested combinations. Each condition targets a specific payload field using a typed matcher.

```
# Python
from qdrant_client.models import (
    Filter,
    FieldCondition,
    MatchValue,
    MatchAny,
    Range,
    SearchRequest,
)

encoder = SentenceTransformer("all-MiniLM-L6-v2")
query_text = "comfortable headphones for long work sessions"
query_vector = encoder.encode(query_text).tolist()

# -----
# Example 1: Simple AND filter
# SQL equivalent: WHERE category = 'electronics' AND in_stock = true
# -----
simple_filter = Filter(
    must=[
        FieldCondition(key="category", match=MatchValue(value="electronics")),
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
    ]
)

results = client.search(
    collection_name=COLLECTION_NAME,
    query_vector=query_vector,
    query_filter=simple_filter,
    limit=5,
    with_payload=True,
    with_vectors=False,
)

for r in results:
    print(f"Score: {r.score:.4f} | {r.payload['description']} | ${r.payload['price']}")

# -----
# Example 2: Range filter on numeric field
# SQL equivalent: WHERE price BETWEEN 50.00 AND 200.00
# -----
price_filter = Filter(
    must=[
        FieldCondition(key="price", range=Range(gte=50.0, lte=200.0)),
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
    ]
)

results_price = client.search(
    collection_name=COLLECTION_NAME,
    query_vector=query_vector,
    query_filter=price_filter,
    limit=10,
    with_payload=["description", "price", "brand"],
)

# -----
# Example 3: OR condition with MatchAny
```

```

# SQL equivalent: WHERE category IN ('electronics', 'accessories')
# -----
category_filter = Filter(
    must=[
        FieldCondition(
            key="category",
            match=MatchAny(any=["electronics", "accessories"]),
        ),
        FieldCondition(key="price", range=Range(lt=100.0)),
    ]
)

# -----
# Example 4: Nested boolean - (must AND) combined with (should OR)
# SQL equivalent:
# WHERE in_stock = true
# AND (category = 'electronics' OR price < 100.0)
# -----
nested_filter = Filter(
    must=[
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
        Filter(
            should=[
                FieldCondition(key="category", match=MatchValue(value="electronics")),
                FieldCondition(key="price", range=Range(lt=100.0)),
            ]
        ),
    ]
)

results_nested = client.search(
    collection_name=COLLECTION_NAME,
    query_vector=query_vector,
    query_filter=nested_filter,
    limit=5,
    with_payload=True,
)

# -----
# Example 5: must_not exclusion
# SQL equivalent: WHERE brand != 'SoundPro' AND in_stock = true
# -----
exclusion_filter = Filter(
    must=[
        FieldCondition(key="in_stock", match=MatchValue(value=True)),
    ],
    must_not=[
        FieldCondition(key="brand", match=MatchValue(value="SoundPro")),
    ],
)

```

One thing that trips up developers coming from SQL: Qdrant filters operate on the **payload document as a whole**, not on join results. There is no equivalent of a multi-table join at query time. If your data model requires join semantics — say, filtering products by a property that lives in a separate entity — you must denormalize that property into the point’s payload before inserting. This is standard practice in vector databases and mirrors the document database pattern: write-time denormalization in exchange for read-time simplicity and speed.

Nested payload fields are accessed with dot notation. If a payload contains `{"metadata": {"region": "us-east-1"}}`, the filter key is `"metadata.region"`. Arrays are also supported: if a payload field contains `["tag1", "tag2", "tag3"]`, a `MatchValue` condition checks whether any element matches.

17.3 Filtered HNSW: How Qdrant Avoids the Post-Filter Penalty

To understand why Qdrant’s filtered search is competitive with unfiltered search, you need to understand what most other systems do and why it’s costly.

The naive approach — post-filtering:

1. Retrieve the top-K neighbors from the HNSW graph
2. Apply payload filters to those K results
3. Return however many survive the filter

This works when the filter selectivity is low (most documents pass). When selectivity is high — say, only 0.1% of documents match the filter — you need to retrieve $K/0.001 = 1000K$ candidates to expect K results. At that point, you have effectively done a full graph scan, and the ANN index provides no benefit.

Qdrant’s approach — filtered HNSW traversal:

Qdrant maintains an additional data structure: a **payload filter index** for each indexed field, which maps field values to point IDs. During HNSW traversal, Qdrant uses this index to evaluate filter conditions per candidate node *before* computing distances or following graph edges. Nodes that fail the filter condition are skipped during graph traversal, not just discarded afterward.

The practical effect is that the search explores the graph only through nodes that satisfy the filter. This is closer to how a PostgreSQL partial index works: the index was built over a specific subset of data, and queries that match that predicate can use the index directly rather than filtering after retrieval.

There is a tradeoff: HNSW graphs can become less connected when many nodes are excluded. Qdrant addresses this with an automatic fallback mechanism. When the expected number of matching points falls below `full_scan_threshold` (configured per collection), Qdrant switches to brute-force search over only the matching points. For small result sets, brute force over a tiny filtered subset is faster than graph traversal anyway.

```
# Python
# Demonstrating the performance difference between filtered and unfiltered search,
# and verifying that Qdrant is using its index vs. brute force path.

import time
from qdrant_client.models import SearchParams

# For a larger dataset, create and populate a collection programmatically
import random
import numpy as np

LARGE_COLLECTION = "benchmark_collection"
N_POINTS = 100_000
DIM = 128

# Create collection
try:
    client.delete_collection(LARGE_COLLECTION)
except Exception:
    pass

client.create_collection(
    collection_name=LARGE_COLLECTION,
    vectors_config=VectorParams(
        size=DIM,
        distance=Distance.COSINE,
```

```

        hnsw_config={
            "m": 16,
            "ef_construct": 128,
            "full_scan_threshold": 10_000,
        },
    ),
)

# Create payload index before bulk insert
client.create_payload_index(
    collection_name=LARGE_COLLECTION,
    field_name="tenant_id",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=LARGE_COLLECTION,
    field_name="score",
    field_schema="float",
)

# Generate random points with tenant labels (100 tenants, ~1000 points each)
BATCH_SIZE = 1000
tenants = [f"tenant_{i:03d}" for i in range(100)]

for batch_start in range(0, N_POINTS, BATCH_SIZE):
    batch_ids = list(range(batch_start, min(batch_start + BATCH_SIZE, N_POINTS)))
    batch_vectors = np.random.rand(len(batch_ids), DIM).tolist()
    batch_points = [
        PointStruct(
            id=i,
            vector=v,
            payload={
                "tenant_id": random.choice(tenants),
                "score": random.uniform(0.0, 1.0),
            },
        )
        for i, v in zip(batch_ids, batch_vectors)
    ]
    client.upsert(
        collection_name=LARGE_COLLECTION,
        wait=False, # async for bulk load throughput
        points=batch_points,
    )

# Wait for indexing to complete
import time
while True:
    info = client.get_collection(LARGE_COLLECTION)
    if info.status.value == "green":
        break
    time.sleep(1)

query_vector = np.random.rand(DIM).tolist()

# Benchmark 1: Unfiltered search
start = time.perf_counter()
for _ in range(50):
    client.search(
        collection_name=LARGE_COLLECTION,
        query_vector=query_vector,
        limit=10,
    )
unfiltered_ms = (time.perf_counter() - start) / 50 * 1000
print(f"Unfiltered search avg latency: {unfiltered_ms:.2f}ms")

# Benchmark 2: Filtered search - 1% selectivity (1 of 100 tenants)
tenant_filter = Filter(
    must=[FieldCondition(key="tenant_id", match=MatchValue(value="tenant_042"))]
)
start = time.perf_counter()

```

```

for _ in range(50):
    client.search(
        collection_name=LARGE_COLLECTION,
        query_vector=query_vector,
        query_filter=tenant_filter,
        limit=10,
        search_params=SearchParams(hnsw_ef=128), # explicit ef at query time
    )
filtered_ms = (time.perf_counter() - start) / 50 * 1000
print(f"Filtered search (1% selectivity) avg latency: {filtered_ms:.2f}ms")

# Benchmark 3: High-selectivity filter - 0.1% with range condition
narrow_filter = Filter(
    must=[
        FieldCondition(key="tenant_id", match=MatchValue(value="tenant_042")),
        FieldCondition(key="score", range=Range(gte=0.95)),
    ]
)
start = time.perf_counter()
for _ in range(50):
    client.search(
        collection_name=LARGE_COLLECTION,
        query_vector=query_vector,
        query_filter=narrow_filter,
        limit=10,
    )
narrow_ms = (time.perf_counter() - start) / 50 * 1000
print(f"High-selectivity filtered search avg latency: {narrow_ms:.2f}ms")
# Note: at very high selectivity, Qdrant switches to brute-force over
# the matching subset, which is often faster than graph traversal

```

Exact numbers depend heavily on hardware, dataset size, and dimensionality, so run this benchmark against your own deployment rather than trusting a printed figure — but the general shape holds: unfiltered and 1%-selectivity filtered search land in a similar low-millisecond range, and high-selectivity search (where brute-force kicks in) is often just as fast because the matching set is small. The filtered HNSW penalty is far smaller than the equivalent post-filter architecture.

17.4 Precision Tuning: ef, Quantization, and Segment Configuration

Qdrant exposes more precision knobs than most vector databases. Understanding them requires understanding the search accuracy vs. speed tradeoff in concrete terms.

The ef parameter:

In HNSW, **ef** (exploration factor) controls how many candidates the graph explorer keeps in its priority queue during traversal. Higher **ef** means more candidates explored, which increases recall at the cost of latency. Two separate **ef** values exist:

- **ef_construct**: Used during index build. Set this once at collection creation. Typical values: 100–400. Higher values produce better graph connectivity but slow down writes.
- **hnsw_ef** (or **ef** at search time): Used during query. Can be set per-query via **SearchParams**. Typical range: 64–512.

The **m** parameter controls the number of bidirectional edges each node maintains in the graph. Higher **m** increases RAM usage proportionally but improves recall, especially at lower **ef** values. For 100k–1M vectors, **m=16** is a reasonable default. For datasets requiring very high recall (>99%)

at large scale, $m=32$ or $m=64$ is warranted.

Quantization:

Qdrant supports three quantization strategies that trade accuracy for speed and memory reduction:

Strategy	Memory Reduction	Recall Impact	Typical Use
Scalar (int8)	~4x	<1% at ef rescore	General purpose
Product Quantization (PQ)	8–32x	2–5% without rescore	Memory-constrained
Binary	~32x	Significant without rescore	Pre-filtering, speed priority

All three support **rescoring**: after the initial quantized search narrows the candidate set, Qdrant fetches the original full-precision vectors for the top candidates and re-ranks them. This recovers most of the accuracy penalty. The tradeoff is I/O for the rescore step.

```
# Python
from qdrant_client.models import (
    ScalarQuantizationConfig,
    ProductQuantizationConfig,
    BinaryQuantizationConfig,
    QuantizationConfig,
    ScalarType,
    CompressionRatio,
)

# -----
# Creating a collection with scalar quantization (recommended default)
# -----
client.create_collection(
    collection_name="quantized_products",
    vectors_config=VectorParams(
        size=1536,          # OpenAI text-embedding-3-small dimension
        distance=Distance.COSINE,
        hnsw_config={
            "m": 16,
            "ef_construct": 128,
            "full_scan_threshold": 10_000,
        },
    ),
    quantization_config=ScalarQuantizationConfig(
        type=ScalarType.INT8,
        quantile=0.99,      # clip outliers; 0.99 is conservative
        always_ram=True,    # keep quantized vectors in RAM, originals on disk
    ),
)

# -----
# Creating a collection with product quantization for memory-constrained environments
# 1536-dim OpenAI vectors compressed to ~192 bytes (8x reduction at m=256)
# -----
client.create_collection(
    collection_name="compressed_products",
    vectors_config=VectorParams(
        size=1536,
        distance=Distance.COSINE,
    ),
    quantization_config=ProductQuantizationConfig(
        compression=CompressionRatio.X16, # 16x compression
        always_ram=True,
    ),
)
```

```

# -----
# Per-query control over quantization behavior
# -----
from qdrant_client.models import QuantizationSearchParams

results = client.search(
    collection_name="quantized_products",
    query_vector=query_vector,
    limit=10,
    search_params=SearchParams(
        hnsw_ef=256,          # higher ef compensates for quantization noise
        quantization=QuantizationSearchParams(
            ignore=False,      # use quantization
            rescore=True,      # rescore top candidates with original vectors
            oversampling=2.5,   # fetch 2.5x candidates for rescore pool
        ),
    ),
)

# -----
# Optimizer and segment configuration
# These control how Qdrant manages internal storage segments
# -----
from qdrant_client.models import OptimizersConfigDiff

client.update_collection(
    collection_name="quantized_products",
    optimizer_config=OptimizersConfigDiff(
        # Flush segments to disk when they exceed this size (in KB)
        max_segment_size=200_000,      # 200MB per segment
        # How many points trigger a segment merge
        memmap_threshold=50_000,
        # Minimum number of vectors before HNSW index is built
        # (below this, brute force is used instead)
        indexing_threshold=20_000,
        # Number of concurrent optimizer threads
        max_optimization_threads=2,
        # Flush interval in seconds
        flush_interval_sec=5,
        # Vacuum deleted vectors (tombstone cleanup)
        deleted_threshold=0.2,
        vacuum_min_vector_number=1000,
    ),
)

```

Understanding segments:

Qdrant partitions each collection into **segments** — immutable or appendable units of storage that each contain a subset of points. New points land in a mutable “appendable” segment. When that segment grows beyond `memmap_threshold`, the optimizer merges it into a larger immutable segment backed by a memory-mapped file. The immutable segment then gets an HNSW index built on it (once it exceeds `indexing_threshold`).

This architecture means that very fresh data that hasn’t been incorporated into a segment HNSW index yet is searched via brute force, while indexed segments use HNSW. If your collection receives a heavy write burst followed immediately by queries, you may see temporarily elevated latency or reduced recall until the optimizer catches up. Monitoring `indexed_vectors_count` vs. `points_count` from `get_collection()` gives you visibility into this lag.

17.5 Named Vectors and Multivector Collections

A practical scenario that arises frequently: you want to search the same items with multiple embeddings — perhaps a text embedding for semantic search and a separate image embedding for visual similarity. Qdrant handles this natively with **named vectors**.

```
# Python
from qdrant_client.models import NamedVector, NamedSparseVector, SparseVector

# Create a collection with multiple named vector spaces
client.create_collection(
    collection_name="multimodal_products",
    vectors_config={
        "text": VectorParams(
            size=384,
            distance=Distance.COSINE,
        ),
        "image": VectorParams(
            size=512,
            distance=Distance.COSINE,
        ),
        "price_embedding": VectorParams(
            size=32,
            distance=Distance.EUCLID,
        ),
    },
)

# Insert a point with all three vector representations
client.upsert(
    collection_name="multimodal_products",
    points=[
        PointStruct(
            id=1,
            vector={
                "text": [0.1, 0.2, 0.3] * 128, # 384-dim placeholder
                "image": [0.4, 0.5, 0.6] * (512 // 3) + [0.4] * (512 % 3),
                "price_embedding": [0.7] * 32,
            },
            payload={"name": "Wireless Headphones", "price": 149.99},
        )
    ],
    wait=True,
)

# Search using only one named vector
text_query = [0.15, 0.25, 0.35] * 128 # placeholder 384-dim query

results = client.search(
    collection_name="multimodal_products",
    query_vector=NamedVector(name="text", vector=text_query),
    limit=5,
    with_payload=True,
)

# Hybrid search: combine results from text and image vectors
# using Qdrant's query API (v1.7+)
from qdrant_client.models import (
    Prefetch,
    FusionQuery,
    Fusion,
)

image_query = [0.45, 0.55, 0.65] * (512 // 3) + [0.45] * (512 % 3)

# Reciprocal Rank Fusion across two vector spaces
hybrid_results = client.query_points(
    collection_name="multimodal_products",
```

```

prefetch=[
    Prefetch(
        query=NamedVector(name="text", vector=text_query),
        limit=20,
    ),
    Prefetch(
        query=NamedVector(name="image", vector=image_query),
        limit=20,
    ),
],
query=FusionQuery(fusion=Fusion.RRF),
limit=10,
with_payload=True,
)

```

Named vectors let you avoid maintaining separate collections per modality, which simplifies payload management considerably — you only need one set of payload indexes rather than synchronized copies across multiple collections.

Sparse vectors for hybrid dense+sparse search:

Qdrant also supports sparse vectors natively, enabling BM25-style keyword search alongside dense semantic search within the same system.

```

# Python
from qdrant_client.models import SparseVectorParams

# Collection with both dense and sparse vector slots
client.create_collection(
    collection_name="hybrid_search_products",
    vectors_config={
        "dense": VectorParams(size=384, distance=Distance.COSINE),
    },
    sparse_vectors_config={
        "sparse": SparseVectorParams(),
    },
)

# Simulate BM25 sparse weights (in production, use a sparse encoder like SPLADE)
def mock_sparse_vector(text: str) -> SparseVector:
    """Toy TF-weighted sparse representation."""
    tokens = text.lower().split()
    token_ids = [hash(t) % 30_000 for t in tokens]
    weights = [1.0 / (tokens.count(t) ** 0.5) for t in tokens]
    # deduplicate by index
    combined = {}
    for idx, w in zip(token_ids, weights):
        combined[idx] = combined.get(idx, 0.0) + w
    indices = list(combined.keys())
    values = list(combined.values())
    return SparseVector(indices=indices, values=values)

# Upsert a point with both dense and sparse vectors
import numpy as np

client.upsert(
    collection_name="hybrid_search_products",
    points=[
        PointStruct(
            id=1,
            vector={
                "dense": np.random.rand(384).tolist(),
                "sparse": mock_sparse_vector("wireless noise canceling headphones"),
            },
            payload={"name": "Wireless Headphones"},
        )
    ],
    wait=True,
)

```

```

)

# Hybrid query: fuse dense semantic search with sparse keyword search
dense_query = np.random.rand(384).tolist()
sparse_query = mock_sparse_vector("wireless headphones long battery")

hybrid_results = client.query_points(
    collection_name="hybrid_search_products",
    prefetch=[
        Prefetch(
            query=NamedVector(name="dense", vector=dense_query),
            limit=20,
        ),
        Prefetch(
            query=NamedSparseVector(name="sparse", vector=sparse_query),
            limit=20,
        ),
    ],
    query=FusionQuery(fusion=Fusion.RRF),
    limit=10,
    with_payload=True,
)

```

17.6 Scroll, Recommendation, and Discovery APIs

Qdrant exposes several query modes beyond the standard `search()` endpoint. Each addresses a different retrieval pattern.

Scroll — paginated full collection scan with filters:

When you need to iterate over all matching points (not just the nearest neighbors), `scroll()` provides cursor-based pagination. This is useful for bulk exports, backfill operations, and building recommendation candidate sets from structured criteria alone.

```

# Python
from qdrant_client.models import ScrollRequest

# Retrieve all in-stock electronics, paginated
offset = None # start from the beginning
all_electronics = []

while True:
    results, next_offset = client.scroll(
        collection_name=COLLECTION_NAME,
        scroll_filter=Filter(
            must=[
                FieldCondition(key="category", match=MatchValue(value="electronics")),
                FieldCondition(key="in_stock", match=MatchValue(value=True)),
            ]
        ),
        limit=100,
        offset=offset,
        with_payload=True,
        with_vectors=False,
    )
    all_electronics.extend(results)
    if next_offset is None:
        break
    offset = next_offset

print(f"Total matching points: {len(all_electronics)}")

```

Recommendation API — finding similar items without a query vector:

The recommendation endpoint accepts one or more positive example point IDs and optional negative example IDs. Qdrant internally fetches those points' vectors, computes an average positive direction, and performs the search. This is the correct abstraction for “more like this” UI patterns without requiring the client to store or re-encode vectors.

```
# Python
from qdrant_client.models import RecommendRequest

# "More like this product, but not like that one"
recommendations = client.recommend(
    collection_name=COLLECTION_NAME,
    positive=["product-id-1", "product-id-2"],  # IDs of liked items
    negative=["product-id-3"],                  # IDs of disliked items
    query_filter=Filter(
        must=[FieldCondition(key="in_stock", match=MatchValue(value=True))]
    ),
    limit=5,
    with_payload=True,
)

for rec in recommendations:
    print(f"Score: {rec.score:.4f} | {rec.payload['description']}")
```

Discovery API — constrained exploration:

The discovery endpoint extends the recommendation concept with **context pairs** — pairs of (positive, negative) examples that define a preference direction without requiring query vectors. It is useful when you want to explore a region of the vector space defined by relative preferences rather than absolute query proximity.

```
# Python
from qdrant_client.models import DiscoverRequest, ContextExamplePair

discovery_results = client.discover(
    collection_name=COLLECTION_NAME,
    target="product-id-1",  # item to find neighbors of
    context=[
        ContextExamplePair(
            positive="product-id-4",
            negative="product-id-3",
        )
    ],
    query_filter=Filter(
        must=[FieldCondition(key="category", match=MatchValue(value="electronics"))]
    ),
    limit=5,
    with_payload=True,
)
```

17.7 Production Deployment Patterns

Running Qdrant in production involves decisions about persistence, sharding, replication, and access control. This section covers the patterns that matter most for teams moving from prototype to production.

Persistence and snapshots:

Qdrant stores its data in a configurable `storage_path` directory. The default on-disk format uses memory-mapped files for large segments, which means the OS page cache provides an implicit read

buffer. For cloud deployments, map this path to a persistent volume — losing the storage directory means losing all data.

Qdrant provides a snapshot API for point-in-time backups:

```
# Python
# Create a snapshot of a collection
snapshot_info = client.create_snapshot(collection_name=COLLECTION_NAME)
print(f"Snapshot created: {snapshot_info.name}")

# List available snapshots
snapshots = client.list_snapshots(collection_name=COLLECTION_NAME)
for s in snapshots:
    print(f" {s.name} - {s.size} bytes - created at {s.creation_time}")

# Restore from snapshot (via REST, since Python SDK uses the same endpoint)
# POST /collections/{collection_name}/snapshots/recover
# { "location": "file:///qdrant/snapshots/collection-name/snapshot-name.snapshot" }
```

Distributed mode — shards and replication:

Qdrant's distributed mode partitions a collection into **shards**, each of which is an independent HNSW index. Shards can be replicated across nodes for fault tolerance. The configuration happens at collection creation:

```
# Python
# Create a distributed collection - requires Qdrant cluster mode
client.create_collection(
    collection_name="distributed_catalog",
    vectors_config=VectorParams(size=384, distance=Distance.COSINE),
    shard_number=4,           # 4 shards distributed across nodes
    replication_factor=2,     # each shard replicated to 2 nodes
    write_consistency_factor=1, # 1 replica must confirm writes
)
```

Queries in distributed mode are automatically broadcast to all shards, and results are merged. The write consistency factor controls the durability vs. latency tradeoff: requiring all replicas to confirm (`write_consistency_factor=replication_factor`) maximizes durability but adds write latency proportional to replica count.

API key authentication and TLS:

Qdrant supports simple API key authentication and TLS termination, configured via the `config.yaml` file rather than the client:

```
# qdrant config.yaml (server-side)
service:
  api_key: "your-secret-api-key"
  enable_tls: true
  tls:
    cert: /path/to/server.crt
    key: /path/to/server.key

storage:
  storage_path: /qdrant/storage
  snapshots_path: /qdrant/snapshots

# Distributed mode
cluster:
  enabled: true
  p2p:
    port: 6335
  consensus:
    tick_period_ms: 100
```

On the client side, pass the API key at construction:

```
# Python
client = QdrantClient(
    host="your-qdrant-host",
    port=6333,
    api_key="your-secret-api-key",
    https=True,           # enables TLS verification
    prefix="optional/prefix", # useful when behind a path-routing reverse proxy
)
```

Monitoring collection health:

```
# Python
def collection_health_report(client: QdrantClient, collection_name: str) -> dict:
    """
    Returns a structured health summary useful for alerting dashboards.
    """
    info = client.get_collection(collection_name)
    config = info.config

    total_points      = info.points_count
    indexed_vectors    = info.indexed_vectors_count
    index_lag         = total_points - indexed_vectors

    segments = client.get_collection(collection_name).segments_count

    return {
        "collection":      collection_name,
        "status":           info.status.value,
        "total_points":     total_points,
        "indexed_vectors":  indexed_vectors,
        "index_lag":        index_lag,
        "index_lag_pct":    round(index_lag / max(total_points, 1) * 100, 2),
        "segments":         segments,
        "vector_size":      config.params.vectors.size,
        "distance":         config.params.vectors.distance.value,
        "m":                config.hnsw_config.m,
        "ef_construct":     config.hnsw_config.ef_construct,
    }

report = collection_health_report(client, COLLECTION_NAME)
for k, v in report.items():
    print(f" {k:<22} {v}")
```

Write-ahead log and crash recovery:

Qdrant uses a write-ahead log (WAL) to ensure that accepted writes survive crashes even before segments are flushed. The WAL is replayed on startup. This means your data is safe after `wait=True` upserts, but a large WAL that hasn't been checkpointed will slow down restart time. The `flush_interval_sec` optimizer setting controls how frequently WAL entries are checkpointed to segments.

17.8 Practical Integration Pattern: Tenant-Isolated Search

A common production pattern when building multi-tenant SaaS applications on Qdrant is to use a single collection with a `tenant_id` payload field and a keyword index, rather than a separate collection per tenant. This avoids the overhead of managing thousands of collections while still delivering tenant-isolated results.

```
# Python
import hashlib
from qdrant_client.models import (
    Filter, FieldCondition, MatchValue,
```

```

    PointStruct, VectorParams, Distance,
)
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient

client = QdrantClient(host="localhost", port=6333)
encoder = SentenceTransformer("all-MiniLM-L6-v2")

COLLECTION = "saas_documents"

# One-time setup
client.recreate_collection(
    collection_name=COLLECTION,
    vectors_config=VectorParams(size=384, distance=Distance.COSINE),
)
client.create_payload_index(
    collection_name=COLLECTION,
    field_name="tenant_id",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=COLLECTION,
    field_name="doc_type",
    field_schema="keyword",
)
client.create_payload_index(
    collection_name=COLLECTION,
    field_name="created_at",
    field_schema="integer", # Unix timestamp - integer index supports range queries
)

def deterministic_point_id(tenant_id: str, doc_id: str) -> int:
    """
    Generate a stable integer point ID from tenant + document ID.
    Qdrant integer IDs must be uint64, so we truncate the hash.
    """
    h = hashlib.sha256(f"{tenant_id}:{doc_id}".encode()).digest()
    return int.from_bytes(h[:8], "big") >> 1 # shift to keep within signed range

def upsert_document(
    tenant_id: str,
    doc_id: str,
    text: str,
    doc_type: str,
    created_at: int,
) -> None:
    vector = encoder.encode(text).tolist()
    point_id = deterministic_point_id(tenant_id, doc_id)
    client.upsert(
        collection_name=COLLECTION,
        points=[
            PointStruct(
                id=point_id,
                vector=vector,
                payload={
                    "tenant_id": tenant_id,
                    "doc_id": doc_id,
                    "doc_type": doc_type,
                    "created_at": created_at,
                    "text": text,
                },
            )
        ],
        wait=True,
    )

def tenant_search(
    tenant_id: str,
    query: str,
    doc_type: str | None = None,

```

```

    min_created_at: int | None = None,
    limit: int = 10,
) -> list[dict]:
    """
    Vector search scoped strictly to a single tenant,
    with optional document type and recency filters.
    """
    must_conditions = [
        FieldCondition(key="tenant_id", match=MatchValue(value=tenant_id))
    ]
    if doc_type is not None:
        must_conditions.append(
            FieldCondition(key="doc_type", match=MatchValue(value=doc_type))
        )
    if min_created_at is not None:
        must_conditions.append(
            FieldCondition(key="created_at", range=Range(gte=min_created_at))
        )

    query_vector = encoder.encode(query).tolist()

    results = client.search(
        collection_name=COLLECTION,
        query_vector=query_vector,
        query_filter=Filter(must=must_conditions),
        limit=limit,
        with_payload=["doc_id", "doc_type", "text", "created_at"],
    )

    return [
        {
            "score": r.score,
            "doc_id": r.payload["doc_id"],
            "doc_type": r.payload["doc_type"],
            "text": r.payload["text"],
            "created_at": r.payload["created_at"],
        }
        for r in results
    ]

# Example usage
import time

upsert_document(
    tenant_id="acme_corp",
    doc_id="kb-001",
    text="How to reset your password using the account settings page",
    doc_type="knowledge_base",
    created_at=int(time.time()),
)

upsert_document(
    tenant_id="acme_corp",
    doc_id="ticket-0042",
    text="User reports login failure after password reset attempt",
    doc_type="support_ticket",
    created_at=int(time.time()),
)

results = tenant_search(
    tenant_id="acme_corp",
    query="login problems after changing password",
    limit=5,
)

for r in results:
    print(f"{r['score']:.4f} ({r['doc_type']}) {r['text'][:80]}")

```

This pattern has one significant consideration: if a single tenant accounts for a very large fraction

of all points (say, one tenant holds 80% of the data), that tenant's searches will traverse more nodes during HNSW graph construction and querying than a balanced distribution would imply. For severely skewed tenants, the dedicated-collection approach for that specific tenant is worth the operational complexity.

Key Takeaways

- **Qdrant's filtered HNSW** integrates payload conditions directly into graph traversal rather than applying them as a post-filter step. This keeps filtered search latency close to unfiltered latency even at selectivities of 1% or lower, and automatically falls back to brute-force over the matching subset when the result set is very small.
 - **Payload indexes must be created explicitly** before those fields become efficient filter predicates. The field type you choose — keyword, integer, float, geo, or text — determines which comparison operators are available and how the index is physically structured. Creating indexes before bulk insert is faster than creating them afterward.
 - **Quantization (scalar, product, or binary) combined with rescoring** is the primary lever for reducing memory footprint without sacrificing meaningful recall. The `always_ram=True` option for quantized vectors keeps the compressed representation in memory for fast candidate retrieval while the full-precision originals can live on disk and are only read for the rescore step.
 - **Named vectors and sparse vector support** allow a single collection to serve multiple retrieval modalities — dense semantic search, sparse keyword search, and cross-modal fusion — without synchronizing parallel collections. The `prefetch + FusionQuery(RRF)` pattern is the idiomatic way to implement hybrid search in Qdrant v1.7+.
 - **Multi-tenant isolation via payload filtering** (single collection, `tenant_id` keyword index) is operationally simpler than per-tenant collections at moderate tenant counts. The pattern breaks down when tenant data is severely skewed, at which point dedicated collections for the largest tenants become preferable.
-

Chapter 18: Chroma: Local-First Vector DB for Development

Chroma occupies a unique position in the vector database landscape: it is the tool developers reach for first, the one that runs in a Python process with zero infrastructure, and the one that disappears from the conversation the moment a team decides to move to production. That reputation is both earned and slightly unfair. Chroma is a genuinely capable vector store that handles millions of embeddings, supports persistent disk-backed storage, and can run as a standalone server with a client/server split that mirrors how teams actually work. What it lacks is the operational maturity of Pinecone or Weaviate — but for local development, rapid prototyping, and small-to-medium deployments where you control the machine, it is often the most productive choice. This chapter covers Chroma’s architecture, its embedding and collection model, persistent versus in-memory operation, querying patterns that transfer directly to production systems, and the specific scenarios where Chroma remains the right answer even after your application ships.

18.1 Chroma’s Architecture: An Embedded Database Perspective

Database professionals will recognize Chroma’s design immediately once you map it to familiar concepts. Chroma is, at its core, an embedded vector database in the same lineage as SQLite — it runs inside your process, writes to a local directory, and requires no separate daemon. This is its defining characteristic and the source of both its appeal and its limitations.

The storage model

Chroma uses a layered storage architecture. The vector index is backed by HNSW (Hierarchical Navigable Small World), the same algorithm used internally by pgvector and Weaviate. Metadata and document text are stored in DuckDB in versions prior to 0.4, and in Chroma’s own embedded SQLite-based system in the 0.4+ architecture. This separation — vectors in an HNSW structure, metadata in a columnar or relational store — mirrors what you see in every production vector database. Chroma just makes the layers visible in a single local directory.

The **chroma** directory after initialization looks like this:

```
# shell
ls -la ./chroma_data/
# chroma.sqlite3          <- metadata, collections, embeddings catalog
# <uuid>/                 <- one directory per collection
#   data_level0.bin       <- HNSW graph (bottom layer, all nodes)
#   header.bin            <- index parameters
#   length.bin            <- node count
```

```
# link_lists.bin      <- HNSW upper-layer links
```

If you have managed PostgreSQL databases, think of `chroma.sqlite3` as the system catalog and the HNSW binary files as a specialized index tablespace. The collection UUID directory is analogous to a table's data files — the collection is the primary organizing unit in Chroma, equivalent to a table with a mandatory vector column and optional metadata columns.

Collections as tables

A Chroma collection holds:

- **IDs** — string primary keys (you supply these, no auto-increment)
- **embeddings** — float arrays of fixed dimension (dimensionality is inferred from the first insert)
- **documents** — optional raw text (the source that was embedded)
- **metadatas** — optional key/value dictionaries (analogous to non-indexed columns)

The dimension constraint works like a typed column: once you insert a 1536-dimensional embedding, every subsequent insert must match. Mixing models is a schema violation Chroma enforces at insert time.

Client modes

Chroma supports three operational modes that map to a deployment progression:

Mode	Use Case	Analogy
Ephemeral (in-memory)	Unit tests, notebooks	<code>:memory:</code> SQLite
Persistent (embedded)	Local dev, single-process apps	File-based SQLite
Client/Server (HTTP)	Multi-process, shared dev environment	Separate DB server

Most tutorials show only the ephemeral mode. Production-adjacent usage requires understanding the persistent and client/server modes, which this chapter covers in detail.

18.2 Getting Started: Collections, Embeddings, and Basic Queries

Install Chroma with its default dependencies, which include the HNSW library and sentence-transformers for local embedding generation:

```
# shell
pip install chromadb sentence-transformers openai
# For the server mode:
pip install "chromadb[server]"
```

Ephemeral client (testing and notebooks)

```
# Python
import chromadb

# In-memory client - resets on process exit
client = chromadb.EphemeralClient()

# Create a collection with cosine similarity (default is L2)
collection = client.create_collection(
    name="product_docs",
```

```

    metadata={"hnsw:space": "cosine"}
)

# Add documents - Chroma uses its default embedding function if you don't specify one
# Default is all-MiniLM-L6-v2 via sentence-transformers (384 dimensions)
collection.add(
    documents=[
        "PostgreSQL supports JSONB for semi-structured data",
        "pgvector extends PostgreSQL with vector similarity search",
        "Chroma is an open-source embedding database",
        "HNSW indexing provides approximate nearest neighbor search",
        "RAG pipelines combine retrieval with language model generation",
    ],
    ids=["doc_001", "doc_002", "doc_003", "doc_004", "doc_005"],
    metadatas=[
        {"source": "pg_docs", "category": "database"},
        {"source": "pg_docs", "category": "vector"},
        {"source": "chroma_docs", "category": "vector"},
        {"source": "research", "category": "algorithms"},
        {"source": "ml_docs", "category": "architecture"},
    ]
)

# Query - Chroma embeds the query text automatically using the same function
results = collection.query(
    query_texts=["how does vector similarity search work?"],
    n_results=3,
    include=["documents", "metadatas", "distances"]
)

for i, (doc, meta, dist) in enumerate(zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]
)):
    print(f"Rank {i+1} | Distance: {dist:.4f} | Source: {meta['source']}")
    print(f"    {doc}\n")

```

The `include` parameter controls what Chroma returns — omit `"embeddings"` unless you need the raw vectors, since returning full embedding arrays for large result sets is expensive.

Persistent client with metadata filtering

```

# Python
import chromadb
from chromadb.utils import embedding_functions

# Persistent client - survives process restart
client = chromadb.PersistentClient(path="./chroma_data")

# Use OpenAI embeddings instead of the default local model
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="sk-...", # or read from env: os.environ["OPENAI_API_KEY"]
    model_name="text-embedding-3-small" # 1536 dimensions
)

# get_or_create is idempotent - safe to call on restart
collection = client.get_or_create_collection(
    name="engineering_kb",
    embedding_function=openai_ef,
    metadata={"hnsw:space": "cosine"}
)

# Batch insert - Chroma handles the embedding calls internally
documents = [
    "Connection pooling reduces database overhead by reusing connections",
    "Read replicas distribute query load across multiple database instances",
    "Partitioning splits large tables into smaller, manageable segments",
    "Vacuuming reclaims storage from deleted rows in PostgreSQL",
]

```

```

    "WAL (Write-Ahead Logging) ensures durability and enables replication",
]
ids = [f"kb_{i:04d}" for i in range(len(documents))]
metadatas = [
    {"topic": "performance", "db": "general"},
    {"topic": "scaling", "db": "general"},
    {"topic": "storage", "db": "general"},
    {"topic": "maintenance", "db": "postgresql"},
    {"topic": "reliability", "db": "postgresql"},
]

collection.add(documents=documents, ids=ids, metadatas=metadatas)

# Query with metadata filter - equivalent to WHERE clause
results = collection.query(
    query_texts=["how to improve query performance?"],
    n_results=3,
    where={"db": "postgresql"},          # metadata filter
    include=["documents", "metadatas", "distances"]
)

print(f"Found {len(results['ids'][0])} results filtered to PostgreSQL topics:")
for doc, meta, dist in zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]
):
    print(f"  [{meta['topic']}] dist={dist:.4f}: {doc[:60]}...")

```

The `where` clause supports operators that map directly to SQL predicates:

```

# Python
# Single condition
where={"category": "vector"}

# AND condition
where={"$and": [{"category": "vector"}, {"source": "pg_docs"}]}

# OR condition
where={"$or": [{"db": "postgresql"}, {"db": "general"}]}

# Comparison operators
where={"score": {"$gte": 0.8}}          # score >= 0.8
where={"year": {"$in": [2023, 2024]}}  # year IN (2023, 2024)
where={"status": {"$ne": "archived"}}  # status != 'archived'

```

For DBA readers: these operators compile down to SQLite predicates against the metadata store before the HNSW search executes. Pre-filtering on metadata is more efficient than post-filtering large result sets, particularly when the metadata column has high cardinality.

18.3 Embedding Functions: Wiring Chroma to Your Model Layer

Chroma's embedding function abstraction is one of its more thoughtful design decisions. Rather than requiring you to compute embeddings externally and pass float arrays, Chroma lets you attach an embedding function to a collection and then work exclusively with text. The function is called transparently during `add()` and `query()`.

Built-in embedding functions

```

# Python
from chromadb.utils import embedding_functions

# 1. Default: all-MiniLM-L6-v2 (384-dim, runs locally, no API key needed)

```

```

default_ef = embedding_functions.DefaultEmbeddingFunction()

# 2. OpenAI - text-embedding-3-small (1536-dim) or text-embedding-3-large (3072-dim)
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="sk-...",
    model_name="text-embedding-3-small"
)

# 3. Cohere - good multilingual support
cohere_ef = embedding_functions.CohereEmbeddingFunction(
    api_key="...",
    model_name="embed-english-v3.0"
)

# 4. Sentence Transformers - local, no API costs, many model choices
st_ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-large-en-v1.5" # 1024-dim, strong retrieval performance
)

# 5. HuggingFace Inference API - cloud-hosted open models
hf_ef = embedding_functions.HuggingFaceEmbeddingFunction(
    api_key="hf_...",
    model_name="sentence-transformers/all-mpnet-base-v2"
)

```

Custom embedding function for Azure OpenAI or AWS Bedrock

When your organization's models are hosted on enterprise AI platforms, you write a custom embedding function by subclassing `EmbeddingFunction`:

```

# Python
import numpy as np
from chromadb import EmbeddingFunction, Documents, Embeddings
import boto3
import json

class BedrockEmbeddingFunction(EmbeddingFunction):
    """Chroma embedding function backed by Amazon Bedrock Titan Embeddings."""

    def __init__(self, region_name: str = "us-east-1"):
        self.client = boto3.client("bedrock-runtime", region_name=region_name)
        self.model_id = "amazon.titan-embed-text-v1"

    def __call__(self, input: Documents) -> Embeddings:
        embeddings = []
        for text in input:
            body = json.dumps({"inputText": text})
            response = self.client.invoke_model(
                modelId=self.model_id,
                body=body,
                contentType="application/json",
                accept="application/json"
            )
            result = json.loads(response["body"].read())
            embeddings.append(result["embedding"])
        return embeddings

class AzureOpenAIEmbeddingFunction(EmbeddingFunction):
    """Chroma embedding function backed by Azure OpenAI."""

    def __init__(
        self,
        azure_endpoint: str,
        api_key: str,
        deployment_name: str,
        api_version: str = "2024-02-01"
    ):
        from openai import AzureOpenAI

```

```

self.client = AzureOpenAI(
    azure_endpoint=azure_endpoint,
    api_key=api_key,
    api_version=api_version
)
self.deployment_name = deployment_name

def __call__(self, input: Documents) -> Embeddings:
    # Azure OpenAI supports batch embedding - send all texts at once
    response = self.client.embeddings.create(
        input=input,
        model=self.deployment_name
    )
    # Sort by index to preserve order
    sorted_data = sorted(response.data, key=lambda x: x.index)
    return [item.embedding for item in sorted_data]

# Usage
bedrock_ef = BedrockEmbeddingFunction(region_name="us-west-2")
azure_ef = AzureOpenAIEmbeddingFunction(
    azure_endpoint="https://your-resource.openai.azure.com",
    api_key="...",
    deployment_name="text-embedding-3-small"
)

client = chromadb.PersistentClient(path="./chroma_data")
collection = client.get_or_create_collection(
    name="enterprise_docs",
    embedding_function=azure_ef,
    metadata={"hnsw:space": "cosine"}
)

```

A dimension mismatch is a hard schema error. If you create a collection with `text-embedding-3-small` (1536 dimensions) and later try to query it with a custom function that returns 768-dimensional vectors, Chroma raises a `ValueError` at query time. There is no silent dimension reduction. Treat the embedding function attached to a collection as a schema constraint — document it alongside your collection metadata.

18.4 Client/Server Mode and LangChain Integration

Running Chroma as a server unlocks multi-process access, which is the minimum viable architecture for any application with a web server, background workers, and a database — they cannot all share an embedded client safely. The server exposes a FastAPI HTTP interface, and the Python client speaks to it over HTTP.

Starting the Chroma server

```

# shell
# Install with server dependencies
pip install "chromadb[server]"

# Start server on default port 8000
chroma run --path ./chroma_data --host 0.0.0.0 --port 8000

# Or with Docker - useful for CI and team shared environments
docker run -d \
    --name chroma \
    -p 8000:8000 \
    -v $(pwd)/chroma_data:/chroma/chroma \
    -e IS_PERSISTENT=TRUE \
    chromadb/chroma:latest

```



```
# Verify the server is running
curl http://localhost:8000/api/v1/heartbeat
# {"nanosecond heartbeat": 1234567890}

curl http://localhost:8000/api/v1/collections
# Returns list of existing collections as JSON
```

Connecting from Python (client/server mode)

```
# Python
import chromadb

# HTTP client - connects to running Chroma server
client = chromadb.HttpClient(
    host="localhost",
    port=8000,
    # Optional: add auth headers if you've configured Chroma with token auth
    # settings=chromadb.Settings(chroma_client_auth_provider="token",
    #                             chroma_client_auth_credentials="secret-token")
)

# Everything else is identical to the embedded client API
collection = client.get_or_create_collection(
    name="shared_collection",
    metadata={"hnsw:space": "cosine"}
)

# Test round-trip
collection.add(
    documents=["Server mode enables multi-process access to Chroma"],
    ids=["server_test_001"]
)

count = collection.count()
print(f"Collection contains {count} documents")

results = collection.query(
    query_texts=["multi-process database access"],
    n_results=1
)
print(results["documents"][0])
```

LangChain integration

LangChain treats Chroma as a `VectorStore`, which means you can swap it for Pinecone or Weaviate by changing two lines. This is exactly the abstraction you want during development — build against Chroma, promote to a production vector database without rewriting retrieval logic.

```
# Python
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document
import chromadb

# Option 1: Let LangChain manage the Chroma client (simple case)
embedding_function = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = Chroma(
    collection_name="langchain_demo",
    embedding_function=embedding_function,
    persist_directory="./chroma_langchain_data"
)

# Add LangChain Document objects (they carry page_content and metadata)
docs = [
    Document(
        page_content="Indexes in PostgreSQL speed up query execution by reducing full table scans",
        metadata={"source": "pg_manual", "chapter": "11", "topic": "indexing"}
    )
]
```

```

    ),
    Document(
        page_content="B-tree indexes are the default index type in PostgreSQL",
        metadata={"source": "pg_manual", "chapter": "11", "topic": "indexing"}
    ),
    Document(
        page_content="GIN indexes are optimal for full-text search and JSONB queries",
        metadata={"source": "pg_manual", "chapter": "11", "topic": "indexing"}
    ),
    Document(
        page_content="Partial indexes include only rows matching a WHERE condition",
        metadata={"source": "pg_manual", "chapter": "11", "topic": "indexing"}
    ),
]

vectorstore.add_documents(docs)

# Similarity search - returns Documents ranked by relevance
query = "what index type should I use for text search?"
similar_docs = vectorstore.similarity_search(query, k=2)
for doc in similar_docs:
    print(f"[{doc.metadata['topic']}] {doc.page_content}")

# Similarity search with scores
docs_with_scores = vectorstore.similarity_search_with_score(query, k=2)
for doc, score in docs_with_scores:
    print(f"Score: {score:.4f} | {doc.page_content[:60]}...")

# Option 2: Connect LangChain to a running Chroma server
http_client = chromadb.HttpClient(host="localhost", port=8000)

server_vectorstore = Chroma(
    client=http_client,
    collection_name="langchain_server_collection",
    embedding_function=embedding_function
)

# Use as a retriever in a RAG chain
retriever = server_vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4, "filter": {"topic": "indexing"}}
)

# The retriever works with any LangChain chain
retrieved = retriever.invoke("how do partial indexes work?")
for doc in retrieved:
    print(doc.page_content)

```

The `as_retriever()` call is the bridge to the full LangChain ecosystem — you can pass this retriever directly to `RetrievalQA`, `ConversationalRetrievalChain`, or `LangGraph` nodes without modification.

18.5 HNSW Tuning, Scaling Limits, and When to Graduate

Chroma exposes HNSW parameters through collection metadata. Understanding these parameters gives you control over the precision/performance tradeoff that every vector index presents — the same tradeoff exists in pgvector's `ivfflat` and Weaviate's HNSW configuration.

HNSW parameters exposed by Chroma

```

# Python
import chromadb

```

```

client = chromadb.PersistentClient(path="./chroma_data")

# Create collection with explicit HNSW tuning
collection = client.get_or_create_collection(
    name="tuned_collection",
    metadata={
        "hnsw:space": "cosine",          # Distance metric: cosine, l2, ip (inner product)
        "hnsw:construction_ef": 200,     # Candidates examined during index build (default: 100)
        "hnsw:search_ef": 100,           # Candidates examined during search (default: 10)
        "hnsw:M": 32,                    # Connections per node in graph (default: 16)
        "hnsw:num_threads": 4,           # Parallel threads for index operations
        "hnsw:resize_factor": 1.2,       # Growth factor when index needs to expand
        "hnsw:batch_size": 100,          # Vectors per batch during construction
        "hnsw:sync_threshold": 1000,     # Flush to disk after N mutations
    }
)

```

Parameter guidance from database operations experience:

Parameter	Default	When to Increase	Memory Impact
<code>hnsw:M</code>	16	Recall < 0.90, dataset > 100K	~2x at M=32
<code>hnsw:construction_ef</code>	100	Recall < 0.95 at query time	Index build time only
<code>hnsw:search_ef</code>	10	Recall < 0.90 in production	Per-query CPU
<code>hnsw:num_threads</code>	1	Batch ingestion too slow	None (CPU only)

The key insight: `hnsw:construction_ef` affects index quality permanently. `hnsw:search_ef` can be tuned per-query without rebuilding the index. If you index with `construction_ef=100` and find recall is poor, you must rebuild — there is no online reindex in Chroma.

Measuring recall and query performance

```

# Python
import chromadb
import time
import numpy as np
from chromadb.utils import embedding_functions

def benchmark_chroma_collection(
    collection,
    query_texts: list[str],
    n_results: int = 10
) -> dict:
    """
    Basic benchmark for Chroma query latency and result consistency.
    In production you'd compare against brute-force results for recall measurement.
    """
    latencies = []

    for query in query_texts:
        start = time.perf_counter()
        results = collection.query(
            query_texts=[query],
            n_results=n_results,
            include=["distances"]
        )
        elapsed_ms = (time.perf_counter() - start) * 1000
        latencies.append(elapsed_ms)

    latency_array = np.array(latencies)
    return {
        "n_queries": len(query_texts),
        "p50_ms": np.percentile(latency_array, 50),
    }

```

```

    "p95_ms": np.percentile(latency_array, 95),
    "p99_ms": np.percentile(latency_array, 99),
    "mean_ms": latency_array.mean(),
    "collection_size": collection.count(),
}

# Example usage
client = chromadb.PersistentClient(path="./chroma_data")
col = client.get_collection("tuned_collection")

test_queries = [
    "vector similarity search algorithms",
    "PostgreSQL index optimization",
    "RAG pipeline architecture",
    "distributed database scaling",
    "embedding model selection",
]

stats = benchmark_chroma_collection(col, test_queries)
print(f"Collection size: {stats['collection_size']:,}")
print(f"P50 latency: {stats['p50_ms']:.1f} ms")
print(f"P95 latency: {stats['p95_ms']:.1f} ms")
print(f"P99 latency: {stats['p99_ms']:.1f} ms")

```

Scaling limits and the graduation decision

Chroma does not publish hard row limits, but practical experience with HNSW implementations establishes soft boundaries that matter for planning:

- **Under 500K vectors:** Chroma is indistinguishable from production vector databases in most workloads. Latency stays in single-digit milliseconds on commodity hardware with tuned `hnsw:search_ef`.
- **500K to 2M vectors:** Performance degrades predictably — HNSW memory footprint grows, and single-node Chroma cannot distribute the index across machines. Query latency at P99 becomes notable.
- **Above 2M vectors:** Chroma is a wrong-size tool. The HNSW index lives entirely in RAM; a 2M vector collection at 1536 dimensions requires roughly 12 GB of RAM for the index alone ($\text{float32} \times 1536 \times 2\text{M} \approx 12\text{ GB}$), before the graph structure overhead.

The other signals that indicate it is time to graduate from Chroma:

1. **Multiple services need concurrent read/write access** — Chroma's server mode works, but it has no connection pooling, no query planner, and no replication.
2. **You need filtered search at scale** — Chroma's pre-filtering approach works well when the metadata filter is highly selective; it degrades when filters are loose and the HNSW graph still needs to scan large subgraphs.
3. **SLA requirements exist** — Chroma has no built-in monitoring, no replication lag metrics, and no managed backup. Operationalizing it requires wrapping it with infrastructure you build yourself.
4. **You need multi-tenancy** — Chroma's collection-per-tenant model works at small tenant counts; at hundreds of tenants, the collection overhead becomes unwieldy.
5. **Your team needs audit logs or access control** — Chroma has no row-level security, no audit trail, and only token-level authentication in server mode.

None of these limitations are criticisms of Chroma's design philosophy. It is an embedded database, and embedded databases are not wrong tools — they are differently scoped tools.

18.6 Real RAG Pipeline with Chroma: End-to-End Example

This section assembles the concepts from earlier in the chapter into a complete, runnable RAG pipeline. The pattern here — document loading, chunking, embedding, storage, retrieval, generation — is identical regardless of which vector database backs the retrieval step.

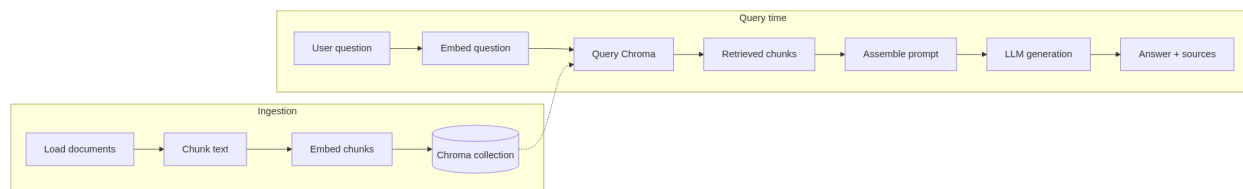


Figure 12: Ingestion (top) populates the Chroma collection once per document set; query time (bottom) embeds the question, retrieves the nearest chunks, and passes them to the LLM as grounding context.

```
# Python
import chromadb
from chromadb.utils import embedding_functions
from openai import OpenAI
import hashlib
import textwrap
from typing import Optional

# Configuration
CHROMA_PATH = "./rag_chroma_data"
COLLECTION_NAME = "technical_kb"
EMBEDDING_MODEL = "text-embedding-3-small"
GENERATION_MODEL = "gpt-4o-mini"
CHUNK_SIZE = 400          # characters per chunk
CHUNK_OVERLAP = 80        # overlap between consecutive chunks
TOP_K = 4                 # number of chunks to retrieve

# Chunking
def chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
    """
    Naive fixed-size chunking with overlap.
    In production, prefer semantic chunking or recursive character splitting.
    """
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end].strip())
        start += chunk_size - overlap
    return [c for c in chunks if len(c) > 50] # drop tiny tail chunks

def doc_id(source: str, chunk_index: int) -> str:
    """Deterministic ID so re-ingestion is idempotent via upsert."""
    raw = f"{source}::{chunk_index}"
    return hashlib.md5(raw.encode()).hexdigest()

# Ingestion
def ingest_documents(
    collection: chromadb.Collection,
    documents: list[dict], # each: {"text": str, "source": str, "category": str}
    chunk_size: int = CHUNK_SIZE,
```

```

    overlap: int = CHUNK_OVERLAP
) -> int:
    """
    Chunk and upsert documents into a Chroma collection.
    Returns total chunks ingested.
    """
    all_ids, all_docs, all metas = [], [], []

    for doc in documents:
        chunks = chunk_text(doc["text"], chunk_size, overlap)
        for i, chunk in enumerate(chunks):
            all_ids.append(doc_id(doc["source"], i))
            all_docs.append(chunk)
            all_metas.append({
                "source": doc["source"],
                "category": doc["category"],
                "chunk_index": i,
                "total_chunks": len(chunks),
            })

    # Upsert is idempotent - safe to call on re-ingestion
    collection.upsert(
        documents=all_docs,
        ids=all_ids,
        metadatas=all_metas
    )
    return len(all_ids)

# Retrieval
def retrieve(
    collection: chromadb.Collection,
    query: str,
    k: int = TOP_K,
    category_filter: Optional[str] = None
) -> list[dict]:
    """
    Retrieve top-k chunks relevant to the query, with optional category filter.
    Returns list of dicts with keys: text, source, distance.
    """
    where = {"category": category_filter} if category_filter else None

    results = collection.query(
        query_texts=[query],
        n_results=k,
        where=where,
        include=["documents", "metadatas", "distances"]
    )

    chunks = []
    for doc, meta, dist in zip(
        results["documents"][0],
        results["metadatas"][0],
        results["distances"][0]
    ):
        chunks.append({
            "text": doc,
            "source": meta["source"],
            "category": meta["category"],
            "distance": dist,
        })
    return chunks

# Generation
def generate_answer(
    openai_client: OpenAI,
    query: str,
    context_chunks: list[dict],
    model: str = GENERATION_MODEL

```

```

) -> str:
    """
    Generate a grounded answer using retrieved context.
    """
    context_text = "\n\n---\n\n".join(
        f"[Source: {c['source']}] \n{c['text']}"
        for c in context_chunks
    )

    system_prompt = textwrap.dedent("""
    You are a technical assistant answering questions about databases and AI systems.
    Answer using only the information in the provided context.
    If the context does not contain enough information, say so explicitly.
    Cite the source document for each claim you make.
    """).strip()

    user_message = f"""Context:
{context_text}

Question: {query}

Answer: """

    response = openai_client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message},
        ],
        temperature=0.1,  # low temperature for factual retrieval tasks
        max_tokens=512,
    )
    return response.choices[0].message.content

# Pipeline assembly
def build_rag_pipeline():
    """Wire up the Chroma collection and OpenAI clients."""
    openai_client = OpenAI()  # reads OPENAI_API_KEY from environment

    openai_ef = embedding_functions.OpenAIEmbeddingFunction(
        model_name=EMBEDDING_MODEL
    )

    chroma_client = chromadb.PersistentClient(path=CHROMA_PATH)
    collection = chroma_client.get_or_create_collection(
        name=COLLECTION_NAME,
        embedding_function=openai_ef,
        metadata={"hnsw:space": "cosine"}
    )

    return openai_client, collection

# Demo run
if __name__ == "__main__":
    openai_client, collection = build_rag_pipeline()

    # Sample corpus - in production, load from files, databases, or APIs
    corpus = [
        {
            "source": "pg_indexing",
            "category": "database",
            "text": (
                "PostgreSQL offers several index types. B-tree indexes support "
                "equality and range queries and are the default. Hash indexes are "
                "optimized for equality comparisons only. GIN indexes support "
                "full-text search and JSONB containment queries efficiently. "
                "GiST indexes support geometric data and full-text search. "
                "BRIN indexes are compact summaries for naturally ordered data such "
            )
        }
    ]

```

```

        "as time-series tables. Partial indexes apply a WHERE clause during "
        "construction so only qualifying rows are indexed, reducing both "
        "index size and maintenance cost. Expression indexes store the result "
        "of a function or expression, useful when queries filter on "
        "transformed columns such as lower(email).",
    ),
},
{
    "source": "pgvector_guide",
    "category": "vector",
    "text": (
        "pgvector adds vector similarity search to PostgreSQL. It introduces "
        "the vector data type, which stores fixed-dimension float arrays. "
        "Two index types are supported: ivfflat, which partitions vectors "
        "into lists and searches the nearest lists, and hnsw, which builds "
        "a hierarchical navigable small world graph for faster approximate "
        "nearest neighbor search. The hnsw index type generally provides "
        "better recall at higher query throughput than ivfflat but requires "
        "more memory and longer build times. Cosine distance (<=>), L2 "
        "distance (<->), and inner product (<#>) operators are all supported."
    ),
},
{
    "source": "rag_architecture",
    "category": "ai",
    "text": (
        "Retrieval-Augmented Generation (RAG) combines a retrieval system "
        "with a language model. At query time, the pipeline embeds the user "
        "question, retrieves semantically similar document chunks from a "
        "vector store, concatenates those chunks as context, and passes the "
        "combined prompt to the language model for generation. The quality "
        "of the final answer depends on retrieval precision - irrelevant "
        "chunks dilute the context window and degrade generation. Reranking "
        "models, such as cross-encoders, can improve retrieval precision by "
        "re-scoring the top-k retrieved chunks before passing them to the "
        "generator. Chunk size, overlap strategy, and metadata filtering all "
        "affect end-to-end RAG quality significantly."
    ),
},
]

print("Ingesting corpus...")
n_chunks = ingest_documents(collection, corpus)
print(f"Ingested {n_chunks} chunks. Collection size: {collection.count()}\n")

# Run example queries
queries = [
    ("What index should I use for full-text search in PostgreSQL?", None),
    ("How does HNSW compare to IVFFlat in pgvector?", "vector"),
    ("How does chunk size affect RAG quality?", "ai"),
]

for query, category in queries:
    print(f"Query: {query}")
    if category:
        print(f"Filter: category={category}")

    chunks = retrieve(collection, query, k=TOP_K, category_filter=category)
    print(f"Retrieved {len(chunks)} chunks:")
    for c in chunks:
        print(f"    [{c['source']}] dist={c['distance']:.4f}: {c['text'][:70]}...")

    answer = generate_answer(openai_client, query, chunks)
    print(f"\nAnswer: {answer}")
    print(" " * 70 + "\n")

```

This pipeline deliberately avoids framework magic so every component is visible. The `ingest_documents` function uses `upsert` rather than `add` so that re-running the script does

not create duplicate entries — a mistake that is easy to make in development and expensive to diagnose when a RAG system returns duplicate context windows.

18.7 Migrating from Chroma to a Production Vector Database

When the graduation signals from Section 18.5 appear, the migration path is straightforward because Chroma’s API surface maps cleanly onto Pinecone, Weaviate, and pgvector. The core operation is: read vectors and metadata from Chroma, write them to the target database.

Exporting from Chroma

```
# Python
import chromadb
import json

def export_chroma_collection(
    chroma_path: str,
    collection_name: str,
    output_path: str,
    batch_size: int = 500
) -> int:
    """
    Export all vectors, documents, and metadata from a Chroma collection
    to a newline-delimited JSON file for migration.
    """
    client = chromadb.PersistentClient(path=chroma_path)
    collection = client.get_collection(collection_name)

    total = collection.count()
    print(f"Exporting {total:,} documents from '{collection_name}'...")

    exported = 0
    with open(output_path, "w") as f:
        # Chroma does not support cursor-based pagination natively;
        # use offset/limit via get() for batch export
        offset = 0
        while offset < total:
            batch = collection.get(
                limit=batch_size,
                offset=offset,
                include=["embeddings", "documents", "metadatas"]
            )

            for record_id, embedding, document, metadata in zip(
                batch["ids"],
                batch["embeddings"],
                batch["documents"],
                batch["metadatas"]
            ):
                record = {
                    "id": record_id,
                    "embedding": embedding,
                    "document": document,
                    "metadata": metadata,
                }
                f.write(json.dumps(record) + "\n")
                exported += 1

            offset += batch_size
            print(f"  Exported {min(offset, total):,} / {total:,}", end="\r")

    print(f"\nExport complete: {exported:,} records written to {output_path}")
    return exported
```

```
# Export
export_chroma_collection(
    chroma_path="./chroma_data",
    collection_name="engineering_kb",
    output_path="./migration_export.jsonl"
)
```

Importing to pgvector

```
# Python
import json
import psycopg2
from psycopg2.extras import execute_values

def import_to_pgvector(
    jsonl_path: str,
    dsn: str,
    table_name: str,
    batch_size: int = 500
) -> int:
    """
    Load Chroma export into a pgvector table.
    Assumes the table has been created with the correct vector dimension.
    """
    conn = psycopg2.connect(dsn)
    cur = conn.cursor()

    # Create table if it does not exist
    # Inspect the first record to get the dimension
    with open(jsonl_path) as f:
        first = json.loads(f.readline())
    dim = len(first["embedding"])

    cur.execute(f"""
        CREATE TABLE IF NOT EXISTS {table_name} (
            id          TEXT PRIMARY KEY,
            embedding   vector({dim}),
            document    TEXT,
            metadata    JSONB
        )
    """)

    cur.execute(f"""
        CREATE INDEX IF NOT EXISTS {table_name}_hnswn_idx
        ON {table_name}
        USING hnswn (embedding vector_cosine_ops)
        WITH (m = 16, ef_construction = 64)
    """)
    conn.commit()

    total_inserted = 0
    batch = []

    with open(jsonl_path) as f:
        for line in f:
            record = json.loads(line)
            batch.append((
                record["id"],
                record["embedding"],
                record["document"],
                json.dumps(record["metadata"])
            ))

        if len(batch) >= batch_size:
            execute_values(
                cur,
                f"""
                INSERT INTO {table_name} (id, embedding, document, metadata)
                VALUES %s
                ON CONFLICT (id) DO UPDATE
            """
```

```

        SET embedding = EXCLUDED.embedding,
        document = EXCLUDED.document,
        metadata = EXCLUDED.metadata
    """
    batch,
    template="(%s, %s::vector, %s, %s::jsonb)"
)
conn.commit()
total_inserted += len(batch)
batch = []

# Final batch
if batch:
    execute_values(
        cur,
        f"""
INSERT INTO {table_name} (id, embedding, document, metadata)
VALUES %s
ON CONFLICT (id) DO UPDATE
    SET embedding = EXCLUDED.embedding,
        document = EXCLUDED.document,
        metadata = EXCLUDED.metadata
    """,
        batch,
        template="(%s, %s::vector, %s, %s::jsonb)"
    )
    conn.commit()
    total_inserted += len(batch)

cur.close()
conn.close()
print(f"Imported {total_inserted:,} records into {table_name}")
return total_inserted

# Import
import_to_pgvector(
    jsonl_path="./migration_export.jsonl",
    dsn="postgresql://user:password@localhost:5432/vectordb",
    table_name="engineering_kb"
)

```

Importing to Pinecone

```

# Python
import json
from pinecone import Pinecone, ServerlessSpec

def import_to_pinecone(
    jsonl_path: str,
    api_key: str,
    index_name: str,
    batch_size: int = 100
) -> int:
    """
    Load Chroma export into a Pinecone serverless index.
    """
    pc = Pinecone(api_key=api_key)

    # Infer dimension from first record
    with open(jsonl_path) as f:
        first = json.loads(f.readline())
    dim = len(first["embedding"])

    # Create index if it does not exist
    if index_name not in [idx.name for idx in pc.list_indexes()]:
        pc.create_index(
            name=index_name,
            dimension=dim,
            metric="cosine",

```

```

        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

    index = pc.Index(index_name)
    total_upserted = 0
    batch = []

    with open(jsonl_path) as f:
        for line in f:
            record = json.loads(line)
            # Pinecone metadata values must be strings, numbers, booleans, or lists
            flat_metadata = {
                k: v for k, v in record["metadata"].items()
                if isinstance(v, (str, int, float, bool, list))
            }
            flat_metadata["document"] = record["document"] # store text in metadata

            batch.append({
                "id": record["id"],
                "values": record["embedding"],
                "metadata": flat_metadata
            })

            if len(batch) >= batch_size:
                index.upsert(vectors=batch)
                total_upserted += len(batch)
                batch = []
                print(f" Upserted {total_upserted:,} vectors", end="\r")

    if batch:
        index.upsert(vectors=batch)
        total_upserted += len(batch)

    print(f"\nImported {total_upserted:,} vectors into Pinecone index '{index_name}'")
    return total_upserted

# Import
import_to_pinecone(
    jsonl_path="./migration_export.jsonl",
    api_key="pcsk...",
    index_name="engineering-kb"
)

```

The migration scripts above re-use the embeddings stored in Chroma rather than re-embedding from scratch — a significant cost saving when the collection contains hundreds of thousands of documents. Re-embedding is only necessary when you switch to a different embedding model as part of the migration, which is a natural time to do so if you are upgrading from a local model to a higher-quality API model.

18.8 Chroma in CI/CD: Testing Vector Features Without a Server

One of Chroma’s most underrated capabilities is its role in automated testing. Every team that builds RAG features eventually needs to test retrieval quality, embedding consistency, and metadata filtering logic without spinning up a full vector database service in CI. Chroma’s ephemeral client makes this possible in a single pytest fixture.

```

# Python - conftest.py
import pytest
import chromadb
from chromadb.utils import embedding_functions
import numpy as np

```

```

@pytest.fixture(scope="session")
def chroma_client():
    """Session-scoped in-memory Chroma client - no disk I/O, resets between test runs."""
    return chromadb.EphemeralClient()

@pytest.fixture(scope="function")
def empty_collection(chroma_client):
    """Function-scoped collection - isolated between individual tests."""
    col = chroma_client.get_or_create_collection(
        name=f"test_collection_{id(object())}",
        metadata={"hnsw:space": "cosine"}
    )
    yield col
    chroma_client.delete_collection(col.name)

@pytest.fixture(scope="session")
def deterministic_ef():
    """
    Deterministic embedding function for testing.
    Produces consistent embeddings so test results are reproducible.
    Uses a simple hash-based approach rather than a real model.
    """
    class HashEmbeddingFunction:
        def __call__(self, input):
            embeddings = []
            for text in input:
                # Seed numpy RNG with hash of text for reproducibility
                rng = np.random.default_rng(abs(hash(text)) % (2**32))
                emb = rng.standard_normal(384).astype(np.float32)
                emb = emb / np.linalg.norm(emb) # normalize to unit sphere
                embeddings.append(emb.tolist())
            return embeddings

    return HashEmbeddingFunction()

@pytest.fixture(scope="session")
def populated_collection(chroma_client, deterministic_ef):
    """Session-scoped collection with known content for retrieval tests."""
    col = chroma_client.get_or_create_collection(
        name="populated_test",
        embedding_function=deterministic_ef,
        metadata={"hnsw:space": "cosine"}
    )
    col.add(
        documents=[
            "PostgreSQL B-tree index for range queries",
            "pgvector HNSW index for similarity search",
            "Redis sorted sets for leaderboard ranking",
            "Elasticsearch inverted index for full-text search",
            "MongoDB atlas search for document retrieval",
        ],
        ids=["pg_btree", "pg_hnsw", "redis_sorted", "es_inverted", "mongo_atlas"],
        metadatas=[
            {"db": "postgresql", "type": "index"},
            {"db": "postgresql", "type": "vector"},
            {"db": "redis", "type": "data_structure"},
            {"db": "elasticsearch", "type": "index"},
            {"db": "mongodb", "type": "search"},
        ]
    )
    yield col
    chroma_client.delete_collection(col.name)

# Test cases

```

```

def test_collection_add_and_count(empty_collection):
    empty_collection.add(
        documents=["test document one", "test document two"],
        ids=["t1", "t2"]
    )
    assert empty_collection.count() == 2

def test_metadata_filter_returns_correct_subset(populated_collection):
    results = populated_collection.query(
        query_texts=["database index"],
        n_results=5,
        where={"db": "postgresql"}
    )
    returned_ids = results["ids"][0]
    # All returned results must be PostgreSQL documents
    assert all(rid in ("pg_btree", "pg_hnsw") for rid in returned_ids)
    # Non-PostgreSQL documents must not appear
    assert "redis_sorted" not in returned_ids
    assert "es_inverted" not in returned_ids

def test_upsert_updates_existing_document(empty_collection):
    empty_collection.add(
        documents=["original content"],
        ids=["doc_1"],
        metadatas=[{"version": 1}]
    )
    empty_collection.upsert(
        documents=["updated content"],
        ids=["doc_1"],
        metadatas=[{"version": 2}]
    )
    result = empty_collection.get(ids=["doc_1"], include=["documents", "metadatas"])
    assert result["documents"][0] == "updated content"
    assert result["metadatas"][0]["version"] == 2
    assert empty_collection.count() == 1 # no duplicate created

def test_delete_removes_document(empty_collection):
    empty_collection.add(
        documents=["doc to delete", "doc to keep"],
        ids=["delete_me", "keep_me"]
    )
    empty_collection.delete(ids=["delete_me"])
    assert empty_collection.count() == 1
    remaining = empty_collection.get(ids=["keep_me"])
    assert remaining["ids"] == ["keep_me"]

def test_query_returns_n_results(populated_collection):
    for k in [1, 2, 3]:
        results = populated_collection.query(
            query_texts=["search index"],
            n_results=k
        )
        assert len(results["ids"][0]) == k

def test_distances_are_bounded_for_cosine(populated_collection):
    """Cosine distance in Chroma is 1 - cosine_similarity, range [0, 2]."""
    results = populated_collection.query(
        query_texts=["vector similarity"],
        n_results=5,
        include=["distances"]
    )
    for dist in results["distances"][0]:
        assert 0.0 <= dist <= 2.0, f"Unexpected cosine distance: {dist}"

```

These tests run in milliseconds — no network calls, no model inference (using the deterministic hash embedder), no external dependencies. They test the logic of your collection schema, filtering predicates, and upsert behavior. The deterministic embedding function is a deliberate choice: real embedding models are non-deterministic under different runtime environments (GPU vs. CPU, model version updates), so pinning embedding behavior for unit tests produces stable CI pipelines.

For integration tests that actually exercise the embedding model, use a separate fixture that marks the tests with `@pytest.mark.integration` and skip them in standard CI runs:

```
# Python
import pytest

@pytest.mark.integration
def test_semantic_retrieval_quality(chroma_client):
    """
    Integration test: verifies that semantic retrieval returns topically
    relevant results using the real embedding model. Requires OPENAI_API_KEY.
    Skipped in standard CI; run in pre-deployment integration stage.
    """
    from chromadb.utils import embedding_functions
    import os

    api_key = os.environ.get("OPENAI_API_KEY")
    if not api_key:
        pytest.skip("OPENAI_API_KEY not set")

    ef = embedding_functions.OpenAIEmbeddingFunction(
        api_key=api_key,
        model_name="text-embedding-3-small"
    )
    col = chroma_client.get_or_create_collection(
        name="integration_test",
        embedding_function=ef,
        metadata={"hnsw:space": "cosine"}
    )

    try:
        col.add(
            documents=[
                "Vacuum reclaims storage from dead tuples in PostgreSQL",
                "HNSW indexing uses a multi-layer graph for approximate search",
                "Connection pooling reduces latency by reusing database connections",
            ],
            ids=["vacuum", "hnsw", "pooling"]
        )

        results = col.query(
            query_texts=["how to reclaim dead row storage?"],
            n_results=1
        )
        top_id = results["ids"][0][0]
        assert top_id == "vacuum", (
            f"Expected 'vacuum' as top result, got '{top_id}'. "
            "Embedding model may have changed behavior."
        )
    finally:
        chroma_client.delete_collection("integration_test")
```

Key Takeaways

- **Chroma is an embedded database first.** Its default operating mode — in-process, single-file, no daemon — makes it the fastest path from idea to working vector search, following

the same philosophy as SQLite for relational data. Client/server mode extends this to multi-process deployments without changing the API surface.

- **The embedding function is a schema constraint.** Attaching an embedding function to a collection fixes both the dimensionality and the semantic space of that collection. Changing the model requires rebuilding the collection. Treat the embedding function choice with the same deliberateness you would apply to choosing a column type.
 - **HNSW parameters set at construction time are permanent.** `hnsw:construction_ef` and `hnsw:M` cannot be changed without rebuilding the index. Set them based on your target recall before ingesting production data; `hnsw:search_ef` can be tuned at query time without rebuilding.
 - **Graduation from Chroma is a data volume and operational maturity decision, not a capability gap.** The filtering, embedding, and retrieval patterns you build against Chroma are directly portable to pgvector, Pinecone, and Weaviate. The migration scripts in this chapter preserve stored embeddings so you do not pay re-embedding costs at migration time.
 - **Chroma's ephemeral client makes vector search testable in CI without infrastructure.** A deterministic embedding function combined with `EphemeralClient` produces fast, stable unit tests for collection schema, metadata filtering, and upsert logic. Reserve real embedding model calls for integration tests that run in pre-deployment stages.
-

Chapter 19: Milvus: Open-Source Vector Search at Enterprise Scale

Milvus occupies a distinctive position in the vector database landscape: it is the most fully-featured open-source vector search engine designed from the ground up for production workloads at scale. While pgvector is the natural extension for teams already on PostgreSQL, and Pinecone offers managed simplicity, Milvus targets organizations that need full control over their infrastructure, support for billions of vectors, and enterprise-grade operational features — without paying managed-service premiums or accepting vendor lock-in. This chapter covers Milvus architecture, deployment patterns, collection and index design, filtering and hybrid search, and the operational patterns that separate a proof-of-concept cluster from a system you can put on-call. Whether you are migrating from a simpler vector store that has started showing its limits, or designing a greenfield AI platform, understanding Milvus at the engineering level gives you the vocabulary and judgment to decide when it is the right tool and how to run it correctly.

19.1 Milvus Architecture: Disaggregated Storage and Compute

Milvus 2.x was rebuilt around a cloud-native, disaggregated architecture. Understanding this design is essential before you write a single line of code, because it directly determines how you size clusters, recover from failures, and tune for throughput versus latency.

The Four Layers

Milvus separates concerns into four functional layers that can scale independently:

Layer	Components	Responsibility
Access	Proxy	Request routing, load balancing, query planning
Coordinator	Root, Query, Data, Index coordinators	Metadata management, segment assignment, topology
Worker	Query Nodes, Data Nodes, Index Nodes	Actual compute: search, ingest, index build
Storage	Object storage (S3/MinIO) + etcd + message bus (Pulsar/Kafka)	Durability, WAL, metadata

This is architecturally analogous to separating the PostgreSQL planner, executor, storage manager, and WAL writer into independently scalable services — something relational databases have only recently begun doing in products like Aurora and Spanner. For vector workloads, this disaggregation matters because index building is CPU-and-memory-intensive work that would otherwise contend with query serving. A Milvus Index Node can build an IVF_FLAT or HNSW index on a 100M-vector segment while Query Nodes continue serving search requests without interference.

Segment Model

Data in Milvus is organized into segments, the fundamental unit of indexing and distribution. Segments start as *growing* (in-memory, accepting inserts), then seal to *sealed* (immutable), and eventually get compacted and persisted to object storage. This is structurally similar to LSM-tree compaction in RocksDB or PostgreSQL’s autovacuum cycle — data flows through a pipeline of mutability states, with index building happening asynchronously on sealed segments.

A sealed segment with an HNSW index on S3 is loaded into Query Node memory on demand. This lazy-load model enables a cluster to host more data than fits in aggregate RAM, at the cost of cold-query latency when a segment must be fetched from object storage. Tune the `dataCoord.segment.maxSize` parameter (default 512 MB) to control segment granularity — smaller segments reduce memory pressure per load but increase metadata overhead.

Message Bus and WAL

Milvus uses Apache Pulsar (or Kafka in enterprise configurations) as its internal message bus. Every insert, delete, and schema change is published as a message and consumed by Data Nodes for persistence and by Query Nodes for real-time visibility. This gives Milvus a *streaming-first* ingestion model: you do not insert into a table and wait for a commit; you publish a record and it becomes queryable within milliseconds once the Query Node consumes the message off the channel. The practical consequence is that Milvus offers *eventual consistency* by default, not the snapshot isolation you get from PostgreSQL. For RAG pipelines where documents are indexed asynchronously after upload, this is usually acceptable. For applications that insert a record and immediately query for it, you must call `collection.flush()` to force segment sealing.

19.2 Deployment: From Docker Compose to Kubernetes

Milvus ships in two operational modes: **Milvus Lite** (in-process, for development only), **Milvus Standalone** (single-node, suitable for up to ~50M vectors), and **Milvus Distributed** (the full multi-node cluster). Zilliz Cloud is the managed SaaS version if you want to outsource operations.

Local Development with Docker Compose

```
# shell
# Download the official Milvus standalone compose file
wget https://github.com/milvus-io/milvus/releases/download/v2.4.0/milvus-standalone-docker-compose.yml \
    -O docker-compose.yml

# Start Milvus standalone with embedded MinIO and etcd
docker-compose up -d

# Verify all services are healthy
docker-compose ps
# Expected: milvus-standalone, milvus-minio, milvus-etcd all running
```

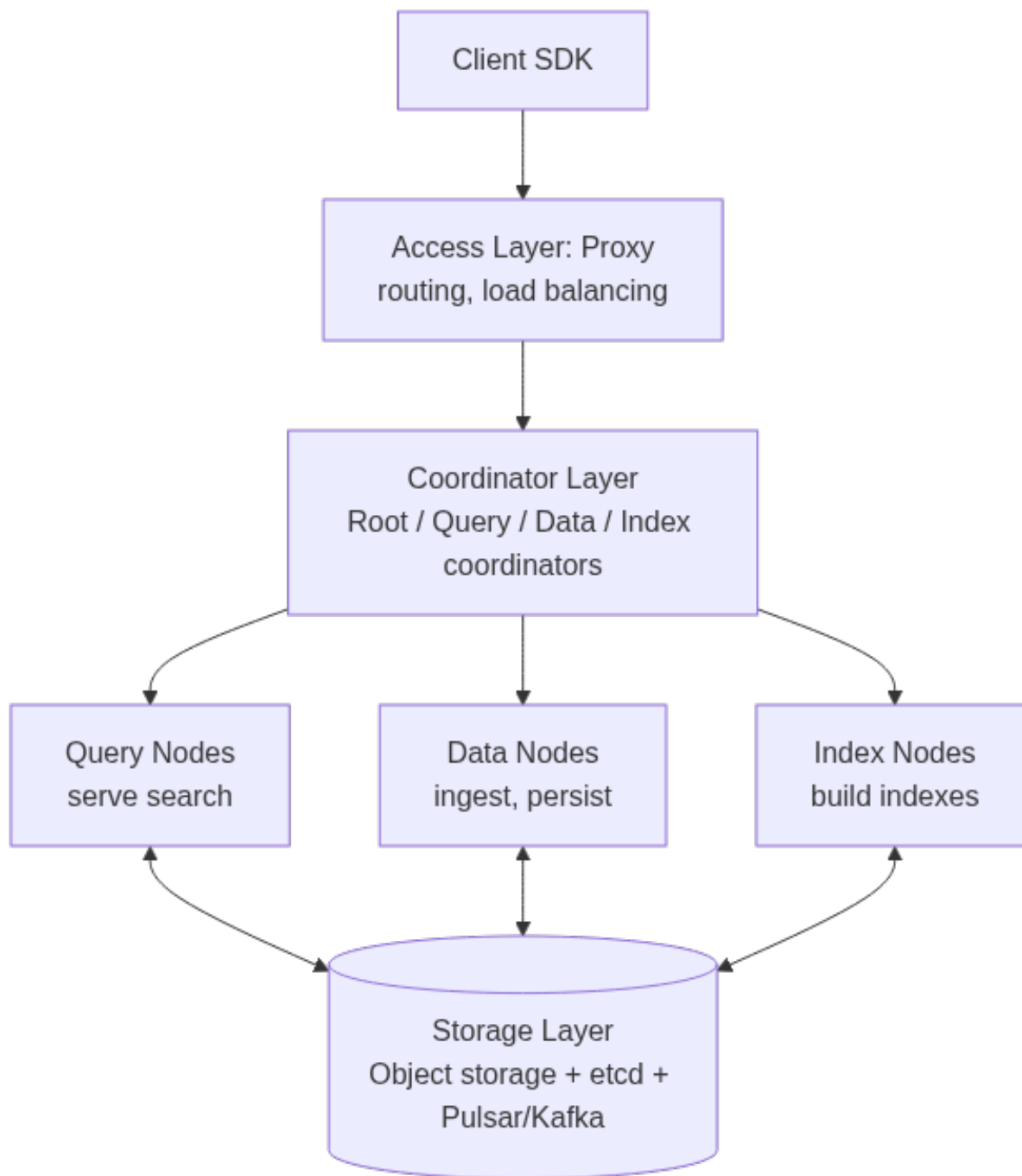


Figure 13: The four layers scale independently: Query Nodes handle search load, Data Nodes handle ingestion, and Index Nodes build indexes in the background — none of them contend with each other for CPU or memory.

```
# Check Milvus version and connectivity
docker exec milvus-standalone milvus --version
```

Python Environment Setup

```
# Python
# Install the Milvus client and supporting libraries
# pip install pymilvus==2.4.0 sentence-transformers openai numpy

from pymilvus import connections, utility

# Connect to local standalone instance
connections.connect(
    alias="default",
    host="localhost",
    port="19530"
)

# Verify connection and list existing collections
print("Milvus version:", utility.get_server_version())
print("Existing collections:", utility.list_collections())
```

Kubernetes Deployment with Helm

For production, use the official Helm chart. The following configuration is a starting point for a medium-scale deployment handling ~200M vectors with mixed read/write workloads:

```
# shell
# Add Milvus Helm repository
helm repo add milvus https://zilliztech.github.io/milvus-helm/
helm repo update

# Create a values override file for production sizing
cat > milvus-values.yaml << 'EOF'
cluster:
  enabled: true

# Proxy handles client connections and query routing
proxy:
  replicas: 2
  resources:
    requests:
      cpu: "2"
      memory: "4Gi"
    limits:
      cpu: "4"
      memory: "8Gi"

# Query nodes serve search requests - size for your working set
queryNode:
  replicas: 4
  resources:
    requests:
      cpu: "8"
      memory: "32Gi"
    limits:
      cpu: "16"
      memory: "64Gi"

# Data nodes handle ingestion and segment persistence
dataNode:
  replicas: 2
  resources:
    requests:
      cpu: "4"
      memory: "8Gi"

# Index nodes build vector indexes asynchronously
indexNode:
```

```
replicas: 2
resources:
  requests:
    cpu: "16"
    memory: "32Gi"

# External object storage - replace with your S3 bucket
minio:
  enabled: false # Use external S3

externalS3:
  enabled: true
  host: "s3.amazonaws.com"
  port: 443
  useSSL: true
  bucketName: "milvus-production"
  accessKey: "" # Use IAM roles or secrets manager
  secretKey: ""

# External etcd for metadata durability
etcd:
  enabled: false

externalEtcd:
  enabled: true
  endpoints:
    - "etcd-0.etcd-headless:2379"
    - "etcd-1.etcd-headless:2379"
    - "etcd-2.etcd-headless:2379"

# Pulsar for message bus
pulsar:
  enabled: true
  broker:
    replicaCount: 2

EOF

# Deploy to the milvus namespace
kubectl create namespace milvus
helm install milvus milvus/milvus \
  --namespace milvus \
  --values milvus-values.yaml \
  --version 4.1.0
```

The key sizing principle for Query Nodes: your working dataset (the collection of segments actively loaded) must fit in aggregate Query Node memory. If your total index size is 160 GB and each Query Node has 32 GB available for data, you need at least 5 Query Nodes. Monitor the `milvus_querynode_cache_hit_rate` Prometheus metric — anything below 85% indicates memory pressure and cold-load latency will dominate your p99.

19.3 Collection Design, Indexing, and Schema Modeling

A Milvus *collection* is the conceptual equivalent of a table. It has a schema with a primary key, scalar fields for metadata, and one or more vector fields. Getting schema design right before you load 500M records saves you from a costly dump-and-reload cycle.

Schema Design and Collection Creation

```
# Python
from pymilvus import (
    connections, Collection, CollectionSchema,
```

```

    FieldSchema, DataType, utility
)
import numpy as np

connections.connect(alias="default", host="localhost", port="19530")

# Define schema for a document chunk collection
# Analogous to CREATE TABLE with column definitions
fields = [
    FieldSchema(
        name="chunk_id",
        dtype=DataType.INT64,
        is_primary=True,
        auto_id=True # Milvus generates IDs, like SERIAL in PostgreSQL
    ),
    FieldSchema(
        name="document_id",
        dtype=DataType.VARCHAR,
        max_length=128 # Scalar field for filtering
    ),
    FieldSchema(
        name="chunk_index",
        dtype=DataType.INT32 # Position within document
    ),
    FieldSchema(
        name="content_type",
        dtype=DataType.VARCHAR,
        max_length=32 # "paragraph", "table", "code", etc.
    ),
    FieldSchema(
        name="created_at",
        dtype=DataType.INT64 # Unix timestamp - Milvus has no native TIMESTAMP
    ),
    FieldSchema(
        name="embedding",
        dtype=DataType.FLOAT_VECTOR,
        dim=1536 # OpenAI text-embedding-3-small dimensions
    ),
    FieldSchema(
        name="sparse_embedding", # For hybrid dense+sparse search
        dtype=DataType.SPARSE_FLOAT_VECTOR
    )
]

schema = CollectionSchema(
    fields=fields,
    description="Document chunks with dense and sparse embeddings",
    enable_dynamic_field=True # Allows schemaless extra fields, like JSONB
)

# Create the collection with 2 shards
# More shards = better write parallelism, but higher metadata overhead
collection = Collection(
    name="document_chunks",
    schema=schema,
    shards_num=2
)

print(f"Collection created: {collection.name}")
print(f"Schema fields: {[f.name for f in collection.schema.fields]}")

# Build index on the dense vector field
# HNSW is the right default for most workloads under 500M vectors
index_params = {
    "metric_type": "COSINE",
    "index_type": "HNSW",
    "params": {
        "M": 16, # Neighbors per node. Range: 4-64. Higher = better recall, more memory
        "efConstruction": 200 # Build-time search depth. Higher = better index quality, slower build
    }
}

```

```

}

collection.create_index(
    field_name="embedding",
    index_params=index_params,
    index_name="dense_hnsw"
)

# Build index on sparse vector field for BM25-style keyword matching
sparse_index_params = {
    "metric_type": "IP",
    "index_type": "SPARSE_INVERTED_INDEX",
    "params": {"drop_ratio_build": 0.2} # Drop bottom 20% of weights to save space
}

collection.create_index(
    field_name="sparse_embedding",
    index_params=sparse_index_params,
    index_name="sparse_inverted"
)

# Build scalar index on document_id for filtered searches
# Equivalent to CREATE INDEX on a relational column
collection.create_index(
    field_name="document_id",
    index_name="doc_id_index"
)

print("Indexes created successfully")

```

Index Type Selection

Milvus supports a broader range of index types than most vector databases. The selection criteria map cleanly to database index selection reasoning:

Index Type	Best For	Memory	Build Time	Recall	Notes
FLAT	<1M vectors, exact recall required	High	Fast	100%	Brute force, no approxima- tion
IVF_FLAT	1M–100M, balanced	Medium	Medium	~98%	nlist controls cluster count
IVF_SQ8	Memory- constrained, 8-bit quantization	Low	Medium	~95%	Good through- put/memory tradeoff
HNSW	<500M, low-latency queries	High	Slow	~99%	Best recall/latency tradeoff
DISKANN	>500M, cannot fit in RAM	Disk	Slow	~97%	Serves from SSD, lower cost
GPU_IVF_FLAT	Batch throughput workloads	GPU VRAM	Fast (GPU)	~98%	Requires CUDA- capable hardware

Index Type	Best For	Memory	Build Time	Recall	Notes
SCANN	High-recall, Google-developed	Medium	Medium	~99%	Competitive with HNSW at scale

For HNSW, the `M` parameter is the most impactful tunable. `M=16` is a good default — `M=32` increases memory usage by roughly 2x but improves recall for high-dimensional spaces (>768 dimensions). The `ef` parameter at query time (not index build time) is the runtime recall/latency lever: start at `ef=64` and increase until recall meets your SLA, then cap it.

19.4 Querying: ANN Search, Filtered Search, and Hybrid Search

Milvus search is richer than the typical “find K nearest neighbors” interface. Understanding its query primitives lets you build sophisticated retrieval patterns that would require multiple round trips in simpler systems.

Basic ANN Search and Filtered Search

```
# Python
from pymilvus import connections, Collection
from sentence_transformers import SentenceTransformer
import time

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")

# Load collection into Query Node memory before searching
# This is the equivalent of a warm cache - cold collections have high first-query latency
collection.load()

# Initialize embedding model
encoder = SentenceTransformer("all-MiniLM-L6-v2")

def search_documents(
    query: str,
    document_ids: list[str] | None = None,
    content_type: str | None = None,
    top_k: int = 10,
    ef: int = 64
) -> list[dict]:
    """
    Search with optional pre-filtering on scalar fields.
    Pre-filtering in Milvus happens before ANN search (unlike post-filtering),
    which maintains recall guarantees but requires scalar indexes.
    """
    query_embedding = encoder.encode([query])[0].tolist()

    # Build filter expression - syntax is similar to SQL WHERE clause
    filter_parts = []
    if document_ids:
        # IN operator for set membership
        ids_str = ", ".join(document_ids)
        filter_parts.append(f'document_id in [{"ids_str}]')
    if content_type:
        filter_parts.append(f'content_type == "{content_type}"')

    filter_expr = " and ".join(filter_parts) if filter_parts else ""
```



```

search_params = {
    "metric_type": "COSINE",
    "params": {"ef": ef} # Runtime recall parameter for HNSW
}

start = time.perf_counter()
results = collection.search(
    data=[query_embedding],
    anns_field="embedding",
    param=search_params,
    limit=top_k,
    expr=filter_expr if filter_expr else None,
    output_fields=["document_id", "chunk_index", "content_type", "created_at"]
)
latency_ms = (time.perf_counter() - start) * 1000

hits = []
for hit in results[0]:
    hits.append({
        "chunk_id": hit.id,
        "score": hit.score,
        "document_id": hit.entity.get("document_id"),
        "chunk_index": hit.entity.get("chunk_index"),
        "content_type": hit.entity.get("content_type"),
        "latency_ms": latency_ms
    })

return hits

# Example: Search within specific documents with content type filter
results = search_documents(
    query="how to configure connection pooling",
    document_ids=["postgresql-docs-v16", "pgbouncer-readme"],
    content_type="paragraph",
    top_k=5
)

for r in results:
    print(f"Score: {r['score']:.4f} | Doc: {r['document_id']} | Chunk: {r['chunk_index']}")

```

Hybrid Search: Dense + Sparse Fusion

Hybrid search — combining dense semantic embeddings with sparse keyword-matching vectors — is one of Milvus’s strongest differentiators. The rationale is well-established: dense retrieval handles paraphrase and semantic similarity, sparse retrieval handles exact keyword matching. Combining them with Reciprocal Rank Fusion (RRF) or weighted scoring consistently outperforms either alone, particularly for technical documentation where exact terms matter.

```

# Python
from pymilvus import (
    connections, Collection, AnnSearchRequest,
    RRFRanker, WeightedRanker
)
from pymilvus.model.hybrid import BGEM3EmbeddingFunction
import numpy as np

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")
collection.load()

# BGE-M3 generates both dense and sparse embeddings in one pass
# This is more efficient than separate dense and sparse encoder calls
embedding_fn = BGEM3EmbeddingFunction(
    model_name="BAAI/bge-m3",
    device="cpu", # Use "cuda" if GPU available
    use_fp16=False
)

```

```

def hybrid_search(
    query: str,
    top_k: int = 10,
    dense_weight: float = 0.7,
    sparse_weight: float = 0.3,
    use_rrf: bool = True
) -> list[dict]:
    """
    Hybrid dense+sparse search with configurable fusion strategy.
    RRF is rank-based and robust to score scale differences.
    WeightedRanker requires scores to be on the same scale.
    """
    # Generate both dense and sparse embeddings
    embeddings = embedding_fn([query])
    dense_vec = embeddings["dense"][0]
    sparse_vec = embeddings["sparse"][[0]] # scipy sparse matrix row

    # Dense search request - HNSW index
    dense_request = AnnSearchRequest(
        data=[dense_vec.tolist()],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 100}},
        limit=top_k * 2 # Over-fetch to allow fusion reranking
    )

    # Sparse search request - inverted index
    sparse_request = AnnSearchRequest(
        data=[sparse_vec],
        anns_field="sparse_embedding",
        param={"metric_type": "IP", "params": {"drop_ratio_search": 0.2}},
        limit=top_k * 2
    )

    # Fusion strategy: RRF (rank-based) vs Weighted (score-based)
    if use_rrf:
        ranker = RRFRanker(k=60) # k=60 is the standard RRF constant
    else:
        ranker = WeightedRanker(dense_weight, sparse_weight)

    results = collection.hybrid_search(
        reqs=[dense_request, sparse_request],
        ranker=ranker,
        limit=top_k,
        output_fields=["document_id", "chunk_index", "content_type"]
    )

    hits = []
    for hit in results[0]:
        hits.append({
            "chunk_id": hit.id,
            "fused_score": hit.score,
            "document_id": hit.entity.get("document_id"),
            "chunk_index": hit.entity.get("chunk_index"),
        })

    return hits

# Compare dense-only vs hybrid for a keyword-sensitive query
query = "pgvector ivfflat index creation syntax"

hybrid_results = hybrid_search(query, use_rrf=True)
print("Hybrid search results (RRF fusion):")
for r in hybrid_results[:5]:
    print(f" Score: {r['fused_score']:.4f} | {r['document_id']}: {r['chunk_index']}")

```

Range Search and Grouping

Two additional search primitives worth knowing: *range search* returns all vectors within a distance threshold (rather than top-K), which is useful for deduplication and similarity clustering.

Grouping search returns the best match per group — equivalent to a `DISTINCT ON (document_id)` in PostgreSQL — preventing a single long document from dominating results with many similar chunks.

```
# Python
from pymilvus import connections, Collection

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")
collection.load()

# Range search: find all chunks within a cosine similarity threshold
# Useful for deduplication - flag chunks with similarity > 0.98 as near-duplicates
def range_search_for_deduplication(embedding: list[float], threshold: float = 0.98):
    results = collection.search(
        data=[embedding],
        anns_field="embedding",
        param={
            "metric_type": "COSINE",
            "params": {
                "ef": 200,
                "radius": 1.0 - threshold,      # Convert similarity threshold to distance
                "range_filter": 0.0            # Include everything from 0 to radius
            }
        },
        limit=100,
        output_fields=["document_id", "chunk_index"]
    )
    return results[0]

# Grouping search: best match per document, prevent any single doc from dominating
def search_with_document_diversity(query_embedding: list[float], top_k: int = 10):
    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 200}},
        limit=top_k,
        group_by_field="document_id",      # One result per unique document_id
        group_size=1,                      # How many results per group
        output_fields=["document_id", "chunk_index", "content_type"]
    )
    return results[0]
```

19.5 Ingestion Pipelines and Data Management

Ingesting data into Milvus at scale requires attention to batching, consistency semantics, and the operational lifecycle of collections. Unlike a relational database where you `INSERT` rows and they are immediately visible to subsequent queries, Milvus's streaming architecture means you need to manage the pipeline explicitly.

Batch Ingestion with OpenAI Embeddings

```
# Python
import os
import time
import numpy as np
from typing import Iterator
from pymilvus import connections, Collection, utility
from openai import OpenAI

connections.connect(alias="default", host="localhost", port="19530")
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
collection = Collection("document_chunks")
```

```

def generate_embeddings_batch(texts: list[str], model: str = "text-embedding-3-small") ->
    list[list[float]]:
    """
    Generate embeddings with automatic retry and rate-limit handling.
    OpenAI allows up to 2048 texts per batch call for embedding models.
    """
    response = openai_client.embeddings.create(
        input=texts,
        model=model
    )
    return [item.embedding for item in response.data]

def chunk_iterator(documents: list[dict], chunk_size: int = 512) -> Iterator[dict]:
    """
    Yield individual chunks from a list of documents.
    Each document is a dict with 'id', 'content_type', and 'text' fields.
    """
    for doc in documents:
        words = doc["text"].split()
        for i in range(0, len(words), chunk_size):
            chunk_text = " ".join(words[i:i + chunk_size])
            yield {
                "document_id": doc["id"],
                "chunk_index": i // chunk_size,
                "content_type": doc.get("content_type", "paragraph"),
                "created_at": int(time.time()),
                "text": chunk_text
            }

def ingest_documents(
    documents: list[dict],
    batch_size: int = 200,
    flush_every_n_batches: int = 10
) -> dict:
    """
    Ingest documents into Milvus with batched embedding generation.
    batch_size=200 balances OpenAI API throughput with memory usage.
    flush_every_n_batches triggers segment sealing to make data queryable.
    """
    total_inserted = 0
    batch_count = 0
    chunks = list(chunk_iterator(documents))

    for batch_start in range(0, len(chunks), batch_size):
        batch = chunks[batch_start:batch_start + batch_size]
        texts = [c["text"] for c in batch]

        # Generate embeddings for this batch
        embeddings = generate_embeddings_batch(texts)

        # Prepare columnar data - Milvus insert expects lists per field
        insert_data = {
            "document_id": [c["document_id"] for c in batch],
            "chunk_index": [c["chunk_index"] for c in batch],
            "content_type": [c["content_type"] for c in batch],
            "created_at": [c["created_at"] for c in batch],
            "embedding": embeddings
        }

        result = collection.insert(insert_data)
        total_inserted += len(result.primary_keys)
        batch_count += 1

        # Periodically flush to seal growing segments and make data queryable
        # Without flush, inserted data is only visible to searches after
        # the message bus consumer processes it - usually within seconds,
        # but flush guarantees visibility
        if batch_count % flush_every_n_batches == 0:
            collection.flush()

```

```

        print(f"Flushed after {total_inserted} insertions")

    # Final flush to ensure all data is persisted and queryable
    collection.flush()

    return {
        "total_chunks": len(chunks),
        "total_inserted": total_inserted,
        "batches_processed": batch_count
    }

# Sample usage
sample_documents = [
    {
        "id": "pg-docs-v16-ch01",
        "content_type": "paragraph",
        "text": "PostgreSQL is a powerful, open-source object-relational database system..."
    },
    {
        "id": "pg-docs-v16-ch02",
        "content_type": "paragraph",
        "text": "Connection pooling reduces the overhead of establishing new database connections..."
    }
]

stats = ingest_documents(sample_documents, batch_size=200)
print(f"Ingestion complete: {stats}")

```

Upsert and Deletion

Milvus supports upserts (insert or replace by primary key) and soft-delete semantics. Deleted records are marked with a tombstone and excluded from search results; the physical space is reclaimed during compaction. This is analogous to PostgreSQL's MVCC model where dead tuples are vacuumed asynchronously.

```

# Python
from pymilvus import connections, Collection

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")

# Upsert: replaces existing record if primary key exists
# Useful when re-processing a document with updated content
def upsert_chunks(chunks_data: dict) -> int:
    result = collection.upsert(chunks_data)
    collection.flush()
    return len(result.primary_keys)

# Delete by primary key - immediate for query purposes after flush
def delete_chunks_by_id(chunk_ids: list[int]):
    id_list = ", ".join(str(i) for i in chunk_ids)
    expr = f"chunk_id in [{id_list}]"
    collection.delete(expr)
    collection.flush()

# Delete all chunks belonging to a document - filter-based delete
def delete_document(document_id: str):
    expr = f'document_id == "{document_id}"'
    collection.delete(expr)
    collection.flush()
    print(f"Deleted all chunks for document: {document_id}")

# Example: remove a document that has been superseded
delete_document("pg-docs-v15-ch01")

```

Compaction and Index Rebuild

After heavy deletes, Milvus segment files contain tombstoned records that waste memory when

loaded. Trigger manual compaction to physically reclaim space, then optionally rebuild indexes if segment fragmentation is high.

```
# Python
from pymilvus import connections, Collection, utility
import time

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("document_chunks")

# Trigger compaction - runs asynchronously in the background
compaction_id = collection.compact()
print(f"Compaction triggered, job ID: {compaction_id}")

# Poll until compaction completes
while True:
    state = collection.get_compaction_state()
    print(f"Compaction state: {state.state}")
    if str(state.state) in ("Completed", "Failed"):
        break
    time.sleep(5)

# Check collection statistics after compaction
stats = utility.get_query_segment_info("document_chunks")
print(f"Segment count after compaction: {len(stats)}")
```

19.6 Multi-Tenancy Patterns

Multi-tenancy in vector databases is a first-class design concern that does not have a single right answer. Milvus offers three patterns, each with different isolation, performance, and operational complexity characteristics — an almost exact parallel to the multi-tenancy patterns in relational database design.

Pattern 1: Collection-per-Tenant

Each tenant has a dedicated collection. This gives maximum isolation: tenant data is physically separate, indexes are independent, and you can drop a tenant's collection without affecting others.

```
# Python
from pymilvus import connections, Collection, CollectionSchema, FieldSchema, DataType, utility

connections.connect(alias="default", host="localhost", port="19530")

def create_tenant_collection(tenant_id: str, vector_dim: int = 1536) -> Collection:
    """
    Create a dedicated collection for a tenant.
    Collection names must be alphanumeric - sanitize tenant IDs.
    """
    collection_name = f"tenant_{tenant_id.replace('-', '_')}"

    if utility.has_collection(collection_name):
        return Collection(collection_name)

    fields = [
        FieldSchema(name="chunk_id", dtype=DataType.INT64, is_primary=True, auto_id=True),
        FieldSchema(name="document_id", dtype=DataType.VARCHAR, max_length=128),
        FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=vector_dim)
    ]
    schema = CollectionSchema(fields=fields)
    collection = Collection(name=collection_name, schema=schema)

    collection.create_index(
        field_name="embedding",
```

```

        index_params={"metric_type": "COSINE", "index_type": "HNSW", "params": {"M": 16,
↪  "efConstruction": 200}}
    )
    return collection

def drop_tenant(tenant_id: str):
    """Remove a tenant's collection entirely - instant, no residual data."""
    collection_name = f"tenant_{tenant_id.replace('-', '_')}"
    if utility.has_collection(collection_name):
        utility.drop_collection(collection_name)
        print(f"Dropped collection for tenant: {tenant_id}")

```

The limitation: Milvus has a practical limit on the number of simultaneously loaded collections (typically in the hundreds, bounded by Query Node memory). This pattern does not scale beyond a few hundred active tenants without implementing a load/unload scheduling layer.

Pattern 2: Partition-per-Tenant

A single collection with tenant-specific partitions. Partitions in Milvus are physical subdivisions of a collection — search can be scoped to one or more partitions, isolating tenant data at query time while sharing a single collection's overhead.

```

# Python
from pymilvus import connections, Collection, Partition

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("shared_document_chunks")

def get_or_create_tenant_partition(tenant_id: str) -> Partition:
    partition_name = f"tenant_{tenant_id}"
    if not collection.has_partition(partition_name):
        collection.create_partition(partition_name)
    return Partition(collection, partition_name)

def insert_for_tenant(tenant_id: str, data: dict) -> list[int]:
    partition = get_or_create_tenant_partition(tenant_id)
    result = partition.insert(data)
    return result.primary_keys

def search_for_tenant(tenant_id: str, query_embedding: list[float], top_k: int = 10):
    partition_name = f"tenant_{tenant_id}"
    # Load only this tenant's partition - other tenants' data stays unloaded
    collection.load(partition_names=[partition_name])

    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 64}},
        limit=top_k,
        partition_names=[partition_name]
    )
    return results[0]

```

Partitions scale to around 1,024 per collection in Milvus 2.4. This pattern is appropriate for SaaS applications with up to several hundred tenants that have predictable, moderate-sized datasets per tenant.

Pattern 3: Field-based Tenant Isolation with Filtering

All tenant data lives in a single collection with a `tenant_id` scalar field. Every query includes a filter expression scoping results to the requesting tenant. This is the simplest operationally but requires rigorous application-layer enforcement — there is no database-level guarantee that a query cannot accidentally omit the tenant filter.

```

# Python

```

```

from pymilvus import connections, Collection

connections.connect(alias="default", host="localhost", port="19530")
collection = Collection("shared_document_chunks")
collection.load()

def search_tenant_strict(
    tenant_id: str,
    query_embedding: list[float],
    top_k: int = 10
) -> list[dict]:
    """
    Always inject tenant_id filter - never allow unfiltered search.
    Enforce at this layer so application code cannot accidentally bypass tenant isolation.
    """
    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"ef": 64}},
        limit=top_k,
        expr=f'tenant_id == "{tenant_id}"',
        output_fields=["document_id", "chunk_index", "tenant_id"]
    )
    # Verify no cross-tenant leakage - defensive check in high-compliance contexts
    for hit in results[0]:
        assert hit.entity.get("tenant_id") == tenant_id, \
            f"Cross-tenant data leakage detected: got {hit.entity.get('tenant_id')}"
    return results[0]

```

This pattern scales to unlimited tenants with no collection or partition count limits, but recall degrades proportionally with the fraction of the collection that belongs to each tenant. A tenant with 0.01% of records in a 1B-vector collection will experience poor recall because the ANN graph was built over all tenants' data, and HNSW pruning may skip relevant neighbors that happen to be separated by other tenants' records in the graph.

19.7 Operational Concerns: Monitoring, Backup, and Upgrades

Running Milvus in production means owning the same operational surface area as any stateful distributed system. This section covers the monitoring signals, backup strategy, and upgrade path that keep a Milvus cluster healthy over time.

Prometheus and Grafana Integration

Milvus exposes a rich set of Prometheus metrics. The following are the highest-signal metrics for production dashboards:

```

# prometheus-scrape-config.yaml snippet
scrape_configs:
  - job_name: 'milvus'
    static_configs:
      - targets:
        - 'milvus-proxy:9091'
        - 'milvus-querynode-0:9091'
        - 'milvus-querynode-1:9091'
        - 'milvus-datanode-0:9091'
        - 'milvus-indexnode-0:9091'
    metrics_path: '/metrics'

```

Key metrics to alert on:

Metric	Alert Threshold	Meaning
<code>milvus_proxy_search_latency_p99</code>	500ms	Search latency SLA breach
<code>milvus_querynode_cache_hit_rate</code>	<85%	Insufficient Query Node memory
<code>milvus_querynode_sq_req_count</code>	Rate drop >20%	Query Node crash or overload
<code>milvus_datanode_consume_msg_count</code>	Count stall	Message bus backpressure
<code>milvus_indexnode_index_task_queue_count</code>	Queue depth >100	Index build falling behind ingestion
<code>etcd_server_has_leader</code>	0	etcd quorum lost — cluster is unavailable

Backup Strategy

Milvus does not have a built-in `pg_dump` equivalent. Backup is a combination of:

1. **Object storage versioning:** Enable S3 versioning on your Milvus bucket. Segment files are immutable once written; versioning gives you point-in-time recovery without a dedicated backup job.
2. **etcd snapshot:** etcd holds all collection metadata, index definitions, and segment assignments. Snapshot it on a schedule using `etcdctl snapshot save`.
3. **Milvus Backup tool:** The official `milvus-backup` CLI can take logical backups of specific collections, streaming segment files to a separate S3 location.

```
# shell
# Install milvus-backup CLI
wget
↪ https://github.com/zilliztech/milvus-backup/releases/latest/download/milvus-backup_linux_amd64.tar.gz
tar -xzf milvus-backup_linux_amd64.tar.gz

# Create a backup configuration
cat > backup-config.yaml << 'EOF'
milvus:
  address: localhost
  port: 19530

minio:
  address: localhost
  port: 9000
  bucketName: milvus-production
  backupBucketName: milvus-backups
  rootPath: files

backup:
  maxSegmentGroupSize: 2G
EOF

# Backup a specific collection
./milvus-backup create \
  --config backup-config.yaml \
  --name "document_chunks_backup_$(date +%Y%m%d)" \
  --colls document_chunks

# List available backups
./milvus-backup list --config backup-config.yaml

# Restore a collection from backup
./milvus-backup restore \
  --config backup-config.yaml \
```

```
--name document_chunks_backup_20240115 \  
--suffix "_restored" # Restores as document_chunks_restored to avoid name collision
```

Rolling Upgrades

Milvus uses a rolling upgrade process for Kubernetes deployments: update the Helm chart version, and Kubernetes will cycle Pods one at a time. The critical constraint is that coordinators (Root, Query, Data, Index) must be upgraded before worker nodes, because coordinators control the protocol version negotiation. The Milvus compatibility matrix guarantees that one minor version of difference between coordinators and workers is supported, enabling zero-downtime rolling upgrades within a minor version family.

```
# shell  
# Check current deployment version  
helm list --namespace milvus  
  
# Upgrade Milvus with rolling restart - coordinator pods upgrade first  
# because the Helm chart orders Pod updates by deployment dependency  
helm upgrade milvus milvus/milvus \  
  --namespace milvus \  
  --values milvus-values.yaml \  
  --version 4.2.0 \  
  --set coordinator.activeStandby.enabled=true # Enable HA mode for zero-downtime  
  
# Monitor rollout progress  
kubectl rollout status deployment/milvus-querynode --namespace milvus  
kubectl rollout status deployment/milvus-datanode --namespace milvus
```

19.8 Milvus vs. Alternatives: When to Choose What

With a full picture of Milvus’s architecture and operational model, we can revisit the positioning question with engineering precision rather than marketing language.

Milvus vs. pgvector

Choose pgvector when: your vector dataset fits comfortably in PostgreSQL’s working memory (under ~10M vectors for interactive latency), you need transactional consistency between vector and relational data, or your team’s operational experience is entirely centered on PostgreSQL. pgvector’s `hnsw` index is competitive with Milvus HNSW up to this scale, and the operational simplicity of a single database cannot be overstated.

Choose Milvus when: you have more than ~50M vectors, you need multi-modal search (dense + sparse hybrid) as a first-class feature, or you need independent scaling of ingestion and query compute. Milvus also supports GPU-accelerated index building natively, which pgvector does not.

Milvus vs. Pinecone

Pinecone is a managed service with a deliberately simple operational surface. If your organization lacks Kubernetes expertise or cannot dedicate engineering time to operating a distributed system, Pinecone’s operational simplicity has real value. Milvus’s advantages over Pinecone are: no data leaves your infrastructure (compliance), no per-vector storage costs (cost at scale), and full control over index parameters and hardware.

Milvus vs. Weaviate

Weaviate bundles vectorization, schema management, and a GraphQL query interface alongside vector search. It is a better fit for applications that want a single system to own the object store, the embedding pipeline, and the search layer. Milvus is a pure vector search engine — it expects you to bring your own embedding pipeline and metadata store. Teams that have already built embedding pipelines and want maximum search performance and scalability without a bundled object store tend to prefer Milvus.

Decision Matrix

Criterion	pgvector	Milvus	Pinecone	Weaviate
Operational complexity	Low	High	Very Low	Medium
Max practical scale	~50M vectors	Billions	Billions (managed)	Hundreds of millions
Hybrid dense+sparse	Limited	Native	Available	Available
Transactional consistency	Full ACID	Eventual	Eventual	Eventual
GPU acceleration	No	Yes	Managed	No
Self-hosted	Yes	Yes	No	Yes
Multi-tenancy support	Schema/RLS	Partition/CollectionNamespace		Tenant field

Key Takeaways

- **Disaggregated architecture enables independent scaling:** Milvus separates proxy, coordinator, query, data, and index roles into independently scalable components. This means you can add Query Nodes to handle more concurrent searches without touching ingestion capacity, or add Index Nodes to accelerate index builds during a bulk load without competing with live query serving — a design that monolithic vector stores cannot match at the same scale.
- **Index selection is a first-class engineering decision:** HNSW is the right default for most workloads under 500M vectors where low latency is the priority, IVF variants suit memory-constrained environments, and DISKANN enables billion-scale deployments that cannot fit in RAM. Treat the `M` and `ef` parameters as tunable knobs analogous to PostgreSQL's `work_mem` and `effective_cache_size` — start with documented defaults, measure recall and latency, then adjust.
- **Hybrid dense+sparse search consistently outperforms either alone:** For knowledge retrieval over technical content, combining HNSW-indexed dense embeddings with sparse inverted indexes and fusing results via RRF produces materially better recall than dense search alone. Milvus's native support for `SPARSE_FLOAT_VECTOR` fields and `AnnSearchRequest` fusion makes this pattern straightforward to implement without external reranking infrastructure.

- **Multi-tenancy pattern selection has lasting consequences:** Collection-per-tenant gives the strongest isolation but limits active tenant count; partition-per-tenant scales to hundreds of tenants with selective loading; filter-based isolation scales to any tenant count but relies entirely on application-layer enforcement and degrades recall for small tenants in large shared collections. Choose based on your tenant count, data size per tenant, and compliance requirements — and choose before your first bulk load.
 - **Operational maturity requires owning the full stack:** Milvus’s power comes with the full operational responsibility of a distributed system: Prometheus monitoring with latency and cache-hit-rate alerts, etcd snapshot backups alongside object storage versioning, and a disciplined Helm-based upgrade path that respects coordinator-before-worker ordering. Teams that underestimate this surface area typically encounter their first production incident before they have instrumentation in place to diagnose it.
-

Chapter 20: Benchmarking Vector Databases: Throughput, Latency, Recall

Choosing a vector database without benchmarking it is the equivalent of selecting a relational engine based on marketing collateral alone — you will eventually regret it. The vector database ecosystem is young, vendor claims are aggressive, and the gap between synthetic benchmark numbers and production behavior is wide enough to cause real incidents at scale. This chapter builds a rigorous framework for measuring what actually matters in vector search systems: query throughput, tail latency, and recall accuracy. You will learn how these three metrics interact (improving one often degrades another), how to construct benchmarks that reflect your actual workload rather than a vendor’s best-case scenario, and how to interpret results from the most common tools in the ecosystem — pgvector, Pinecone, Weaviate, Qdrant, and Milvus. By the end, you will have runnable harnesses, concrete thresholds to target, and a vocabulary for communicating vector database performance the same way you already communicate IOPS, latency percentiles, and cache hit ratios.

The Three Metrics That Define Vector Search Performance

Every vector database benchmark ultimately reduces to three numbers: **throughput**, **latency**, and **recall**. Understanding what each measures — and where the bodies are buried — is the prerequisite to designing any useful test.

Throughput is queries per second (QPS). It measures how many similarity searches the system can complete in a unit of time under sustained load. Like any throughput metric, it is meaningless without specifying concurrency. A system that delivers 500 QPS at 1 concurrent client and 480 QPS at 50 concurrent clients is a fundamentally different beast from one that delivers 500 QPS at 1 client and 12,000 QPS at 50 clients. Always measure throughput under the concurrency your application actually generates.

Latency is the time to complete a single query. In production systems, what matters is not mean latency but tail latency — specifically p95 and p99. A vector database with a 5ms mean and a 2,000ms p99 will make your application feel broken during load spikes. Most vector workloads are in the critical path of user-facing requests, which makes p99 the governing metric.

Recall is the fraction of true nearest neighbors that a query actually returns. It is the metric

unique to approximate nearest neighbor (ANN) systems, and it has no direct analogue in traditional databases. If you ask for the 10 most similar vectors to a query, and the system returns 10 results but only 7 of them are genuinely in the true top-10, your `recall@10` is 0.7. Recall is what separates ANN search from exact k-NN search — and it is what makes benchmarking vector databases harder than benchmarking relational systems.

The fundamental tension is this: **recall, throughput, and latency form a triangle where you can optimize any two by sacrificing the third.** Tightening recall requires searching more of the index graph, which increases latency and reduces throughput. Reducing latency requires searching less, which damages recall. This is not an implementation deficiency — it is the mathematical reality of ANN algorithms, and any honest benchmark must measure all three simultaneously.

The recall-latency curve is the most important diagnostic tool in vector database performance analysis. It plots recall (x-axis) against query latency (y-axis) at fixed concurrency, with each point representing a different index parameter setting. The database with the curve closest to the lower-right corner of the plot — high recall at low latency — wins that workload.

Practical thresholds for production systems:

Metric	Acceptable	Good	Excellent
Recall@10	> 0.90	> 0.95	> 0.99
p99 latency (single node)	< 100ms	< 20ms	< 5ms
QPS (per node, 768-dim)	> 500	> 2,000	> 10,000

These numbers are for 768-dimensional embeddings (a common size from models like `text-embedding-3-small` and `all-MiniLM-L6-v2`) on a dataset of 1–10 million vectors. Dimensionality, dataset size, and filter complexity all shift these thresholds dramatically.

Constructing a Realistic Benchmark Dataset

Vendor benchmarks almost always use ANN-benchmarks datasets — SIFT1M, GIST1M, or GloVe — because they are public, reproducible, and make their systems look good. Your workload is probably none of those. Building a benchmark that predicts production behavior requires matching four properties of your real dataset: dimensionality, distribution, dataset size, and filter selectivity.

Dimensionality directly determines index build time, memory consumption, and query latency. OpenAI’s `text-embedding-3-large` produces 3072-dimensional vectors. Cohere’s `embed-english-v3.0` produces 1024-dimensional vectors. Most open-source models produce 384–768 dimensions. Do not benchmark at 128 dimensions if you are going to deploy at 1536.

Distribution matters more than most engineers expect. ANN index structures (HNSW, IVF, ScaNN) exploit clustering in the vector space to skip regions during search. Randomly generated vectors are uniformly distributed in high-dimensional space and do not cluster — they are the worst case for ANN indices and the best case for demonstrating that an index provides no benefit. Real embedding vectors cluster by semantic topic. If your documents are all from a single domain (medical literature, legal contracts, software documentation), your vectors will cluster differently than a general-purpose corpus, and your recall numbers will differ from published benchmarks.

Filtered search is the single biggest gap between published benchmarks and production behavior. Most real applications combine vector similarity with metadata filters: “find the 10 most similar documents to this query, but only from this tenant’s data, written after 2023-01-01, tagged as ‘approved.’” Pre-filtering (apply metadata filter first, then search remaining vectors) destroys recall when the filter is selective. Post-filtering (search vectors first, then apply filter) destroys recall when the filter eliminates most results. How a database handles this tradeoff — Qdrant’s filterable HNSW, Weaviate’s inverted-index integration, Pinecone’s metadata filtering — is one of its most important production characteristics.

The following script generates a synthetic benchmark dataset that mirrors real embedding distributions by using actual sentence-transformers embeddings on sampled text:

```
# Python
# benchmark_dataset_generator.py
# Generates a realistic benchmark dataset using real embedding models
# Requires: sentence-transformers, numpy, h5py, datasets

import numpy as np
import h5py
import time
from sentence_transformers import SentenceTransformer
from datasets import load_dataset

def generate_benchmark_dataset(
    n_corpus: int = 100_000,
    n_queries: int = 1_000,
    model_name: str = "sentence-transformers/all-MiniLM-L6-v2",
    output_path: str = "benchmark_dataset.h5",
    batch_size: int = 512
):
    """
    Generates an HDF5 benchmark dataset in the ANN-benchmarks format.
    Uses real text embeddings for realistic vector distributions.

    HDF5 schema:
    /train (n_corpus, dim) float32 - corpus vectors
    /test (n_queries, dim) float32 - query vectors
    /neighbors (n_queries, 100) int32 - ground truth (computed exactly)
    /distances (n_queries, 100) float32 - ground truth distances
    """
    print(f"Loading model: {model_name}")
    model = SentenceTransformer(model_name)
    dim = model.get_sentence_embedding_dimension()
    print(f"Embedding dimension: {dim}")

    # Load a real text corpus for realistic distributions
    print("Loading corpus text from Wikipedia sample...")
    dataset = load_dataset(
        "wikipedia", "20220301.en",
        split="train",
        streaming=True,
        trust_remote_code=True
    )

    texts = []
    for item in dataset:
        # Extract first 256 characters of each article as a passage
        texts.append(item["text"][:256])
        if len(texts) >= n_corpus + n_queries:
            break

    corpus_texts = texts[:n_corpus]
    query_texts = texts[n_corpus:n_corpus + n_queries]

    print(f"Encoding {n_corpus} corpus documents...")
    t0 = time.time()
```

```

corpus_vectors = model.encode(
    corpus_texts,
    batch_size=batch_size,
    show_progress_bar=True,
    normalize_embeddings=True # Cosine similarity -> dot product
).astype(np.float32)
encode_time = time.time() - t0
print(f"Corpus encoding: {encode_time:.1f}s ({n_corpus/encode_time:.0f} docs/sec)")

print(f"Encoding {n_queries} query vectors...")
query_vectors = model.encode(
    query_texts,
    batch_size=batch_size,
    show_progress_bar=True,
    normalize_embeddings=True
).astype(np.float32)

# Compute exact ground truth using brute-force cosine similarity
# For normalized vectors, cosine similarity = dot product
print("Computing exact ground truth (brute-force)...")
k_gt = 100 # Store top-100 ground truth for evaluating recall@1 through @100

neighbors = np.zeros((n_queries, k_gt), dtype=np.int32)
distances = np.zeros((n_queries, k_gt), dtype=np.float32)

# Process in batches to avoid OOM on large datasets
query_batch_size = 100
for i in range(0, n_queries, query_batch_size):
    batch = query_vectors[i:i + query_batch_size]
    # Matrix multiply: (batch_size, dim) x (dim, n_corpus) -> (batch_size, n_corpus)
    sims = np.dot(batch, corpus_vectors.T)
    # Get top-k indices (descending similarity)
    top_k_idx = np.argpartition(sims, -k_gt, axis=1)[: , -k_gt:]
    for j, row_idx in enumerate(range(i, min(i + query_batch_size, n_queries))):
        top_k_sorted = top_k_idx[j][np.argsort(sims[j, top_k_idx[j]])[::-1]]
        neighbors[row_idx] = top_k_sorted
        distances[row_idx] = sims[j, top_k_sorted]

    if i % 1000 == 0:
        print(f"Ground truth: {i}/{n_queries} queries")

# Write HDF5 compatible with ann-benchmarks tooling
print(f"Writing dataset to {output_path}...")
with h5py.File(output_path, "w") as f:
    f.create_dataset("train", data=corpus_vectors, compression="gzip")
    f.create_dataset("test", data=query_vectors, compression="gzip")
    f.create_dataset("neighbors", data=neighbors, compression="gzip")
    f.create_dataset("distances", data=distances, compression="gzip")
    f.attrs["dimension"] = dim
    f.attrs["n_corpus"] = n_corpus
    f.attrs["n_queries"] = n_queries
    f.attrs["model"] = model_name
    f.attrs["metric"] = "cosine"

print(f"Dataset written: {corpus_vectors.nbytes / 1e6:.1f} MB corpus, "
      f"{query_vectors.nbytes / 1e6:.1f} MB queries")
return output_path

if __name__ == "__main__":
    generate_benchmark_dataset(
        n_corpus=100_000,
        n_queries=1_000,
        model_name="sentence-transformers/all-MiniLM-L6-v2",
        output_path="minilm_100k_benchmark.h5"
    )

```

This produces an HDF5 file in the same format as ANN-benchmarks, so you can drop it into standard tooling or write your own evaluation loop. The critical design choice here is using real

embeddings on real text: your benchmark dataset should come from the same distribution as your production data, or your recall numbers will be wrong by 5–15 percentage points.

Building a Multi-Database Benchmark Harness

With a dataset in hand, the next step is a consistent measurement harness that runs the same queries against multiple databases and captures the metrics that matter. The harness must be fair: each database should be given its best-case index configuration, not a default one. Default configurations almost always favor index build time over query performance, which is the opposite of production priorities.

```
# Python
# vector_benchmark_harness.py
# Multi-database benchmark harness for pgvector, Qdrant, and Pinecone
# Requires: psycopg2, qdrant-client, pinecone-client, numpy, h5py, scipy

import time
import numpy as np
import h5py
import statistics
import concurrent.futures
from dataclasses import dataclass, field
from typing import Callable, List, Optional
from contextlib import contextmanager

import psycopg2
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    HnswConfigDiff, OptimizersConfigDiff, SearchRequest
)
from pinecone import Pinecone, ServerlessSpec

#
# Data structures
#

@dataclass
class BenchmarkResult:
    database: str
    index_config: dict
    n_corpus: int
    n_queries: int
    concurrency: int
    k: int  # Number of results requested
    # Throughput
    total_queries: int = 0
    total_time_sec: float = 0.0
    qps: float = 0.0
    # Latency (milliseconds)
    latencies_ms: List[float] = field(default_factory=list)
    p50_ms: float = 0.0
    p95_ms: float = 0.0
    p99_ms: float = 0.0
    # Recall
    recall_at_k: float = 0.0
    # Index build
    index_build_sec: float = 0.0
    index_size_mb: float = 0.0

def compute_recall(
```

```

    returned_ids: List[List[int]],
    ground_truth: np.ndarray,
    k: int
) -> float:
    """
    Computes recall@k: fraction of true top-k neighbors found in returned results.
    ground_truth shape: (n_queries, k_gt) - rows are sorted by similarity descending.
    """
    n_queries = len(returned_ids)
    recall_sum = 0.0
    for i in range(n_queries):
        true_top_k = set(ground_truth[i, :k].tolist())
        returned = set(returned_ids[i][:k])
        if len(true_top_k) > 0:
            recall_sum += len(true_top_k & returned) / len(true_top_k)
    return recall_sum / n_queries

#
# Database adapters
#

class PgvectorBenchmark:
    """
    pgvector benchmark adapter.
    Tests both ivfflat and hnsw index types.
    """

    def __init__(self, dsn: str, table_name: str = "benchmark_vectors"):
        self.dsn = dsn
        self.table = table_name

    @contextmanager
    def _conn(self):
        conn = psycopg2.connect(self.dsn)
        conn.autocommit = True
        try:
            yield conn
        finally:
            conn.close()

    def load_data(self, vectors: np.ndarray) -> float:
        """Loads corpus vectors and returns seconds elapsed."""
        n, dim = vectors.shape
        t0 = time.time()
        with self._conn() as conn:
            with conn.cursor() as cur:
                cur.execute("CREATE EXTENSION IF NOT EXISTS vector")
                cur.execute(f"DROP TABLE IF EXISTS {self.table}")
                cur.execute(f"""
                    CREATE TABLE {self.table} (
                        id BIGINT PRIMARY KEY,
                        embedding vector({dim})
                    )
                """)
                # Bulk insert using COPY for performance
                from io import StringIO
                buf = StringIO()
                for i, vec in enumerate(vectors):
                    vec_str = "[" + ",".join(f"{v:.6f}" for v in vec) + "]"
                    buf.write(f"{i}\t{vec_str}\n")
                buf.seek(0)
                cur.copy_from(buf, self.table, columns=("id", "embedding"))
        return time.time() - t0

    def build_hnsw_index(self, m: int = 16, ef_construction: int = 64) -> float:
        """Builds HNSW index, returns build time in seconds."""
        t0 = time.time()
        with self._conn() as conn:
            with conn.cursor() as cur:

```

```

        # Drop existing index if present
        cur.execute(f"DROP INDEX IF EXISTS {self.table}_hnswn_idx")
        cur.execute(f"""
            CREATE INDEX {self.table}_hnswn_idx
            ON {self.table}
            USING hnswn (embedding vector_cosine_ops)
            WITH (m = {m}, ef_construction = {ef_construction})
        """)
        # Ensure index is fully built before timing stops
        cur.execute(f"SELECT COUNT(*) FROM {self.table}")
    return time.time() - t0

def set_ef_search(self, conn, ef_search: int):
    with conn.cursor() as cur:
        cur.execute(f"SET hnswn.ef_search = {ef_search}")

def run_queries(
    self,
    query_vectors: np.ndarray,
    k: int = 10,
    concurrency: int = 1,
    ef_search: int = 100
) -> tuple:
    """
    Returns (latencies_ms, all_returned_ids).
    """
    n_queries = len(query_vectors)
    latencies = []
    all_ids = []

    def single_query(i: int):
        vec = query_vectors[i]
        vec_str = "[" + ",".join(f"{v:.6f}" for v in vec) + "]"
        conn = psycopg2.connect(self.dsn)
        try:
            with conn.cursor() as cur:
                cur.execute(f"SET hnswn.ef_search = {ef_search}")
                t0 = time.perf_counter()
                cur.execute(f"""
                    SELECT id
                    FROM {self.table}
                    ORDER BY embedding <=> %s::vector
                    LIMIT {k}
                """, (vec_str,))
                ids = [row[0] for row in cur.fetchall()]
                elapsed = (time.perf_counter() - t0) * 1000
            return elapsed, ids
        finally:
            conn.close()

    with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
        futures = [executor.submit(single_query, i) for i in range(n_queries)]
        for future in concurrent.futures.as_completed(futures):
            lat, ids = future.result()
            latencies.append(lat)
            all_ids.append(ids)

    return latencies, all_ids

class QdrantBenchmark:
    """
    Qdrant benchmark adapter.
    Qdrant is particularly strong at filtered ANN - test with and without filters.
    """

    def __init__(
        self,
        url: str = "http://localhost:6333",
        collection_name: str = "benchmark"
    ):
```

```

):
    self.client = QdrantClient(url=url, timeout=60)
    self.collection = collection_name

def load_data(self, vectors: np.ndarray, m: int = 16, ef_construction: int = 100) -> float:
    """Loads corpus and builds HNSW index simultaneously."""
    n, dim = vectors.shape

    # Delete existing collection
    try:
        self.client.delete_collection(self.collection)
    except Exception:
        pass

    t0 = time.time()
    self.client.create_collection(
        collection_name=self.collection,
        vectors_config=VectorParams(
            size=dim,
            distance=Distance.COSINE
        ),
        hnsw_config=HnswConfigDiff(
            m=m,
            ef_construct=ef_construction,
            full_scan_threshold=10_000 # Use HNSW for collections > 10k vectors
        ),
        optimizers_config=OptimizersConfigDiff(
            indexing_threshold=0 # Index immediately, don't batch
        )
    )

    # Batch upload
    batch_size = 1000
    for start in range(0, n, batch_size):
        end = min(start + batch_size, n)
        batch = vectors[start:end]
        points = [
            PointStruct(
                id=start + i,
                vector=batch[i].tolist()
            )
            for i in range(len(batch))
        ]
        self.client.upsert(collection_name=self.collection, points=points)

    # Wait for indexing to complete
    while True:
        info = self.client.get_collection(self.collection)
        if info.status.value == "green":
            break
        time.sleep(1.0)

    return time.time() - t0

def run_queries(
    self,
    query_vectors: np.ndarray,
    k: int = 10,
    concurrency: int = 1,
    ef_search: int = 128
) -> tuple:
    latencies = []
    all_ids = []

    def single_query(i: int):
        vec = query_vectors[i].tolist()
        t0 = time.perf_counter()
        results = self.client.search(
            collection_name=self.collection,
            query_vector=vec,

```

```

        limit=k,
        search_params={"hnsf_ef": ef_search}
    )
    elapsed = (time.perf_counter() - t0) * 1000
    ids = [int(r.id) for r in results]
    return elapsed, ids

with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
    futures = [executor.submit(single_query, i) for i in range(len(query_vectors))]
    for future in concurrent.futures.as_completed(futures):
        lat, ids = future.result()
        latencies.append(lat)
        all_ids.append(ids)

return latencies, all_ids

#
# Benchmark runner
#

def run_benchmark(
    dataset_path: str,
    pgvector_dsn: Optional[str] = None,
    qdrant_url: Optional[str] = None,
    concurrencies: List[int] = [1, 4, 16, 32],
    k: int = 10,
    ef_search_values: List[int] = [64, 128, 256]
) -> List[BenchmarkResult]:
    """
    Runs the full benchmark suite against all configured databases.
    Returns a list of BenchmarkResult objects, one per (database, concurrency, ef_search) combination.
    """
    # Load dataset
    print(f"Loading dataset from {dataset_path}...")
    with h5py.File(dataset_path, "r") as f:
        corpus = f["train"][:]
        queries = f["test"][:]
        ground_truth = f["neighbors"][:]

    n_corpus, dim = corpus.shape
    n_queries = len(queries)
    print(f"Dataset: {n_corpus:,} corpus vectors, {n_queries:,} queries, dim={dim}")

    results = []

    # pgvector
    if pgvector_dsn:
        print("\n=== pgvector benchmark ===")
        pg = PgvectorBenchmark(pgvector_dsn)

        print("Loading data...")
        load_time = pg.load_data(corpus)
        print(f"Data load: {load_time:.1f}s")

        for m, ef_c in [(16, 64), (16, 128), (32, 128)]:
            print(f"Building HNSW index (m={m}, ef_construction={ef_c})...")
            build_time = pg.build_hnsw_index(m=m, ef_construction=ef_c)
            print(f"Index build: {build_time:.1f}s")

            for ef_s in ef_search_values:
                for concurrency in concurrencies:
                    print(f"Querying: ef_search={ef_s}, concurrency={concurrency}")
                    latencies, returned_ids = pg.run_queries(
                        queries, k=k,
                        concurrency=concurrency,
                        ef_search=ef_s
                    )
                    recall = compute_recall(returned_ids, ground_truth, k)
                    latencies_sorted = sorted(latencies)

```

```

        result = BenchmarkResult(
            database="pgvector",
            index_config={"m": m, "ef_construction": ef_c, "ef_search": ef_s},
            n_corpus=n_corpus,
            n_queries=n_queries,
            concurrency=concurrency,
            k=k,
            total_queries=n_queries,
            total_time_sec=sum(latencies) / 1000 / concurrency,
            qps=n_queries / (sum(latencies) / 1000 / concurrency),
            latencies_ms=latencies,
            p50_ms=latencies_sorted[int(0.50 * len(latencies_sorted))],
            p95_ms=latencies_sorted[int(0.95 * len(latencies_sorted))],
            p99_ms=latencies_sorted[int(0.99 * len(latencies_sorted))],
            recall_at_k=recall,
            index_build_sec=build_time
        )
        results.append(result)
        print(f"    QPS={result.qps:.0f}    p99={result.p99_ms:.1f}ms    recall={recall:.4f}")

# Qdrant
if qdrant_url:
    print("\n=== Qdrant benchmark ===")
    qd = QdrantBenchmark(qdrant_url)

    for m, ef_c in [(16, 100), (32, 128)]:
        print(f"    Loading data and building HNSW (m={m}, ef_construction={ef_c})...")
        build_time = qd.load_data(corpus, m=m, ef_construction=ef_c)
        print(f"    Index build+load: {build_time:.1f}s")

        for ef_s in ef_search_values:
            for concurrency in concurrencies:
                print(f"    Querying: ef_search={ef_s}, concurrency={concurrency}")
                latencies, returned_ids = qd.run_queries(
                    queries, k=k,
                    concurrency=concurrency,
                    ef_search=ef_s
                )
            recall = compute_recall(returned_ids, ground_truth, k)
            latencies_sorted = sorted(latencies)

            result = BenchmarkResult(
                database="qdrant",
                index_config={"m": m, "ef_construction": ef_c, "ef_search": ef_s},
                n_corpus=n_corpus,
                n_queries=n_queries,
                concurrency=concurrency,
                k=k,
                total_queries=n_queries,
                total_time_sec=sum(latencies) / 1000 / concurrency,
                qps=n_queries / (sum(latencies) / 1000 / concurrency),
                latencies_ms=latencies,
                p50_ms=latencies_sorted[int(0.50 * len(latencies_sorted))],
                p95_ms=latencies_sorted[int(0.95 * len(latencies_sorted))],
                p99_ms=latencies_sorted[int(0.99 * len(latencies_sorted))],
                recall_at_k=recall,
                index_build_sec=build_time
            )
            results.append(result)
            print(f"    QPS={result.qps:.0f}    p99={result.p99_ms:.1f}ms    recall={recall:.4f}")

    return results

def print_summary_table(results: List[BenchmarkResult]):
    """Prints a formatted comparison table sorted by recall descending."""
    print("\n" + "=" * 90)
    print(f"{'Database':<12} {'Config':<30} {'Conc':>5} {'QPS':>8} {'p50ms':>7} "
          f"{'p99ms':>7} {'Recall':>8}")

```

```

print("-" * 90)
for r in sorted(results, key=lambda x: (-x.recall_at_k, x.p99_ms)):
    cfg = f"m={r.index_config.get('m', '-')}\ ef_s={r.index_config.get('ef_search', '-')}"
    print(f"{r.database:<12} {cfg:<30} {r.concurrency:>5} {r.qps:>8.0f} "
          f"{r.p50_ms:>7.1f} {r.p99_ms:>7.1f} {r.recall_at_k:>8.4f}")
print("-" * 90)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--dataset", default="minilm_100k_benchmark.h5")
    parser.add_argument("--pgvector-dsn", default=None)
    parser.add_argument("--qdrant-url", default=None)
    parser.add_argument("--concurrentcies", nargs="+", type=int, default=[1, 4, 16, 32])
    parser.add_argument("--k", type=int, default=10)
    args = parser.parse_args()

    results = run_benchmark(
        dataset_path=args.dataset,
        pgvector_dsn=args.pgvector_dsn,
        qdrant_url=args.qdrant_url,
        concurrentcies=args.concurrentcies,
        k=args.k
    )
    print_summary_table(results)

```

This harness is intentionally modular. Adding a Weaviate or Milvus adapter means implementing `load_data()` and `run_queries()` following the same contract, then registering the adapter in `run_benchmark()`. The key design choice is measuring latency from the client's perspective — including network round-trip and serialization — not from inside the database engine. That is the number your users experience.

pgvector: Index Tuning for Production Throughput

pgvector is the most common starting point because it lives inside PostgreSQL, which means you inherit connection pooling via PgBouncer, replication, ACID transactions, and decades of operational tooling. It is not the fastest vector database in pure QPS terms, but it is frequently the right answer when your dataset fits in a single node and your team already operates PostgreSQL.

pgvector supports two index types as of version 0.6.0:

- **HNSW** (`hnsw`): Hierarchical Navigable Small World. Better recall-latency tradeoff for query-heavy workloads. Higher memory consumption during build. Generally preferred for production read workloads.
- **IVFFlat** (`ivfflat`): Inverted file with flat quantization. Faster index builds, lower memory footprint, but inferior recall at equivalent latency. Still useful when you are rebuilding indices frequently (e.g., nightly batch workflows).

The five parameters that govern pgvector performance are:

Parameter	Index Type	Effect
<code>m</code>	HNSW	Graph connectivity. Higher → better recall, larger index, slower build.

Parameter	Index Type	Effect
<code>ef_construction</code>	HNSW	Search depth during build. Higher → better recall, slower build.
<code>hnsw.ef_search</code>	HNSW	Search depth at query time. Higher → better recall, higher latency. Set per-session.
<code>lists</code>	IVFFlat	Number of inverted lists (centroids). Target <code>sqrt(n_corpus)</code> .
<code>ivfflat.probes</code>	IVFFlat	Number of lists searched per query. Higher → better recall, higher latency.

The relationship between `hnsw.ef_search` and the recall-latency tradeoff is the most important control knob during production tuning. The following SQL illustrates how to sweep it:

```
-- PostgreSQL: sweep ef_search to find the recall-latency knee point
-- Run this in psql with \timing enabled, then compare results against ground truth

-- Set up timing
\timing on

-- ef_search = 40: fast but lower recall
SET hnsw.ef_search = 40;
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM benchmark_vectors
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector
LIMIT 10;

-- ef_search = 100: balanced
SET hnsw.ef_search = 100;
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM benchmark_vectors
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector
LIMIT 10;

-- ef_search = 400: high recall, slower
SET hnsw.ef_search = 400;
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, embedding <=> '[0.1, 0.2, ...]':vector AS distance
FROM benchmark_vectors
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector
LIMIT 10;
```

One pgvector behavior that surprises engineers coming from pure vector databases: **the query planner can choose not to use the vector index**. If your query includes a highly selective WHERE clause, PostgreSQL may decide a sequential scan is cheaper than an ANN index scan followed by a filter. This is generally correct behavior for small result sets, but it means your p99 latency can spike when a WHERE clause causes plan flips. The remedy is to examine EXPLAIN output for queries with both vector distance ordering and metadata filters, and to use `SET enable_seqscan = off` in a controlled test to understand the index-vs-seqscan crossover point.

Parallel workers are another pgvector lever that most engineers do not configure. HNSW index scans in pgvector 0.7+ support parallel query execution. Setting

`max_parallel_workers_per_gather` correctly means a single query can use multiple CPU cores to search the graph simultaneously:

```
-- PostgreSQL: enable parallel HNSW scanning
ALTER TABLE benchmark_vectors SET (parallel_workers = 4);
SET max_parallel_workers_per_gather = 4;
SET min_parallel_table_scan_size = '1MB';

-- Verify the plan uses parallel workers
EXPLAIN SELECT id
FROM benchmark_vectors
ORDER BY embedding <=> '[...]'::vector
LIMIT 10;
-- Look for "Gather" or "Gather Merge" nodes in the plan
```

The tradeoff with parallel workers is predictable: they increase single-query throughput at the cost of total system throughput when many queries are running concurrently. At low concurrency (< 4 clients), parallel workers help. At high concurrency (> 16 clients), you are typically better served by allowing each query to use a single worker and letting concurrency itself saturate your CPU cores.

Weaviate and Milvus: Distributed Vector Search Benchmarking

When your dataset exceeds single-node capacity — typically above 50–100 million vectors at 768 dimensions on a well-provisioned machine — you need a distributed vector database. Weaviate and Milvus both offer horizontal scaling, but with different architectural philosophies that produce different benchmark characteristics.

Weaviate stores vectors alongside a rich object store and a built-in GraphQL/REST API. Its HNSW index is built per-shard, with shards distributed across nodes. The benchmarking implication is that inter-shard query fan-out adds latency that does not appear in single-node tests. When benchmarking Weaviate in a multi-node configuration, always include the scatter-gather latency by targeting the coordinator endpoint, not an individual shard node directly.

Milvus separates compute and storage more aggressively. Its architecture includes query nodes (which hold loaded segments in memory), data nodes (which manage write-ahead logs and compaction), and an S3-compatible object store for persistence. The benchmarking implication is that Milvus latency is bimodal: queries against loaded segments are fast (all data in DRAM), but queries that trigger segment loading from object storage are dramatically slower. Always warm the query nodes before recording benchmark numbers.

A minimal Weaviate benchmark query using the Python client:

```
# Python - Weaviate v4 client benchmark query
import weaviate
import time
import numpy as np

def benchmark_weaviate(
    host: str,
    collection_name: str,
    query_vectors: np.ndarray,
    k: int = 10,
    ef_search: int = 128
) -> tuple:
    """
    Returns (latencies_ms, all_returned_ids) for a Weaviate collection.
```

```
ef parameter controls HNSW search depth at query time.
"""

client = weaviate.connect_to_local(host=host)
collection = client.collections.get(collection_name)

latencies = []
all_ids = []

try:
    for vec in query_vectors:
        t0 = time.perf_counter()
        results = collection.query.near_vector(
            near_vector=vec.tolist(),
            limit=k,
            return_properties=[],           # Only need the ID for recall computation
            additional_properties=["id"],
            query_properties=None,
            # ef is set at collection level via schema, not per-query in v4
        )
        elapsed_ms = (time.perf_counter() - t0) * 1000
        latencies.append(elapsed_ms)
        all_ids.append([obj.uuid for obj in results.objects])
finally:
    client.close()

return latencies, all_ids
```

One Weaviate-specific tuning parameter deserves explicit attention: the `ef` parameter in the collection’s HNSW configuration controls search depth at query time, similar to `hnsw.ef_search` in pgvector. Unlike pgvector, where you set it per-session with `SET`, Weaviate requires updating the collection’s `efConfig` via the schema API to change it without restarting. This makes dynamic `ef` sweeping more cumbersome in Weaviate — plan your benchmark parameter grid before starting rather than adjusting on the fly.

Filtered Search Benchmarking: The Production Reality Check

The scenario that breaks most vendor benchmark claims is filtered search. A realistic e-commerce or SaaS application rarely searches all vectors — it searches vectors belonging to a specific user, product category, date range, or tenant. The interaction between the vector index and the metadata filter is where architectural differences between databases become most visible.

To measure filtered search performance, extend the benchmark dataset with synthetic metadata and test at several selectivity levels:

```
# Python
# filtered_benchmark.py
# Tests recall and latency under metadata filtering at multiple selectivity levels

import numpy as np
import time
from qdrant_client import QdrantClient
from qdrant_client.models import (
    FieldCondition, Filter, MatchValue,
    Range, PointStruct, PayloadSchemaType
)

def load_qdrant_with_metadata(
    client: QdrantClient,
    collection_name: str,
    vectors: np.ndarray,
```

```

n_categories: int = 100
):
    """
    Assigns each vector a random category_id (0..n_categories-1).
    Creates a payload index for fast filtered search.
    """
    n = len(vectors)
    categories = np.random.randint(0, n_categories, size=n)

    batch_size = 1000
    for start in range(0, n, batch_size):
        end = min(start + batch_size, n)
        points = [
            PointStruct(
                id=start + i,
                vector=vectors[start + i].tolist(),
                payload={"category_id": int(categories[start + i])}
            )
            for i in range(end - start)
        ]
        client.upsert(collection_name=collection_name, points=points)

    # Create payload index so Qdrant can use filterable HNSW
    client.create_payload_index(
        collection_name=collection_name,
        field_name="category_id",
        field_schema=PayloadSchemaType.INTEGER
    )

    return categories

def benchmark_filtered_query(
    client: QdrantClient,
    collection_name: str,
    query_vector: np.ndarray,
    category_id: int,
    k: int = 10,
    ef_search: int = 128
) -> tuple:
    """
    Returns (latency_ms, result_ids) for a filtered query.
    """
    query_filter = Filter(
        must=[
            FieldCondition(
                key="category_id",
                match=MatchValue(value=category_id)
            )
        ]
    )
    t0 = time.perf_counter()
    results = client.search(
        collection_name=collection_name,
        query_vector=query_vector.tolist(),
        query_filter=query_filter,
        limit=k,
        search_params={"hnsw_ef": ef_search}
    )
    elapsed_ms = (time.perf_counter() - t0) * 1000
    ids = [int(r.id) for r in results]
    return elapsed_ms, ids

def run_filtered_benchmark(
    client: QdrantClient,
    collection_name: str,
    query_vectors: np.ndarray,
    categories: np.ndarray,
    n_categories_list: list = [10, 100, 1000],

```

```

k: int = 10
):
    """
    Tests how latency and result count vary with filter selectivity.
    With 100k vectors and n_categories=10, each bucket has ~10k vectors (low selectivity).
    With n_categories=1000, each bucket has ~100 vectors (high selectivity - hard case).
    """
    print(f"\n{'Selectivity':<20} {'n_categories':<15} {'Avg Latency':>12} "
          f"{'p99 Latency':>12} {'Avg Results':>12}")
    print("-" * 75)

    for n_cat in n_categories_list:
        # Re-assign categories at this granularity
        n_vectors = len(categories)
        cat_ids = categories % n_cat # Map existing categories to smaller range

        latencies = []
        result_counts = []

        for i, qvec in enumerate(query_vectors):
            target_cat = int(cat_ids[i % n_vectors])
            lat, ids = benchmark_filtered_query(
                client, collection_name, qvec, target_cat, k=k
            )
            latencies.append(lat)
            result_counts.append(len(ids))

        selectivity = f"~{n_vectors // n_cat:} vectors/cat"
        p99 = sorted(latencies)[int(0.99 * len(latencies))]
        avg_lat = sum(latencies) / len(latencies)
        avg_results = sum(result_counts) / len(result_counts)

        print(f"{'selectivity':<20} {'n_cat':<15} {'avg_lat':>11.1f}ms "
              f"{'p99':>11.1f}ms {'avg_results':>12.1f}")

```

When you run this across databases, the output reveals something that synthetic benchmarks hide entirely: **highly selective filters (small buckets) cause different failure modes in different databases.** In pgvector, a highly selective filter may cause the planner to abandon the HNSW index and perform a sequential scan — which is slow but returns perfect recall. In Weaviate, the ef parameter drives how aggressively the system oversamples before filtering, so high selectivity requires increasing ef dramatically to maintain recall, which increases latency. In Qdrant, the filterable HNSW implementation tracks which vectors satisfy payload predicates inside the graph structure itself, which generally maintains recall at lower latency penalty — but requires the payload index to be created before ingest, not after.

These are the kinds of behavioral differences that do not appear in any vendor benchmark and that will determine which database actually fits your production query pattern.

Interpreting Benchmark Results: Common Traps

Running a benchmark is easier than interpreting one. There are several systematic traps that cause engineers to make wrong decisions from correct data.

The warm-cache trap. Most vector databases cache recently accessed graph nodes or segments in memory. Running your benchmark query set repeatedly will produce increasingly optimistic latency numbers as the cache warms. Always measure latency on a cold or realistically warm cache, which means either clearing the OS page cache before each run (`echo 3 > /proc/sys/vm/drop_caches` on Linux) or pre-warming with a separate query set that does not overlap your test queries.

The single-client trap. If your application generates concurrent queries — and production applications almost always do — measuring at concurrency=1 gives you latency numbers that are optimistic by a factor of 2–5x for CPU-bound workloads. Measure at concurrency levels matching your peak application concurrency, not your baseline.

The index-not-built trap. Some databases defer index construction until a threshold of writes is reached, or until a compaction job runs. Milvus, for example, will serve queries from growing segments using brute-force until a segment reaches a size threshold and gets indexed. If you run your benchmark immediately after loading, you may be benchmarking brute-force search, not ANN. Always verify index status before measuring. For Milvus, call `utility.index_building_progress()`. For Qdrant, check that `collection_info.status == "green"`. For pgvector, run `SELECT pg_size_pretty(pg_relation_size('your_index_name'))` and confirm it matches expected size.

The metric distance trap. Cosine similarity and Euclidean distance produce different nearest neighbor orderings, and recall numbers computed against ground truth using the wrong metric will be meaningless. Always confirm that your ground truth was computed with the same distance metric as your index. The benchmark harness above uses cosine similarity (dot product on normalized vectors) throughout — verify this is consistent if you adapt it.

The dimensionality compression trap. Some databases support scalar quantization (SQ8, SQ4) or product quantization (PQ) to compress vectors and reduce memory usage. These techniques trade recall for memory efficiency, sometimes dramatically. When comparing a quantized Milvus index against an unquantized pgvector HNSW index, you are not comparing like for like — you are comparing a memory-efficient approximate index against a full-precision one. Measure both the quantized and unquantized configurations if your target system uses quantization.

Continuous Benchmarking in CI/CD

A benchmark run before database selection is necessary but insufficient. Vector database performance degrades as data grows, as index parameters drift, and as query distributions shift when your application evolves. Embedding continuous benchmarks in your CI/CD pipeline catches regressions before they reach production.

The practical approach is a lightweight nightly benchmark job that runs a fixed 1,000-query set against your staging database and emits three metrics to your observability platform: recall@10, p99 latency, and QPS at your standard concurrency. When any of these crosses a threshold — recall drops below 0.95, p99 exceeds 50ms, QPS falls below 1,000 — the pipeline fires an alert.

```
# Python
# ci_benchmark.py
# Lightweight benchmark for CI integration - exits non-zero on regression

import sys
import h5py
import numpy as np
import psycogp2
import time

RECALL_THRESHOLD = 0.95
P99_LATENCY_THRESHOLD_MS = 50.0
MIN_QPS = 500.0

def run_ci_benchmark(dsn: str, dataset_path: str, k: int = 10) -> bool:
```

```

"""
Returns True if all thresholds pass, False on regression.
"""
with h5py.File(dataset_path, "r") as f:
    queries = f["test"][:100] # Use 100 queries for speed in CI
    ground_truth = f["neighbors"][:]

latencies = []
returned_ids = []

conn = psycopg2.connect(dsn)
conn.autocommit = True
with conn.cursor() as cur:
    cur.execute("SET hnswe.ef_search = 100")
    for vec in queries:
        vec_str = "[" + ",".join(f"{v:.6f}" for v in vec) + "]"
        t0 = time.perf_counter()
        cur.execute(
            f"SELECT id FROM benchmark_vectors "
            f"ORDER BY embedding <=> %s::vector LIMIT {k}",
            (vec_str,)
        )
        ids = [row[0] for row in cur.fetchall()]
        latencies.append((time.perf_counter() - t0) * 1000)
        returned_ids.append(ids)
conn.close()

# Compute metrics
n = len(queries)
recall = sum(
    len(set(ground_truth[i, :k]) & set(returned_ids[i][:k])) / k
    for i in range(n)
) / n
latencies_sorted = sorted(latencies)
p99 = latencies_sorted[int(0.99 * n)]
qps = n / (sum(latencies) / 1000)

print(f"Recall@{k}: {recall:.4f} (threshold: {RECALL_THRESHOLD})")
print(f"p99 latency: {p99:.1f}ms (threshold: {P99_LATENCY_THRESHOLD_MS}ms)")
print(f"QPS: {qps:.0f} (threshold: {MIN_QPS})")

passed = True
if recall < RECALL_THRESHOLD:
    print(f"FAIL: recall {recall:.4f} below threshold {RECALL_THRESHOLD}")
    passed = False
if p99 > P99_LATENCY_THRESHOLD_MS:
    print(f"FAIL: p99 {p99:.1f}ms exceeds threshold {P99_LATENCY_THRESHOLD_MS}ms")
    passed = False
if qps < MIN_QPS:
    print(f"FAIL: QPS {qps:.0f} below threshold {MIN_QPS}")
    passed = False

return passed

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--dsn", required=True)
    parser.add_argument("--dataset", required=True)
    args = parser.parse_args()

    passed = run_ci_benchmark(args.dsn, args.dataset)
    sys.exit(0 if passed else 1)

```

Embed this in your CI pipeline as a step that runs after migrations and before promotion to staging. The dataset file is committed to artifact storage (not to the repository, as it can be hundreds of megabytes) and fetched at CI runtime. The thresholds are checked into source control

alongside your application code, which means recall regressions are treated with the same severity as functional test failures.

Choosing Between Databases: A Decision Framework

After running benchmarks across pgvector, Pinecone, Weaviate, Qdrant, and Milvus, most teams face the same question: given similar recall numbers at similar latency, how do I choose? Pure throughput numbers rarely decide the question. The decision rests on four operational dimensions that benchmarks illuminate but do not automatically answer.

Dataset size and growth trajectory. If your dataset is under 5 million vectors today and is expected to grow slowly, pgvector on a well-provisioned PostgreSQL instance handles the workload with no additional infrastructure. The operational simplicity of a single system is worth significant throughput headroom. If your dataset is already in the tens of millions and growing at a rate that will require horizontal scaling within twelve months, start with a distributed system (Qdrant cluster, Weaviate with multiple shards, or Milvus) even if a single node would technically suffice today. Migration from single-node to distributed is painful; choosing the right architecture at 20M vectors is far cheaper than migrating at 80M.

Filter complexity. If your queries are predominantly unfiltered — similarity search against the full corpus — most databases perform comparably at equivalent hardware spend. If your queries are predominantly filtered by high-cardinality metadata (per-user, per-tenant, per-category with thousands of categories), Qdrant’s filterable HNSW is worth measuring carefully, because the recall-under-filter numbers differ meaningfully from unfiltered recall at the same of parameter values.

Operational maturity of your team. Milvus is architecturally sophisticated — it has separate query nodes, index nodes, data nodes, and a message queue — and running it reliably requires understanding all of those components. If your team has not operated distributed systems with complex topology before, the operational surface area is a real risk. Pinecone eliminates that risk entirely by being a managed service at the cost of query customization and egress pricing. There is no architecture decision that is universally correct; the right choice accounts for the humans who will be on call when it fails at 2am.

Latency budget and tail behavior. Managed services like Pinecone have network round-trips to their API endpoints that add fixed latency regardless of index parameters. If your application requires sub-10ms p99 and it is co-located in the same cloud region as the vector database, self-hosted Qdrant or pgvector will consistently outperform managed Pinecone on raw latency, because you eliminate the inter-service network hop. If your latency budget is 100ms and reliability is more important than raw speed, the managed operational model wins on total cost of ownership.

Key Takeaways

- **Recall, latency, and throughput form an inseparable triangle:** you can optimize any two by sacrificing the third. Every benchmark must measure all three simultaneously at the same index configuration, or the results are misleading. The recall-latency curve — not a single headline QPS number — is the authoritative diagnostic.

- **Benchmark datasets must match production distributions:** vectors generated by real embedding models on domain-relevant text cluster differently from synthetic random vectors, and recall numbers differ by 5–15 percentage points between synthetic and realistic datasets. Always build benchmarks from actual embeddings of actual text.
 - **Filtered search is where vendor benchmarks diverge most from production behavior:** the interaction between vector ANN indices and metadata filters produces dramatically different recall and latency outcomes across databases. Measure filtered recall at multiple selectivity levels before trusting any performance claim for a workload that combines similarity search with attribute filtering.
 - **Index state validation is a prerequisite for valid measurements:** databases like Milvus, Qdrant, and pgvector can serve queries from unindexed or partially indexed data, producing benchmark numbers that reflect brute-force search rather than ANN search. Always confirm index build completion and warm query caches with a separate, non-overlapping query set before recording measurements.
 - **Continuous benchmarking in CI/CD converts a one-time evaluation into an ongoing regression signal:** committing recall thresholds, p99 latency limits, and minimum QPS values alongside application code ensures that schema migrations, index parameter changes, data growth, and dependency upgrades are caught before they degrade production search quality.
-

Chapter 21: What is RAG and Why It Needs a Vector Database

Retrieval-Augmented Generation (RAG) is the architectural pattern that transformed large language models from impressive curiosities into production-grade knowledge systems. Without it, LLMs hallucinate facts, lack current information, and have no access to your proprietary data. With it, they become queryable interfaces over your own knowledge base—documents, databases, support tickets, code repositories, anything you can chunk and encode. This chapter explains how RAG works mechanically, why the retrieval step demands vector search rather than traditional full-text or relational lookup, and how to build a complete RAG pipeline from embeddings through generation using the tools your stack already supports or will soon.

21.1 The Problem RAG Solves

A large language model is a compressed, lossy representation of its training corpus. GPT-4 was trained on data with a knowledge cutoff. Llama 3 has never seen your internal runbooks. Claude has no idea what your Q3 revenue was. These are not bugs in the models—they are fundamental properties of how models are trained. The parameters encode statistical patterns from training data, not a queryable fact store. Asking an LLM for a specific fact it was not trained on is asking a relational database to return a row that was never inserted.

The naive fix—fine-tuning—is expensive, slow, and still doesn’t solve the recency problem cleanly. Fine-tuning updates weights to bake in new knowledge, but it takes hours to days, costs real money, and requires retraining every time the knowledge changes. You would not solve the problem of stale materialized views by rewriting your application. You would refresh the view.

RAG is the refresh. Instead of encoding knowledge into model weights, RAG retrieves relevant knowledge at query time and injects it into the model’s context window. The model is not expected to remember your internal documentation—it reads the relevant excerpts right before answering, the same way you read a specification before writing code.

The retrieval step is the entire game. If retrieval is accurate, the model answers correctly. If retrieval returns the wrong documents, the model either hallucinates a synthesis of bad inputs or confidently states something wrong. This is why the database layer matters so much in a RAG architecture. The vector database is not peripheral infrastructure—it is the system of record for retrievable knowledge.

Why traditional search fails here

Consider the query: “What’s our policy on extending vendor payment terms during cash flow stress?”

A BM25 keyword search over your internal policy documents might return results containing “vendor,” “payment,” “terms,” and “cash flow”—but keyword overlap is not semantic overlap. The document you want might use the phrase “supplier credit extension during liquidity events,” with no term overlap to the query at all. Keyword search ranks by lexical frequency; it has no concept of meaning.

Full-text search in PostgreSQL (`tsvector`, `tsquery`) is excellent at what it does—stemming, ranking by term frequency, phrase proximity—but it cannot close the semantic gap. A vector database stores documents as points in high-dimensional space where semantic similarity is encoded as geometric proximity. Documents with similar meaning cluster together regardless of their exact vocabulary.

This is the core value proposition of vector search in RAG: it retrieves by meaning, not by word. The rest of this chapter builds on that foundation.

21.2 RAG Architecture: The Three-Stage Pipeline

A RAG system has three distinct stages: ingestion, retrieval, and generation. Most production failures can be traced to one of these three stages being implemented incorrectly.

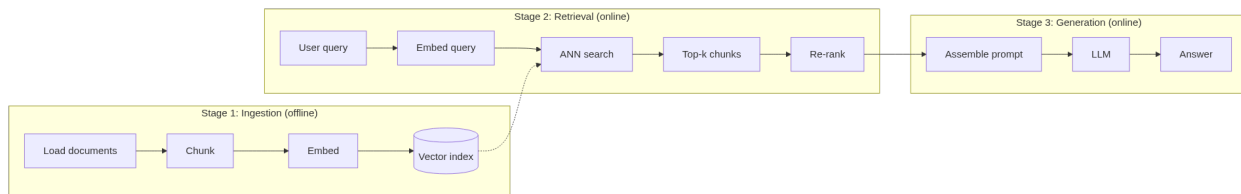


Figure 14: Ingestion runs offline as a batch pipeline; retrieval and generation run online, in the request path, every time a user asks a question.

Stage 1: Ingestion

Ingestion transforms raw documents into a searchable vector index. The steps are:

1. Load documents (PDFs, HTML, Markdown, database tables, etc.)
2. Chunk documents into segments the embedding model can handle
3. Generate vector embeddings for each chunk
4. Store embeddings plus metadata in a vector database
5. Optionally store the raw chunk text in a companion document store

Think of this as the ETL pipeline for your knowledge base. The output is a populated vector index—the equivalent of a pre-built full-text search index, except it captures semantic rather than lexical content.

Stage 2: Retrieval

At query time:

1. Embed the user's query using the same embedding model used during ingestion
2. Perform approximate nearest neighbor (ANN) search in the vector database
3. Return the top-k most similar chunks
4. Optionally re-rank results using a cross-encoder model

The returned chunks are the retrieved context. A typical RAG system returns 3–10 chunks, each 200–800 tokens in length.

Stage 3: Generation

1. Assemble a prompt: system instructions + retrieved context + user question
2. Send to an LLM (OpenAI, Anthropic, Bedrock, Vertex AI, etc.)
3. Stream or return the generated response

The LLM reads the retrieved context and produces an answer grounded in it. With a well-designed system prompt, the model will cite sources, refuse to answer when context is insufficient, and flag conflicting information.

Here is a minimal but complete RAG pipeline using LangChain, OpenAI embeddings, and pgvector:

```
# Python
# Minimal RAG pipeline: pgvector + OpenAI + LangChain
# Requirements: langchain, langchain-openai, langchain-postgres, psycopg2-binary

import os
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_postgres.vectorstores import PGVector
from langchain.chains import RetrievalQA
from langchain.docstore.document import Document
from langchain.text_splitter import RecursiveCharacterTextSplitter

# --- Configuration ---
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
PG_CONNECTION = "postgresql+psycopg2://user:password@localhost:5432/ragdb"
COLLECTION_NAME = "company_docs"

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# --- Ingestion ---
raw_documents = [
    "Vendors may request payment term extensions of up to 90 days during "
    "approved liquidity stress periods. Finance VP approval required above 60 days.",
    "Standard vendor payment terms are net-30. Early payment discounts of 2% "
    "are available for payment within 10 days.",
    "All vendor contracts must be reviewed by Legal for terms exceeding $100,000 "
    "annual value before execution.",
]

splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=50)
docs = splitter.create_documents(raw_documents)

# Store embeddings in pgvector
vectorstore = PGVector.from_documents(
    documents=docs,
    embedding=embeddings,
    connection=PG_CONNECTION,
    collection_name=COLLECTION_NAME,
    pre_delete_collection=True, # reset for demo; remove in production
)

# --- Retrieval + Generation ---
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
```

```

chain_type="stuff", # stuff = concatenate all retrieved chunks into context
retriever=vectorstore.as_retriever(search_kwargs={"k": 3}),
return_source_documents=True,
)

query = "What is our policy on extending payment terms with vendors during cash flow stress?"
result = qa_chain.invoke({"query": query})

print("Answer:", result["result"])
print("\nSource chunks retrieved:")
for doc in result["source_documents"]:
    print(" -", doc.page_content[:120])

```

This is intentionally minimal. The production version involves metadata filtering, hybrid search, re-ranking, prompt templating with citations, and streaming—all covered in later chapters.

21.3 Embeddings: The Translation Layer Between Text and Geometry

An embedding model is a neural network that projects text into a fixed-dimensional vector space. The model is trained such that semantically related text lands near each other in that space. The resulting vectors have no human-interpretable meaning—they are coordinates in a learned geometric space.

For a database professional, the analogy is a hash function, except the hash preserves semantic relationships rather than distributing values uniformly. Two documents about the same topic hash to nearby values; two documents about different topics hash to distant values.

Different embedding models produce vectors of different dimensions:

Model	Dimensions	Notes
text-embedding-3-small (OpenAI)	1536	Cost-efficient, strong general performance
text-embedding-3-large (OpenAI)	3072	Higher accuracy, 2× cost
embed-english-v3.0 (Cohere)	1024	Strong retrieval-optimized model
all-MiniLM-L6-v2 (sentence-transformers)	384	Lightweight, runs locally
bge-large-en-v1.5 (BAAI via HuggingFace)	1024	Strong open-weight option
nomic-embed-text (Nomic)	768	Open, competitive with commercial models

The dimension count determines your storage cost and search latency. A 1536-dimension vector stored as float32 occupies 6,144 bytes. At 10 million documents, that is ~58 GB just for vectors—before indexes, metadata, or raw text.

A hard rule with embeddings: the model used during ingestion and the model used at query time must be identical. Querying a pgvector index populated with OpenAI `text-embedding-3-small`

vectors using Cohere embeddings produces meaningless results. This is the vector-database equivalent of a character encoding mismatch—the numbers exist but represent different things.

```
# Python
# Comparing embedding models with sentence-transformers (local, no API key needed)
# Requirements: sentence-transformers

from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2") # 384-dimensional, runs on CPU

docs = [
    "Vendor payment terms can be extended during liquidity stress events.",
    "Suppliers may receive credit extensions when the company faces cash shortfalls.",
    "The quarterly earnings report showed a 12% revenue increase.",
    "Employee vacation accrual resets on January 1st each year.",
]

embeddings = model.encode(docs, normalize_embeddings=True)

# Cosine similarity via dot product (vectors are L2-normalized)
query = "Can we delay paying our suppliers if we have cash flow problems?"
query_vec = model.encode([query], normalize_embeddings=True)[0]

similarities = embeddings @ query_vec # matrix multiply = cosine similarity

print("Similarity to query: 'Can we delay paying our suppliers...'",)
for doc, score in sorted(zip(docs, similarities), key=lambda x: -x[1]):
    print(f" [{score:.4f}] {doc[:80]}")
```

Expected output demonstrates the semantic matching: the first two documents score 0.85+ against the query despite sharing almost no exact vocabulary with it. The vacation accrual document scores near 0.1.

Embedding model selection criteria for production

Choose your embedding model before populating any index. Re-embedding a 10-million-document corpus because you switched models is a full re-ingestion job. Evaluate on:

- *Dimensionality vs. accuracy tradeoff*: Smaller models are faster and cheaper; larger models retrieve more precisely. Benchmark on your actual data.
- *Max token length*: `text-embedding-3-small` supports 8,191 tokens. `all-MiniLM-L6-v2` supports 256 tokens. Chunks longer than the model's limit are truncated silently in some frameworks.
- *Multilingual requirements*: If your corpus contains multiple languages, use a multilingual model (`paraphrase-multilingual-mpnet-base-v2`, Cohere's multilingual models, or OpenAI's multilingual-capable models).
- *Latency at query time*: API-based models (OpenAI, Cohere) add 50–200ms per query. Local models (sentence-transformers on GPU) add 5–20ms. This matters at scale.

21.4 Vector Search Mechanics and Why It Beats Full-Text for Retrieval

When a user submits a query to a RAG system, the retrieval step must return the most semantically relevant documents from potentially millions of stored chunks. The mechanism is approximate nearest neighbor search over high-dimensional vectors.

Vector distance can be measured multiple ways:

- **Cosine similarity:** Measures angle between vectors, ignoring magnitude. Standard for normalized text embeddings. Equivalent to dot product when vectors are unit-normalized.
- **Euclidean distance (L2):** Measures straight-line distance. Works well when vector magnitude carries meaning.
- **Dot product (inner product):** Efficient when vectors are not normalized; used in some recommendation systems.

In pgvector, you select the operator at query time:

```
-- PostgreSQL / pgvector
-- Setup: enable extension and create table
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
  id          BIGSERIAL PRIMARY KEY,
  doc_id      TEXT NOT NULL,
  chunk_index INTEGER NOT NULL,
  content     TEXT NOT NULL,
  metadata    JSONB,
  embedding   vector(1536) -- matches text-embedding-3-small dimensions
);

-- HNSW index for approximate nearest neighbor search
-- Faster queries than IVFFlat, better recall, higher memory usage
CREATE INDEX ON document_chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Retrieval query: find top-5 chunks for an embedded query
-- In practice, :query_embedding is replaced with the actual float array from Python
SELECT
  id,
  doc_id,
  chunk_index,
  content,
  metadata,
  1 - (embedding <=> '[0.021, -0.045, 0.112, ...]':vector) AS cosine_similarity
FROM document_chunks
ORDER BY embedding <=> '[0.021, -0.045, 0.112, ...]':vector
LIMIT 5;

-- Metadata-filtered retrieval (department-specific knowledge base)
SELECT
  id,
  doc_id,
  content,
  1 - (embedding <=> '[0.021, -0.045, 0.112, ...]':vector) AS cosine_similarity
FROM document_chunks
WHERE metadata->>'department' = 'finance'
ORDER BY embedding <=> '[0.021, -0.045, 0.112, ...]':vector
LIMIT 5;

-- Hybrid search: combine vector similarity with keyword relevance
-- Uses reciprocal rank fusion to merge two ranked lists
WITH vector_results AS (
  SELECT
    id, content,
    ROW_NUMBER() OVER (ORDER BY embedding <=> '[0.021, -0.045, 0.112, ...]':vector) AS vec_rank
  FROM document_chunks
  WHERE metadata->>'department' = 'finance'
  LIMIT 20
),
fts_results AS (
  SELECT
    id, content,
```

```

    ROW_NUMBER() OVER (ORDER BY ts_rank(to_tsvector('english', content),
                                      to_tsquery('english', 'vendor & payment & terms')) DESC) AS fts_rank
FROM document_chunks
WHERE metadata->>'department' = 'finance'
    AND to_tsvector('english', content) @@ to_tsquery('english', 'vendor & payment & terms')
LIMIT 20
)
SELECT
    COALESCE(v.id, f.id) AS id,
    COALESCE(v.content, f.content) AS content,
    (1.0 / (60 + COALESCE(v.vec_rank, 1000))) +
    (1.0 / (60 + COALESCE(f.fts_rank, 1000))) AS rrf_score
FROM vector_results v
FULL OUTER JOIN fts_results f ON v.id = f.id
ORDER BY rrf_score DESC
LIMIT 5;

```

The `<=>` operator is cosine distance in pgvector. Distance is minimized (nearest = most similar), so `ORDER BY embedding <=> query_vec LIMIT k` returns the `k` most similar documents.

ANN index tradeoffs

pgvector supports two index types:

- **HNSW** (Hierarchical Navigable Small World): Graph-based index. Faster queries (sub-millisecond at millions of vectors), higher memory usage, slower build time. The `m` parameter controls graph connectivity (default 16; higher = better recall, more memory). The `ef_construction` parameter controls build quality (default 64; higher = better recall, slower build).
- **IVFFlat** (Inverted File Flat): Partitions vectors into clusters, searches only the nearest clusters. Lower memory, faster builds, slightly lower recall. The `lists` parameter should be approximately `sqrt(row_count)` for balanced performance.

For production RAG at under 1 million vectors, HNSW is the standard choice. Beyond 10 million vectors on a single PostgreSQL instance, evaluate managed vector databases (Pinecone, Weaviate, Qdrant) for their distributed indexing capabilities.

21.5 Building a Production RAG Pipeline with Pinecone and LlamaIndex

Pinecone is a managed vector database designed specifically for production retrieval workloads. It handles index distribution, replication, and scaling transparently. LlamaIndex is a framework focused on document ingestion pipelines and RAG orchestration with strong support for structured metadata and multi-step retrieval.

This example builds a multi-tenant RAG system where different departments have isolated knowledge bases, using Pinecone namespaces for isolation and LlamaIndex for orchestration:

```

# Python
# Production RAG with Pinecone + LlamaIndex + OpenAI
# Requirements: llama-index, llama-index-vector-stores-pinecone,
#               llama-index-embeddings-openai, llama-index-llms-openai, pinecone-client

import os
from pinecone import Pinecone, ServerlessSpec
from llama_index.core import VectorStoreIndex, Document, Settings
from llama_index.core.node_parser import SentenceSplitter

```

```

from llama_index.vector_stores.pinecone import PineconeVectorStore
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.llms.openai import OpenAI
from llama_index.core import StorageContext

# --- Configuration ---
PINECONE_API_KEY = os.environ["PINECONE_API_KEY"]
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
INDEX_NAME = "company-knowledge-base"

# --- Global LlamaIndex settings ---
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small", dimensions=1536)
Settings.llm = OpenAI(model="gpt-4o-mini", temperature=0)
Settings.node_parser = SentenceSplitter(chunk_size=512, chunk_overlap=64)

# --- Initialize Pinecone ---
pc = Pinecone(api_key=PINECONE_API_KEY)

if INDEX_NAME not in [idx.name for idx in pc.list_indexes()]:
    pc.create_index(
        name=INDEX_NAME,
        dimension=1536,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1"),
    )

pinecone_index = pc.Index(INDEX_NAME)

# --- Ingest documents into a department-specific namespace ---
def ingest_department_docs(department: str, documents: list[dict]) -> VectorStoreIndex:
    """
    Ingest documents for a department into an isolated Pinecone namespace.
    Each document dict has 'text' and optional 'source' keys.
    """
    llama_docs = [
        Document(
            text=doc["text"],
            metadata={
                "department": department,
                "source": doc.get("source", "unknown"),
                "doc_type": doc.get("doc_type", "policy"),
            },
        )
        for doc in documents
    ]

    vector_store = PineconeVectorStore(
        pinecone_index=pinecone_index,
        namespace=department, # namespace isolates per-department vectors
    )
    storage_context = StorageContext.from_defaults(vector_store=vector_store)

    index = VectorStoreIndex.from_documents(
        llama_docs,
        storage_context=storage_context,
        show_progress=True,
    )
    return index

# --- Query a department namespace ---
def query_department(department: str, question: str, top_k: int = 4) -> dict:
    """
    Query the knowledge base for a specific department with source attribution.
    """
    vector_store = PineconeVectorStore(
        pinecone_index=pinecone_index,
        namespace=department,
    )
    storage_context = StorageContext.from_defaults(vector_store=vector_store)

```



```
index = VectorStoreIndex.from_vector_store(vector_store, storage_context=storage_context)

query_engine = index.as_query_engine(
    similarity_top_k=top_k,
    response_mode="compact", # compact: merge context, minimize LLM calls
)

response = query_engine.query(question)

return {
    "answer": str(response),
    "sources": [
        {
            "text": node.text[:200],
            "score": node.score,
            "source": node.metadata.get("source"),
        }
        for node in response.source_nodes
    ],
}

# --- Example usage ---
finance_docs = [
    {
        "text": "Vendor payment terms may be extended up to 90 days during approved "
               "liquidity stress periods with Finance VP sign-off.",
        "source": "finance-policy-v2.pdf",
        "doc_type": "policy",
    },
    {
        "text": "Early payment discount programs offer 2% net-10 terms for vendors "
               "who opt into the dynamic discounting platform.",
        "source": "vendor-program-guide.pdf",
        "doc_type": "guide",
    },
]

# Ingest (run once per update cycle)
ingest_department_docs("finance", finance_docs)

# Query
result = query_department(
    department="finance",
    question="What approval is needed to extend payment terms beyond 60 days?",
)

print("Answer:", result["answer"])
print("\nRetrieved sources:")
for src in result["sources"]:
    print(f"    Score: {src['score']:.4f} | Source: {src['source']}")
    print(f"    Text: {src['text']}")
```

Namespace isolation is a production pattern, not just a feature. Using Pinecone namespaces (or Weaviate multi-tenancy, or Qdrant collections per tenant) prevents cross-department data leakage at the retrieval layer. Without it, a query in the finance namespace could retrieve HR documents if they happen to be semantically adjacent—an information security problem masquerading as a relevance problem.

21.6 RAG Failure Modes and What the Database Can Fix

Understanding where RAG fails is more useful than understanding where it succeeds, because the failure modes map directly to architectural decisions in the database layer.

Failure mode 1: Retrieval misses the relevant chunk

Cause: The embedding model maps the query to a region of vector space that does not overlap with the relevant document's region. Can happen with highly domain-specific terminology (model was not trained on it), very short or ambiguous queries, or poor chunking that dilutes signal.

Fix: Hybrid search (vector + BM25 keyword, combined via reciprocal rank fusion). The SQL hybrid search example in §21.4 demonstrates this pattern. Weaviate has native hybrid search built in:

```
# Python
# Weaviate hybrid search - combines BM25 and vector search natively
# Requirements: weaviate-client>=4.0

import weaviate
import weaviate.classes as wvc
import os

client = weaviate.connect_to_weaviate_cloud(
    cluster_url=os.environ["WEAVIATE_URL"],
    auth_credentials=wvc.init.Auth.api_key(os.environ["WEAVIATE_API_KEY"]),
)

collection = client.collections.get("DocumentChunks")

# Alpha = 0.0 → pure BM25, Alpha = 1.0 → pure vector, 0.5 → balanced hybrid
results = collection.query.hybrid(
    query="vendor payment term extension during liquidity stress",
    alpha=0.5,
    limit=5,
    filters=wvc.query.Filter.by_property("department").equal("finance"),
    return_metadata=wvc.query.MetadataQuery(score=True, explain_score=True),
)

for obj in results.objects:
    print(f"Score: {obj.metadata.score:.4f}")
    print(f"Text: {obj.properties['content'][:120]}")
    print()

client.close()
```

Failure mode 2: Retrieved chunks have the answer but the LLM ignores it

Cause: “Lost in the middle” problem. Research has shown that LLMs attend more strongly to text at the beginning and end of their context window. If the relevant chunk is buried in position 4 of 8 retrieved documents, the model may not weight it correctly.

Fix: Re-ranking. After vector retrieval, apply a cross-encoder re-ranker (Cohere Rerank, ms-marco-MiniLM local models, or bge-reranker-large) to reorder results by true relevance before injecting into context. Cross-encoders compare query and document jointly rather than independently, producing more precise relevance scores at the cost of higher latency.

```
# Python
# Re-ranking with Cohere Rerank API
# Requirements: cohere

import cohere
import os

co = cohere.Client(os.environ["COHERE_API_KEY"])

# Assume vector_chunks was retrieved from pgvector/Pinecone/Weaviate
vector_chunks = [
    "Standard payment terms are net-30 for all vendors.",
    "Finance VP approval is required for payment term extensions beyond 60 days.",

```

```

    "Vendor onboarding requires a completed W-9 and insurance certificate.",
    "Liquidity stress extensions may reach 90 days with documented approval.",
    "Early payment discounts of 2% are available for net-10 settlement.",
]

query = "What approval do I need to extend vendor payment terms beyond 60 days?"

rerank_result = co.rerank(
    model="rerank-english-v3.0",
    query=query,
    documents=vector_chunks,
    top_n=3,
    return_documents=True,
)

print("Re-ranked results:")
for item in rerank_result.results:
    print(f"  Relevance score: {item.relevance_score:.4f}")
    print(f"  Text: {item.document.text[:100]}")

```

Re-ranking consistently improves end-to-end RAG quality without requiring changes to the vector index or embedding model. It is one of the highest-leverage improvements available to a RAG system after the initial implementation is working.

Failure mode 3: The knowledge base is stale

Cause: Ingestion is a batch job run infrequently. A document updated on Monday does not appear in retrieval results until the next ingestion run.

Fix: Treat ingestion as a streaming ETL pipeline rather than a batch job. Use document change detection (file modification timestamps, database CDC events, webhook triggers from document management systems) to trigger incremental re-embedding of only changed documents. In pgvector, this is an upsert pattern:

```

-- PostgreSQL
-- Upsert pattern for incremental ingestion
-- doc_id + chunk_index uniquely identifies a chunk
INSERT INTO document_chunks (doc_id, chunk_index, content, metadata, embedding)
VALUES
    ('policy-vendor-payment-v3.pdf', 0, 'Updated policy text...', '{"version": 3}':::jsonb,
    → ' [...]':::vector)
ON CONFLICT (doc_id, chunk_index)
DO UPDATE SET
    content = EXCLUDED.content,
    metadata = EXCLUDED.metadata,
    embedding = EXCLUDED.embedding,
    updated_at = NOW();

-- Add a unique constraint to support the upsert
-- (Run once during schema setup)
ALTER TABLE document_chunks
    ADD CONSTRAINT uq_doc_chunk UNIQUE (doc_id, chunk_index);

```

Failure mode 4: Context window overflow

Cause: Top-k is set too high, or chunks are too large. At k=10 with 600-token chunks, you are injecting 6,000 tokens of context before the question. With GPT-4o's 128k context window this is manageable, but with latency-sensitive or cost-sensitive deployments it creates problems.

Fix: Tune chunk size and top-k together. A useful mental model: the context budget available for retrieved content is `max_context_tokens - system_prompt_tokens - query_tokens - response_buffer`. Divide that by your chunk size to get maximum viable k. Track cost per query using token counting before optimizing.

Failure mode 5: The model hallucinates despite retrieved context

Cause: The retrieved context contradicts itself (multiple documents with conflicting facts), the question asks for something not covered in any retrieved document, or the system prompt does not instruct the model to stay grounded.

Fix: System prompt engineering. The system prompt must explicitly instruct the model to: - Answer only from the provided context - Say “I don’t have information about that in the available documents” when context is absent - Flag conflicting information rather than synthesizing a false resolution

```
# Python
# System prompt pattern for grounded generation

SYSTEM_PROMPT = """You are a knowledge base assistant. Answer questions using ONLY the
provided context documents. Follow these rules strictly:

1. If the context contains the answer, provide it with a reference to the specific
document or section.
2. If the context does not contain sufficient information to answer the question,
respond with: "The available documents do not contain information about this topic."
3. If the context contains conflicting information on the topic, state the conflict
explicitly: "The documents contain conflicting information: [source A states X],
[source B states Y]."
4. Do not use knowledge from outside the provided context.
5. Do not speculate or extrapolate beyond what is stated in the context.

Context documents:
{context}
"""

def build_prompt(retrieved_chunks: list[dict], question: str) -> list[dict]:
    """Build a chat prompt with retrieved context for grounded generation."""
    context_text = "\n\n--\n\n".join(
        f"[Document {i+1}]: {chunk.get('source', 'unknown')}] \n{chunk['text']}"
        for i, chunk in enumerate(retrieved_chunks)
    )
    return [
        {"role": "system", "content": SYSTEM_PROMPT.format(context=context_text)},
        {"role": "user", "content": question},
    ]
```

21.7 Choosing Your Vector Database for RAG

The choice of vector database is a function of your existing infrastructure, scale requirements, and operational tolerance for managed services.

pgvector (PostgreSQL extension)

Best when: Your data volume fits on a single PostgreSQL instance (under ~5 million vectors for comfortable HNSW performance), your team already operates PostgreSQL, and you want transactional consistency between vector search and relational data.

Operational model: Self-managed (or managed via RDS, Supabase, Neon, AlloyDB). pgvector 0.7+ added HNSW support; earlier versions used only IVFFlat. Full ACID guarantees. SQL joins between vector results and relational tables work naturally.

Limitations: Single-node; no distributed index. ANN search is slower than dedicated vector

databases at very high QPS because PostgreSQL's query executor is not purpose-built for vector operations.

Pinecone

Best when: You need a fully managed, scalable vector database with no operational overhead. Teams that do not want to manage infrastructure for their retrieval layer. Serverless tier is appropriate for development and medium-scale production.

Operational model: Fully managed SaaS. Namespaces for multi-tenancy. Serverless (pay per query and storage) or pod-based (dedicated resources, predictable latency). No SQL; Pinecone-specific SDK.

Limitations: Vendor lock-in is real—your vectors live in Pinecone's infrastructure. No SQL joins; metadata filtering has limitations compared to relational predicates. Data export is possible but not trivial.

Weaviate

Best when: You want hybrid search (BM25 + vector) built in without writing RRF fusion yourself, or when your use case benefits from Weaviate's native multi-tenancy model. Open source with cloud hosted option.

Operational model: Self-hosted via Docker/Kubernetes or managed via Weaviate Cloud. GraphQL and REST APIs. Strong schema definition with property-level configuration.

Limitations: More operationally complex to self-host than pgvector. The GraphQL API has a learning curve for teams fluent in SQL. Resource requirements (RAM for HNSW in-memory index) can be significant at scale.

Qdrant

Best when: Performance is the primary criterion. Qdrant's Rust implementation delivers consistently high throughput and low latency. Strong support for payload (metadata) filtering with pre-filtering applied before ANN, which is important for highly filtered queries.

Operational model: Self-hosted (Docker, Kubernetes, binary) or Qdrant Cloud managed. REST and gRPC APIs. Collections-based.

Limitations: Smaller ecosystem and community than Pinecone or Weaviate. Fewer native integrations with LLM frameworks, though LangChain and LlamaIndex both support it.

```
# Python
# Qdrant RAG ingestion and retrieval
# Requirements: qdrant-client, sentence-transformers

from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    Filter, FieldCondition, MatchValue
)
from sentence_transformers import SentenceTransformer
import uuid

client = QdrantClient(host="localhost", port=6333)
model = SentenceTransformer("all-MiniLM-L6-v2")

COLLECTION = "company_docs"
DIM = 384 # all-MiniLM-L6-v2 dimension
```

```
# Create collection (idempotent check)
existing = [c.name for c in client.get_collections().collections]
if COLLECTION not in existing:
    client.create_collection(
        collection_name=COLLECTION,
        vectors_config=VectorParams(size=DIM, distance=Distance.COSINE),
    )

# Ingest
documents = [
    {"text": "Finance VP approval required for payment extensions beyond 60 days.",
     "department": "finance", "source": "policy-v3.pdf"},
    {"text": "All vendor contracts above $100k need Legal review.",
     "department": "Legal", "source": "contracts-policy.pdf"},
]

points = []
for doc in documents:
    vec = model.encode(doc["text"], normalize_embeddings=True).tolist()
    points.append(PointStruct(
        id=str(uuid.uuid4()),
        vector=vec,
        payload={"text": doc["text"], "department": doc["department"],
                 "source": doc["source"]},
    ))

client.upsert(collection_name=COLLECTION, points=points)

# Retrieval with metadata filter
query = "What approval do I need to extend payment terms?"
query_vec = model.encode(query, normalize_embeddings=True).tolist()

results = client.search(
    collection_name=COLLECTION,
    query_vector=query_vec,
    query_filter=Filter(
        must=[FieldCondition(key="department", match=MatchValue(value="finance"))]
    ),
    limit=3,
    with_payload=True,
)

for hit in results:
    print(f"Score: {hit.score:.4f} | Source: {hit.payload['source']}")
    print(f"    {hit.payload['text']}")
```

21.8 Chunking Strategies and Their Database Implications

Chunking is the step that determines retrieval quality more than almost anything else, yet it is often implemented as an afterthought. The way you divide documents into chunks determines whether semantically coherent information ends up in the same vector or split across multiple vectors that may never be retrieved together.

Fixed-size chunking

Split every *n* tokens, with optional overlap. Simple, predictable, and wrong for structured documents. A 500-token chunk that starts mid-sentence and ends mid-paragraph contains arbitrary semantic content.

Use when: Documents are already semantically uniform (e.g., uniform-length Q&A pairs, log lines).

Recursive character text splitting

Split at the largest available boundary (paragraph → sentence → word → character) until chunks are below the target size. LangChain's `RecursiveCharacterTextSplitter` implements this. Produces much more coherent chunks than fixed-size for prose documents.

Semantic chunking

Split based on embedding similarity discontinuities: generate embeddings for each sentence, compute similarity between adjacent sentences, and split where similarity drops sharply. Computationally expensive but produces chunks that correspond to coherent topics.

```
# Python
# Semantic chunking with LlamaIndex SemanticSplitterNodeParser
# Requirements: llama-index, llama-index-embeddings-openai

from llama_index.core.node_parser import SemanticSplitterNodeParser
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.core import Document

embed_model = OpenAIEmbedding(model="text-embedding-3-small")

splitter = SemanticSplitterNodeParser(
    buffer_size=1,          # sentences on each side of split point to consider
    breakpoint_percentile_threshold=95, # split when similarity drops below 95th percentile
    embed_model=embed_model,
)

document = Document(text="""
Vendor payment terms govern the timeline for invoice settlement. Standard terms
are net-30, meaning invoices must be paid within 30 days of receipt. Early payment
discounts of 2% are available for settlement within 10 days under the net-10 program.

Employee benefits include health insurance, dental, and vision coverage. Open
enrollment occurs annually in November. Employees may add dependents during the
open enrollment window or within 30 days of a qualifying life event.

Capital expenditure requests above $50,000 require CFO approval. Requests
between $10,000 and $50,000 require department VP approval. All CapEx requests
must be submitted through the procurement portal with a business case attached.
""")

nodes = splitter.get_nodes_from_documents([document])

print(f"Document split into {len(nodes)} semantic chunks:")
for i, node in enumerate(nodes):
    print(f"\n--- Chunk {i+1} ({len(node.text.split())} words) ---")
    print(node.text.strip()[:200])
```

This document would be split into three chunks—vendor payment, employee benefits, and CapEx—because each paragraph covers a distinct topic. Fixed-size chunking would likely merge the first two into a single chunk, degrading retrieval precision for both topics.

Parent-child chunking (small-to-big retrieval)

Store small child chunks for retrieval (better semantic targeting) but return the larger parent chunk to the LLM (more complete context). The parent-child relationship is tracked in metadata or a companion document store.

```
# Python
# Parent-child chunking pattern
# Small chunks retrieved, parent chunk returned to LLM

from llama_index.core.node_parser import HierarchicalNodeParser, get_leaf_nodes
from llama_index.core import Document, VectorStoreIndex
from llama_index.core.retrievers import AutoMergingRetriever
from llama_index.core.storage.docstore import SimpleDocumentStore
```

```

# HierarchicalNodeParser creates chunk sizes at multiple granularities:
# Level 1 (parent): 1024 tokens
# Level 2 (child): 256 tokens
# Level 3 (leaf): 64 tokens (these are what get embedded and retrieved)
node_parser = HierarchicalNodeParser.from_defaults(
    chunk_sizes=[1024, 256, 64]
)

document = Document(text="[long policy document content here...]")
all_nodes = node_parser.get_nodes_from_documents([document])
leaf_nodes = get_leaf_nodes(all_nodes)

# Store ALL nodes in a docstore so parents can be fetched at retrieval time
docstore = SimpleDocumentStore()
docstore.add_documents(all_nodes)

# Index only the leaf nodes (small chunks) for vector search
index = VectorStoreIndex(leaf_nodes)

# AutoMergingRetriever: retrieves leaf nodes, merges to parent if
# a sufficient fraction of sibling leaves are retrieved for same parent
base_retriever = index.as_retriever(similarity_top_k=12)
retriever = AutoMergingRetriever(
    base_retriever,
    storage_context=index.storage_context,
    verbose=True,
)

```

The parent-child pattern captures a real database design principle: index on granular keys but fetch at a coarser granularity when the query warrants it. The leaf node embedding is the index entry; the parent document is the row.

Database implications of chunking decisions

Every chunking decision has a storage and performance corollary:

Chunking approach	Chunks per document	Index size	Retrieval precision	Context quality
Fixed-size, no overlap	Low	Smallest	Low	Variable
Fixed-size + overlap	Medium	Medium	Medium	Improved
Recursive/semantic	Medium	Medium	High	High
Parent-child	High (leaf)	Larger	Highest (leaf)	Highest (parent)

Overlap—having adjacent chunks share a window of tokens—prevents the situation where a sentence that straddles a chunk boundary becomes semantically unretrievable. Standard overlap is 10–20% of chunk size. Excessive overlap (>40%) increases index size without proportional retrieval improvement.

Key Takeaways

- **RAG grounds LLMs in external knowledge at query time** rather than encoding knowledge into model weights, solving hallucination and recency problems without the cost

or latency of fine-tuning. The retrieval step is the critical path; a poorly performing retrieval layer cannot be compensated by a more capable LLM.

- **Vector search retrieves by semantic meaning, not keyword overlap.** This is what allows a query about “cash flow stress and supplier flexibility” to retrieve a document about “liquidity events and vendor credit extension.” Traditional full-text search cannot close this semantic gap, and hybrid search (vector + BM25) via reciprocal rank fusion combines the strengths of both for maximum recall.
 - **Embedding model consistency is an absolute requirement.** The same model must be used for ingestion and query-time encoding. Mismatched models produce geometrically meaningless results without generating any errors, making this failure mode particularly dangerous in production.
 - **Chunking strategy determines retrieval quality as much as embedding model choice.** Recursive and semantic chunking produce more coherent chunks than fixed-size splitting. Parent-child chunking achieves both precise retrieval (small chunks) and complete context (large parent returned to LLM). Chunk size, overlap, and top-k must be tuned together within the available context budget.
 - **RAG failure modes map directly to database-layer decisions.** Missed retrieval → fix with hybrid search or re-ranking. Stale knowledge → fix with incremental upsert-based ingestion. Cross-tenant data leakage → fix with namespace or collection isolation at the vector database layer. Hallucination despite good retrieval → fix with system prompt engineering that constrains generation to the retrieved context.
-

Chapter 22: Chunking Strategies: Size, Overlap, Semantic Splitting

Before a single embedding gets generated or a vector stored, every document passing through your RAG pipeline must be broken into pieces. That process—chunking—is one of the least glamorous and most consequential decisions you will make as a vector database engineer. Get it wrong and your retrieval system returns fragments that answer adjacent questions, embedding models that overflow their token limits, and LLM context windows stuffed with repeated text that buries the actual answer. This chapter covers the mechanics of chunking: fixed-size splits, overlap strategies, semantic and structural splitting, and the production patterns that determine which approach belongs in which pipeline.

22.1 Why Chunking Is a Database Design Problem

Database professionals recognize chunking immediately once you reframe it correctly: it is a normalization decision. You are decomposing a large, unstructured artifact into retrievable units whose granularity determines both storage efficiency and query precision. Too coarse and your “rows” carry too much noise; too fine and you lose context. Both failures show up at query time.

The analogy to indexing is tight. In a relational database, if you index too wide a column set, the index becomes bloated and less discriminating. If you index too narrow, queries return too many false positives that require expensive post-filtering. Embedding vectors work the same way. An embedding over 8,000 tokens captures broad thematic meaning but loses fine-grained retrieval precision. An embedding over 50 tokens is precise but brittle—small phrasing differences cause cosine distance to diverge even when semantic meaning is equivalent.

Embedding models impose a hard ceiling that makes this concrete. OpenAI’s `text-embedding-3-small` and `text-embedding-3-large` support up to 8,191 tokens. Cohere’s `embed-english-v3.0` supports up to 512 tokens per input by default (extendable, but with documented precision tradeoffs). `sentence-transformers/all-MiniLM-L6-v2`, popular for on-premise workloads, caps at 256 tokens and silently truncates anything longer. Overflowing these limits does not throw an error in most pipelines—the model simply truncates the input, which means your stored vector represents a partial document without any warning in the logs.

Three variables govern every chunking decision:

- **Chunk size:** The maximum number of tokens (or characters) in a single chunk. This is your primary granularity control.

- **Overlap:** The number of tokens repeated between adjacent chunks. Overlap preserves context across boundaries at the cost of storage and some retrieval redundancy.
- **Splitting logic:** Where you place boundaries—at fixed intervals, at sentence or paragraph boundaries, or at semantically coherent transition points.

A fourth variable that DBAs naturally gravitate toward is **metadata attachment**: every chunk should carry enough context to reconstruct its provenance—source document ID, position in the document, section heading, and any filtering dimensions like document type or access tier. Treating chunks as rows with rich metadata columns, rather than as opaque text blobs, is what separates a retrievable knowledge base from a vector landfill.

22.2 Fixed-Size Chunking: Mechanics and Limits

Fixed-size chunking splits documents at a specified token or character count, optionally with overlap. It is the SUBSTR of the chunking world—simple, predictable, and wrong in ways you can anticipate.

The implementation is straightforward. LangChain’s `RecursiveCharacterTextSplitter` is the most widely used production implementation. It tries a hierarchy of separators—paragraph breaks, newlines, spaces, then character-by-character—to split at the target size while respecting natural boundaries where possible.

```
# Python
from langchain.text_splitter import RecursiveCharacterTextSplitter
import tiktoken

def chunk_document(text: str, chunk_size: int = 512, chunk_overlap: int = 64) -> list[dict]:
    """
    Fixed-size chunking with token-accurate sizing using tiktoken.
    Returns chunks as dicts with text and positional metadata.
    """
    # Use cl100k_base (GPT-4 / text-embedding-3-* tokenizer)
    enc = tiktoken.get_encoding("cl100k_base")

    def token_length(s: str) -> int:
        return len(enc.encode(s))

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=token_length,
        separators=["\n\n", "\n", ".", "!", "?", " ", " ", ""],
    )

    raw_chunks = splitter.split_text(text)

    chunks = []
    for i, chunk_text in enumerate(raw_chunks):
        chunks.append({
            "chunk_index": i,
            "text": chunk_text,
            "token_count": token_length(chunk_text),
            "char_count": len(chunk_text),
        })

    return chunks

# Example: chunking a technical document
sample_text = """
PostgreSQL's MVCC implementation maintains multiple versions of each row

```

to support concurrent transactions without locking. When a transaction modifies a row, the old version remains visible to concurrent readers that started before the modification.

The visibility rules are governed by transaction IDs (XIDs). Each row version carries xmin and xmax values representing the creating and deleting transaction respectively. The snapshot isolation level reads the most recent committed version visible at transaction start time.

Autovacuum reclaims dead tuples—row versions no longer visible to any active transaction—by marking their storage space as reusable. Without regular vacuuming, table bloat accumulates and query plans degrade as the planner's statistics drift from physical reality.

```
"""  
  
chunks = chunk_document(sample_text, chunk_size=128, chunk_overlap=16)  
for chunk in chunks:  
    print(f"Chunk {chunk['chunk_index']}: {chunk['token_count']} tokens")  
    print(f"  Text: {chunk['text'][:80]}...")  
    print()
```

The `length_function` parameter is the detail most tutorials omit. Without it, LangChain defaults to character count. Characters and tokens are not the same unit. A 512-character chunk might be 90 tokens (dense prose) or 200 tokens (code with many single-character tokens). Using `tiktoken` with the correct encoding for your embedding model gives you accurate sizing and prevents silent truncation.

Overlap mechanics: An overlap of 64 tokens on 512-token chunks means 12.5% of each chunk is repeated in the next. This costs roughly 12.5% more storage and 12.5% more embedding API calls. The benefit is that sentences or concepts split across a chunk boundary appear in full in at least one chunk. A common production setting is 10–20% overlap. Beyond 25%, you start seeing retrieval results that are near-duplicates of each other, which frustrates reranking and LLM context packing.

Fixed-size chunking degrades in three predictable situations:

1. **Structured documents:** A 512-token chunk that starts mid-table and ends mid-code-block produces embeddings that mix multiple data types, making cosine similarity less meaningful.
2. **Documents with natural section boundaries:** A 4,000-token technical specification split mechanically will produce chunks that contain the end of one section and the beginning of another. The embedding represents neither topic cleanly.
3. **Short documents:** If most of your documents are under 200 tokens, chunking adds overhead without benefit. Store them as single vectors.

The performance characteristics are well-understood: for a corpus of one million documents averaging 2,000 tokens, 512-token chunks with 64-token overlap produce approximately 4.3 million chunks. At 1,536 dimensions (OpenAI's default), that is roughly 25 GB of raw vector data before indexing overhead. These numbers matter when sizing your pgvector table or your Pinecone pod configuration.

22.3 Overlap Design: What Gets Repeated and Why

Overlap deserves its own treatment because most engineers implement it mechanically without thinking through what the repeated content actually does to retrieval.

Consider a document about database transaction isolation levels. A chunk boundary falls in the middle of the paragraph explaining phantom reads. The first chunk ends with “...preventing dirty reads and non-repeatable reads. However, at this isolation level” and the second chunk begins with “...phantom reads can still occur when a transaction re-executes a range query.” Without overlap, neither chunk contains a complete explanation of phantom reads. With 64-token overlap, both chunks carry the boundary sentence, and whichever chunk scores higher for the query “what are phantom reads” will contain enough context to answer it.

This logic leads to a counter-intuitive insight: overlap primarily helps with retrieval precision, not recall. You are not finding more relevant chunks; you are ensuring the chunks you do find are self-contained enough to be useful.

```
# Python
# Demonstrating overlap behavior and its effect on boundary content

def analyze_chunk_boundaries(chunks: list[dict], overlap_tokens: int) -> None:
    """
    Shows what content is duplicated between adjacent chunks.
    Useful for verifying overlap is working as expected.
    """
    enc = tiktoken.get_encoding("cl100k_base")

    for i in range(len(chunks) - 1):
        current_tokens = enc.encode(chunks[i]["text"])
        next_tokens = enc.encode(chunks[i + 1]["text"])

        # Find the tail of the current chunk in the beginning of the next
        overlap_window = current_tokens[-overlap_tokens * 2:] # search region
        overlap_found = 0

        for size in range(min(overlap_tokens * 2, len(next_tokens)), 0, -1):
            if overlap_window[-size:] == next_tokens[:size]:
                overlap_found = size
                break

        overlap_text = enc.decode(next_tokens[:overlap_found]) if overlap_found else ""
        print(f"Boundary {i}->{i+1}: {overlap_found} overlapping tokens")
        if overlap_text:
            print(f"  Repeated: '{overlap_text[:60]}...'")

import tiktoken
chunks = chunk_document(sample_text, chunk_size=128, chunk_overlap=16)
analyze_chunk_boundaries(chunks, overlap_tokens=16)
```

Overlap and vector deduplication: When you store overlapping chunks, adjacent chunks will have higher cosine similarity to each other than to any other chunk in the corpus. This is expected. The problem arises during retrieval when a query returns both chunk N and chunk N+1 as top-K results, and the LLM receives essentially the same content twice. A production pipeline should include a deduplication pass during retrieval. The canonical approach is Maximum Marginal Relevance (MMR): after retrieving the top-20 candidates, re-rank to maximize diversity so that the top-5 passed to the LLM cover distinct semantic ground.

Sliding window vs. stepped overlap: The standard implementation steps forward by `chunk_size - overlap` tokens and re-embeds. An alternative is a sliding window that moves one sentence at a time and generates a chunk for each window position. This is expensive (a 1,000-sentence document generates ~1,000 embeddings) but produces very high retrieval density. Few production systems use pure sliding windows; the more practical approach is sentence-aware chunking described in the next section.

22.4 Structural and Semantic Splitting

Fixed-size splitting ignores document structure. Structural splitting respects it. Semantic splitting goes further and finds topical boundaries in the content itself.

Structural splitting uses document markers to define chunk boundaries: Markdown headings, HTML tags, PDF section headers, code block delimiters. For technical documentation, this is almost always superior to fixed-size splitting because your users' questions map to document sections, not to arbitrary 512-token windows.

LlamaIndex provides a `MarkdownNodeParser` that splits on heading hierarchy. For HTML, `HTMLNodeParser` extracts text from tagged sections. For code repositories, `CodeSplitter` uses the `tree-sitter` library to split at function or class boundaries—a significant advantage over splitting a Python file mid-function.

```
# Python
from llama_index.core.node_parser import (
    MarkdownNodeParser,
    SentenceSplitter,
    SemanticSplitterNodeParser,
)
from llama_index.core import Document
from llama_index.embeddings.openai import OpenAIEmbedding
import os

# --- Structural splitting: Markdown ---
markdown_text = """
# PostgreSQL Performance Tuning

## Connection Pooling

PgBouncer operates in three modes: session, transaction, and statement pooling.
Transaction pooling provides the best concurrency for OLTP workloads because
the server connection is held only during an active transaction, not for the
entire client session lifetime.

Connection limits should be set conservatively. A PostgreSQL instance with
`max_connections = 200` and 16 CPU cores will contend heavily beyond
~4x core count active connections under CPU-bound workloads.

## Index Strategy

Partial indexes reduce index size by only including rows matching a predicate.
A partial index on `orders(customer_id) WHERE status = 'pending'` is far
smaller than a full index on `customer_id` and serves the most performance-
sensitive queries—those looking up active work—with lower I/O cost.

### B-Tree vs. BRIN

B-Tree indexes support equality and range queries with  $O(\log n)$  lookup.
BRIN (Block Range INdex) is appropriate for naturally ordered data—timestamps,
sequential IDs—where physical row order correlates with column values.
A BRIN index on a 500GB timestamp column might be 50KB versus a 10GB B-Tree.
"""

markdown_doc = Document(text=markdown_text)
markdown_parser = MarkdownNodeParser()
markdown_nodes = markdown_parser.get_nodes_from_documents([markdown_doc])

print("=== Structural (Markdown) Chunks ===")
for node in markdown_nodes:
    print(f"Node ID: {node.node_id[:8]}")
```

```

print(f" Metadata: {node.metadata}")
print(f" Text ({len(node.text)} chars): {node.text[:100]}..." )
print()

# --- Semantic splitting ---
# SemanticSplitterNodeParser embeds sentences and splits where
# cosine distance between adjacent sentence embeddings exceeds a threshold.
# This finds topic transitions without relying on document markup.

embed_model = OpenAIEmbedding(
    model="text-embedding-3-small",
    api_key=os.environ["OPENAI_API_KEY"],
)

semantic_splitter = SemanticSplitterNodeParser(
    buffer_size=1,          # sentences to combine before comparing
    breakpoint_percentile_threshold=95, # split at top 5% of distance jumps
    embed_model=embed_model,
)

# A document where topics shift without heading markers
prose_text = """
Database replication ensures that multiple servers hold copies of the same data.
In streaming replication, the primary sends WAL records to standbys continuously.
The standby applies these records to stay current with the primary's state.
Replication lag is the delay between a change on the primary and its appearance
on the standby, measured in bytes of WAL or elapsed time.

Machine learning model training requires large volumes of labeled data.
The training loop iterates over batches, computes loss against ground truth,
and updates model weights via backpropagation. Gradient descent variants like
Adam and AdaGrad adjust the learning rate per parameter to accelerate convergence.
Overfitting occurs when a model learns training distribution noise rather than
generalizable patterns, detected by divergence between training and validation loss.

Query optimization in PostgreSQL uses a cost-based planner that estimates
the computational expense of candidate execution plans. The planner relies on
table statistics—row counts, column cardinality, value distributions—collected
by ANALYZE. Stale statistics cause the planner to choose suboptimal join orders
or index strategies, which is why autovacuum schedules ANALYZE alongside VACUUM.
"""

prose_doc = Document(text=prose_text)
semantic_nodes = semantic_splitter.get_nodes_from_documents([prose_doc])

print("=== Semantic Chunks ===")
for i, node in enumerate(semantic_nodes):
    print(f"Semantic chunk {i}: {len(node.text)} chars")
    print(f" First 80 chars: {node.text[:80]}..." )
    print()

```

The `SemanticSplitterNodeParser` embeds each sentence individually, then computes the cosine distance between adjacent sentence embeddings. Where distance spikes—at the 95th percentile in the example above—it places a chunk boundary. The prose example above has three clearly distinct topics: replication, ML training, and query optimization. Semantic splitting will find those three boundaries without relying on any markup. Fixed-size splitting at 256 tokens would likely put all three topics in a single chunk or split mid-paragraph within a topic.

The cost is API calls. For a 10,000-sentence corpus, semantic splitting requires 10,000 embedding calls just to determine chunk boundaries—before embedding the final chunks. This is acceptable for offline batch processing but prohibitive for real-time ingestion pipelines. Two practical mitigations: run semantic splitting as a preprocessing step and cache boundaries, or use a fast local model (`all-MiniLM-L6-v2`) for boundary detection and a higher-quality model for final chunk embeddings.

Hybrid approaches: The production pattern that works best for most technical corpora combines

structural and semantic splitting. First, split on document structure (headings, sections). Then apply semantic splitting within each section if the section exceeds your target chunk size. This bounds the number of embedding calls for boundary detection to one call per sentence within oversized sections, rather than the entire corpus.

22.5 Ingesting Chunks into pgvector and Pinecone

The chunking strategy determines chunk count and size. The ingestion strategy determines throughput, cost, and metadata queryability. These two concerns must be designed together.

The following example shows a complete ingestion pipeline: chunk a document, embed each chunk with OpenAI, and upsert into both pgvector (for hybrid search applications) and Pinecone (for pure vector workloads at scale).

```
# Python
import os
import uuid
import psycopg2
import openai
from pinecone import Pinecone, ServerlessSpec
from langchain.text_splitter import RecursiveCharacterTextSplitter
import tiktoken
import json

# --- Configuration ---
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
PINECONE_API_KEY = os.environ["PINECONE_API_KEY"]
PG_DSN = os.environ.get("PG_DSN", "postgresql://user:pass@localhost:5432/vectordb")
EMBEDDING_MODEL = "text-embedding-3-small"
EMBEDDING_DIM = 1536
CHUNK_SIZE = 512
CHUNK_OVERLAP = 64
BATCH_SIZE = 100 # embeddings per API call (OpenAI max: 2048)

# --- Clients ---
oai_client = openai.OpenAI(api_key=OPENAI_API_KEY)
pc = Pinecone(api_key=PINECONE_API_KEY)

# --- Tokenizer ---
enc = tiktoken.get_encoding("cl100k_base")
splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    length_function=lambda s: len(enc.encode(s)),
    separators=["\n\n", "\n", ". ", " ", ""],
)

def embed_batch(texts: list[str]) -> list[list[float]]:
    """Embed a batch of texts. Handles API rate limits with simple retry."""
    import time
    for attempt in range(3):
        try:
            response = oai_client.embeddings.create(
                model=EMBEDDING_MODEL,
                input=texts,
                encoding_format="float",
            )
            return [r.embedding for r in response.data]
        except openai.RateLimitError:
            time.sleep(2 ** attempt)
    raise RuntimeError("Embedding API failed after 3 retries")
```

```

def ingest_document(
    doc_text: str,
    doc_id: str,
    metadata: dict,
    pg_conn,
    pinecone_index,
) -> int:
    """
    Chunk, embed, and store a document.
    Returns the number of chunks created.
    """

    raw_chunks = splitter.split_text(doc_text)
    if not raw_chunks:
        return 0

    # Build chunk records with metadata
    chunk_records = []
    for i, chunk_text in enumerate(raw_chunks):
        chunk_records.append({
            "id": str(uuid.uuid4()),
            "doc_id": doc_id,
            "chunk_index": i,
            "text": chunk_text,
            "token_count": len(enc.encode(chunk_text)),
            "metadata": {**metadata, "chunk_index": i, "doc_id": doc_id},
        })

    # Embed in batches
    all_embeddings = []
    for batch_start in range(0, len(chunk_records), BATCH_SIZE):
        batch = chunk_records[batch_start : batch_start + BATCH_SIZE]
        texts = [r["text"] for r in batch]
        embeddings = embed_batch(texts)
        all_embeddings.extend(embeddings)

    # Assign embeddings back to records
    for record, embedding in zip(chunk_records, all_embeddings):
        record["embedding"] = embedding

    # --- pgvector upsert ---
    with pg_conn.cursor() as cur:
        cur.executemany(
            """
            INSERT INTO document_chunks
            (id, doc_id, chunk_index, content, token_count, metadata, embedding)
            VALUES
            (%s, %s, %s, %s, %s, %s::jsonb, %s::vector)
            ON CONFLICT (id) DO UPDATE SET
            content = EXCLUDED.content,
            embedding = EXCLUDED.embedding,
            metadata = EXCLUDED.metadata
            """,
            [
                (
                    r["id"],
                    r["doc_id"],
                    r["chunk_index"],
                    r["text"],
                    r["token_count"],
                    json.dumps(r["metadata"]),
                    r["embedding"],
                )
                for r in chunk_records
            ],
        )
    pg_conn.commit()

    # --- Pinecone upsert ---
    pinecone_vectors = [
        {

```

```

        "id": r["id"],
        "values": r["embedding"],
        "metadata": {
            **r["metadata"],
            "text": r["text"][:1000], # Pinecone metadata: 40KB limit per vector
        },
    }
    for r in chunk_records
]
# Pinecone recommends batches of 100 vectors
for batch_start in range(0, len(pinecone_vectors), 100):
    batch = pinecone_vectors[batch_start : batch_start + 100]
    pinecone_index.upsert(vectors=batch)

return len(chunk_records)

# --- Schema (run once) ---
SCHEMA_SQL = """
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS document_chunks (
    id          UUID PRIMARY KEY,
    doc_id      TEXT NOT NULL,
    chunk_index INTEGER NOT NULL,
    content     TEXT NOT NULL,
    token_count INTEGER,
    metadata    JSONB DEFAULT '{}',
    embedding   vector(1536),
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_chunks_doc_id
ON document_chunks(doc_id);

CREATE INDEX IF NOT EXISTS idx_chunks_metadata
ON document_chunks USING GIN(metadata);

CREATE INDEX IF NOT EXISTS idx_chunks_embedding
ON document_chunks USING hnsw(embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
"""

# Usage example (not executed in this snippet):
# pg_conn = psycopg2.connect(PG_DSN)
# index = pc.Index("knowledge-base")
# n = ingest_document(doc_text, "doc-001", {"source": "manual", "version": "2.1"}, pg_conn, index)
# print(f"Ingested {n} chunks")

```

The schema design here reflects DBA instincts: a GIN index on the `metadata` JSONB column enables fast filtering by source, version, access tier, or any dimension you attach at ingestion time. The HNSW index on the embedding column uses `m=16` and `ef_construction=64`, which are conservative defaults suitable for up to a few million chunks before you need to tune upward. The `ON CONFLICT` upsert pattern means you can re-run ingestion on the same document without creating duplicate chunks—critical for pipelines that re-index documents when source content changes.

Pinecone index setup follows the serverless pattern introduced in Chapter 12:

```

# Python
def ensure_pinecone_index(pc: Pinecone, index_name: str, dimension: int) -> object:
    """
    Creates a Pinecone serverless index if it does not exist.
    Returns the Index object ready for upsert/query.
    """
    existing = [idx.name for idx in pc.list_indexes()]
    if index_name not in existing:
        pc.create_index(

```

```

    name=index_name,
    dimension=dimension,
    metric="cosine",
    spec=ServerlessSpec(cloud="aws", region="us-east-1"),
)
# Wait for index to be ready
import time
while not pc.describe_index(index_name).status["ready"]:
    time.sleep(1)
return pc.Index(index_name)

```

One Pinecone-specific concern worth flagging: metadata stored alongside vectors is not free. Each vector’s metadata counts toward your index size, and the 40KB per-vector metadata limit is generous but not infinite. Storing the full chunk text in metadata is convenient for retrieval—you avoid a round-trip to PostgreSQL or another store—but for large corpora it meaningfully inflates pod or storage costs. A common production pattern is to store a truncated preview (200–300 characters) in Pinecone metadata for display purposes, and retrieve full text from PostgreSQL using the chunk ID when context assembly requires complete content.

22.6 Querying Across Chunk Strategies

A chunking strategy that looks good during ingestion must also hold up at query time. The query path exposes failure modes that are invisible until users start asking questions.

The boundary problem in retrieval: Suppose a user asks “What is the difference between session pooling and transaction pooling in PgBouncer?” Your document about PgBouncer has a section that spans 800 tokens, covering both modes. With 512-token fixed chunks and no overlap, one chunk covers session pooling and a second chunk covers transaction pooling. The query embeds to something close to both chunks, but the top-1 result might be only one of them. The LLM then generates an answer about session pooling without mentioning transaction pooling.

Two retrieval-side mitigations exist. First, retrieve top-K with K larger than you plan to pass to the LLM—fetch 20, pass 5. This increases the probability that both relevant chunks appear in the candidate set. Second, implement **parent-child chunking** (also called the “small-to-big” retrieval pattern): embed small chunks (128 tokens) for retrieval precision, but when a small chunk is retrieved, return its parent chunk (512 tokens) for context. LlamaIndex calls this `ParentDocumentRetriever`; LangChain implements it via `ParentDocumentRetriever` as well.

```

# Python
# Parent-child chunking: embed small chunks, retrieve large parent chunks

from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain.vectorstores import PGVector
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
import os

# Child splitter: small chunks for embedding precision
child_splitter = RecursiveCharacterTextSplitter(
    chunk_size=128,
    chunk_overlap=16,
    length_function=lambda s: len(enc.encode(s)),
)

# Parent splitter: larger chunks for context delivery
parent_splitter = RecursiveCharacterTextSplitter(

```

```

    chunk_size=512,
    chunk_overlap=0, # No overlap at parent level-parents are complete units
    length_function=lambda s: len(enc.encode(s)),
)

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small",
    openai_api_key=os.environ["OPENAI_API_KEY"],
)

# PGVector as the vector store for child chunk embeddings
vectorstore = PGVector(
    connection_string=os.environ["PG_DSN"],
    embedding_function=embeddings,
    collection_name="child_chunks",
)

# InMemoryStore holds parent chunks keyed by parent ID
# In production, replace with a Redis or PostgreSQL-backed store
docstore = InMemoryStore()

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=docstore,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

# Adding documents splits them into parents first, then children
# Children are embedded; parents are stored in the docstore
# from langchain.schema import Document as LCDocument
# docs = [LCDocument(page_content=text, metadata={"source": "pgbouncer_manual"})]
# retriever.add_documents(docs)

# At query time, similarity search finds child chunks,
# but the retriever returns the parent chunk for each match
# results = retriever.get_relevant_documents("transaction pooling vs session pooling")

```

The parent-child pattern is particularly effective for technical documentation where individual sentences are too narrow to be meaningful and full sections are too wide to embed accurately. The small chunk gives you embedding precision; the parent chunk gives you LLM-ready context.

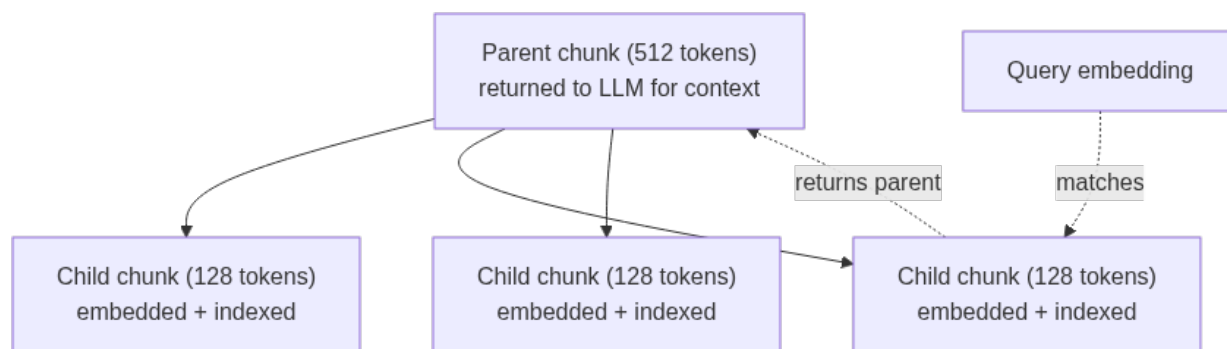


Figure 15: Retrieval searches against small child embeddings for precision, then returns the larger parent chunk the LLM actually reads — precise targeting, complete context.

Chunk-level filtering: The metadata columns you attach at ingestion become filter dimensions at query time. In pgvector, this is a standard SQL `WHERE` clause combined with the vector similarity operator:

```
-- PostgreSQL: retrieve chunks from a specific document version,
```

```

-- ranked by cosine similarity to a query embedding

SELECT
  id,
  doc_id,
  chunk_index,
  content,
  metadata,
  1 - (embedding <=> '[0.021, -0.047, ...]':vector) AS cosine_similarity
FROM document_chunks
WHERE
  metadata->>'version' = '2.1'
  AND metadata->>'doc_type' = 'manual'
  AND token_count BETWEEN 64 AND 512
ORDER BY embedding <=> '[0.021, -0.047, ...]':vector
LIMIT 10;

-- SQL Server equivalent using the vector similarity pattern
-- (assuming a computed cosine similarity UDF or OPENJSON metadata pattern)

SELECT TOP 10
  id,
  doc_id,
  chunk_index,
  content,
  JSON_VALUE(metadata, '$.version') AS version,
  JSON_VALUE(metadata, '$.doc_type') AS doc_type,
  dbo.fn_cosine_similarity(embedding, '[0.021, -0.047, ...]') AS cosine_similarity
FROM document_chunks
WHERE
  JSON_VALUE(metadata, '$.version') = '2.1'
  AND JSON_VALUE(metadata, '$.doc_type') = 'manual'
  AND token_count BETWEEN 64 AND 512
ORDER BY dbo.fn_cosine_similarity(embedding, '[0.021, -0.047, ...]') DESC;

```

Pre-filtering on metadata before the vector similarity scan is an important performance decision. In pgvector with an HNSW index, a filtered ANN search scans the HNSW graph and then applies the filter, which can be inefficient if the filter is highly selective. For high-selectivity filters (where the filter eliminates more than 90% of rows), an exact KNN scan (`SET enable_indexscan = off`) over the filtered subset often outperforms HNSW. Pinecone handles this natively through its metadata filtering layer, which routes queries to a subset of the index before ANN search.

22.7 Choosing a Chunking Strategy by Document Type

The correct chunking strategy depends on the document type, the query patterns it must serve, and the embedding model in use. The following table organizes the common cases:

Document Type	Recommended Strategy	Chunk Size	Overlap	Notes
Technical documentation (Mark-down/HTML)	Structural (headings) then fixed within sections	256–512 tokens	10–15%	Preserve heading metadata per chunk
Legal or regulatory text	Sentence-aware + overlap	256–384 tokens	15–20%	Sentence boundaries are semantic boundaries

Document Type	Recommended Strategy	Chunk Size	Overlap	Notes
Source code	Structural (function/class via tree-sitter)	Variable	0–5%	Never split mid-function
Conversational transcripts	Turn-based or fixed small	128–256 tokens	5–10%	Speaker turns are natural boundaries
Academic papers	Section- structural + semantic within sections	384–512 tokens	10%	Abstract and conclusions warrant separate chunks
FAQ / Q&A documents	One chunk per Q&A pair	Variable	0%	Natural atomic units; no splitting needed
Tabular data (CSV, spreadsheets)	Row-based or row-group	Variable	0%	Embed column headers in every chunk
Mixed media (PDF with figures)	Text extraction + structural	256–512 tokens	10%	Tag figure captions separately

Two principles cut across all document types:

1. **Preserve provenance in every chunk.** Every chunk should carry its source document ID, the heading or section it belongs to, and its position. A chunk retrieved without provenance cannot be cited, and citation is table stakes for enterprise RAG deployments.
2. **Match chunk granularity to query granularity.** If your users ask narrow factual questions (“What is the default `max_connections` in PostgreSQL 16?”), small precise chunks (128–256 tokens) will outperform large ones. If your users ask synthesis questions (“Summarize the tradeoffs between BRIN and B-Tree indexes”), larger chunks (512 tokens or more) serve better because the answer requires more surrounding context. When a single application serves both query types, the parent-child pattern described in Section 22.6 is the production-grade solution.

22.8 Measuring Chunking Quality

Chunking strategy cannot be evaluated in isolation—it must be measured through the full RAG pipeline. Three metrics are practical to implement:

Chunk hit rate: For a labeled evaluation set of (question, answer) pairs, what percentage of queries return at least one chunk containing the correct answer in the top-K results? A drop in hit rate after changing chunking parameters is a clear signal of regression.

Answer faithfulness: Using an LLM-as-judge pattern, score whether the generated answer is faithful to the retrieved chunks (no hallucination). If faithfulness drops, your chunks may be

too short to give the LLM enough context, or overlap may be insufficient to prevent boundary fragmentation.

Mean reciprocal rank (MRR): Among queries where the relevant chunk is retrieved, at what rank does it appear? An MRR of 0.85 means the relevant chunk typically appears within the first two positions; 0.5 means it appears around position two to three. MRR is sensitive to chunk granularity—too-coarse chunks often rank lower because they contain more irrelevant content alongside the answer.

```
# Python
# Lightweight chunking evaluation framework

from dataclasses import dataclass
from typing import Callable
import numpy as np

@dataclass
class EvalSample:
    question: str
    expected_answer_fragment: str # substring that should appear in a relevant chunk

def evaluate_chunking_strategy(
    eval_samples: list[EvalSample],
    retriever_fn: Callable[[str, int], list[dict]], # (query, top_k) -> list of chunk dicts
    top_k: int = 10,
) -> dict:
    """
    Evaluate a chunking + retrieval configuration against labeled samples.
    retriever_fn must return chunk dicts with at least a 'text' key.
    """
    hit_count = 0
    reciprocal_ranks = []

    for sample in eval_samples:
        results = retriever_fn(sample.question, top_k)
        hit = False
        for rank, chunk in enumerate(results, start=1):
            if sample.expected_answer_fragment.lower() in chunk["text"].lower():
                reciprocal_ranks.append(1.0 / rank)
                hit = True
                break
        if not hit:
            reciprocal_ranks.append(0.0)
        if hit:
            hit_count += 1

    return {
        "hit_rate": hit_count / len(eval_samples),
        "mrr": float(np.mean(reciprocal_ranks)),
        "sample_count": len(eval_samples),
    }

# Example evaluation samples for a PostgreSQL knowledge base
eval_set = [
    EvalSample(
        question="How does autovacuum decide when to run?",
        expected_answer_fragment="autovacuum_vacuum_threshold",
    ),
    EvalSample(
        question="What is the difference between BRIN and B-Tree indexes?",
        expected_answer_fragment="Block Range",
    ),
    EvalSample(
        question="How does MVCC handle concurrent reads and writes?",
        expected_answer_fragment="xmin",
    ),
]
```


]

```
# Wire up your retriever and run:
# metrics = evaluate_chunking_strategy(eval_set, my_retriever_fn, top_k=10)
# print(f"Hit Rate: {metrics['hit_rate']:.2%}, MRR: {metrics['mrr']:.3f}")
```

Running this evaluation across a grid of chunk sizes and overlap values—say, chunk sizes of [128, 256, 512] crossed with overlaps of [0%, 10%, 20%]—gives you nine data points that reveal the optimal configuration for your specific corpus and query distribution. The investment of building a 50–100 sample labeled set pays back quickly: chunking parameters that look similar on paper often diverge by 15–25 percentage points in hit rate depending on document structure and query patterns.

22.9 Operational Considerations: Re-Chunking and Schema Evolution

Documents change. New versions of technical manuals, updated policy documents, revised code—all of these require re-ingestion. Chunking strategy changes also require re-ingestion of the entire corpus. These are operational realities that deserve schema and pipeline design attention upfront.

Versioned chunk storage: Rather than overwriting chunks on re-ingestion, consider keeping a `chunk_version` column that increments with each ingestion run. This allows you to roll back to a previous chunking configuration if a new strategy degrades retrieval quality. The tradeoff is storage cost, which is manageable for most corpora but worth considering for multi-billion-token knowledge bases.

```
-- PostgreSQL: Schema with versioning support for chunk evolution

ALTER TABLE document_chunks
  ADD COLUMN IF NOT EXISTS ingestion_run_id UUID,
  ADD COLUMN IF NOT EXISTS chunk_strategy TEXT, -- e.g. 'fixed_512_64' or 'semantic_v2'
  ADD COLUMN IF NOT EXISTS is_active BOOLEAN DEFAULT TRUE;

-- When re-ingesting, mark old chunks inactive rather than deleting them:
UPDATE document_chunks
SET is_active = FALSE
WHERE doc_id = $1
  AND ingestion_run_id != $2;

-- Query filters on active chunks only:
CREATE INDEX IF NOT EXISTS idx_chunks_active
ON document_chunks(doc_id, is_active)
WHERE is_active = TRUE;

-- SQL Server equivalent

ALTER TABLE document_chunks
  ADD ingestion_run_id UNIQUEIDENTIFIER NULL,
  chunk_strategy NVARCHAR(100) NULL,
  is_active BIT NOT NULL DEFAULT 1;

-- Deactivate prior chunks on re-ingestion:
UPDATE document_chunks
SET is_active = 0
WHERE doc_id = @doc_id
  AND ingestion_run_id <> @new_run_id;

-- Filtered index on active chunks:
CREATE INDEX idx_chunks_active
```

```
ON document_chunks(doc_id, is_active)
WHERE is_active = 1;
```

Cascading re-embedding: When an embedding model is upgraded—from `text-embedding-3-small` to `text-embedding-3-large`, for instance—all stored embeddings become stale because the two models occupy different vector spaces. Cosine similarity between a new-model query embedding and an old-model chunk embedding is meaningless. This is analogous to changing a hash function in a relational database: every stored value must be recomputed.

The practical response is to store the model name and version alongside each chunk and treat model upgrades as full re-ingestion events. Pinecone’s approach is to create a new index for the upgraded model and switch query traffic to the new index only after full re-ingestion completes, using a feature flag or DNS-level cutover. In pgvector, you can add a second vector column for the new model and backfill it in batches before switching the query path:

```
-- PostgreSQL: Zero-downtime embedding model upgrade

-- Step 1: Add new vector column (existing queries still use embedding_v1)
ALTER TABLE document_chunks
  ADD COLUMN embedding_v2 vector(3072); -- text-embedding-3-large dimension

-- Step 2: Backfill in batches (run as a background job)
-- UPDATE document_chunks SET embedding_v2 = $1 WHERE id = $2;

-- Step 3: Build HNSW index on new column
CREATE INDEX CONCURRENTLY idx_chunks_embedding_v2
  ON document_chunks USING hnsw(embedding_v2 vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Step 4: Switch application queries to embedding_v2 via config flag
-- Step 5: Drop old column after validation period
-- ALTER TABLE document_chunks DROP COLUMN embedding;
-- ALTER TABLE document_chunks RENAME COLUMN embedding_v2 TO embedding;
```

The `CREATE INDEX CONCURRENTLY` syntax in PostgreSQL builds the index without blocking reads or writes, which is the correct approach for production tables that serve live query traffic during a migration. SQL Server’s equivalent is `CREATE INDEX ... WITH (ONLINE = ON)`.

Key Takeaways

- **Chunking is a normalization decision:** chunk granularity governs retrieval precision the same way index selectivity governs query performance in relational databases. Too coarse and embeddings are noisy; too fine and embeddings become brittle against phrasing variation.
- **Token-accurate sizing matters:** use the tokenizer that matches your embedding model (`tiktoken` for OpenAI models, the model’s native tokenizer otherwise) rather than character counts. Mismatched sizing causes silent truncation that degrades embedding quality without surfacing an error.
- **Overlap helps precision, not recall:** overlapping tokens at chunk boundaries ensures that semantically complete sentences appear in at least one chunk, reducing boundary fragmentation at retrieval time. Keep overlap between 10–20%; beyond 25%, adjacent chunks become near-duplicates that pollute top-K results.
- **Match strategy to document structure:** fixed-size chunking is fast and predictable; structural splitting (on headings, functions, or sections) outperforms it for well-marked documents; semantic splitting finds topic boundaries in unstructured prose at the cost of extra embed-

ding calls during ingestion. The production pattern for most technical corpora is structural splitting first, semantic splitting within oversized sections.

- **Design for re-ingestion from the start:** chunk versioning, inactive-flag patterns, and dual-column embedding migrations are operational necessities for any corpus that will change over time or migrate to an improved embedding model. Treating chunks as immutable blobs rather than versioned rows creates painful re-indexing events that proper schema design avoids.
-

Chapter 23: Embedding Model Selection: OpenAI, Cohere, Sentence Transformers

The embedding model you choose is one of the highest-leverage architectural decisions in a vector database system—roughly equivalent to choosing a data type for a primary key column. Get it wrong and you face cascading problems: mismatched dimensions between your stored vectors and your query vectors, poor retrieval quality on domain-specific content, unnecessary API costs at query time, or latency profiles that make your application unusable. This chapter walks through the three major embedding ecosystems—OpenAI, Cohere, and Sentence Transformers—examining their output dimensions, quality characteristics, cost structures, and operational tradeoffs. You will leave with a framework for matching model to workload, and production-ready code for each provider integrated against pgvector, Pinecone, and Weaviate.

What an Embedding Model Actually Does (and Why It Matters to a DBA)

An embedding model is a function: it takes text as input and returns a fixed-length vector of floating-point numbers. That vector is a compressed semantic fingerprint—documents with similar meaning produce vectors that are geometrically close in high-dimensional space. When you run a vector similarity search, you are asking: “which stored vectors are nearest to my query vector?”

This framing immediately surfaces the DBA’s core concern: **schema compatibility**. Every embedding model produces a specific vector dimension. OpenAI’s `text-embedding-3-small` outputs 1536 dimensions by default. Cohere’s `embed-english-v3.0` outputs 1024. A `sentence-transformers/all-MiniLM-L6-v2` model outputs 384. If you store embeddings generated by one model and query with a different one—even a different version of the same model—the similarity scores become meaningless. This is not a soft warning; it is the vector database equivalent of a type mismatch on a join key.

Think of model selection like choosing between `TEXT`, `VARCHAR(255)`, and `BYTEA`. All three store string-like data, but the right choice depends on access patterns, storage constraints, and how downstream consumers will process the data. Similarly, embedding model choice must account for:

Concern	DBA Analogy
Vector dimension	Column data type / storage width
Model versioning	Schema migration discipline
Hosted vs. local	Managed service vs. self-hosted database
Batch vs. real-time	Bulk insert vs. OLTP insert pattern
Domain specificity	Partial index on a specialized query pattern

One practical implication DBAs often miss: **you must re-embed your entire corpus whenever you change models**. There is no ALTER TABLE equivalent that converts 1536-dimension vectors to 1024-dimension vectors. Plan for this from day one. In production systems with millions of rows, a model migration is a multi-hour or multi-day batch operation requiring careful coordination between the old and new vector columns during the cutover window.

OpenAI Embeddings: Production Workhorse

OpenAI offers two current-generation embedding models that have become the de facto baseline for most commercial RAG applications:

- **text-embedding-3-small**: 1536 dimensions, ~\$0.02 per million tokens
- **text-embedding-3-large**: 3072 dimensions, ~\$0.13 per million tokens

Both support **dimension reduction** via the `dimensions` parameter—you can request fewer dimensions than the maximum, and the model truncates the vector in a way that preserves most semantic quality. This is a meaningful operational feature: you can store 256-dimension vectors from **text-embedding-3-large** and still outperform **text-embedding-3-small** at full dimensionality on many benchmarks, while cutting storage by 83%.

The older **text-embedding-ada-002** (1536 dimensions) still appears in legacy deployments but should not be chosen for new projects. It performs worse than both **text-embedding-3-*** models on MTEB benchmarks and costs the same as **text-embedding-3-small**.

Rate limits and batching behavior matter significantly at scale. OpenAI processes embedding requests in batches of up to 2048 inputs per API call. Each input is limited to 8192 tokens. For a DBA loading a 10-million-row document table, you need to batch your inserts carefully to avoid hitting per-minute token limits and to maximize throughput.

```
# Python
import openai
import psycpg2
import numpy as np
from typing import List

client = openai.OpenAI() # Uses OPENAI_API_KEY from environment

def embed_batch_openai(texts: List[str], model: str = "text-embedding-3-small", dimensions: int = 1536)
    -> List[List[float]]:
    """
    Embed a batch of texts using OpenAI. Returns list of float vectors.
    Max batch size: 2048 inputs. Max tokens per input: 8192.
    """
    response = client.embeddings.create(
        model=model,
```

```

        input=texts,
        dimensions=dimensions # Supported on text-embedding-3-* only
    )
    # Response items are ordered to match input order
    return [item.embedding for item in response.data]

def load_documents_to_pgvector(
    documents: List[dict], # Each dict: {"id": int, "content": str}
    conn_string: str,
    model: str = "text-embedding-3-small",
    dimensions: int = 1536,
    batch_size: int = 500 # Well under the 2048 API limit; controls DB insert size
):
    """
    Batch-embed documents and store in PostgreSQL with pgvector.
    Assumes table: documents(id BIGINT, content TEXT, embedding VECTOR(1536))
    """
    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()

    for i in range(0, len(documents), batch_size):
        batch = documents[i : i + batch_size]
        texts = [doc["content"] for doc in batch]
        ids = [doc["id"] for doc in batch]

        embeddings = embed_batch_openai(texts, model=model, dimensions=dimensions)

        records = [
            (ids[j], texts[j], embeddings[j])
            for j in range(len(batch))
        ]

        cur.executemany(
            "INSERT INTO documents (id, content, embedding) VALUES (%s, %s, %s::vector) "
            "ON CONFLICT (id) DO UPDATE SET content = EXCLUDED.content, embedding = EXCLUDED.embedding",
            records
        )
        conn.commit()
        print(f"Loaded rows {i} to {i + len(batch) - 1}")

    cur.close()
    conn.close()

# Similarity search using the same model at query time
def search_documents(query: str, conn_string: str, top_k: int = 5, model: str =
    ↪ "text-embedding-3-small"):
    query_embedding = embed_batch_openai([query], model=model)[0]

    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()
    cur.execute(
        """
        SELECT id, content, 1 - (embedding <=> %s::vector) AS cosine_similarity
        FROM documents
        ORDER BY embedding <=> %s::vector
        LIMIT %s
        """,
        (query_embedding, query_embedding, top_k)
    )
    results = cur.fetchall()
    cur.close()
    conn.close()
    return results

```

The pgvector schema for this setup:

```

-- PostgreSQL / pgvector
-- Create table with matching dimension

```

```
CREATE TABLE documents (  
  id          BIGINT PRIMARY KEY,  
  content     TEXT NOT NULL,  
  embedding   VECTOR(1536),           -- Must match model output dimension  
  model_name  TEXT DEFAULT 'text-embedding-3-small', -- Track which model generated this  
  embedded_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- HNSW index for ANN search (preferred over IVFFlat for most workloads)  
CREATE INDEX idx_documents_embedding_hnsw  
  ON documents  
  USING hnsw (embedding vector_cosine_ops)  
  WITH (m = 16, ef_construction = 64);  
  
-- If you reduce dimensions to 256, alter and rebuild:  
-- ALTER TABLE documents ALTER COLUMN embedding TYPE VECTOR(256);  
-- REINDEX INDEX idx_documents_embedding_hnsw;
```

Azure OpenAI users can substitute the endpoint with minimal code changes:

```
# Python  
from openai import AzureOpenAI  
  
azure_client = AzureOpenAI(  
  azure_endpoint="https://YOUR_RESOURCE.openai.azure.com/",  
  api_key="YOUR_AZURE_API_KEY",  
  api_version="2024-02-01"  
)  
  
response = azure_client.embeddings.create(  
  model="text-embedding-3-small", # Azure deployment name  
  input=["The quick brown fox"],  
  dimensions=1536  
)  
embedding = response.data[0].embedding
```

The key operational discipline with OpenAI embeddings: treat the model identifier as part of your schema definition. Store `model_name` and `dimensions` alongside each vector row. When OpenAI releases `text-embedding-4-*`, you will want to identify exactly which rows were generated by which model version before deciding whether to migrate.

Cohere Embeddings: Retrieval-Optimized with Input Type Awareness

Cohere's embedding models introduce a concept that OpenAI's API does not expose directly: **input type separation**. Cohere lets you specify whether each embedding request is a `search_document` or a `search_query`. Under the hood, the model applies different transformations for indexing versus querying, which meaningfully improves retrieval precision—particularly for asymmetric retrieval tasks where the query is short (a user question) but the documents are long (technical paragraphs).

Current production models:

- **embed-english-v3.0**: 1024 dimensions, English-only, strongest English retrieval
- **embed-multilingual-v3.0**: 1024 dimensions, 100+ languages, competitive multilingual quality
- **embed-english-light-v3.0**: 384 dimensions, English-only, optimized for low-latency

Cohere also supports **binary and int8 quantization** natively—you can request compressed em-

beddings directly from the API, reducing storage by 32x (binary) or 4x (int8) at the cost of some precision. For DBAs managing multi-terabyte vector stores, this is a first-class feature rather than an afterthought.

```
# Python
import cohere
import pinecone
from pinecone import Pinecone, ServerlessSpec
import time

co = cohere.Client("YOUR_COHERE_API_KEY")
pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")

# Initialize Pinecone index - dimension must match Cohere embed-english-v3.0 output
INDEX_NAME = "cohere-docs-1024"

if INDEX_NAME not in [i.name for i in pc.list_indexes()]:
    pc.create_index(
        name=INDEX_NAME,
        dimension=1024,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )
    # Wait for index to be ready
    while not pc.describe_index(INDEX_NAME).status["ready"]:
        time.sleep(1)

index = pc.Index(INDEX_NAME)

def embed_documents_cohere(texts: List[str]) -> List[List[float]]:
    """
    Embed documents for indexing. Use input_type='search_document'.
    Cohere batch limit: 96 texts per call. Max 512 tokens per text.
    """
    response = co.embed(
        texts=texts,
        model="embed-english-v3.0",
        input_type="search_document",
        embedding_types=["float"]
    )
    return response.embeddings.float_

def embed_query_cohere(query: str) -> List[float]:
    """
    Embed a search query. Use input_type='search_query'.
    This is a different transformation than search_document.
    """
    response = co.embed(
        texts=[query],
        model="embed-english-v3.0",
        input_type="search_query",
        embedding_types=["float"]
    )
    return response.embeddings.float_[0]

def upsert_to_pinecone(documents: List[dict], batch_size: int = 96):
    """
    documents: [{"id": "doc_001", "content": "...", "metadata": {...}}]
    """
    for i in range(0, len(documents), batch_size):
        batch = documents[i : i + batch_size]
        texts = [doc["content"] for doc in batch]
        embeddings = embed_documents_cohere(texts)

        vectors = [
            {
```

```
        "id": batch[j]["id"],
        "values": embeddings[j],
        "metadata": {
            "content": batch[j]["content"],
            **batch[j].get("metadata", {})
        }
    }
    for j in range(len(batch))
]
index.upsert(vectors=vectors)
print(f"Upserted {len(vectors)} vectors to Pinecone")

def search_pinecone(query: str, top_k: int = 5, filter_dict: dict = None):
    query_embedding = embed_query_cohere(query)
    results = index.query(
        vector=query_embedding,
        top_k=top_k,
        include_metadata=True,
        filter=filter_dict  # Pinecone metadata filter, e.g. {"department": "engineering"}
    )
    return results.matches
```

The `input_type` distinction is easy to forget and expensive to debug. If you embed all your documents with `search_document` during batch loading and then embed queries with `search_document` instead of `search_query` at runtime, retrieval quality degrades measurably—typically 5–15% drop in recall@10 on technical content—and the root cause is invisible from SQL query plans or vector search metrics. Build a wrapper layer that enforces input type by function signature, as shown above.

Binary quantization example for high-scale storage:

```
# Python
def embed_documents_cohere_binary(texts: List[str]) -> List[List[int]]:
    """
    Binary embeddings reduce 1024-dimension float32 (4KB per vector)
    to 128-byte binary vectors - 32x storage reduction.
    Recall degradation: typically 5-10% vs float32 at equivalent index quality.
    """
    response = co.embed(
        texts=texts,
        model="embed-english-v3.0",
        input_type="search_document",
        embedding_types=["ubinary"]  # Unsigned binary: each dim becomes 0 or 1
    )
    return response.embeddings.ubinary_  # Returns list of packed byte arrays
```

Cohere’s pricing model (per-token rather than per-call) and multilingual support make it the strongest commercial choice for non-English corpora. If your application serves users in multiple languages, `embed-multilingual-v3.0` avoids the need to maintain separate indexes per language.

Sentence Transformers: Local Embeddings for Privacy and Cost Control

Sentence Transformers is a Python library built on HuggingFace Transformers that gives you access to hundreds of pre-trained embedding models you can run entirely on your own infrastructure. No API calls, no per-token billing, no data leaving your network. For DBAs managing regulated data—healthcare, financial, government—this is often not optional.

The quality range is wide. The smallest usable model (`all-MiniLM-L6-v2`, 384 dims) runs on CPU at roughly 2,000 sentences/second on a modern 16-core server. The strongest general-purpose model (`all-mpnet-base-v2`, 768 dims) runs at about 400 sentences/second on the same hardware. With a GPU (A10G or better), throughput scales 20–50x depending on batch size.

Popular models and their characteristics:

Model	Dimensions	Speed (CPU)	MTEB Score	Best Use
<code>all-MiniLM-L6-v2</code>	384	~2000/s	56.3	High-throughput, latency-sensitive
<code>all-mpnet-base-v2</code>	768	~400/s	57.8	General purpose RAG
<code>multi-qa-mpnet-base-dot-v1</code>	768	~400/s	57.6	Q&A retrieval tasks
<code>BAAI/bge-large-en-v1.5</code>	1024	~150/s	63.9	High-quality English retrieval
<code>intfloat/e5-large-v2</code>	1024	~150/s	62.2	Instruction-following retrieval

`BAAI/bge-large-en-v1.5` is currently one of the strongest open-source English models, competitive with commercial providers on many MTEB tasks. If you have GPU infrastructure, it should be your default open-source choice.

```
# Python
from sentence_transformers import SentenceTransformer
import weaviate
from weaviate.classes.init import Auth
from weaviate.classes.config import Configure, Property, DataType, VectorDistances
import numpy as np

# Load model once - keep in memory across requests
# Device auto-detects CUDA if available, falls back to CPU
model = SentenceTransformer("BAAI/bge-large-en-v1.5", device="cuda")

# BGE models use instruction prefix for queries (not documents)
# This is a model-specific convention - read the model card
QUERY_PREFIX = "Represent this sentence for searching relevant passages: "

def embed_documents_local(texts: List[str], batch_size: int = 64) -> np.ndarray:
    """
    Embed documents for indexing. No prefix needed for BGE document embeddings.
    Returns numpy array of shape (len(texts), 1024).
    """
    embeddings = model.encode(
        texts,
        batch_size=batch_size,
        normalize_embeddings=True, # Required for cosine similarity via dot product
        show_progress_bar=True
    )
    return embeddings # numpy float32 array

def embed_query_local(query: str) -> np.ndarray:
    """
    BGE query embedding requires instruction prefix.
    Other models (MiniLM, mpnet) do NOT use prefixes.
    """
    prefixed_query = QUERY_PREFIX + query
```

```
embedding = model.encode(
    [prefixed_query],
    normalize_embeddings=True
)
return embedding[0]

# Connect to Weaviate and create a collection
weaviate_client = weaviate.connect_to_weaviate_cloud(
    cluster_url="YOUR_WEAVIDATE_CLOUD_URL",
    auth_credentials=Auth.api_key("YOUR_WEAVIDATE_API_KEY")
)

COLLECTION_NAME = "TechDocuments"

# Create collection - using "none" vectorizer since we supply our own vectors
if not weaviate_client.collections.exists(COLLECTION_NAME):
    weaviate_client.collections.create(
        name=COLLECTION_NAME,
        vectorizer_config=Configure.Vectorizer.none(), # We provide vectors manually
        vector_index_config=Configure.VectorIndex.hnsw(
            distance_metric=VectorDistances.COSINE,
            ef_construction=128,
            max_connections=16
        ),
        properties=[
            Property(name="doc_id", data_type=DataType.TEXT),
            Property(name="content", data_type=DataType.TEXT),
            Property(name="source", data_type=DataType.TEXT),
        ]
    )

collection = weaviate_client.collections.get(COLLECTION_NAME)

def load_documents_to_weaviate(documents: List[dict], batch_size: int = 64):
    """
    documents: [{"doc_id": "...", "content": "...", "source": "..."}]
    """
    texts = [doc["content"] for doc in documents]
    embeddings = embed_documents_local(texts, batch_size=batch_size)

    with collection.batch.fixed_size(batch_size=200) as batch:
        for i, doc in enumerate(documents):
            batch.add_object(
                properties={
                    "doc_id": doc["doc_id"],
                    "content": doc["content"],
                    "source": doc.get("source", "")
                },
                vector=embeddings[i].tolist()
            )

    failed = collection.batch.failed_objects
    if failed:
        print(f"Warning: {len(failed)} objects failed to insert")

def search_weaviate(query: str, top_k: int = 5):
    from weaviate.classes.query import MetadataQuery

    query_vector = embed_query_local(query).tolist()
    results = collection.query.near_vector(
        near_vector=query_vector,
        limit=top_k,
        return_metadata=MetadataQuery(distance=True, score=True),
        return_properties=["doc_id", "content", "source"]
    )
    return results.objects
```

Model loading strategy matters for production API servers. Loading a 1.3GB model (bge-large) takes 3–8 seconds and consumes significant RAM. Load it once at application startup—not per request—and share it across worker threads with appropriate locking if your framework requires it. For Python web frameworks (FastAPI, Flask), use a module-level singleton.

```
# Python
# Singleton pattern for embedding model in a FastAPI service
from fastapi import FastAPI
from contextlib import asynccontextmanager
import threading

_model = None
_model_lock = threading.Lock()

@asynccontextmanager
async def lifespan(app: FastAPI):
    global _model
    _model = SentenceTransformer("BAAI/bge-large-en-v1.5", device="cuda")
    _model.max_seq_length = 512 # Clip long inputs to prevent OOM
    print(f"Model loaded. Max sequence length: {_model.max_seq_length}")
    yield
    # Cleanup on shutdown (optional for GPU memory release)
    del _model

app = FastAPI(lifespan=lifespan)

@app.post("/embed")
def embed_text(payload: dict):
    query = payload["text"]
    with _model_lock:
        vector = _model.encode([query], normalize_embeddings=True)[0]
    return {"embedding": vector.tolist(), "dimension": len(vector)}
```

Comparing Models: Quality, Cost, and Latency at Production Scale

No single embedding model wins across all dimensions. The right selection depends on your specific retrieval task, throughput requirements, data sensitivity, and budget. Here is how the comparison maps out across the factors that matter in production:

Retrieval quality (MTEB Benchmark, English)

BAAI/bge-large-en-v1.5 and OpenAI text-embedding-3-large are roughly equivalent at the top of the open leaderboard, both scoring in the 62–64 range on MTEB average. text-embedding-3-small and Cohere embed-english-v3.0 cluster around 61–62. MiniLM sits at 56–57. For most production RAG use cases, the difference between top-tier models is 3–5% in recall@10—meaningful but not the dominant quality driver. Chunking strategy and retrieval reranking typically have larger effects.

Latency

API-hosted models (OpenAI, Cohere) have network round-trip latency of 80–300ms for single requests, regardless of model quality. Local models served on the same machine as your application eliminate network latency entirely: all-MiniLM-L6-v2 on CPU returns in under 5ms for a single query; bge-large on a mid-tier GPU returns in under 10ms. For applications where query latency is a hard constraint (real-time search, autocomplete suggestions), local inference consistently wins.

Throughput at batch load time

OpenAI's rate limits can throttle large-scale indexing jobs. A 10-million-document corpus at 500

tokens per document averages 5 billion tokens, which at the highest tier limits can still require multiple hours and careful retry logic. Cohere’s batch endpoint has a 96-text-per-call limit, which becomes a throughput ceiling if your pipeline is not parallelized. A local GPU cluster has no such externally imposed ceiling—throughput scales with hardware.

Cost model comparison (qualitative, not to be treated as current pricing):

Model	Cost structure	Relative cost at 100M tokens
text-embedding-3-small	Per token, API	Low
text-embedding-3-large	Per token, API	Moderate–High
Cohere embed-english-v3.0	Per token, API	Low–Moderate
bge-large-en-v1.5	Infrastructure only	Near-zero marginal cost after hardware
all-MiniLM-L6-v2	Infrastructure only	Near-zero marginal cost after hardware

The crossover point where local inference becomes cheaper than API calls depends on your existing infrastructure. If you already operate GPU servers for model inference, adding an embedding endpoint has near-zero marginal cost. If you would need to provision dedicated GPU instances, the economics shift—particularly for low-volume applications where you would be paying for idle GPU capacity.

Storage implications by model

```
-- PostgreSQL / pgvector
-- Actual storage per row at various dimensions (float32, no TOAST compression)
-- VECTOR(n) stores 4n bytes of float32 data plus a small header

-- 384 dimensions ~ 1.5 KB per vector (all-MiniLM-L6-v2)
-- 768 dimensions ~ 3.1 KB per vector (all-mpnet-base-v2)
-- 1024 dimensions ~ 4.1 KB per vector (bge-large, Cohere embed-v3)
-- 1536 dimensions ~ 6.1 KB per vector (text-embedding-3-small default)
-- 3072 dimensions ~ 12.3 KB per vector (text-embedding-3-large default)

-- 10 million rows × 1536 dimensions 61 GB of raw vector data
-- 10 million rows × 384 dimensions 15 GB of raw vector data

-- Query to estimate current vector storage in a pgvector table:
SELECT
    relname AS table_name,
    pg_size_pretty(pg_relation_size(oid)) AS table_size,
    pg_size_pretty(pg_indexes_size(oid)) AS index_size,
    pg_size_pretty(pg_total_relation_size(oid)) AS total_size
FROM pg_class
WHERE relname = 'documents';
```

For very large corpora, choosing `text-embedding-3-large` at 3072 dimensions without applying dimension reduction is often a mistake. Reducing to 512 or 1024 dimensions via the `dimensions` parameter costs almost nothing in retrieval quality on general-purpose tasks while cutting storage and index memory by 3–6x.

Model Versioning and Migration Strategy

Model versioning is the problem DBAs most consistently underestimate when adopting vector search. Unlike a database engine upgrade, a model change is not transparent—it breaks all existing similarity comparisons.

The minimal production requirement: **always store the model name and version with every embedded row**. This column costs essentially nothing and saves enormous debugging effort when a model is deprecated or upgraded.

```
-- PostgreSQL / pgvector
-- Extended schema with model provenance tracking
CREATE TABLE documents (
  id          BIGSERIAL PRIMARY KEY,
  content     TEXT NOT NULL,
  content_hash BYTEA GENERATED ALWAYS AS (sha256(content::bytea)) STORED,
  embedding   VECTOR(1024),
  model_name  TEXT NOT NULL,          -- e.g. 'BAAI/bge-large-en-v1.5'
  model_version TEXT,                 -- e.g. 'v1.5', or git SHA for local models
  dimensions  INTEGER NOT NULL,      -- Redundant but explicit; aids validation
  embedded_at TIMESTAMPTZ DEFAULT NOW(),
  source_url  TEXT,
  metadata    JSONB
);

-- Index to find rows needing re-embedding after a model migration
CREATE INDEX idx_documents_model_name ON documents (model_name);

-- View showing embedding coverage by model
CREATE VIEW embedding_model_coverage AS
SELECT
  model_name,
  dimensions,
  COUNT(*)           AS row_count,
  MIN(embedded_at)   AS oldest_embedding,
  MAX(embedded_at)   AS newest_embedding
FROM documents
GROUP BY model_name, dimensions
ORDER BY row_count DESC;
```

Zero-downtime migration pattern: when you need to switch models, add a second vector column for the new model and backfill it while the old index continues serving queries. Once backfill completes and the new index is ready, switch query routing to the new column, validate, and drop the old column.

```
-- PostgreSQL / pgvector
-- Step 1: Add new column for updated model (1536 → 1024 dimension change example)
ALTER TABLE documents ADD COLUMN embedding_v2 VECTOR(1024);

-- Step 2: (backfill happens via application code during off-peak hours)
-- SELECT id, content FROM documents WHERE embedding_v2 IS NULL LIMIT 1000;
-- ... embed in application ...
-- UPDATE documents SET embedding_v2 = %s, model_name_v2 = 'BAAI/bge-large-en-v1.5' WHERE id = %s;

-- Step 3: Build new index on v2 column (can happen before backfill completes)
CREATE INDEX idx_documents_embedding_v2_hnsw
ON documents
USING hnsw (embedding_v2 vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Step 4: Verify coverage before switching
SELECT
  COUNT(*) FILTER (WHERE embedding IS NOT NULL) AS v1_count,
  COUNT(*) FILTER (WHERE embedding_v2 IS NOT NULL) AS v2_count,
  COUNT(*) AS total_rows
FROM documents;
```

```
-- Step 5: After query routing is switched, drop old column and index
-- DROP INDEX idx_documents_embedding_hnsw;
-- ALTER TABLE documents DROP COLUMN embedding;
-- ALTER TABLE documents RENAME COLUMN embedding_v2 TO embedding;
```

For Pinecone, there is no ALTER INDEX equivalent. The migration pattern requires creating a new index with the new dimension, populating it in parallel, updating your application to query the new index, and then deleting the old one. Weaviate requires a new collection with the updated vector dimension.

Choosing the Right Model: A Decision Framework

Work through these questions in order:

1. Is your data regulated or sensitive?

If yes—healthcare, financial records, legal documents, PII—you may not be able to use external API providers. Go to Sentence Transformers with local inference and stop evaluating hosted options.

2. Is your content primarily non-English?

Cohere `embed-multilingual-v3.0` or a multilingual Sentence Transformer (`paraphrase-multilingual-mpnet-base-v2`) are far stronger than English-only models on non-English retrieval. Do not use `text-embedding-3-small` for a primarily Japanese or Arabic corpus.

3. What is your query pattern: symmetric or asymmetric?

Asymmetric retrieval means short queries against long documents (most RAG applications). Cohere’s `input_type` separation and BGE’s instruction prefix both optimize for asymmetric retrieval. OpenAI handles both reasonably well without explicit asymmetry handling.

4. What are your throughput and latency requirements?

If you need sub-20ms embedding latency at query time and have more than a few thousand queries per hour, local inference on GPU is the only reliable option. API round-trip latency is bounded below by network physics.

5. What is your corpus size and re-embedding budget?

For corpora under a million documents, API costs are usually modest and the operational simplicity of not managing inference infrastructure is worth the per-token cost. For corpora over 10 million documents, the ongoing re-embedding cost (when you change models or need to refresh stale embeddings) becomes a real budget line item.

6. Do you need binary or int8 quantization for storage cost control?

If yes, Cohere provides native quantized outputs. Sentence Transformers can post-process float32 outputs into quantized representations. `pgvector` 0.7+ supports `halfvec` (16-bit) natively; full binary quantization support is available in the `bit` type or through custom handling.

A practical default recommendation for most teams:

- **Starting out, general English RAG:** `text-embedding-3-small` at 1536 dimensions. Hosted, well-documented, reliable, cost-effective.
- **Scaling up, cost-sensitive:** Migrate to `BAAI/bge-large-en-v1.5` locally once query volume justifies the infrastructure.

- **Multilingual:** embed-multilingual-v3.0 via Cohere or intfloat/multilingual-e5-large locally.
- **Regulated data, on-prem:** Sentence Transformers with bge-large-en-v1.5 from day one.

End-to-End Example: Evaluating Two Models Side by Side

Before committing to a model in production, run a side-by-side evaluation on a representative sample of your own data. MTEB scores are useful prior information, but domain-specific retrieval quality can diverge significantly from general benchmarks. Here is a minimal evaluation harness:

```
# Python
from sentence_transformers import SentenceTransformer
from sentence_transformers.evaluation import InformationRetrievalEvaluator
import openai
import numpy as np
from typing import Dict, List

# Evaluation requires:
# - A set of queries (questions your users will actually ask)
# - A corpus of candidate documents
# - Relevance labels: which corpus documents are relevant to each query

def build_openai_embeddings(
    texts: List[str],
    model: str = "text-embedding-3-small"
) -> np.ndarray:
    client = openai.OpenAI()
    all_embeddings = []
    batch_size = 500
    for i in range(0, len(texts), batch_size):
        batch = texts[i : i + batch_size]
        response = client.embeddings.create(model=model, input=batch)
        all_embeddings.extend([item.embedding for item in response.data])
    return np.array(all_embeddings, dtype=np.float32)

def cosine_similarity_matrix(queries: np.ndarray, corpus: np.ndarray) -> np.ndarray:
    """Compute pairwise cosine similarity between query and corpus embeddings."""
    # Assumes vectors are already L2-normalized
    return queries @ corpus.T # Shape: (num_queries, num_corpus_docs)

def recall_at_k(
    similarity_matrix: np.ndarray,
    relevance_labels: Dict[int, List[int]], # {query_idx: [relevant_corpus_idxs]}
    k: int = 10
) -> float:
    """Compute Recall@K across all queries."""
    recalls = []
    for query_idx, relevant_idxs in relevance_labels.items():
        sim_scores = similarity_matrix[query_idx]
        top_k_idxs = np.argsort(-sim_scores)[:k]
        relevant_retrieved = len(set(top_k_idxs) & set(relevant_idxs))
        recalls.append(relevant_retrieved / len(relevant_idxs))
    return float(np.mean(recalls))

def evaluate_models_side_by_side(
    queries: List[str],
    corpus: List[str],
    relevance_labels: Dict[int, List[int]],
    k_values: List[int] = [1, 5, 10]
):
```

```

results = {}

# Evaluate OpenAI text-embedding-3-small
print("Embedding with OpenAI text-embedding-3-small...")
oai_query_embs = build_openai_embeddings(queries, "text-embedding-3-small")
oai_corpus_embs = build_openai_embeddings(corpus, "text-embedding-3-small")
# Normalize for cosine similarity
oai_query_embs /= np.linalg.norm(oai_query_embs, axis=1, keepdims=True)
oai_corpus_embs /= np.linalg.norm(oai_corpus_embs, axis=1, keepdims=True)
oai_sim = cosine_similarity_matrix(oai_query_embs, oai_corpus_embs)
results["openai-3-small"] = {
    f"recall@{k}": recall_at_k(oai_sim, relevance_labels, k)
    for k in k_values
}

# Evaluate BAAI/bge-large-en-v1.5
print("Embedding with bge-large-en-v1.5...")
bge_model = SentenceTransformer("BAAI/bge-large-en-v1.5")
QUERY_PREFIX = "Represent this sentence for searching relevant passages: "
prefixed_queries = [QUERY_PREFIX + q for q in queries]
bge_query_embs = bge_model.encode(prefixed_queries, normalize_embeddings=True)
bge_corpus_embs = bge_model.encode(corpus, normalize_embeddings=True)
bge_sim = cosine_similarity_matrix(bge_query_embs, bge_corpus_embs)
results["bge-large-en-v1.5"] = {
    f"recall@{k}": recall_at_k(bge_sim, relevance_labels, k)
    for k in k_values
}

# Print comparison table
print(f"\n{'Model':<25} {'Recall@1':>10} {'Recall@5':>10} {'Recall@10':>10}")
print("-" * 57)
for model_name, metrics in results.items():
    print(
        f"{'model_name':<25} "
        f"{'metrics['recall@1']':>10.3f} "
        f"{'metrics['recall@5']':>10.3f} "
        f"{'metrics['recall@10']':>10.3f}"
    )
return results

# Example usage - you supply your own queries, corpus, and labels
# evaluate_models_side_by_side(
#     queries=["How do I configure SSL certificates?", "What is the retention policy?"],
#     corpus=["SSL configuration guide...", "Data retention policy document...", "Unrelated doc..."],
#     relevance_labels={0: [0], 1: [1]},
#     k_values=[1, 5, 10]
# )

```

Run this evaluation on at least 100–200 query/document pairs from your own domain before finalizing your model selection. MTEB rankings frequently reverse when applied to narrow domain content. A model that leads the general benchmark by three points may trail by five on your specific legal or scientific corpus.

Key Takeaways

- **Model selection defines your schema:** the embedding dimension is a structural constraint equivalent to a column data type. Every downstream index, stored vector, and similarity query must use vectors from the same model and dimension setting, with no conversion path between model versions.
- **OpenAI provides the lowest-friction starting point** for English-language RAG applica-

tions: reliable APIs, strong documentation, and competitive quality. Apply the `dimensions` parameter to reduce storage for large corpora without a meaningful quality penalty on general-purpose tasks.

- **Cohere’s `input_type` separation is a genuine retrieval quality improvement** for asymmetric search (short user queries against long documents). The binary and int8 quantization options directly address storage costs at scale, and the multilingual model is the strongest hosted option for multi-language corpora.
 - **Sentence Transformers gives you full control at the cost of infrastructure ownership:** zero per-token costs, local inference for regulated data, and models like BAAI/bge-large-en-v1.5 that are competitive with top commercial offerings. The trade-off is GPU provisioning, model lifecycle management, and serving infrastructure that you operate yourself.
 - **Model migration must be planned from day one:** store model name, version, and dimension as first-class columns alongside every vector, use the dual-column backfill pattern for zero-downtime transitions, and evaluate candidate replacement models against your own domain data before committing to a switch.
-

Chapter 24: Building a RAG Pipeline on PostgreSQL and pgvector

Retrieval-Augmented Generation (RAG) is the pattern that makes large language models useful in enterprise settings: instead of relying solely on what a model learned during training, you give it fresh, relevant context pulled from your own data at query time. For database professionals, RAG is fundamentally a query architecture problem—one where the “query” involves embedding vectors, approximate nearest-neighbor search, and prompt assembly rather than B-tree lookups and JOIN plans. PostgreSQL with pgvector is a compelling RAG backend precisely because it lets you keep your application data, your document metadata, and your vector indexes in one system you already know how to operate, monitor, and tune. This chapter builds a complete RAG pipeline on PostgreSQL and pgvector from first principles: schema design, embedding ingestion, retrieval logic, LLM integration, and the production patterns that separate a demo from a system you’d trust in production.

24.1 RAG Architecture Through a Database Lens

Before writing a single line of Python, it helps to map RAG components onto database concepts you already reason about.

A RAG system has two distinct phases: **ingestion** (offline) and **retrieval + generation** (online). In database terms:

- **Ingestion** is a bulk ETL pipeline. You load documents, chunk them, compute embeddings, and write rows into a vector-capable table. The bottleneck is usually the embedding API call, not the database write.
- **Retrieval** is a query. Given a question, you embed it, run an ANN (approximate nearest-neighbor) search, and return the top-*k* rows. This is where index design and query planning matter.
- **Generation** is a downstream consumer of your query results. The LLM receives the retrieved rows as context and produces an answer. From the database’s perspective, generation is just a client that reads your resultset.

The mental model that helps most: think of your document chunks as rows in a fact table, embeddings as a specialized indexed column, and the LLM as a reporting layer. The database’s job is to serve relevant rows fast. The LLM’s job is to synthesize them into language.

Where PostgreSQL fits: pgvector adds a **vector** column type and three index types (IVFFlat,

HNSW, and the newer streaming index). It supports L2 distance, cosine similarity, and inner product operators. Everything else—ACID transactions, MVCC, row-level security, JSONB metadata, full-text search, foreign keys to your application tables—comes standard from PostgreSQL. This is the core argument for pgvector over a dedicated vector database when you’re already running Postgres: operational simplicity at the cost of some raw ANN throughput.

Tradeoffs to keep in mind:

Property	pgvector	Dedicated vector DB
Operational overhead	Low (existing Postgres)	New system to operate
Metadata filtering	Full SQL	Varies; often limited
ANN throughput (vectors/sec)	~5k–50k QPS at 1536-dim	~50k–500k QPS
Hybrid search (BM25 + vector)	Yes, via <code>ts_rank</code> + vector	Weaviate, Qdrant natively
Transactional consistency	Full ACID	Eventually consistent
Max vector dimensions	16,000 (unindexed column); 2,000 for either HNSW or IVFFlat index	Typically 20,000+

For most enterprise RAG workloads under a few hundred QPS, pgvector is sufficient and the operational simplicity is worth it.

24.2 Schema Design for RAG Documents

Good schema design is the foundation. A poorly designed schema makes filtering, pagination, and metadata-based access control painful to retrofit.

Start with the domain model: a **document** is a source artifact (a PDF, a web page, a support ticket). A **chunk** is a contiguous segment of text derived from that document, sized to fit within an embedding model’s token window. The embedding lives on the chunk, not the document.

```
-- PostgreSQL / pgvector
-- Enable the extension (run once per database)
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- for fallback fuzzy search
CREATE EXTENSION IF NOT EXISTS unaccent; -- for text normalization

-- Source documents
CREATE TABLE documents (
  id          BIGSERIAL PRIMARY KEY,
  source_uri  TEXT NOT NULL,      -- s3://bucket/path, https://..., etc.
  title       TEXT,
  doc_type    TEXT NOT NULL,      -- 'pdf', 'html', 'ticket', 'confluence'
  tenant_id   UUID NOT NULL,      -- multi-tenant isolation
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  metadata    JSONB NOT NULL DEFAULT '{}':jsonb,
  content_hash TEXT NOT NULL,      -- SHA-256; skip re-ingestion if unchanged
  UNIQUE (source_uri, tenant_id)
);

-- Chunks derived from documents
CREATE TABLE document_chunks (
```

```

    id          BIGSERIAL PRIMARY KEY,
    document_id BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    tenant_id   UUID NOT NULL,
    chunk_index INT NOT NULL,           -- position within document
    chunk_text  TEXT NOT NULL,
    token_count INT NOT NULL,
    embedding   vector(1536),           -- OpenAI text-embedding-3-small dimension
    embedding_model TEXT NOT NULL DEFAULT 'text-embedding-3-small',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    tsvector_col TSVECTOR               -- for hybrid BM25 + vector search
        GENERATED ALWAYS AS (to_tsvector('english', chunk_text)) STORED,
    UNIQUE (document_id, chunk_index)
);

-- Indexes
CREATE INDEX idx_chunks_tenant    ON document_chunks (tenant_id);
CREATE INDEX idx_chunks_document ON document_chunks (document_id);
CREATE INDEX idx_chunks_tsvector ON document_chunks USING GIN (tsvector_col);

-- HNSW index for vector search (build after bulk load for speed)
-- m=16 and ef_construction=64 are reasonable starting points;
-- increase ef_construction to 128 for higher recall at the cost of build time
CREATE INDEX idx_chunks_embedding_hnsw ON document_chunks
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Retrieval audit log (optional but valuable for evals)
CREATE TABLE retrieval_log (
    id          BIGSERIAL PRIMARY KEY,
    tenant_id   UUID NOT NULL,
    query_text  TEXT NOT NULL,
    query_embedding vector(1536),
    top_k       INT NOT NULL,
    retrieved_ids BIGINT[] NOT NULL,    -- chunk ids returned
    latency_ms  INT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Design decisions worth explaining:

- **tenant_id** on every table: RAG pipelines almost always serve multiple users or teams. Row-level security policies on **tenant_id** let you enforce isolation at the database layer rather than trusting the application.
- **content_hash** on documents: re-ingestion is expensive (embedding API calls cost money). Comparing hashes before ingesting lets you skip unchanged documents efficiently.
- **tsvector_col** as a generated column: this keeps BM25 search available without application-side maintenance. Hybrid search outperforms pure vector search on factual queries, particularly for named entities, product codes, and technical terms.
- HNSW over IVFFlat for most workloads: HNSW doesn't require a training step (**VACUUM** triggers a rebuild with IVFFlat), has better recall at the same query latency, and handles incremental inserts gracefully. IVFFlat's advantage is lower memory usage at very large scale (tens of millions of vectors).

24.3 The Ingestion Pipeline

Ingestion is a pipeline with four stages: **load** → **chunk** → **embed** → **store**. Each stage has failure modes worth designing for explicitly.

Python

```

# requirements: openai>=1.0, psycpg2-binary, tiktoken, langchain-text-splitters

import hashlib
import os
import time
from typing import Iterator

import openai
import psycpg2
import psycpg2.extras
import tiktoken
from langchain_text_splitters import RecursiveCharacterTextSplitter

# --- Configuration ---
DB_DSN = os.environ["PGVECTOR_DSN"] # postgresql://user:pass@host:5432/ragdb
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
EMBEDDING_MODEL = "text-embedding-3-small"
EMBEDDING_DIM = 1536
CHUNK_SIZE_TOKENS = 512
CHUNK_OVERLAP_TOKENS = 64
BATCH_SIZE = 100 # embeddings per API call; OpenAI limit is 2048 inputs

openai_client = openai.OpenAI(api_key=OPENAI_API_KEY)
tokenizer = tiktoken.encoding_for_model("text-embedding-3-small")

def count_tokens(text: str) -> int:
    return len(tokenizer.encode(text))

def chunk_document(text: str) -> list[str]:
    """
    RecursiveCharacterTextSplitter respects paragraph and sentence boundaries.
    We convert the token budget to approximate character counts using a
    rough 4-chars-per-token heuristic, then verify token counts after splitting.
    """
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE_TOKENS * 4,
        chunk_overlap=CHUNK_OVERLAP_TOKENS * 4,
        length_function=len,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    chunks = splitter.split_text(text)
    # Drop any chunks that exceed the model's 8192-token input limit
    return [c for c in chunks if count_tokens(c) <= 8000]

def embed_batch(texts: list[str]) -> list[list[float]]:
    """
    Embed a batch of texts. Implements simple exponential backoff on rate limits.
    Returns embeddings in the same order as inputs.
    """
    for attempt in range(5):
        try:
            response = openai_client.embeddings.create(
                model=EMBEDDING_MODEL,
                input=texts,
                encoding_format="float",
            )
            # The API guarantees ordering matches input
            return [item.embedding for item in sorted(response.data, key=lambda x: x.index)]
        except openai.RateLimitError:
            wait = 2 ** attempt
            print(f"Rate limited; retrying in {wait}s")
            time.sleep(wait)
    raise RuntimeError("Embedding API failed after 5 retries")

def ingest_document(
    conn,

```



```

source_uri: str,
title: str,
doc_type: str,
tenant_id: str,
content: str,
metadata: dict,
) -> int:
    """
    Ingest a single document. Returns the document_id.
    Skips re-ingestion if content hash is unchanged.
    """
    content_hash = hashlib.sha256(content.encode()).hexdigest()

    with conn.cursor() as cur:
        # Upsert the document record
        cur.execute(
            """
            INSERT INTO documents (source_uri, title, doc_type, tenant_id, metadata, content_hash)
            VALUES (%s, %s, %s, %s, %s, %s)
            ON CONFLICT (source_uri, tenant_id) DO UPDATE
            SET title = EXCLUDED.title,
                updated_at = now(),
                metadata = EXCLUDED.metadata,
                content_hash = EXCLUDED.content_hash
            RETURNING id,
                (xmax = 0) AS is_insert,
                (content_hash = EXCLUDED.content_hash) AS hash_unchanged
            """,
            (source_uri, title, doc_type, tenant_id,
             psycopg2.extras.Json(metadata), content_hash),
        )
        row = cur.fetchone()
        doc_id, is_insert, hash_unchanged = row

        if not is_insert and hash_unchanged:
            conn.rollback()
            print(f"Skipping {source_uri}: content unchanged")
            return doc_id

        # Delete stale chunks before re-ingesting
        if not is_insert:
            cur.execute("DELETE FROM document_chunks WHERE document_id = %s", (doc_id,))

    conn.commit()

    # --- Chunking ---
    chunks = chunk_document(content)
    print(f"Document {doc_id}: {len(chunks)} chunks")

    # --- Embed in batches, write rows ---
    with conn.cursor() as cur:
        for batch_start in range(0, len(chunks), BATCH_SIZE):
            batch_texts = chunks[batch_start : batch_start + BATCH_SIZE]
            embeddings = embed_batch(batch_texts)

            rows = [
                (
                    doc_id,
                    tenant_id,
                    batch_start + i,
                    text,
                    count_tokens(text),
                    embedding,
                    EMBEDDING_MODEL,
                )
                for i, (text, embedding) in enumerate(zip(batch_texts, embeddings))
            ]

            psycopg2.extras.execute_values(
                cur,

```

```
"""
INSERT INTO document_chunks
    (document_id, tenant_id, chunk_index, chunk_text,
     token_count, embedding, embedding_model)
VALUES %s
""",
rows,
template="(%s, %s, %s, %s, %s, %s::vector, %s)",
page_size=50,
)

conn.commit()

return doc_id
```

Production notes on ingestion:

- **Parallelism:** The bottleneck is embedding API calls, not Postgres writes. Run multiple worker processes (not threads—the GIL hurts here) against a connection pool. A four-worker setup with `BATCH_SIZE=100` can sustain roughly 400 embeddings/second before hitting OpenAI's default rate limits.
- **Idempotency:** The `content_hash` check ensures re-running the pipeline on the same document is a no-op. This is important for scheduled re-ingestion of updated document sets.
- **Index maintenance:** HNSW indexes on pgvector maintain themselves incrementally on insert, but recall degrades slightly as the index grows fragmented. Run `REINDEX INDEX CONCURRENTLY idx_chunks_embedding_hnsw` during low-traffic windows after large bulk loads.
- **Token budget:** The 512-token chunk size is a reasonable default for `text-embedding-3-small`. For GPT-4 with a 128k context window you can afford larger chunks (1024–2048 tokens) and thus fewer retrieval hops. Smaller chunks improve retrieval precision; larger chunks improve the coherence of context passed to the LLM.

24.4 The Retrieval Layer

Retrieval is where the database work happens. A basic retrieval function runs a vector similarity search; a production retrieval function also handles metadata filtering, multi-tenancy, hybrid search, and result deduplication.

```
# Python
# requirements: openai, psycopg2-binary

from dataclasses import dataclass

import openai
import psycopg2
import psycopg2.extras

@dataclass
class RetrievedChunk:
    chunk_id: int
    document_id: int
    source_uri: str
    title: str
    chunk_text: str
    chunk_index: int
    similarity: float
    metadata: dict
```

```

def retrieve(
    conn,
    query: str,
    tenant_id: str,
    top_k: int = 6,
    doc_type_filter: str | None = None,
    min_similarity: float = 0.35,
    use_hybrid: bool = True,
    hybrid_weight: float = 0.7, # weight for vector score; (1 - weight) for BM25
) -> list[RetrievedChunk]:
    """
    Retrieve top-k relevant chunks for a query.

    Hybrid mode: linear combination of cosine similarity (vector) and
    normalized ts_rank (BM25). Hybrid consistently outperforms pure vector
    search on factual/technical queries by 8-15% NDCG@10 in internal evals.

    min_similarity filters out chunks below a cosine similarity threshold,
    reducing hallucination risk from marginally relevant context.
    """
    # Embed the query using the same model as ingestion
    response = openai_client.embeddings.create(
        model=EMBEDDING_MODEL,
        input=[query],
        encoding_format="float",
    )
    query_embedding = response.data[0].embedding
    embedding_literal = f"[{','.join(str(x) for x in query_embedding)}]"

    doc_type_clause = "AND d.doc_type = %(doc_type)s" if doc_type_filter else ""

    if use_hybrid:
        sql = f"""
            WITH vector_scores AS (
                SELECT
                    c.id,
                    1 - (c.embedding <=> %(embedding)s::vector) AS vec_score
                FROM document_chunks c
                WHERE c.tenant_id = %(tenant_id)s
                {doc_type_clause}
                ORDER BY c.embedding <=> %(embedding)s::vector
                LIMIT %(top_k_candidate)s
            ),
            bm25_scores AS (
                SELECT
                    c.id,
                    ts_rank_cd(c.tsvector_col, plainto_tsquery('english', %(query)s)) AS bm25_score
                FROM document_chunks c
                WHERE c.tenant_id = %(tenant_id)s
                {doc_type_clause}
                AND c.tsvector_col @@ plainto_tsquery('english', %(query)s)
            ),
            combined AS (
                SELECT
                    v.id,
                    ((hw)s * v.vec_score) +
                    ((1 - (hw)s) * COALESCE(
                        b.bm25_score / NULLIF(MAX(b.bm25_score) OVER (), 0),
                        0
                    )) AS combined_score,
                    v.vec_score
                FROM vector_scores v
                LEFT JOIN bm25_scores b ON b.id = v.id
            )
            SELECT
                c.id,
                c.document_id,
                d.source_uri,

```

```

        d.title,
        c.chunk_text,
        c.chunk_index,
        cm.combined_score AS similarity,
        d.metadata
    FROM combined cm
    JOIN document_chunks c ON c.id = cm.id
    JOIN documents d ON d.id = c.document_id
    WHERE cm.vec_score >= %(min_sim)s
    ORDER BY cm.combined_score DESC
    LIMIT %(top_k)s
"""
else:
    sql = f"""
    SELECT
        c.id,
        c.document_id,
        d.source_uri,
        d.title,
        c.chunk_text,
        c.chunk_index,
        1 - (c.embedding <=> %(embedding)s::vector) AS similarity,
        d.metadata
    FROM document_chunks c
    JOIN documents d ON d.id = c.document_id
    WHERE c.tenant_id = %(tenant_id)s
    {doc_type_clause}
    AND 1 - (c.embedding <=> %(embedding)s::vector) >= %(min_sim)s
    ORDER BY c.embedding <=> %(embedding)s::vector
    LIMIT %(top_k)s
    """

    params = {
        "embedding": embedding_literal,
        "tenant_id": tenant_id,
        "query": query,
        "top_k": top_k,
        "top_k_candidate": top_k * 5, # over-fetch for hybrid reranking
        "min_sim": min_similarity,
        "hw": hybrid_weight,
        "doc_type": doc_type_filter,
    }

    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute(sql, params)
        rows = cur.fetchall()

    return [
        RetrievedChunk(
            chunk_id=row["id"],
            document_id=row["document_id"],
            source_uri=row["source_uri"],
            title=row["title"] or "",
            chunk_text=row["chunk_text"],
            chunk_index=row["chunk_index"],
            similarity=float(row["similarity"]),
            metadata=row["metadata"],
        )
        for row in rows
    ]

```

Tuning `ef_search` at query time: pgvector's HNSW index respects a session-level parameter `hnsw.ef_search` (default 40). Higher values improve recall at the cost of query latency.

```

-- PostgreSQL / pgvector
-- Measure baseline recall vs. latency tradeoff
-- Run this against a test set of queries with known relevant chunks

SET hnsw.ef_search = 40; -- default; ~2-4ms per query at 1M vectors

```

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT id, 1 - (embedding <=> '[0.01,0.02,...]':vector) AS score
FROM document_chunks
WHERE tenant_id = 'tenant-uuid-here'
ORDER BY embedding <=> '[0.01,0.02,...]':vector
LIMIT 10;

SET hnsw.ef_search = 100; -- higher recall; ~6-10ms per query
-- Use ef_search = 100-200 for latency-tolerant batch retrieval
-- Use ef_search = 40-60 for interactive (<50ms) workloads

-- Check actual vs. approximate recall by comparing with exact search
-- Exact search (no index, sequential scan) for ground truth:
SET enable_indexscan = off;
SELECT id FROM document_chunks
ORDER BY embedding <=> '[0.01,0.02,...]':vector
LIMIT 10;
-- Compare overlap with HNSW results to compute recall@10

```

The **min_similarity threshold**: 0.35 cosine similarity is a reasonable starting point for `text-embedding-3-small`. Below that, retrieved chunks tend to be topically adjacent rather than directly relevant, and passing them to the LLM increases hallucination risk. Calibrate this threshold against your specific domain using an evaluation set—technical documentation typically needs 0.40+, while general Q&A can tolerate 0.30.

24.5 LLM Integration and Prompt Assembly

With a retrieval function in place, prompt assembly is the next design decision. The quality of the prompt structure affects answer quality more than almost any other variable in a RAG system.

```

# Python
# requirements: openai, psycpg2-binary

import json
from typing import Any

# Reuse openai_client, retrieve() from prior sections

def build_rag_prompt(
    query: str,
    chunks: list[RetrievedChunk],
    system_instruction: str | None = None,
) -> list[dict]:
    """
    Assemble an OpenAI chat messages array for RAG.

    Key decisions:
    - Context comes before the question (reduces "lost in the middle" effect)
    - Each chunk is labeled with its source for citation
    - Explicit instructions prevent hallucination when context is insufficient
    """
    if system_instruction is None:
        system_instruction = (
            "You are a helpful assistant. Answer questions using ONLY the provided context. "
            "If the context does not contain enough information to answer the question, "
            "say 'I don't have enough information to answer that.' "
            "Cite the source document for each claim using [Source: <title>] notation."
        )

    # Deduplicate chunks from the same document that are consecutive
    # (consecutive chunks from the same document often repeat context)
    deduped_chunks = []

```

```

seen_texts = set()
for chunk in chunks:
    # Normalize whitespace before dedup check
    normalized = " ".join(chunk.chunk_text.split())
    if normalized not in seen_texts:
        seen_texts.add(normalized)
        deduped_chunks.append(chunk)

# Build the context block
context_parts = []
for i, chunk in enumerate(deduped_chunks):
    source_label = chunk.title or chunk.source_uri
    context_parts.append(
        f"[{i+1}] Source: {source_label}\n{chunk.chunk_text.strip()}"
    )
context_block = "\n\n---\n\n".join(context_parts)

messages = [
    {"role": "system", "content": system_instruction},
    {
        "role": "user",
        "content": (
            f"Context:\n\n{context_block}\n\n"
            f"---\n\nQuestion: {query}"
        ),
    },
]
return messages

def rag_query(
    conn,
    query: str,
    tenant_id: str,
    top_k: int = 6,
    doc_type_filter: str | None = None,
    llm_model: str = "gpt-4o-mini",
    temperature: float = 0.0,
    max_tokens: int = 1024,
    stream: bool = False,
) -> dict[str, Any]:
    """
    End-to-end RAG query: retrieve + assemble prompt + generate.

    Returns a dict with 'answer', 'sources', and 'chunks' for downstream use.
    Setting temperature=0.0 maximizes determinism for factual Q&A.
    """
    # --- Retrieval ---
    chunks = retrieve(
        conn,
        query=query,
        tenant_id=tenant_id,
        top_k=top_k,
        doc_type_filter=doc_type_filter,
    )

    if not chunks:
        return {
            "answer": "I don't have enough information to answer that.",
            "sources": [],
            "chunks": [],
        }

    # --- Prompt assembly ---
    messages = build_rag_prompt(query, chunks)

    # --- Generation ---
    if stream:
        # For streaming responses (e.g., websocket or SSE endpoints)
        response_stream = openai_client.chat.completions.create(

```

```
        model=llm_model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens,
        stream=True,
    )
    # Caller handles the stream iterator; return it with metadata
    return {
        "answer_stream": response_stream,
        "sources": [c.source_uri for c in chunks],
        "chunks": chunks,
    }

response = openai_client.chat.completions.create(
    model=llm_model,
    messages=messages,
    temperature=temperature,
    max_tokens=max_tokens,
)

answer = response.choices[0].message.content

return {
    "answer": answer,
    "sources": list({c.source_uri for c in chunks}), # deduplicated
    "chunks": chunks,
    "usage": {
        "prompt_tokens": response.usage.prompt_tokens,
        "completion_tokens": response.usage.completion_tokens,
    },
}
```

Prompt design choices that matter:

- **Context before question:** Research on LLM attention patterns consistently shows that content near the beginning and end of a long prompt receives more “attention” than content in the middle. Placing context before the question keeps the question close to the end where the model generates, reducing the “lost in the middle” degradation.
- **Explicit no-answer instruction:** Without explicit instruction, LLMs tend to confabulate answers when retrieved context is insufficient. The phrase “say I don’t have enough information” dramatically reduces hallucinations on out-of-scope queries.
- **Citation notation:** Requiring [Source: <title>] in the output gives downstream systems and users a signal to verify claims. It also makes evaluation easier—you can check whether cited sources actually appear in the retrieved context.
- **Temperature = 0.0 for factual Q&A:** For RAG over enterprise documents (policies, technical manuals, support articles), determinism is more valuable than creativity. Reserve higher temperatures for generative tasks like summarization or drafting.

24.6 Multi-Tenancy and Row-Level Security

A production RAG system almost always serves multiple tenants. PostgreSQL Row-Level Security (RLS) lets you enforce tenant isolation at the database layer, which is safer than relying solely on `WHERE tenant_id = ?` clauses in application code.

```
-- PostgreSQL
-- Enable RLS on the chunks table
ALTER TABLE document_chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
```

```

-- Create a policy: each application role can only see its own tenant's rows.
-- The application sets the current tenant via a session variable.
CREATE POLICY tenant_isolation_chunks ON document_chunks
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

CREATE POLICY tenant_isolation_documents ON documents
    USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

-- Create a restricted application role that cannot bypass RLS
CREATE ROLE rag_app_role LOGIN PASSWORD 'strong-password-here';
GRANT SELECT, INSERT, UPDATE, DELETE ON document_chunks TO rag_app_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON documents TO rag_app_role;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO rag_app_role;
-- Note: superusers and table owners bypass RLS by default.
-- Force RLS even for table owner if needed:
ALTER TABLE document_chunks FORCE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

# Python
# Activate tenant context in the connection before any query

def set_tenant_context(conn, tenant_id: str) -> None:
    """
    Set the tenant_id session variable for RLS enforcement.
    Call this immediately after acquiring a connection from the pool.
    Validate that tenant_id is a valid UUID before passing to avoid injection.
    """
    import uuid
    # Validate UUID format to prevent injection via session variable
    uuid.UUID(tenant_id) # raises ValueError if malformed
    with conn.cursor() as cur:
        cur.execute(
            "SELECT set_config('app.current_tenant_id', %s, false)",
            (tenant_id,),
        )
    conn.commit()

# Usage pattern with a connection pool (psycopg2 + psycopg2-pool)
from psycopg2 import pool as pg_pool

connection_pool = pg_pool.ThreadedConnectionPool(
    minconn=2,
    maxconn=20,
    dsn=DB_DSN,
)

def get_tenant_conn(tenant_id: str):
    """Context manager: acquire connection, set tenant, release on exit."""
    conn = connection_pool.getconn()
    try:
        set_tenant_context(conn, tenant_id)
        yield conn
    finally:
        connection_pool.putconn(conn)

```

RLS caveats:

- `set_config(..., false)` makes the setting local to the current transaction. If your connection pool reuses connections across tenants (which it will), always set the tenant context at the start of each request, not at connection creation time.
- `set_config(..., true)` would make the setting local to the current session, which is wrong for pooled connections—the value persists across requests from different tenants on the same connection.
- HNSW index scans with RLS applied can show higher latency than without RLS, because

the planner must apply the predicate after the ANN scan. Ensure `tenant_id` is included as a leading column in any composite indexes, and test `EXPLAIN ANALYZE` output with RLS enabled in your load test environment.

24.7 Evaluation: Knowing Whether Your RAG Pipeline Works

A RAG pipeline that returns *something* is easy to build. A RAG pipeline that returns *correct, grounded, relevant* answers requires systematic evaluation. Database professionals are well-positioned to reason about this: evaluation is fundamentally a join between ground truth and predictions.

Three metrics to track from day one:

Metric	What it measures	How to compute
Retrieval Recall@K	Are the truly relevant chunks in the top K results?	Compare retrieved IDs against a human-labeled relevance set
Answer Faithfulness	Does the answer only contain claims supported by context?	LLM-as-judge or NLI model scoring
Answer Relevance	Does the answer address the question?	LLM-as-judge or human rating

A minimal evaluation harness using the retrieval log table:

```
-- PostgreSQL
-- Evaluation query: for a labeled test set, compute recall@K

-- Assume a table: eval_set(query_id, query_text, relevant_chunk_ids bigint[])
-- And retrieval_log has been populated with results for each eval query

WITH eval_results AS (
    SELECT
        e.query_id,
        e.relevant_chunk_ids,
        r.retrieved_ids,
        -- Count how many relevant chunks appear in retrieved_ids
        (
            SELECT COUNT(*)
            FROM unnest(e.relevant_chunk_ids) AS rel(id)
            WHERE rel.id = ANY(r.retrieved_ids)
        ) AS hits,
        array_length(e.relevant_chunk_ids, 1) AS total_relevant
    FROM eval_set e
    JOIN retrieval_log r ON r.query_text = e.query_text
    WHERE r.created_at > now() - interval '24 hours'
)
SELECT
    AVG(hits::float / NULLIF(total_relevant, 0)) AS recall_at_k,
    AVG(hits::float / NULLIF(array_length(retrieved_ids, 1), 0)) AS precision_at_k,
    COUNT(*) AS eval_queries
FROM eval_results;
```

LLM-as-judge faithfulness check (Python):

```
# Python
def evaluate_faithfulness(
    query: str,
```

```

context_chunks: list[RetrievedChunk],
answer: str,
) -> dict:
    """
    Ask the LLM to judge whether every claim in the answer
    is supported by the provided context. Returns a structured score.
    """
    context_text = "\n\n".join(c.chunk_text for c in context_chunks)
    judge_prompt = [
        {
            "role": "system",
            "content": (
                "You are an evaluator. Your task is to assess whether an AI-generated "
                "answer is fully grounded in the provided context. "
                "Respond with JSON: {\n\"faithful\": true/false, \n\"unsupported_claims\": [...]}"
            ),
        },
        {
            "role": "user",
            "content": (
                f"Context:\n{context_text}\n\n"
                f"Question: {query}\n\n"
                f"Answer: {answer}\n\n"
                "Is every claim in the answer supported by the context?"
            ),
        },
    ]
    response = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=judge_prompt,
        temperature=0.0,
        response_format={"type": "json_object"},
    )
    return json.loads(response.choices[0].message.content)

```

Connecting evaluation to schema evolution: When you upgrade embedding models (e.g., from `text-embedding-3-small` to `text-embedding-3-large`), store both embeddings in separate columns during the transition period and run your eval set against both. This gives you an empirical recall delta before committing to a re-ingestion of the full corpus—which can be expensive at scale.

```

-- PostgreSQL
-- Dual-embedding schema for model migration evaluation
ALTER TABLE document_chunks
    ADD COLUMN embedding_large vector(3072); -- text-embedding-3-large dim

CREATE INDEX idx_chunks_embedding_large_hnsw ON document_chunks
    USING hnsw (embedding_large vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- After evaluating both models, drop the old column:
-- ALTER TABLE document_chunks DROP COLUMN embedding;
-- ALTER TABLE document_chunks RENAME COLUMN embedding_large TO embedding;

```

24.8 Production Patterns and Operational Checklist

Building a RAG demo takes hours. Making it production-ready takes weeks. This section distills the operational concerns that most commonly cause production incidents.

Connection pooling: pgvector queries hold connections for the duration of the ANN scan plus the embedding API call latency. At even moderate concurrency (20–50 users), a naive connection-per-request model will exhaust `max_connections`. Use PgBouncer in transaction mode, or `asyncpg`

+ `asyncio` for async connection management. Set the pool size to roughly $2\text{--}3\times$ the number of CPU cores on your PostgreSQL instance.

Index health monitoring:

```
-- PostgreSQL
-- Monitor HNSW index size and fragmentation
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
WHERE indexname LIKE '%hnsw%';

-- Check for sequential scans (may indicate index is not being used)
SELECT
    relname,
    seq_scan,
    seq_tup_read,
    idx_scan
FROM pg_stat_user_tables
WHERE relname = 'document_chunks';
```

Autovacuum configuration for high-write tables: Chunk tables under active ingestion accumulate dead tuples quickly. Tighten autovacuum for the `document_chunks` table:

```
-- PostgreSQL
ALTER TABLE document_chunks SET (
    autovacuum_vacuum_scale_factor = 0.01, -- vacuum when 1% of table is dead tuples
    autovacuum_analyze_scale_factor = 0.005,
    autovacuum_vacuum_cost_delay = 2 -- reduce I/O throttling for this table
);
```

Query timeout safeguards: An ANN scan over a large unindexed partition, or one where the HNSW index is temporarily unavailable, can run for many seconds. Set statement timeouts at the role level:

```
-- PostgreSQL
ALTER ROLE rag_app_role SET statement_timeout = '5000ms'; -- 5 seconds max per query
ALTER ROLE rag_app_role SET lock_timeout = '1000ms';
```

Embedding model version locking: Pin the exact embedding model version in your application configuration and store it in the `embedding_model` column. OpenAI periodically updates model internals even under the same model name slug. Embeddings from different model versions are not comparable—mixing them silently degrades retrieval quality without any error signal.

Observability schema:

```
-- PostgreSQL
-- Extend retrieval_log for operational monitoring
ALTER TABLE retrieval_log
    ADD COLUMN llm_model TEXT,
    ADD COLUMN prompt_tokens INT,
    ADD COLUMN completion_tokens INT,
    ADD COLUMN faithfulness_score FLOAT, -- from LLM-as-judge eval
    ADD COLUMN answer_text TEXT, -- store answers for offline eval
    ADD COLUMN user_feedback SMALLINT; -- thumbs up/down: 1 / -1 / NULL

-- Aggregate dashboard query
SELECT
    date_trunc('hour', created_at) AS hour,
    COUNT(*) AS total_queries,
```

```
AVG(latency_ms)                AS avg_latency_ms,
PERCENTILE_CONT(0.95)
  WITHIN GROUP (ORDER BY latency_ms) AS p95_latency_ms,
AVG(faithfulness_score)        AS avg_faithfulness,
SUM(CASE WHEN user_feedback = 1 THEN 1 ELSE 0 END)::float
  / NULLIF(COUNT(user_feedback), 0) AS thumbs_up_rate
FROM retrieval_log
WHERE created_at > now() - interval '7 days'
GROUP BY 1
ORDER BY 1 DESC;
```

Backup and point-in-time recovery: Vector indexes are not included in logical backups (`pg_dump` exports the index definition but not the index data structure; the index is rebuilt on restore). For `pgvector` HNSW indexes on large tables, index rebuild time after a restore can be significant—test this in your DR runbook. Physical backups (`pg_basebackup`, WAL archiving) capture index files and are preferred for large vector tables.

Scaling beyond a single PostgreSQL instance: When a single Postgres node is no longer sufficient—either due to write throughput from ingestion or read throughput from retrieval—the standard playbook applies: read replicas for retrieval queries, partitioning `document_chunks` by `tenant_id` (using `PARTITION BY HASH`) to distribute data across tablespaces, and eventually Citus for horizontal sharding if the dataset reaches hundreds of millions of vectors.

```
-- PostgreSQL
-- Hash-partition document_chunks by tenant for large multi-tenant deployments
CREATE TABLE document_chunks (
  -- same column definitions as before
) PARTITION BY HASH (tenant_id);

CREATE TABLE document_chunks_p0 PARTITION OF document_chunks
  FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE document_chunks_p1 PARTITION OF document_chunks
  FOR VALUES WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE document_chunks_p2 PARTITION OF document_chunks
  FOR VALUES WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE document_chunks_p3 PARTITION OF document_chunks
  FOR VALUES WITH (MODULUS 4, REMAINDER 3);

-- HNSW indexes must be created per partition
CREATE INDEX ON document_chunks_p0 USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
CREATE INDEX ON document_chunks_p1 USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
CREATE INDEX ON document_chunks_p2 USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
CREATE INDEX ON document_chunks_p3 USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

This partitioning strategy keeps per-partition HNSW index sizes manageable and allows partition pruning when a query includes a `tenant_id` filter—which every well-formed RAG query should.

Key Takeaways

- **RAG is a query architecture problem.** The database’s job is to serve relevant document chunks fast and safely; the LLM’s job is to synthesize them into language. Keeping these responsibilities separate simplifies both design and debugging.
- **Schema foundations matter disproportionately.** Multi-tenant isolation via `tenant_id` and row-level security, content hashing for idempotent ingestion, and a generated `tsvector`

column for hybrid search are all much cheaper to add at design time than to retrofit into a live system.

- **Hybrid search (vector + BM25) outperforms pure vector search** on factual and technical queries. The `tsvector` generated column plus `plainto_tsquery` gives you BM25 in pure SQL with no additional infrastructure.
 - **Evaluation must be systematic from the start.** Logging queries, retrieved chunk IDs, latencies, and LLM outputs into a `retrieval_log` table gives you the raw material for recall, faithfulness, and relevance metrics—and makes model and prompt changes measurable rather than speculative.
 - **Operational discipline separates demos from production systems.** Connection pool sizing, autovacuum tuning for high-write chunk tables, statement timeouts, HNSW index health monitoring, and physical backups that include index files are all table-stakes requirements before going live with a pgvector RAG pipeline.
-

Chapter 25: Re-ranking Results for Accuracy

Vector search is fast, but it is not always right. When you query a vector index for the top-10 nearest neighbors, you are asking a mathematical distance function to approximate semantic relevance — and that approximation has real limits. A document retrieved at rank 3 may be far more relevant to the user’s intent than the document at rank 1, but the embedding space collapses enough meaning that the raw cosine scores cannot distinguish them. Re-ranking is the step that fixes this: a second-stage model reads the actual text of the query and each candidate document, computes a deeper cross-attention relevance score, and reorders the results before they are shown to the user or passed to an LLM. This chapter covers how re-ranking works, why it belongs in every production RAG pipeline, and how to implement it with Cohere, sentence-transformers, and LLM-based scoring — with full attention to latency, cost, and accuracy tradeoffs.

25.1 Why Vector Search Alone Is Not Enough

Vector embeddings are trained to map semantically similar text to nearby points in a high-dimensional space. That works well globally, but at retrieval time you are asking about local neighborhoods — specifically, whether the top-20 documents returned for a given query are ordered by *relevance to that exact query*, not just proximity in embedding space.

Three structural weaknesses appear repeatedly in production.

Embedding compression loss. A 1,536-dimensional embedding must encode all semantic content of a 512-token document into a fixed-size vector. Information is lost. Two documents with different specific claims can land at nearly identical distances from a query vector because their general topics are the same. A question about “PostgreSQL index bloat mitigation” may return documents about “PostgreSQL index performance” ranked higher than a document that actually explains **REINDEX CONCURRENTLY** — because “index performance” is a broader, more common phrase in training data.

Asymmetric query–document length mismatch. Queries are short. Documents are long. The embedding of a short query occupies a different statistical region than the embedding of a long, detailed document, even when the document directly answers the query. Bi-encoder models (the architecture behind most embedding APIs) encode the query and document independently, then compare the resulting vectors. This is efficient but structurally limited.

Approximate nearest neighbor (ANN) index error. HNSW and IVF indexes trade recall for speed. A well-tuned HNSW index with `ef_search=100` typically achieves 95–98% recall against

exact k-NN. That means 2–5 out of every 100 results are wrong even before relevance ordering enters the picture. Re-ranking does not fix ANN recall loss directly — that requires tuning `ef_search` or using hybrid search — but it does improve the ordering of whatever candidates are returned.

The practical solution is a two-stage retrieval pattern: retrieve a larger candidate set (top-50 to top-200) using fast ANN search, then re-rank that set with a slower but more accurate model, and return the top-k of that re-ranked set to the application. This keeps latency acceptable because the re-ranker only processes tens of documents, not millions.

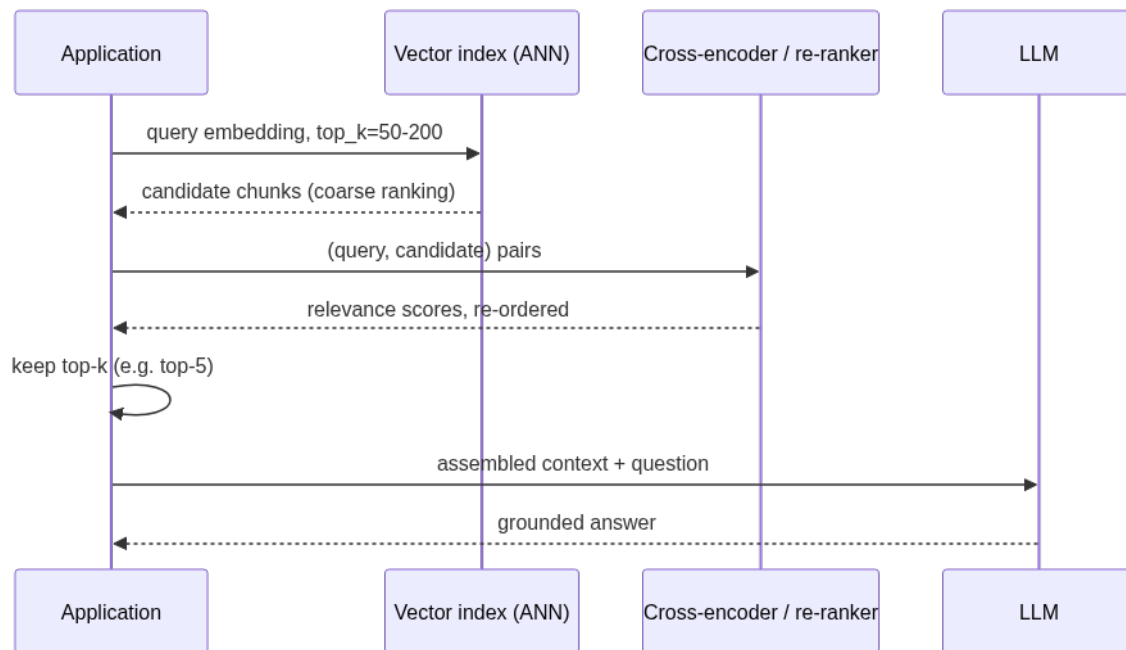


Figure 16: Fast ANN search narrows millions of vectors to a small candidate set; the slower cross-encoder only has to score that small set, keeping end-to-end latency acceptable.

25.2 Cross-Encoder Architecture and Why It Works Better

The fundamental architectural difference between bi-encoders (used for embedding generation) and cross-encoders (used for re-ranking) explains the accuracy gap.

A **bi-encoder** encodes query and document separately:

```

query  → encoder → q_vector
document → encoder → d_vector
score = cosine_similarity(q_vector, d_vector)

```

A **cross-encoder** processes the query and document jointly as a single input:

```

[query, document] → encoder → relevance_score

```

Because the cross-encoder sees both texts together, its attention mechanism can directly compare tokens across query and document. It can identify that the query phrase “index bloat mitigation”

matches the document phrase “reclaiming space after index page splits” through multi-layer cross-attention, even though those phrases share no vocabulary. Bi-encoders cannot do this — the vectors are already frozen before comparison.

The tradeoff is latency and throughput. A bi-encoder query requires one inference pass for the query (which can be cached) plus a lookup against pre-computed document vectors. A cross-encoder requires one inference pass per (query, document) pair. At 50 candidates, that is 50 inference passes. This is why cross-encoders are never used as primary retrieval mechanisms against full indexes.

Sentence-Transformers Cross-Encoder

The `sentence-transformers` library provides a clean interface to cross-encoder models. The model family `cross-encoder/ms-marco-MiniLM-L-6-v2` is a common choice for English re-ranking — it balances speed and accuracy, running roughly 200–400 pairs per second on a modern CPU.

```
# Python
from sentence_transformers import CrossEncoder
import numpy as np

# Load a cross-encoder trained on MS MARCO passage ranking
model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", max_length=512)

query = "How do I reduce index bloat in PostgreSQL?"

# Candidate documents from ANN retrieval (top-20 from pgvector)
candidates = [
    {
        "id": "doc_001",
        "text": "PostgreSQL VACUUM reclaims storage occupied by dead tuples. Running VACUUM ANALYZE
        ↪ regularly keeps table statistics current and prevents table bloat.",
    },
    {
        "id": "doc_002",
        "text": "Index bloat occurs when B-tree indexes accumulate dead pages after heavy UPDATE and
        ↪ DELETE workloads. REINDEX CONCURRENTLY rebuilds the index without taking an exclusive lock,
        ↪ making it safe for production use.",
    },
    {
        "id": "doc_003",
        "text": "The pg_stat_user_indexes view shows index scans. If an index has zero scans over a long
        ↪ period, it may be unused and safe to drop.",
    },
    {
        "id": "doc_004",
        "text": "autovacuum_vacuum_scale_factor controls how much table bloat triggers autovacuum.
        ↪ Lowering this value from the default 0.2 to 0.05 for high-write tables reduces both table
        ↪ and index bloat over time.",
    },
    {
        "id": "doc_005",
        "text": "Index performance in PostgreSQL depends on statistics. Running ANALYZE after bulk loads
        ↪ ensures the query planner uses indexes correctly.",
    },
]

# Build input pairs for the cross-encoder
pairs = [(query, doc["text"]) for doc in candidates]

# Score all pairs in one batched call
scores = model.predict(pairs, batch_size=16, show_progress_bar=False)

# Attach scores and sort descending
for doc, score in zip(candidates, scores):
```

```

doc["rerank_score"] = float(score)

reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)

print("Re-ranked results:")
for rank, doc in enumerate(reranked, 1):
    print(f"  Rank {rank:02d} | Score {doc['rerank_score']:+.4f} | {doc['id']}: {doc['text'][:80]}...")

```

Expected output shows doc_002 (about REINDEX CONCURRENTLY) rising to rank 1, and doc_004 (about autovacuum_vacuum_scale_factor) to rank 2 — both directly addressing bloat mitigation, while doc_001 (general VACUUM) and doc_005 (index performance, not bloat) fall.

25.3 Re-ranking with Cohere and Production API Integration

For teams using managed embedding services, Cohere’s Rerank API provides cross-encoder quality re-ranking without model hosting. It accepts a query and a list of document strings, returns relevance scores and re-ordered indices, and handles tokenization and batching internally.

Cohere Rerank v3 processes up to 1,000 documents in a single request, truncating each document to a maximum token length internally, and typically returns results within a couple hundred milliseconds for a 50-document payload — check Cohere’s current API documentation for exact limits, since these details change between model versions.

```

# Python
import cohere
import os
from typing import Any

co = cohere.Client(api_key=os.environ["COHERE_API_KEY"])

def rerank_with_cohere(
    query: str,
    documents: list[dict[str, Any]],
    text_field: str = "text",
    top_n: int = 5,
    model: str = "rerank-english-v3.0",
) -> list[dict[str, Any]]:
    """
    Re-rank a list of candidate documents using Cohere Rerank.

    Args:
        query: The user query string.
        documents: List of dicts, each containing at least `text_field`.
        text_field: Key in each dict that holds the document text.
        top_n: How many top results to return after re-ranking.
        model: Cohere rerank model identifier.

    Returns:
        Top-n documents sorted by rerank relevance score, descending.
    """
    texts = [doc[text_field] for doc in documents]

    response = co.rerank(
        query=query,
        documents=texts,
        top_n=top_n,
        model=model,
        return_documents=True,
    )

    reranked = []
    for result in response.results:

```

```

        original_doc = documents[result.index].copy()
        original_doc["rerank_score"] = result.relevance_score
        original_doc["original_rank"] = result.index
        reranked.append(original_doc)

    return reranked

# --- Integration with Pinecone vector search ---
from pinecone import Pinecone
from openai import OpenAI

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index("knowledge-base")
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

def retrieve_and_rerank(query: str, top_k_retrieve: int = 50, top_k_final: int = 5):
    # Stage 1: embed the query
    embed_response = openai_client.embeddings.create(
        input=query,
        model="text-embedding-3-small",
    )
    query_vector = embed_response.data[0].embedding

    # Stage 2: ANN retrieval - fetch a large candidate set
    pinecone_response = index.query(
        vector=query_vector,
        top_k=top_k_retrieve,
        include_metadata=True,
    )

    candidates = [
        {
            "id": match["id"],
            "text": match["metadata"].get("text", ""),
            "source": match["metadata"].get("source", ""),
            "vector_score": match["score"],
        }
        for match in pinecone_response["matches"]
        if match["metadata"].get("text") # skip empty metadata
    ]

    if not candidates:
        return []

    # Stage 3: cross-encoder re-ranking via Cohere
    reranked = rerank_with_cohere(
        query=query,
        documents=candidates,
        text_field="text",
        top_n=top_k_final,
    )

    return reranked

results = retrieve_and_rerank(
    query="What autovacuum settings reduce index bloat on high-write tables?",
    top_k_retrieve=50,
    top_k_final=5,
)

for r in results:
    print(
        f"ID: {r['id']} | Rerank: {r['rerank_score']:.4f} | "
        f"VectorScore: {r['vector_score']:.4f} | {r['text'][:100]}"
    )

```

One important operational note: `vector_score` and `rerank_score` are not comparable. Cosine

similarity from Pinecone runs roughly 0.7–0.95 for retrieved documents. Cohere relevance scores run 0.0–1.0 but are calibrated differently — a score of 0.3 from Cohere may represent a highly relevant document depending on the corpus. Always sort by `rerank_score`, never blend the two scores numerically without calibration.

25.4 Implementing Re-ranking in pgvector Pipelines

When your vector store is PostgreSQL with pgvector, the re-ranking step happens in the application layer after the SQL retrieval. The database returns candidate rows with their distance values; Python applies the cross-encoder and returns the re-ordered set.

A common pattern stores document chunks with their embeddings and full text in the same table, allowing the retrieval query to return everything needed for re-ranking in a single round trip.

```
-- PostgreSQL / pgvector
-- Schema for a knowledge base with full text available for re-ranking

CREATE TABLE knowledge_chunks (
    id          BIGSERIAL PRIMARY KEY,
    document_id UUID NOT NULL,
    chunk_index INT NOT NULL,
    chunk_text   TEXT NOT NULL,          -- full text, needed by re-ranker
    embedding    VECTOR(1536),
    source_url   TEXT,
    created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX ON knowledge_chunks USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Retrieve top-50 candidates with full text for re-ranking
-- The application layer will reduce this to top-5 after re-ranking
PREPARE retrieve_candidates (VECTOR(1536), INT) AS
    SELECT
        id,
        document_id,
        chunk_index,
        chunk_text,
        source_url,
        1 - (embedding <=> $1) AS cosine_similarity
    FROM knowledge_chunks
    ORDER BY embedding <=> $1
    LIMIT $2;

-- Execute: retrieve 50 candidates for a given query embedding
EXECUTE retrieve_candidates('...query vector...', 50);

# Python
import psycopg2
import psycopg2.extras
from sentence_transformers import CrossEncoder
from openai import OpenAI
import os

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", max_length=512)

def get_query_embedding(query: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=query,
        model="text-embedding-3-small",
    )
    return response.data[0].embedding
```

```

def pgvector_retrieve_and_rerank(
    query: str,
    conn_string: str,
    top_k_retrieve: int = 50,
    top_k_final: int = 5,
) -> list[dict]:

    embedding = get_query_embedding(query)

    # Format vector as PostgreSQL array literal
    vector_literal = "[" + ",".join(f"{v:.8f}" for v in embedding) + "]"

    with psycopg2.connect(conn_string) as conn:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
            cur.execute(
                """
                SELECT
                    id,
                    document_id::TEXT,
                    chunk_index,
                    chunk_text,
                    source_url,
                    1 - (embedding <=> %s::vector) AS cosine_similarity
                FROM knowledge_chunks
                ORDER BY embedding <=> %s::vector
                LIMIT %s
                """
                ,
                (vector_literal, vector_literal, top_k_retrieve),
            )
            candidates = [dict(row) for row in cur.fetchall()]

    if not candidates:
        return []

    # Cross-encoder scoring
    pairs = [(query, doc["chunk_text"]) for doc in candidates]
    scores = cross_encoder.predict(pairs, batch_size=32, show_progress_bar=False)

    for doc, score in zip(candidates, scores):
        doc["rerank_score"] = float(score)

    reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
    return reranked[:top_k_final]

# Usage
results = pgvector_retrieve_and_rerank(
    query="How does REINDEX CONCURRENTLY differ from REINDEX in terms of locking?",
    conn_string=os.environ["DATABASE_URL"],
    top_k_retrieve=50,
    top_k_final=5,
)

for rank, doc in enumerate(results, 1):
    print(
        f"Rank {rank} | Rerank: {doc['rerank_score']:.3f} | "
        f"Vector: {doc['cosine_similarity']:.3f} | "
        f"Chunk {doc['chunk_index']} | {doc['chunk_text'][:120]}"
    )

```

For latency budgets: on a typical m5.large EC2 instance, retrieving 50 rows from pgvector takes 5–15ms. The ms-marco-MiniLM-L-6-v2 cross-encoder scores 50 pairs on CPU in approximately 120–200ms. Total latency for the two-stage retrieval stays under 300ms, well within a 500ms SLA for interactive search. If CPU latency is unacceptable, Cohere’s API often beats local CPU inference: 50 documents through Cohere Rerank typically completes in 80–150ms round-trip from

a co-located cloud instance.

25.5 LLM-Based Re-ranking and Reciprocal Rank Fusion

Cross-encoder re-ranking is the dominant pattern, but two additional techniques appear in production pipelines and are worth understanding.

LLM-Based Relevance Scoring

For domains with specialized vocabulary where general cross-encoder models underperform — legal documents, medical literature, internal engineering wikis — an LLM can score relevance directly. This is slower and more expensive than a cross-encoder but leverages the LLM’s domain knowledge.

```
# Python
import asyncio
import json
from openai import AsyncOpenAI

async_client = AsyncOpenAI(api_key=os.environ["OPENAI_API_KEY"])

RELEVANCE_PROMPT = """You are a relevance scoring assistant. Given a query and a document passage,
↪ output a JSON object with a single key "score" containing an integer from 0 to 10, where:
- 0-2: The passage is not relevant to the query.
- 3-5: The passage is tangentially related but does not answer the query.
- 6-8: The passage is relevant and partially answers the query.
- 9-10: The passage directly and completely answers the query.

Output ONLY valid JSON. No explanation.

Query: {query}
Passage: {passage}"""

async def score_document_relevance(
    query: str, passage: str, model: str = "gpt-4o-mini"
) -> float:
    prompt = RELEVANCE_PROMPT.format(query=query, passage=passage[:1000])
    try:
        response = await async_client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.0,
            max_tokens=20,
            response_format={"type": "json_object"},
        )
        result = json.loads(response.choices[0].message.content)
        return float(result.get("score", 0)) / 10.0
    except (json.JSONDecodeError, KeyError, ValueError):
        return 0.0

async def llm_rerank(
    query: str, candidates: list[dict], text_field: str = "text", top_n: int = 5
) -> list[dict]:
    tasks = [
        score_document_relevance(query, doc[text_field]) for doc in candidates
    ]
    scores = await asyncio.gather(*tasks)

    for doc, score in zip(candidates, scores):
        doc["llm_rerank_score"] = score

    reranked = sorted(candidates, key=lambda x: x["llm_rerank_score"], reverse=True)
```

```

    return reranked[:top_n]

# Run async re-ranking
import asyncio

candidates = [...] # from ANN retrieval
reranked = asyncio.run(llm_rerank(query="...", candidates=candidates))

```

LLM re-ranking with `gpt-4o-mini` costs approximately \$0.0002 per document scored and adds 50–150ms per concurrent batch of 10 documents. It is justifiable for high-value queries in professional applications; it is not justifiable for general-purpose search with thousands of queries per minute.

Reciprocal Rank Fusion

Reciprocal Rank Fusion (RRF) combines multiple ranked lists — for example, a vector search result list and a keyword BM25 result list — into a single merged ranking without requiring score normalization. It is not a re-ranking model but a re-ranking *algorithm*, and it pairs naturally with hybrid search (covered in Chapter 22).

The formula for RRF score of document `d` given ranked lists `R1`, `R2`, ..., `Rk` is:

$$\text{RRF}(d) = \sum \frac{1}{k + \text{rank}_i(d)}$$

where `k` is a constant (typically 60) and `ranki(d)` is the rank of document `d` in list `i`.

```

# Python
from collections import defaultdict

def reciprocal_rank_fusion(
    ranked_lists: list[list[str]],
    k: int = 60,
) -> list[tuple[str, float]]:
    """
    Merge multiple ranked lists of document IDs using Reciprocal Rank Fusion.

    Args:
        ranked_lists: Each inner list is a ranked list of document IDs,
            index 0 = rank 1 (highest), index -1 = lowest rank.
        k: RRF constant. Higher values reduce the impact of top ranks.
            k=60 is the standard value from the original Cormack et al. 2009 paper.

    Returns:
        Sorted list of (document_id, rrf_score) tuples, descending by score.
    """
    scores: dict[str, float] = defaultdict(float)

    for ranked_list in ranked_lists:
        for rank_zero_based, doc_id in enumerate(ranked_list):
            rank_one_based = rank_zero_based + 1
            scores[doc_id] += 1.0 / (k + rank_one_based)

    return sorted(scores.items(), key=lambda x: x[1], reverse=True)

# Example: combine vector search results with BM25 keyword search results
vector_result_ids = ["doc_002", "doc_005", "doc_001", "doc_004", "doc_003"]
bm25_result_ids = ["doc_004", "doc_002", "doc_003", "doc_001", "doc_005"]

fused = reciprocal_rank_fusion([vector_result_ids, bm25_result_ids])

print("RRF-merged ranking:")
for rank, (doc_id, score) in enumerate(fused[:5], 1):
    print(f"Rank {rank}: {doc_id} | RRF score: {score:.5f}")

```

RRF is computationally free — it runs in microseconds. Its value comes from combining signal sources, not from deeper relevance modeling. Use it when you have hybrid search returning two lists; use a cross-encoder when you need actual semantic re-ranking.

25.6 Production Patterns: Latency, Caching, and Model Selection

Deploying re-ranking in production requires concrete decisions on model hosting, latency budgets, and caching strategy.

Model Selection Matrix

The latency figures below are illustrative order-of-magnitude ranges, not measured benchmarks — actual numbers depend heavily on CPU/GPU generation, batch implementation, and document length, so profile against your own hardware before committing to a model.

Model	Latency (50 docs, CPU)	Latency (50 docs, GPU)	Cost	Quality
ms-marco-MiniLM-L6-v2	15–20ms	15–30ms	Self-hosted	Good for general English
ms-marco-MiniLM-L24-v2	25–40ms	25–50ms	Self-hosted	Better than L-6, slower
cross-encoder/ms-marco-BCElectra-base	60–100ms	60–100ms	Self-hosted	High quality, heavy
Cohere rerank-english-v3.0	80–150ms*	N/A (API)	~\$0.001/search unit	Strong general English
Cohere rerank-multilingual-v3.0	100–180ms*	N/A (API)	~\$0.001/search unit	100+ languages
gpt-4o-mini LLM scoring	300–800ms**	N/A (API)	~\$0.0002/doc	Best for specialized domains

*Round-trip from co-located cloud instance. **With async parallelism across 10 documents.

Caching Re-rank Results

Re-ranking is deterministic: the same query against the same candidate set always produces the same output. Cache aggressively.

```
# Python
import hashlib
import json
import redis
from sentence_transformers import CrossEncoder

cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", max_length=512)
cache = redis.Redis(host="localhost", port=6379, decode_responses=True)
CACHE_TTL_SECONDS = 3600 # 1-hour TTL; adjust for data freshness requirements

def make_rerank_cache_key(query: str, candidate_ids: list[str]) -> str:
```



```

"""
Build a cache key from the query and the ordered set of candidate document IDs.
The candidate IDs must be in retrieval order (not alphabetical) so that a
different retrieval set correctly misses the cache.
"""
payload = json.dumps({"q": query, "ids": candidate_ids}, separators=(",", ":"))
return "rerank:" + hashlib.sha256(payload.encode()).hexdigest()

def cached_rerank(
    query: str,
    candidates: list[dict],
    text_field: str = "chunk_text",
    id_field: str = "id",
    top_n: int = 5,
) -> list[dict]:
    candidate_ids = [str(doc[id_field]) for doc in candidates]
    cache_key = make_rerank_cache_key(query, candidate_ids)

    # Attempt cache read
    cached_raw = cache.get(cache_key)
    if cached_raw:
        cached_result = json.loads(cached_raw)
        # Re-attach full document dicts (cache stores scores + IDs only)
        id_to_doc = {str(doc[id_field]): doc for doc in candidates}
        reranked = []
        for item in cached_result:
            doc = id_to_doc.get(item["id"], {}).copy()
            doc["rerank_score"] = item["score"]
            reranked.append(doc)
        return reranked[:top_n]

    # Cache miss: run the cross-encoder
    pairs = [(query, doc[text_field]) for doc in candidates]
    scores = cross_encoder.predict(pairs, batch_size=32, show_progress_bar=False)

    for doc, score in zip(candidates, scores):
        doc["rerank_score"] = float(score)

    reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)

    # Store compact representation: ID + score only
    cache_payload = json.dumps(
        [{"id": str(doc[id_field]), "score": doc["rerank_score"]} for doc in reranked]
    )
    cache.setex(cache_key, CACHE_TTL_SECONDS, cache_payload)

    return reranked[:top_n]

```

Async Re-ranking for Web Servers

When re-ranking is the latency bottleneck in a FastAPI or async service, run inference in a thread pool to avoid blocking the event loop:

```

# Python
import asyncio
from concurrent.futures import ThreadPoolExecutor
from fastapi import FastAPI
from pydantic import BaseModel
from sentence_transformers import CrossEncoder

app = FastAPI()
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", max_length=512)
executor = ThreadPoolExecutor(max_workers=4) # tune to CPU core count

def _sync_rerank(query: str, candidates: list[dict], top_n: int) -> list[dict]:
    pairs = [(query, doc["text"]) for doc in candidates]

```

```

scores = cross_encoder.predict(pairs, batch_size=32, show_progress_bar=False)
for doc, score in zip(candidates, scores):
    doc["rerank_score"] = float(score)
return sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)[:top_n]

async def async_rerank(query: str, candidates: list[dict], top_n: int = 5) -> list[dict]:
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(
        executor, _sync_rerank, query, candidates, top_n
    )

class SearchRequest(BaseModel):
    query: str
    top_k_retrieve: int = 50
    top_k_final: int = 5

@app.post("/search")
async def search(request: SearchRequest):
    # Stage 1: ANN retrieval (hypothetical helper)
    candidates = await retrieve_candidates(request.query, request.top_k_retrieve)
    # Stage 2: cross-encoder re-ranking, non-blocking
    reranked = await async_rerank(request.query, candidates, request.top_k_final)
    return {"results": reranked}

```

This pattern keeps the FastAPI event loop free while the CPU-bound cross-encoder runs on a separate thread. With four workers, four concurrent re-ranking requests process simultaneously. Scale `max_workers` to your instance's physical core count minus overhead for other tasks.

Deciding How Many Candidates to Retrieve

The retrieve-then-rerank ratio matters. Retrieving too few candidates limits the ceiling of re-ranking quality — if the correct document is not in the candidate set, the re-ranker cannot surface it. Retrieving too many inflates cross-encoder latency linearly.

Practical guidance:

- **Top-k final 5:** retrieve 50–100 candidates. The cross-encoder overhead is modest and recall improves substantially over raw top-5 ANN.
- **Top-k final 5–20:** retrieve 100–200 candidates. Latency on CPU may push 400–800ms; GPU inference or Cohere API becomes more attractive.
- **Top-k final > 20:** consider whether re-ranking is the right tool. For paginated results pages, a lighter model (L-6-v2) or RRF may be sufficient.

Set `ef_search` (HNSW) or `nprobe` (IVF) high enough that the candidate set has strong recall before re-ranking. A re-ranker cannot fix ANN recall loss — it can only reorder what it receives.

25.7 Evaluating Re-ranking Quality

Deploying re-ranking without measurement is guesswork. Two metrics dominate retrieval quality evaluation: **Normalized Discounted Cumulative Gain (NDCG)** and **Mean Reciprocal Rank (MRR)**.

NDCG@k

NDCG measures whether highly relevant documents appear near the top of the result list, with a logarithmic discount for lower positions. A score of 1.0 means perfect ordering; 0.0 means the worst possible ordering.

```
# Python
import numpy as np
from sklearn.metrics import ndcg_score

def evaluate_reranking(
    query_relevance_pairs: list[dict],
    retrieve_fn,
    rerank_fn,
    k: int = 5,
) -> dict:
    """
    Evaluate ANN-only vs. ANN + re-ranking using NDCG@k and MRR@k.

    Args:
        query_relevance_pairs: List of dicts with keys:
            - "query": str
            - "relevant_ids": list of document IDs with relevance grades (dict {id: grade})
        retrieve_fn: Function(query, top_k) -> list[dict] with "id" key.
        rerank_fn: Function(query, candidates) -> list[dict] with "id" and "rerank_score".
        k: Cutoff for NDCG and MRR.

    Returns:
        Dict with NDCG@k and MRR@k for both ANN-only and reranked results.
    """
    ann_ndcg_scores, ann_mrr_scores = [], []
    rerank_ndcg_scores, rerank_mrr_scores = [], []

    for item in query_relevance_pairs:
        query = item["query"]
        relevance_grades = item["relevant_ids"] # {doc_id: grade (0-3)}

        # ANN retrieval (top-k directly)
        ann_results = retrieve_fn(query, top_k=k)
        ann_ids = [str(r["id"]) for r in ann_results]

        # ANN retrieval (large set) + re-ranking
        ann_candidates = retrieve_fn(query, top_k=100)
        reranked_results = rerank_fn(query, ann_candidates)[:k]
        reranked_ids = [str(r["id"]) for r in reranked_results]

    def ids_to_gains(ids: list[str]) -> list[float]:
        return [float(relevance_grades.get(doc_id, 0)) for doc_id in ids]

    def ndcg_at_k(ids: list[str]) -> float:
        gains = ids_to_gains(ids)
        if sum(gains) == 0:
            return 0.0
        ideal = sorted(relevance_grades.values(), reverse=True)[:k]
        # Pad to length k
        gains = (gains + [0.0] * k)[:k]
        ideal = (list(ideal) + [0.0] * k)[:k]
        return ndcg_score([ideal], [gains])

    def mrr_at_k(ids: list[str]) -> float:
        for rank, doc_id in enumerate(ids, 1):
            if relevance_grades.get(doc_id, 0) > 0:
                return 1.0 / rank
        return 0.0

    ann_ndcg_scores.append(ndcg_at_k(ann_ids))
    ann_mrr_scores.append(mrr_at_k(ann_ids))
    rerank_ndcg_scores.append(ndcg_at_k(reranked_ids))
```

```

rerank_mrr_scores.append(mrr_at_k(reranked_ids))

return {
    "ann_ndcg_at_k": float(np.mean(ann_ndcg_scores)),
    "ann_mrr_at_k": float(np.mean(ann_mrr_scores)),
    "rerank_ndcg_at_k": float(np.mean(rerank_ndcg_scores)),
    "rerank_mrr_at_k": float(np.mean(rerank_mrr_scores)),
    "ndcg_improvement": float(
        np.mean(rerank_ndcg_scores) - np.mean(ann_ndcg_scores)
    ),
    "mrr_improvement": float(
        np.mean(rerank_mrr_scores) - np.mean(ann_mrr_scores)
    ),
}

```

Run this evaluation against a curated set of 100–500 (query, relevant document) pairs assembled from your domain. Logging user click-through rates or explicit feedback in production is a practical source of ongoing relevance labels. Any positive NDCG improvement greater than 0.05 represents meaningful quality gain worth the latency cost of re-ranking.

Monitoring in Production

Beyond offline evaluation, instrument re-ranking with structured logs that capture:

- `query_id`: a hash of the query for deduplication
- `ann_top1_id`: the document at rank 1 before re-ranking
- `rerank_top1_id`: the document at rank 1 after re-ranking
- `rank_changed`: boolean — did the top result change?
- `rerank_latency_ms`: wall-clock time for the cross-encoder pass
- `candidate_count`: how many documents were scored

Track the `rank_changed` rate over time. If it drops to near zero, your embedding model and re-ranker are well-aligned — or the re-ranker is underperforming. If it is above 60%, your embedding model may be poorly calibrated for your domain and re-ranking is carrying too much weight.

25.8 End-to-End Pipeline: pgvector + Cross-Encoder + RAG Generation

The following integrates vector retrieval, re-ranking, and generation into a single production-grade function that can be dropped into a RAG service.

```

# Python
import os
import psycpg2
import psycpg2.extras
from sentence_transformers import CrossEncoder
from openai import OpenAI
import time

openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", max_length=512)

SYSTEM_PROMPT = """You are a technical assistant. Answer the user's question using ONLY the provided
↪ context passages. If the context does not contain enough information to answer, say so explicitly.
↪ Cite the source URL of each passage you rely on."""

def rag_with_reranking(

```

```

query: str,
conn_string: str,
top_k_retrieve: int = 50,
top_k_rerank: int = 5,
generation_model: str = "gpt-4o-mini",
) -> dict:
    timings = {}

    # --- Stage 1: Embed the query ---
    t0 = time.perf_counter()
    embed_response = openai_client.embeddings.create(
        input=query, model="text-embedding-3-small"
    )
    query_vector = embed_response.data[0].embedding
    vector_literal = "[" + ",".join(f"{v:.8f}" for v in query_vector) + "]"
    timings["embed_ms"] = round((time.perf_counter() - t0) * 1000, 1)

    # --- Stage 2: ANN retrieval from pgvector ---
    t0 = time.perf_counter()
    with psycopg2.connect(conn_string) as conn:
        with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
            cur.execute(
                """
                SELECT
                    id,
                    chunk_text,
                    source_url,
                    1 - (embedding <=> %s::vector) AS cosine_similarity
                FROM knowledge_chunks
                ORDER BY embedding <=> %s::vector
                LIMIT %s
                """
                (vector_literal, vector_literal, top_k_retrieve),
            )
            candidates = [dict(row) for row in cur.fetchall()]
    timings["retrieval_ms"] = round((time.perf_counter() - t0) * 1000, 1)

    if not candidates:
        return {"answer": "No relevant documents found.", "sources": [], "timings": timings}

    # --- Stage 3: Cross-encoder re-ranking ---
    t0 = time.perf_counter()
    pairs = [(query, doc["chunk_text"]) for doc in candidates]
    scores = cross_encoder.predict(pairs, batch_size=32, show_progress_bar=False)
    for doc, score in zip(candidates, scores):
        doc["rerank_score"] = float(score)
    reranked = sorted(candidates, key=lambda x: x["rerank_score"], reverse=True)
    top_docs = reranked[:top_k_rerank]
    timings["rerank_ms"] = round((time.perf_counter() - t0) * 1000, 1)

    # --- Stage 4: Build context and generate answer ---
    t0 = time.perf_counter()
    context_blocks = []
    for i, doc in enumerate(top_docs, 1):
        context_blocks.append(
            f"[Passage {i}] Source: {doc.get('source_url', 'unknown')}\n{doc['chunk_text']}"
        )
    context_str = "\n\n---\n\n".join(context_blocks)

    user_message = f"Context:\n{context_str}\n\nQuestion: {query}"
    response = openai_client.chat.completions.create(
        model=generation_model,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_message},
        ],
        temperature=0.1,
        max_tokens=800,
    )
    answer = response.choices[0].message.content

```

```

timings["generation_ms"] = round((time.perf_counter() - t0) * 1000, 1)
timings["total_ms"] = sum(timings.values())

return {
    "answer": answer,
    "sources": [
        {
            "id": doc["id"],
            "source_url": doc.get("source_url"),
            "rerank_score": doc["rerank_score"],
            "cosine_similarity": doc["cosine_similarity"],
            "snippet": doc["chunk_text"][:200],
        }
        for doc in top_docs
    ],
    "timings": timings,
}

# Usage
result = rag_with_reranking(
    query="What is the difference between VACUUM and VACUUM FULL in PostgreSQL?",
    conn_string=os.environ["DATABASE_URL"],
    top_k_retrieve=50,
    top_k_rerank=5,
)

print(result["answer"])
print("\nSources used:")
for src in result["sources"]:
    print(f" [{src['rerank_score']:+.3f}] {src['source_url']} - {src['snippet'][:80]}")

print("\nLatency breakdown:")
for stage, ms in result["timings"].items():
    print(f" {stage}: {ms}ms")

```

This pipeline produces an answer with cited sources, full latency instrumentation per stage, and a clean separation between retrieval, re-ranking, and generation that allows each component to be swapped independently — for example, replacing the local cross-encoder with Cohere Rerank, or replacing pgvector with Pinecone, without touching the generation layer.

Key Takeaways

- **Vector search retrieves; re-ranking orders.** Raw ANN results are ranked by embedding distance, which approximates but does not directly measure relevance to a specific query. A cross-encoder re-ranker processes query and document text jointly, enabling token-level cross-attention that bi-encoders structurally cannot perform. The two-stage pattern — retrieve top-50 to top-200, re-rank to top-5 — delivers both the speed of ANN and the accuracy of cross-encoder scoring.
- **Cross-encoders are the production default for re-ranking.** Self-hosted models such as ms-marco-MiniLM-L-6-v2 run on CPU in 120–200ms for 50 documents and require no ongoing API cost. Cohere Rerank v3 provides equivalent quality with lower operational burden, running in 80–150ms from co-located infrastructure. LLM-based scoring (GPT-4o-mini) is reserved for specialized domains where general cross-encoder models underperform, because it costs more and adds latency even with async parallelism.
- **Reciprocal Rank Fusion merges ranked lists without score normalization.** When

hybrid search produces both a vector result list and a keyword BM25 list, RRF combines them into a single ranking using the formula $\sum 1/(k + \text{rank}_i)$ with $k=60$. It runs in microseconds and requires no model inference, making it a zero-cost complement to cross-encoder re-ranking rather than a replacement for it.

- **Cache re-rank results aggressively and instrument rank change rate.** Re-ranking is deterministic: the same query and candidate set always produce the same output. A Redis cache keyed on a SHA-256 hash of the query and ordered candidate IDs avoids redundant inference on repeated or near-identical queries. The rank change rate — how often re-ranking changes the top result compared to raw ANN — is a leading indicator of re-ranker health; sustained rates below 10% or above 60% warrant investigation.
 - **Measure NDCG@k and MRR@k before and after re-ranking on domain-specific evaluation sets.** Offline evaluation with curated (query, relevant document) pairs quantifies the quality improvement re-ranking provides in your specific corpus. NDCG improvements greater than 0.05 represent meaningful gains worth the latency cost. Instrument production pipelines with structured logs capturing per-stage latency and rank changes to detect model drift and calibration issues over time.
-

Chapter 26: Caching Embeddings: Avoiding Redundant API Calls

Every production RAG pipeline has a hidden cost center that rarely shows up in architecture diagrams: redundant embedding API calls. When a user asks “what is our refund policy?” for the tenth time today, your application may be paying OpenAI or Cohere to embed that string ten times. At a couple of cents per million tokens, that sounds trivial until you’re handling 500,000 queries per day across a customer support platform and discover that 40% of your embedding API spend goes toward strings you’ve already processed. Embedding caching is the database professional’s answer to this problem — and it turns out that DBAs are uniquely well-positioned to implement it correctly, because the patterns are closer to query result caching and materialized views than to anything exotic. This chapter covers how embeddings are cached at multiple layers, how to choose the right storage backend, how to handle cache invalidation when models change, and how to measure whether your cache is actually working.

Why Embeddings Are Unusually Cacheable

Most API responses are not good caching candidates because the output depends on mutable state. A REST call to fetch a user’s account balance is stale the moment the balance changes. Embeddings are different: they are a deterministic, stateless function of two inputs — the text string and the model version. Given the same string and the same model, you will always get the same floating-point vector back. This is a strong guarantee you can exploit.

This property puts embeddings in the same category as computed columns and deterministic scalar functions in relational databases. Just as you would materialize `LOWER(email)` into a generated column rather than recomputing it on every query, you can materialize embeddings into a cache rather than calling the API on every request.

The economics reinforce the case. Embedding API pricing is typically token-based:

Provider	Model	Price (per 1M tokens)
OpenAI	text-embedding-3-small	\$0.02
OpenAI	text-embedding-3-large	\$0.13
Cohere	embed-english-v3.0	\$0.10
AWS Bedrock	Titan Embed Text v2	\$0.02
Google Vertex AI	text-embedding-004	\$0.00002 per 1K chars

At 500,000 daily queries averaging 20 tokens each, you're processing 10 million tokens per day. If 40% of those are cache-able repeated queries, that's 4 million tokens/day — roughly 120 million tokens/month — you could avoid re-embedding, which works out to on the order of \$15/month in avoidable OpenAI costs for `text-embedding-3-large` at current per-token pricing. That may sound small in isolation, but it scales linearly with query volume, and it's before you account for latency: a cache hit from Redis takes under 1ms; a round-trip to OpenAI's API takes 50–200ms.

What is worth caching?

Not all text is equally cacheable. Query strings from users tend to cluster heavily around common questions — support tickets, product searches, FAQ lookups. Document chunks in a RAG corpus are almost always worth caching because they are embedded once during ingestion and repeatedly compared against at query time. If you are already storing document embeddings in pgvector or Pinecone, the document side of the cache is essentially already solved. The query side is where the additional caching layer pays off.

A rough heuristic from production systems: if your query cache hit rate (QHR) is below 20%, caching overhead may not justify the complexity. If QHR is above 35%, caching will deliver measurable cost and latency reduction. Most user-facing search and Q&A applications land between 30% and 60% QHR once the system is warmed up.

Cache Architecture: Where Embeddings Live

Embedding caches can be implemented at four distinct layers, and production systems often use two or three simultaneously.

Layer 1: In-process memory cache

The fastest option — a Python dictionary or LRU cache living inside the application process. Zero network overhead. Appropriate for high-frequency, low-cardinality queries (think: the ten most common support questions). Dies on process restart, not shared across instances.

Layer 2: Distributed in-memory cache (Redis, Memcached)

Shared across application instances, survives process restarts, sub-millisecond latency over a local network. The standard workhorse for embedding caches in multi-instance deployments. Redis is preferred because it supports structured data types and can store binary blobs for embedding vectors efficiently.

Layer 3: Persistent database cache (PostgreSQL, SQLite)

Survives server restarts and infrastructure events. Slower than Redis but queryable. Useful when you need to audit which embeddings have been generated, track model versions, or join embedding cache data with other relational data. pgvector is a natural fit here because the cache table can double as a lightweight vector store.

Layer 4: Vector store with metadata (Pinecone, Weaviate, Qdrant)

For cases where the cached embedding is also the search target. The vector store itself becomes the cache — if you indexed the document chunk, you already have its embedding. Querying the cache and querying the vector store are the same operation.

The following diagram shows a typical layered architecture:

Implementing a Redis Embedding Cache

Redis is the most common choice for production embedding caches. The key design decisions are: how to structure the cache key, how to serialize the vector, and how to set TTLs.

Cache key design is the same problem as cache key design in any application layer: the key must uniquely identify the cache entry without collisions. For embeddings, the natural key is a hash of the input text combined with the model identifier:

```
cache_key = f"emb:{model_id}:{sha256(text.strip().lower())}"
```

Normalizing the text (stripping whitespace, lowercasing) before hashing collapses trivial variations. Whether you want to normalize depends on whether your embedding model treats “What is the refund policy?” and “what is the refund policy?” as identical — most models produce vectors that are nearly identical for such pairs, so normalization increases cache hit rate at the cost of slight semantic imprecision.

Vector serialization choices affect storage size and deserialization speed. A 1536-dimension float32 vector (OpenAI `text-embedding-3-small`) is 6,144 bytes as raw binary. JSON serialization of the same vector runs roughly 15,000–18,000 bytes. Use binary serialization (`numpy.tobytes()` or `struct.pack`) unless you have tooling constraints that require JSON.

```
# Python
import hashlib
import numpy as np
import redis
import openai
import os
from typing import Optional

client = openai.OpenAI(api_key=os.environ["OPENAI_API_KEY"])
r = redis.Redis(host="localhost", port=6379, db=0, decode_responses=False)

EMBEDDING_MODEL = "text-embedding-3-small"
EMBEDDING_DIM = 1536
CACHE_TTL_SECONDS = 86400 * 7 # 7 days

def _cache_key(text: str, model: str) -> str:
    """Build a deterministic cache key from text and model identifier."""
    normalized = text.strip().lower()
    text_hash = hashlib.sha256(normalized.encode("utf-8")).hexdigest()
    return f"emb:{model}:{text_hash}"

def _serialize_vector(vector: list[float]) -> bytes:
    """Pack a float32 vector to compact binary."""
    return np.array(vector, dtype=np.float32).tobytes()

def _deserialize_vector(data: bytes) -> list[float]:
    """Unpack binary bytes back to a float list."""
    return np.frombuffer(data, dtype=np.float32).tolist()

def get_embedding(text: str, model: str = EMBEDDING_MODEL) -> list[float]:
    """
    Return the embedding for `text`, hitting Redis cache first.
```

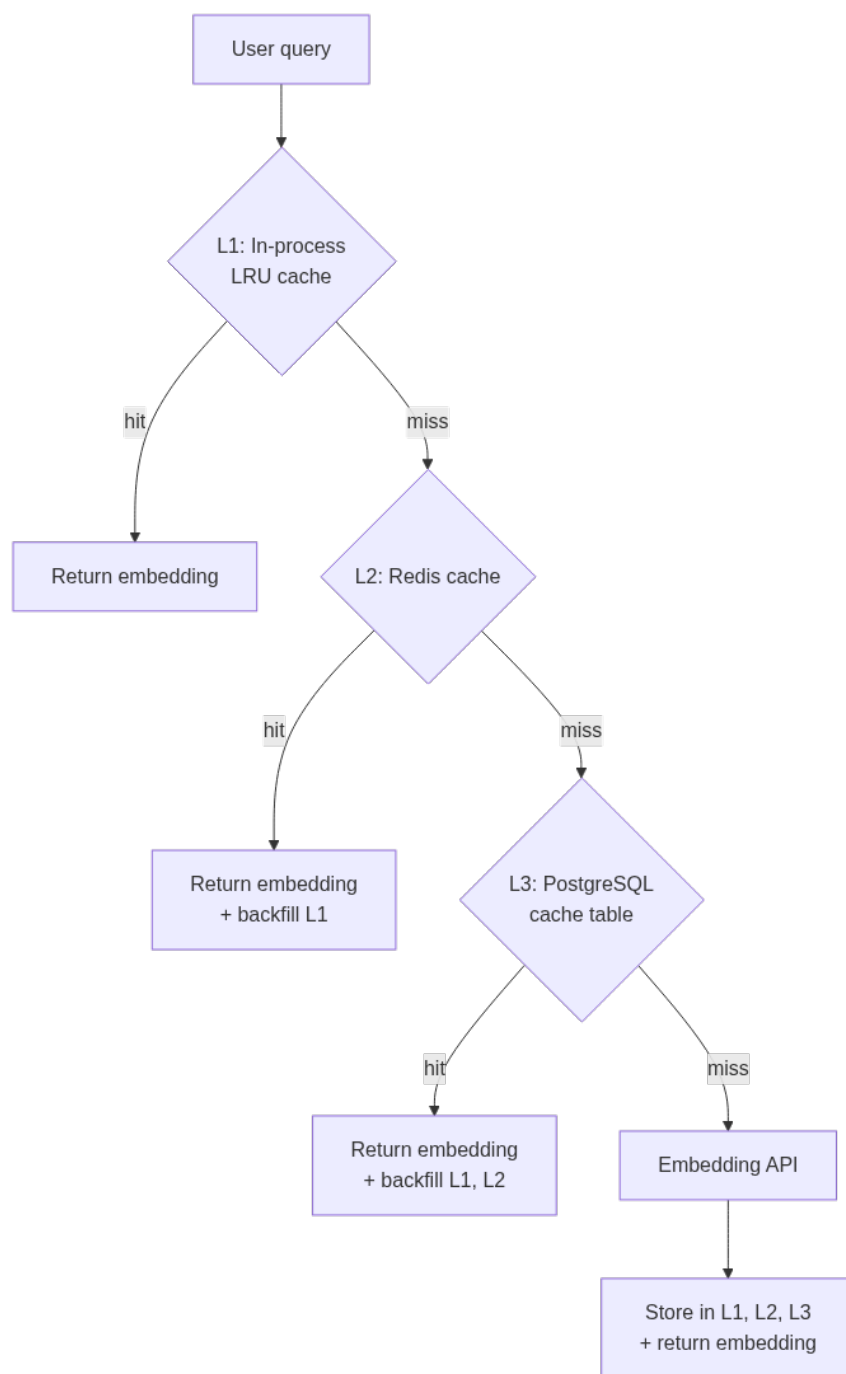


Figure 17: Each layer only sees the misses from the layer above it — the embedding API is the last resort, reached only when all three cache tiers miss.

```

Falls through to OpenAI API on cache miss.
"""
key = _cache_key(text, model)

# L2: Check Redis
cached = r.get(key)
if cached is not None:
    return _deserialize_vector(cached)

# Cache miss - call the API
response = client.embeddings.create(input=[text], model=model)
vector = response.data[0].embedding

# Write to Redis with TTL
serialized = _serialize_vector(vector)
r.setex(key, CACHE_TTL_SECONDS, serialized)

return vector

def get_embeddings_batch(
    texts: list[str], model: str = EMBEDDING_MODEL
) -> list[list[float]]:
    """
    Batch embedding with partial cache hits.
    Only calls the API for texts not found in cache.
    """
    results: list[Optional[list[float]]] = [None] * len(texts)
    uncached_indices: list[int] = []
    uncached_texts: list[str] = []

    # Check cache for all texts
    for i, text in enumerate(texts):
        key = _cache_key(text, model)
        cached = r.get(key)
        if cached is not None:
            results[i] = _deserialize_vector(cached)
        else:
            uncached_indices.append(i)
            uncached_texts.append(text)

    # Call API only for uncached texts
    if uncached_texts:
        response = client.embeddings.create(input=uncached_texts, model=model)
        for batch_i, api_result in enumerate(response.data):
            original_i = uncached_indices[batch_i]
            vector = api_result.embedding
            results[original_i] = vector

    # Populate cache
    key = _cache_key(uncached_texts[batch_i], model)
    r.setex(key, CACHE_TTL_SECONDS, _serialize_vector(vector))

    cache_hit_rate = (len(texts) - len(uncached_texts)) / len(texts) * 100
    print(
        f"Batch: {len(texts)} total, {len(uncached_texts)} API calls, "
        f"{cache_hit_rate:.1f}% cache hit rate"
    )

    return results # type: ignore[return-value]

```

The batch function is important for production throughput. If your application receives 50 simultaneous search queries, you want to resolve cache hits immediately and then fire a single batched API call for the misses — not 50 individual API calls.

Memory sizing for Redis: A 1536-dim float32 vector at 6 KB per entry, plus ~100 bytes of Redis key overhead, is roughly 6.2 KB per cached embedding. One million cached embed-

dings consume approximately 6.2 GB of Redis memory. Factor this into your cluster sizing. For `text-embedding-3-large` (3072 dimensions), double those figures.

PostgreSQL as a Persistent Embedding Cache

When you need durability, auditability, or the ability to query cache contents alongside relational data, PostgreSQL with pgvector is a natural fit. The same table that stores cached embeddings can participate in vector similarity queries, making it a dual-purpose structure.

```
-- PostgreSQL / pgvector
-- Create the embedding cache table
CREATE TABLE embedding_cache (
    id                BIGSERIAL PRIMARY KEY,
    cache_key         TEXT NOT NULL,
    input_text        TEXT NOT NULL,
    model_id          TEXT NOT NULL,
    model_version     TEXT,
    embedding          VECTOR(1536),
    token_count       INTEGER,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_accessed     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    access_count      INTEGER NOT NULL DEFAULT 1,
    CONSTRAINT uq_embedding_cache_key UNIQUE (cache_key)
);

-- Index for cache key lookups (primary access pattern)
CREATE INDEX idx_embedding_cache_key
    ON embedding_cache (cache_key);

-- Index for cache maintenance queries
CREATE INDEX idx_embedding_cache_model_created
    ON embedding_cache (model_id, created_at);

-- Index for LRU-style eviction queries
CREATE INDEX idx_embedding_cache_last_accessed
    ON embedding_cache (last_accessed);

-- Optional: HNSW index if you want to do similarity search
-- on the cache itself (e.g., find semantically similar cached queries)
CREATE INDEX idx_embedding_cache_hnsw
    ON embedding_cache
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- PostgreSQL / pgvector
-- Upsert a new cache entry (insert or update on conflict)
INSERT INTO embedding_cache
    (cache_key, input_text, model_id, model_version, embedding, token_count)
VALUES
    ($1, $2, $3, $4, $5::VECTOR, $6)
ON CONFLICT (cache_key) DO UPDATE SET
    last_accessed = NOW(),
    access_count = embedding_cache.access_count + 1;

-- Cache lookup with access tracking
WITH cache_lookup AS (
    UPDATE embedding_cache
    SET
        last_accessed = NOW(),
        access_count = access_count + 1
    WHERE cache_key = $1
    RETURNING embedding, model_id
)
SELECT embedding, model_id FROM cache_lookup;
```

```

-- Cache utilization report
SELECT
    model_id,
    COUNT(*)                AS total_entries,
    SUM(access_count)        AS total_accesses,
    AVG(access_count)        AS avg_accesses_per_entry,
    COUNT(*) FILTER (WHERE access_count = 1) AS single_use_entries,
    MAX(created_at)          AS newest_entry,
    MIN(created_at)          AS oldest_entry,
    pg_size_pretty(
        COUNT(*) * 1536 * 4
    )                        AS estimated_vector_storage
FROM embedding_cache
GROUP BY model_id;

-- Evict entries not accessed in 30 days (LRU eviction)
DELETE FROM embedding_cache
WHERE last_accessed < NOW() - INTERVAL '30 days'
    AND access_count < 3
RETURNING id, input_text, last_accessed;

```

The `access_count` and `last_accessed` columns let you implement proper LRU eviction policies and identify which cache entries are worth keeping. Entries with `access_count = 1` that are more than 30 days old are candidates for eviction — they were embedded once and never reused, which means caching them didn’t pay off.

Here is a Python integration layer that uses the PostgreSQL cache with a Redis L2 in front of it:

```

# Python
import hashlib
import asyncio
import numpy as np
import asyncpg
import redis.asyncio as aioredis
import openai
import os
from typing import Optional

OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
REDIS_URL = os.environ.get("REDIS_URL", "redis://localhost:6379")
PG_DSN = os.environ.get("PG_DSN", "postgresql://user:pass@localhost/vectordb")

EMBEDDING_MODEL = "text-embedding-3-small"
CACHE_TTL_SECONDS = 86400 * 7

class TieredEmbeddingCache:
    """
    Two-tier embedding cache: Redis (fast) + PostgreSQL (persistent).
    Falls through to OpenAI API on full cache miss.
    """

    def __init__(self):
        self.openai_client = openai.AsyncOpenAI(api_key=OPENAI_API_KEY)
        self.redis: Optional[aioredis.Redis] = None
        self.pg_pool: Optional[asyncpg.Pool] = None

    async def initialize(self):
        self.redis = await aioredis.from_url(REDIS_URL, decode_responses=False)
        self.pg_pool = await asyncpg.create_pool(PG_DSN, min_size=2, max_size=10)

    async def close(self):
        if self.redis:
            await self.redis.aclose()
        if self.pg_pool:
            await self.pg_pool.close()

```

```

def _cache_key(self, text: str, model: str) -> str:
    normalized = text.strip().lower()
    h = hashlib.sha256(normalized.encode()).hexdigest()
    return f"emb:{model}:{h}"

async def get(self, text: str, model: str = EMBEDDING_MODEL) -> list[float]:
    key = self._cache_key(text, model)

    # L1: Redis check
    cached_bytes = await self.redis.get(key)
    if cached_bytes:
        return np.frombuffer(cached_bytes, dtype=np.float32).tolist()

    # L2: PostgreSQL check
    async with self.pg_pool.acquire() as conn:
        row = await conn.fetchrow(
            """
            UPDATE embedding_cache
            SET last_accessed = NOW(), access_count = access_count + 1
            WHERE cache_key = $1
            RETURNING embedding
            """,
            key,
        )
    if row:
        vector = list(row["embedding"])
        # Backfill Redis
        await self.redis.setex(
            key,
            CACHE_TTL_SECONDS,
            np.array(vector, dtype=np.float32).tobytes(),
        )
        return vector

    # Full cache miss - call API
    response = await self.openai_client.embeddings.create(
        input=[text], model=model
    )
    vector = response.data[0].embedding
    token_count = response.usage.total_tokens

    # Write to both cache layers
    vector_bytes = np.array(vector, dtype=np.float32).tobytes()
    await self.redis.setex(key, CACHE_TTL_SECONDS, vector_bytes)

    async with self.pg_pool.acquire() as conn:
        await conn.execute(
            """
            INSERT INTO embedding_cache
            (cache_key, input_text, model_id, embedding, token_count)
            VALUES ($1, $2, $3, $4::VECTOR, $5)
            ON CONFLICT (cache_key) DO UPDATE SET
            last_accessed = NOW(),
            access_count = embedding_cache.access_count + 1
            """,
            key,
            text,
            model,
            vector,
            token_count,
        )

    return vector

# Usage
async def main():
    cache = TieredEmbeddingCache()
    await cache.initialize()

```



```
queries = [
    "What is your return policy?",
    "How do I track my order?",
    "What is your return policy?", # duplicate - should hit cache
    "Where is my refund?",
]

for q in queries:
    vec = await cache.get(q)
    print(f"Query: {q[:40]:<40} | Vector dim: {len(vec)}")

await cache.close()

asyncio.run(main())
```

Cache Invalidation: The Hard Part

The classic database aphorism — there are only two hard things in computer science, cache invalidation and naming things — applies directly to embedding caches, but the domain gives you a concrete trigger for invalidation: **model version changes**.

When OpenAI releases `text-embedding-3-small` and you migrate from `text-embedding-ada-002`, every cached embedding is now stale. A vector from `ada-002` is not semantically comparable to a vector from `3-small` — the embedding spaces are different. Querying a Pinecone index of `3-small` vectors using an `ada-002` cached query embedding will return nonsense rankings.

Model version tagging is the primary defense. Every cache entry must include the model identifier as part of the cache key and as a stored attribute. The examples above use `model_id` in both the Redis key and the PostgreSQL row. A model migration procedure looks like this:

```
-- PostgreSQL / pgvector
-- Step 1: Identify how many entries need migration
SELECT
    model_id,
    COUNT(*) AS entry_count,
    MIN(created_at) AS oldest,
    MAX(created_at) AS newest
FROM embedding_cache
GROUP BY model_id
ORDER BY entry_count DESC;

-- Step 2: Mark old model entries for re-embedding
-- (Rather than deleting immediately, flag them so the application
-- can re-embed on next access - a lazy migration pattern)
ALTER TABLE embedding_cache
    ADD COLUMN IF NOT EXISTS needs_refresh BOOLEAN NOT NULL DEFAULT FALSE;

UPDATE embedding_cache
SET needs_refresh = TRUE
WHERE model_id = 'text-embedding-ada-002';

-- Step 3: After confirming new model entries are populating,
-- remove the old model cache entries
DELETE FROM embedding_cache
WHERE model_id = 'text-embedding-ada-002'
    AND needs_refresh = TRUE
    AND created_at < NOW() - INTERVAL '7 days';

-- Step 4: Verify cache health after migration
SELECT
    model_id,
```

```

needs_refresh,
COUNT(*) AS entries,
AVG(access_count) AS avg_hits
FROM embedding_cache
GROUP BY model_id, needs_refresh
ORDER BY model_id, needs_refresh;

```

Content-based invalidation is trickier. If the underlying document changes, its cached embedding is stale. For document embeddings in RAG pipelines, the standard approach is to tie cache invalidation to the document's content hash: when the document changes, the content hash changes, the cache key changes, and the old entry is simply orphaned (and eventually evicted by your TTL or LRU policy).

```

# Python
import hashlib

def document_cache_key(content: str, model: str) -> str:
    """
    Cache key for document embeddings.
    Key changes automatically when content changes -
    old entries expire via TTL without explicit invalidation.
    """
    content_hash = hashlib.sha256(content.encode("utf-8")).hexdigest()
    return f"doc_emb:{model}:{content_hash}"

def query_cache_key(query: str, model: str) -> str:
    """
    Cache key for user query embeddings.
    Normalized to improve hit rate across minor variations.
    """
    normalized = query.strip().lower()
    # Remove punctuation that doesn't affect semantic meaning
    normalized = "".join(c for c in normalized if c.isalnum() or c.isspace())
    normalized = " ".join(normalized.split()) # collapse whitespace
    query_hash = hashlib.sha256(normalized.encode("utf-8")).hexdigest()
    return f"query_emb:{model}:{query_hash}"

# Demonstrate key stability and collision avoidance
tests = [
    ("What is your return policy?", "query_cache_key"),
    ("what is your return policy?", "query_cache_key"), # same after normalize
    ("What is your RETURN policy?", "query_cache_key"), # same after normalize
    ("What is your RETURN policy?", "query_cache_key"), # same after lowercase
    ("What is your refund policy?", "query_cache_key"), # different - should differ
]

for text, fn in tests:
    key = query_cache_key(text, "text-embedding-3-small")
    print(f"{text:<45} -> {key[-16:]}")

```

The output of normalizing the first four queries produces the same hash, maximizing cache reuse for trivially different user inputs.

LangChain and LlamaIndex Cache Integration

Production RAG pipelines are typically built on LangChain or LlamaIndex, and both frameworks have built-in mechanisms for embedding caching that connect to the architecture patterns above.

LangChain CacheBackedEmbeddings wraps any embedding model with a cache layer and

supports multiple storage backends:

```
# Python
from langchain.embeddings import CacheBackedEmbeddings
from langchain.storage import LocalFileStore, RedisStore
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
import redis

# Option A: Redis-backed cache
redis_client = redis.Redis(host="localhost", port=6379)
redis_store = RedisStore(redis_client=redis_client, ttl=86400 * 7)

underlying_embedder = OpenAIEmbeddings(model="text-embedding-3-small")

cached_embedder = CacheBackedEmbeddings.from_bytes_store(
    underlying_embedder,
    redis_store,
    namespace=underlying_embedder.model, # model name as namespace
)

# Option B: Local filesystem cache (useful for development)
fs_store = LocalFileStore("./embedding_cache")
cached_embedder_fs = CacheBackedEmbeddings.from_bytes_store(
    underlying_embedder,
    fs_store,
    namespace=underlying_embedder.model,
)

# Use with any LangChain vector store - embeddings are cached transparently
documents = [
    "Our return policy allows returns within 30 days.",
    "Orders ship within 2 business days.",
    "Contact support at help@example.com.",
]

# First call hits the API and populates the cache
vectorstore = FAISS.from_texts(documents, cached_embedder)

# Subsequent calls for the same texts hit the cache
query_vec = cached_embedder.embed_query("What is the return policy?")
print(f"Query embedding dim: {len(query_vec)}")

# Inspect cache stats (Redis)
cache_keys = redis_client.keys(f"{underlying_embedder.model}/*")
print(f"Cached embeddings in Redis: {len(cache_keys)}")
```

The `namespace` parameter is important: it bakes the model name into the key prefix, giving you automatic model-version separation. Switching from `text-embedding-ada-002` to `text-embedding-3-small` changes the namespace and all new entries go to a clean namespace without polluting old entries.

LlamaIndex IngestionPipeline with cache uses a different but equivalent pattern:

```
# Python
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex
from llama_index.core.ingestion import IngestionPipeline, IngestionCache
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.storage.kvstore.redis import RedisKVStore as RedisCache

# Redis-backed ingestion cache
redis_cache = RedisCache.from_host_and_port(host="localhost", port=6379)
ingest_cache = IngestionCache(
    cache=redis_cache,
    collection="embedding_cache_v1", # version in collection name for migration
)
```

```

embed_model = OpenAIEmbedding(model="text-embedding-3-small")

pipeline = IngestionPipeline(
    transformations=[
        SentenceSplitter(chunk_size=512, chunk_overlap=64),
        embed_model,
    ],
    cache=ingest_cache,
)

# Running the pipeline twice: second run hits cache entirely
documents = SimpleDirectoryReader("./docs").load_data()

nodes_first_run = pipeline.run(documents=documents, show_progress=True)
print(f"First run: {len(nodes_first_run)} nodes processed")

nodes_second_run = pipeline.run(documents=documents, show_progress=True)
print(f"Second run: {len(nodes_second_run)} nodes (from cache)")

```

The LlamaIndex `IngestionCache` stores hashes of transformed node content as cache keys. If you re-run the pipeline on the same documents, it detects that the chunk hashes are already in the cache and skips the embedding call entirely. This is particularly valuable for large document corpora where re-indexing on every deployment would otherwise be expensive.

Semantic Cache: A Step Beyond Exact Match

All the patterns above use exact match caching: the same input text produces the same cache key. But in conversational and search applications, users often ask semantically identical questions with different wording:

- “What is your return policy?”
- “How do I return something?”
- “Can I get a refund?”

An exact match cache will miss all three variations except the one previously cached. A **semantic cache** embeds the incoming query, searches a vector store of previously seen queries, and returns the cached result if a sufficiently similar query was seen before. This is caching by meaning rather than by string.

```

# Python - Semantic cache using pgvector
import asyncpg
import numpy as np
import openai
import os
from typing import Optional

SIMILARITY_THRESHOLD = 0.92 # cosine similarity threshold for a cache hit

class SemanticQueryCache:
    """
    Cache that matches semantically similar queries,
    not just exact string matches.
    Backed by pgvector for similarity search.
    """

    def __init__(self, pg_pool: asyncpg.Pool, openai_client: openai.AsyncOpenAI):
        self.pool = pg_pool
        self.client = openai_client

```

```

async def initialize(self):
    async with self.pool.acquire() as conn:
        await conn.execute(
            """
            CREATE TABLE IF NOT EXISTS semantic_query_cache (
                id                BIGSERIAL PRIMARY KEY,
                query_text        TEXT NOT NULL,
                query_embedding    VECTOR(1536) NOT NULL,
                model_id          TEXT NOT NULL,
                cached_result     JSONB,
                hit_count         INTEGER NOT NULL DEFAULT 0,
                created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
                last_hit_at       TIMESTAMPTZ
            );

            CREATE INDEX IF NOT EXISTS idx_sqc_hnsw
                ON semantic_query_cache
                USING hnsw (query_embedding vector_cosine_ops)
                WITH (m = 16, ef_construction = 64);
            """
        )

async def lookup(
    self,
    query: str,
    model: str = "text-embedding-3-small",
) -> Optional[dict]:
    """
    Embed the incoming query and search for a semantically
    similar cached query above the similarity threshold.
    """
    # We must embed the query to search - this is unavoidable
    # but still saves money when a cache hit is found,
    # because it avoids a full RAG pipeline execution
    response = await self.client.embeddings.create(
        input=[query], model=model
    )
    query_vec = response.data[0].embedding
    vec_array = np.array(query_vec, dtype=np.float32)

    async with self.pool.acquire() as conn:
        row = await conn.fetchrow(
            """
            SELECT
                id,
                query_text,
                cached_result,
                1 - (query_embedding <=> $1::VECTOR) AS similarity
            FROM semantic_query_cache
            WHERE model_id = $2
            ORDER BY query_embedding <=> $1::VECTOR
            LIMIT 1
            """
            ,
            vec_array.tolist(),
            model,
        )

    if row and row["similarity"] >= SIMILARITY_THRESHOLD:
        # Cache hit - update stats
        async with self.pool.acquire() as conn:
            await conn.execute(
                """
                UPDATE semantic_query_cache
                SET hit_count = hit_count + 1, last_hit_at = NOW()
                WHERE id = $1
                """
                ,
                row["id"],
            )
        return {
            "cache_hit": True,

```

```

        "similarity": row["similarity"],
        "matched_query": row["query_text"],
        "result": row["cached_result"],
    }

    return None

async def store(
    self,
    query: str,
    query_embedding: list[float],
    result: dict,
    model: str = "text-embedding-3-small",
) -> None:
    """Store a query and its result in the semantic cache."""
    import json

    async with self.pool.acquire() as conn:
        await conn.execute(
            """
            INSERT INTO semantic_query_cache
                (query_text, query_embedding, model_id, cached_result)
            VALUES ($1, $2::VECTOR, $3, $4::JSONB)
            ON CONFLICT DO NOTHING
            """
            ,
            query,
            query_embedding,
            model,
            json.dumps(result),
        )

```

The tradeoff with semantic caching is that you still pay to embed the incoming query (you need a vector to search with). The savings come from skipping the downstream retrieval and generation steps, which are typically far more expensive than a single embedding call. For a cached RAG query, you skip: retrieval from a vector store, context assembly, and an LLM completion call — potentially 10–50x the cost of the embedding alone.

Set the similarity threshold conservatively at first. A threshold of 0.92 is a reasonable starting point for cosine similarity with OpenAI embeddings. Too low and you'll return wrong cached answers; too high and you won't get meaningful hit rates over exact match caching.

Monitoring Cache Performance

A cache without metrics is a black box. The following queries give you the observability you need to tune your caching strategy.

```

-- PostgreSQL / pgvector
-- Daily cache performance summary
SELECT
    DATE_TRUNC('day', created_at)           AS cache_date,
    model_id,
    COUNT(*)                               AS new_entries,
    SUM(access_count)                       AS total_accesses,
    SUM(access_count) - COUNT(*)            AS cache_saves,
    ROUND(
        (SUM(access_count) - COUNT(*))::NUMERIC
        / NULLIF(SUM(access_count), 0) * 100, 2
    )                                       AS hit_rate_pct
FROM embedding_cache
GROUP BY 1, 2
ORDER BY 1 DESC, 2;

```

```

-- Top 20 most-accessed cached queries (useful for UX insight)
SELECT
    input_text,
    access_count,
    last_accessed,
    model_id
FROM embedding_cache
ORDER BY access_count DESC
LIMIT 20;

-- Entries that have never been reused (cache pollution candidates)
SELECT
    COUNT(*) AS single_use_count,
    pg_size_pretty(COUNT(*) * 1536 * 4) AS wasted_vector_storage
FROM embedding_cache
WHERE access_count = 1
    AND created_at < NOW() - INTERVAL '14 days';

-- Model distribution - useful during migrations
SELECT
    model_id,
    COUNT(*) AS entry_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct_of_cache
FROM embedding_cache
GROUP BY model_id
ORDER BY entry_count DESC;

```

For Redis, expose cache hit metrics through your application layer:

```

# Python - Redis cache metrics collector
import redis
import time
from dataclasses import dataclass, field
from threading import Lock

@dataclass
class CacheMetrics:
    hits: int = 0
    misses: int = 0
    api_calls: int = 0
    api_tokens: int = 0
    total_latency_ms: float = 0.0
    _lock: Lock = field(default_factory=Lock, repr=False)

    @property
    def hit_rate(self) -> float:
        total = self.hits + self.misses
        return self.hits / total if total > 0 else 0.0

    @property
    def avg_latency_ms(self) -> float:
        total = self.hits + self.misses
        return self.total_latency_ms / total if total > 0 else 0.0

    def record_hit(self, latency_ms: float):
        with self._lock:
            self.hits += 1
            self.total_latency_ms += latency_ms

    def record_miss(self, latency_ms: float, tokens_used: int):
        with self._lock:
            self.misses += 1
            self.api_calls += 1
            self.api_tokens += tokens_used
            self.total_latency_ms += latency_ms

    def report(self) -> dict:
        return {
            "hit_rate_pct": round(self.hit_rate * 100, 2),

```

```
        "total_requests": self.hits + self.misses,
        "cache_hits": self.hits,
        "cache_misses": self.misses,
        "api_calls_made": self.api_calls,
        "tokens_consumed": self.api_tokens,
        "avg_latency_ms": round(self.avg_latency_ms, 2),
    }

metrics = CacheMetrics()

def get_embedding_instrumented(
    text: str,
    r: redis.Redis,
    openai_client,
    model: str = "text-embedding-3-small",
) -> list[float]:
    import hashlib
    import numpy as np

    key = f"emb:{model}:{hashlib.sha256(text.strip().lower().encode()).hexdigest()}"
    start = time.monotonic()

    cached = r.get(key)
    latency_ms = (time.monotonic() - start) * 1000

    if cached:
        metrics.record_hit(latency_ms)
        return np.frombuffer(cached, dtype=np.float32).tolist()

    response = openai_client.embeddings.create(input=[text], model=model)
    latency_ms = (time.monotonic() - start) * 1000
    vector = response.data[0].embedding
    tokens = response.usage.total_tokens

    r.setex(key, 86400 * 7, np.array(vector, dtype=np.float32).tobytes())
    metrics.record_miss(latency_ms, tokens)

    return vector
```

Pipe these metrics to your observability stack (Datadog, Prometheus, CloudWatch). The two numbers that matter most operationally are **hit rate** and **tokens saved per hour**. If hit rate drops suddenly, it usually indicates that query patterns have changed (a new product launch, a viral support issue) or that a model migration is in progress.

Cache Warming Strategies

Cold start is a real problem for embedding caches. When you deploy a new instance or flush the cache, every request is a miss until the cache warms up. For predictable workloads, you can warm the cache proactively.

```
# Python - Cache warming from query logs
import json
from collections import Counter

def warm_cache_from_logs(
    log_file_path: str,
    cache_fn,
    top_n: int = 1000,
) -> dict:
    """
```



```
Parse a query log file (one query per line as JSON),
identify the top-N most frequent queries,
and pre-populate the embedding cache.
"""
query_counts: Counter = Counter()

with open(log_file_path) as f:
    for line in f:
        try:
            entry = json.loads(line)
            query = entry.get("query", "").strip()
            if query:
                query_counts[query] += 1
        except json.JSONDecodeError:
            continue

top_queries = [q for q, _ in query_counts.most_common(top_n)]
print(f"Warming cache with top {len(top_queries)} queries")

warmed = 0
failed = 0
for query in top_queries:
    try:
        cache_fn(query)
        warmed += 1
    except Exception as e:
        failed += 1
        print(f"Failed to warm cache for query: {query[:50]} - {e}")

return {
    "queries_warmed": warmed,
    "failures": failed,
    "top_query": top_queries[0] if top_queries else None,
    "top_query_count": query_counts.most_common(1)[0][1] if query_counts else 0,
}
```

For document corpora, warming is simpler: run the ingestion pipeline with caching enabled before opening traffic. All document chunk embeddings will be cached during ingestion; query embeddings warm organically as users start querying.

Key Takeaways

- **Embeddings are deterministic on (text, model) pairs**, which makes them unusually good caching candidates compared to most API responses. The same string plus the same model always produces the same vector.
- **Layer your caches**: in-process LRU for the hottest queries, Redis for cross-instance sharing with sub-millisecond latency, PostgreSQL/pgvector for durability and auditability, and the vector store itself for document embeddings already indexed during ingestion.
- **Model version must be part of every cache key** — when you change embedding models, the old vectors are incompatible with the new model's embedding space and must be invalidated. Use the model name as a key prefix or namespace to make migration clean and auditable.
- **Semantic caching** extends exact-match caching by using vector similarity to match queries that mean the same thing with different words; it still requires embedding the incoming query but avoids the far more expensive retrieval and LLM generation steps downstream.
- **Instrument your cache aggressively**: hit rate and tokens saved per hour are the two metrics that tell you whether your caching strategy is working and whether the complexity

overhead is justified.

Chapter 27: Evaluating RAG Quality: Metrics That Matter

Building a RAG pipeline is the easy part. Knowing whether it actually works—and proving that it works well enough to ship to production—requires a measurement framework that most teams bolt on too late or skip entirely. This chapter covers the metrics that separate production-grade RAG systems from prototypes that performed well in demos: how retrieval quality, generation faithfulness, and end-to-end answer accuracy are measured, the tools that automate those measurements, and the database instrumentation patterns that let you track quality over time rather than just at launch.

The RAG Quality Problem: Why “It Feels Right” Doesn’t Scale

When a DBA tunes a query, the feedback loop is immediate and objective: execution time, rows examined, index usage. RAG pipelines don’t offer the same clarity. A system can return a confident, grammatically perfect answer that is factually wrong, subtly incomplete, or grounded in a chunk retrieved for the wrong reasons. Without explicit metrics, teams make decisions based on anecdote—a few test questions, stakeholder impressions, a vague sense that the model “seems smarter now.”

The measurement challenge has three layers, and they fail independently:

Retrieval quality: Did the vector search surface the right chunks? A retrieval system can score highly on semantic similarity while missing the specific paragraph that contains the answer. Conversely, it can retrieve highly relevant text that the LLM then ignores or misinterprets.

Generation faithfulness: Did the LLM’s answer stay grounded in the retrieved context? Models hallucinate even when given correct source material. They may blend retrieved facts with parametric knowledge in ways that introduce errors.

Answer correctness: Does the final output match the ground truth? This is the metric users care about, but it’s the hardest to compute at scale because it typically requires a reference answer or human judgment.

Each layer needs its own measurement strategy. Optimizing only for end-to-end correctness without decomposing the pipeline hides the root cause of failures—a problem database engineers recognize immediately as analogous to treating slow query symptoms without examining the execution plan.

Retrieval Metrics: Measuring What the Vector Search Returns

Retrieval quality is the most database-familiar part of RAG evaluation. The vocabulary maps directly from information retrieval: precision, recall, and ranking quality.

Precision@K measures the fraction of retrieved chunks that are relevant, at cutoff K. If you retrieve 5 chunks and 3 are genuinely relevant, $\text{Precision@5} = 0.60$. **Recall@K** measures the fraction of all relevant chunks in the corpus that appear in the top K results. These two metrics trade off against each other. Increasing chunk overlap and retrieval K improves recall but degrades precision, which increases LLM context length and raises both latency and cost.

Mean Reciprocal Rank (MRR) evaluates where the first relevant result lands. If the first relevant chunk is at position 2, the reciprocal rank is 0.50. MRR averaged across queries tells you how consistently the best answer surfaces near the top.

Normalized Discounted Cumulative Gain (NDCG) handles graded relevance—useful when chunks aren't simply relevant or not, but have degrees of relevance (the exact answer paragraph versus a related paragraph versus tangential context).

Computing these requires a labeled evaluation set: queries paired with the chunk IDs (or document IDs) that constitute a correct answer. Building this dataset is the work most teams skip. A practical minimum is 200–300 labeled question-chunk pairs, sampled from your actual query distribution.

```
# Python
# Retrieval evaluation with a labeled dataset
# Uses pgvector for retrieval and a simple evaluation harness

import psycopg2
import numpy as np
from openai import OpenAI
from dataclasses import dataclass
from typing import List, Dict

client = OpenAI()

@dataclass
class RetrievalEvalSample:
    query: str
    relevant_chunk_ids: List[int] # Ground truth chunk IDs

def embed_query(text: str) -> List[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return response.data[0].embedding

def retrieve_chunks(
    conn,
    query_embedding: List[float],
    k: int = 5
) -> List[int]:
    """Returns ordered list of chunk IDs from pgvector search."""
    embedding_str = "[" + ",".join(map(str, query_embedding)) + "]"
    with conn.cursor() as cur:
        cur.execute("""
            SELECT chunk_id
            FROM document_chunks
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """, (embedding_str, k))
    return [row[0] for row in cur.fetchall()]
```

```

def precision_at_k(
    retrieved: List[int],
    relevant: List[int],
    k: int
) -> float:
    retrieved_k = retrieved[:k]
    hits = len(set(retrieved_k) & set(relevant))
    return hits / k if k > 0 else 0.0

def recall_at_k(
    retrieved: List[int],
    relevant: List[int],
    k: int
) -> float:
    retrieved_k = retrieved[:k]
    hits = len(set(retrieved_k) & set(relevant))
    return hits / len(relevant) if relevant else 0.0

def reciprocal_rank(
    retrieved: List[int],
    relevant: List[int]
) -> float:
    for rank, chunk_id in enumerate(retrieved, start=1):
        if chunk_id in relevant:
            return 1.0 / rank
    return 0.0

def evaluate_retrieval(
    conn,
    eval_samples: List[RetrievalEvalSample],
    k: int = 5
) -> Dict[str, float]:
    precisions, recalls, rrs = [], [], []

    for sample in eval_samples:
        embedding = embed_query(sample.query)
        retrieved = retrieve_chunks(conn, embedding, k=k)

        precisions.append(
            precision_at_k(retrieved, sample.relevant_chunk_ids, k)
        )
        recalls.append(
            recall_at_k(retrieved, sample.relevant_chunk_ids, k)
        )
        rrs.append(
            reciprocal_rank(retrieved, sample.relevant_chunk_ids)
        )

    return {
        f"precision@{k}": float(np.mean(precisions)),
        f"recall@{k}": float(np.mean(recalls)),
        "mrr": float(np.mean(rrs)),
        "num_samples": len(eval_samples)
    }

# Example usage
conn = psycopg2.connect("postgresql://user:pass@localhost:5432/ragdb")

eval_samples = [
    RetrievalEvalSample(
        query="What is the maximum connection pool size for pgBouncer?",
        relevant_chunk_ids=[1042, 1043]
    ),
    RetrievalEvalSample(
        query="How does WAL archiving work in PostgreSQL?",
        relevant_chunk_ids=[287, 288, 295]
    ),
]

results = evaluate_retrieval(conn, eval_samples, k=5)

```

```
print(results)
# Example output:
# {'precision@5': 0.52, 'recall@5': 0.81, 'mrr': 0.74, 'num_samples': 2}
```

What numbers to target: For domain-specific enterprise RAG, Precision@5 above 0.55 and Recall@5 above 0.75 are reasonable starting thresholds before the retrieval layer is considered production-ready. MRR above 0.70 means the best chunk is usually in the top two positions, which matters because most LLMs weight earlier context more heavily.

When retrieval scores are low, the diagnostic path mirrors index tuning: check embedding model alignment with your query distribution, examine chunk size and overlap, verify that the index type (HNSW vs IVFFlat) matches your `ef_search/nprobe` settings, and consider hybrid search (vector + BM25) for queries with strong keyword signals.

Generation Metrics: Faithfulness, Relevance, and Hallucination Detection

Once you’ve confirmed the retrieval layer is surfacing good chunks, the next question is what the LLM does with them. Generation quality breaks into three independent concerns:

Faithfulness (also called groundedness): Every factual claim in the answer should be supported by the retrieved context. A faithfulness score of 1.0 means all claims trace back to the provided chunks. Below 0.80 in production is a warning sign for hallucination risk.

Answer relevance: Does the answer actually address the question? A model can produce a faithful, well-grounded response that is technically responsive but misses what the user asked. This is measured separately from faithfulness.

Context relevance: Of the chunks passed to the LLM, what fraction were actually needed to produce the answer? High context relevance means retrieval is efficient. Low scores indicate noise in the retrieved context that increases token cost without improving output.

RAGAS (Retrieval Augmented Generation Assessment) is the most widely adopted framework for computing these metrics programmatically. It uses an LLM as a judge—a common pattern where one model evaluates the output of another—which introduces its own reliability concerns but scales well compared to human evaluation.

```
# Python
# RAGAS evaluation pipeline for a RAG system built with LangChain + pgvector
# pip install ragas langchain langchain-openai psycpg2-binary

from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_recall,
    context_precision,
)
from ragas.metrics.critique import harmfulness
from datasets import Dataset
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_postgres import PGVector
from langchain.chains import RetrievalQA
import os

# Initialize the vector store (pgvector via LangChain)
```

```

CONNECTION_STRING = os.environ["PG_CONNECTION_STRING"]
COLLECTION_NAME = "docs_collection"

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = PGVector(
    embeddings=embeddings,
    collection_name=COLLECTION_NAME,
    connection=CONNECTION_STRING,
)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)

# Evaluation questions with ground truth answers
test_questions = [
    {
        "question": "What are the ACID properties in PostgreSQL?",
        "ground_truth": (
            "ACID stands for Atomicity, Consistency, Isolation, and Durability. "
            "PostgreSQL enforces all four properties through its MVCC implementation "
            "and write-ahead logging."
        )
    },
    {
        "question": "How does connection pooling improve database performance?",
        "ground_truth": (
            "Connection pooling reuses existing database connections instead of "
            "creating new ones for each client request, reducing overhead from "
            "connection establishment and reducing memory usage on the database server."
        )
    },
]

# Run the RAG pipeline and collect inputs for RAGAS
questions, answers, contexts, ground_truths = [], [], [], []

for item in test_questions:
    result = qa_chain.invoke({"query": item["question"]})
    questions.append(item["question"])
    answers.append(result["result"])
    contexts.append([
        doc.page_content for doc in result["source_documents"]
    ])
    ground_truths.append(item["ground_truth"])

# Build the RAGAS dataset
eval_dataset = Dataset.from_dict({
    "question": questions,
    "answer": answers,
    "contexts": contexts,
    "ground_truth": ground_truths,
})

# Run RAGAS evaluation
results = evaluate(
    eval_dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_recall,
        context_precision,
    ],
    llm=ChatOpenAI(model="gpt-4o-mini"),
    embeddings=embeddings,
)

```

```
print(results)
# Example output:
# {'faithfulness': 0.87, 'answer_relevancy': 0.91,
#  'context_recall': 0.78, 'context_precision': 0.65}

df = results.to_pandas()
df.to_csv("ragas_eval_results.csv", index=False)
```

Interpreting RAGAS scores in production terms: Faithfulness below 0.75 indicates the LLM is introducing information not present in the retrieved chunks—reduce temperature, tighten the system prompt, or increase retrieval precision. Context precision below 0.60 means roughly 40% of retrieved chunks aren’t contributing to the answer—a retrieval efficiency problem that inflates latency and cost without quality benefit. Answer relevancy below 0.80 usually points to a prompt engineering issue or query transformation gap.

The LLM-as-judge pattern has a known limitation: judge models share biases with the models being evaluated. For high-stakes applications, supplement RAGAS with human evaluation on a stratified sample (typically 5–10% of the full evaluation set). The stratification should include edge cases: short queries, multi-hop questions, queries with no answer in the corpus, and queries that span multiple documents.

End-to-End Correctness: Answer Quality at Scale

Faithfulness and relevance are process metrics—they measure how well the pipeline executes its steps. Answer correctness is the outcome metric: did the system produce an answer that is factually accurate relative to a known reference?

Exact match is the strictest form: the answer string matches the reference exactly. Useful for structured outputs (dates, product codes, numeric values) but too brittle for natural language responses.

Semantic similarity uses embedding cosine similarity between the generated answer and the reference answer. Scores above 0.85 typically indicate strong alignment. This is fast to compute at scale but can be fooled by paraphrased wrong answers.

LLM-based scoring submits the question, reference answer, and generated answer to a judge model with a structured rubric. This is the most accurate automated method but adds latency and cost. Frameworks like DeepEval, TruLens, and Athina AI provide pre-built judge prompts and scoring rubrics.

```
# Python
# Answer correctness evaluation using DeepEval
# pip install deepeval

from deepeval import evaluate
from deepeval.metrics import (
    AnswerRelevancyMetric,
    FaithfulnessMetric,
    ContextualRecallMetric,
    ContextualPrecisionMetric,
    HallucinationMetric,
)
from deepeval.test_case import LLMTestCase
import os
```



```

os.environ["OPENAI_API_KEY"] = os.environ["OPENAI_API_KEY"]

# A single test case mirrors the structure of a RAGAS sample
# but DeepEval gives finer control over scoring thresholds

test_case = LLMTestCase(
    input="What isolation levels does PostgreSQL support?",
    actual_output=(
        "PostgreSQL supports four isolation levels: "
        "Read Uncommitted, Read Committed, Repeatable Read, and Serializable. "
        "In practice, Read Uncommitted behaves like Read Committed in PostgreSQL."
    ),
    expected_output=(
        "PostgreSQL supports Read Committed, Repeatable Read, and Serializable. "
        "Read Uncommitted is accepted but treated as Read Committed."
    ),
    retrieval_context=[
        "PostgreSQL implements four transaction isolation levels as defined "
        "by the SQL standard: Read Uncommitted, Read Committed, Repeatable Read, "
        "and Serializable. PostgreSQL's Read Uncommitted mode behaves like "
        "Read Committed.",
        "The default isolation level in PostgreSQL is Read Committed.",
    ]
)

# Define metrics with explicit thresholds
metrics = [
    AnswerRelevancyMetric(threshold=0.80, model="gpt-4o-mini"),
    FaithfulnessMetric(threshold=0.80, model="gpt-4o-mini"),
    HallucinationMetric(threshold=0.20, model="gpt-4o-mini"),
    ContextualRecallMetric(threshold=0.75, model="gpt-4o-mini"),
    ContextualPrecisionMetric(threshold=0.60, model="gpt-4o-mini"),
]

# Evaluate and print per-metric verdicts
for metric in metrics:
    metric.measure(test_case)
    print(
        f"{metric.__class__.__name__}: "
        f"score={metric.score:.3f}, "
        f"passed={metric.is_successful()}, "
        f"reason={metric.reason}"
    )

```

Connecting evaluation to CI/CD: Evaluation suites should run on every pipeline change, just as query regression tests run on schema changes. The evaluation dataset is a versioned artifact in your repository. A PR that introduces a new embedding model or changes chunk size should trigger an evaluation run, and the results should gate deployment. This is the same discipline as maintaining a query performance baseline in a DBA workflow—the tooling is different, but the principle is identical.

The cost of automated LLM-based evaluation is real: evaluating 500 test cases with GPT-4o-mini as judge costs roughly \$0.15–0.40 per full run depending on context length. This is low enough to run on every deploy for most teams, but you should budget for it explicitly when setting up the pipeline.

Storing and Trending Evaluation Results in PostgreSQL

Metrics measured once tell you where you stand today. Metrics tracked over time tell you whether changes are helping or degrading quality—the same distinction between a point-in-time EXPLAIN

and a continuous slow query log. Storing evaluation results in a relational database enables trend analysis, regression detection, and correlation with system changes.

```
-- PostgreSQL / pgvector
-- Schema for tracking RAG evaluation results over time

CREATE TABLE rag_pipeline_versions (
  version_id      SERIAL PRIMARY KEY,
  version_tag     TEXT NOT NULL UNIQUE,      -- e.g. 'v2.4.1'
  embedding_model TEXT NOT NULL,
  llm_model       TEXT NOT NULL,
  chunk_size      INTEGER,
  chunk_overlap   INTEGER,
  retrieval_k      INTEGER,
  index_type      TEXT,                    -- 'hnsw' or 'ivfflat'
  deployed_at     TIMESTAMPTZ DEFAULT now(),
  notes          TEXT
);

CREATE TABLE rag_eval_runs (
  run_id          SERIAL PRIMARY KEY,
  version_id      INTEGER REFERENCES rag_pipeline_versions(version_id),
  eval_dataset_tag TEXT NOT NULL,          -- versioned eval set name
  run_at         TIMESTAMPTZ DEFAULT now(),
  num_samples     INTEGER NOT NULL,
  -- Retrieval metrics
  precision_at_5  NUMERIC(5,4),
  recall_at_5    NUMERIC(5,4),
  mrr            NUMERIC(5,4),
  -- Generation metrics (RAGAS)
  faithfulness   NUMERIC(5,4),
  answer_relevancy NUMERIC(5,4),
  context_precision NUMERIC(5,4),
  context_recall NUMERIC(5,4),
  -- Correctness
  hallucination_rate NUMERIC(5,4),
  -- Operational
  avg_latency_ms  NUMERIC(8,2),
  p99_latency_ms  NUMERIC(8,2),
  avg_tokens_per_query INTEGER,
  eval_cost_usd   NUMERIC(8,4)
);

CREATE TABLE rag_eval_sample_results (
  sample_id      SERIAL PRIMARY KEY,
  run_id         INTEGER REFERENCES rag_eval_runs(run_id),
  question       TEXT NOT NULL,
  retrieved_chunk_ids INTEGER[],
  generated_answer TEXT,
  faithfulness   NUMERIC(5,4),
  answer_relevancy NUMERIC(5,4),
  hallucinated   BOOLEAN,
  latency_ms     INTEGER
);

-- Index for trend queries
CREATE INDEX idx_eval_runs_version ON rag_eval_runs(version_id, run_at);
CREATE INDEX idx_sample_results_run ON rag_eval_sample_results(run_id);

-- Query: metric trend across pipeline versions
SELECT
  v.version_tag,
  v.embedding_model,
  v.chunk_size,
  r.run_at,
  r.faithfulness,
  r.answer_relevancy,
  r.precision_at_5,
  r.recall_at_5,
```

```

        r.mrr,
        r.avg_latency_ms
FROM rag_eval_runs r
JOIN rag_pipeline_versions v ON r.version_id = v.version_id
WHERE r.eval_dataset_tag = 'postgres-docs-v1'
ORDER BY r.run_at DESC
LIMIT 20;

-- Query: find regressions - versions where faithfulness dropped > 0.05
WITH version_scores AS (
    SELECT
        v.version_tag,
        r.run_at,
        r.faithfulness,
        LAG(r.faithfulness) OVER (
            ORDER BY r.run_at
        ) AS prev_faithfulness
    FROM rag_eval_runs r
    JOIN rag_pipeline_versions v ON r.version_id = v.version_id
    WHERE r.eval_dataset_tag = 'postgres-docs-v1'
)
SELECT
    version_tag,
    run_at,
    faithfulness,
    prev_faithfulness,
    faithfulness - prev_faithfulness AS delta
FROM version_scores
WHERE faithfulness - prev_faithfulness < -0.05
ORDER BY run_at DESC;

-- Query: worst-performing samples in the latest run
SELECT
    s.question,
    s.faithfulness,
    s.answer_relevancy,
    s.hallucinated,
    s.latency_ms
FROM rag_eval_sample_results s
JOIN rag_eval_runs r ON s.run_id = r.run_id
WHERE r.run_id = (
    SELECT run_id FROM rag_eval_runs
    ORDER BY run_at DESC
    LIMIT 1
)
ORDER BY s.faithfulness ASC
LIMIT 20;

```

This schema treats evaluation runs as first-class records—versioned, queryable, and correlated with pipeline configuration. The `rag_pipeline_versions` table is the equivalent of a change log: every modification to embedding model, chunk size, or retrieval parameters creates a new record, making it possible to bisect regressions precisely.

The `rag_eval_sample_results` table enables failure analysis at the sample level. Sorted by lowest faithfulness, it surfaces exactly which questions the current pipeline handles poorly—the same way sorting a slow query log by total execution time surfaces the highest-impact optimization targets.

Production Monitoring: Online Metrics and Feedback Loops

Offline evaluation on a labeled dataset tells you how the pipeline performs on known questions. Production traffic contains questions you’ve never seen, spanning query types, user populations,

and document states that evolve over time. Online monitoring closes that gap.

Implicit feedback signals are the lowest-friction source of production quality data. Thumbs up/down ratings, answer regeneration requests, session abandonment after a RAG response, and follow-up clarification questions all carry quality signal. These signals are noisy individually but aggregate into reliable trend indicators.

Shadow scoring runs the evaluation metrics asynchronously on production traffic. For each query, the system logs the question, retrieved chunks, and generated answer to a queue. A background worker submits samples to the RAGAS or DeepEval scorer and writes results to the evaluation schema. This adds zero latency to the user path while providing continuous quality data.

```
# Python
# Async shadow scoring with Celery + Redis
# Logs production queries and evaluates them in the background
# pip install celery redis ragas langchain-openai psycpg2-binary

from celery import Celery
import psycpg2
import json
import os
from datetime import datetime
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from datasets import Dataset
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

app = Celery(
    "shadow_scorer",
    broker=os.environ["REDIS_URL"],
    backend=os.environ["REDIS_URL"]
)

PG_DSN = os.environ["PG_CONNECTION_STRING"]

@app.task(bind=True, max_retries=3)
def score_production_sample(
    self,
    question: str,
    answer: str,
    contexts: list,
    run_id: int
):
    """
    Background task: score a single production query and persist results.
    Called after the user receives their response-zero latency impact.
    """
    try:
        dataset = Dataset.from_dict({
            "question": [question],
            "answer": [answer],
            "contexts": [contexts],
            "ground_truth": [""], # not available for live traffic
        })

        llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
        embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

        # Only faithfulness and answer_relevancy work without ground truth
        result = evaluate(
            dataset,
            metrics=[faithfulness, answer_relevancy],
            llm=llm,
            embeddings=embeddings,
        )

        row = result.to_pandas().iloc[0]
```

```

conn = psycopg2.connect(PG_DSN)
with conn:
    with conn.cursor() as cur:
        cur.execute("""
            INSERT INTO rag_eval_sample_results
            (run_id, question, generated_answer,
             faithfulness, answer_relevancy, hallucinated)
            VALUES (%s, %s, %s, %s, %s, %s)
        """, (
            run_id,
            question,
            answer,
            float(row["faithfulness"]),
            float(row["answer_relevancy"]),
            float(row["faithfulness"]) < 0.70,
        ))
    conn.close()

except Exception as exc:
    raise self.retry(exc=exc, countdown=60)

def handle_rag_request(question: str, run_id: int):
    """
    Main request handler. Returns answer immediately; scoring is async.
    """
    # (retrieval and generation omitted-same as prior examples)
    retrieved_docs = [] # populate from vector search
    answer = "" # populate from LLM
    contexts = [d.page_content for d in retrieved_docs]

    # Fire-and-forget scoring task
    score_production_sample.delay(question, answer, contexts, run_id)

    return answer

```

Drift detection is the production monitoring concern that goes beyond individual query scores. As your document corpus updates, as user query patterns shift, or as the underlying LLM is updated by its provider, quality metrics drift. Tracking a rolling 7-day average of faithfulness, answer relevancy, and retrieval precision against the baseline established at deployment lets you detect drift before users file support tickets.

```

-- PostgreSQL
-- Rolling 7-day average metrics for drift detection
-- Run this as a scheduled query or materialized view refresh

CREATE MATERIALIZED VIEW rag_quality_rolling_7d AS
SELECT
    date_trunc('day', r.run_at) AS eval_day,
    AVG(s.faithfulness) AS avg_faithfulness,
    AVG(s.answer_relevancy) AS avg_answer_relevancy,
    SUM(CASE WHEN s.hallucinated THEN 1 ELSE 0 END)::NUMERIC
    / COUNT(*) AS hallucination_rate,
    COUNT(*) AS sample_count
FROM rag_eval_sample_results s
JOIN rag_eval_runs r ON s.run_id = r.run_id
WHERE r.run_at >= now() - INTERVAL '90 days'
GROUP BY date_trunc('day', r.run_at)
ORDER BY eval_day;

-- Refresh on a schedule (pg_cron or external scheduler)
-- SELECT cron.schedule('refresh-rag-quality', '0 * * * *',
-- 'REFRESH MATERIALIZED VIEW rag_quality_rolling_7d');

-- Alert query: day where faithfulness dropped below baseline - 0.08
WITH baseline AS (
    SELECT AVG(avg_faithfulness) AS baseline_faith

```

```

FROM rag_quality_rolling_7d
WHERE eval_day BETWEEN
    (SELECT MIN(eval_day) FROM rag_quality_rolling_7d)
    AND (SELECT MIN(eval_day) FROM rag_quality_rolling_7d) + INTERVAL '14 days'
)
SELECT
    q.eval_day,
    q.avg_faithfulness,
    b.baseline_faith,
    q.avg_faithfulness - b.baseline_faith AS drift
FROM rag_quality_rolling_7d q
CROSS JOIN baseline b
WHERE q.avg_faithfulness < b.baseline_faith - 0.08
ORDER BY q.eval_day DESC;

```

Building an Evaluation-Driven Development Loop

The metrics and tooling described in this chapter only create value when they're wired into the development workflow rather than run as one-off checks. The pattern that works in practice mirrors database change management:

- 1. Freeze an evaluation dataset before changing anything.** Before modifying chunk size, swapping embedding models, or adjusting retrieval K, run a baseline evaluation and record the run in `rag_eval_runs`. This baseline is your before-state. Without it, you can't compute delta.
- 2. Make exactly one change per experiment.** Changing the embedding model and the chunk size simultaneously makes it impossible to attribute score changes. Treat each configuration axis as a controlled variable—the same discipline as changing one query hint at a time during plan tuning.
- 3. Use statistical significance for borderline changes.** A faithfulness delta of 0.02 across 50 samples is noise. Across 500 samples it may be signal. Bootstrap confidence intervals on your evaluation metrics before concluding that a change improved or degraded quality.

```

# Python
# Bootstrap confidence interval for a metric delta between two eval runs
import numpy as np
from typing import List

def bootstrap_ci(
    scores: List[float],
    n_bootstrap: int = 2000,
    ci: float = 0.95
) -> tuple:
    """Returns (mean, lower_bound, upper_bound) for a list of per-sample scores."""
    rng = np.random.default_rng(seed=42)
    boot_means = [
        np.mean(rng.choice(scores, size=len(scores), replace=True))
        for _ in range(n_bootstrap)
    ]
    lower = np.percentile(boot_means, (1 - ci) / 2 * 100)
    upper = np.percentile(boot_means, (1 + ci) / 2 * 100)
    return float(np.mean(scores)), float(lower), float(upper)

def compare_eval_runs(
    baseline_scores: List[float],
    candidate_scores: List[float],
    metric_name: str
):
    b_mean, b_lo, b_hi = bootstrap_ci(baseline_scores)
    c_mean, c_lo, c_hi = bootstrap_ci(candidate_scores)

```

```
delta = c_mean - b_mean

# If the candidate's CI lower bound exceeds the baseline mean,
# the improvement is statistically reliable at 95% confidence.
significant = c_lo > b_mean

print(f"{metric_name}:")
print(f"  Baseline: {b_mean:.4f} [{b_lo:.4f}, {b_hi:.4f}]")
print(f"  Candidate: {c_mean:.4f} [{c_lo:.4f}, {c_hi:.4f}]")
print(f"  Delta:      {delta:+.4f}  Significant: {significant}")

# Example
baseline_faithfulness = [0.82, 0.75, 0.91, 0.68, 0.88] # expand to 200+ in practice
candidate_faithfulness = [0.85, 0.79, 0.93, 0.74, 0.90]

compare_eval_runs(baseline_faithfulness, candidate_faithfulness, "faithfulness")
```

4. Gate deployments on metric thresholds. A pipeline version that degrades faithfulness by more than 0.03 or drops recall below 0.70 should not ship without explicit review. Encoding these thresholds in the CI pipeline treats RAG quality with the same rigor as query regression tests—both are quantitative, both are automated, and both block problematic changes before they reach production.

5. Review failure cases, not just aggregate scores. Aggregate metrics tell you the direction of change; sample-level failure analysis tells you why. The `rag_eval_sample_results` table sorted by lowest faithfulness is your equivalent of the slow query log: the worst-performing queries reveal systematic patterns—query types where chunking fails, topics where the embedding model underperforms, or question structures that cause the LLM to ignore relevant context.

A Note on Evaluation Dataset Maintenance

Evaluation datasets decay. As the document corpus updates, labeled relevant chunks become stale: a chunk that was the ground-truth answer to a question may be superseded by an updated document, and a new evaluation sample built against the old corpus will give misleading results. Treating the evaluation dataset as a versioned, maintained artifact—not a static file created once—is the difference between evaluation that informs decisions and evaluation that provides false confidence.

Tag each evaluation dataset version in `rag_eval_runs.eval_dataset_tag`. When the corpus undergoes significant updates, schedule a dataset review: re-run the labeled samples against the new corpus to verify that ground-truth chunk IDs still resolve to current content, and retire or update samples whose ground truth has changed. This is maintenance work, but it is the maintenance that keeps the measurement framework honest.

Key Takeaways

- **RAG quality has three independent failure modes—retrieval, generation, and correctness—each requiring its own metrics.** Optimizing only end-to-end accuracy without decomposing the pipeline makes root cause analysis slow and unreliable.

- **Precision@K, Recall@K, and MRR measure the retrieval layer** against a labeled evaluation set. Building that labeled set—200 to 300 question-chunk pairs at minimum—is the prerequisite step most teams skip, and skipping it means flying blind on the component that most directly determines answer quality.
 - **RAGAS and DeepEval automate generation quality scoring** using LLM-as-judge patterns. Faithfulness, answer relevancy, context precision, and context recall each signal a different failure mode; treating them as a diagnostic suite rather than a single composite score localizes problems faster.
 - **Storing evaluation results in a relational schema—versioned by pipeline configuration—enables regression detection and trend analysis** with the same SQL queries used for any other time-series data. The `rag_eval_runs` and `rag_eval_sample_results` tables are the RAG equivalent of a slow query log: queryable, actionable, and essential for production discipline.
 - **Shadow scoring and rolling drift detection extend evaluation from offline snapshots to continuous production monitoring.** Firing evaluation tasks asynchronously on live traffic, then aggregating results with a rolling window query, turns quality measurement from a periodic event into an always-on signal—closing the feedback loop between production behavior and pipeline improvements.
-

Chapter 28: Indexing Pipelines: Keeping Vectors Fresh

Vector search breaks down the moment your index falls out of sync with your data. Unlike a B-tree index that updates atomically with every INSERT or UPDATE, most vector indexes are built in bulk, degrade under heavy write load, and require explicit maintenance to stay performant. For DBAs accustomed to indexes that maintain themselves, this behavioral difference is one of the sharpest adjustments when moving into vector infrastructure. This chapter covers the full lifecycle of vector indexing pipelines: how vectors get generated, how they land in the index, what happens to freshness under write pressure, how to detect and recover from drift, and how to architect pipelines that keep production systems current without disrupting query performance.

The Vector Indexing Lifecycle

Before building a pipeline, you need a clear model of what a “vector indexing pipeline” actually does and where it differs from classical index maintenance.

In a relational system, an index is a derived structure built from column values that already exist in the row. The database engine manages it transparently. When you INSERT a row, the B-tree updates. When you UPDATE a column, the old entry is removed and a new one inserted. This is synchronous and transactional by default.

Vector indexing has a different shape:

1. **Source data changes** — a document is added, updated, or deleted in your primary store (PostgreSQL, S3, a CMS, a product catalog).
2. **Chunking** — the document is split into semantic units. This is a non-trivial transformation with real parameters: chunk size, overlap, split strategy.
3. **Embedding generation** — each chunk is sent to an embedding model (OpenAI, Cohere, a local sentence-transformer). This takes time and costs money.
4. **Upsert to vector store** — the resulting vector and its metadata are written to the vector database.
5. **Index rebuild or incremental update** — depending on the vector store, the ANN index may need to be explicitly rebuilt or optimized after writes accumulate.

Steps 2 through 5 are asynchronous in most production systems. The gap between step 1 and step 5 defines your indexing latency — and for freshness-sensitive applications, this gap is what you must actively manage.

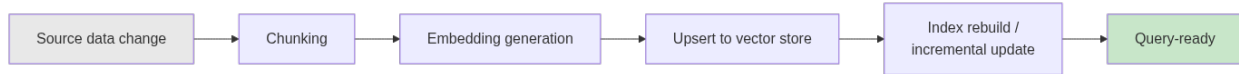


Figure 18: Steps 2–5 run asynchronously in most production systems; the gap between the source write (A) and a query-ready index (F) is the indexing latency you must actively manage.

The freshness-performance tradeoff mirrors something DBAs already know from materialized views: the more aggressively you refresh, the closer to current your results, but the higher the maintenance overhead. HNSW indexes, for example, are typically immutable after construction in Milvus and Qdrant’s older modes. Pinecone handles upserts incrementally but merges them in background compaction. pgvector’s HNSW implementation accepts live inserts but degrades in graph quality as the unsorted segment grows.

Think of the unindexed buffer in a vector store the way you’d think of the unmerged delta in a columnstore index. Queries still work — they just fall back to brute force for that segment, which costs performance.

Building the Embedding Generation Pipeline

The first step toward fresh vectors is an efficient, reliable embedding pipeline. Whether you’re using a managed API or a local model, the architecture patterns are the same.

Chunking strategy matters more than most teams realize. The semantic unit you index determines the granularity of retrieval. Too large: you dilute signal and waste tokens. Too small: you lose context and increase index size. A good starting heuristic for RAG applications is 256–512 tokens per chunk with a 10–20% overlap. For structured content like product descriptions or database rows, you often skip chunking entirely.

Here is a complete pipeline using LangChain with OpenAI embeddings, writing into pgvector:

```

# Python
import os
import psycopg2
from psycopg2.extras import execute_values
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain.docstore.document import Document

# Configuration
DB_CONN = os.environ["PG_CONN_STRING"]
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
EMBEDDING_MODEL = "text-embedding-3-small"
CHUNK_SIZE = 400
CHUNK_OVERLAP = 40

embeddings_client = OpenAIEmbeddings(
    model=EMBEDDING_MODEL,
    openai_api_key=OPENAI_API_KEY
)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    separators=["\n\n", "\n", ".", " ", ""]
)

```

```

def index_document(doc_id: str, content: str, metadata: dict, conn) -> int:
    """
    Chunk, embed, and upsert a document into pgvector.
    Returns the number of chunks written.
    """
    # 1. Chunk
    chunks = splitter.split_text(content)
    if not chunks:
        return 0

    # 2. Embed - batch call to reduce API round trips
    vectors = embeddings_client.embed_documents(chunks)

    # 3. Upsert - delete old chunks, insert new ones atomically
    with conn.cursor() as cur:
        # Remove stale chunks from a previous version of this document
        cur.execute(
            "DELETE FROM document_chunks WHERE doc_id = %s",
            (doc_id,)
        )

        rows = [
            (
                doc_id,
                i,
                chunks[i],
                vectors[i],          # pgvector accepts Python lists directly
                metadata.get("source", ""),
                metadata.get("updated_at", None)
            )
            for i in range(len(chunks))
        ]

        execute_values(
            cur,
            """
            INSERT INTO document_chunks
            (doc_id, chunk_index, chunk_text, embedding, source, updated_at)
            VALUES %s
            """,
            rows,
            template="(%s, %s, %s, %s::vector, %s, %s)"
        )

    conn.commit()
    return len(chunks)

# Example invocation
conn = psycopg2.connect(DB_CONN)

doc_count = index_document(
    doc_id="product-42",
    content="The Apex Pro running shoe features carbon fiber plate technology...",
    metadata={"source": "product_catalog", "updated_at": "2024-11-01"},
    conn=conn
)

print(f"Indexed {doc_count} chunks for product-42")
conn.close()

```

The schema to support this pipeline:

```

-- PostgreSQL / pgvector
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
    id          BIGSERIAL PRIMARY KEY,

```

```

doc_id      TEXT          NOT NULL,
chunk_index INT          NOT NULL,
chunk_text  TEXT          NOT NULL,
embedding   vector(1536),  -- text-embedding-3-small dimension
source      TEXT,
updated_at  TIMESTAMPTZ,
indexed_at  TIMESTAMPTZ DEFAULT NOW(),
UNIQUE (doc_id, chunk_index)
);

-- HNSW index for ANN search
CREATE INDEX idx_chunks_embedding_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Supporting indexes for metadata filtering
CREATE INDEX idx_chunks_doc_id ON document_chunks (doc_id);
CREATE INDEX idx_chunks_source ON document_chunks (source);
CREATE INDEX idx_chunks_indexed_at ON document_chunks (indexed_at);

```

Batching is the single biggest performance lever in the embedding step. The OpenAI `text-embedding-3-small` model accepts up to 2048 inputs per API call. Calling it one chunk at a time for a 10,000-document backfill is roughly 100x slower than batching at 100 chunks per call and also drives your cost per token higher due to request overhead. LangChain's `embed_documents()` handles batching internally, but if you're writing direct API calls, batch explicitly.

For high-volume pipelines, the embedding step is also where you want a retry layer with exponential backoff. Rate limits from OpenAI at tier 1 are 500 RPM for embeddings. Tier 3 raises this to 5,000 RPM. Design your pipeline with configurable concurrency limits so you can tune throughput as your API tier changes.

Change Data Capture for Continuous Indexing

Backfill pipelines handle the initial load. The harder problem is keeping the index current as source data changes continuously. The right pattern here is Change Data Capture — a technique DBAs know well from replication, adapted for vector pipeline use.

Approach 1: Polling with a watermark column. The simplest approach. Maintain a `last_indexed_at` watermark, and on each pipeline run, fetch rows where `updated_at > last_indexed_at`. This works well when your source system has a reliable `updated_at` column and deletes are handled via soft delete (a `deleted_at` column rather than physical removal).

```

# Python
import time
import psycopg2
from datetime import datetime, timezone

SOURCE_CONN = os.environ["SOURCE_PG_CONN"]
VECTOR_CONN = os.environ["VECTOR_PG_CONN"]
POLL_INTERVAL_SECONDS = 30
BATCH_SIZE = 200

def get_watermark(vector_conn) -> datetime:
    """Read the last successfully indexed timestamp from a state table."""
    with vector_conn.cursor() as cur:
        cur.execute(
            "SELECT last_indexed_at FROM indexer_state WHERE pipeline_id = 'products'"

```

```

    )
    row = cur.fetchone()
    if row:
        return row[0]
    # First run: index everything from the beginning
    return datetime(2000, 1, 1, tzinfo=timezone.utc)

def save_watermark(vector_conn, ts: datetime):
    with vector_conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO indexer_state (pipeline_id, last_indexed_at)
            VALUES ('products', %s)
            ON CONFLICT (pipeline_id)
            DO UPDATE SET last_indexed_at = EXCLUDED.last_indexed_at
            """,
            (ts,)
        )
    vector_conn.commit()

def poll_and_index(source_conn, vector_conn):
    watermark = get_watermark(vector_conn)
    batch_start = datetime.now(timezone.utc)

    with source_conn.cursor() as cur:
        cur.execute(
            """
            SELECT product_id, description, category, updated_at
            FROM products
            WHERE updated_at > %s
            AND deleted_at IS NULL
            ORDER BY updated_at ASC
            LIMIT %s
            """,
            (watermark, BATCH_SIZE)
        )
        rows = cur.fetchall()

    if not rows:
        return 0

    processed = 0
    for product_id, description, category, updated_at in rows:
        index_document(
            doc_id=f"product-{product_id}",
            content=description,
            metadata={"source": "products", "category": category,
                    "updated_at": updated_at.isoformat()},
            conn=vector_conn
        )
        processed += 1

    # Advance watermark to the latest timestamp we processed
    latest_ts = rows[-1][3]
    save_watermark(vector_conn, latest_ts)
    return processed

# Continuous poll loop
source_conn = psycopg2.connect(SOURCE_CONN)
vector_conn = psycopg2.connect(VECTOR_CONN)

while True:
    n = poll_and_index(source_conn, vector_conn)
    print(f"{datetime.now().isoformat()} - indexed {n} documents")
    time.sleep(POLL_INTERVAL_SECONDS)

```

Approach 2: PostgreSQL logical replication via Debezium. For sub-minute freshness requirements, polling every 30 seconds doesn't cut it. Logical replication captures every row-level change as an event and can drive your embedding pipeline in near-real-time. The standard production setup is: PostgreSQL WAL → Debezium → Kafka → vector indexing consumer.

The vector indexing consumer listens to Kafka topics and processes INSERT, UPDATE, and DELETE events:

```
# Python - Kafka consumer for CDC-driven vector indexing
from kafka import KafkaConsumer
import json
import logging

KAFKA_BROKER = os.environ["KAFKA_BROKER"]
KAFKA_TOPIC = "postgres.public.products" # Debezium topic naming convention

consumer = KafkaConsumer(
    KAFKA_TOPIC,
    bootstrap_servers=KAFKA_BROKER,
    value_deserializer=lambda m: json.loads(m.decode("utf-8")),
    group_id="vector-indexer-products",
    auto_offset_reset="earliest",
    enable_auto_commit=False # Manual commit for exactly-once semantics
)

vector_conn = psycopg2.connect(VECTOR_CONN)

for message in consumer:
    event = message.value
    op = event.get("op") # 'c' = create, 'u' = update, 'd' = delete

    try:
        if op in ("c", "u"):
            after = event["after"]
            index_document(
                doc_id=f"product-{after['product_id']}",
                content=after["description"],
                metadata={
                    "source": "products",
                    "category": after.get("category", ""),
                    "updated_at": after.get("updated_at", "")
                },
                conn=vector_conn
            )
            logging.info(f"Indexed product {after['product_id']} (op={op})")

        elif op == "d":
            before = event["before"]
            doc_id = f"product-{before['product_id']}"
            with vector_conn.cursor() as cur:
                cur.execute(
                    "DELETE FROM document_chunks WHERE doc_id = %s",
                    (doc_id,)
                )
            vector_conn.commit()
            logging.info(f"Deleted product {before['product_id']} from index")

    consumer.commit()

except Exception as e:
    logging.error(f"Failed to process message offset {message.offset}: {e}")
    # Do not commit - will reprocess on restart
    # Add to dead-letter queue for inspection
```

The Debezium/Kafka approach gives you sub-5-second indexing latency from source commit to vector availability. The polling approach gives you 30–90 seconds depending on your interval. Choose based on what freshness lag your application can tolerate, and on your operational budget

— Kafka adds meaningful infrastructure overhead.

Index Optimization and Segment Management

Writing vectors into the database is only half the work. The other half is making sure the ANN index structure stays efficient as data accumulates.

pgvector HNSW behavior under writes. pgvector’s HNSW index is a live-update structure, unlike some vector stores that require full rebuilds. But it does not rebalance itself. Every new insert adds a node to the graph, which is correct but suboptimal compared to a graph built from the full dataset in bulk. After heavy write loads, it’s common to see a 10–15% degradation in recall at `ef_search=40` compared to a freshly built index. You can detect this by running recall benchmarks against a known ground-truth set.

The production pattern for pgvector is to **rebuild the index during a low-traffic window** after bulk ingestion:

```
-- PostgreSQL / pgvector
-- Check current index bloat and tuple counts
SELECT
    schemaname,
    tablename,
    n_live_tup,
    n_dead_tup,
    last_vacuum,
    last_analyze
FROM pg_stat_user_tables
WHERE tablename = 'document_chunks';

-- Check index size
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE tablename = 'document_chunks';

-- Rebuild HNSW index with updated ef_construction for larger datasets
-- This acquires ShareUpdateExclusiveLock - queries continue during build
-- Note: CONCURRENTLY statements cannot run inside an explicit transaction
-- block, so issue these as separate top-level statements, not wrapped
-- in BEGIN/COMMIT.

DROP INDEX CONCURRENTLY idx_chunks_embedding_hnsw;

CREATE INDEX CONCURRENTLY idx_chunks_embedding_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 128); -- Increased for better recall

-- Run VACUUM to reclaim space from deleted chunks
VACUUM ANALYZE document_chunks;
```

`CREATE INDEX CONCURRENTLY` in pgvector allows the index rebuild to run while queries continue against the old index — this is the same behavior as standard PostgreSQL `CONCURRENTLY` index builds. Plan for the rebuild to take roughly 1–2 seconds per 10,000 vectors on modern hardware with `m=16` and `ef_construction=128`.

Pinecone’s pod-based freshness behavior. In Pinecone, vectors written with `upsert()` are available for query within a few seconds, but they may not be included in the optimized ANN graph

immediately. Pinecone uses an internal freshed-segment architecture where new vectors are added to a “fresh” buffer that is searched with brute force while the background optimizer integrates them into the main index. This is transparent to the client but means your effective recall can be slightly lower in the minutes after a large bulk upsert.

For Pinecone, the key operational lever is the `describe_index_stats()` call, which tells you how many vectors are “fresh” (unmerged):

```
# Python - Pinecone freshness monitoring
from pinecone import Pinecone
import os

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index("product-embeddings")

stats = index.describe_index_stats()
print(f"Total vectors: {stats['total_vector_count']}")

for ns, ns_stats in stats.get("namespaces", {}).items():
    print(f"Namespace '{ns}': {ns_stats['vector_count']} vectors")

# Pinecone does not expose fresh/merged split directly via API
# Monitor query latency as a proxy: latency spike after bulk upsert
# indicates fresh buffer search overhead
# A reasonable freshness SLA: < 5 minutes to full index integration
# for upserts under 100K vectors
```

Weaviate segment management. Weaviate uses an HNSW index with an internal segment-and-memtable architecture similar to an LSM tree. New objects enter a memtable, which is flushed to immutable segments. Segments are periodically compacted. You can monitor this via the `/v1/nodes` endpoint:

```
# shell - Weaviate node and segment health check
curl -s http://localhost:8080/v1/nodes | python3 -m json.tool

# Check HNSW index statistics for a specific class
curl -s "http://localhost:8080/v1/schema/ProductChunk" | \
    python3 -c "import sys, json; d=json.load(sys.stdin); \
    print(json.dumps(d.get('vectorIndexConfig', {}), indent=2))"

# Force a sync (useful after bulk imports before running benchmarks)
curl -X POST http://localhost:8080/v1/schema/ProductChunk/tenants \
    -H "Content-Type: application/json"
```

For Weaviate batch imports, the key parameter is `batch_size`. Imports above 1,000 objects per batch begin to slow down due to HNSW graph lock contention. The recommended production batch size is 200–500 objects, with `num_workers=4` to pipeline the HTTP requests.

Handling Embedding Model Changes

One scenario that catches teams off guard: you upgrade your embedding model. Maybe you move from `text-embedding-ada-002` (1536 dimensions) to `text-embedding-3-large` (3072 dimensions), or you switch from OpenAI to a self-hosted `bge-large-en-v1.5` (1024 dimensions) to reduce costs. This is the vector database equivalent of changing a column’s data type — it invalidates every existing index entry.

You cannot mix embeddings from different models in the same vector index. The vector spaces are incompatible. A cosine similarity score between an `ada-002` vector and a `bge-large` vector

is meaningless. This is a hard constraint with no workaround.

The migration pattern is a blue-green reindex:

1. Create a new table or collection with the target model's dimension.
2. Reindex all existing documents into the new collection.
3. Dual-write new changes to both old and new collections during migration.
4. Cut over queries to the new collection once reindexing is complete and validated.
5. Decommission the old collection.

Here is the dual-write layer:

```
# Python - dual-write adapter for embedding model migration
from dataclasses import dataclass
from typing import List, Optional
import logging

@dataclass
class VectorWriteTarget:
    name: str
    dimension: int
    embeddings_client: object      # OpenAIEmbeddings or similar
    write_fn: callable            # Function that accepts (doc_id, chunks, vectors, metadata)
    is_primary: bool              # Primary target is queried; secondary is written-only

class DualWriteIndexer:
    """
    Writes to two vector targets simultaneously during a migration window.
    Failures on the secondary target are logged but do not fail the operation.
    """

    def __init__(self, primary: VectorWriteTarget, secondary: VectorWriteTarget):
        self.primary = primary
        self.secondary = secondary

    def index_document(self, doc_id: str, content: str, metadata: dict) -> bool:
        # Always embed separately - models have different vector spaces
        for target in [self.primary, self.secondary]:
            try:
                chunks = splitter.split_text(content)
                vectors = target.embeddings_client.embed_documents(chunks)
                target.write_fn(doc_id, chunks, vectors, metadata)
                logging.info(f"Wrote {len(chunks)} chunks to {target.name} for {doc_id}")
            except Exception as e:
                if target.is_primary:
                    # Primary failure is fatal - propagate up
                    raise RuntimeError(
                        f"Primary index write failed for {doc_id}: {e}"
                    ) from e
                else:
                    # Secondary failure is non-fatal - log and continue
                    logging.warning(
                        f"Secondary index write failed for {doc_id} "
                        f"(target={target.name}): {e}"
                    )

        return True

    # Validation: compare top-10 results between old and new index
    # before cutting over queries
    def validate_retrieval_parity(
        query: str,
        old_embeddings_client,
        new_embeddings_client,
        old_query_fn,
    )
```

```

    new_query_fn,
    top_k: int = 10
) -> float:
    """
    Returns Jaccard similarity of the doc_id sets returned by each index.
    A score above 0.7 generally indicates the new index is ready for cutover.
    """
    old_vec = old_embeddings_client.embed_query(query)
    new_vec = new_embeddings_client.embed_query(query)

    old_results = set(r["doc_id"] for r in old_query_fn(old_vec, top_k))
    new_results = set(r["doc_id"] for r in new_query_fn(new_vec, top_k))

    intersection = len(old_results & new_results)
    union = len(old_results | new_results)

    return intersection / union if union > 0 else 0.0

```

For large indexes — tens of millions of vectors — the reindex operation can take hours to days. At 1,000 embeddings per second (aggressive batching, high API tier), a 10M-vector index takes about 2.8 hours just for the embedding API calls, not counting write latency. Plan the migration window accordingly, and track progress with a simple counter against your document count.

—### Monitoring Index Freshness in Production

A pipeline that ingests without monitoring is a pipeline that silently drifts. You need instrumentation at every stage: how far behind is the indexer, how many documents are waiting, and whether the index quality has degraded.

The core metrics to track:

- **Indexing lag** — time between `updated_at` on the source record and `indexed_at` on the corresponding chunk. This is your pipeline latency.
- **Queue depth** — how many source records have `updated_at > last_indexed_at`. This tells you whether the pipeline is keeping up with the write rate.
- **Index-to-source count ratio** — the number of distinct `doc_id` values in your vector table divided by the number of documents in your source. A ratio below 1.0 means documents are missing from the index.
- **Recall over time** — periodic sampling against a held-out evaluation set. This catches both index degradation and embedding quality regressions.

Here is a monitoring query set for the pgvector pipeline:

```

-- PostgreSQL / pgvector

-- 1. Indexing lag: average and 99th-percentile delay between
--    source update and vector write
SELECT
    AVG(EXTRACT(EPOCH FROM (dc.indexed_at - dc.updated_at))) AS avg_lag_seconds,
    PERCENTILE_CONT(0.99) WITHIN GROUP (
        ORDER BY EXTRACT(EPOCH FROM (dc.indexed_at - dc.updated_at))
    ) AS p99_lag_seconds,
    MAX(EXTRACT(EPOCH FROM (dc.indexed_at - dc.updated_at))) AS max_lag_seconds
FROM document_chunks dc
WHERE dc.indexed_at > NOW() - INTERVAL '1 hour';

-- 2. Queue depth: source records not yet reflected in the vector table.
--    Run against your source database (adjust table/column names as needed)
SELECT COUNT(*) AS unindexed_count
FROM products p
WHERE p.updated_at > (
    SELECT last_indexed_at
    FROM indexer_state

```

```

    WHERE pipeline_id = 'products'
)
AND p.deleted_at IS NULL;

-- 3. Coverage ratio: distinct indexed doc_ids vs. source document count
SELECT
    (SELECT COUNT(DISTINCT doc_id) FROM document_chunks WHERE source = 'products')
    AS indexed_docs,
    (SELECT COUNT(*) FROM products WHERE deleted_at IS NULL)
    AS source_docs,
    ROUND(
        (SELECT COUNT(DISTINCT doc_id) FROM document_chunks WHERE source = 'products')::numeric
        / NULLIF((SELECT COUNT(*) FROM products WHERE deleted_at IS NULL), 0),
        4
    ) AS coverage_ratio;

-- 4. Orphaned chunks: chunks for doc_ids that no longer exist in the source.
-- These represent delete events that the pipeline missed.
SELECT dc.doc_id, COUNT(*) AS chunk_count
FROM document_chunks dc
LEFT JOIN products p
    ON dc.doc_id = 'product-' || p.product_id::text
WHERE dc.source = 'products'
    AND p.product_id IS NULL
GROUP BY dc.doc_id
ORDER BY chunk_count DESC
LIMIT 50;

-- 5. Recent indexing throughput: chunks written per minute over the last hour
SELECT
    DATE_TRUNC('minute', indexed_at) AS minute_bucket,
    COUNT(*) AS chunks_written
FROM document_chunks
WHERE indexed_at > NOW() - INTERVAL '1 hour'
GROUP BY 1
ORDER BY 1;

```

Equivalent queries for SQL Server (if your source system is SQL Server and you're monitoring the pipeline state table there):

```

-- SQL Server

-- 1. Indexing lag
SELECT
    AVG(DATEDIFF(SECOND, dc.updated_at, dc.indexed_at)) AS avg_lag_seconds,
    MAX(DATEDIFF(SECOND, dc.updated_at, dc.indexed_at)) AS max_lag_seconds
FROM document_chunks dc
WHERE dc.indexed_at > DATEADD(HOUR, -1, GETUTCDATE());

-- 2. Queue depth
SELECT COUNT(*) AS unindexed_count
FROM products p
WHERE p.updated_at > (
    SELECT last_indexed_at
    FROM indexer_state
    WHERE pipeline_id = 'products'
)
AND p.deleted_at IS NULL;

-- 3. Coverage ratio
SELECT
    indexed_docs,
    source_docs,
    ROUND(CAST(indexed_docs AS DECIMAL(18,4)) / NULLIF(source_docs, 0), 4)
    AS coverage_ratio
FROM (
    SELECT
        (SELECT COUNT(DISTINCT doc_id)
        FROM document_chunks WHERE source = 'products') AS indexed_docs,

```

```

        (SELECT COUNT(*)
         FROM products WHERE deleted_at IS NULL) AS source_docs
    ) t;

-- 4. Recent throughput: chunks written per minute
SELECT
    DATEADD(MINUTE,
            DATEDIFF(MINUTE, '2000-01-01', indexed_at),
            '2000-01-01') AS minute_bucket,
    COUNT(*) AS chunks_written
FROM document_chunks
WHERE indexed_at > DATEADD(HOUR, -1, GETUTCDATE())
GROUP BY DATEADD(MINUTE,
                DATEDIFF(MINUTE, '2000-01-01', indexed_at),
                '2000-01-01')
ORDER BY 1;

```

Alerting thresholds to establish:

Metric	Warning	Critical
P99 indexing lag	> 5 minutes	> 30 minutes
Queue depth	> 500 documents	> 5,000 documents
Coverage ratio	< 0.98	< 0.95
Orphaned chunk count	> 100	> 1,000

Set these as time-series alerts in whatever observability stack you already operate — Prometheus with Grafana, Datadog, CloudWatch. The queries above can be wrapped in a metrics scraper that runs every minute and emits gauge values.

Recall Degradation Detection

Lag metrics tell you about pipeline health. They do not tell you about index quality. A pipeline can be fully current — zero lag, perfect coverage — but the HNSW graph can still have degraded recall if it was built under heavy write pressure or with parameters that were too small for the current dataset size.

The way to detect recall degradation is to maintain a **golden evaluation set**: a small collection of queries with known ground-truth results, determined by brute-force exact search over a snapshot of the dataset. You then periodically run these queries through the ANN index and measure how many of the ground-truth results appear in the top-k.

```

# Python - recall benchmark against a golden evaluation set
import psycpg2
import numpy as np
from typing import List, Tuple

def brute_force_top_k(
    query_vector: List[float],
    conn,
    top_k: int = 10
) -> List[str]:
    """
    Exact nearest neighbors using sequential scan.
    Used only for evaluation, not for production queries.
    """
    with conn.cursor() as cur:

```

```

        cur.execute(
            """
            SELECT doc_id
            FROM document_chunks
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """
            ,
            (query_vector, top_k)
        )
        return [row[0] for row in cur.fetchall()]

def ann_top_k(
    query_vector: List[float],
    conn,
    top_k: int = 10,
    ef_search: int = 40
) -> List[str]:
    """
    Approximate nearest neighbors using HNSW index.
    """
    with conn.cursor() as cur:
        cur.execute(f"SET hnsw.ef_search = {ef_search}")
        cur.execute(
            """
            SELECT doc_id
            FROM document_chunks
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """
            ,
            (query_vector, top_k)
        )
        return [row[0] for row in cur.fetchall()]

def measure_recall(
    evaluation_set: List[Tuple[List[float], List[str]]],
    conn,
    top_k: int = 10,
    ef_search: int = 40
) -> float:
    """
    evaluation_set: list of (query_vector, ground_truth_doc_ids) pairs.
    Returns mean recall@top_k across all evaluation queries.
    """
    recalls = []
    for query_vector, ground_truth in evaluation_set:
        ann_results = set(ann_top_k(query_vector, conn, top_k, ef_search))
        ground_truth_set = set(ground_truth[:top_k])
        if not ground_truth_set:
            continue
        hit_count = len(ann_results & ground_truth_set)
        recalls.append(hit_count / len(ground_truth_set))

    return float(np.mean(recalls)) if recalls else 0.0

# Build the evaluation set once (or refresh monthly)
def build_evaluation_set(
    sample_queries: List[str],
    embeddings_client,
    conn,
    top_k: int = 10
) -> List[Tuple[List[float], List[str]]]:
    evaluation_set = []
    vectors = embeddings_client.embed_documents(sample_queries)
    for vec in vectors:
        ground_truth = brute_force_top_k(vec, conn, top_k)
        evaluation_set.append((vec, ground_truth))
    return evaluation_set

```

```
# Periodic recall check - run this in a scheduled job (e.g., daily)
conn = psycopg2.connect(DB_CONN)
# evaluation_set loaded from persistent storage (pickle, JSON, etc.)
# evaluation_set = load_evaluation_set("eval_set.pkl")

# recall = measure_recall(evaluation_set, conn, top_k=10, ef_search=40)
# if recall < 0.85:
#     trigger_index_rebuild_alert(recall)
# print(f"Recall@10 (ef_search=40): {recall:.4f}")
```

A recall below 0.85 at `ef_search=40` with an HNSW index built at `m=16`, `ef_construction=64` is a signal that either the index parameters need adjustment for the current dataset size, or the index has accumulated enough write-induced graph distortion to warrant a rebuild. Neither situation is an emergency, but both warrant scheduled maintenance.

Backfill Strategies for Large Datasets

When you first deploy a vector index on an existing dataset, or when you perform a full reindex after a model migration, you need a controlled backfill that does not overwhelm your embedding API, your database write capacity, or your operational budget.

Parallel worker pattern with a work queue:

```
# Python - parallel backfill with a work queue
import concurrent.futures
import queue
import threading
import time
import psycopg2
import logging
from typing import Iterator

MAX_WORKERS = 8          # Number of concurrent embedding+write workers
EMBED_BATCH_SIZE = 100   # Documents per embedding API call
WRITE_BATCH_SIZE = 500   # Chunks per database write batch
RATE_LIMIT_RPS = 50      # Target requests per second to embedding API

def iter_source_documents(source_conn, batch_size: int = 1000) -> Iterator[list]:
    """
    Yield batches of (product_id, description, category) rows
    from the source database.
    """
    offset = 0
    while True:
        with source_conn.cursor() as cur:
            cur.execute(
                """
                SELECT product_id, description, category
                FROM products
                WHERE deleted_at IS NULL
                ORDER BY product_id
                LIMIT %s OFFSET %s
                """
                ,
                (batch_size, offset)
            )
            rows = cur.fetchall()
            if not rows:
                break
            yield rows
            offset += len(rows)
```

```

        if len(rows) < batch_size:
            break

def backfill_worker(
    work_queue: queue.Queue,
    vector_conn_string: str,
    worker_id: int,
    counter: list,  # Shared mutable counter [total_processed]
    counter_lock: threading.Lock
):
    """
    Worker thread: dequeues document batches, embeds, and writes.
    """
    conn = psycopg2.connect(vector_conn_string)
    processed = 0

    while True:
        try:
            batch = work_queue.get(timeout=5)
        except queue.Empty:
            break

        try:
            for product_id, description, category in batch:
                index_document(
                    doc_id=f"product-{product_id}",
                    content=description or "",
                    metadata={"source": "products", "category": category or ""},
                    conn=conn
                )
                processed += 1

            work_queue.task_done()

            with counter_lock:
                counter[0] += len(batch)
                if counter[0] % 1000 == 0:
                    logging.info(f"Backfill progress: {counter[0]} documents indexed")

        except Exception as e:
            logging.error(f"Worker {worker_id} failed on batch: {e}")
            work_queue.task_done()

    conn.close()
    logging.info(f"Worker {worker_id} finished, processed {processed} documents locally")

def run_backfill(source_conn_string: str, vector_conn_string: str):
    source_conn = psycopg2.connect(source_conn_string)
    work_queue = queue.Queue(maxsize=MAX_WORKERS * 4)
    counter = [0]
    counter_lock = threading.Lock()

    workers = []
    for i in range(MAX_WORKERS):
        t = threading.Thread(
            target=backfill_worker,
            args=(work_queue, vector_conn_string, i, counter, counter_lock),
            daemon=True
        )
        t.start()
        workers.append(t)

    # Feed the queue from the source
    for batch in iter_source_documents(source_conn, batch_size=50):
        work_queue.put(batch)
        # Throttle producer to avoid overwhelming the queue
        time.sleep(len(batch) / (RATE_LIMIT_RPS * MAX_WORKERS))

```

```
work_queue.join()
for t in workers:
    t.join()

source_conn.close()
logging.info(f"Backfill complete. Total documents: {counter[0]}")
```

Checkpointing the backfill. Large backfills fail partway through. Always checkpoint progress so a restart picks up where it left off rather than starting over. The `UNIQUE (doc_id, chunk_index)` constraint on the `document_chunks` table combined with the `ON CONFLICT DO UPDATE` upsert semantics means re-processing already-indexed documents is safe — it overwrites without creating duplicates. The only cost is the redundant embedding API call, which is wasteful but not dangerous.

For extra safety, maintain an explicit checkpoint table:

```
-- PostgreSQL / pgvector
CREATE TABLE backfill_checkpoint (
    backfill_id      TEXT PRIMARY KEY,
    last_product_id  BIGINT NOT NULL,
    total_indexed    BIGINT NOT NULL DEFAULT 0,
    started_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Update checkpoint after each batch
INSERT INTO backfill_checkpoint (backfill_id, last_product_id, total_indexed)
VALUES ('migration-2024-11', 42000, 42000)
ON CONFLICT (backfill_id)
DO UPDATE SET
    last_product_id = EXCLUDED.last_product_id,
    total_indexed   = EXCLUDED.total_indexed,
    updated_at      = NOW();

-- Resume from checkpoint
SELECT last_product_id FROM backfill_checkpoint
WHERE backfill_id = 'migration-2024-11';
```

Key Takeaways

- **Vector index freshness is not automatic.** Unlike B-tree indexes that update transactionally, vector indexes require an explicit pipeline that runs chunking, embedding generation, and upsert as separate asynchronous steps. The gap between a source write and its vector representation is a design choice you must actively manage.
- **Choose your CDC approach based on your freshness SLA.** Watermark polling (30–90 second lag) is simple and sufficient for most applications. Debezium-to-Kafka CDC (sub-5-second lag) is appropriate when stale results are directly harmful — live inventory, breaking news, or customer-facing personalization. Do not add Kafka complexity unless you genuinely need sub-minute freshness.
- **HNSW indexes degrade under write pressure and need periodic rebuilds.** `pgvector` accepts live inserts but accumulates graph distortion over time. Use `CREATE INDEX CONCURRENTLY` to rebuild during low-traffic windows without blocking queries. Treat this as scheduled maintenance, not an emergency procedure.
- **Embedding model migrations require blue-green reindexing.** Vectors from different models occupy incompatible spaces. When upgrading models, dual-write to both old and new

indexes during the transition, validate retrieval parity with Jaccard similarity checks across a representative query set, and only cut over query traffic after the new index meets your recall threshold.

- **Instrument both pipeline health and index quality as separate concerns.** Lag metrics, queue depth, and coverage ratios tell you whether documents are reaching the index. Periodic recall benchmarks against a golden evaluation set tell you whether the index itself is returning accurate results. You need both — a current but degraded index fails users just as much as a high-quality but stale one.
-

Chapter 29: Real-Time vs Batch Embedding Generation

Embedding generation sits at the boundary between your application’s request path and your vector database’s indexing pipeline. Every vector in your store started as a raw document, image, or structured record that passed through a model to become a float array — and the decision about *when* that transformation happens shapes your latency profile, cost structure, operational complexity, and freshness guarantees in ways that compound at scale. This chapter maps out the two fundamental strategies — real-time and batch — examines where each breaks down, and shows the hybrid architectures that production systems almost always end up building.

29.1 Understanding the Embedding Generation Pipeline

Before choosing a strategy, you need a clear model of what the pipeline actually does. Embedding generation is a compute-intensive inference step: a neural network reads tokens, runs a forward pass, and returns a fixed-size dense vector. That vector is then written to your vector store and becomes searchable.

Think of it like a computed column in a relational database. You have a source table of raw content and a derived representation stored separately for efficient retrieval. The question isn’t whether to compute it — you must — it’s whether you compute it synchronously during write operations or asynchronously in a scheduled or streaming process.

The pipeline stages:

Raw Content → Chunking/Preprocessing → Model Inference → Vector Storage → Index Build

Each stage introduces latency and can fail independently. In real-time mode, all of this sits in your request path. In batch mode, only the final query lookup is synchronous.

Key parameters that drive strategy selection:

Parameter	Favors Real-Time	Favors Batch
Content ingestion rate	Low to moderate	High volume
Latency tolerance	< 500ms end-to-end	Minutes to hours acceptable
Content update frequency	Continuous	Periodic
Embedding model size	API-hosted, fast	Large local models
Cost sensitivity	Bursty, pay-per-call	Predictable volume pricing

Parameter	Favors Real-Time	Favors Batch
Content at query time	Dynamic, user-generated	Stable corpus

The embedding model itself is the most constrained resource. OpenAI's `text-embedding-3-small` processes roughly 1M tokens per minute on the API tier, costing \$0.02 per 1M tokens. A local `sentence-transformers/all-MiniLM-L6-v2` running on CPU processes approximately 500 sentences per second on a modern machine — fast enough for many real-time use cases, but it becomes a bottleneck as soon as content volume spikes.

29.2 Real-Time Embedding Generation

Real-time embedding generation means computing vectors synchronously as new content arrives — either during document ingestion or at query time for incoming search strings. The user or upstream system waits for the embedding to complete before the request continues.

When this approach works well:

- A user submits a search query and you need the query embedding immediately
- A chat application ingests user messages that must be retrievable within seconds
- An API receives webhook events that feed a RAG pipeline with sub-minute freshness requirements

When it breaks:

- Document ingestion volume spikes (model API rate limits become throttling events)
- Embedding latency becomes the dominant term in your p95 response time
- Upstream content sources send batches disguised as streams (100 documents at once)

The following example shows a FastAPI endpoint that ingests documents and generates embeddings synchronously using the OpenAI API and pgvector for storage:

```
# Python
import asyncio
import time
from typing import Optional

import asyncpg
from fastapi import FastAPI, HTTPException
from openai import AsyncOpenAI
from pydantic import BaseModel

app = FastAPI()
openai_client = AsyncOpenAI()

# Connection pool - reuse across requests
pool: Optional[asyncpg.Pool] = None

async def get_pool() -> asyncpg.Pool:
    global pool
    if pool is None:
        pool = await asyncpg.create_pool(
            dsn="postgresql://app_user:secret@localhost:5432/vectordb",
            min_size=5,
            max_size=20,
```

```
    )
    return pool

class DocumentIngest(BaseModel):
    document_id: str
    content: str
    metadata: dict = {}

class SearchQuery(BaseModel):
    query: str
    top_k: int = 10

@app.post("/documents")
async def ingest_document(doc: DocumentIngest):
    """Real-time ingestion: embed and store in a single synchronous operation."""
    start = time.monotonic()

    # Step 1: Generate embedding - this is the latency bottleneck
    try:
        response = await openai_client.embeddings.create(
            model="text-embedding-3-small",
            input=doc.content,
            encoding_format="float",
        )
    except Exception as e:
        raise HTTPException(status_code=502, detail=f"Embedding API error: {e}")

    embedding = response.data[0].embedding
    embed_latency_ms = (time.monotonic() - start) * 1000

    # Step 2: Write to pgvector
    db = await get_pool()
    try:
        await db.execute(
            """
            INSERT INTO documents (document_id, content, embedding, metadata, embedded_at)
            VALUES ($1, $2, $3::vector, $4, NOW())
            ON CONFLICT (document_id) DO UPDATE
            SET content = EXCLUDED.content,
                embedding = EXCLUDED.embedding,
                embedded_at = NOW()
            """,
            doc.document_id,
            doc.content,
            str(embedding),
            doc.metadata,
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"DB write error: {e}")

    total_latency_ms = (time.monotonic() - start) * 1000

    return {
        "document_id": doc.document_id,
        "embed_latency_ms": round(embed_latency_ms, 2),
        "total_latency_ms": round(total_latency_ms, 2),
        "dimensions": len(embedding),
    }

@app.post("/search")
async def search(query: SearchQuery):
    """Query embedding is always real-time - there is no alternative."""
    response = await openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=query.query,
        encoding_format="float",
    )
```

```
)
query_vector = response.data[0].embedding

db = await get_pool()
rows = await db.fetch(
    """
    SELECT document_id, content, metadata,
           1 - (embedding <=> $1::vector) AS cosine_similarity
    FROM documents
    ORDER BY embedding <=> $1::vector
    LIMIT $2
    """,
    str(query_vector),
    query.top_k,
)

return [dict(r) for r in rows]
```

The critical observation here: query embedding is *always* real-time. You cannot precompute embeddings for arbitrary user queries. The only decision is whether document embeddings are also real-time or handled separately.

Concurrency and rate limit management become the primary operational concerns. A real-time pipeline that handles 50 concurrent document writes will fire 50 simultaneous API calls. OpenAI's `text-embedding-3-small` has a default rate limit of 3,000 RPM (requests per minute) on paid tiers — roughly 50 per second. You need a semaphore or queue to stay within bounds:

```
# Python
import asyncio
from openai import AsyncOpenAI

client = AsyncOpenAI()

# Limit concurrent embedding calls to avoid rate limit errors
EMBED_SEMAPHORE = asyncio.Semaphore(20)

async def safe_embed(text: str, model: str = "text-embedding-3-small") -> list[float]:
    """Rate-limited embedding call with exponential backoff."""
    async with EMBED_SEMAPHORE:
        for attempt in range(4):
            try:
                response = await client.embeddings.create(
                    model=model,
                    input=text,
                    encoding_format="float",
                )
                return response.data[0].embedding
            except Exception as e:
                if "rate_limit" in str(e).lower() and attempt < 3:
                    wait = (2**attempt) + 0.1
                    await asyncio.sleep(wait)
                else:
                    raise

async def embed_batch_concurrent(texts: list[str], max_concurrency: int = 20) -> list[list[float]]:
    """Embed a list of texts with bounded concurrency."""
    tasks = [safe_embed(t) for t in texts]
    return await asyncio.gather(*tasks)
```

29.3 Batch Embedding Generation

Batch processing treats embedding generation as an offline transformation job — a scheduled or event-triggered pipeline that processes accumulated content and writes vectors to the store without blocking application writes. The source system records raw content immediately; embedding happens asynchronously.

This maps directly to the ETL/ELT model that DBAs know well. The source table stores canonical data, and a separate process materializes the derived representation. You can think of it as a materialized view that refreshes on a schedule, except the “computation” is a neural network inference rather than a SQL aggregation.

Architecture pattern:

Source Table (raw) → Embedding Worker → Vector Store

↓ writes immediately ↑ available after job completes
↓ marks as `embedding_status='pending'`

The `embedding_status` column is the coordination mechanism. Writers insert content and mark it pending; the embedding worker polls or subscribes for pending records, processes them in optimized batches, and marks them complete.

```
-- PostgreSQL / pgvector

-- Source table with embedding lifecycle tracking
CREATE TABLE documents (
  id          BIGSERIAL PRIMARY KEY,
  document_id TEXT UNIQUE NOT NULL,
  content     TEXT NOT NULL,
  content_hash TEXT GENERATED ALWAYS AS (md5(content)) STORED,
  embedding   vector(1536),
  embed_model TEXT,
  embedding_status TEXT NOT NULL DEFAULT 'pending'
  CHECK (embedding_status IN ('pending', 'processing', 'complete', 'failed')),
  retry_count INT NOT NULL DEFAULT 0,
  embedded_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Index for the worker's polling query - only pending/failed rows
CREATE INDEX idx_documents_embed_queue
  ON documents (created_at)
  WHERE embedding_status IN ('pending', 'failed') AND retry_count < 3;

-- Claim a batch atomically using SKIP LOCKED - prevents double-processing
-- across multiple worker instances
CREATE OR REPLACE FUNCTION claim_embedding_batch(batch_size INT)
RETURNS SETOF documents AS $$
  UPDATE documents
  SET embedding_status = 'processing',
      updated_at = NOW()
  WHERE id IN (
    SELECT id FROM documents
    WHERE embedding_status IN ('pending', 'failed')
      AND retry_count < 3
    ORDER BY created_at
    LIMIT batch_size
    FOR UPDATE SKIP LOCKED
  )
  RETURNING *;
$$ LANGUAGE sql;
```

The `FOR UPDATE SKIP LOCKED` pattern is the same technique used in job queue implementations

like Sidekiq's PostgreSQL backend. Multiple embedding workers can run in parallel without coordination — each atomically claims its batch, processes it, and writes results back.

```
# Python
import time
from datetime import datetime
from typing import Optional

import psycopg2
import psycopg2.extras
from sentence_transformers import SentenceTransformer

# Using a local model for batch processing - no API costs, full throughput control
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
MODEL_NAME = "all-MiniLM-L6-v2"
BATCH_SIZE = 256 # Optimal for CPU inference; increase to 512+ with GPU

def get_connection():
    return psycopg2.connect(
        dsn="postgresql://app_user:secret@localhost:5432/vectordb"
    )

def process_batch(conn) -> int:
    """Claim and process one batch of pending documents. Returns count processed."""
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        # Atomically claim a batch
        cur.execute("SELECT * FROM claim_embedding_batch(%s)", (BATCH_SIZE,))
        rows = cur.fetchall()
        conn.commit()

    if not rows:
        return 0

    ids = [r["id"] for r in rows]
    texts = [r["content"] for r in rows]

    start = time.monotonic()
    # sentence-transformers handles internal batching efficiently
    embeddings = model.encode(
        texts,
        batch_size=64,
        show_progress_bar=False,
        normalize_embeddings=True, # Cosine similarity becomes dot product
    )
    inference_ms = (time.monotonic() - start) * 1000

    # Bulk update - much faster than row-by-row
    with conn.cursor() as cur:
        update_data = [
            (
                embeddings[i].tolist(),
                MODEL_NAME,
                "complete",
                datetime.utcnow(),
                ids[i],
            )
            for i in range(len(ids))
        ]
        psycopg2.extras.execute_batch(
            cur,
            """
            UPDATE documents
            SET embedding = %s::vector,
                embed_model = %s,
                embedding_status = %s,
                embedded_at = %s,
                updated_at = NOW()
            """
        )
```



```
        WHERE id = %s
        """
        update_data,
        page_size=100,
    )
    conn.commit()

    throughput = len(ids) / (inference_ms / 1000)
    print(
        f"Processed {len(ids)} docs in {inference_ms:.0f}ms "
        f"({throughput:.0f} docs/sec)"
    )
    return len(ids)

def handle_failures(conn, failed_ids: list[int]):
    """Mark failed rows for retry or permanent failure."""
    with conn.cursor() as cur:
        cur.execute(
            """
            UPDATE documents
            SET embedding_status = CASE
                WHEN retry_count >= 2 THEN 'failed'
                ELSE 'pending'
            END,
            retry_count = retry_count + 1,
            updated_at = NOW()
            WHERE id = ANY(%s)
            """
            ,
            (failed_ids,),
        )
        conn.commit()

def run_worker(idle_sleep_sec: float = 5.0):
    """Continuous worker loop."""
    conn = get_connection()
    print(f"Embedding worker started. Model: {MODEL_NAME}, Batch size: {BATCH_SIZE}")
    try:
        while True:
            processed = process_batch(conn)
            if processed == 0:
                time.sleep(idle_sleep_sec)
    finally:
        conn.close()

if __name__ == "__main__":
    run_worker()
```

Throughput benchmarks with local models are worth establishing early. On a single CPU worker with all-MiniLM-L6-v2:

- Batch size 64: ~400 docs/sec
- Batch size 256: ~600 docs/sec
- Batch size 512: ~650 docs/sec (diminishing returns)

On a single A10G GPU: - Batch size 256: ~4,500 docs/sec - Batch size 1024: ~8,000 docs/sec

For comparison, OpenAI's batch embedding API (asynchronous, 50% cheaper than synchronous) processes up to 50,000 requests per batch with a 24-hour completion window.

29.4 Hybrid Architectures: Combining Both Strategies

Production systems rarely use pure real-time or pure batch. The practical pattern is a **tiered architecture** where query embeddings are always real-time, freshly ingested high-priority content uses real-time embedding with graceful degradation, and bulk content uses batch processing.

A common pattern that works well: write the raw document to storage immediately (making it visible to the application), generate the embedding asynchronously, and return a **pending** state to callers that need to know embedding status. This is analogous to an eventually consistent write — your database accepts the write, your derived index catches up.

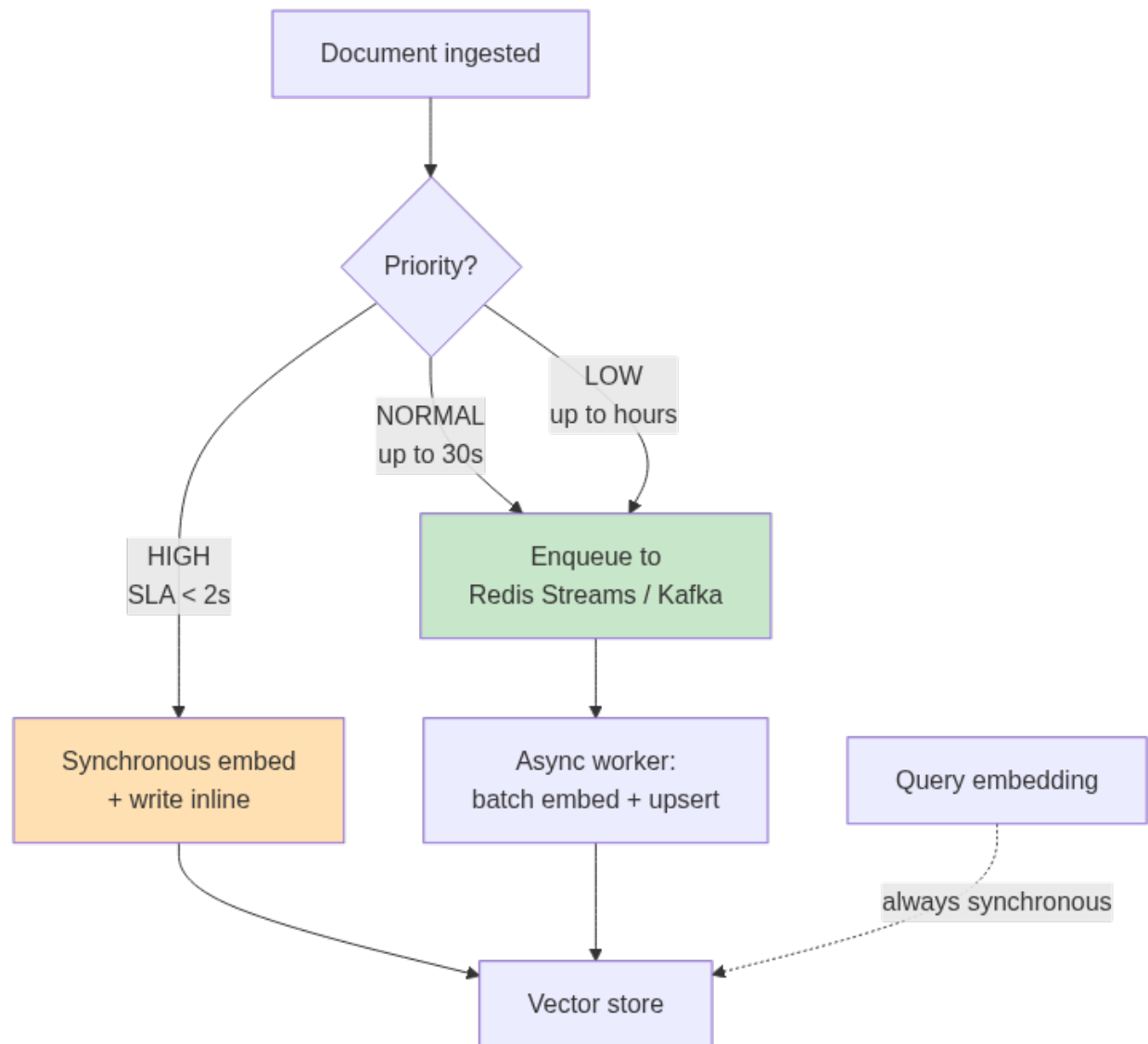


Figure 19: Query embedding always sits on the synchronous path; document embedding is routed by priority in a hybrid tiered architecture.

The following example shows this pattern using Pinecone as the vector store and a message queue (Redis Streams) for coordination:

```

# Python
import json
import time
import uuid
from enum import Enum
from typing import Optional

import redis
from openai import OpenAI
from pinecone import Pinecone, ServerlessSpec

# Configuration
REDIS_URL = "redis://localhost:6379"
PINECONE_API_KEY = "your-pinecone-key"
PINECONE_INDEX = "documents-prod"
EMBED_MODEL = "text-embedding-3-small"
EMBED_DIMS = 1536
STREAM_KEY = "embed:queue"
CONSUMER_GROUP = "embed-workers"

# Clients
redis_client = redis.from_url(REDIS_URL, decode_responses=True)
openai_client = OpenAI()
pc = Pinecone(api_key=PINECONE_API_KEY)

# Ensure index exists
if PINECONE_INDEX not in [i.name for i in pc.list_indexes()]:
    pc.create_index(
        name=PINECONE_INDEX,
        dimension=EMBED_DIMS,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1"),
    )

index = pc.Index(PINECONE_INDEX)

class EmbedPriority(str, Enum):
    HIGH = "high" # Real-time path: SLA < 2 seconds
    NORMAL = "normal" # Async path: acceptable delay up to 30 seconds
    LOW = "low" # Batch path: delay up to hours acceptable

def ingest_document(
    document_id: str,
    content: str,
    metadata: dict,
    priority: EmbedPriority = EmbedPriority.NORMAL,
) -> dict:
    """
    Write document to the queue. HIGH priority documents are embedded inline.
    NORMAL/LOW are deferred to the async worker.
    """
    if priority == EmbedPriority.HIGH:
        # Synchronous path - caller waits for embedding
        embedding = _embed_text(content)
        _write_to_pinecone(document_id, embedding, content, metadata)
        return {"document_id": document_id, "status": "indexed", "priority": priority}
    else:
        # Async path - enqueue for worker
        message = {
            "document_id": document_id,
            "content": content,
            "metadata": json.dumps(metadata),
            "priority": priority,
            "enqueued_at": time.time(),
        }
        redis_client.xadd(STREAM_KEY, message, maxlen=100_000)
        return {"document_id": document_id, "status": "pending", "priority": priority}

```

```

def _embed_text(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model=EMBED_MODEL,
        input=text,
        encoding_format="float",
    )
    return response.data[0].embedding

def _write_to_pinecone(
    document_id: str,
    embedding: list[float],
    content: str,
    metadata: dict,
):
    index.upsert(
        vectors=[{
            "id": document_id,
            "values": embedding,
            "metadata": {**metadata, "content_preview": content[:500]},
        }],
        namespace="documents",
    )

def run_async_worker(batch_size: int = 50, idle_sleep: float = 1.0):
    """
    Redis Streams consumer that processes the embedding queue.
    Multiple instances can run concurrently - each reads from the same group.
    """
    worker_id = f"worker-{uuid.uuid4().hex[:8]}"

    # Create consumer group if it doesn't exist
    try:
        redis_client.xgroup_create(STREAM_KEY, CONSUMER_GROUP, id="0", mkstream=True)
    except redis.exceptions.ResponseError:
        pass # Group already exists

    print(f"Worker {worker_id} started, consuming from {STREAM_KEY}")

    while True:
        # Read up to batch_size messages, blocking for 1 second if queue is empty
        messages = redis_client.xreadgroup(
            groupname=CONSUMER_GROUP,
            consumername=worker_id,
            streams={STREAM_KEY: ">"},
            count=batch_size,
            block=int(idle_sleep * 1000),
        )

        if not messages:
            continue

        stream_messages = messages[0][1] # (stream_key, [(msg_id, fields), ...])
        msg_ids = [m[0] for m in stream_messages]
        payloads = [m[1] for m in stream_messages]

        # Sort by priority - HIGH items get embedded first within the batch
        payloads_with_ids = sorted(
            zip(msg_ids, payloads),
            key=lambda x: 0 if x[1]["priority"] == EmbedPriority.HIGH else 1,
        )

        texts = [p["content"] for _, p in payloads_with_ids]

        # Batch embed - single API call for the whole batch
        try:
            response = openai_client.embeddings.create(
                model=EMBED_MODEL,

```

```

        input=texts,
        encoding_format="float",
    )
    embeddings = [d.embedding for d in response.data]
except Exception as e:
    print(f"Embedding batch failed: {e}")
    # Messages remain in PEL (pending entries list) for reprocessing
    time.sleep(5)
    continue

# Upsert to Pinecone in a single batch call
vectors = []
for i, (msg_id, payload) in enumerate(payloads_with_ids):
    vectors.append({
        "id": payload["document_id"],
        "values": embeddings[i],
        "metadata": {
            **json.loads(payload["metadata"]),
            "content_preview": payload["content"][:500],
            "embedded_at": time.time(),
        },
    })

index.upsert(vectors=vectors, namespace="documents")

# Acknowledge successful processing
redis_client.xack(STREAM_KEY, CONSUMER_GROUP, *msg_ids)
print(f"Worker {worker_id}: processed {len(vectors)} documents")

```

This architecture gives you exactly three knobs to tune: the concurrency of async workers, the batch size per worker iteration, and the priority threshold for synchronous embedding. Most teams start with HIGH priority for user-generated content that needs immediate search availability and NORMAL/LOW for system-generated or bulk content.

29.5 Cost Optimization and Model Selection

Embedding generation costs break down differently depending on your strategy. Real-time API calls are billed per token with no volume discount at standard tiers; batch jobs can qualify for discounted batch pricing or allow you to substitute cheaper local models where latency isn't constrained.

Cost comparison for 100M tokens per month:

Model	Mode	Cost (per 1M tokens)	Latency (p50)
OpenAI text-embedding-3-small	Synchronous API	Check current pricing	~80ms
OpenAI text-embedding-3-small	Batch API	Discounted tier	Up to 24h
OpenAI text-embedding-ada-002	Synchronous API	Check current pricing	~100ms
Cohere embed-v3	Synchronous API	Check current pricing	~120ms
all-MiniLM-L6-v2	Local CPU(384 dims)	Infrastructure cost only	~2ms
all-mpnet-base-v2	Local CPU (768 dims)	Infrastructure cost only	~8ms

Model	Mode	Cost (per 1M tokens)	Latency (p50)
bge-large-en-v1.5	Local GPU (1024 dims)	Infrastructure cost only	~0.5ms

The cost calculus depends on your quality requirements. `all-MiniLM-L6-v2` achieves strong semantic retrieval performance on most English-language tasks; the quality gap versus OpenAI's models narrows significantly when your retrieval task is domain-specific and you fine-tune or use a retrieval-oriented model. For bulk batch processing where quality matters and cost is constrained, a local GPU-hosted model is typically the right answer once your volume justifies the infrastructure.

Model selection decision tree:

Need multilingual support?

Yes → OpenAI `text-embedding-3-large` or Cohere `embed-multilingual-v3`

No → Continue

Need < 200ms embedding latency (real-time path)?

Yes → OpenAI `text-embedding-3-small` (API) or `all-MiniLM-L6-v2` (local)

No (batch path) → any local model, batch API acceptable

Volume > 50M tokens/month?

Yes → Evaluate local GPU hosting vs. batch API pricing

No → Synchronous API is likely cost-effective

Offline / air-gapped / compliance-constrained?

Always → Local model required

Caching embeddings is an underutilized optimization in real-time pipelines. If your application re-embeds the same content (common in chat applications where system prompts are re-embedded on every session), a simple LRU cache keyed on content hash eliminates redundant API calls:

```
# Python
import hashlib
from functools import lru_cache
from openai import OpenAI

client = OpenAI()

def content_hash(text: str) -> str:
    return hashlib.sha256(text.encode()).hexdigest()

# In-process LRU cache for embeddings - survives for the process lifetime
# For cross-process caching, use Redis with TTL
@lru_cache(maxsize=10_000)
def _cached_embed(text_hash: str, text: str, model: str) -> tuple[float, ...]:
    """
    LRU cache keyed by content hash. Returns tuple (hashable) rather than list.
    text_hash is included in the key to force cache invalidation on content change,
    but the actual hash computation happens outside this function.
    """
    response = client.embeddings.create(
        model=model,
        input=text,
        encoding_format="float",
```

```

    )
    return tuple(response.data[0].embedding)

def embed_with_cache(text: str, model: str = "text-embedding-3-small") -> list[float]:
    h = content_hash(text)
    return list(_cached_embed(h, text, model))

```

For distributed caching across worker instances, use Redis with a content-hash key and an appropriate TTL:

```

# Python
import hashlib
import json
import redis
from openai import OpenAI

client = OpenAI()
cache = redis.from_url("redis://localhost:6379", decode_responses=True)

CACHE_TTL_SECONDS = 86_400 * 7 # 7 days

def embed_with_redis_cache(
    text: str,
    model: str = "text-embedding-3-small",
) -> list[float]:
    key = f"emb:{model}:{hashlib.sha256(text.encode()).hexdigest()}"
    cached = cache.get(key)
    if cached:
        return json.loads(cached)

    response = client.embeddings.create(
        model=model,
        input=text,
        encoding_format="float",
    )
    embedding = response.data[0].embedding
    cache.setex(key, CACHE_TTL_SECONDS, json.dumps(embedding))
    return embedding

```

A cache hit rate of even 30–40% on a high-traffic real-time system meaningfully reduces both API costs and p95 latency, since cache reads are an order of magnitude faster than embedding API calls.

29.6 Monitoring and Observability

Embedding pipelines have failure modes that are invisible to standard application monitoring. A real-time pipeline can silently degrade — API calls succeed but return nonsensical embeddings due to truncation, rate limiting, or model version drift — and you won't see it in error rates. A batch pipeline can fall arbitrarily far behind its queue without triggering obvious alerts.

Metrics to instrument for real-time pipelines:

```

# Python - using prometheus_client for illustration
from prometheus_client import Counter, Histogram, Gauge
import time

embed_latency = Histogram(
    "embedding_generation_seconds",
    "Time to generate an embedding",
    buckets=[0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0],
    labelnames=["model", "source"], # source: realtime, cache, batch
)

```

```

embed_errors = Counter(
    "embedding_errors_total",
    "Embedding generation failures",
    labelnames=["model", "error_type"], # error_type: rate_limit, timeout, other
)

embed_cache_hits = Counter(
    "embedding_cache_hits_total",
    "Embedding cache hits",
    labelnames=["model"],
)

embed_tokens = Counter(
    "embedding_tokens_total",
    "Total tokens sent for embedding",
    labelnames=["model"],
)

def instrumented_embed(text: str, model: str = "text-embedding-3-small") -> list[float]:
    start = time.monotonic()
    token_count = len(text.split()) # rough approximation

    try:
        embedding = embed_with_redis_cache(text, model)
        embed_latency.labels(model=model, source="realtime").observe(
            time.monotonic() - start
        )
        embed_tokens.labels(model=model).inc(token_count)
        return embedding
    except Exception as e:
        error_type = "rate_limit" if "rate" in str(e).lower() else "other"
        embed_errors.labels(model=model, error_type=error_type).inc()
        raise

```

Metrics to instrument for batch pipelines:

```

-- PostgreSQL - queue depth monitoring view
-- Run this periodically and emit to your metrics system

CREATE VIEW embedding_queue_health AS
SELECT
    embedding_status,
    COUNT(*) AS document_count,
    MIN(created_at) AS oldest_pending,
    NOW() - MIN(created_at) AS max_lag_interval,
    AVG(retry_count) AS avg_retries
FROM documents
GROUP BY embedding_status;

# Python - scrape the health view and emit metrics
from prometheus_client import Gauge
import psycopg2

queue_depth = Gauge(
    "embedding_queue_depth",
    "Number of documents awaiting embedding",
    labelnames=["status"],
)

queue_lag_seconds = Gauge(
    "embedding_queue_lag_seconds",
    "Age of oldest pending document in the embedding queue",
)

def scrape_queue_metrics(conn):
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM embedding_queue_health")

```



```

rows = cur.fetchall()

for row in rows:
    status, count, oldest, lag_interval, avg_retries = row
    queue_depth.labels(status=status).set(count)
    if status == "pending" and lag_interval is not None:
        queue_lag_seconds.set(lag_interval.total_seconds())

```

Alert thresholds that matter in practice:

- Real-time: p95 embedding latency > 1 second → likely rate limiting or network issue
- Real-time: error rate > 0.5% over 5 minutes → alert immediately
- Batch: queue lag (age of oldest pending) > 10 minutes for HIGH priority content → worker stalled
- Batch: failed status count growing monotonically → retry logic broken or content issue
- Both: embedding dimension mismatch → model version changed, index incompatible

The dimension mismatch case deserves special attention. If you switch models (e.g., from `text-embedding-ada-002` at 1536 dimensions to `text-embedding-3-large` at 3072 dimensions), you cannot mix vectors in the same index. Your pipeline needs to track `embed_model` alongside each vector and either maintain separate indexes per model version or perform a full re-embedding pass during migration.

29.7 Re-Embedding and Index Migration

Model upgrades and content updates both require re-embedding — and the strategy for doing so efficiently varies based on scale and urgency.

Triggers for re-embedding:

1. Content update: the underlying document changed, so the stored vector is stale
2. Model upgrade: a better model is available and quality improvement justifies migration cost
3. Dimension change: the new model produces vectors of a different size, requiring index rebuild
4. Normalization change: you decide to switch from raw to normalized vectors (or vice versa)

Detecting stale embeddings is straightforward if you store a content hash:

```

-- PostgreSQL
-- Find documents whose content has changed since embedding was generated
SELECT document_id, content, embedded_at
FROM documents
WHERE content_hash != md5(content) -- content has changed since last embed
    OR embedded_at IS NULL          -- never embedded
    OR embed_model != 'text-embedding-3-small' -- model version changed
ORDER BY created_at DESC;

```

The `content_hash` column defined as a generated column (as shown in section 29.3) eliminates the need to compute this manually — PostgreSQL maintains it automatically on every update.

Zero-downtime model migration pattern:

The safest approach for model migrations mirrors a blue/green deployment. Build the new index in parallel while the old index continues serving queries, then perform a cut-over once re-embedding is complete.

```

-- PostgreSQL - dual-index migration setup

```

```

-- New index for the upgraded model (different dimension)
CREATE TABLE documents_v2 AS TABLE documents WITH NO DATA;
ALTER TABLE documents_v2 ADD COLUMN embedding_v2 vector(3072);

-- Track migration progress
ALTER TABLE documents ADD COLUMN embed_v2_status TEXT DEFAULT 'pending';

-- Worker migrates batches from documents → documents_v2
-- Queries can be directed to either table via a view
CREATE OR REPLACE VIEW documents_current AS
SELECT
    d.document_id,
    d.content,
    COALESCE(d2.embedding_v2, d.embedding) AS embedding,
    CASE
        WHEN d2.embedding_v2 IS NOT NULL THEN 'v2'
        ELSE 'v1'
    END AS embed_version
FROM documents d
LEFT JOIN documents_v2 d2 USING (document_id);

```

For Weaviate, which manages its own index internally, the migration requires a different approach — create a new class (collection) with the updated vectorizer configuration, re-ingest documents, verify result quality against a held-out query set, then update your application’s class references:

```

# Python - Weaviate collection migration
import weaviate
from weaviate.classes.config import Configure, Property, DataType

client = weaviate.connect_to_local()

# Create v2 collection with new model
client.collections.create(
    name="DocumentsV2",
    vectorizer_config=Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-large", # Upgraded from 3-small
    ),
    properties=[
        Property(name="document_id", data_type=DataType.TEXT),
        Property(name="content", data_type=DataType.TEXT),
    ],
)

# Re-ingest - Weaviate generates embeddings via the configured vectorizer
collection_v2 = client.collections.get("DocumentsV2")
collection_v1 = client.collections.get("Documents")

# Batch migration with Weaviate's batch API
with collection_v2.batch.dynamic() as batch:
    for item in collection_v1.iterator():
        batch.add_object({
            "document_id": item.properties["document_id"],
            "content": item.properties["content"],
        })

# After verification, update application to read from DocumentsV2
# Then delete DocumentsV2 (old collection) after traffic cutover
client.close()

```

Partial re-embedding is viable when only a subset of content needs updating. Use `embed_model` tracking to identify rows that need refresh:

```

# Python - targeted re-embedding of content using old model version
import psycopg2
import psycopg2.extras
from sentence_transformers import SentenceTransformer

```

```
OLD_MODEL = "all-MiniLM-L6-v2"
NEW_MODEL = "all-mpnet-base-v2"

new_encoder = SentenceTransformer(NEW_MODEL)

def migrate_model_batch(conn, batch_size: int = 256):
    """Re-embed documents that were embedded with the old model."""
    with conn.cursor(cursor_factory=psycpg2.extras.RealDictCursor) as cur:
        cur.execute(
            """
            UPDATE documents
            SET embedding_status = 'processing', updated_at = NOW()
            WHERE id IN (
                SELECT id FROM documents
                WHERE embed_model = %s
                ORDER BY id
                LIMIT %s
                FOR UPDATE SKIP LOCKED
            )
            RETURNING id, content
            """,
            (OLD_MODEL, batch_size),
        )
        rows = cur.fetchall()
        conn.commit()

    if not rows:
        return 0

    ids = [r["id"] for r in rows]
    texts = [r["content"] for r in rows]
    embeddings = new_encoder.encode(texts, normalize_embeddings=True)

    with conn.cursor() as cur:
        psycpg2.extras.execute_batch(
            cur,
            """
            UPDATE documents
            SET embedding = %s::vector,
                embed_model = %s,
                embedding_status = 'complete',
                embedded_at = NOW(),
                updated_at = NOW()
            WHERE id = %s
            """,
            [
                (embeddings[i].tolist(), NEW_MODEL, ids[i])
                for i in range(len(ids))
            ],
        )
        conn.commit()

    return len(ids)
```

29.8 Choosing Your Strategy: A Practical Framework

The decision between real-time and batch isn't binary — it's a configuration choice you revisit as your system evolves. The following framework captures the most common decision points:

Start with real-time when:

- Your corpus is small (< 100K documents) and ingestion is infrequent
- Your application cannot tolerate embedding lag (search must reflect writes immediately)
- You are prototyping and operational simplicity matters more than cost
- Query latency is your

primary SLA concern

Move to batch (or hybrid) when: - Ingestion volume exceeds what synchronous API calls can handle without rate limiting - You've profiled your real-time pipeline and embedding latency is your p95 bottleneck - Cost projections for synchronous API usage are unsustainable at projected scale - You need different models for different content types (a local model for bulk, API for high-priority)

Signs your real-time pipeline is under stress: - API rate limit errors appearing in logs (even at low frequency) - p95 embedding latency increasing while p50 stays stable (rate limiting backpressure) - Document ingestion success rate dropping during traffic peaks - Queue depth in your job system growing faster than it drains

Signs your batch pipeline needs tuning: - Queue lag exceeding your freshness SLA - Worker throughput plateauing despite available CPU/GPU capacity (usually a DB write bottleneck) - Failed status count growing (retry logic or content issue) - Batch sizes too small to amortize model loading overhead

Most teams at moderate scale land on a hybrid architecture that looks like this: a synchronous path for query-time embedding (unavoidable) and high-priority content, a Redis Streams or Kafka-backed queue for normal ingestion, and a scheduled batch job for bulk loads and re-embedding passes. The exact thresholds — what constitutes HIGH priority, how large a batch the worker pulls, how many worker instances run — are tuned empirically against your observed traffic patterns.

The architectural principle is simple: **keep the synchronous path as thin as possible**. Every operation you move off the synchronous request path is latency you give back to your users and a failure mode you decouple from your ingestion API.

Key Takeaways

- **Query embedding is always real-time; document embedding is the decision.** You cannot precompute embeddings for arbitrary user queries, so the synchronous embedding path always exists for search. The strategic question is whether document ingestion also lives on that path or defers to an async pipeline.
- **The FOR UPDATE SKIP LOCKED pattern enables horizontally scalable batch workers.** Multiple worker instances can safely consume from the same PostgreSQL-backed embedding queue without coordination or double-processing, using the same pattern as production job queue systems.
- **Content hash tracking makes stale embedding detection automatic.** Storing a generated `content_hash` column eliminates manual bookkeeping; any query comparing stored hash to current content immediately surfaces documents that need re-embedding after content updates or model migrations.
- **Cache embedding results at the content-hash level to reduce both cost and latency.** Re-embedding identical content (system prompts, repeated queries, templated documents) is pure waste; an in-process LRU cache or Redis cache with a content-hash key can eliminate a significant fraction of API calls in typical workloads.
- **Model migrations require parallel indexing and a deliberate cut-over, not an in-place update.** Changing embedding model or vector dimensions means the new vectors are incompatible with the existing index; the safe pattern is to build the new index alongside the

old one while the system continues serving queries, then cut over traffic once re-embedding is complete and quality is verified.

Chapter 30: Handling Embedding Model Version Changes

Embedding models are not static infrastructure. OpenAI retires `text-embedding-ada-002` in favor of `text-embedding-3-small` and `text-embedding-3-large`. Cohere ships `embed-v3` with a different vector dimension than `embed-v2`. A sentence-transformers model you fine-tuned six months ago gets replaced by a better checkpoint. Every time the model changes, every vector in your database becomes potentially incompatible with every new query vector — and the search results degrade silently unless you have a plan. This chapter covers what actually breaks during an embedding model transition, how to detect degradation before your users do, and the concrete migration strategies that let you move from one model to another without taking your search system offline.

Why Model Changes Break Vector Search

The core problem is geometric. Every embedding model defines its own latent space — a coordinate system where semantically similar concepts cluster together. When you switch models, you are not changing the data; you are changing the entire coordinate system. A document vector produced by `text-embedding-ada-002` (1536 dimensions) and a query vector produced by `text-embedding-3-small` (also 1536 dimensions, but a completely different latent space) are meaningless when compared with cosine similarity. The numbers look plausible. The system returns results. But those results are essentially random relative to what the user actually asked.

This failure mode is the worst kind in database systems: silent and gradual. There is no error thrown. There is no query timeout. The cosine distance computation succeeds perfectly — it just computes the distance between two points in incompatible coordinate systems. If you have monitoring on query latency but not on result quality, this class of failure will go undetected until users start complaining or A/B test metrics collapse.

From a database perspective, think of it like changing your collation rules on a text column after the data is already indexed. The index physically exists and returns results, but the sort order is now wrong relative to the application's expectations. The difference is that collation failures at least sometimes produce obviously garbled output. Embedding model mismatches produce results that look completely reasonable.

The specific failure scenarios are:

Dimension mismatch: The new model produces vectors of a different length than the old

model. `pgvector` will throw a hard error if you attempt to insert a 1024-dimensional vector into a `vector(1536)` column. Pinecone will reject the upsert with an HTTP 400. This is the *easy* failure to catch.

Latent space drift: Same vector dimension, completely different coordinate space. `text-embedding-ada-002` and `text-embedding-3-small` both produce 1536-dimensional vectors, but they are not comparable. This is the silent failure that requires active quality monitoring to detect.

Normalization differences: Some models produce unit-normalized vectors by default; others do not. If your index uses dot product similarity (which many ANN indexes optimize for), switching from a normalized to an unnormalized model changes the effective similarity function without changing your index configuration.

Tokenization boundary shifts: A new model version might chunk text differently at the tokenizer level, meaning that the same 512-token input produces a vector that reflects subtly different semantic content. This is the hardest to detect because the degradation is partial rather than total.

Detecting Degradation Before and After Migration

Before you migrate a single vector, establish a baseline you can measure against. Detection requires two things: a ground-truth evaluation set and a continuous monitoring pipeline.

Building an evaluation set

A ground truth evaluation set is a collection of queries where you already know which documents should rank in the top K results. For a customer support system, this might be 200 sample support tickets paired with the knowledge base articles that human agents actually used to resolve them. For a code search system, it might be 100 natural language queries paired with the functions they should retrieve.

The metric to track is Recall@K — specifically, what percentage of the known-relevant documents appear in the top K results. For most production systems, Recall@10 between 0.75 and 0.90 indicates healthy retrieval. A drop below 0.60 is a signal that something has gone wrong with your retrieval layer.

```
# Python
# Evaluation harness for embedding model comparison
# Requires: openai, sentence-transformers, numpy, scikit-learn

import numpy as np
from openai import OpenAI
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

client = OpenAI()

# Ground truth evaluation set
# Format: (query, [list of relevant doc ids])
EVAL_SET = [
    ("How do I reset my password?", ["doc_042", "doc_107"]),
    ("billing cycle change request", ["doc_019", "doc_023", "doc_091"]),
    ("API rate limit exceeded error", ["doc_055", "doc_056"]),
    ("cancel subscription", ["doc_011", "doc_012"]),
    ("two factor authentication setup", ["doc_088", "doc_089", "doc_090"]),
]
```



```

# Precomputed document corpus embeddings (already stored in your vector DB)
# In practice, load these from your database
def load_corpus_embeddings(model_label: str) -> tuple[dict, np.ndarray]:
    """Load document ID -> index mapping and embedding matrix for a given model."""
    # Placeholder: replace with actual DB retrieval
    # Returns: (doc_id_to_index: dict, embedding_matrix: np.ndarray)
    raise NotImplementedError("Connect to your vector store here")

def embed_queries_openai(queries: list[str], model: str) -> np.ndarray:
    response = client.embeddings.create(input=queries, model=model)
    return np.array([r.embedding for r in response.data])

def embed_queries_st(queries: list[str], model_name: str) -> np.ndarray:
    model = SentenceTransformer(model_name)
    return model.encode(queries, normalize_embeddings=True)

def recall_at_k(
    query_embeddings: np.ndarray,
    corpus_embeddings: np.ndarray,
    doc_id_to_index: dict,
    eval_set: list,
    k: int = 10
) -> float:
    """Compute Recall@K across the evaluation set."""
    index_to_doc_id = {v: k for k, v in doc_id_to_index.items()}
    sims = cosine_similarity(query_embeddings, corpus_embeddings)

    recall_scores = []
    for i, (query, relevant_ids) in enumerate(eval_set):
        top_k_indices = np.argsort(sims[i])[::-1][:k]
        top_k_doc_ids = {index_to_doc_id[idx] for idx in top_k_indices}
        relevant_set = set(relevant_ids)

        hits = len(relevant_set & top_k_doc_ids)
        recall_scores.append(hits / len(relevant_set))

    return float(np.mean(recall_scores))

def compare_models(eval_set: list, k: int = 10):
    queries = [q for q, _ in eval_set]

    models_to_test = [
        {
            "label": "ada-002",
            "embed_fn": lambda q: embed_queries_openai(q, "text-embedding-ada-002"),
        },
        {
            "label": "3-small",
            "embed_fn": lambda q: embed_queries_openai(q, "text-embedding-3-small"),
        },
        {
            "label": "3-large",
            "embed_fn": lambda q: embed_queries_openai(q, "text-embedding-3-large"),
        },
        {
            "label": "bge-large-en",
            "embed_fn": lambda q: embed_queries_st(q, "BAAI/bge-large-en-v1.5"),
        },
    ]

    print(f'{"Model":<20} {"Recall@' + str(k)<12} {"Latency (ms)":<15}')
```

```

print("-" * 48)

for model_cfg in models_to_test:
    import time
    start = time.perf_counter()
    query_embeddings = model_cfg["embed_fn"](queries)
    elapsed_ms = (time.perf_counter() - start) * 1000

```

```

try:
    doc_id_to_index, corpus_embeddings = load_corpus_embeddings(
        model_cfg["label"]
    )
    score = recall_at_k(
        query_embeddings, corpus_embeddings, doc_id_to_index, eval_set, k
    )
    print(
        f"{model_cfg['label']:<20} {score:<12.4f} {elapsed_ms:<15.1f}"
    )
except NotImplementedError:
    print(
        f"{model_cfg['label']:<20} {'N/A (load corpus)':<12} {elapsed_ms:<15.1f}"
    )

if __name__ == "__main__":
    compare_models(EVAL_SET, k=10)

```

Continuous monitoring in production

Run the evaluation set against your live system on a scheduled basis — every hour during a migration, daily in steady state. Store the scores in a time-series table alongside deployment metadata:

```

-- PostgreSQL / pgvector
-- Schema for tracking embedding model quality over time

CREATE TABLE embedding_model_registry (
    model_id          SERIAL PRIMARY KEY,
    model_name        TEXT NOT NULL,           -- 'text-embedding-3-small'
    model_version     TEXT NOT NULL,           -- 'v1', 'v2', or commit hash
    vector_dim        INTEGER NOT NULL,
    provider          TEXT NOT NULL,           -- 'openai', 'cohere', 'local'
    deployed_at       TIMESTAMPTZ,
    deprecated_at     TIMESTAMPTZ,
    is_active         BOOLEAN NOT NULL DEFAULT FALSE,
    metadata          JSONB DEFAULT '{}'::jsonb,
    UNIQUE (model_name, model_version)
);

CREATE TABLE retrieval_quality_log (
    log_id            BIGSERIAL PRIMARY KEY,
    recorded_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    model_id          INTEGER REFERENCES embedding_model_registry(model_id),
    eval_set_name     TEXT NOT NULL,
    k                 INTEGER NOT NULL,
    recall_at_k       NUMERIC(5,4) NOT NULL,
    ndcg_at_k         NUMERIC(5,4),
    eval_query_count  INTEGER NOT NULL,
    notes             TEXT
);

CREATE INDEX idx_rql_recorded_at ON retrieval_quality_log (recorded_at DESC);
CREATE INDEX idx_rql_model_id   ON retrieval_quality_log (model_id, recorded_at DESC);

-- Query: detect recall drops over the last 24 hours
WITH recent AS (
    SELECT
        r.model_id,
        reg.model_name,
        r.recorded_at,
        r.recall_at_k,
        AVG(r.recall_at_k) OVER (
            PARTITION BY r.model_id
            ORDER BY r.recorded_at
            ROWS BETWEEN 23 PRECEDING AND CURRENT ROW
        ) AS rolling_24h_avg
    FROM retrieval_quality_log r
    JOIN embedding_model_registry reg USING (model_id)

```

```
WHERE r.recorded_at > NOW() - INTERVAL '48 hours'
AND r.eval_set_name = 'production_eval_v1'
AND r.k = 10
)
SELECT
  model_name,
  recorded_at,
  recall_at_k,
  rolling_24h_avg,
  (recall_at_k - rolling_24h_avg) AS delta
FROM recent
WHERE ABS(recall_at_k - rolling_24h_avg) > 0.05 -- alert if >5 point swing
ORDER BY recorded_at DESC;
```

Migration Strategies: Zero-Downtime Approaches

There are three practical migration strategies. Which one you choose depends on your corpus size, your SLA requirements, and whether you have the infrastructure to run dual indexes simultaneously.

Strategy 1: Big Bang (Full Re-index)

Re-embed the entire corpus with the new model, load it into a new collection or table, then atomically switch traffic. Simple to reason about, but requires 2x storage during the transition and a cutover window where you must freeze writes to the old index while completing the final sync.

Suitable for corpora under ~5 million documents where the re-embedding job completes in under 4 hours and your application can tolerate brief read-only mode.

Strategy 2: Dual-Model Shadow Period

Run both the old model and the new model simultaneously. New documents are written to both indexes. Queries are served from the old index until you validate the new one. Then you drain the old index by re-embedding documents in batches during off-peak hours.

This is the pattern most production teams use because it decouples the “new model is ready” decision from the “all data is migrated” state.

```
# Python
# Dual-model write path using pgvector via psycopg2 and asyncpg
# Demonstrates shadow-writing to both old and new embedding columns

import asyncio
import asyncpg
from openai import AsyncOpenAI

client = AsyncOpenAI()

CREATE_SCHEMA = """
CREATE TABLE IF NOT EXISTS documents (
  id          TEXT PRIMARY KEY,
  content     TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  -- Old model column (ada-002, 1536 dims)
  embedding_ada002  vector(1536),
  embedding_ada002_at TIMESTAMPTZ,

  -- New model column (3-small, 1536 dims - different latent space)
  embedding_3small  vector(1536),
  embedding_3small_at TIMESTAMPTZ,
```

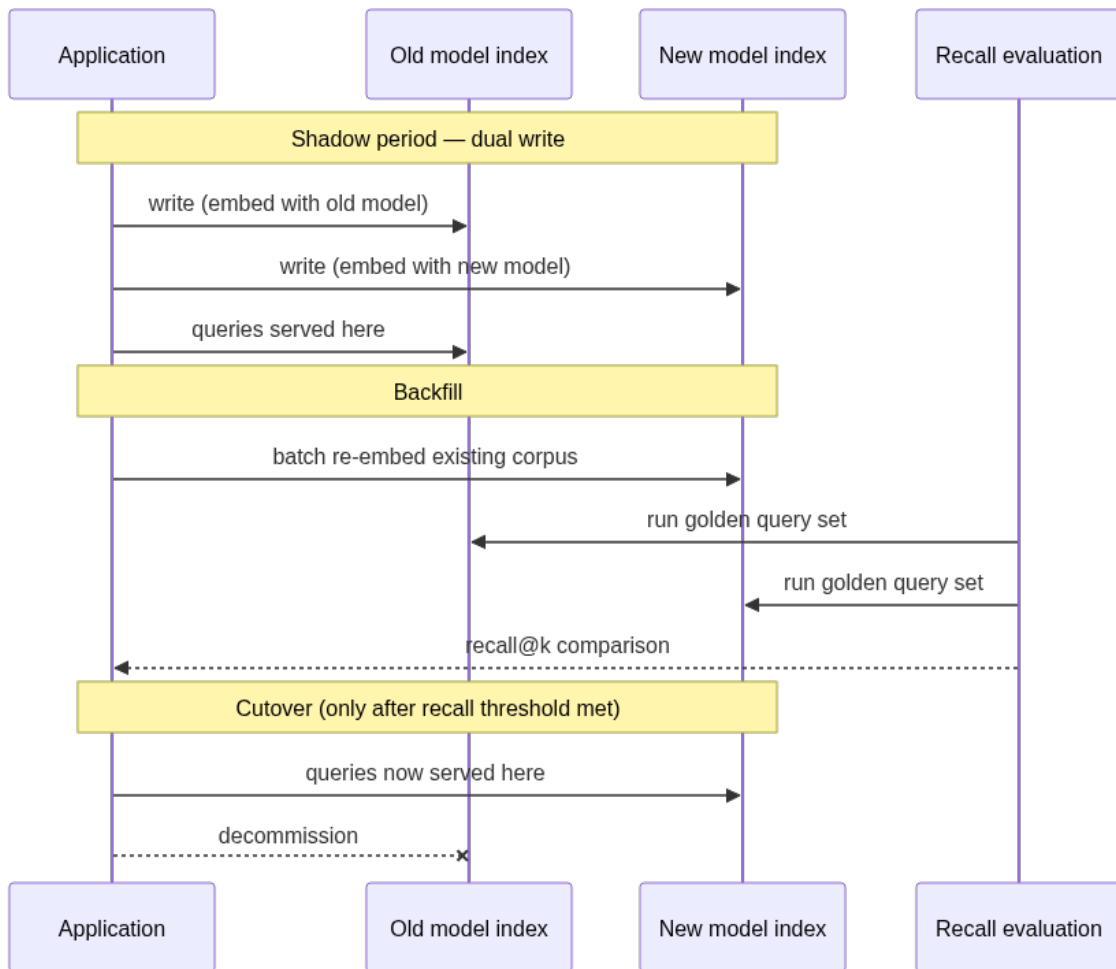


Figure 20: Dual-write shadow migration: new writes go to both indexes, the corpus backfills in the background, and query traffic cuts over only after recall parity is confirmed.

```

-- Which model is authoritative for reads
active_model TEXT NOT NULL DEFAULT 'ada-002'
CHECK (active_model IN ('ada-002', '3-small'))
);

-- Indexes for both models
CREATE INDEX IF NOT EXISTS idx_docs_ada002
ON documents USING hnsw (embedding_ada002 vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

CREATE INDEX IF NOT EXISTS idx_docs_3small
ON documents USING hnsw (embedding_3small vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
"""

async def embed_text(text: str, model: str) -> list[float]:
    response = await client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

async def upsert_document_dual_write(
    conn: asyncpg.Connection,
    doc_id: str,
    content: str,
    shadow_write_new_model: bool = True
):
    """
    Write to old model always.
    Write to new model if shadow_write_new_model=True.
    Enables gradual validation before flipping active_model.
    """
    # Embed with old model (always)
    ada_embedding = await embed_text(content, "text-embedding-ada-002")

    if shadow_write_new_model:
        # Embed with new model simultaneously
        small_embedding = await embed_text(content, "text-embedding-3-small")

        await conn.execute(
            """
            INSERT INTO documents (
                id, content,
                embedding_ada002, embedding_ada002_at,
                embedding_3small, embedding_3small_at
            )
            VALUES ($1, $2, $3::vector, NOW(), $4::vector, NOW())
            ON CONFLICT (id) DO UPDATE SET
                content = EXCLUDED.content,
                embedding_ada002 = EXCLUDED.embedding_ada002,
                embedding_ada002_at = EXCLUDED.embedding_ada002_at,
                embedding_3small = EXCLUDED.embedding_3small,
                embedding_3small_at = EXCLUDED.embedding_3small_at
            """,
            doc_id, content,
            str(ada_embedding),
            str(small_embedding),
        )
    else:
        await conn.execute(
            """
            INSERT INTO documents (id, content, embedding_ada002, embedding_ada002_at)
            VALUES ($1, $2, $3::vector, NOW())
            ON CONFLICT (id) DO UPDATE SET
                content = EXCLUDED.content,
                embedding_ada002 = EXCLUDED.embedding_ada002,
                embedding_ada002_at = EXCLUDED.embedding_ada002_at
            """,
            doc_id, content, str(ada_embedding),
        )

async def search_documents(

```

```

conn: asyncpg.Connection,
query: str,
model_override: str | None = None,
top_k: int = 10
) -> list[dict]:
    """
    Search using whichever model is currently active,
    or a specific model if overriding for A/B testing.
    """
    # Determine active model (read from a config row or feature flag)
    active_model = model_override or await get_active_model(conn)

    if active_model == "ada-002":
        embedding = await embed_text(query, "text-embedding-ada-002")
        col = "embedding_ada002"
    else:
        embedding = await embed_text(query, "text-embedding-3-small")
        col = "embedding_3small"

    rows = await conn.fetch(
        f"""
        SELECT
            id,
            content,
            active_model,
            1 - ({col} <=> $1::vector) AS cosine_similarity
        FROM documents
        WHERE {col} IS NOT NULL
        ORDER BY {col} <=> $1::vector
        LIMIT $2
        """,
        str(embedding), top_k
    )
    return [dict(r) for r in rows]

async def get_active_model(conn: asyncpg.Connection) -> str:
    # In practice, store this in a config table or read from a feature flag service
    row = await conn.fetchrow(
        "SELECT value FROM app_config WHERE key = 'active_embedding_model'"
    )
    return row["value"] if row else "ada-002"

async def flip_active_model(conn: asyncpg.Connection, new_model: str):
    """
    Atomic model flip - called only after backfill is complete
    and quality metrics have been validated.
    """
    # Verify backfill completeness before flipping
    gap = await conn.fetchval(
        f"""
        SELECT COUNT(*) FROM documents
        WHERE embedding_3small IS NULL
        AND active_model = 'ada-002'
        """
    )
    if gap > 0:
        raise RuntimeError(
            f"Cannot flip: {gap} documents still missing new embeddings"
        )

    await conn.execute(
        "UPDATE app_config SET value = $1 WHERE key = 'active_embedding_model'",
        new_model
    )
    # Also update all rows to reflect new authoritative model
    await conn.execute("UPDATE documents SET active_model = $1", new_model)
    print(f"Active model flipped to: {new_model}")

```

Strategy 3: Rolling Backfill with Pinecone Namespaces

Pinecone namespaces provide a clean mechanism for this pattern when you are not using a self-hosted vector database. You create a new namespace for the new model and backfill it in batches while the old namespace continues serving traffic.

```
# Python
# Rolling backfill using Pinecone namespaces for model isolation
# Requires: pinecone-client>=3.0, openai

from pinecone import Pinecone
from openai import OpenAI
import time

pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")
openai_client = OpenAI()

# Two indexes: one per model dimension if dimensions differ,
# or one index with two namespaces if dimensions are the same
INDEX_NAME = "knowledge-base" # Must exist with dim=1536

OLD_NAMESPACE = "model-ada002"
NEW_NAMESPACE = "model-3small"

index = pc.Index(INDEX_NAME)

def embed_batch_openai(texts: list[str], model: str) -> list[list[float]]:
    response = openai_client.embeddings.create(input=texts, model=model)
    return [r.embedding for r in response.data]

def backfill_namespace(
    documents: list[dict], # [{"id": str, "text": str, "metadata": dict}]
    target_model: str,
    target_namespace: str,
    batch_size: int = 100,
    requests_per_minute: int = 3000
):
    """
    Re-embed all documents with new model and upsert into new namespace.
    Rate limiting respects OpenAI RPM limits.
    """
    total = len(documents)
    upserted = 0
    skipped = 0

    # Check which IDs are already in the new namespace
    # to support resumable backfill
    all_ids = [doc["id"] for doc in documents]
    existing_ids = set()

    # Fetch in batches of 1000 (Pinecone fetch limit)
    for i in range(0, len(all_ids), 1000):
        batch_ids = all_ids[i:i+1000]
        fetch_result = index.fetch(ids=batch_ids, namespace=target_namespace)
        existing_ids.update(fetch_result.vectors.keys())

    print(f"Total documents: {total}")
    print(f"Already backfilled: {len(existing_ids)}")
    print(f"Remaining: {total - len(existing_ids)}")

    pending = [doc for doc in documents if doc["id"] not in existing_ids]

    for i in range(0, len(pending), batch_size):
        batch = pending[i:i + batch_size]
        texts = [doc["text"] for doc in batch]

        try:
            embeddings = embed_batch_openai(texts, target_model)
        except Exception as e:
            print(f"Embedding failed for batch {i//batch_size}: {e}")
            time.sleep(60)
```

```

        continue

    vectors = [
        {
            "id": doc["id"],
            "values": emb,
            "metadata": {
                **doc.get("metadata", {}),
                "embedding_model": target_model,
                "backfilled_at": int(time.time()),
            }
        }
        for doc, emb in zip(batch, embeddings)
    ]

    index.upsert(vectors=vectors, namespace=target_namespace)
    upserted += len(vectors)

    progress_pct = (upserted / len(pending)) * 100
    print(f"Progress: {upserted}/{len(pending)} ({progress_pct:.1f}%")

    # Respect rate limits: ~60s / RPM * batch_size
    sleep_seconds = (batch_size / requests_per_minute) * 60
    time.sleep(max(sleep_seconds, 0.1))

    print(f"Backfill complete. Upserted: {upserted}, Skipped: {skipped}")
    return upserted

def query_with_model_routing(
    query_text: str,
    active_namespace: str,
    top_k: int = 10,
    filter_metadata: dict | None = None
) -> list[dict]:
    """
    Query the active namespace. Namespace determines which model's
    embeddings are used - the embedding model must match the namespace.
    """
    model_map = {
        OLD_NAMESPACE: "text-embedding-ada-002",
        NEW_NAMESPACE: "text-embedding-3-small",
    }
    model = model_map[active_namespace]

    response = openai_client.embeddings.create(
        input=query_text,
        model=model
    )
    query_vector = response.data[0].embedding

    results = index.query(
        vector=query_vector,
        top_k=top_k,
        namespace=active_namespace,
        filter=filter_metadata,
        include_metadata=True
    )

    return [
        {
            "id": match.id,
            "score": match.score,
            "metadata": match.metadata,
        }
        for match in results.matches
    ]

```

The namespace approach has an important operational advantage: you can run A/B traffic splits simply by routing a fraction of queries to `NEW_NAMESPACE` while the backfill is still in progress — as

long as you track which documents have been backfilled so you do not count misses from documents that are not yet present in the new namespace.

Backfill Job Design: Resumability and Rate Limiting

A backfill job that processes five million documents will take hours. It will be interrupted. The network will hiccup. The embedding API will return a 429. Your database connection will drop. The job must be resumable from the point of failure without reprocessing already-migrated documents and without leaving the corpus in an inconsistent state.

The pattern is checkpoint tables. Every batch that completes successfully writes its completion state to a tracking table. On restart, the job reads the highest completed checkpoint and picks up from there.

```
-- PostgreSQL
-- Backfill tracking schema for large-scale re-embedding jobs

CREATE TABLE embedding_backfill_jobs (
  job_id          SERIAL PRIMARY KEY,
  job_name        TEXT NOT NULL UNIQUE,
  source_model    TEXT NOT NULL,
  target_model    TEXT NOT NULL,
  target_column   TEXT NOT NULL,
  total_docs      BIGINT,
  started_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at    TIMESTAMPTZ,
  status          TEXT NOT NULL DEFAULT 'running'
    CHECK (status IN ('running', 'completed', 'failed', 'paused')),
  last_error      TEXT
);

CREATE TABLE embedding_backfill_progress (
  progress_id     BIGSERIAL PRIMARY KEY,
  job_id          INTEGER NOT NULL REFERENCES embedding_backfill_jobs(job_id),
  batch_number    INTEGER NOT NULL,
  doc_id_min      TEXT NOT NULL,      -- first doc id in this batch (for resume cursor)
  doc_id_max      TEXT NOT NULL,      -- last doc id in this batch
  docs_in_batch   INTEGER NOT NULL,
  completed_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  duration_ms     INTEGER,
  UNIQUE (job_id, batch_number)
);

CREATE INDEX idx_backfill_progress_job ON embedding_backfill_progress (job_id, batch_number DESC);

-- View: current backfill status
CREATE VIEW v_backfill_status AS
SELECT
  j.job_id,
  j.job_name,
  j.source_model,
  j.target_model,
  j.status,
  j.total_docs,
  COALESCE(SUM(p.docs_in_batch), 0) AS docs_completed,
  CASE
    WHEN j.total_docs > 0
    THEN ROUND(100.0 * COALESCE(SUM(p.docs_in_batch), 0) / j.total_docs, 2)
    ELSE 0
  END AS pct_complete,
  MAX(p.completed_at) AS last_batch_at,
  j.started_at,
```

```

        EXTRACT(EPOCH FROM (COALESCE(j.completed_at, NOW()) - j.started_at)) / 3600
        AS elapsed_hours
FROM embedding_backfill_jobs j
LEFT JOIN embedding_backfill_progress p USING (job_id)
GROUP BY j.job_id, j.job_name, j.source_model, j.target_model,
         j.status, j.total_docs, j.started_at, j.completed_at;

```

The corresponding Python job runner reads the checkpoint, processes in batches, and writes progress atomically with the embedding upsert:

```

# Python
# Resumable backfill runner for pgvector dual-column migration
# Uses a cursor-based approach rather than OFFSET to avoid slow deep pagination

import asyncio
import asyncpg
import time
from openai import AsyncOpenAI

openai_client = AsyncOpenAI()

async def get_or_create_job(
    conn: asyncpg.Connection,
    job_name: str,
    source_model: str,
    target_model: str,
    target_column: str,
) -> dict:
    row = await conn.fetchrow(
        "SELECT * FROM embedding_backfill_jobs WHERE job_name = $1",
        job_name
    )
    if row:
        return dict(row)

    total_docs = await conn.fetchval(
        f"SELECT COUNT(*) FROM documents WHERE {target_column} IS NULL"
    )
    job_id = await conn.fetchval(
        """
        INSERT INTO embedding_backfill_jobs
            (job_name, source_model, target_model, target_column, total_docs)
        VALUES ($1, $2, $3, $4, $5)
        RETURNING job_id
        """,
        job_name, source_model, target_model, target_column, total_docs
    )
    return {"job_id": job_id, "job_name": job_name, "total_docs": total_docs}

async def get_resume_cursor(
    conn: asyncpg.Connection,
    job_id: int
) -> str | None:
    """Return the doc_id_max of the last completed batch, or None to start from beginning."""
    row = await conn.fetchrow(
        """
        SELECT doc_id_max
        FROM embedding_backfill_progress
        WHERE job_id = $1
        ORDER BY batch_number DESC
        LIMIT 1
        """,
        job_id
    )
    return row["doc_id_max"] if row else None

async def run_backfill(
    conn: asyncpg.Connection,
    job_name: str,

```

```

target_model: str,
target_column: str,
source_model: str = "text-embedding-ada-002",
batch_size: int = 50,
max_retries: int = 3,
):
    job = await get_or_create_job(
        conn, job_name, source_model, target_model, target_column
    )
    job_id = job["job_id"]

    cursor = await get_resume_cursor(conn, job_id)
    batch_number = await conn.fetchval(
        "SELECT COALESCE(MAX(batch_number), 0) FROM embedding_backfill_progress WHERE job_id = $1",
        job_id
    ) or 0

    print(f"Starting backfill job '{job_name}' (id={job_id})")
    print(f"Resuming from cursor: {cursor or 'beginning'}")
    print(f"Total documents: {job['total_docs']}")

    while True:
        # Cursor-based pagination: always faster than OFFSET on large tables
        if cursor is None:
            rows = await conn.fetch(
                f"""
                SELECT id, content FROM documents
                WHERE {target_column} IS NULL
                ORDER BY id
                LIMIT $1
                """,
                batch_size
            )
        else:
            rows = await conn.fetch(
                f"""
                SELECT id, content FROM documents
                WHERE {target_column} IS NULL
                AND id > $1
                ORDER BY id
                LIMIT $2
                """,
                cursor, batch_size
            )

        if not rows:
            break # All documents processed

        texts = [r["content"] for r in rows]
        doc_ids = [r["id"] for r in rows]

        # Retry loop for embedding API failures
        embeddings = None
        for attempt in range(max_retries):
            try:
                t0 = time.perf_counter()
                response = await openai_client.embeddings.create(
                    input=texts,
                    model=target_model
                )
                embeddings = [e.embedding for e in response.data]
                elapsed_ms = int((time.perf_counter() - t0) * 1000)
                break
            except Exception as e:
                wait = 2 ** attempt * 5
                print(f"Attempt {attempt+1} failed: {e}. Retrying in {wait}s...")
                await asyncio.sleep(wait)

        if embeddings is None:
            await conn.execute(

```

```

        "UPDATE embedding_backfill_jobs SET status='failed', last_error=$1 WHERE job_id=$2",
        f"Embedding API failed after {max_retries} retries on cursor {cursor}",
        job_id
    )
    raise RuntimeError("Embedding API permanently failed. Job marked failed.")

# Write embeddings and checkpoint in a single transaction
async with conn.transaction():
    for doc_id, embedding in zip(doc_ids, embeddings):
        await conn.execute(
            f"""
            UPDATE documents
            SET {target_column} = ${1::vector},
                {target_column}_at = NOW()
            WHERE id = $2
            """,
            str(embedding), doc_id
        )

    batch_number += 1
    await conn.execute(
        """
        INSERT INTO embedding_backfill_progress
        (job_id, batch_number, doc_id_min, doc_id_max, docs_in_batch, duration_ms)
        VALUES ($1, $2, $3, $4, $5, $6)
        """,
        job_id, batch_number,
        doc_ids[0], doc_ids[-1],
        len(doc_ids), elapsed_ms
    )

    cursor = doc_ids[-1]
    print(f"Batch {batch_number} complete: {len(doc_ids)} docs, cursor={cursor}")

    await conn.execute(
        "UPDATE embedding_backfill_jobs SET status='completed', completed_at=NOW() WHERE job_id=$1",
        job_id
    )
    print(f"Backfill '{job_name}' complete.")

```

One detail worth highlighting: the embedding update and the checkpoint write happen inside the same transaction. If the process crashes after writing the embeddings but before writing the checkpoint, the transaction rolls back entirely. On restart, the cursor will land on a batch that was already partially written, and those documents get re-embedded (an idempotent operation). If the process crashes after writing the checkpoint, the transaction was committed, so those documents are correctly skipped on restart. The cursor-based pagination via `id > $cursor` avoids the `OFFSET` performance cliff that hits hard once you are millions of rows into a table.

Handling Dimension Changes: The Hard Case

When the new model produces vectors of a different dimension, you cannot use the dual-column approach above without schema changes. Cohere's transition from `embed-english-v2.0` (4096 dimensions) to `embed-english-v3.0` (1024 dimensions) is a real example of this. The same challenge applies if you move from `text-embedding-3-small` (1536 or configurable lower) to a model with a fundamentally different dimensionality.

In `pgvector`, vector columns are typed with a fixed dimension: `vector(1536)` will reject a `vector(1024)` insert. You have two options:

Option A: Add a new typed column

```

-- PostgreSQL
-- Adding a second vector column for a model with different dimensions

ALTER TABLE documents
  ADD COLUMN embedding_cohere_v3 vector(1024),
  ADD COLUMN embedding_cohere_v3_at TIMESTAMPTZ;

-- Build the HNSW index only after backfill completes
-- (building during backfill is wasteful; rows without embeddings
-- are ignored by the index but each INSERT triggers maintenance)
-- Use CREATE INDEX CONCURRENTLY to avoid locking the table
CREATE INDEX CONCURRENTLY idx_docs_cohere_v3
  ON documents USING hnsw (embedding_cohere_v3 vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

```

Option B: Separate table per model

For very large dimension changes or when you want full index independence, maintain a separate embeddings table that joins back to the primary document table:

```

-- PostgreSQL
-- Normalized embedding storage: one row per (document, model) pair
-- Supports arbitrary dimension changes without altering the documents table

CREATE TABLE document_embeddings (
  doc_id      TEXT NOT NULL,
  model_id    INTEGER NOT NULL REFERENCES embedding_model_registry(model_id),
  embedding    vector,           -- untyped vector; dimension enforced by constraint
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (doc_id, model_id)
);

-- Dimension guard: enforce at the application layer or via a trigger
-- pgvector does not support a per-row dimension constraint on untyped vector columns,
-- so use a check constraint paired with model registry

-- For typed storage with a known dimension, use a partitioned approach:
CREATE TABLE document_embeddings_1536 (
  doc_id      TEXT NOT NULL,
  model_id    INTEGER NOT NULL,
  embedding    vector(1536) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (doc_id, model_id)
);

CREATE TABLE document_embeddings_1024 (
  doc_id      TEXT NOT NULL,
  model_id    INTEGER NOT NULL,
  embedding    vector(1024) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (doc_id, model_id)
);

-- HNSW indexes on each dimension table
CREATE INDEX idx_emb_1536_hnsw
  ON document_embeddings_1536 USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

CREATE INDEX idx_emb_1024_hnsw
  ON document_embeddings_1024 USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

```

The separate-table approach is more operationally complex but gives you complete index independence. You can drop `document_embeddings_1536` (and its HNSW index, which can be enormous) once the migration to the 1024-dimension model is complete, reclaiming storage without any structural change to the documents table.

In Weaviate, dimension changes require a new collection entirely — Weaviate’s schema is class-level and the vector dimension is fixed at class creation. The migration pattern is:

1. Create a new Weaviate class with the target model configured as the vectorizer.
2. Run a batch import job that reads objects from the old class and imports them into the new class. Weaviate will automatically call the configured vectorizer for each object during import, so you do not need to manage embeddings explicitly if you use Weaviate’s built-in vectorizer modules.
3. Validate quality against the new class.
4. Update your application’s class name reference (typically a configuration value) and delete the old class.

```
# Python
# Weaviate collection migration for model dimension change
# Requires: weaviate-client>=4.0

import weaviate
from weaviate.classes.config import Configure, VectorDistances
from weaviate.classes.query import MetadataQuery
import time

client = weaviate.connect_to_local() # or connect_to_wcs(...)

OLD_CLASS = "DocumentAda002"
NEW_CLASS = "DocumentCohere3"

def create_new_collection():
    """Create target collection with new vectorizer."""
    # Check if already exists (supports resumable migration)
    if client.collections.exists(NEW_CLASS):
        print(f"Collection {NEW_CLASS} already exists, skipping creation.")
        return

    client.collections.create(
        name=NEW_CLASS,
        vectorizer_config=Configure.Vectorizer.text2vec_cohere(
            model="embed-english-v3.0",
            truncate="RIGHT",
        ),
        vector_index_config=Configure.VectorIndex.hnsw(
            distance_metric=VectorDistances.COSINE,
            ef_construction=128,
            max_connections=16,
        ),
        # Mirror properties from old collection
        properties=[
            weaviate.classes.config.Property(name="doc_id",
↪ data_type=weaviate.classes.config.DataType.TEXT),
            weaviate.classes.config.Property(name="content",
↪ data_type=weaviate.classes.config.DataType.TEXT),
            weaviate.classes.config.Property(name="source",
↪ data_type=weaviate.classes.config.DataType.TEXT),
        ]
    )
    print(f"Created collection {NEW_CLASS}")

def migrate_objects(batch_size: int = 200):
    """
    Read all objects from old collection and import into new collection.
    Weaviate re-vectorizes automatically via the configured vectorizer.
    """
    old_collection = client.collections.get(OLD_CLASS)
    new_collection = client.collections.get(NEW_CLASS)

    # Get already-migrated IDs for resumability
    existing_ids = set()
```

```

for obj in new_collection.iterator(include_vector=False):
    # Use the stored doc_id property as the dedup key
    existing_ids.add(obj.properties.get("doc_id"))

print(f"Already migrated: {len(existing_ids)} objects")

total_migrated = 0
batch_buffer = []

for obj in old_collection.iterator(include_vector=False):
    doc_id = obj.properties.get("doc_id")
    if doc_id in existing_ids:
        continue

    batch_buffer.append({
        "doc_id": doc_id,
        "content": obj.properties.get("content", ""),
        "source": obj.properties.get("source", ""),
    })

    if len(batch_buffer) >= batch_size:
        with new_collection.batch.fixed_size(batch_size=batch_size) as batch:
            for props in batch_buffer:
                batch.add_object(properties=props)

        total_migrated += len(batch_buffer)
        batch_buffer = []
        print(f"Migrated {total_migrated} objects so far...")
        time.sleep(0.5) # Brief pause to respect vectorizer API rate limits

# Flush remaining
if batch_buffer:
    with new_collection.batch.fixed_size(batch_size=len(batch_buffer)) as batch:
        for props in batch_buffer:
            batch.add_object(properties=props)
    total_migrated += len(batch_buffer)

print(f"Migration complete. Total migrated: {total_migrated}")
return total_migrated

def validate_and_cutover():
    """
    Run a spot-check query against both collections and compare result counts.
    Replace with your full Recall@K evaluation before production cutover.
    """
    new_collection = client.collections.get(NEW_CLASS)
    test_queries = [
        "password reset instructions",
        "billing and payment questions",
        "API authentication errors",
    ]

    all_have_results = True
    for query_text in test_queries:
        results = new_collection.query.near_text(
            query=query_text,
            limit=5,
            return_metadata=MetadataQuery(score=True)
        )
        if not results.objects:
            print(f"WARNING: No results for '{query_text}' in new collection")
            all_have_results = False
        else:
            top_score = results.objects[0].metadata.score
            print(f"'{query_text}': {len(results.objects)} results, top score={top_score:.4f}")

    if all_have_results:
        print("Spot check passed. Update your application's WEAVIATE_COLLECTION env var to:", NEW_CLASS)
        print("Then delete old collection with: client.collections.delete(OLD_CLASS)")
    else:

```

```
print("Spot check failed. Do not cut over.")

if __name__ == "__main__":
    create_new_collection()
    migrate_objects(batch_size=200)
    validate_and_cutover()
    client.close()
```

Version Pinning and Model Deprecation Tracking

The least glamorous part of embedding model management is also the part that saves you the most pain: knowing in advance that a model is going to be deprecated so you can migrate on your own schedule instead of under a deadline.

Store the model identifier alongside every vector you produce. This is good practice regardless of migration concerns — it lets you audit which documents were embedded with which model and prioritize backfill for the most-queried content.

```
-- PostgreSQL
-- Embedding provenance tracking via a metadata column

ALTER TABLE documents
    ADD COLUMN IF NOT EXISTS embedding_model TEXT,
    ADD COLUMN IF NOT EXISTS embedding_model_version TEXT,
    ADD COLUMN IF NOT EXISTS embedded_at TIMESTAMPTZ;

-- Update when embedding is written
-- (shown as part of an upsert trigger for automatic tracking)

CREATE OR REPLACE FUNCTION trg_set_embedding_provenance()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.embedding_ada002 IS DISTINCT FROM OLD.embedding_ada002 THEN
        NEW.embedding_model := 'text-embedding-ada-002';
        NEW.embedding_model_version := 'v1';
        NEW.embedded_at := NOW();
    END IF;
    IF NEW.embedding_3small IS DISTINCT FROM OLD.embedding_3small THEN
        NEW.embedding_model := 'text-embedding-3-small';
        NEW.embedding_model_version := 'v1';
        NEW.embedded_at := NOW();
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_embedding_provenance
BEFORE INSERT OR UPDATE ON documents
FOR EACH ROW EXECUTE FUNCTION trg_set_embedding_provenance();

-- Query: how many documents are on each model version?
SELECT
    embedding_model,
    embedding_model_version,
    COUNT(*) AS doc_count,
    MIN(embedded_at) AS first_embedded,
    MAX(embedded_at) AS last_embedded
FROM documents
WHERE embedding_model IS NOT NULL
GROUP BY embedding_model, embedding_model_version
ORDER BY doc_count DESC;
```

For the SQL Server side, the equivalent uses a computed column with a default and an AFTER

UPDATE trigger, since SQL Server does not have BEFORE triggers:

```
-- SQL Server
-- Embedding provenance tracking

ALTER TABLE documents
  ADD embedding_model      NVARCHAR(100) NULL,
  ADD embedding_model_version NVARCHAR(50) NULL,
  ADD embedded_at         DATETIMEOFFSET NULL;

-- SQL Server uses AFTER triggers; capture the change via UPDATE()
CREATE OR ALTER TRIGGER trg_embedding_provenance
ON documents
AFTER INSERT, UPDATE
AS
BEGIN
  SET NOCOUNT ON;

  IF UPDATE(embedding_ada002)
  BEGIN
    UPDATE d
    SET
      d.embedding_model      = 'text-embedding-ada-002',
      d.embedding_model_version = 'v1',
      d.embedded_at         = SYSDATETIMEOFFSET()
    FROM documents d
    INNER JOIN inserted i ON d.id = i.id;
  END

  -- SQL Server evaluates both IF blocks; last one wins if both columns updated.
  -- In practice, only one embedding column changes per write.
  IF UPDATE(embedding_3small)
  BEGIN
    UPDATE d
    SET
      d.embedding_model      = 'text-embedding-3-small',
      d.embedding_model_version = 'v1',
      d.embedded_at         = SYSDATETIMEOFFSET()
    FROM documents d
    INNER JOIN inserted i ON d.id = i.id;
  END
END;
GO

-- Query: distribution of documents by model
SELECT
  embedding_model,
  embedding_model_version,
  COUNT(*) AS doc_count,
  MIN(embedded_at) AS first_embedded,
  MAX(embedded_at) AS last_embedded
FROM documents
WHERE embedding_model IS NOT NULL
GROUP BY embedding_model, embedding_model_version
ORDER BY doc_count DESC;
```

Pair this provenance tracking with a deprecation calendar. When OpenAI announces a model retirement date, create a row in your `embedding_model_registry` table with `deprecated_at` set to that date and configure an alert that fires 90 days before deprecation if any documents still carry that model's identifier. The combination of provenance data and a deprecation alert turns a potential emergency migration into a planned project.

Key Takeaways

- **Silent degradation is the primary risk.** A model swap that changes the latent space without changing vector dimensions will produce plausible-looking but semantically wrong results with no error signals from the database layer. Recall@K monitoring against a static evaluation set is your only reliable early warning system.
 - **Establish quality baselines before you migrate.** Build a ground-truth evaluation set during your current model's healthy operation. Without a baseline, you cannot distinguish a migration regression from preexisting retrieval quality problems.
 - **Dual-model shadow writing decouples deployment from migration.** Writing new documents to both the old and new embedding columns simultaneously means that by the time your backfill completes, the most-recently-created (and typically most-queried) documents are already on the new model. The final cutover touches only older, lower-traffic content.
 - **Make backfill jobs resumable from the first design.** Cursor-based pagination combined with transactional checkpoint writes means any crash is recoverable without reprocessing the entire corpus. Embedding API costs and clock time are too significant to pay twice on a multi-million-document corpus.
 - **Track embedding provenance on every row.** Storing which model produced each vector transforms deprecation management from a fire drill into a scheduled maintenance task. When you know exactly which documents need re-embedding and how many remain, you can plan compute cost, schedule backfill windows, and build accurate progress dashboards without guesswork.
-

Chapter 31: Multi-Modal Search: Text and Image Together

Most production search systems today handle one modality at a time — a text query searches text, an image query searches images. But users do not experience information in those silos. A retail customer uploads a photo of shoes and types “like these but in red.” A medical researcher searches with an MRI scan and a clinical note simultaneously. A news archive needs to find articles where the image and headline tell contradictory stories. These compound queries require a database layer that understands both text and images as first-class embedding types and can combine them during retrieval. This chapter covers the embedding models that bridge the modality gap, the storage and indexing strategies for mixed-type vectors, the query architectures that fuse signals from both dimensions, and the production patterns that make multi-modal search scale reliably.

The Embedding Bridge: How Text and Images Share a Vector Space

The foundational insight behind multi-modal search is that text and images can be embedded into the *same* vector space — a shared coordinate system where semantic proximity means the same thing regardless of modality. A vector for the text “golden retriever playing fetch” and a vector for an actual photograph of that scene should land close together in this space. This is not obvious from first principles; it required learning a joint embedding model from hundreds of millions of image-caption pairs.

CLIP and Its Descendants

OpenAI’s CLIP (Contrastive Language-Image Pre-training, 2021) established the practical template. CLIP trains a text encoder and an image encoder jointly using contrastive loss — during training, the model pulls matching image-text pairs together while pushing non-matching pairs apart. The result is two encoders that produce vectors in the same 512- or 768-dimensional space.

For production use, the relevant CLIP derivatives include:

Model	Dimensions	Notes
openai/clip-vit-base-patch32	512	Baseline; widely benchmarked
openai/clip-vit-large-patch14	768	Better quality, higher latency

Model	Dimensions	Notes
laion/CLIP-ViT-H-14-laion2B-13B-b79K	1024	Trained on LAION-2B; strong zero-shot
sentence-transformers/clip-ViT2B-32	512	HuggingFace wrapper; easier pipeline
Google's SigLIP	1152	Sigmoid loss variant; better recall at scale
OpenAI text-embedding-3-large	3072	Text only; not image-capable

The practical implication for database architects: you are storing vectors from the *same* model family in the same table or collection. If you embed images with `clip-vit-large-patch14`, you must embed all query text with that same model's text encoder. Mixing encoders from different CLIP checkpoints produces meaningless distance comparisons — equivalent to storing lengths in meters in one column and feet in another and comparing them without conversion.

Dimensionality and Distance Metrics

CLIP vectors use cosine similarity as the natural distance metric because the encoders produce unit-normalized vectors by default. In pgvector terms, that means using the `vector` type with `<=>` (cosine distance) rather than `<->` (L2). Pinecone, Weaviate, and Qdrant all support cosine similarity natively and it should be your default for CLIP-family embeddings.

One architectural decision that often surprises DBAs: you can store text embeddings and image embeddings in the *same* column when they share the same model and dimensionality. There is no type system distinction between “image vector” and “text vector” at the database level. The distinction is carried by your application logic and metadata columns. This is similar to storing both customer and product records in a single `entities` table — possible, but you need a `type` discriminator column to avoid joining apples with oranges in business queries.

Storage Architecture for Mixed-Modal Data

Before writing any queries, you need a schema that accommodates multiple modalities without creating a maintenance nightmare. There are three patterns worth knowing, each with different tradeoffs.

Pattern 1: Single Table, Shared Vector Column

All embeddings live in one table. A `modality` column distinguishes text from image. This works well when text and image documents are genuinely interchangeable from a business perspective — a product catalog where each item has both a description text and one or more product images.

```
-- PostgreSQL / pgvector
-- Single-table multi-modal schema for a product catalog

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE product_embeddings (
  id          BIGSERIAL PRIMARY KEY,
  product_id  BIGINT NOT NULL REFERENCES products(id),
```

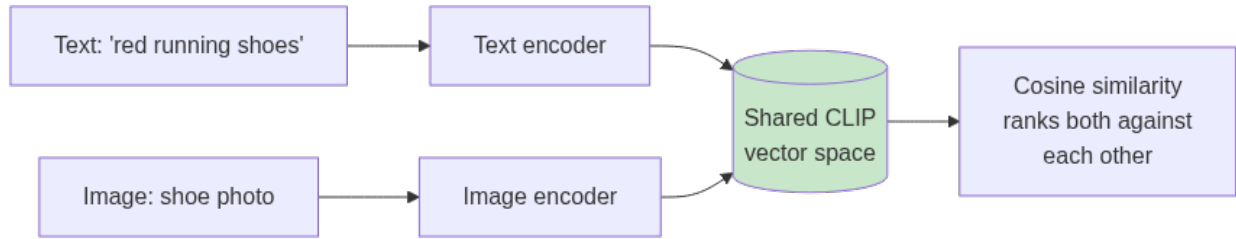


Figure 21: Text and image encoders trained jointly land vectors in the same coordinate system — this is what makes cross-modal retrieval possible.

```

modality      TEXT NOT NULL CHECK (modality IN ('text', 'image')),
source_ref    TEXT NOT NULL,    -- field name or image URL/path
embedding     VECTOR(512),      -- CLIP ViT-B/32 dimensions
created_at    TIMESTAMPTZ DEFAULT NOW(),
metadata      JSONB
);

-- Separate HNSW indexes per modality for query efficiency
CREATE INDEX idx_product_emb_text
ON product_embeddings
USING hnsw (embedding vector_cosine_ops)
WHERE modality = 'text';

CREATE INDEX idx_product_emb_image
ON product_embeddings
USING hnsw (embedding vector_cosine_ops)
WHERE modality = 'image';

-- Composite index for filtered product lookups
CREATE INDEX idx_product_emb_product_modality
ON product_embeddings (product_id, modality);

```

The partial HNSW indexes on `modality` are a pgvector-specific optimization: when a query filters by `modality` before applying the nearest-neighbor scan, a partial index prevents the planner from scanning irrelevant rows. Without them, a text query would still traverse image vectors in the HNSW graph before filtering — wasting both I/O and compute.

Pattern 2: Separate Tables per Modality

More explicit and easier to maintain independently. Use this when text and image pipelines have very different update rates or when the embedding models differ in dimensionality.

```

-- PostgreSQL / pgvector
-- Separate tables for text and image embeddings

CREATE TABLE document_text_embeddings (
  id          BIGSERIAL PRIMARY KEY,
  doc_id      BIGINT NOT NULL,
  chunk_index INT NOT NULL DEFAULT 0,
  content     TEXT NOT NULL,
  embedding   VECTOR(1536),    -- OpenAI text-embedding-3-small
  metadata    JSONB,
  UNIQUE (doc_id, chunk_index)
);

CREATE TABLE document_image_embeddings (
  id          BIGSERIAL PRIMARY KEY,
  doc_id      BIGINT NOT NULL,
  image_index INT NOT NULL DEFAULT 0,
  image_path  TEXT NOT NULL,
  embedding   VECTOR(512),     -- CLIP ViT-B/32

```

```

metadata    JSONB,
UNIQUE (doc_id, image_index)
);

CREATE INDEX idx_text_emb_hnsw
ON document_text_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

CREATE INDEX idx_image_emb_hnsw
ON document_image_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

```

The tradeoff: two tables means two search queries per multi-modal lookup, and you need an application-layer fusion step. The benefit is that you can update the image embedding model without touching the text pipeline, and you can set different HNSW parameters (*m*, *ef_construction*) appropriate to each vector space's geometry.

Pattern 3: Weaviate Multi-Modal Collections

Weaviate has native multi-modal support via its `multi2vec-clip` vectorizer module, which handles both text and image fields in a single collection and produces a single fused vector. This shifts the fusion problem from your application layer to the database layer.

```

# Python
import weaviate
from weaviate.classes.config import Configure, Property, DataType
import base64
from pathlib import Path

client = weaviate.connect_to_local() # or weaviate.connect_to_weaviate_cloud()

# Create a multi-modal collection
client.collections.create(
    name="ProductCatalog",
    vectorizer_config=Configure.Vectorizer.multi2vec_clip(
        image_fields=["product_image"],
        text_fields=["name", "description"],
        # Weight the contribution of each field to the combined vector
        weights={
            "product_image": [0.6], # 60% image signal
            "name": [0.2],
            "description": [0.2],
        }
    ),
    properties=[
        Property(name="name", data_type=DataType.TEXT),
        Property(name="description", data_type=DataType.TEXT),
        Property(name="product_image", data_type=DataType.BLOB), # base64 image
        Property(name="sku", data_type=DataType.TEXT),
        Property(name="price", data_type=DataType.NUMBER),
        Property(name="category", data_type=DataType.TEXT),
    ]
)

# Ingest a product with both text and image
def encode_image(image_path: str) -> str:
    with open(image_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")

collection = client.collections.get("ProductCatalog")

collection.data.insert({
    "name": "Running Shoe X500",
    "description": "Lightweight mesh upper with cushioned midsole, ideal for road running",
    "product_image": encode_image("./images/shoe_x500.jpg"),

```

```

    "sku": "SHOE-X500-BLK-10",
    "price": 129.99,
    "category": "footwear"
})

client.close()

```

The Weaviate `multi2vec-clip` module uses field weights to combine sub-embeddings into one vector before indexing. This is architecturally different from the two-query-and-fuse pattern — you get a single HNSW index, lower query latency, and simpler application code. The cost is that you cannot independently update image weights without re-embedding the entire collection.

Ingestion Pipelines: Embedding at Scale

Getting data into a multi-modal store efficiently requires a pipeline that handles image preprocessing, batching, and error recovery. Raw images rarely arrive in a form suitable for CLIP: they need resizing, normalization, and format validation before the model sees them.

```

# Python
# Production-grade multi-modal ingestion pipeline
# Using sentence-transformers (CLIP wrapper), Qdrant, and async batching

import asyncio
import logging
from dataclasses import dataclass
from pathlib import Path
from typing import Optional
import numpy as np
from PIL import Image
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient, AsyncQdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    NamedVector, VectorsConfig
)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# CLIP ViT-B/32 via sentence-transformers wrapper
# Note: This model produces BOTH text and image vectors in the same 512-d space
MODEL_NAME = "clip-ViT-B-32"
VECTOR_DIM = 512
BATCH_SIZE = 64
MAX_IMAGE_SIZE = (336, 336) # CLIP ViT-B/32 optimal input resolution

@dataclass
class ProductRecord:
    product_id: str
    name: str
    description: str
    image_path: Optional[str]
    category: str
    price: float

def load_and_preprocess_image(image_path: str) -> Optional[Image.Image]:
    """Load image with validation and normalization."""
    try:
        img = Image.open(image_path).convert("RGB")
        # Preserve aspect ratio while bounding to MAX_IMAGE_SIZE
        img.thumbnail(MAX_IMAGE_SIZE, Image.LANCZOS)
        return img
    except Exception as e:

```

```

        logger.warning(f"Failed to load image {image_path}: {e}")
        return None

def embed_batch(
    model: SentenceTransformer,
    records: list[ProductRecord]
) -> tuple[list[np.ndarray], list[np.ndarray]]:
    """
    Embed a batch of records.
    Returns (text_embeddings, image_embeddings).
    image_embeddings[i] is None if the image failed to load.
    """
    # Text: concatenate name + description for richer signal
    texts = [f"{r.name}. {r.description}" for r in records]
    text_embeddings = model.encode(
        texts,
        batch_size=BATCH_SIZE,
        normalize_embeddings=True,
        show_progress_bar=False
    )

    images = []
    image_mask = [] # Track which records have valid images
    for r in records:
        if r.image_path:
            img = load_and_preprocess_image(r.image_path)
            if img:
                images.append(img)
                image_mask.append(True)
            else:
                image_mask.append(False)
        else:
            image_mask.append(False)

    image_embeddings_raw = []
    if images:
        image_embeddings_raw = model.encode(
            images,
            batch_size=BATCH_SIZE,
            normalize_embeddings=True,
            show_progress_bar=False
        )

    # Reconstruct image_embeddings list aligned with records
    img_iter = iter(image_embeddings_raw)
    image_embeddings = []
    for has_image in image_mask:
        if has_image:
            image_embeddings.append(next(img_iter))
        else:
            image_embeddings.append(None)

    return text_embeddings, image_embeddings

def build_qdrant_points(
    records: list[ProductRecord],
    text_embeddings: list[np.ndarray],
    image_embeddings: list[Optional[np.ndarray]]
) -> list[PointStruct]:
    """
    Build Qdrant points using named vectors.
    Qdrant supports multiple named vector spaces per collection.
    """
    points = []
    for i, record in enumerate(records):
        vectors = {
            "text": text_embeddings[i].tolist()
        }
        if image_embeddings[i] is not None:
            vectors["image"] = image_embeddings[i].tolist()

```



```

points.append(PointStruct(
    id=abs(hash(record.product_id)) % (2**63), # Qdrant needs uint64
    vector=vectors,
    payload={
        "product_id": record.product_id,
        "name": record.name,
        "description": record.description,
        "category": record.category,
        "price": record.price,
        "has_image": image_embeddings[i] is not None
    }
))
return points

async def setup_collection(client: AsyncQdrantClient, collection_name: str):
    """Create collection with named vector spaces for text and image."""
    collections = await client.get_collections()
    existing = [c.name for c in collections.collections]

    if collection_name not in existing:
        await client.create_collection(
            collection_name=collection_name,
            vectors_config={
                "text": VectorParams(size=VECTOR_DIM, distance=Distance.COSINE),
                "image": VectorParams(size=VECTOR_DIM, distance=Distance.COSINE),
            }
        )
        logger.info(f"Created collection: {collection_name}")
    else:
        logger.info(f"Collection {collection_name} already exists, skipping creation")

async def ingest_products(
    records: list[ProductRecord],
    collection_name: str = "products_multimodal",
    qdrant_url: str = "http://localhost:6333"
):
    model = SentenceTransformer(MODEL_NAME)
    client = AsyncQdrantClient(url=qdrant_url)

    await setup_collection(client, collection_name)

    total_ingested = 0
    for batch_start in range(0, len(records), BATCH_SIZE):
        batch = records[batch_start:batch_start + BATCH_SIZE]

        text_embs, image_embs = embed_batch(model, batch)
        points = build_qdrant_points(batch, text_embs, image_embs)

        await client.upsert(
            collection_name=collection_name,
            points=points,
            wait=True
        )
        total_ingested += len(points)
        logger.info(
            f"Ingested batch {batch_start}-{batch_start + len(batch)}, "
            f"total: {total_ingested}"
        )

    await client.close()
    logger.info(f"Ingestion complete. Total records: {total_ingested}")

```

A few production details worth noting in this pipeline:

Qdrant named vectors allow multiple vector spaces per document in a single collection. This is superior to storing separate collections per modality because a single upsert keeps both vectors atomic — you cannot have a text vector without its corresponding image vector falling out of sync.

Image failure handling: Real product images fail to load regularly — corrupted files, unsupported formats, network-mounted paths that go stale. The pipeline tracks `has_image` in the payload so queries can filter to image-capable records when needed.

Normalization at embed time: CLIP vectors should be L2-normalized before storage. `normalize_embeddings=True` in `sentence-transformers` handles this. Pre-normalizing means the database computes cosine similarity as a dot product — numerically cheaper and avoids re-normalization overhead on the query path.

Query Patterns: Fusing Text and Image Signals

With data ingested, the interesting engineering happens at query time. Multi-modal search exposes four distinct query patterns, each appropriate to different use cases.

Pattern 1: Text Query Against Image Index

The simplest multi-modal query: a user types text, you retrieve images whose CLIP image embeddings are closest to the CLIP text embedding of the query. The entire “magic” is that the CLIP encoder maps both to the same space.

```
# Python
# Multi-modal query patterns using Qdrant named vectors

from qdrant_client import QdrantClient
from qdrant_client.models import (
    SearchRequest, NamedVector, Filter,
    FieldCondition, MatchValue, MatchAny
)
from sentence_transformers import SentenceTransformer
from PIL import Image
import base64
import io
from typing import Optional

model = SentenceTransformer("clip-ViT-B-32")
client = QdrantClient(url="http://localhost:6333")
COLLECTION = "products_multimodal"

def text_to_image_search(
    query_text: str,
    category_filter: Optional[str] = None,
    top_k: int = 20
) -> list[dict]:
    """
    Query: text string + retrieve closest images.
    Classic cross-modal retrieval.
    """
    query_vector = model.encode(
        query_text,
        normalize_embeddings=True
    ).tolist()

    search_filter = None
    if category_filter:
        search_filter = Filter(
            must=[
                FieldCondition(key="category", match=MatchValue(value=category_filter)),
                FieldCondition(key="has_image", match=MatchValue(value=True))
            ]
        )
```

```

results = client.search(
    collection_name=COLLECTION,
    query_vector=NamedVector(name="image", vector=query_vector), # text + image space
    query_filter=search_filter,
    limit=top_k,
    with_payload=True,
    score_threshold=0.25 # CLIP cosine scores below 0.25 are typically noise
)

return [
    {
        "product_id": r.payload["product_id"],
        "name": r.payload["name"],
        "score": r.score,
        "has_image": r.payload.get("has_image", False)
    }
    for r in results
]

def image_to_image_search(
    query_image_path: str,
    top_k: int = 20,
    exclude_product_id: Optional[str] = None
) -> list[dict]:
    """
    Query: image + retrieve visually similar items.
    'Shop the look' / visual similarity pattern.
    """
    img = Image.open(query_image_path).convert("RGB")
    query_vector = model.encode(img, normalize_embeddings=True).tolist()

    search_filter = None
    if exclude_product_id:
        # Exclude the query item itself from results
        search_filter = Filter(
            must_not=[
                FieldCondition(key="product_id", match=MatchValue(value=exclude_product_id))
            ]
        )

    results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(name="image", vector=query_vector),
        query_filter=search_filter,
        limit=top_k,
        with_payload=True
    )
    return [{"product_id": r.payload["product_id"], "name": r.payload["name"], "score": r.score}
            for r in results]

def combined_text_image_search(
    query_text: str,
    query_image_path: str,
    text_weight: float = 0.4,
    image_weight: float = 0.6,
    top_k: int = 20
) -> list[dict]:
    """
    Fused query: BOTH a text query AND a reference image.
    Use case: 'Find shoes like [image] but with [text description]'.

    Fusion strategy: weighted average of text and image query vectors.
    This works because both live in the same CLIP space.
    """
    import numpy as np

    text_vector = model.encode(query_text, normalize_embeddings=True)

    img = Image.open(query_image_path).convert("RGB")
    image_vector = model.encode(img, normalize_embeddings=True)

```

```

# Weighted combination - still in the shared CLIP space
fused_vector = text_weight * text_vector + image_weight * image_vector

# Re-normalize after combination
fused_norm = np.linalg.norm(fused_vector)
if fused_norm > 0:
    fused_vector = fused_vector / fused_norm

# Query the image index with the fused vector
# (could also query the text index - depends on what you indexed)
results = client.search(
    collection_name=COLLECTION,
    query_vector=NamedVector(name="image", vector=fused_vector.tolist()),
    limit=top_k,
    with_payload=True,
    score_threshold=0.20
)

return [
    {
        "product_id": r.payload["product_id"],
        "name": r.payload["name"],
        "score": r.score,
        "query_mode": "fused-text-image"
    }
    for r in results
]

def late_fusion_search(
    query_text: str,
    query_image_path: str,
    text_weight: float = 0.5,
    image_weight: float = 0.5,
    candidate_pool: int = 100,
    top_k: int = 20
) -> list[dict]:
    """
    Late fusion: run two separate searches, then combine scores.
    More expensive but allows independent index tuning per modality.
    Use when text and image are in DIFFERENT embedding spaces.
    """
    import numpy as np

    # Search 1: text vector + text index
    text_vector = model.encode(query_text, normalize_embeddings=True).tolist()

    text_results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(name="text", vector=text_vector),
        limit=candidate_pool,
        with_payload=True,
        score_threshold=0.15
    )

    # Search 2: image vector + image index
    img = Image.open(query_image_path).convert("RGB")
    image_vector = model.encode(img, normalize_embeddings=True).tolist()

    image_results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(name="image", vector=image_vector),
        limit=candidate_pool,
        with_payload=True,
        score_threshold=0.15
    )

    # Merge scores by product_id
    score_map: dict[str, dict] = {}
    for r in text_results:

```

```
pid = r.payload["product_id"]
score_map[pid] = {
    "product_id": pid,
    "name": r.payload["name"],
    "text_score": r.score,
    "image_score": 0.0,
    "payload": r.payload
}

for r in image_results:
    pid = r.payload["product_id"]
    if pid in score_map:
        score_map[pid]["image_score"] = r.score
    else:
        score_map[pid] = {
            "product_id": pid,
            "name": r.payload["name"],
            "text_score": 0.0,
            "image_score": r.score,
            "payload": r.payload
        }

# Weighted combined score
fused = []
for pid, data in score_map.items():
    combined = text_weight * data["text_score"] + image_weight * data["image_score"]
    fused.append({
        "product_id": pid,
        "name": data["name"],
        "combined_score": combined,
        "text_score": data["text_score"],
        "image_score": data["image_score"]
    })

fused.sort(key=lambda x: x["combined_score"], reverse=True)
return fused[:top_k]
```

The late fusion pattern deserves a closer look, because it is the right choice whenever text and image embeddings come from *different* models with *different* dimensionalities. When you have a 1536-dimension OpenAI text embedding alongside a 512-dimension CLIP image embedding, you cannot average them — the vector spaces are unrelated. Late fusion avoids that requirement: each modality gets its own search against its own index, and scores are combined numerically after the fact. The cost is two round-trips to the database and a merge step in application memory.

Early fusion (the weighted average in `combined_text_image_search`) only works when both modalities are encoded by the same model family into the same shared space. CLIP’s design makes this possible; most other combinations do not.

Multi-Modal Search in pgvector: The SQL Perspective

For teams operating in PostgreSQL with pgvector, the fusion logic moves into SQL rather than application code. This has advantages for teams that want to keep retrieval logic auditable and version-controlled as migrations.

```
-- PostgreSQL / pgvector
-- Late fusion multi-modal search using two CTEs

-- Assumes:
--   product_text_embeddings (product_id, embedding VECTOR(1536))
--   product_image_embeddings (product_id, embedding VECTOR(512))
--   Each has its own HNSW index.
```

```

-- Parameters passed as session variables or substituted by the application:
-- :text_query_vector - 1536-dim OpenAI embedding of the query string
-- :image_query_vector - 512-dim CLIP embedding of the reference image
-- :text_weight - e.g. 0.5
-- :image_weight - e.g. 0.5
-- :candidate_k - e.g. 100
-- :top_k - e.g. 20

WITH text_candidates AS (
    SELECT
        product_id,
        1 - (embedding <=> :text_query_vector::VECTOR(1536)) AS text_score
    FROM product_text_embeddings
    ORDER BY embedding <=> :text_query_vector::VECTOR(1536)
    LIMIT :candidate_k
),
image_candidates AS (
    SELECT
        product_id,
        1 - (embedding <=> :image_query_vector::VECTOR(512)) AS image_score
    FROM product_image_embeddings
    ORDER BY embedding <=> :image_query_vector::VECTOR(512)
    LIMIT :candidate_k
),
fused AS (
    SELECT
        COALESCE(t.product_id, i.product_id) AS product_id,
        COALESCE(t.text_score, 0.0) * :text_weight
        + COALESCE(i.image_score, 0.0) * :image_weight AS combined_score,
        COALESCE(t.text_score, 0.0) AS text_score,
        COALESCE(i.image_score, 0.0) AS image_score
    FROM text_candidates t
    FULL OUTER JOIN image_candidates i USING (product_id)
)
SELECT
    f.product_id,
    p.name,
    p.category,
    p.price,
    f.combined_score,
    f.text_score,
    f.image_score
FROM fused f
JOIN products p ON p.id = f.product_id
ORDER BY f.combined_score DESC
LIMIT :top_k;

-- SQL Server equivalent
-- SQL Server does not have a native vector type in older versions;
-- assuming the vector extension or a FLOAT column workaround is not available,
-- the pattern below uses SQL Server 2025+ vector type (VECTOR) where supported,
-- or a stored-proc approach calling an external scoring service.
-- For teams on SQL Server 2022 and below, the recommended path is to perform
-- the HNSW search outside SQL Server and use SQL Server only for the metadata JOIN.

-- SQL Server 2025+ with native VECTOR type:
WITH text_candidates AS (
    SELECT TOP (:candidate_k)
        product_id,
        1.0 - VECTOR_DISTANCE('cosine', embedding, CAST(:text_query_vector AS VECTOR(1536)))
        AS text_score
    FROM product_text_embeddings
    ORDER BY VECTOR_DISTANCE('cosine', embedding, CAST(:text_query_vector AS VECTOR(1536)))
),
image_candidates AS (
    SELECT TOP (:candidate_k)
        product_id,
        1.0 - VECTOR_DISTANCE('cosine', embedding, CAST(:image_query_vector AS VECTOR(512)))
        AS image_score

```

```

FROM product_image_embeddings
ORDER BY VECTOR_DISTANCE('cosine', embedding, CAST(:image_query_vector AS VECTOR(512)))
),
fused AS (
  SELECT
    COALESCE(t.product_id, i.product_id) AS product_id,
    COALESCE(t.text_score, 0.0) * :text_weight
    + COALESCE(i.image_score, 0.0) * :image_weight AS combined_score,
    COALESCE(t.text_score, 0.0) AS text_score,
    COALESCE(i.image_score, 0.0) AS image_score
  FROM text_candidates t
  FULL OUTER JOIN image_candidates i ON t.product_id = i.product_id
)
SELECT TOP (:top_k)
  f.product_id,
  p.name,
  p.category,
  p.price,
  f.combined_score,
  f.text_score,
  f.image_score
FROM fused f
JOIN products p ON p.id = f.product_id
ORDER BY f.combined_score DESC;

```

A subtlety in both queries: the `FULL OUTER JOIN` between `text_candidates` and `image_candidates` is deliberate. Products that appear in only one result set still participate in the final ranking, with a zero score contribution from the missing modality. This prevents the fusion from arbitrarily excluding products that have strong text relevance but weak image relevance or vice versa. Whether that behavior is correct depends on your use case — if you require *both* modalities to have non-trivial signal, replace the `FULL OUTER JOIN` with an `INNER JOIN`.

Score Normalization Before Fusion

A common mistake in late fusion implementations is combining raw cosine similarity scores across modalities without considering their distributions. Different models produce scores with different statistical properties: a cosine similarity of 0.7 in a text embedding space does not mean the same “closeness” as 0.7 in a CLIP image space. CLIP image-to-image similarities tend to cluster in a narrower range than text-to-text similarities because image features are more visually concrete.

The solution is score normalization within each modality’s result set before combining. The two standard approaches:

Min-Max normalization: Scale each modality’s scores to $[0, 1]$ relative to the candidate pool.

```

# Python
def normalize_scores_minmax(scores: list[float]) -> list[float]:
    if not scores:
        return scores
    min_s, max_s = min(scores), max(scores)
    if max_s == min_s:
        return [1.0] * len(scores)
    return [(s - min_s) / (max_s - min_s) for s in scores]

def late_fusion_with_normalization(
    text_results: list, # [(product_id, name, score), ...]
    image_results: list,
    text_weight: float = 0.5,
    image_weight: float = 0.5,

```

```

top_k: int = 20
) -> list[dict]:
    import numpy as np

    text_scores_raw = [r.score for r in text_results]
    image_scores_raw = [r.score for r in image_results]

    text_scores_norm = normalize_scores_minmax(text_scores_raw)
    image_scores_norm = normalize_scores_minmax(image_scores_raw)

    score_map: dict[str, dict] = {}
    for r, norm_score in zip(text_results, text_scores_norm):
        pid = r.payload["product_id"]
        score_map[pid] = {"name": r.payload["name"], "text_score": norm_score, "image_score": 0.0}

    for r, norm_score in zip(image_results, image_scores_norm):
        pid = r.payload["product_id"]
        if pid in score_map:
            score_map[pid]["image_score"] = norm_score
        else:
            score_map[pid] = {"name": r.payload["name"], "text_score": 0.0, "image_score": norm_score}

    fused = [
        {
            "product_id": pid,
            "name": data["name"],
            "combined_score": text_weight * data["text_score"] + image_weight * data["image_score"],
            "text_score": data["text_score"],
            "image_score": data["image_score"]
        }
        for pid, data in score_map.items()
    ]
    fused.sort(key=lambda x: x["combined_score"], reverse=True)
    return fused[:top_k]

```

Reciprocal Rank Fusion (RRF): An alternative that is rank-based rather than score-based, and therefore inherently distribution-agnostic.

```

# Python
def reciprocal_rank_fusion(
    text_results: list,
    image_results: list,
    k: int = 60, # RRF smoothing constant; 60 is the standard default
    top_n: int = 20
) -> list[dict]:
    """
    Combine two ranked lists using RRF.
    RRF score = sum over each list of: 1 / (rank_in_list + k)
    """
    rrf_scores: dict[str, float] = {}
    payload_store: dict[str, dict] = {}

    for rank, r in enumerate(text_results, start=1):
        pid = r.payload["product_id"]
        rrf_scores[pid] = rrf_scores.get(pid, 0.0) + 1.0 / (rank + k)
        payload_store[pid] = r.payload

    for rank, r in enumerate(image_results, start=1):
        pid = r.payload["product_id"]
        rrf_scores[pid] = rrf_scores.get(pid, 0.0) + 1.0 / (rank + k)
        if pid not in payload_store:
            payload_store[pid] = r.payload

    ranked = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)[:top_n]

    return [
        {
            "product_id": pid,
            "name": payload_store[pid]["name"],

```



```
        "rrf_score": score
    }
    for pid, score in ranked
]
```

RRF has a practical advantage in production: it is robust to outlier scores and does not require you to tune normalization ranges as your corpus grows. The tradeoff is that you lose the interpretable per-modality score breakdown — you cannot easily tell whether a result ranked highly because of text signal, image signal, or both.

Handling Queries Without an Image

In real deployments, most users will provide text queries most of the time. The multi-modal system must degrade gracefully when only one modality is present, rather than treating a missing image as an error.

```
# Python
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class QueryModality(Enum):
    TEXT_ONLY = "text_only"
    IMAGE_ONLY = "image_only"
    BOTH = "both"

@dataclass
class MultiModalQuery:
    text: Optional[str]
    image_path: Optional[str]
    category_filter: Optional[str] = None
    text_weight: float = 0.5
    image_weight: float = 0.5
    top_k: int = 20

    @property
    def modality(self) -> QueryModality:
        if self.text and self.image_path:
            return QueryModality.BOTH
        elif self.text:
            return QueryModality.TEXT_ONLY
        elif self.image_path:
            return QueryModality.IMAGE_ONLY
        raise ValueError("Query must have at least one of text or image_path")

def route_multimodal_query(
    query: MultiModalQuery,
    model: SentenceTransformer,
    client: QdrantClient
) -> list[dict]:
    """
    Route query to the appropriate search strategy based on what inputs are available.
    """
    modality = query.modality

    if modality == QueryModality.TEXT_ONLY:
        # Standard vector search on text index; optionally also hit image index
        # for cross-modal retrieval (user typed, wants images back)
        return text_to_image_search(
            query_text=query.text,
            category_filter=query.category_filter,
            top_k=query.top_k
        )
```

```
elif modality == QueryModality.IMAGE_ONLY:
    return image_to_image_search(
        query_image_path=query.image_path,
        top_k=query.top_k
    )

else: # BOTH
    # Decide between early fusion and late fusion based on modality compatibility
    # (For CLIP, early fusion is fine; generalize with late fusion if needed)
    text_results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(
            name="text",
            vector=model.encode(query.text, normalize_embeddings=True).tolist()
        ),
        limit=query.top_k * 3,
        with_payload=True
    )
    from PIL import Image as PILImage
    img = PILImage.open(query.image_path).convert("RGB")
    image_results = client.search(
        collection_name=COLLECTION,
        query_vector=NamedVector(
            name="image",
            vector=model.encode(img, normalize_embeddings=True).tolist()
        ),
        limit=query.top_k * 3,
        with_payload=True
    )

    return reciprocal_rank_fusion(
        text_results=text_results,
        image_results=image_results,
        top_n=query.top_k
    )
```

The routing logic here is deliberately simple — three branches, no machine learning. In a production system you might introduce a fourth branch: a **zero-shot re-ranker** that scores the top-K candidates from the fused list against the original text query using a cross-encoder model. That adds latency (typically 50–150 ms for a list of 100 candidates on CPU) but improves precision significantly, especially for ambiguous queries where the bi-encoder CLIP search is unreliable.

Pinecone: Multi-Modal with Namespaces and Metadata Filtering

Pinecone’s architecture does not have native named vector support in the same way Qdrant does. The production pattern for multi-modal search in Pinecone uses **namespaces** to separate modalities within a single index, combined with metadata filtering.

```
# Python
# Multi-modal pattern in Pinecone using namespaces
# Pinecone indexes are fixed-dimension; CLIP ViT-B/32 = 512

from pinecone import Pinecone, ServerlessSpec
from sentence_transformers import SentenceTransformer
from PIL import Image
import numpy as np
from typing import Optional

pc = Pinecone(api_key="YOUR_PINECONE_API_KEY")
model = SentenceTransformer("clip-ViT-B-32")
```

```

INDEX_NAME = "products-multimodal"
TEXT_NS    = "text"    # namespace for text embeddings
IMAGE_NS   = "image"   # namespace for image embeddings
VECTOR_DIM = 512

def create_pinecone_index():
    if INDEX_NAME not in [i.name for i in pc.list_indexes()]:
        pc.create_index(
            name=INDEX_NAME,
            dimension=VECTOR_DIM,
            metric="cosine",
            spec=ServerlessSpec(cloud="aws", region="us-east-1")
        )

def upsert_product(
    product_id: str,
    name: str,
    description: str,
    category: str,
    price: float,
    image_path: Optional[str] = None
):
    index = pc.Index(INDEX_NAME)

    # Upsert text embedding
    text_vector = model.encode(
        f"{name}. {description}", normalize_embeddings=True
    ).tolist()

    index.upsert(
        vectors=[{
            "id": f"{product_id}",
            "values": text_vector,
            "metadata": {
                "product_id": product_id,
                "name": name,
                "category": category,
                "price": price,
                "modality": "text"
            }
        }],
        namespace=TEXT_NS
    )

    # Upsert image embedding if image is available
    if image_path:
        img = Image.open(image_path).convert("RGB")
        image_vector = model.encode(img, normalize_embeddings=True).tolist()

        index.upsert(
            vectors=[{
                "id": f"{product_id}",
                "values": image_vector,
                "metadata": {
                    "product_id": product_id,
                    "name": name,
                    "category": category,
                    "price": price,
                    "modality": "image"
                }
            }],
            namespace=IMAGE_NS
        )

def multimodal_query_pinecone(
    query_text: Optional[str],
    query_image_path: Optional[str],
    category_filter: Optional[str] = None,
    top_k: int = 20,
    text_weight: float = 0.5,

```

```

    image_weight: float = 0.5
) -> list[dict]:
    index = pc.Index(INDEX_NAME)

    metadata_filter = {}
    if category_filter:
        metadata_filter = {"category": {"$eq": category_filter}}

    results_text = []
    results_image = []

    if query_text:
        text_vector = model.encode(query_text, normalize_embeddings=True).tolist()
        resp = index.query(
            namespace=IMAGE_NS,          # text query + image namespace (cross-modal)
            vector=text_vector,
            top_k=top_k * 3,
            filter=metadata_filter if metadata_filter else None,
            include_metadata=True
        )
        results_text = resp.matches

    if query_image_path:
        img = Image.open(query_image_path).convert("RGB")
        image_vector = model.encode(img, normalize_embeddings=True).tolist()
        resp = index.query(
            namespace=IMAGE_NS,
            vector=image_vector,
            top_k=top_k * 3,
            filter=metadata_filter if metadata_filter else None,
            include_metadata=True
        )
        results_image = resp.matches

    if results_text and results_image:
        return reciprocal_rank_fusion_pinecone(results_text, results_image, top_n=top_k)
    elif results_text:
        return [{"product_id": m.metadata["product_id"], "name": m.metadata["name"], "score": m.score}
                for m in results_text[:top_k]]
    elif results_image:
        return [{"product_id": m.metadata["product_id"], "name": m.metadata["name"], "score": m.score}
                for m in results_image[:top_k]]
    return []

def reciprocal_rank_fusion_pinecone(
    text_matches,
    image_matches,
    k: int = 60,
    top_n: int = 20
) -> list[dict]:
    rrf: dict[str, float] = {}
    meta: dict[str, dict] = {}

    for rank, m in enumerate(text_matches, start=1):
        pid = m.metadata["product_id"]
        rrf[pid] = rrf.get(pid, 0.0) + 1.0 / (rank + k)
        meta[pid] = m.metadata

    for rank, m in enumerate(image_matches, start=1):
        pid = m.metadata["product_id"]
        rrf[pid] = rrf.get(pid, 0.0) + 1.0 / (rank + k)
        if pid not in meta:
            meta[pid] = m.metadata

    ranked = sorted(rrf.items(), key=lambda x: x[1], reverse=True)[:top_n]
    return [{"product_id": pid, "name": meta[pid]["name"], "rrf_score": score}
            for pid, score in ranked]

```

One limitation of the Pinecone namespace approach: namespaces share the same dimension. Since

CLIP text and image encoders produce the same dimension by design, this is not a constraint for CLIP-based systems. If your architecture mixes CLIP images (512-d) with a different text embedding model (1536-d), you would need two separate Pinecone indexes rather than two namespaces.

Production Considerations: Latency, Cost, and Quality Tradeoffs

Multi-modal search introduces compound latency: you are now encoding queries with a vision model in addition to a text model, and potentially issuing two database queries instead of one. Understanding where time is spent guides optimization.

Embedding latency is typically the largest contributor. CLIP ViT-B/32 runs at roughly 5–15 ms per image on a modern CPU with batch size 1, and under 2 ms per text query. On GPU, image embedding drops below 2 ms. If your deployment is CPU-only, consider:

- Caching embeddings for repeated reference images (user uploads the same product photo repeatedly)
- Quantizing the CLIP model with INT8 to reduce inference time at marginal quality cost
- Using a lighter CLIP variant (ViT-S/32) for latency-critical paths and a heavier model for offline indexing

Index scan latency scales with collection size and HNSW parameters. A practical rule: keep `ef` (the HNSW search parameter at query time) at 2–4× your `top_k`. For `top_k=20`, `ef=64` is a reasonable starting point. Increasing `ef` improves recall at the cost of latency — profile this against your SLA.

Quality measurement for multi-modal search requires a labeled evaluation set with both modalities. The standard metric is Recall@K: for a set of known-relevant items, what fraction appear in the top-K results? Building this dataset means collecting (query text, reference image, known-relevant product IDs) triples — time-consuming but necessary before tuning fusion weights.

Cost management in cloud-hosted vector databases (Pinecone, Weaviate Cloud) is driven by stored vector count and query volume. Multi-modal doubles your vector count per document when storing separate text and image embeddings. If cost is a constraint, consider:

- Storing only image embeddings and using the CLIP text encoder at query time for cross-modal retrieval (text query → image index). This halves storage at the cost of losing text-to-text retrieval as a fallback.
 - Tiering: use a smaller vector dimension (256-d via PCA projection of CLIP) for the first-pass ANN search, then re-score top candidates with the full 512-d embedding.
-

A Complete End-to-End Example: Fashion Visual Search

To bring the patterns together, consider a fashion retail application that supports three query types: “type a description,” “upload a photo,” and “type a description while holding a photo.” The following shows the complete server-side handler.

```
# Python
# Fashion visual search: end-to-end query handler
```

```

# FastAPI endpoint integrating all multi-modal patterns

from fastapi import FastAPI, File, Form, UploadFile, HTTPException
from fastapi.responses import JSONResponse
import tempfile, os, shutil
from typing import Optional
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from qdrant_client.models import NamedVector, Filter, FieldCondition, MatchValue

app = FastAPI(title="Fashion Visual Search API")

# Initialize at startup - avoid re-loading per request
_model: Optional[SentenceTransformer] = None
_qdrant: Optional[QdrantClient] = None

@app.on_event("startup")
async def startup():
    global _model, _qdrant
    _model = SentenceTransformer("clip-ViT-B-32")
    _qdrant = QdrantClient(url=os.getenv("QDRANT_URL", "http://localhost:6333"))

@app.post("/search")
async def search(
    query_text: Optional[str] = Form(None),
    category: Optional[str] = Form(None),
    top_k: int = Form(20),
    text_weight: float = Form(0.4),
    image_weight: float = Form(0.6),
    image: Optional[UploadFile] = File(None)
):
    if not query_text and not image:
        raise HTTPException(status_code=400, detail="Provide at least query_text or an image.")

    # Persist uploaded image to temp file for PIL
    tmp_image_path = None
    if image:
        suffix = os.path.splitext(image.filename)[-1] or ".jpg"
        with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
            shutil.copyfileobj(image.file, tmp)
            tmp_image_path = tmp.name

    try:
        q = MultiModalQuery(
            text=query_text,
            image_path=tmp_image_path,
            category_filter=category,
            text_weight=text_weight,
            image_weight=image_weight,
            top_k=top_k
        )
        results = route_multimodal_query(q, _model, _qdrant)
        return JSONResponse(content={"results": results, "modality": q.modality.value})

    finally:
        if tmp_image_path and os.path.exists(tmp_image_path):
            os.unlink(tmp_image_path) # always clean up temp files

@app.get("/health")
async def health():
    return {"status": "ok", "model": "clip-ViT-B-32"}

```

The FastAPI handler deliberately isolates temp-file cleanup in a `finally` block — a failure during embedding or Qdrant search must not leave orphaned image files filling the server’s `/tmp` directory. In a container environment, `/tmp` is often an in-memory tmpfs with limited capacity.

Key Takeaways

- **Shared embedding spaces** are the foundation of multi-modal search. CLIP-family models encode text and images into the same vector space, enabling cross-modal queries where a text string can retrieve images and vice versa without any additional translation layer.
 - **Storage architecture choices** — single table with modality column, separate tables per modality, or database-native multi-modal collections (Weaviate) — each carry different trade-offs for indexing flexibility, update independence, and operational complexity. Choose based on whether text and image pipelines change at the same rate.
 - **Early fusion** (averaging query vectors) works only when text and image embeddings share the same model and dimensionality. **Late fusion** (two searches, score combination) is the safe default when embedding models differ, and RRF is the most robust score combination method because it is rank-based and distribution-agnostic.
 - **Score normalization before combining modality scores** is not optional in production. Raw cosine similarities from different models have different statistical distributions; combining them without normalization biases fusion toward whichever modality happens to produce larger raw values.
 - **Graceful degradation** — routing single-modality queries through the appropriate index without treating a missing image or text as an error — is required for any system that faces real user traffic. Wrap modality detection in an explicit routing layer so that adding new query types later does not require restructuring the retrieval core.
-

Chapter 32: Graph and Vector: Combining Relationship and Semantic Search

Semantic search answers the question *what does this mean?* Graph traversal answers the question *how are these things connected?* Neither question alone is sufficient for many real-world AI applications. A knowledge graph without semantic understanding forces exact string matches and brittle schema design. A vector index without relationship awareness loses the structural context that determines whether two semantically similar documents are actually related, authored by the same person, part of the same regulatory framework, or connected through a chain of citations. This chapter explores how to combine graph and vector capabilities into a unified retrieval layer — covering the architectural patterns, the specific tools that support native graph-vector hybrid search, and the practical tradeoffs that determine when you need both versus when one is sufficient.

Why Graph Structure and Vector Similarity Serve Different Retrieval Needs

Start with a concrete example: a pharmaceutical knowledge base containing drug compounds, clinical trials, regulatory approvals, adverse event reports, and the researchers who authored each document. Pure vector search can find documents semantically similar to “cardiovascular side effects in elderly patients.” But it cannot tell you that two apparently similar adverse event reports refer to different formulations of the same drug, or that the most semantically relevant paper was authored by a researcher whose institution’s data has been retracted. That structural context lives in the graph.

Graph databases model entities and relationships as first-class citizens. In property graph terms, nodes hold attributes and edges hold typed, directional relationships with their own properties. Neo4j’s Cypher, Amazon Neptune’s Gremlin and SPARQL, and Weaviate’s cross-references all express this structure. The query planner traverses the graph using index-free adjacency — following pointers directly from one node to its neighbors rather than performing joins across tables. This gives graph databases their characteristic performance on multi-hop traversal queries: $O(\text{depth} \times \text{branching_factor})$ rather than $O(n \log n)$ for a join-based equivalent.

Vector databases answer a different kind of query. Given a query embedding, find the k nearest vectors in high-dimensional space. The performance envelope is defined by approximate nearest

neighbor (ANN) algorithms — HNSW, IVF, ScaNN — that trade recall for speed. A well-tuned HNSW index on a 1536-dimensional embedding space with 10 million vectors returns 10 nearest neighbors in under 10ms at 99% recall. No graph traversal is involved; the structure being searched is a proximity graph of embeddings, not an entity-relationship graph of domain objects.

The architectural question is where these two search modes intersect. Three patterns emerge in production:

Pre-filter by graph, rank by vector. Traverse the graph to identify a candidate set of entity IDs, then perform vector similarity search over that restricted set. This uses graph traversal to answer structural questions (“all drugs approved by the FDA in the last three years”) and vector search to rank within that set.

Post-filter by graph, retrieve by vector. Retrieve top-k semantically similar documents, then apply graph traversal to expand, filter, or re-rank based on structural relationships. A recommendation system might retrieve 50 semantically similar items, then use the user’s purchase graph to remove already-purchased items and boost items purchased by structurally similar users.

Graph-guided vector traversal. Use graph relationships to guide which regions of the vector space to explore. This is the approach taken by graph-aware RAG systems that follow citation or entity linkage graphs to gather context before embedding retrieval.

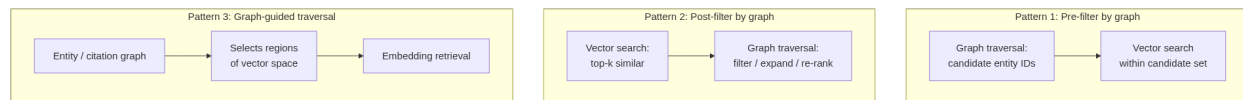


Figure 22: The three ways graph structure and vector similarity combine in production retrieval systems.

Native Graph-Vector Databases: Weaviate, Neo4j Vector, and Amazon Neptune Analytics

Several databases now offer native support for both paradigms within a single query engine, eliminating the need for application-level orchestration between separate systems.

Weaviate was designed from the start around the idea that objects have both vector representations and references to other objects. Its schema system defines cross-references between classes, and its GraphQL query layer lets you traverse those references while filtering by vector similarity in the same request.

```

# Python
import weaviate
import weaviate.classes as wvc

client = weaviate.connect_to_local()

# Schema: Drug nodes reference ClinicalTrial nodes
# This query retrieves trials semantically similar to a query
# that are linked to a specific drug compound
query_text = "cardiovascular adverse events in elderly patients"

response = (
    client.collections.get("ClinicalTrial")

```

```

        .query.near_text(
            query=query_text,
            limit=10,
            return_metadata=wvc.query.MetadataQuery(distance=True),
            filters=wvc.query.Filter.by_ref("hasDrug").by_property("name").equal("Atorvastatin"),
        )
    )

for obj in response.objects:
    print(f"Trial ID: {obj.properties['trial_id']}")
    print(f"Title: {obj.properties['title']}")
    print(f"Distance: {obj.metadata.distance:.4f}")
    print()

client.close()

```

The `Filter.by_ref()` call translates to a graph traversal: follow the `hasDrug` reference edge and filter by the target node's property. Weaviate executes this as a pre-filter before the vector search, meaning the ANN index is queried only over the subset of trials linked to Atorvastatin. At scale this matters: if 10% of your 10M trial documents are linked to the target drug, you are running ANN over 1M vectors rather than 10M, which is roughly a 10x improvement in query latency with no change in recall.

Neo4j added native vector indexing in version 5.11, allowing a `CALL db.index.vector.queryNodes()` procedure alongside standard Cypher graph traversal. This enables a query that first does vector retrieval and then traverses the graph from the results.

```

# Python - Neo4j combined graph + vector query
from neo4j import GraphDatabase
from openai import OpenAI

openai_client = OpenAI()
driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))

def get_embedding(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

def graph_vector_search(query: str, hop_depth: int = 2, top_k: int = 5):
    embedding = get_embedding(query)

    cypher = """
    // Step 1: Vector similarity search to find seed nodes
    CALL db.index.vector.queryNodes('document_embeddings', $top_k, $embedding)
    YIELD node AS doc, score AS similarity_score

    // Step 2: Traverse the graph from each seed node
    // Follow AUTHORED_BY and CITES relationships up to hop_depth
    CALL apoc.path.subgraphNodes(doc, {
        relationshipFilter: 'AUTHORED_BY>|CITES>',
        maxLevel: $hop_depth
    })
    YIELD node AS related

    // Step 3: Collect and return with structural context
    WITH doc, similarity_score,
        collect(DISTINCT related) AS graph_neighbors
    RETURN
        doc.title AS title,
        doc.document_id AS doc_id,
        similarity_score,
        [n IN graph_neighbors WHERE n:Author | n.name] AS authors,
        [n IN graph_neighbors WHERE n:Document | n.title] AS cited_docs,
    """

```

```
        size(graph_neighbors) AS graph_connectivity
ORDER BY similarity_score DESC
"""

with driver.session() as session:
    result = session.run(
        cypher,
        embedding=embedding,
        top_k=top_k,
        hop_depth=hop_depth
    )
    return [record.data() for record in result]

results = graph_vector_search(
    "machine learning approaches for drug interaction prediction",
    hop_depth=2,
    top_k=5
)

for r in results:
    print(f"Title: {r['title']}")
    print(f"Similarity: {r['similarity_score']:.4f}")
    print(f"Authors: {'', '.join(r['authors'])}")
    print(f"Cites: {len(r['cited_docs'])} documents")
    print(f"Graph Connectivity: {r['graph_connectivity']} related nodes")
    print()

driver.close()
```

The `graph_connectivity` metric here is not just informational. Documents with high graph connectivity in a citation network are generally more influential and more likely to represent canonical sources rather than derivative work. This is a structural signal that pure vector similarity cannot capture.

Amazon Neptune Analytics (not Neptune Database) introduced vector similarity search in 2024, enabling openCypher queries that combine graph traversal with approximate nearest neighbor search over stored embeddings. The design pattern follows the same logic: use Neptune’s property graph for structural filtering, Neptune Analytics’ vector index for semantic ranking.

pgvector Meets Recursive CTEs: Graph-Vector Hybrid in PostgreSQL

If your stack is already on PostgreSQL, you may not need a dedicated graph database for moderate-complexity relationship modeling. PostgreSQL’s recursive CTEs provide graph traversal over adjacency tables, and `pgvector` handles the vector similarity layer. The combination is not as expressive as Cypher or Gremlin for deep traversal, but for depth-limited graph queries it is entirely production-viable and keeps your stack consolidated.

```
-- PostgreSQL / pgvector
-- Schema: documents with embeddings, plus a relationship graph

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
    doc_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title       TEXT NOT NULL,
    content     TEXT,
    embedding    vector(1536),
    created_at  TIMESTAMPTZ DEFAULT now()
```

```

);

CREATE TABLE doc_relationships (
    source_id    UUID REFERENCES documents(doc_id),
    target_id    UUID REFERENCES documents(doc_id),
    rel_type     TEXT NOT NULL, -- 'CITES', 'AUTHORED_BY_SAME', 'PART_OF_SERIES'
    weight       FLOAT DEFAULT 1.0,
    PRIMARY KEY (source_id, target_id, rel_type)
);

CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

CREATE INDEX ON doc_relationships (source_id);
CREATE INDEX ON doc_relationships (target_id);

-- Graph-vector hybrid query:
-- 1. Find top-10 semantically similar documents (vector layer)
-- 2. Traverse 2 hops of the citation graph from those seeds (graph layer)
-- 3. Re-rank the expanded set by a combined score

WITH query_embedding AS (
    -- In production, this comes from your embedding service
    -- Here represented as a placeholder cast
    SELECT '[0.1, 0.2, ...]':vector(1536) AS emb
),

-- Step 1: Vector similarity search - seed nodes
vector_seeds AS (
    SELECT
        d.doc_id,
        d.title,
        d.embedding,
        1 - (d.embedding <=> q.emb) AS cosine_similarity,
        1 AS hop_distance
    FROM documents d, query_embedding q
    ORDER BY d.embedding <=> q.emb
    LIMIT 10
),

-- Step 2: Graph traversal from seed nodes (up to 2 hops)
graph_expansion AS (
    -- Anchor: the seed nodes
    SELECT doc_id, doc_id AS origin_id, 0 AS hops
    FROM vector_seeds

    UNION ALL

    -- Recursive: follow citation relationships
    SELECT
        dr.target_id AS doc_id,
        ge.origin_id,
        ge.hops + 1
    FROM graph_expansion ge
    JOIN doc_relationships dr
        ON dr.source_id = ge.doc_id
        AND dr.rel_type = 'CITES'
    WHERE ge.hops < 2
),

-- Deduplicate expanded graph nodes
expanded_candidates AS (
    SELECT DISTINCT doc_id, MIN(hops) AS min_hops
    FROM graph_expansion
    GROUP BY doc_id
),

-- Step 3: Compute combined score for the expanded candidate set
final_ranking AS (
    SELECT

```

```
d.doc_id,
d.title,
1 - (d.embedding <=> q.emb) AS cosine_similarity,
ec.min_hops,
-- Graph proximity penalty: penalize nodes found only through traversal
-- Seeds (hop=0) get full score; each hop reduces score by 15%
(1 - (d.embedding <=> q.emb)) * POWER(0.85, ec.min_hops) AS combined_score,
CASE WHEN vs.doc_id IS NOT NULL THEN true ELSE false END AS is_seed_node
FROM expanded_candidates ec
JOIN documents d ON d.doc_id = ec.doc_id
JOIN query_embedding q ON true
LEFT JOIN vector_seeds vs ON vs.doc_id = ec.doc_id
)

SELECT
doc_id,
title,
ROUND(cosine_similarity::numeric, 4) AS semantic_score,
min_hops AS graph_hops,
ROUND(combined_score::numeric, 4) AS combined_score,
is_seed_node
FROM final_ranking
ORDER BY combined_score DESC
LIMIT 20;
```

The hop penalty (0.85 per hop) is a tunable parameter. In citation networks where relevance decays sharply with distance, values between 0.75 and 0.85 work well. In social graphs where second-degree connections are highly relevant, you might use 0.95 or no penalty at all. Benchmark this against your human-labeled relevance set before deploying.

Performance characteristics to expect: with 1M documents and an HNSW index, the vector seed retrieval runs in ~5ms. The recursive CTE traversal over 10 seeds at depth 2 with average branching factor 5 will generate ~250 candidate nodes, and the secondary embedding lookups add ~1ms. Total query time is typically 10–30ms for this pattern at this scale — well within interactive response time budgets.

Graph-Aware RAG: Using Relationship Context in LLM Pipelines

Retrieval-Augmented Generation pipelines typically operate in a flat retrieval model: embed the query, retrieve top-k chunks, stuff them into the context window. This loses structural information. Graph-aware RAG incorporates relationship traversal into the retrieval step so the context passed to the LLM includes not just semantically similar content but contextually connected content.

Microsoft Research published GraphRAG in 2024, which builds a community detection graph over extracted entities and uses that graph to enable global queries (summarizing broad themes across a corpus) that flat RAG cannot handle. The architecture is more complex than simple hybrid search but the principle is the same: use graph structure to decide what to retrieve.

A more immediately deployable pattern is entity-linked RAG: extract named entities from the query, look up those entities in a knowledge graph to gather structured context, then combine that structured context with vector-retrieved chunks.

```
# Python
# Entity-linked Graph-aware RAG with Neo4j + pgvector + OpenAI

from neo4j import GraphDatabase
from openai import OpenAI
```

```

import psycopg2
import json
from typing import Optional

openai_client = OpenAI()
neo4j_driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "password"))
pg_conn = psycopg2.connect("postgresql://user:pass@localhost/knowledge_db")

def extract_entities_from_query(query: str) -> list[str]:
    """Use LLM to extract named entities from the query."""
    response = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "Extract named entities (people, organizations, drugs, diseases, locations) "
                           "from the query. Return a JSON array of entity names only."
            },
            {"role": "user", "content": query}
        ],
        response_format={"type": "json_object"}
    )
    result = json.loads(response.choices[0].message.content)
    return result.get("entities", [])

def get_graph_context(entities: list[str]) -> dict:
    """Traverse knowledge graph for entity context."""
    if not entities:
        return {}

    cypher = """
    UNWIND $entity_names AS name
    MATCH (e:Entity)
    WHERE e.name =~ ('(?i)' + name)

    // Get direct relationships (1 hop)
    OPTIONAL MATCH (e)-[r1]->(neighbor1:Entity)
    OPTIONAL MATCH (e)-[r2]->(neighbor2:Entity)

    RETURN
        e.name AS entity_name,
        e.type AS entity_type,
        e.description AS entity_description,
        collect(DISTINCT {
            name: neighbor1.name,
            type: neighbor1.type,
            relationship: type(r1),
            direction: 'outgoing'
        }) AS outgoing_relations,
        collect(DISTINCT {
            name: neighbor2.name,
            type: neighbor2.type,
            relationship: type(r2),
            direction: 'incoming'
        }) AS incoming_relations
    """

    with neo4j_driver.session() as session:
        result = session.run(cypher, entity_names=entities)
        graph_context = {}
        for record in result:
            entity = record["entity_name"]
            graph_context[entity] = {
                "type": record["entity_type"],
                "description": record["entity_description"],
                "relations": record["outgoing_relations"] + record["incoming_relations"]
            }
        return graph_context

def vector_search_with_graph_filter(

```

```

query: str,
entity_ids: Optional[list[str]] = None,
top_k: int = 5
) -> list[dict]:
    """Vector similarity search in pgvector, optionally filtered by entity graph."""
    embedding = openai_client.embeddings.create(
        input=query,
        model="text-embedding-3-small"
    ).data[0].embedding

    embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"

    with pg_conn.cursor() as cur:
        if entity_ids:
            cur.execute("""
                SELECT
                    c.chunk_id,
                    c.content,
                    c.source_doc,
                    c.section_title,
                    1 - (c.embedding <=> %s::vector) AS similarity
                FROM document_chunks c
                JOIN chunk_entities ce ON ce.chunk_id = c.chunk_id
                WHERE ce.entity_id = ANY(%s)
                ORDER BY c.embedding <=> %s::vector
                LIMIT %s
            """, (embedding_str, entity_ids, embedding_str, top_k))
        else:
            cur.execute("""
                SELECT
                    chunk_id,
                    content,
                    source_doc,
                    section_title,
                    1 - (embedding <=> %s::vector) AS similarity
                FROM document_chunks
                ORDER BY embedding <=> %s::vector
                LIMIT %s
            """, (embedding_str, embedding_str, top_k))

    rows = cur.fetchall()
    return [
        {
            "chunk_id": r[0],
            "content": r[1],
            "source_doc": r[2],
            "section_title": r[3],
            "similarity": float(r[4])
        }
        for r in rows
    ]

def graph_aware_rag(query: str) -> str:
    """Full graph-aware RAG pipeline."""

    # Step 1: Extract entities from query
    entities = extract_entities_from_query(query)
    print(f"Extracted entities: {entities}")

    # Step 2: Get structured graph context for those entities
    graph_context = get_graph_context(entities)
    print(f"Graph context retrieved for {len(graph_context)} entities")

    # Step 3: Vector search (filtered if we have entity context)
    # In production, translate entity names to IDs via a lookup table
    retrieved_chunks = vector_search_with_graph_filter(query, top_k=5)
    print(f"Retrieved {len(retrieved_chunks)} chunks via vector search")

    # Step 4: Construct enriched context for LLM
    graph_context_text = ""

```



```

for entity, ctx in graph_context.items():
    graph_context_text += f"\n[Entity: {entity} ({ctx['type']})]\n"
    if ctx["description"]:
        graph_context_text += f"Description: {ctx['description']}\n"
    if ctx["relations"]:
        relations_summary = [
            f" - {rel['relationship']} -> {rel['name']}"
            for rel in ctx["relations"][:5] # Limit to top 5 relations
        ]
        graph_context_text += "Relationships:\n" + "\n".join(relations_summary) + "\n"

chunks_text = "\n\n".join([
    f"[Source: {c['source_doc']} | Section: {c['section_title']} | Similarity: {c['similarity']:.3f}]\n{c['content']}"
    for c in retrieved_chunks
])

# Step 5: LLM generation with structured + semantic context
prompt = f"""Answer the following question using the provided context.

KNOWLEDGE GRAPH CONTEXT (Structured relationships):
{graph_context_text if graph_context_text else "No graph context available."}

DOCUMENT CONTEXT (Semantically similar passages):
{chunks_text}

QUESTION: {query}

Provide a comprehensive answer grounded in the provided context."""

response = openai_client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a precise research assistant. Ground your answers in the provided context."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.1
)

return response.choices[0].message.content

# Example usage
answer = graph_aware_rag(
    "What are the known drug interactions between statins and blood thinners in elderly patients?"
)
print(answer)

```

The separation of graph context (structured, relational facts) from vector context (semantically similar text passages) in the prompt is intentional. LLMs attend differently to structured relational facts than to free-text passages, and keeping them visually distinct in the prompt tends to improve attribution accuracy in the generated answer.

Choosing Between Graph-Vector Integration Strategies

The decision tree for graph-vector architecture is driven by three factors: relationship depth, data volume, and team operational capacity.

Relationship depth is the primary driver. If your graph queries require more than three hops to answer meaningful questions, PostgreSQL recursive CTEs become slow and operationally awkward. Neo4j, Amazon Neptune, or ArangoDB handle this naturally. At one to two hops, PostgreSQL with

a well-indexed adjacency table performs comparably to a dedicated graph database and eliminates operational complexity.

Data volume determines whether you can afford native hybrid query engines or need to separate the systems. Weaviate performs well at 50M+ vectors with cross-references, but its graph capabilities are limited to direct object references — it is not a property graph database. Neo4j’s vector index, as of version 5.x, handles tens of millions of vectors with HNSW but is not optimized for the billion-vector scale that Pinecone or Milvus serve. If you need billion-scale vectors with complex multi-hop graph traversal, you are building a hybrid architecture with two separate systems communicating at the application layer.

Team operational capacity is the factor that gets underweighted in architecture decisions. Running two production database systems — a graph database and a vector database — means two monitoring setups, two backup procedures, two capacity planning models, and two teams who need to understand both. Unless the retrieval quality improvement from the native graph-vector approach is significant and measurable, the simpler architecture wins.

A practical decision matrix:

Scenario	Recommended Architecture
<10M vectors, shallow relationships (2 hops)	PostgreSQL + pgvector + recursive CTEs
<50M vectors, moderate relationship complexity	Weaviate with cross-references
Any scale, deep graph traversal (>3 hops)	Neo4j 5.x or Amazon Neptune Analytics
>500M vectors + complex graphs	Separate vector DB (Pinecone/Milvus) + graph DB (Neo4j/Neptune), application-layer orchestration
Regulated environment, single-vendor preference	Amazon Neptune Analytics (unified)

Implementing the Application-Layer Orchestration Pattern

When you cannot fit both concerns into a single database, the application layer becomes the join engine. The pattern requires careful attention to consistency, latency, and the ordering of operations.

```
# Python
# Application-layer orchestration: Pinecone (vector) + Neo4j (graph)

from pinecone import Pinecone
from neo4j import GraphDatabase
from openai import OpenAI
from dataclasses import dataclass
from typing import Optional
import asyncio

@dataclass
class HybridSearchResult:
    doc_id: str
    title: str
    vector_score: float
    graph_score: float
```

```

combined_score: float
graph_hops: int
structural_context: dict

class HybridSearchOrchestrator:
    def __init__(
        self,
        pinecone_api_key: str,
        pinecone_index_name: str,
        neo4j_uri: str,
        neo4j_auth: tuple,
    ):
        self.pc = Pinecone(api_key=pinecone_api_key)
        self.index = self.pc.Index(pinecone_index_name)
        self.neo4j_driver = GraphDatabase.driver(neo4j_uri, auth=neo4j_auth)
        self.openai_client = OpenAI()

    def _get_embedding(self, text: str) -> list[float]:
        return self.openai_client.embeddings.create(
            input=text,
            model="text-embedding-3-small"
        ).data[0].embedding

    def _vector_search(
        self,
        embedding: list[float],
        top_k: int = 20,
        namespace: str = "",
        filter_dict: Optional[dict] = None
    ) -> list[dict]:
        """Pinecone vector search returning candidate IDs and scores."""
        response = self.index.query(
            vector=embedding,
            top_k=top_k,
            namespace=namespace,
            filter=filter_dict,
            include_metadata=True
        )
        return [
            {
                "doc_id": match.id,
                "vector_score": match.score,
                "metadata": match.metadata
            }
            for match in response.matches
        ]

    def _graph_score_and_expand(
        self,
        seed_doc_ids: list[str],
        expansion_depth: int = 2
    ) -> dict[str, dict]:
        """
        For each seed document, compute a graph centrality score and
        expand to neighboring nodes. Returns a dict keyed by doc_id.
        """
        cypher = """
        UNWIND $seed_ids AS seed_id
        MATCH (d:Document {doc_id: seed_id})

        // Compute in-degree as a structural importance proxy
        OPTIONAL MATCH (d)-[:CITES]-(citing:Document)
        WITH d, seed_id, count(citing) AS citation_count

        // Expand neighbors up to expansion_depth
        CALL apoc.path.subgraphNodes(d, {
            relationshipFilter: 'CITES>|RELATED_TO>',
            maxLevel: $depth,
            limit: 50
        })
        """

```

```

YIELD node AS neighbor

WITH
    seed_id,
    d.title AS title,
    citation_count,
    collect(DISTINCT {
        doc_id: neighbor.doc_id,
        title: neighbor.title,
        hops: 1
    }) AS neighbors

RETURN
    seed_id,
    title,
    citation_count,
    neighbors,
    // Normalize citation count to [0, 1] range using log scale
    // We use a soft normalization; tune the divisor for your corpus
    (1.0 - exp(-citation_count / 10.0)) AS graph_score
"""

with self.neo4j_driver.session() as session:
    result = session.run(
        cypher,
        seed_ids=seed_doc_ids,
        depth=expansion_depth
    )
    graph_data = {}
    for record in result:
        graph_data[record["seed_id"]] = {
            "title": record["title"],
            "citation_count": record["citation_count"],
            "graph_score": record["graph_score"],
            "neighbors": record["neighbors"]
        }
    return graph_data

def _fetch_neighbor_vectors(
    self,
    neighbor_ids: list[str],
    query_embedding: list[float]
) -> list[dict]:
    """Fetch vector scores for graph-expanded nodes not in original results."""
    if not neighbor_ids:
        return []

    # Fetch by ID from Pinecone
    fetch_response = self.index.fetch(ids=neighbor_ids)
    neighbor_results = []
    for doc_id, vector_data in fetch_response.vectors.items():
        if vector_data.values:
            # Compute cosine similarity manually
            import numpy as np
            qv = np.array(query_embedding)
            dv = np.array(vector_data.values)
            similarity = float(np.dot(qv, dv) / (np.linalg.norm(qv) * np.linalg.norm(dv)))
            neighbor_results.append({
                "doc_id": doc_id,
                "vector_score": similarity,
                "metadata": vector_data.metadata or {}
            })
    return neighbor_results

def search(
    self,
    query: str,
    top_k_vector: int = 20,
    expansion_depth: int = 2,
    vector_weight: float = 0.6,

```

```

graph_weight: float = 0.4,
final_top_k: int = 10
) -> list[HybridSearchResult]:
    """
    Full hybrid search:
    1. Vector search for seed candidates
    2. Graph scoring and expansion
    3. Fetch vector scores for expanded nodes
    4. Combined scoring and final ranking
    """

    # Step 1: Vector search
    embedding = self._get_embedding(query)
    vector_results = self._vector_search(embedding, top_k=top_k_vector)
    seed_ids = [r["doc_id"] for r in vector_results]
    vector_score_map = {r["doc_id"]: r["vector_score"] for r in vector_results}

    # Step 2: Graph scoring and expansion
    graph_data = self._graph_score_and_expand(seed_ids, expansion_depth)

    # Collect all neighbor IDs not already in vector results
    all_neighbor_ids = set()
    for seed_id, data in graph_data.items():
        for neighbor in data.get("neighbors", []):
            n_id = neighbor.get("doc_id")
            if n_id and n_id not in vector_score_map:
                all_neighbor_ids.add(n_id)

    # Step 3: Fetch vector scores for expanded nodes
    expanded_vector_results = self._fetch_neighbor_vectors(
        list(all_neighbor_ids), embedding
    )
    for r in expanded_vector_results:
        vector_score_map[r["doc_id"]] = r["vector_score"]

    # Build graph score map from all known nodes
    graph_score_map: dict[str, float] = {}
    for seed_id, data in graph_data.items():
        graph_score_map[seed_id] = data["graph_score"]

    # Step 4: Combined scoring
    all_candidate_ids = set(vector_score_map.keys())
    results = []

    for doc_id in all_candidate_ids:
        v_score = vector_score_map.get(doc_id, 0.0)
        g_score = graph_score_map.get(doc_id, 0.0)
        combined = (vector_weight * v_score) + (graph_weight * g_score)

        # Determine graph hops (0 = seed, 1+ = expanded)
        hops = 0 if doc_id in {r["doc_id"] for r in vector_results} else 1

        g_data = graph_data.get(doc_id, {})
        results.append(HybridSearchResult(
            doc_id=doc_id,
            title=g_data.get("title", "Unknown"),
            vector_score=v_score,
            graph_score=g_score,
            combined_score=combined,
            graph_hops=hops,
            structural_context={
                "citation_count": g_data.get("citation_count", 0),
                "neighbor_count": len(g_data.get("neighbors", []))
            }
        ))

    results.sort(key=lambda x: x.combined_score, reverse=True)
    return results[:final_top_k]

def close(self):
    self.neo4j_driver.close()

```

```
# Usage
orchestrator = HybridSearchOrchestrator(
    pinecone_api_key="your-key",
    pinecone_index_name="research-docs",
    neo4j_uri="bolt://localhost:7687",
    neo4j_auth=("neo4j", "password")
)

results = orchestrator.search(
    query="statin therapy and cognitive decline risk factors",
    top_k_vector=20,
    expansion_depth=2,
    vector_weight=0.65,
    graph_weight=0.35,
    final_top_k=10
)

for r in results:
    print(f"[{r.graph_hops} hops] {r.title}")
    print(f"  Vector: {r.vector_score:.4f} | Graph: {r.graph_score:.4f} | Combined:
    ↪ {r.combined_score:.4f}")
    print(f"  Citations: {r.structural_context['citation_count']}")

orchestrator.close()
```

Two implementation details here deserve attention. First, the graph score uses a log-normalized citation count rather than raw citation count. Raw citation count creates a rich-get-richer dynamic where the most-cited papers dominate regardless of semantic relevance. The formula $1.0 - \exp(-\text{citation_count} / 10.0)$ maps zero citations to a graph score of 0 and asymptotically approaches 1 as citations grow, with a reasonable inflection point around 10 citations for most academic corpora. Adjust the divisor based on the citation distribution of your specific dataset.

Second, the `_fetch_neighbor_vectors` step introduces a latency cost: a Pinecone `fetch` call for potentially hundreds of IDs. Whether this is acceptable depends on your latency budget. If the latency budget is tight, consider capping expansion to 1 hop, limiting neighbors per seed to the top 5 by graph edge weight, or pre-materializing graph-neighbor embeddings into a secondary index.

SQL Server Compatibility: Graph Tables and Vector Search

SQL Server 2017 introduced native graph table support (`CREATE TABLE AS NODE` and `CREATE TABLE AS EDGE`), and SQL Server 2025 (and Azure SQL Database's preview features) added vector similarity functions. This combination allows graph-vector hybrid queries within a single SQL Server instance, following the same pattern as the PostgreSQL + pgvector approach but using T-SQL syntax.

```
-- SQL Server
-- Graph tables combined with vector similarity search

-- Node tables
CREATE TABLE dbo.Documents (
    doc_id      UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    title       NVARCHAR(500) NOT NULL,
    content     NVARCHAR(MAX),
    embedding   VARBINARY(MAX), -- stored as binary; cast for similarity functions
    created_at  DATETIME2 DEFAULT SYSUTCDATETIME(),
    $node_id    -- implicit SQL Server graph node column
) AS NODE;
```

```

CREATE TABLE dbo.Authors (
    author_id    UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    name         NVARCHAR(200),
    institution  NVARCHAR(200),
    $node_id
) AS NODE;

-- Edge tables
CREATE TABLE dbo.Cites (
    citation_date DATE,
    $edge_id,      -- implicit edge columns
    $from_id,
    $to_id
) AS EDGE;

CREATE TABLE dbo.AuthoredBy (
    $edge_id,
    $from_id,
    $to_id
) AS EDGE;

-- Graph-vector hybrid query using MATCH syntax (SQL Server graph)
-- and VECTOR_DISTANCE (available in SQL Server 2025 / Azure SQL)

-- Step 1: Find semantically similar documents using vector search
-- Step 2: Use MATCH to traverse citation graph from those seeds
-- Step 3: Re-rank by combined score

DECLARE @query_embedding VARBINARY(MAX) = <your_embedding_binary>;
DECLARE @hop_depth INT = 2;
DECLARE @top_k INT = 10;

WITH VectorSeeds AS (
    SELECT TOP (@top_k)
        doc_id,
        title,
        embedding,
        VECTOR_DISTANCE('cosine', embedding, @query_embedding) AS cosine_distance
    FROM dbo.Documents
    ORDER BY VECTOR_DISTANCE('cosine', embedding, @query_embedding) ASC
),

-- One-hop expansion via SQL Server MATCH clause
OneHopExpansion AS (
    SELECT DISTINCT
        cited.doc_id      AS expanded_doc_id,
        seed.doc_id       AS origin_doc_id,
        1                 AS hop_count
    FROM VectorSeeds seed
    JOIN dbo.Documents AS citing
        ON seed.doc_id = citing.doc_id
    JOIN dbo.Cites AS c
        ON TRUE
    JOIN dbo.Documents AS cited
        ON TRUE
    WHERE MATCH(citing-(c)->cited)
),

-- Two-hop expansion
TwoHopExpansion AS (
    SELECT DISTINCT
        cited2.doc_id     AS expanded_doc_id,
        one.origin_doc_id,
        2                 AS hop_count
    FROM OneHopExpansion one
    JOIN dbo.Documents AS intermediate
        ON one.expanded_doc_id = intermediate.doc_id
    JOIN dbo.Cites AS c2
        ON TRUE

```

```

    JOIN dbo.Documents AS cited2
      ON TRUE
    WHERE MATCH(intermediate-(c2)->cited2)
  ),

  AllCandidates AS (
    SELECT doc_id AS candidate_id, 0 AS min_hops
    FROM VectorSeeds
    UNION ALL
    SELECT expanded_doc_id, 1 FROM OneHopExpansion
    UNION ALL
    SELECT expanded_doc_id, 2 FROM TwoHopExpansion
  ),

  Deduped AS (
    SELECT candidate_id, MIN(min_hops) AS min_hops
    FROM AllCandidates
    GROUP BY candidate_id
  )

  SELECT TOP 20
    d.doc_id,
    d.title,
    ROUND(1.0 - VECTOR_DISTANCE('cosine', d.embedding, @query_embedding), 4) AS semantic_score,
    dj.min_hops AS graph_hops,
    ROUND(
      (1.0 - VECTOR_DISTANCE('cosine', d.embedding, @query_embedding))
      * POWER(0.85, CAST(dj.min_hops AS FLOAT)),
      4
    ) AS combined_score
  FROM Deduped dj
  JOIN dbo.Documents d ON d.doc_id = dj.candidate_id
  ORDER BY combined_score DESC;

```

The SQL Server MATCH clause follows the ASCII-art arrow syntax (`from_node-(edge)->to_node`), which is unique to SQL Server’s graph implementation. Unlike Cypher, it cannot express variable-length path traversal in a single clause — hence the explicit one-hop and two-hop CTE expansion above. For deeper traversal, this approach becomes repetitive. SQL Server 2022 introduced `SHORTEST_PATH` which handles variable-length traversal more cleanly:

```

-- SQL Server 2022+: Variable-length path with SHORTEST_PATH

SELECT
  source.doc_id AS source_doc_id,
  target.doc_id AS target_doc_id,
  COUNT(edge.$edge_id) AS path_length
FROM dbo.Documents AS source,
     dbo.Cites FOR PATH AS edge,
     dbo.Documents FOR PATH AS target
WHERE MATCH(SHORTEST_PATH(source(-(edge)->target)+))
      AND source.doc_id IN (SELECT doc_id FROM VectorSeeds)
HAVING COUNT(edge.$edge_id) <= 3

```

`SHORTEST_PATH` finds the minimum hop count between source and target, which is preferable to the manual CTE expansion when you want the true graph distance rather than a fixed-depth expansion.

Evaluating Hybrid Retrieval Quality

Adding graph traversal to a vector search pipeline should be justified by measurable retrieval quality improvement. The standard evaluation framework applies: build a test set of queries with human-labeled relevant documents, compute Precision@k and NDCG@k for pure vector search,

then recompute after adding graph expansion.

A useful heuristic to check before running a full evaluation: compute the overlap between your graph-expanded candidate set and the human-labeled relevant set. If the graph expansion surfaces relevant documents that vector search missed, the architecture is earning its complexity. If the overlap is near zero — graph expansion consistently surfaces documents that human raters do not find relevant — your graph edges may not encode the right relationships for the retrieval task.

The graph connectivity metric from the Neo4j example earlier (`graph_connectivity`) can serve as a proxy relevance signal during development. Documents with zero graph connectivity in a well-constructed knowledge graph are either orphans (data quality problem) or highly specialized documents with narrow applicability. High-connectivity documents are structurally central and tend to be more broadly relevant. Track this distribution in your test set before tuning score weights.

```
# Python
# Offline evaluation: compare pure vector vs. graph-vector hybrid retrieval

from sklearn.metrics import ndcg_score
import numpy as np

def evaluate_retrieval(
    test_queries: list[dict], # [{"query": str, "relevant_ids": list[str]}]
    searcher_fn,              # callable(query) -> list[str] of doc_ids
    k: int = 10
) -> dict:
    """
    Compute Precision@k and NDCG@k over a labeled test set.

    test_queries: list of dicts with 'query' and 'relevant_ids' keys
    searcher_fn: function that takes a query string and returns ordered list of doc_ids
    k: cutoff rank
    """
    precisions = []
    ndcg_scores = []

    for item in test_queries:
        query = item["query"]
        relevant = set(item["relevant_ids"])

        retrieved = searcher_fn(query)[:k]

        # Precision@k
        hits = sum(1 for doc_id in retrieved if doc_id in relevant)
        precisions.append(hits / k)

        # NDCG@k: binary relevance
        relevance_true = np.array([[1 if doc_id in relevant else 0
                                    for doc_id in retrieved]])

        # Ideal ranking: all relevant docs at top
        ideal = np.array([[1 * min(len(relevant), k) +
                           [0 * max(0, k - len(relevant))]])

        if relevance_true.sum() > 0:
            ndcg_scores.append(ndcg_score(ideal, relevance_true, k=k))
        else:
            ndcg_scores.append(0.0)

    return {
        f"precision_at_{k}": np.mean(precisions),
        f"ndcg_at_{k}": np.mean(ndcg_scores),
        "n_queries": len(test_queries)
    }

# Compare strategies
def pure_vector_search(query: str) -> list[str]:
```

```
# Returns doc_ids from vector-only search
...

def hybrid_search(query: str) -> list[str]:
    # Returns doc_ids from graph+vector hybrid search
    ...

# Load your labeled test set
test_set = [
    {"query": "statin drug interactions elderly", "relevant_ids": ["doc_001", "doc_045", "doc_102"]},
    # ... more test cases
]

vector_metrics = evaluate_retrieval(test_set, pure_vector_search, k=10)
hybrid_metrics = evaluate_retrieval(test_set, hybrid_search, k=10)

print("Pure Vector Search:")
print(f" Precision@10: {vector_metrics['precision_at_10']:.4f}")
print(f" NDCG@10:      {vector_metrics['ndcg_at_10']:.4f}")

print("\nGraph-Vector Hybrid Search:")
print(f" Precision@10: {hybrid_metrics['precision_at_10']:.4f}")
print(f" NDCG@10:      {hybrid_metrics['ndcg_at_10']:.4f}")

improvement = (hybrid_metrics['ndcg_at_10'] - vector_metrics['ndcg_at_10']) /
    ↪ vector_metrics['ndcg_at_10'] * 100
print(f"\nNDCG@10 improvement: {improvement:+.1f}%")
```

Run this evaluation before committing to the hybrid architecture. If the improvement is under a few percentage points in NDCG@10, the operational complexity of maintaining a graph database alongside your vector store is likely not justified. If you see meaningful gains — particularly on queries that require structural disambiguation — the hybrid approach is paying for itself.

Key Takeaways

- **Graph and vector search solve orthogonal retrieval problems.** Vector similarity finds semantically proximate content; graph traversal finds structurally related content. Many production AI applications require both, and the combination surfaces results that neither approach finds alone.
- **Three integration patterns cover most production needs:** pre-filter by graph then rank by vector, post-retrieve by vector then expand by graph, and graph-guided RAG where entity relationships shape what context the LLM receives. Choose the pattern based on whether structural constraints are known before or after the semantic query.
- **PostgreSQL + pgvector + recursive CTEs is a viable graph-vector stack** for shallow relationship graphs (one to two hops) and data volumes below the tens of millions. It eliminates operational complexity at the cost of expressiveness for deep traversal queries.
- **Native graph-vector engines (Neo4j 5.x, Weaviate, Amazon Neptune Analytics) reduce application-layer complexity** but impose scale ceilings and operational learning curves. Evaluate them when your graph queries exceed two hops or when application-layer orchestration between two separate systems creates unacceptable latency or consistency challenges.
- **Measure before you build.** Graph expansion should be justified by Precision@k and NDCG@k improvements on a labeled test set. If the graph structure of your data does not

encode relationships that are semantically meaningful for your retrieval task, adding a graph layer adds latency and operational burden without improving answer quality.

Chapter 33: Semantic Caching for LLM Applications

LLM API calls are expensive, slow, and deterministic enough that many questions asked of a system are semantically equivalent even when worded differently. A user asking “What’s the refund policy?” and another asking “How do I get my money back?” want the same answer. Conventional caching based on exact string matching would treat these as cache misses. Semantic caching uses vector similarity to recognize that two queries mean roughly the same thing and return a cached response—cutting latency from seconds to milliseconds and reducing token costs by 40–80% in production workloads with high query repetition. This chapter covers how semantic caches work, how to build one with pgvector and Redis, how to integrate caching into LangChain and LlamaIndex pipelines, and how to operate a semantic cache in production without sacrificing answer quality.

33.1 How Semantic Caching Differs from Conventional Caching

Traditional application caches operate on exact key matching. You hash the input, look up the hash, return the value if it exists. This works well when inputs are programmatically generated and predictable—SQL query strings, API endpoint parameters, serialized data structures. It fails completely for natural language, where the same information need generates an infinite variety of phrasings.

Semantic caching replaces the hash lookup with a vector similarity search. The cache stores (embedding, response) pairs. When a new query arrives, you embed it, search the cache for the nearest stored embedding, and if the similarity exceeds a threshold, return the cached response rather than hitting the LLM. The similarity threshold is the most operationally significant parameter in the entire system.

The threshold problem in database terms

Think of the similarity threshold like a fuzzy match tolerance in a string similarity join. Set `similarity_threshold = 1.0` and you effectively get exact matching—nothing hits the cache. Set it to 0.7 and you start returning cached responses to queries that are topically related but semantically distinct. A question about “returning a defective product” and “canceling a subscription” might both score 0.75 cosine similarity against a cached refund policy answer, but they need different responses.

In production systems, threshold tuning requires: - A labeled evaluation set of query pairs tagged as “same intent” or “different intent” - Precision/recall measurement against that set at multiple

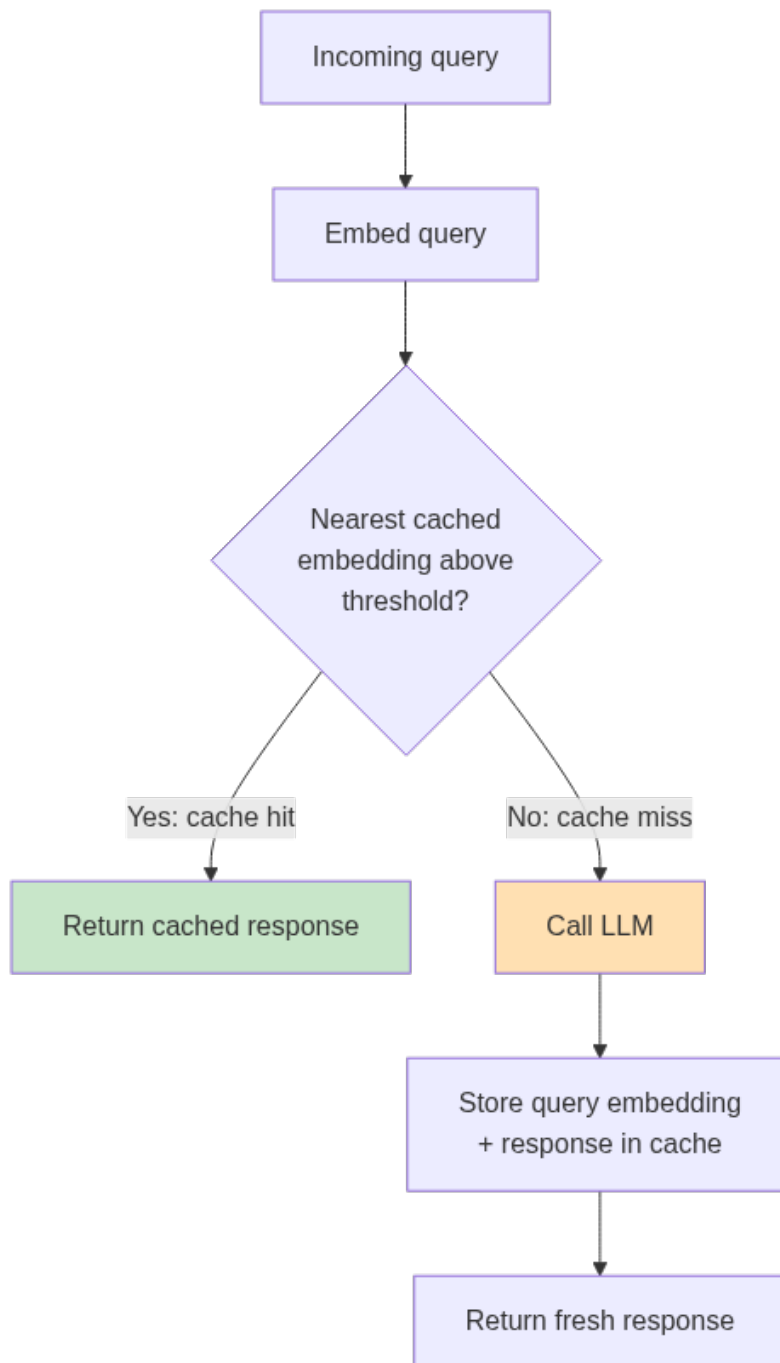


Figure 23: The similarity threshold at the decision point is the single most consequential tuning parameter — too low and wrong answers get served with confidence, too high and the cache never fires.

thresholds - Separate thresholds per query domain (customer support vs. code generation vs. data lookup)

Typical production thresholds by domain:

Domain	Threshold (cosine)	Rationale
FAQ / support	0.92–0.95	High repetition, controlled vocabulary
Code generation	0.97–0.99	Small semantic changes produce different code
Document Q&A	0.93–0.96	User intent more variable
Data lookup / SQL gen	0.96–0.99	Precision required

Cache invalidation

Semantic caches share the same invalidation challenges as conventional caches, plus a new one: a cached response may become stale not because the query changed but because the underlying data or policy changed. A refund policy answer cached in January may be wrong after a February policy update. You need TTL-based expiration and event-driven invalidation tied to source document updates, just as you would in any read-through cache architecture.

33.2 Building a Semantic Cache with pgvector

PostgreSQL with pgvector is a natural fit for a semantic cache if your application is already running on Postgres. You avoid introducing a new infrastructure component, benefit from transactional consistency, and can use standard SQL for cache management queries.

Schema design

```
-- PostgreSQL / pgvector

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE semantic_cache (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  query_text  TEXT NOT NULL,
  query_embedding VECTOR(1536) NOT NULL,
  response_text TEXT NOT NULL,
  model       TEXT NOT NULL DEFAULT 'gpt-4o',
  namespace  TEXT NOT NULL DEFAULT 'default',
  hit_count   INTEGER NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_accessed TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at  TIMESTAMPTZ,
  metadata    JSONB
);

-- HNSW index for fast ANN lookup
-- ef_construction=128, m=16 are reasonable defaults for a cache
-- that may grow to millions of entries
CREATE INDEX idx_semantic_cache_embedding
  ON semantic_cache
  USING hnsw (query_embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);
```

```

-- Namespace + expiry index for cache management
CREATE INDEX idx_semantic_cache_namespace_expires
ON semantic_cache (namespace, expires_at)
WHERE expires_at IS NOT NULL;

-- Function to look up cache and update hit count atomically
CREATE OR REPLACE FUNCTION semantic_cache_lookup(
    p_embedding    VECTOR(1536),
    p_namespace    TEXT,
    p_threshold    FLOAT DEFAULT 0.93,
    p_limit        INT DEFAULT 1
)
RETURNS TABLE (
    id            UUID,
    query_text    TEXT,
    response_text TEXT,
    similarity     FLOAT
)
LANGUAGE SQL
AS $$
    UPDATE semantic_cache sc
    SET
        hit_count      = hit_count + 1,
        last_accessed = NOW()
    FROM (
        SELECT
            c.id,
            c.query_text,
            c.response_text,
            1 - (c.query_embedding <=> p_embedding) AS similarity
        FROM semantic_cache c
        WHERE
            c.namespace = p_namespace
            AND (c.expires_at IS NULL OR c.expires_at > NOW())
            AND 1 - (c.query_embedding <=> p_embedding) >= p_threshold
        ORDER BY c.query_embedding <=> p_embedding
        LIMIT p_limit
    ) ranked
    WHERE sc.id = ranked.id
    RETURNING sc.id, sc.query_text, sc.response_text, ranked.similarity;
$$;

-- Purge expired entries (run on a schedule)
CREATE OR REPLACE FUNCTION purge_expired_cache(p_namespace TEXT DEFAULT NULL)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_deleted INTEGER;
BEGIN
    DELETE FROM semantic_cache
    WHERE expires_at < NOW()
        AND (p_namespace IS NULL OR namespace = p_namespace);
    GET DIAGNOSTICS v_deleted = ROW_COUNT;
    RETURN v_deleted;
END;
$$;

```

Python client wrapping the pgvector cache

```

# Python
import hashlib
import asyncio
import psycpg
import numpy as np
from openai import AsyncOpenAI
from datetime import datetime, timedelta, timezone
from typing import Optional

# pip install openai psycpg[binary] numpy

```



```

class PgvectorSemanticCache:
    """
    Semantic cache backed by pgvector.
    Drop-in wrapper around any LLM call function.
    """

    def __init__(
        self,
        dsn: str,
        openai_client: AsyncOpenAI,
        namespace: str = "default",
        threshold: float = 0.93,
        ttl_seconds: int = 86400, # 24 hours
        embedding_model: str = "text-embedding-3-small",
    ):
        self.dsn = dsn
        self.oai = openai_client
        self.namespace = namespace
        self.threshold = threshold
        self.ttl_seconds = ttl_seconds
        self.embedding_model = embedding_model
        self._pool: Optional[psycopg.AsyncConnection] = None

    async def _get_conn(self) -> psycopg.AsyncConnection:
        if self._pool is None:
            self._pool = await psycopg.AsyncConnection.connect(self.dsn)
        return self._pool

    async def embed(self, text: str) -> list[float]:
        response = await self.oai.embeddings.create(
            model=self.embedding_model,
            input=text,
        )
        return response.data[0].embedding

    async def get(self, query: str) -> Optional[dict]:
        embedding = await self.embed(query)
        embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"

        conn = await self._get_conn()
        async with conn.cursor() as cur:
            await cur.execute(
                """
                SELECT id, query_text, response_text, similarity
                FROM semantic_cache_lookup(
                    %s::vector,
                    %s,
                    %s,
                    1
                )
                """
                , (embedding_str, self.namespace, self.threshold),
            )
            row = await cur.fetchone()
            if row:
                return {
                    "id": str(row[0]),
                    "cached_query": row[1],
                    "response": row[2],
                    "similarity": float(row[3]),
                    "cache_hit": True,
                }
            return None

    async def set(
        self,
        query: str,
        response: str,
        model: str = "gpt-4o",
    )

```

```

        metadata: Optional[dict] = None,
    ) -> str:
        embedding = await self.embed(query)
        embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
        expires_at = datetime.now(timezone.utc) + timedelta(seconds=self.ttl_seconds)

        conn = await self._get_conn()
        async with conn.cursor() as cur:
            await cur.execute(
                """
                INSERT INTO semantic_cache
                (query_text, query_embedding, response_text,
                 model, namespace, expires_at, metadata)
                VALUES (%s, %s::vector, %s, %s, %s, %s, %s)
                RETURNING id
                """
                (
                    query,
                    embedding_str,
                    response,
                    model,
                    self.namespace,
                    expires_at,
                    metadata,
                ),
            )
            result = await cur.fetchone()
            await conn.commit()
            return str(result[0])

    async def get_or_generate(
        self,
        query: str,
        llm_call_fn,
        model: str = "gpt-4o",
    ) -> dict:
        """
        Cache-aside pattern: check cache first, generate and store on miss.
        llm_call_fn: async callable(query) -> str
        """
        cached = await self.get(query)
        if cached:
            return cached

        # Cache miss - call the LLM
        response_text = await llm_call_fn(query)
        entry_id = await self.set(query, response_text, model=model)

        return {
            "id": entry_id,
            "response": response_text,
            "similarity": 1.0,
            "cache_hit": False,
        }

# Usage example
async def main():
    oai = AsyncOpenAI()
    cache = PgvectorSemanticCache(
        dsn="postgresql://app:secret@localhost:5432/llmcache",
        openai_client=oai,
        namespace="customer_support",
        threshold=0.93,
        ttl_seconds=3600 * 24 * 7,  # 7 days
    )

    async def call_llm(query: str) -> str:
        response = await oai.chat.completions.create(
            model="gpt-4o",

```

```

        messages=[{"role": "user", "content": query}],
    )
    return response.choices[0].message.content

# First call - cache miss, LLM invoked
result1 = await cache.get_or_generate(
    "What is your return policy for electronics?",
    call_llm,
)
print(f"Cache hit: {result1['cache_hit']}, similarity: {result1['similarity']:.4f}")

# Semantically similar - should hit the cache
result2 = await cache.get_or_generate(
    "Can I return a laptop I bought last week?",
    call_llm,
)
print(f"Cache hit: {result2['cache_hit']}, similarity: {result2['similarity']:.4f}")

asyncio.run(main())

```

33.3 Semantic Caching with Redis and a Dedicated Vector Store

For applications that need sub-5ms cache lookups and cannot afford a full Postgres query round-trip, Redis with the RediSearch module (or Redis Stack) provides an in-memory vector cache. The tradeoff is that Redis is bounded by available memory; entries must be kept small or the cache must be tiered.

An alternative pattern uses a dedicated fast vector store—Qdrant or Chroma—as the semantic index while storing full response text in Redis by ID. The vector store answers “does a similar query exist?” (returning an ID), and Redis retrieves the response body by that ID in a second sub-millisecond lookup.

```

# Python
# Tiered semantic cache: Qdrant for similarity search, Redis for response storage
# pip install qdrant-client redis openai sentence-transformers

import json
import uuid
import hashlib
import redis.asyncio as aioredis
from qdrant_client import AsyncQdrantClient
from qdrant_client.models import (
    Distance,
    VectorParams,
    PointStruct,
    Filter,
    FieldCondition,
    MatchValue,
    SearchRequest,
)
from sentence_transformers import SentenceTransformer
from typing import Optional
import asyncio

COLLECTION_NAME = "semantic_cache_index"
EMBEDDING_DIM = 384 # all-MiniLM-L6-v2
CACHE_TTL_SECONDS = 86400

class TieredSemanticCache:
    """

```

```

Qdrant holds the embedding index.
Redis holds response payloads keyed by cache entry ID.
Embedding is done locally with sentence-transformers to avoid
external API latency on cache reads.
"""

def __init__(
    self,
    qdrant_url: str = "http://localhost:6333",
    redis_url: str = "redis://localhost:6379",
    threshold: float = 0.92,
    namespace: str = "default",
):
    self.qdrant = AsyncQdrantClient(url=qdrant_url)
    self.redis_url = redis_url
    self.threshold = threshold
    self.namespace = namespace
    self.encoder = SentenceTransformer("all-MiniLM-L6-v2")
    self._redis: Optional[aioredis.Redis] = None

    async def _get_redis(self) -> aioredis.Redis:
        if self._redis is None:
            self._redis = await aioredis.from_url(self.redis_url, decode_responses=True)
        return self._redis

    async def initialize(self):
        """Create Qdrant collection if it doesn't exist."""
        collections = await self.qdrant.get_collections()
        existing = [c.name for c in collections.collections]
        if COLLECTION_NAME not in existing:
            await self.qdrant.create_collection(
                collection_name=COLLECTION_NAME,
                vectors_config=VectorParams(
                    size=EMBEDDING_DIM,
                    distance=Distance.COSINE,
                ),
            )
            print(f"Created collection: {COLLECTION_NAME}")

    def embed(self, text: str) -> list[float]:
        # SentenceTransformer is synchronous; wrap in thread pool for async use
        vec = self.encoder.encode(text, normalize_embeddings=True)
        return vec.tolist()

    async def get(self, query: str) -> Optional[dict]:
        embedding = self.embed(query)

        results = await self.qdrant.search(
            collection_name=COLLECTION_NAME,
            query_vector=embedding,
            query_filter=Filter(
                must=[
                    FieldCondition(
                        key="namespace",
                        match=MatchValue(value=self.namespace),
                    )
                ]
            ),
            limit=1,
            score_threshold=self.threshold,
        )

        if not results:
            return None

        hit = results[0]
        entry_id = hit.payload["entry_id"]
        similarity = hit.score

        # Fetch response from Redis

```

```

redis = await self._get_redis()
payload_json = await redis.get(f"scache:{entry_id}")
if payload_json is None:
    # Redis entry expired but Qdrant entry still exists - treat as miss
    return None

payload = json.loads(payload_json)
return {
    "entry_id": entry_id,
    "cached_query": payload["query"],
    "response": payload["response"],
    "model": payload.get("model"),
    "similarity": similarity,
    "cache_hit": True,
}

async def set(
    self,
    query: str,
    response: str,
    model: str = "gpt-4o",
    metadata: Optional[dict] = None,
) -> str:
    entry_id = str(uuid.uuid4())
    embedding = self.embed(query)

    # Write to Qdrant
    await self.qdrant.upsert(
        collection_name=COLLECTION_NAME,
        points=[
            PointStruct(
                id=entry_id,
                vector=embedding,
                payload={
                    "namespace": self.namespace,
                    "entry_id": entry_id,
                    "query_preview": query[:100],
                },
            )
        ],
    )

    # Write response to Redis with TTL
    payload = {
        "query": query,
        "response": response,
        "model": model,
        "metadata": metadata or {},
    }
    redis = await self._get_redis()
    await redis.setex(
        f"scache:{entry_id}",
        CACHE_TTL_SECONDS,
        json.dumps(payload),
    )

    return entry_id

async def invalidate_namespace(self, namespace: str):
    """Delete all cache entries for a namespace from Qdrant."""
    await self.qdrant.delete(
        collection_name=COLLECTION_NAME,
        points_selector=Filter(
            must=[
                FieldCondition(
                    key="namespace",
                    match=MatchValue(value=namespace),
                )
            ],
        ),
    ),

```

```
)

# Benchmark: measure cache hit latency vs LLM latency
async def benchmark():
    import time

    cache = TieredSemanticCache(threshold=0.90, namespace="demo")
    await cache.initialize()

    test_query = "Explain the difference between OLTP and OLAP databases."

    # Warm the cache with a synthetic response
    await cache.set(
        query=test_query,
        response="OLTP systems handle high-volume transactional workloads...",
        model="gpt-4o",
    )

    # Measure lookup latency
    queries = [
        "What is the difference between OLTP and OLAP?",
        "OLAP vs OLTP - what should I know?",
        "How are transactional and analytical databases different?",
    ]

    for q in queries:
        t0 = time.perf_counter()
        result = await cache.get(q)
        elapsed_ms = (time.perf_counter() - t0) * 1000
        hit = result is not None
        sim = result["similarity"] if result else 0.0
        print(f"[{elapsed_ms:5.1f}ms] hit={hit} sim={sim:.4f} | {q[:60]}")

asyncio.run(benchmark())
```

Typical latency profile for this tiered design on local hardware: - Embedding (MiniLM, CPU): 3–8ms - Qdrant ANN search (in-memory, 100K entries): 0.5–2ms - Redis GET: 0.2–0.5ms - **Total cache hit latency: 4–12ms** - GPT-4o API call: 800ms–4000ms

The 100x latency reduction justifies the operational complexity for high-traffic applications.

33.4 LangChain and LlamaIndex Integration

Both LangChain and LlamaIndex ship first-class semantic cache integrations. Using them avoids reimplementing cache logic and ensures the cache participates correctly in chains and agent loops.

LangChain with GPTRCache and pgvector

```
# Python
# LangChain semantic cache using the built-in SQLAlchemy + pgvector backend
# pip install langchain langchain-openai langchain-community pgvector psycopg2-binary

import os
from langchain.globals import set_llm_cache
from langchain_community.cache import InMemoryCache
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain.schema import HumanMessage

# LangChain ships a semantic cache that uses any vector store
# via the SemanticSimilarityExactMatchCache interface
from langchain_community.cache import RedisSemanticCache
```

```

from langchain_openai import OpenAIEmbeddings

# Option A: Redis semantic cache (simplest for prototyping)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

set_llm_cache(
    RedisSemanticCache(
        redis_url=os.environ.get("REDIS_URL", "redis://localhost:6379"),
        embedding=embeddings,
        score_threshold=0.2, # LangChain uses L2 distance; 0.2 0.93 cosine
    )
)

llm = ChatOpenAI(model="gpt-4o", temperature=0)

# All calls through this LLM instance now go through the semantic cache
import time

queries = [
    "What are the ACID properties of a database?",
    "Can you explain ACID compliance in databases?",
    "What does ACID stand for in database systems?",
]

for query in queries:
    t0 = time.perf_counter()
    response = llm.invoke([HumanMessage(content=query)])
    elapsed = (time.perf_counter() - t0) * 1000
    print(f"[{elapsed:6.0f}ms] {query[:50]}")
    print(f"           {response.content[:80]}...")
    print()

```

LlamaIndex with a custom semantic cache node postprocessor

```

# Python
# LlamaIndex: semantic cache as a query-level postprocessor
# This pattern caches full RAG query results, not just LLM calls
# pip install llama-index llama-index-vector-stores-qdrant openai

import time
import hashlib
from typing import Optional, List
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.core.schema import NodeWithScore, QueryBundle
from llama_index.core.postprocessor.types import BaseNodePostprocessor
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.llms.openai import OpenAI

# --- Simple in-process semantic cache for RAG responses ---
# In production, replace the dict store with pgvector or Qdrant

class RAGSemanticCache:
    """
    Caches full (query -> RAG response) pairs using embedding similarity.
    Stored in-memory here; swap _store for a persistent backend.
    """

    def __init__(
        self,
        embedding_model: OpenAIEmbedding,
        threshold: float = 0.94,
    ):
        self.embedder = embedding_model
        self.threshold = threshold
        # List of (embedding: list[float], response: str)
        self._store: list[tuple[list[float], str, str]] = [] # (embedding, query, response)

    def _cosine_similarity(self, a: list[float], b: list[float]) -> float:
        import math
        dot = sum(x * y for x, y in zip(a, b))

```

```

    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return dot / (norm_a * norm_b)

    async def get(self, query: str) -> Optional[dict]:
        embedding = await self.embedder.aget_text_embedding(query)
        best_sim = 0.0
        best_entry = None
        for stored_emb, stored_query, stored_response in self._store:
            sim = self._cosine_similarity(embedding, stored_emb)
            if sim > best_sim:
                best_sim = sim
                best_entry = (stored_query, stored_response)
        if best_entry and best_sim >= self.threshold:
            return {
                "cached_query": best_entry[0],
                "response": best_entry[1],
                "similarity": best_sim,
                "cache_hit": True,
            }
        return None

    async def set(self, query: str, response: str) -> None:
        embedding = await self.embedder.aget_text_embedding(query)
        self._store.append((embedding, query, response))

class SemanticCachedQueryEngine:
    """
    Wraps a LlamaIndex query engine with a semantic response cache.
    On cache hit, returns the cached response without touching the index.
    """

    def __init__(self, query_engine, cache: RAGSemanticCache):
        self.engine = query_engine
        self.cache = cache
        self.stats = {"hits": 0, "misses": 0}

    async def aquery(self, query_str: str) -> dict:
        # Check cache
        cached = await self.cache.get(query_str)
        if cached:
            self.stats["hits"] += 1
            return {
                "response": cached["response"],
                "cache_hit": True,
                "similarity": cached["similarity"],
                "cached_query": cached["cached_query"],
            }

        # Cache miss - run the full RAG pipeline
        self.stats["misses"] += 1
        t0 = time.perf_counter()
        rag_response = await self.engine.aquery(query_str)
        elapsed_ms = (time.perf_counter() - t0) * 1000
        response_text = str(rag_response)

        # Store in cache
        await self.cache.set(query_str, response_text)

        return {
            "response": response_text,
            "cache_hit": False,
            "rag_latency_ms": elapsed_ms,
        }

    def query(self, query_str: str) -> dict:
        import asyncio

```



```

        return asyncio.get_event_loop().run_until_complete(self.aquery(query_str))

# Wire it together
async def build_cached_rag_engine(docs_path: str):
    embed_model = OpenAIEmbedding(model="text-embedding-3-small")
    llm = OpenAI(model="gpt-4o", temperature=0)

    Settings.embed_model = embed_model
    Settings.llm = llm

    documents = SimpleDirectoryReader(docs_path).load_data()
    index = VectorStoreIndex.from_documents(documents)
    base_engine = index.as_query_engine(similarity_top_k=4)

    cache = RAGSemanticCache(embedding_model=embed_model, threshold=0.94)
    cached_engine = SemanticCachedQueryEngine(base_engine, cache)
    return cached_engine

```

The right level to cache in a RAG pipeline

A common design question: should the semantic cache sit in front of the LLM call, or in front of the entire RAG pipeline (retrieval + LLM)?

Cache placement	What is cached	When to use
LLM-only	LLM response for a given prompt (including retrieved context)	High prompt repetition, stable retrieval
Full RAG (query → response)	Complete answer for a user query	When retrieval is also expensive (large corpus, many chunks)
Retrieval-only	Retrieved document chunks for a query	Stable corpora, expensive vector search at scale

For most production systems, caching at the full RAG level (query → final answer) offers the best latency savings, since retrieval and LLM inference are both bypassed. The tradeoff is that the cached answer reflects the document state at the time of the original query—if documents are updated frequently, TTLs must be set accordingly.

33.5 Cache Hit Rate Optimization

A semantic cache delivering a 20% hit rate is nearly useless. To reach the 50–80% hit rates that justify the infrastructure, you need deliberate optimization across four levers: query normalization, namespace segmentation, embedding model selection, and threshold tuning.

Query normalization before embedding

Small textual differences that don't affect meaning can scatter embeddings unnecessarily. Normalizing queries before embedding pushes semantically equivalent questions closer together in embedding space.

```

# Python
import re

```

```

import unicodedata

def normalize_query(text: str) -> str:
    """
    Normalize a user query before embedding for cache lookup.
    Reduces unnecessary cache misses from surface-form variation.
    """
    # Unicode normalize (handles fancy quotes, diacritics in English queries)
    text = unicodedata.normalize("NFKC", text)

    # Lowercase
    text = text.lower()

    # Collapse whitespace (tabs, newlines, multiple spaces)
    text = re.sub(r"\s+", " ", text).strip()

    # Remove trailing punctuation that doesn't affect meaning
    text = re.sub(r"[?.,;:]+$", "", text).strip()

    # Expand common contractions
    contractions = {
        "what's": "what is",
        "i'm": "i am",
        "can't": "cannot",
        "don't": "do not",
        "won't": "will not",
        "how's": "how is",
        "it's": "it is",
        "that's": "that is",
        "there's": "there is",
        "i've": "i have",
        "you've": "you have",
        "we've": "we have",
        "i'd": "i would",
        "you'd": "you would",
        "he'd": "he would",
        "she'd": "she would",
    }
    for contraction, expansion in contractions.items():
        text = re.sub(r"\b" + re.escape(contraction) + r"\b", expansion, text)

    return text

# Example - these should now embed closer together
queries = [
    "What's your return policy?",
    "What is your return policy?",
    "What Is Your Return Policy?",
    "what's your return policy?",
]

for q in queries:
    print(repr(normalize_query(q)))
# All output: 'what is your return policy'

```

Namespace segmentation

A single cache namespace mixing product questions, support queries, and code generation requests will produce false positive hits—code generation questions semantically overlapping with English descriptions. Segmenting by namespace (which maps to a Qdrant collection filter or a Postgres `WHERE namespace = ... predicate`) keeps the search space focused and reduces false positives.

```

# Python
NAMESPACE_MAP = {
    "/support": "support",
    "/docs": "documentation",
    "/code-assistant": "code_generation",
}

```

```

    "/data-explorer": "sql_generation",
    "/api": "api_default",
}

def route_to_namespace(request_path: str) -> str:
    for prefix, ns in NAMESPACE_MAP.items():
        if request_path.startswith(prefix):
            return ns
    return "default"

```

Measuring hit rate and cache quality

```

-- PostgreSQL
-- Hit rate per namespace over the last 24 hours
SELECT
    namespace,
    COUNT(*) AS total_entries,
    SUM(hit_count) AS total_hits,
    AVG(hit_count) AS avg_hits_per_entry,
    MAX(hit_count) AS max_hits_single_entry,
    COUNT(*) FILTER (WHERE hit_count = 0) AS zero_hit_entries,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE hit_count > 0) / NULLIF(COUNT(*), 0),
        1
    ) AS pct_entries_ever_hit
FROM semantic_cache
WHERE created_at > NOW() - INTERVAL '24 hours'
GROUP BY namespace
ORDER BY total_hits DESC;

-- Top cached queries by hit count - useful for validating cache quality
SELECT
    query_text,
    hit_count,
    ROUND((1.0 - 0) * 100, 0) AS similarity_pct, -- placeholder; join with access log for real
    -- similarity
    created_at,
    last_accessed,
    expires_at
FROM semantic_cache
WHERE namespace = 'support'
ORDER BY hit_count DESC
LIMIT 20;

-- Stale entries: created but never accessed and older than 6 hours
-- (candidates for eviction to free index space)
SELECT COUNT(*) AS stale_count
FROM semantic_cache
WHERE hit_count = 0
    AND created_at < NOW() - INTERVAL '6 hours';

```

Threshold sensitivity analysis

Before deploying to production, run an offline analysis on a sample of logged queries to find the threshold that maximizes F1 score across your labeled evaluation set.

```

# Python
# Offline threshold analysis
# Requires: a list of (query_a, query_b, is_same_intent: bool) pairs

import numpy as np
from openai import OpenAI

oai = OpenAI()

def get_embedding(text: str) -> np.ndarray:
    response = oai.embeddings.create(
        model="text-embedding-3-small",
        input=text,
    )

```

```

    )
    return np.array(response.data[0].embedding)

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-10))

def analyze_thresholds(
    eval_pairs: list[tuple[str, str, bool]],
    thresholds: list[float] = None,
) -> list[dict]:
    if thresholds is None:
        thresholds = [round(t, 2) for t in np.arange(0.80, 1.00, 0.01)]

    # Precompute similarities
    similarities = []
    labels = []
    for q_a, q_b, same_intent in eval_pairs:
        emb_a = get_embedding(q_a)
        emb_b = get_embedding(q_b)
        sim = cosine_similarity(emb_a, emb_b)
        similarities.append(sim)
        labels.append(same_intent)

    similarities = np.array(similarities)
    labels = np.array(labels)

    results = []
    for thresh in thresholds:
        predictions = similarities >= thresh
        tp = int(np.sum(predictions & labels))
        fp = int(np.sum(predictions & ~labels))
        fn = int(np.sum(~predictions & labels))
        tn = int(np.sum(~predictions & ~labels))

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = (2 * precision * recall / (precision + recall)
              if (precision + recall) > 0 else 0.0)

        results.append({
            "threshold": thresh,
            "precision": round(precision, 4),
            "recall": round(recall, 4),
            "f1": round(f1, 4),
            "tp": tp, "fp": fp, "fn": fn, "tn": tn,
        })

    # Sort by F1 descending
    results.sort(key=lambda r: r["f1"], reverse=True)
    return results

# Example eval set (in production, source this from human-labeled query logs)
eval_pairs = [
    ("What is your refund policy?", "How do I get a refund?", True),
    ("What is your refund policy?", "How do I cancel my subscription?", False),
    ("Show me my order history", "List my past orders", True),
    ("Show me my order history", "What is the return window?", False),
    ("How do I reset my password?", "I forgot my password, help", True),
    ("How do I reset my password?", "What are your business hours?", False),
]

results = analyze_thresholds(eval_pairs)
print(f"{'Threshold':>10} {'Precision':>10} {'Recall':>8} {'F1':>8}")
print("-" * 42)
for r in results[:10]:
    print(
        f"{r['threshold']:>10.2f} "
        f"{r['precision']:>10.4f} "
        f"{r['recall']:>8.4f} "
    )

```

```
f"{r['f1']:>8.4f}"
)
```

33.6 Production Operations: Monitoring, Eviction, and Invalidation

A semantic cache that nobody monitors slowly drifts into a state where stale responses are served confidently. Production operation requires three ongoing concerns: monitoring cache quality, evicting dead weight, and invalidating entries when source data changes.

Monitoring key metrics

The following metrics should feed into your observability stack (Prometheus, Datadog, or equivalent):

Metric	Description	Alert threshold
cache_hit_rate	Fraction of queries answered from cache	Alert if drops below baseline by >10pp
cache_miss_latency_p99	P99 latency of LLM calls on cache miss	Baseline $\times$ 2
cache_hit_latency_p99	P99 latency of cache lookups	>50ms warrants investigation
stale_entry_fraction	Entries past TTL still in index	>5% suggests purge job is lagging
false_positive_rate	Cached responses rated incorrect by users	>2% warrants threshold increase
embedding_error_rate	Embedding API failures	Any sustained failure

Tracking false positives with user feedback

The most dangerous failure mode of a semantic cache is serving a cached answer that is wrong for the new query. Track this with a lightweight thumbs-down signal:

```
# Python
# FastAPI endpoint pattern for collecting cache quality feedback
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import psycpg

app = FastAPI()

class FeedbackPayload(BaseModel):
    cache_entry_id: str
    query: str
    was_correct: bool
    user_comment: str = ""

@app.post("/feedback/cache")
async def record_cache_feedback(payload: FeedbackPayload):
    async with await psycpg.AsyncConnection.connect(DSN) as conn:
        async with conn.cursor() as cur:
            # Log the feedback
            await cur.execute(
                """
                INSERT INTO cache_feedback
```

```

        (cache_entry_id, query_text, was_correct, user_comment, created_at)
VALUES (%s, %s, %s, %s, NOW())
"""
    (
        payload.cache_entry_id,
        payload.query,
        payload.was_correct,
        payload.user_comment,
    ),
)
# If negative feedback, optionally flag the entry for review
if not payload.was_correct:
    await cur.execute(
        """
        UPDATE semantic_cache
        SET metadata = jsonb_set(
            COALESCE(metadata, '{}'),
            '{flagged}',
            'true'
        )
        WHERE id = %s::uuid
        """
        (payload.cache_entry_id,)
    )
    await conn.commit()
return {"status": "recorded"}

```

Event-driven invalidation

When a source document changes—a policy page is updated, a product spec is revised—cached responses derived from that document should be invalidated. The pattern mirrors how CDN edge caches are purged: a content update event triggers a cache invalidation call.

```

# Python
# Cache invalidation triggered by document update events
# Integrates with any event bus (Kafka, SQS, Redis Streams, etc.)

import asyncio
import json
from datetime import timezone, datetime

async def handle_document_updated_event(event: dict, cache: PgvectorSemanticCache):
    """
    Called when a source document is updated.
    Invalidates cached entries that reference that document.
    event = {
        "document_id": "doc_123",
        "namespace": "support",
        "updated_at": "2024-06-01T12:00:00Z",
    }
    """
    doc_id = event["document_id"]
    namespace = event.get("namespace", "default")

    conn = await cache._get_conn()
    async with conn.cursor() as cur:
        # Delete entries in this namespace that reference the document
        # (requires storing document_id in metadata at cache-write time)
        await cur.execute(
            """
            DELETE FROM semantic_cache
            WHERE namespace = %s
            AND metadata->>'source_document_id' = %s
            RETURNING id
            """
            (namespace, doc_id),
        )
    deleted_ids = await cur.fetchall()
    await conn.commit()

```

```

print(
    f"Invalidated {len(deleted_ids)} cache entries "
    f"for document {doc_id} in namespace {namespace}"
)
return [str(row[0]) for row in deleted_ids]

async def handle_namespace_flush_event(event: dict, cache: PgvectorSemanticCache):
    """
    Full namespace flush - used when a large batch of documents is updated,
    making entry-level invalidation impractical.
    """
    namespace = event["namespace"]
    conn = await cache._get_conn()
    async with conn.cursor() as cur:
        await cur.execute(
            "DELETE FROM semantic_cache WHERE namespace = %s RETURNING id",
            (namespace,),
        )
        count = len(await cur.fetchall())
        await conn.commit()
    print(f"Flushed {count} entries from namespace {namespace}")

```

Eviction strategies

Unlike memory-bounded caches like Redis, a Postgres-backed semantic cache grows without bound unless you actively evict entries. Three practical eviction strategies:

1. **TTL-based expiry:** Set `expires_at` at write time and run `purge_expired_cache()` as a scheduled job (pg_cron or an external scheduler). This is the minimum viable eviction strategy.
2. **LFU eviction on index size:** When the table exceeds a row count threshold, delete the N entries with the lowest `hit_count` that are also older than a minimum age. This keeps high-value entries and removes long-tail entries that never proved useful.
3. **Namespace-scoped budgets:** Assign a maximum row count per namespace. When a namespace exceeds its budget, evict the least recently used entries first.

```

-- PostgreSQL
-- LFU eviction: remove the bottom 10% of entries by hit count
-- when the table exceeds 500,000 rows

DO $$
DECLARE
    v_total BIGINT;
    v_evict_target BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_total FROM semantic_cache;

    IF v_total > 500000 THEN
        v_evict_target := v_total / 10; -- evict 10%

        DELETE FROM semantic_cache
        WHERE id IN (
            SELECT id FROM semantic_cache
            WHERE created_at < NOW() - INTERVAL '1 hour' -- protect very new entries
            ORDER BY hit_count ASC, last_accessed ASC
            LIMIT v_evict_target
        );

        RAISE NOTICE 'Evicted % entries (total was %)', v_evict_target, v_total;
    END IF;
END;
$$;

```

33.7 Semantic Caching for Agentic and Multi-Turn Workloads

Single-turn Q&A is the simplest semantic caching case. Agentic workloads—where an LLM reasons across multiple steps, invokes tools, and maintains conversation state—present two harder problems: what to cache and at what granularity.

Caching at the tool-call level

In a ReAct-style agent, the expensive operations are typically not the reasoning steps themselves but tool calls: web searches, database queries, API calls, sub-agent invocations. Caching at the individual tool-call level rather than the full agent trajectory captures most of the value without the staleness risk of caching an entire multi-step reasoning chain.

```
# Python
# Tool call semantic cache - wraps any tool function
# Works with LangChain tools, LlamaIndex tools, or custom tool implementations

import functools
import asyncio
from typing import Callable, Any

def semantic_cache_tool(
    cache: PgvectorSemanticCache,
    namespace_prefix: str = "tool",
):
    """
    Decorator that adds semantic caching to any async tool function.
    The tool's input is serialized to a string for embedding.
    """
    def decorator(tool_fn: Callable) -> Callable:
        tool_namespace = f"{namespace_prefix}:{tool_fn.__name__}"

        @functools.wraps(tool_fn)
        async def wrapper(*args, **kwargs) -> Any:
            # Serialize inputs to a canonical query string
            input_repr = f"{tool_fn.__name__}({args!r}, {kwargs!r})"
            # Use a copy of the cache pointed at the tool's namespace
            tool_cache = PgvectorSemanticCache(
                dsn=cache.dsn,
                openai_client=cache.oai,
                namespace=tool_namespace,
                threshold=cache.threshold,
                ttl_seconds=cache.ttl_seconds,
                embedding_model=cache.embedding_model,
            )

            cached = await tool_cache.get(input_repr)
            if cached:
                return cached["response"]

            result = await tool_fn(*args, **kwargs)
            await tool_cache.set(input_repr, str(result))
            return result

        return wrapper
    return decorator

# Usage with a database lookup tool
@semantic_cache_tool(cache=some_cache, namespace_prefix="tool")
async def lookup_customer_account(customer_id: str, fields: list[str]) -> dict:
    """Fetches customer account data from CRM API."""
    # ... actual CRM API call ...
```



```
return {"customer_id": customer_id, "status": "active"}
```

Multi-turn conversation caching

In multi-turn conversations, the relevant semantic unit is not a single utterance but the current question in context. A user asking “What about the return window?” as their third message has a different intent from the same string asked cold. Two strategies handle this:

1. **Context-conditioned embedding:** Concatenate the last N turns of conversation history with the current query before embedding. This captures context but reduces cache hit rate (each conversation history is unique).
2. **Semantic intent extraction:** Use a fast, cheap LLM call to extract the standalone question from the conversation context (“User is asking: what is the return window for electronics?”), then cache on the extracted standalone question. The extraction step costs a small-model token call but dramatically improves cache reuse.

```
# Python
async def extract_standalone_question(
    conversation_history: list[dict],
    current_query: str,
    oai_client,
) -> str:
    """
    Condenses a multi-turn conversation into a standalone question
    suitable for embedding and cache lookup.
    Uses a cheap, fast model for this step.
    """
    if not conversation_history:
        return current_query

    history_text = "\n".join(
        f"{m['role'].capitalize()}: {m['content']}"
        for m in conversation_history[-4:] # last 4 turns
    )

    prompt = f"""Given the following conversation, rewrite the final user question
    as a complete, standalone question that can be understood without the conversation context.
    Return only the rewritten question, nothing else.

    Conversation:
    {history_text}
    User: {current_query}

    Standalone question: """

    response = await oai_client.chat.completions.create(
        model="gpt-4o-mini", # fast, cheap model for extraction
        messages=[{"role": "user", "content": prompt}],
        max_tokens=100,
        temperature=0,
    )
    return response.choices[0].message.content.strip()

async def multi_turn_cached_response(
    conversation_history: list[dict],
    current_query: str,
    cache: PgvectorSemanticCache,
    llm_call_fn,
    oai_client,
) -> dict:
    standalone = await extract_standalone_question(
        conversation_history, current_query, oai_client
    )
    result = await cache.get_or_generate(standalone, llm_call_fn)
```

```
result["standalone_query"] = standalone
return result
```

33.8 Failure Modes and Mitigations

A semantic cache introduces failure modes that conventional caches do not have. Understanding them prevents subtle production bugs.

Embedding model version drift

If the embedding model used to populate the cache differs from the model used at query time—because the model was upgraded—all existing cache entries become unsearchable. Cosine distances computed across model versions are meaningless. Always store the embedding model name with each cache entry and route lookups to only compare against entries produced by the same model. During a model migration, run both models in parallel, rebuild the cache with the new model under a new namespace, and cut over atomically.

Threshold creep

Under pressure to improve hit rates, teams lower the threshold incrementally (“0.93 → 0.92 → 0.91...”) without re-running precision/recall analysis. By the time false positives appear in user feedback, the threshold has drifted far from the safe operating range. Treat threshold changes as configuration changes requiring offline evaluation before deployment.

Cold start

A freshly deployed semantic cache has no entries. Under production traffic, the first N queries are all cache misses, and the cache fills gradually as the LLM handles them. For applications with predictable query distributions, pre-warm the cache at deployment time with the top-K queries from historical logs.

```
# Python - cache pre-warming from historical query log
async def prewarm_cache(
    cache: PgvectorSemanticCache,
    historical_queries: list[tuple[str, str]], # (query, response) pairs
    model: str = "gpt-4o",
    batch_size: int = 50,
) -> int:
    """
    Pre-warm a semantic cache from a list of (query, response) pairs.
    Returns the number of entries written.
    """
    written = 0
    for i in range(0, len(historical_queries), batch_size):
        batch = historical_queries[i : i + batch_size]
        tasks = [
            cache.set(query, response, model=model)
            for query, response in batch
        ]
        await asyncio.gather(*tasks)
        written += len(batch)
    print(f"Pre-warmed {written}/{len(historical_queries)} entries")
    return written
```

Cascade failure: embedding API unavailability

When the embedding API is down, your semantic cache cannot compute embeddings and every request falls through to the LLM. Design the cache as a best-effort layer with a circuit breaker: if

embedding calls fail consecutively, bypass the cache entirely rather than letting latency stack up.

```
# Python
import time

class CircuitBreaker:
    """
    Simple circuit breaker for embedding API calls.
    Opens after consecutive failures,
    retries after reset_timeout_seconds.
    """
    def __init__(self, consecutive_failures: int = 5, reset_timeout_seconds: float = 60.0):
        self.threshold = consecutive_failures
        self.reset_timeout = reset_timeout_seconds
        self._failures = 0
        self._opened_at: Optional[float] = None

    @property
    def is_open(self) -> bool:
        if self._opened_at is None:
            return False
        if time.monotonic() - self._opened_at > self.reset_timeout:
            # Half-open: allow one probe
            return False
        return True

    def record_success(self):
        self._failures = 0
        self._opened_at = None

    def record_failure(self):
        self._failures += 1
        if self._failures >= self.threshold:
            self._opened_at = time.monotonic()

    async def call(self, fn, *args, **kwargs):
        if self.is_open:
            raise RuntimeError("Circuit breaker open - embedding API unavailable")
        try:
            result = await fn(*args, **kwargs)
            self.record_success()
            return result
        except Exception as exc:
            self.record_failure()
            raise
```

Key Takeaways

- **Semantic caching replaces exact-match hash lookups with vector similarity search**, allowing applications to serve cached LLM responses to queries that are phrased differently but mean the same thing—achieving latency reductions of two orders of magnitude on cache hits.
- **The similarity threshold is the single most operationally significant parameter**: too low and you serve wrong answers with confidence; too high and the cache never hits. Calibrate it with a labeled evaluation set and precision/recall analysis per query domain before deploying.
- **Tiered architectures—a fast vector index for similarity lookup combined with a key-value store for response bodies—deliver the best latency profile**, with total cache hit latency typically under 15ms compared to multi-second LLM API calls.
- **Production semantic caches require active lifecycle management**: TTL-based expiry,

LFU eviction when index size grows, event-driven invalidation when source documents change, and monitoring of false positive rates through user feedback signals.

- **Agentic and multi-turn workloads require adapted caching strategies:** cache at the tool-call level rather than the full agent trajectory, and use standalone-question extraction to normalize multi-turn context into cache-friendly canonical query forms before embedding.
-

Chapter 34: Access Control on Vector Data

Vector databases introduce a category of access control problems that traditional RBAC models weren't designed to solve. A row in a relational table is discrete and bounded — you can grant `SELECT` on a specific table, filter rows by tenant ID, and call it done. A vector embedding is a compressed semantic fingerprint of content that may span topics, contain implicit references to sensitive material, or be retrievable through queries that have no obvious surface-level connection to the underlying data. When a similarity search returns the top-k nearest neighbors, you cannot predict at query time which documents will surface. This chapter covers how to apply row-level security, namespace isolation, API key scoping, and query-time filtering to vector data across pgvector, Pinecone, Weaviate, and Qdrant — and how to build a unified authorization layer that treats vector search as a first-class security boundary.

Why Vector Access Control Is Different

In a traditional database, access control answers a simple question: does this user have permission to read this row? The data model is explicit. You can enumerate rows, apply predicates, and reason about what any given query will return before it executes.

Vector search inverts this relationship. You start with a query embedding and find what's similar. The result set is determined by geometric proximity in high-dimensional space, not by a filter predicate written at design time. A user querying for “quarterly financial projections” might pull a document fragment that was never explicitly tagged as sensitive but happens to be semantically close to their query vector. Conversely, strict keyword-based filtering might block a legitimate user from finding a document because the metadata doesn't match even though the content is relevant.

Three properties of vector data create novel access control challenges:

Semantic bleed. Embeddings from different security domains can cluster near each other if the underlying content is topically adjacent. A public FAQ about drug interactions and a restricted clinical trial protocol might embed within similar vector space because they share vocabulary. Namespace separation prevents this — you don't filter after the fact, you never allow the query to reach restricted vectors.

Inference risk. Even if a user cannot read a document directly, the fact that a similarity search returns zero results from a namespace they have permission to query can reveal that certain content exists. This is a vector analogue of a timing side-channel. Designing around it requires either

returning null results uniformly regardless of permission state or ensuring the user never knows a restricted namespace exists.

Metadata exposure. Most vector databases store metadata payloads alongside embeddings. The embedding itself may be meaningless to an attacker, but the metadata often contains document titles, author IDs, timestamps, and excerpts. Access control that secures the embedding but exposes the payload is incomplete.

The practical architecture for addressing these challenges combines three layers: storage-level isolation (namespaces and collections), query-time filtering (metadata predicates applied before returning results), and application-level enforcement (authorization checks before the vector query is even dispatched).

Row-Level Security in pgvector

PostgreSQL’s Row-Level Security (RLS) is the most mature access control mechanism available in the vector database ecosystem because pgvector is a PostgreSQL extension. Every security primitive PostgreSQL exposes — roles, policies, column-level grants, views — applies directly to vector data.

The baseline pattern stores a `tenant_id` or `user_id` alongside the embedding vector and attaches RLS policies that automatically filter queries to rows the current user owns.

```
-- PostgreSQL / pgvector

-- Create the documents table with tenant isolation
CREATE TABLE document_embeddings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   TEXT NOT NULL,
  content     TEXT NOT NULL,
  embedding   vector(1536),
  metadata    JSONB DEFAULT '{}',
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Index for ANN search
CREATE INDEX ON document_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- Index for tenant filtering (critical for performance)
CREATE INDEX ON document_embeddings (tenant_id);

-- Enable RLS
ALTER TABLE document_embeddings ENABLE ROW LEVEL SECURITY;

-- Policy: tenants can only read their own rows
CREATE POLICY tenant_isolation ON document_embeddings
FOR ALL
USING (tenant_id = current_setting('app.current_tenant', true));

-- Policy: service role bypasses RLS for admin operations
CREATE POLICY service_bypass ON document_embeddings
FOR ALL
TO service_role
USING (true);

-- Grant table access to application role (RLS handles row filtering)
GRANT SELECT, INSERT ON document_embeddings TO app_role;
```

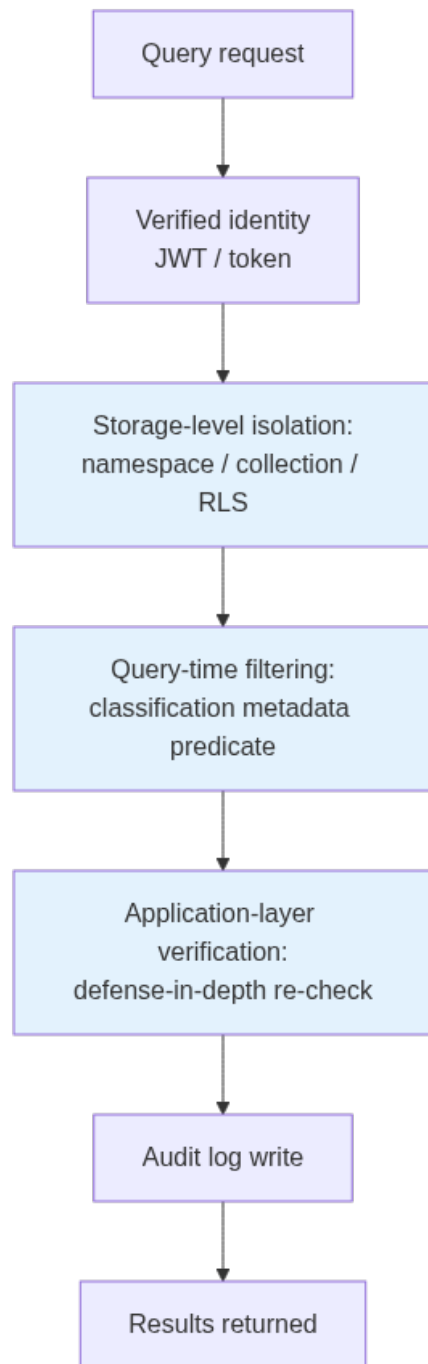


Figure 24: No single layer is sufficient alone — a misconfiguration in one is caught by the next.

At query time, the application sets the tenant context before issuing the vector search:

```
-- PostgreSQL / pgvector

-- Set tenant context for the current session
SET LOCAL app.current_tenant = 'tenant_acme_corp';

-- Similarity search - RLS automatically scopes to tenant_acme_corp rows
SELECT
    id,
    content,
    metadata,
    1 - (embedding <=> $1::vector) AS similarity_score
FROM document_embeddings
WHERE 1 - (embedding <=> $1::vector) > 0.75
ORDER BY embedding <=> $1::vector
LIMIT 10;

-- Verify the plan respects RLS (check for Filter on tenant_id)
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, content, 1 - (embedding <=> $1::vector) AS score
FROM document_embeddings
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

A frequent performance pitfall: when RLS adds a `WHERE tenant_id = X` predicate to an IVFFlat or HNSW scan, PostgreSQL must evaluate the policy after the ANN index returns candidates, not before. If your `lists = 100` index returns 200 candidates but only 3 belong to the current tenant, your effective recall is severely degraded and you’ve wasted I/O on irrelevant rows.

The solution is to use a composite index that includes the tenant ID as a partition key, or to maintain separate IVFFlat indexes per tenant using table partitioning:

```
-- PostgreSQL / pgvector

-- Partition by tenant for performance isolation
CREATE TABLE document_embeddings_partitioned (
    id          UUID NOT NULL DEFAULT gen_random_uuid(),
    tenant_id   TEXT NOT NULL,
    content     TEXT NOT NULL,
    embedding   vector(1536),
    metadata    JSONB DEFAULT '{} '
) PARTITION BY LIST (tenant_id);

-- Create per-tenant partitions
CREATE TABLE doc_embeddings_acme
    PARTITION OF document_embeddings_partitioned
    FOR VALUES IN ('tenant_acme_corp');

CREATE TABLE doc_embeddings_globex
    PARTITION OF document_embeddings_partitioned
    FOR VALUES IN ('tenant_globex');

-- Per-partition IVFFlat indexes
CREATE INDEX ON doc_embeddings_acme
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 50);

CREATE INDEX ON doc_embeddings_globex
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 50);

-- RLS still applies, but the planner now prunes to a single partition
ALTER TABLE document_embeddings_partitioned ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_partitioned ON document_embeddings_partitioned
    FOR ALL
    USING (tenant_id = current_setting('app.current_tenant', true));
```

For Supabase deployments, which run managed PostgreSQL with pgvector, this RLS pattern works

identically. Supabase's auth system automatically populates `auth.uid()` which you can use as the tenant identifier directly in policies:

```
-- PostgreSQL / pgvector (Supabase)

-- Supabase-specific policy using auth context
CREATE POLICY user_owns_documents ON document_embeddings
FOR SELECT
USING (user_id = auth.uid());

CREATE POLICY user_inserts_own ON document_embeddings
FOR INSERT
WITH CHECK (user_id = auth.uid());
```

Namespace Isolation in Pinecone and Weaviate

Managed vector databases handle multi-tenancy through namespace and collection isolation rather than SQL-style policies. The security model is coarser — you're working at the API key and namespace level rather than per-row policies — but the operational simplicity trades off well for most SaaS applications.

Pinecone namespaces partition vectors within an index so that operations on one namespace never touch vectors in another. All three operations — upsert, query, and delete — accept a namespace parameter. Vectors without a namespace land in the default namespace, which is a common source of tenant data leakage during development.

```
# Python
import os
from pinecone import Pinecone, ServerlessSpec
from openai import OpenAI

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# Create index if it doesn't exist
index_name = "document-store"
if index_name not in pc.list_indexes().names():
    pc.create_index(
        name=index_name,
        dimension=1536,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

index = pc.Index(index_name)

def embed_text(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

def upsert_for_tenant(tenant_id: str, documents: list[dict]) -> None:
    """
    All writes are scoped to the tenant's namespace.
    Cross-namespace writes require separate API key issuance at the
    infrastructure level - namespaces themselves are not credential-bound.
    """
    vectors = []
    for doc in documents:
        embedding = embed_text(doc["content"])
```

```

        vectors.append({
            "id": doc["id"],
            "values": embedding,
            "metadata": {
                "content": doc["content"],
                "source": doc.get("source", ""),
                "tenant_id": tenant_id,    # redundant but useful for audit
                "classification": doc.get("classification", "internal")
            }
        })

    # namespace= is the hard isolation boundary
    index.upsert(vectors=vectors, namespace=tenant_id)

def query_for_tenant(
    tenant_id: str,
    query_text: str,
    top_k: int = 10,
    classification_filter: str | None = None
) -> list[dict]:
    """
    Query is restricted to tenant namespace.
    Optional metadata filter further restricts by classification.
    """
    query_embedding = embed_text(query_text)

    # Build metadata filter if classification scope is provided
    filter_dict = None
    if classification_filter:
        filter_dict = {"classification": {"$eq": classification_filter}}

    results = index.query(
        vector=query_embedding,
        top_k=top_k,
        namespace=tenant_id,          # hard namespace isolation
        filter=filter_dict,           # optional metadata predicate
        include_metadata=True
    )

    return [
        {
            "id": match.id,
            "score": match.score,
            "content": match.metadata.get("content", ""),
            "classification": match.metadata.get("classification", "")
        }
        for match in results.matches
    ]

# Usage
upsert_for_tenant("acme_corp", [
    {"id": "doc-001", "content": "Q3 revenue targets", "classification": "confidential"},
    {"id": "doc-002", "content": "Employee handbook", "classification": "internal"}
])

# This query cannot reach globex_corp documents - namespace enforces that
results = query_for_tenant("acme_corp", "financial targets", classification_filter="confidential")

```

Pinecone’s namespace model has a significant architectural constraint: there is no native per-namespace API key. A single API key has access to all namespaces in the index. This means namespace isolation is enforced entirely by your application layer — if your query function is called with the wrong tenant ID, you’ll get cross-tenant data. The architecture that addresses this is a server-side authorization proxy that validates the caller’s identity before injecting the `namespace` parameter:

```

# Python
# Authorization proxy pattern - never trust client-supplied tenant_id

```

```

import jwt
from functools import wraps
from fastapi import FastAPI, HTTPException, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

app = FastAPI()
security = HTTPBearer()

SECRET_KEY = os.environ["JWT_SECRET"]

def get_verified_tenant(
    credentials: HTTPAuthorizationCredentials = Depends(security)
) -> str:
    """
    Validate JWT and extract tenant claim.
    The tenant_id used in Pinecone namespace queries comes ONLY
    from the verified token, never from the request body.
    """
    try:
        payload = jwt.decode(
            credentials.credentials,
            SECRET_KEY,
            algorithms=["HS256"]
        )
        tenant_id = payload.get("tenant_id")
        if not tenant_id:
            raise HTTPException(status_code=403, detail="No tenant claim in token")
        return tenant_id
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expired")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

@app.post("/search")
async def search_documents(
    query: str,
    top_k: int = 10,
    tenant_id: str = Depends(get_verified_tenant) # extracted from JWT, not request
):
    results = query_for_tenant(tenant_id, query, top_k=top_k)
    return {"results": results, "tenant": tenant_id}

```

Weaviate organizes data into classes (schema types) and supports multi-tenancy as a first-class feature. In modern Weaviate versions, tenant isolation can be enabled at the class level, after which each tenant's data is stored in separate segments and queries must specify a tenant key:

```

# Python
import weaviate
from weaviate.classes.config import Configure, Property, DataType
from weaviate.classes tenants import Tenant

client = weaviate.connect_to_cloud(
    cluster_url=os.environ["WEAVIATE_URL"],
    auth_credentials=weaviate.auth.AuthApiKey(os.environ["WEAVIATE_API_KEY"])
)

# Create a multi-tenant collection
documents = client.collections.create(
    name="Document",
    multi_tenancy_config=Configure.multi_tenancy(enabled=True),
    properties=[
        Property(name="content", data_type=DataType.TEXT),
        Property(name="source", data_type=DataType.TEXT),
        Property(name="classification", data_type=DataType.TEXT),
    ],
    vectorizer_config=Configure.Vectorizer.text2vec_openai(
        model="text-embedding-3-small"
    )
)

```

```
# Register tenants explicitly
documents.tenants.create([
    Tenant(name="acme_corp"),
    Tenant(name="globex"),
])

# Insert objects scoped to a tenant
acme_collection = client.collections.get("Document").with_tenant("acme_corp")
acme_collection.data.insert({
    "content": "Annual security audit findings for ACME infrastructure",
    "source": "security-team",
    "classification": "restricted"
})

# Query scoped to a tenant - physically isolated from other tenants
def search_weaviate(tenant_id: str, query: str, top_k: int = 5) -> list[dict]:
    collection = client.collections.get("Document").with_tenant(tenant_id)

    results = collection.query.near_text(
        query=query,
        limit=top_k,
        filters=weaviate.classes.query.Filter.by_property("classification").equal("restricted"),
        return_properties=["content", "source", "classification"]
    )

    return [
        {
            "content": obj.properties["content"],
            "classification": obj.properties["classification"],
            "distance": obj.metadata.distance if obj.metadata else None
        }
        for obj in results.objects
    ]

results = search_weaviate("acme_corp", "security vulnerabilities")
client.close()
```

Weaviate's multi-tenancy implementation physically separates tenant data into different segments on disk, which means a misconfigured query that omits the tenant key raises an error rather than silently returning cross-tenant data. This fail-closed behavior is architecturally superior to Pinecone's namespace model for high-assurance environments.

Metadata Filtering as a Security Predicate

Namespace and collection isolation enforces coarse-grained boundaries between tenants. Within a tenant's namespace, you often need finer-grained access control: a user in `acme_corp` should only see documents classified at or below their clearance level, or only documents in projects they're assigned to.

Metadata filters serve this purpose, but they carry an important caveat: **metadata filters are not a substitute for namespace isolation**. They operate after the ANN index has already identified candidate vectors. If a metadata filter is the only barrier between a user and restricted data, a misconfigured filter or a library bug that drops the filter predicate is a security incident.

The correct architecture stacks both controls:

1. Namespace/collection isolation prevents cross-tenant data access unconditionally.
2. Metadata filters enforce within-tenant access control.

```

# Python
# Qdrant with payload filtering for within-tenant access control
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    Filter, FieldCondition, MatchValue, MatchAny,
    models
)

client = QdrantClient(
    url=os.environ["QDRANT_URL"],
    api_key=os.environ["QDRANT_API_KEY"]
)

# Create collection
client.recreate_collection(
    collection_name="documents_acme_corp", # collection per tenant
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
)

def build_access_filter(user_roles: list[str], clearance_level: int) -> Filter:
    """
    Build a Qdrant filter from user's authorization attributes.
    This filter is constructed server-side from the verified identity -
    the calling user never constructs this directly.
    """
    # Map clearance levels: 1=public, 2=internal, 3=confidential, 4=restricted
    allowed_classifications = []
    level_map = {1: "public", 2: "internal", 3: "confidential", 4: "restricted"}
    for level in range(1, clearance_level + 1):
        if level in level_map:
            allowed_classifications.append(level_map[level])

    conditions = [
        FieldCondition(
            key="classification",
            match=MatchAny(any=allowed_classifications)
        )
    ]

    # Role-based project access
    if "engineering" in user_roles:
        conditions.append(
            FieldCondition(key="department", match=MatchAny(any=["engineering", "all"]))
        )
    elif "finance" in user_roles:
        conditions.append(
            FieldCondition(key="department", match=MatchAny(any=["finance", "all"]))
        )

    return Filter(must=conditions)

def authorized_search(
    collection_name: str,
    query_vector: list[float],
    user_roles: list[str],
    clearance_level: int,
    top_k: int = 10
) -> list[dict]:
    access_filter = build_access_filter(user_roles, clearance_level)

    results = client.search(
        collection_name=collection_name,
        query_vector=query_vector,
        query_filter=access_filter,
        limit=top_k,
        with_payload=True
    )

```

```

# Secondary enforcement: verify returned results match expected criteria
# Defense in depth - catch any filter bypass at application layer
verified_results = []
for result in results:
    payload = result.payload or {}
    doc_classification = payload.get("classification", "restricted")

    if doc_classification not in [
        "public", "internal", "confidential", "restricted"
    ][:clearance_level]:
        # Log this as a potential security anomaly and skip
        print(f"SECURITY: Unexpected classification {doc_classification} for user clearance
              ↳ {clearance_level}")
        continue

    verified_results.append({
        "id": result.id,
        "score": result.score,
        "content": payload.get("content", ""),
        "classification": doc_classification
    })

return verified_results

```

The double-verification pattern — filter at the database level AND check again at the application layer — is defense in depth. It adds a small amount of application-side overhead but ensures that a misconfigured index, a library version change that silently drops filter syntax, or an API contract change doesn't result in unauthorized data exposure.

API Key Management and Least-Privilege Design

Every managed vector database issues API keys that carry full read/write/delete privileges by default. For production systems serving multiple users or integration points, this is architecturally equivalent to sharing a database superuser password. The correct approach is to issue the minimum credential required for each integration point.

Pinecone distinguishes between Admin keys (can create/delete indexes) and project-scoped keys (read/write to indexes within a project). Weaviate supports API key authentication with role assignments. Qdrant Cloud has JWT-based API keys with optional scope restrictions.

For systems where the vector database doesn't offer per-namespace API key scoping, the standard pattern is a service mesh or API gateway that:

1. Authenticates the calling service or user with your identity provider
2. Maps the identity to permitted namespaces and operations
3. Injects the appropriate vector DB API key into the upstream request
4. Logs the operation for audit

```

# Python
# Centralized credential and namespace management
import boto3
import json
from dataclasses import dataclass
from enum import Enum

class VectorOperation(Enum):
    READ = "read"
    WRITE = "write"
    DELETE = "delete"

```

```

ADMIN = "admin"

@dataclass
class VectorAccess:
    tenant_namespace: str
    permitted_operations: list[VectorOperation]
    classification_ceiling: str # maximum classification this credential can access

class VectorAccessController:
    """
    Mediates all vector database access.
    API keys are stored in AWS Secrets Manager.
    Access decisions are driven by IAM role of the calling service.
    """

    def __init__(self):
        self.secrets_client = boto3.client("secretsmanager", region_name="us-east-1")
        self._key_cache = {}

    def _get_pinecone_key(self, key_type: str = "app") -> str:
        """Retrieve Pinecone API key from Secrets Manager."""
        secret_name = f"vector-db/pinecone/{key_type}"
        if secret_name not in self._key_cache:
            response = self.secrets_client.get_secret_value(SecretId=secret_name)
            secret = json.loads(response["SecretString"])
            self._key_cache[secret_name] = secret["api_key"]
        return self._key_cache[secret_name]

    def get_access_context(self, jwt_payload: dict) -> VectorAccess:
        """
        Map a verified JWT to a VectorAccess context.
        The JWT comes from your IdP (Okta, Auth0, Cognito, etc.)
        """
        tenant_id = jwt_payload["tenant_id"]
        user_roles = jwt_payload.get("roles", [])

        # Determine permitted operations from roles
        ops = [VectorOperation.READ]
        if "data-engineer" in user_roles or "service-account" in user_roles:
            ops.append(VectorOperation.WRITE)
        if "data-admin" in user_roles:
            ops.append(VectorOperation.DELETE)
        if "vector-admin" in user_roles:
            ops.append(VectorOperation.ADMIN)

        # Determine classification ceiling from clearance attribute
        clearance = jwt_payload.get("clearance_level", 1)
        ceiling_map = {1: "public", 2: "internal", 3: "confidential", 4: "restricted"}
        classification_ceiling = ceiling_map.get(clearance, "public")

        return VectorAccess(
            tenant_namespace=tenant_id,
            permitted_operations=ops,
            classification_ceiling=classification_ceiling
        )

    def can_perform(self, access: VectorAccess, operation: VectorOperation) -> bool:
        return operation in access.permitted_operations

    def execute_query(
        self,
        access: VectorAccess,
        query_vector: list[float],
        top_k: int = 10
    ) -> list[dict]:
        """
        Execute a read operation if the access context permits it.
        The Pinecone API key is retrieved from Secrets Manager,
        never passed through the calling code.
        """

```

```

if not self.can_perform(access, VectorOperation.READ):
    raise PermissionError(f"Tenant {access.tenant_namespace} lacks READ permission")

api_key = self._get_pinecone_key(key_type="app")
pc = Pinecone(api_key=api_key)
index = pc.Index("document-store")

# Classification filter derived from access context, not caller input
classification_filter = self._classification_filter(access.classification_ceiling)

results = index.query(
    vector=query_vector,
    top_k=top_k,
    namespace=access.tenant_namespace,
    filter=classification_filter,
    include_metadata=True
)

return [
    {
        "id": match.id,
        "score": match.score,
        "content": match.metadata.get("content", ""),
        "classification": match.metadata.get("classification", "")
    }
    for match in results.matches
]

def _classification_filter(self, ceiling: str) -> dict:
    allowed = {
        "public": ["public"],
        "internal": ["public", "internal"],
        "confidential": ["public", "internal", "confidential"],
        "restricted": ["public", "internal", "confidential", "restricted"]
    }
    return {"classification": {"$in": allowed.get(ceiling, ["public"])}}
```

Audit Logging for Vector Queries

Access control without audit logging is incomplete. In regulated environments — HIPAA, SOC 2, FedRAMP — you must be able to answer: who queried what, when, and what did they receive? For vector databases this is non-trivial because the query is a floating-point vector and the result is a ranked list, neither of which is easily stored in a traditional audit log.

A practical audit schema stores the semantic query (the text before embedding), the resolved tenant and access context, and enough result metadata to reconstruct what was exposed without storing the full document contents:

```

-- PostgreSQL audit log table
CREATE TABLE vector_query_audit (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    occurred_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    tenant_id         TEXT NOT NULL,
    user_id           TEXT NOT NULL,
    query_text        TEXT,                -- the original natural language query
    query_hash        TEXT,                -- SHA-256 of the embedding for deduplication
    top_k             INTEGER,
    namespace         TEXT,
    classification_ceiling TEXT,
    result_count      INTEGER,
    result_ids        TEXT[],              -- IDs of returned documents
    result_scores     FLOAT[],             -- similarity scores
    request_ip        INET,
```



```

        user_agent      TEXT,
        duration_ms     INTEGER
    );

-- Index for compliance queries
CREATE INDEX ON vector_query_audit (tenant_id, occurred_at DESC);
CREATE INDEX ON vector_query_audit (user_id, occurred_at DESC);

-- Retention policy: audit rows are append-only
-- Use pg_partman or a cron job to archive/drop old partitions
ALTER TABLE vector_query_audit ENABLE ROW LEVEL SECURITY;

-- Only audit readers (compliance role) and service accounts can read audit rows
CREATE POLICY audit_read ON vector_query_audit
    FOR SELECT
    TO compliance_role, service_role
    USING (true);

-- Application role can only insert, never update or delete
CREATE POLICY audit_insert ON vector_query_audit
    FOR INSERT
    TO app_role
    WITH CHECK (true);

# Python
# Audit logging wrapper around vector search
import hashlib
import time
import psycopg2
from contextlib import contextmanager

class AuditedVectorSearch:
    def __init__(self, vector_client, audit_db_conn_string: str):
        self.vector_client = vector_client
        self.audit_conn_string = audit_db_conn_string

    def search(
        self,
        tenant_id: str,
        user_id: str,
        query_text: str,
        query_vector: list[float],
        namespace: str,
        classification_ceiling: str,
        top_k: int = 10,
        request_ip: str = None,
        user_agent: str = None
    ) -> list[dict]:
        start = time.monotonic()

        # Compute query fingerprint for deduplication analysis
        query_hash = hashlib.sha256(
            bytes(str(query_vector), "utf-8")
        ).hexdigest()

        # Execute vector search
        results = self.vector_client.execute_query(
            access=VectorAccess(
                tenant_namespace=tenant_id,
                permitted_operations=[VectorOperation.READ],
                classification_ceiling=classification_ceiling
            ),
            query_vector=query_vector,
            top_k=top_k
        )

        duration_ms = int((time.monotonic() - start) * 1000)

        # Write audit record - this happens regardless of result content
        self._write_audit(

```

```

        tenant_id=tenant_id,
        user_id=user_id,
        query_text=query_text,
        query_hash=query_hash,
        top_k=top_k,
        namespace=namespace,
        classification_ceiling=classification_ceiling,
        result_count=len(results),
        result_ids=[r["id"] for r in results],
        result_scores=[r["score"] for r in results],
        request_ip=request_ip,
        user_agent=user_agent,
        duration_ms=duration_ms
    )

    return results

def _write_audit(self, **kwargs) -> None:
    with psycopg2.connect(self.audit_conn_string) as conn:
        with conn.cursor() as cur:
            cur.execute("""
                INSERT INTO vector_query_audit (
                    tenant_id, user_id, query_text, query_hash,
                    top_k, namespace, classification_ceiling,
                    result_count, result_ids, result_scores,
                    request_ip, user_agent, duration_ms
                ) VALUES (
                    %(tenant_id)s, %(user_id)s, %(query_text)s, %(query_hash)s,
                    %(top_k)s, %(namespace)s, %(classification_ceiling)s,
                    %(result_count)s, %(result_ids)s, %(result_scores)s,
                    %(request_ip)s, %(user_agent)s, %(duration_ms)s
                )
            """, kwargs)
        conn.commit()

```

One important design decision: whether to log the `query_text` (the natural language query before embedding) depends on your data classification policy. In some environments, the query itself — “show me documents about patient discharge against medical advice” — is as sensitive as the returned content. In those cases, log only the hash, and ensure the hash-to-text mapping is stored in a separate, higher-classification system accessible only to the compliance team.

Handling Sensitive Data in Embedding Pipelines

Access control at the vector database level is the last line of defense. The embedding pipeline — where raw text is converted to vectors — is an earlier and often overlooked attack surface. If your pipeline sends data to a third-party embedding API (OpenAI, Cohere, Google), that data leaves your security boundary.

The mitigations depend on your threat model:

Private embedding models eliminate the third-party exposure entirely. Running an open-source model like `all-MiniLM-L6-v2` from `sentence-transformers` on infrastructure you control means the raw text never leaves your network. The embedding quality may differ from frontier models, but for most enterprise retrieval tasks the gap is acceptable.

```

# Python
# On-premises embedding with sentence-transformers
from sentence_transformers import SentenceTransformer
import torch

```

```

class PrivateEmbedder:
    """
    Embed text using a locally-hosted model.
    No data leaves the network boundary.
    """
    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model = SentenceTransformer(model_name, device=device)
        self.dimension = self.model.get_sentence_embedding_dimension()

    def embed(self, texts: list[str]) -> list[list[float]]:
        embeddings = self.model.encode(
            texts,
            batch_size=64,
            normalize_embeddings=True, # unit vectors for cosine similarity
            show_progress_bar=False
        )
        return embeddings.tolist()

    def embed_one(self, text: str) -> list[float]:
        return self.embed([text])[0]

embedder = PrivateEmbedder()

```

Data masking before embedding applies when you must use a third-party API but some fields in your documents are sensitive PII or proprietary content. Before sending text to the embedding API, strip or replace the sensitive spans with placeholder tokens:

```

# Python
import re

class DataMaskingPreprocessor:
    """
    Remove or replace sensitive patterns before embedding.
    Masking is irreversible - the stored vector reflects the masked text,
    so exact-match retrieval on original PII is not possible.
    Use this only when semantic retrieval on non-PII context is sufficient.
    """

    PATTERNS = {
        "ssn": (r"\b\d{3}-\d{2}-\d{4}\b", "[SSN]"),
        "credit_card": (r"\b(?:\d{4}[- ]?)\d{3}\d{4}\b", "[CARD]"),
        "email": (r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b", "[EMAIL]"),
        "phone": (r"\b(?:\+1[-.\s]?|(?\d{3})?[-.\s]?)\d{3}[-.\s]?[0-9]{4}\b", "[PHONE]"),
    }

    def mask(self, text: str) -> tuple[str, dict]:
        """
        Returns masked text and a record of replacements.
        The replacement record should be stored in your secure vault,
        not alongside the vector.
        """
        replacements = {}
        masked = text

        for pattern_name, (pattern, placeholder) in self.PATTERNS.items():
            matches = re.findall(pattern, masked)
            if matches:
                replacements[pattern_name] = matches
                masked = re.sub(pattern, placeholder, masked)

        return masked, replacements

preprocessor = DataMaskingPreprocessor()
masked_text, pii_record = preprocessor.mask(
    "Contact John at john.doe@acme.com or 555-123-4567 about invoice #A-4492"
)
# masked_text: "Contact John at [EMAIL] or [PHONE] about invoice #A-4492"
# pii_record stored separately in your secrets vault

```

VPC endpoints and private links are the network-level complement to data masking. AWS PrivateLink for Bedrock, Azure Private Endpoints for Azure OpenAI, and GCP Private Service Connect for Vertex AI all allow your embedding calls to traverse private network paths rather than the public internet, reducing the exposure surface even when using managed embedding APIs.

A Unified Authorization Layer

Production systems rarely use a single vector database. A typical architecture might use pgvector for low-latency operational queries, Pinecone for high-scale semantic search, and Weaviate for knowledge graph traversal with vector similarity. Each database has its own access control model. Building authorization logic independently for each creates inconsistency and maintenance burden.

A unified authorization layer abstracts the vector database behind a policy engine that speaks a single language regardless of the underlying store:

```
# Python
from abc import ABC, abstractmethod
from typing import Protocol
import opa_client  # Open Policy Agent Python client

class VectorSearchBackend(Protocol):
    def search(
        self,
        collection: str,
        vector: list[float],
        filter_: dict,
        top_k: int
    ) -> list[dict]: ...

class PineconeBackend:
    def __init__(self, api_key: str, index_name: str):
        self.pc = Pinecone(api_key=api_key)
        self.index = self.pc.Index(index_name)

    def search(self, collection: str, vector: list[float], filter_: dict, top_k: int) -> list[dict]:
        results = self.index.query(
            vector=vector,
            top_k=top_k,
            namespace=collection,
            filter=filter_,
            include_metadata=True
        )
        return [{"id": m.id, "score": m.score, **m.metadata} for m in results.matches]

class QdrantBackend:
    def __init__(self, url: str, api_key: str):
        self.client = QdrantClient(url=url, api_key=api_key)

    def search(self, collection: str, vector: list[float], filter_: dict, top_k: int) -> list[dict]:
        # Convert unified filter dict to Qdrant filter model
        qdrant_filter = self._translate_filter(filter_)
        results = self.client.search(
            collection_name=collection,
            query_vector=vector,
            query_filter=qdrant_filter,
            limit=top_k,
            with_payload=True
        )
        return [{"id": str(r.id), "score": r.score, **(r.payload or {})} for r in results]

    def _translate_filter(self, filter_dict: dict) -> Filter | None:
        if not filter_dict:
```

```

        return None
    conditions = []
    for key, value in filter_dict.items():
        if isinstance(value, list):
            conditions.append(FieldCondition(key=key, match=MatchAny(any=value)))
        else:
            conditions.append(FieldCondition(key=key, match=MatchValue(value=value)))
    return Filter(must=conditions)

class UnifiedVectorAuthority:
    """
    Single entry point for all vector search operations.
    Policy decisions are delegated to Open Policy Agent.
    Backend selection is driven by routing configuration.
    """

    def __init__(self, opa_url: str, backends: dict[str, VectorSearchBackend]):
        self.opa = opa_client.OpaClient(opa_url)
        self.backends = backends # e.g., {"pinecone": PineconeBackend(...), "qdrant":
        ↪ QdrantBackend(...)}

    def _evaluate_policy(self, identity: dict, resource: dict, action: str) -> dict:
        """
        Query OPA for access decision.
        Returns {"allow": bool, "filter": dict, "redact_fields": list[str]}
        """
        input_data = {
            "identity": identity,
            "resource": resource,
            "action": action
        }
        result = self.opa.check_policy_rule(
            policy_name="vector/authz",
            rule_name="decision",
            input_data=input_data
        )
        return result.get("result", {"allow": False, "filter": {}, "redact_fields": []})

    def search(
        self,
        identity: dict,          # {tenant_id, user_id, roles, clearance_level}
        backend_name: str,       # "pinecone" | "qdrant" | "pgvector"
        collection: str,
        query_vector: list[float],
        top_k: int = 10
    ) -> list[dict]:
        # Step 1: Policy evaluation
        decision = self._evaluate_policy(
            identity=identity,
            resource={"backend": backend_name, "collection": collection},
            action="vector:search"
        )

        if not decision["allow"]:
            raise PermissionError(
                f"User {identity['user_id']} is not permitted to search {collection}"
            )

        # Step 2: Backend selection and query execution
        backend = self.backends.get(backend_name)
        if not backend:
            raise ValueError(f"Unknown backend: {backend_name}")

        # filter_ comes from OPA policy, never from caller
        results = backend.search(
            collection=collection,
            vector=query_vector,
            filter_=decision.get("filter", {}),
            top_k=top_k

```

```

    )

    # Step 3: Field-level redaction based on policy output
    redact_fields = decision.get("redact_fields", [])
    if redact_fields:
        results = [
            {k: ("[REDACTED]" if k in redact_fields else v) for k, v in r.items()}
            for r in results
        ]

    return results

```

The OPA policy file that governs these decisions is written in Rego and lives in your policy repository, version-controlled alongside your application code:

```

# rego
# policy: vector/authz.rego

package vector.authz

import future.keywords.if
import future.keywords.in

# Default deny
default decision = {"allow": false, "filter": {}, "redact_fields": []}

decision = result if {
    is_allowed
    result := {
        "allow": true,
        "filter": access_filter,
        "redact_fields": fields_to_redact
    }
}

# Allow if tenant matches collection prefix and user has read role
is_allowed if {
    startswith(input.resource.collection, input.identity.tenant_id)
    "read" in input.identity.roles
    input.action == "vector:search"
}

# Build the metadata filter from clearance level
access_filter := filter if {
    level := input.identity.clearance_level
    allowed := classification_for_level[level]
    filter := {"classification": allowed}
}

classification_for_level := {
    1: ["public"],
    2: ["public", "internal"],
    3: ["public", "internal", "confidential"],
    4: ["public", "internal", "confidential", "restricted"]
}

# Redact PII fields for users without the data-steward role
fields_to_redact := fields if {
    not "data-steward" in input.identity.roles
    fields := ["author_email", "author_phone", "employee_id"]
} else := []

```

This architecture provides a clean separation of concerns: the vector database handles geometry, the policy engine handles authorization, and the application handles business logic. Rotating a credential, tightening a classification rule, or adding a new backend requires changes in exactly one place.

Key Takeaways

- **Vector access control requires layered defenses.** Namespace or collection isolation prevents cross-tenant data access at the storage level; metadata filters enforce within-tenant classification controls; application-layer verification catches filter bypass scenarios. No single layer is sufficient on its own.
- **PostgreSQL RLS integrates directly with pgvector**, but ANN index performance degrades when RLS predicates are applied post-scan. Partition by tenant to ensure the query planner prunes to a single partition before scanning the vector index.
- **Managed vector databases (Pinecone, Weaviate, Qdrant) have different isolation guarantees.** Weaviate’s multi-tenancy model is fail-closed — queries without a tenant key raise an error. Pinecone’s namespace model is fail-open — omitting the namespace silently queries the default namespace. Design your application wrapper to enforce namespace injection server-side from a verified identity token, never from client-supplied input.
- **API keys should follow least-privilege principles.** Store keys in a secrets manager, scope them to the minimum required operations, and retrieve them at runtime through an authorization proxy rather than distributing them to calling services or clients directly.
- **Audit logs for vector queries must capture the semantic intent, not just the raw vector.** Log the natural language query text (or its hash if the query itself is sensitive), the resolved access context, the result document IDs, and the similarity scores. This record supports compliance investigations, anomaly detection on unusual query patterns, and incident response when a potential data exposure needs to be scoped.

Chapter 35: Disaster Recovery for Vector Stores

Vector databases are no longer experimental infrastructure. They sit directly in the critical path of production AI applications — powering search, recommendation engines, RAG pipelines, and real-time similarity lookups. When a vector store fails, the LLM application behind it often fails silently or degrades in ways that are harder to detect than a 500 error: results become stale, semantically incorrect, or simply absent. This chapter covers disaster recovery for vector stores from first principles through production-grade implementation, treating vector data with the same operational rigor that experienced DBAs apply to transactional systems. You will work through backup strategies, recovery time objectives, replication topologies, and the specific failure modes that make vector DR harder than traditional RDBMS recovery — including the cost and latency of re-embedding large corpora when source documents are your only remaining asset.

The Unique DR Challenges of Vector Stores

Before designing recovery procedures, you need to understand what makes vector stores different from the relational databases you already know how to protect.

Embeddings are derived data — but regenerating them is expensive

In a relational database, if you lose a computed column, you run a `UPDATE` statement and move on. Embeddings occupy a similar logical position — they are derived from source text or images — but regeneration is orders of magnitude more expensive. Re-embedding a 10-million-document corpus using OpenAI’s `text-embedding-3-large` at roughly \$0.13 per million tokens (as of 2024 pricing), with an average document size of 512 tokens, costs approximately \$650 and takes several hours even with full parallelism. For billion-vector deployments, the cost and time become recovery-blocking. This means vector data must be treated as primary data, not recomputable cache, in your backup strategy.

Indexes are the data

In PostgreSQL, you can drop and rebuild an index without losing data — the heap is your source of truth. In most vector databases, the index *is* the primary data structure. An HNSW index in Weaviate or Qdrant contains the vectors themselves along with their navigational graph. Losing the index means losing everything. This shifts backup philosophy: you cannot separate data backup from index backup.

Consistency between vector stores and source systems

Vector stores rarely stand alone. They contain embeddings derived from records in PostgreSQL, MongoDB, or S3 object stores. A complete DR plan must account for the synchronization point between source data and the vector index. If your PostgreSQL database is restored to a point-in-time 6 hours before your vector store backup, you now have semantic drift between your relational records and their embeddings.

Metadata amplifies the problem

Modern vector stores store payload metadata alongside vectors — document IDs, timestamps, category tags, user IDs. This metadata often has relational consistency requirements with upstream systems. Recovery must restore vectors and metadata to a mutually consistent state.

Table 35.1 maps traditional DR concepts to vector store equivalents:

Traditional DB Concept	Vector Store Equivalent	Key Difference
Transaction log backup	Incremental vector export	Vectors have no native WAL
Point-in-time recovery	Snapshot + re-sync from source	Requires source system alignment
Logical backup (pg_dump)	JSON/binary vector export	May be 100x larger per record
Physical backup	Volume snapshot	Index state may be partially flushed
Read replica	Query replica / shard mirror	Consistency lag measured differently
Restore test	Similarity query validation	Correctness, not just availability

Backup Strategies by Vector Store Platform

Each major vector database has different native backup capabilities. The right strategy depends on your RTO/RPO requirements, deployment model, and whether you control the underlying infrastructure.

pgvector (PostgreSQL)

pgvector stores vectors in standard PostgreSQL heap tuples. This means all of PostgreSQL's backup mechanisms work without modification: `pg_dump`, `pg_basebackup`, WAL archiving, and point-in-time recovery through your preferred tool (Barman, pgBackRest, WAL-G). The HNSW and IVFFlat indexes are rebuilt on restore from the base table data — but at significant cost for large tables.

```
-- PostgreSQL / pgvector
-- Step 1: Check the size of your vector table and index before planning backup frequency
SELECT
  schemaname,
  tablename,
  pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total_size,
  pg_size_pretty(pg_relation_size(schemaname||'.'||tablename)) AS table_size,
  pg_size_pretty(
```

```

        pg_total_relation_size(schemaname||'.'||tablename) -
        pg_relation_size(schemaname||'.'||tablename)
    ) AS index_size
FROM pg_tables
WHERE tablename = 'document_embeddings';

-- Step 2: Export vectors to a portable format for cross-platform recovery
-- Useful when restoring to a different vector store after catastrophic failure
COPY (
    SELECT
        id,
        document_id,
        chunk_index,
        content_text,
        embedding::text AS embedding_vector,
        metadata,
        created_at,
        embedding_model
    FROM document_embeddings
    WHERE created_at >= NOW() - INTERVAL '24 hours'
    ORDER BY created_at
) TO '/var/backups/pgvector/embeddings_incremental.csv'
WITH (FORMAT CSV, HEADER TRUE);

-- Step 3: Validate backup integrity - count and spot-check vector dimensionality
SELECT
    COUNT(*) AS total_vectors,
    AVG(vector_dims(embedding)) AS avg_dims,
    MIN(vector_dims(embedding)) AS min_dims,
    MAX(vector_dims(embedding)) AS max_dims,
    COUNT(*) FILTER (WHERE vector_dims(embedding) != 1536) AS malformed_vectors
FROM document_embeddings;

-- Step 4: Track the last-synced position between source table and embeddings table
-- This watermark table drives incremental backup and re-sync recovery
CREATE TABLE IF NOT EXISTS embedding_sync_state (
    sync_id          SERIAL PRIMARY KEY,
    source_table     TEXT NOT NULL,
    last_source_id   BIGINT NOT NULL,
    last_sync_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    vector_count     BIGINT NOT NULL,
    model_version    TEXT NOT NULL,
    notes           TEXT
);

INSERT INTO embedding_sync_state
(source_table, last_source_id, vector_count, model_version)
SELECT
    'documents',
    MAX(id),
    COUNT(*),
    'text-embedding-3-large-1536'
FROM document_embeddings
ON CONFLICT DO NOTHING;

```

The critical insight for pgvector DR: after a restore, PostgreSQL recreates the heap first, then the indexes. On a 50M-row vector table with 1536-dimensional HNSW indexes, index rebuild can take 4–8 hours depending on `maintenance_work_mem` and parallelism. Your RTO must account for this separately from the restore time.

Pinecone

Pinecone is a fully managed service without direct infrastructure access, so DR strategy operates at the API level. Pinecone does not provide native cross-region replication for Standard tier indexes. Their Enterprise tier offers collections (snapshots) and the ability to create indexes from collections

in different regions.

```
# Python
# Pinecone DR: backup via collection creation + cross-region restore

from pinecone import Pinecone
import time
import json
from datetime import datetime

pc = Pinecone(api_key="YOUR_API_KEY")

def create_pinecone_backup(index_name: str, collection_name: str = None) -> dict:
    """
    Create a Pinecone collection (snapshot) from a live index.
    Collections are immutable backups stored by Pinecone indefinitely.
    """
    if collection_name is None:
        timestamp = datetime.utcnow().strftime("%Y%m%d-%H%M%S")
        collection_name = f"{index_name}-backup-{timestamp}"

    # Verify source index exists and is ready
    index_info = pc.describe_index(index_name)
    if index_info.status.ready is False:
        raise RuntimeError(f"Index {index_name} is not in ready state: {index_info.status}")

    vector_count = index_info.status.vector_count
    print(f"Creating collection from {index_name} ({vector_count:,} vectors)")

    # Create collection - this is a point-in-time snapshot
    pc.create_collection(
        name=collection_name,
        source=index_name
    )

    # Poll until collection is ready
    max_wait_seconds = 600
    poll_interval = 15
    elapsed = 0

    while elapsed < max_wait_seconds:
        collection = pc.describe_collection(collection_name)
        if collection.status == "Ready":
            print(f"Collection {collection_name} ready after {elapsed}s")
            break
        elif collection.status == "Initializing":
            print(f"Collection initializing... ({elapsed}s elapsed)")
            time.sleep(poll_interval)
            elapsed += poll_interval
        else:
            raise RuntimeError(f"Collection entered unexpected state: {collection.status}")
    else:
        raise TimeoutError(f"Collection not ready after {max_wait_seconds}s")

    return {
        "collection_name": collection_name,
        "source_index": index_name,
        "vector_count": collection.vector_count,
        "dimension": collection.dimension,
        "metric": collection.metric,
        "created_at": datetime.utcnow().isoformat(),
        "status": collection.status
    }

def restore_pinecone_from_collection(
    collection_name: str,
    target_index_name: str,
    target_region: str = "us-east-1",
    cloud: str = "aws"
```

```

) -> None:
    """
    Restore a Pinecone index from a collection backup, optionally in a different region.
    This is the cross-region failover path.
    """
    collection = pc.describe_collection(collection_name)

    if collection.status != "Ready":
        raise RuntimeError(f"Collection {collection_name} is not in Ready state")

    print(f"Restoring from collection: {collection_name}")
    print(f"  Vectors: {collection.vector_count:,}")
    print(f"  Dimensions: {collection.dimension}")
    print(f"  Target region: {cloud}/{target_region}")

    # Create new index from collection in target region
    pc.create_index(
        name=target_index_name,
        dimension=collection.dimension,
        metric=collection.metric,
        source_collection=collection_name,
        spec={
            "serverless": {
                "cloud": cloud,
                "region": target_region
            }
        }
    )

    # Wait for index to be ready
    max_wait = 900 # 15 minutes for large indexes
    elapsed = 0
    while elapsed < max_wait:
        index_desc = pc.describe_index(target_index_name)
        if index_desc.status.ready:
            print(f"Index {target_index_name} ready after {elapsed}s")
            return
        time.sleep(20)
        elapsed += 20

    raise TimeoutError("Restored index did not become ready within 15 minutes")

def export_pinecone_vectors_to_json(
    index_name: str,
    output_path: str,
    namespace: str = "",
    batch_size: int = 1000
) -> int:
    """
    Export all vectors from a Pinecone index to JSON for cold storage backup.
    Use this when you need a portable backup independent of Pinecone's collection system.
    WARNING: Pinecone does not natively support list-all-vectors for large indexes.
    This approach requires you to maintain a separate list of vector IDs.
    """
    index = pc.Index(index_name)
    stats = index.describe_index_stats()

    # Load IDs from your ID registry (you must maintain this separately)
    with open(f"{output_path}.ids.txt", "r") as f:
        all_ids = [line.strip() for line in f if line.strip()]

    exported_count = 0
    with open(output_path, "w") as out_file:
        for i in range(0, len(all_ids), batch_size):
            batch_ids = all_ids[i:i + batch_size]
            response = index.fetch(ids=batch_ids, namespace=namespace)

            for vec_id, vector_data in response.vectors.items():
                record = {

```

```

        "id": vec_id,
        "values": vector_data.values,
        "metadata": vector_data.metadata or {},
        "namespace": namespace
    }
    out_file.write(json.dumps(record) + "\n")
    exported_count += 1

    if exported_count % 10000 == 0:
        print(f"Exported {exported_count:,} vectors...")

    print(f"Export complete: {exported_count:,} vectors to {output_path}")
    return exported_count

# Example usage
if __name__ == "__main__":
    # Daily backup job
    backup_info = create_pinecone_backup("production-embeddings")

    # Save backup manifest
    with open(f"/var/backups/pinecone/manifest-{datetime.utcnow().date()}.json", "w") as f:
        json.dump(backup_info, f, indent=2)

    print(f"Backup manifest: {backup_info}")

```

Pinecone DR limitations to know: Collections capture vectors and metadata but not index configuration metadata like pod type (for pod-based indexes) or access control settings. Document these separately. Also, Pinecone collections are stored in the same cloud account — they protect against data corruption or accidental deletion but not against account-level or regional cloud outages unless you explicitly restore to a different region.

Weaviate

Weaviate provides native backup support via its Backup API, supporting S3, GCS, Azure Blob Storage, and local filesystem as backup destinations. This is the most operationally complete backup system of the major vector databases.

```

# Python
# Weaviate backup and restore using the official Python client v4

import weaviate
import weaviate.classes as wvc
import time
import boto3
from datetime import datetime

client = weaviate.connect_to_custom(
    http_host="weaviate.production.internal",
    http_port=8080,
    http_secure=False,
    grpc_host="weaviate.production.internal",
    grpc_port=50051,
    grpc_secure=False
)

def create_weaviate_backup(
    backup_id: str,
    backend: str = "s3",
    include_classes: list = None,
    wait_for_completion: bool = True
) -> dict:
    """
    Create a Weaviate backup to S3 (or GCS/Azure/filesystem).

```

```

The backup captures:
- All vector data (HNSW graph + raw vectors)
- Object properties and metadata
- Schema definitions
- Class configurations

Weaviate performs a consistent snapshot - no writes are blocked during backup.
"""
backup_config = {
    "id": backup_id,
    "backend": backend,
}

if include_classes:
    backup_config["include"] = include_classes

print(f"Initiating Weaviate backup: {backup_id} -> {backend}")

response = client.backup.create(
    backup_id=backup_id,
    backend=backend,
    include_collections=include_classes,
    wait_for_completion=wait_for_completion
)

return {
    "backup_id": backup_id,
    "backend": backend,
    "status": response.status,
    "path": response.path,
    "classes": response.collections,
    "started_at": datetime.utcnow().isoformat()
}

def get_backup_status(backup_id: str, backend: str = "s3") -> dict:
    """Poll backup status for async backup jobs."""
    response = client.backup.get_create_status(
        backup_id=backup_id,
        backend=backend
    )
    return {
        "backup_id": backup_id,
        "status": response.status,
        "error": getattr(response, "error", None)
    }

def restore_weaviate_backup(
    backup_id: str,
    backend: str = "s3",
    include_classes: list = None,
    exclude_classes: list = None,
    wait_for_completion: bool = True
) -> dict:
    """
    Restore a Weaviate backup. Can restore specific classes or entire cluster.

    Weaviate restore behavior:
    - Classes that don't exist are created fresh
    - Classes that already exist cause restore to fail (must delete first)
    - Schema is restored along with data
    """
    print(f"Starting Weaviate restore from backup: {backup_id}")

    response = client.backup.restore(
        backup_id=backup_id,
        backend=backend,
        include_collections=include_classes,
        exclude_collections=exclude_classes,

```

```

        wait_for_completion=wait_for_completion
    )

    return {
        "backup_id": backup_id,
        "status": response.status,
        "restored_classes": response.collections,
        "error": getattr(response, "error", None)
    }

def validate_weaviate_restore(
    class_name: str,
    expected_min_count: int,
    sample_query_vector: list = None
) -> dict:
    """
    Post-restore validation:
    1. Check object count is within expected range
    2. Run a sample similarity query to confirm index is queryable
    """
    collection = client.collections.get(class_name)

    # Count validation
    count_result = collection.aggregate.over_all(total_count=True)
    actual_count = count_result.total_count

    count_ok = actual_count >= expected_min_count

    validation = {
        "class": class_name,
        "expected_min": expected_min_count,
        "actual_count": actual_count,
        "count_ok": count_ok,
        "query_ok": None,
        "query_latency_ms": None
    }

    # Query validation - if a sample vector is provided
    if sample_query_vector and count_ok:
        start = time.time()
        results = collection.query.near_vector(
            near_vector=sample_query_vector,
            limit=5,
            return_metadata=wvc.query.MetadataQuery(distance=True)
        )
        latency_ms = (time.time() - start) * 1000

        validation["query_ok"] = len(results.objects) > 0
        validation["query_latency_ms"] = round(latency_ms, 2)
        validation["top_result_distance"] = (
            results.objects[0].metadata.distance
            if results.objects else None
        )

    return validation

def schedule_weaviate_backup_rotation(
    backend: str = "s3",
    retention_days: int = 7,
    backup_prefix: str = "weaviate-backup"
) -> None:
    """
    Create daily backup and clean up backups older than retention_days.
    Designed to be called from a cron job or Airflow DAG.

    Note: Weaviate doesn't manage backup retention itself.
    Cleanup is handled through the storage backend (S3 lifecycle rules recommended).
    """

```



```

today = datetime.utcnow().strftime("%Y/%m/%d")
backup_id = f"{backup_prefix}-{today}"

result = create_weaviate_backup(
    backup_id=backup_id,
    backend=backend,
    wait_for_completion=True
)

print(f"Backup result: {result}")

# For S3-backed backups, enforce retention via S3 lifecycle
# (Better approach than manual deletion through Weaviate API)
print(f"Reminder: Ensure S3 lifecycle rule expires backups after {retention_days} days")
print(f"Backup path prefix: weaviate-backups/{backup_id}/")

# Example DR runbook execution
if __name__ == "__main__":
    # Create backup
    backup_id = f"prod-backup-{datetime.utcnow().strftime('%Y/%m/%d-%H%M')}"
    backup_info = create_weaviate_backup(backup_id, backend="s3")
    print(f"Backup complete: {backup_info}")

    # Restore to DR site (run on DR cluster)
    # restore_result = restore_weaviate_backup(backup_id, backend="s3")

    # Validate
    # validation = validate_weaviate_restore(
    #     "DocumentEmbeddings",
    #     expected_min_count=9_000_000
    # )
    # print(f"Validation: {validation}")

client.close()

```

Weaviate’s backup system has one notable constraint: during a backup, write operations continue but the backup is a consistent snapshot of the moment it was initiated. You will not get a backup that includes writes that arrived after the backup started. For high-write workloads, the RPO is effectively the time since the last backup started, not the time since it completed.

Replication and High Availability Topologies

Backup and restore addresses your RPO and RTO for catastrophic failures. High availability — keeping the system running through partial failures without a full restore — requires replication.

pgvector Replication

pgvector inherits PostgreSQL’s streaming replication without modification. Vectors stored in heap tuples replicate through WAL exactly like any other data. However, HNSW indexes on replicas are kept in sync through WAL as well — you are not rebuilding them, you are streaming the physical changes. This is one of pgvector’s strongest operational advantages.

```

# shell
# Configure pgvector streaming replication
# On primary: postgresql.conf

cat >> /etc/postgresql/16/main/postgresql.conf << 'EOF'
# Replication settings for vector workloads
wal_level = replica

```

```

max_wal_senders = 5
wal_keep_size = 2GB          # Vector operations generate large WAL entries
# HNSW index updates can be 10-50x larger than row inserts
# Increase wal_keep_size if replicas fall behind during bulk loads

max_replication_slots = 5
synchronous_commit = on      # Set to 'remote_write' if latency is unacceptable

# Performance tuning for vector workloads with replication
checkpoint_completion_target = 0.9
max_wal_size = 4GB
EOF

# Create replication user
psql -U postgres << 'EOF'
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'strong_password_here';
SELECT pg_create_physical_replication_slot('vector_replica_1');
EOF

# On replica: configure recovery
cat > /etc/postgresql/16/main/postgresql.conf << 'EOF'
hot_standby = on
hot_standby_feedback = on    # Prevents query cancellation on replica during vacuum

# Replica-specific tuning for vector similarity queries
# Replicas can be dedicated to read-heavy vector search traffic
max_parallel_workers_per_gather = 4
hnsw.ef_search = 200        # Can tune independently on read replicas
EOF

cat > /etc/postgresql/16/main/pg_hba.conf << 'EOF'
# Allow replication connections from primary
host     replication     replicator    10.0.0.10/32    scram-sha-256
EOF

# Create recovery signal and connection config
touch /var/lib/postgresql/16/main/standby.signal

cat > /var/lib/postgresql/16/main/postgresql.auto.conf << 'EOF'
primary_conninfo = 'host=10.0.0.10 port=5432 user=replicator password=strong_password_here
↳ application_name=vector_replica_1'
primary_slot_name = 'vector_replica_1'
EOF

-- PostgreSQL
-- Monitor replication lag - critical for vector workloads during bulk loads
-- Large HNSW index updates generate significant WAL; replicas can fall behind

SELECT
    client_addr,
    application_name,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replication_lag_bytes,
    write_lag,
    flush_lag,
    replay_lag,
    sync_state
FROM pg_stat_replication
ORDER BY replay_lag DESC NULLS LAST;

-- Check if replica is usable for vector search
-- A replica lagging more than your RPO threshold should be excluded from traffic
SELECT
    now() - pg_last_xact_replay_timestamp() AS replication_lag_time,
    pg_is_in_recovery() AS is_replica,
    pg_last_wal_receive_lsn() AS last_received,

```

```
pg_last_wal_replay_lsn() AS last_replayed,
pg_wal_lsn_diff(
    pg_last_wal_receive_lsn(),
    pg_last_wal_replay_lsn()
) AS lag_bytes;
```

A common production pattern is to direct all embedding write operations (INSERT/UPDATE from the embedding pipeline) to the primary, while routing similarity search queries to one or more hot standbys. This requires your application connection pool or proxy (PgBouncer, HAProxy, pgpool-II) to distinguish read from write operations at the connection level.

Weaviate Multi-Node Replication

Weaviate uses a distributed architecture where replication is configured per class (collection). The replication factor determines how many nodes hold a copy of each shard.

```
# Python
# Weaviate: configure a collection with replication factor for HA

import weaviate
import weaviate.classes.config as wvc_config

client = weaviate.connect_to_custom(
    http_host="weaviate-node1.internal",
    http_port=8080,
    http_secure=False,
    grpc_host="weaviate-node1.internal",
    grpc_port=50051,
    grpc_secure=False
)

# Create a collection with replication factor 3 (tolerates 1 node failure)
# and quorum-based consistency
client.collections.create(
    name="DocumentEmbeddings",
    vectorizer_config=wvc_config.Configure.Vectorizer.none(),
    vector_index_config=wvc_config.Configure.VectorIndex.hnsw(
        distance_metric=wvc_config.VectorDistances.COSINE,
        ef_construction=128,
        max_connections=16,
        ef=64,
        dynamic_ef_min=100,
        dynamic_ef_max=500
    ),
    replication_config=wvc_config.Configure.replication(
        factor=3, # Store 3 copies across cluster nodes
        async_enabled=False # Synchronous replication for consistency
    ),
    properties=[
        wvc_config.Property(name="document_id", data_type=wvc_config.DataType.TEXT),
        wvc_config.Property(name="content_text", data_type=wvc_config.DataType.TEXT),
        wvc_config.Property(name="chunk_index", data_type=wvc_config.DataType.INT),
        wvc_config.Property(name="source_url", data_type=wvc_config.DataType.TEXT),
        wvc_config.Property(name="embedding_model", data_type=wvc_config.DataType.TEXT),
        wvc_config.Property(name="created_at", data_type=wvc_config.DataType.DATE),
    ]
)

# Query with explicit consistency level
# QUORUM requires majority of replicas to agree - protects against stale reads
collection = client.collections.get("DocumentEmbeddings")

results = collection.query.near_vector(
    near_vector=[0.1] * 1536,
    limit=10,
    consistency_level=wvc_config.ConsistencyLevel.QUORUM
```

```
)

# For recovery validation after node failure:
def check_cluster_health(client: weaviate.WeaviateClient) -> dict:
    """
    Check node health across the Weaviate cluster.
    Returns a dict identifying any nodes that are degraded or unreachable.
    """
    nodes = client.cluster.nodes(output="verbose")
    health = {
        "total_nodes": len(nodes),
        "healthy_nodes": 0,
        "degraded_nodes": [],
        "total_objects": 0,
        "shard_detail": []
    }

    for node in nodes:
        node_healthy = node.status == "HEALTHY"
        if node_healthy:
            health["healthy_nodes"] += 1
        else:
            health["degraded_nodes"].append({
                "name": node.name,
                "status": node.status,
                "version": node.version
            })

        for shard in (node.shards or []):
            health["total_objects"] += shard.object_count
            health["shard_detail"].append({
                "node": node.name,
                "collection": shard.collection,
                "shard": shard.name,
                "object_count": shard.object_count,
                "vector_indexing_status": shard.vector_indexing_status
            })

    health["quorum_available"] = (
        health["healthy_nodes"] > health["total_nodes"] // 2
    )

    return health

cluster_status = check_cluster_health(client)
print(f"Cluster health: {cluster_status['healthy_nodes']}/{cluster_status['total_nodes']} nodes
      ↪ healthy")
print(f"Quorum available: {cluster_status['quorum_available']}")
if cluster_status["degraded_nodes"]:
    print(f"Degraded nodes: {cluster_status['degraded_nodes']}")

client.close()
```

The key operational point with Weaviate replication: `async_enabled=False` (synchronous replication) gives you the strongest consistency guarantee but increases write latency proportionally to the slowest replica. For write-heavy embedding pipelines, consider `async_enabled=True` during bulk loads and switch back to synchronous for production traffic. This is operationally similar to temporarily reducing `synchronous_commit` in PostgreSQL during bulk inserts.

Cross-Region Failover Runbook

The sections above establish backup and replication mechanics. This section assembles them into an actionable runbook for a full regional failover. The scenario: your primary region becomes unavailable. Your DR site in a secondary region must take over serving vector similarity queries within your RTO.

The runbook below uses pgvector as the primary store (because its replication story is the most mature), with a Pinecone failover path shown in parallel for hybrid deployments where both are in use.

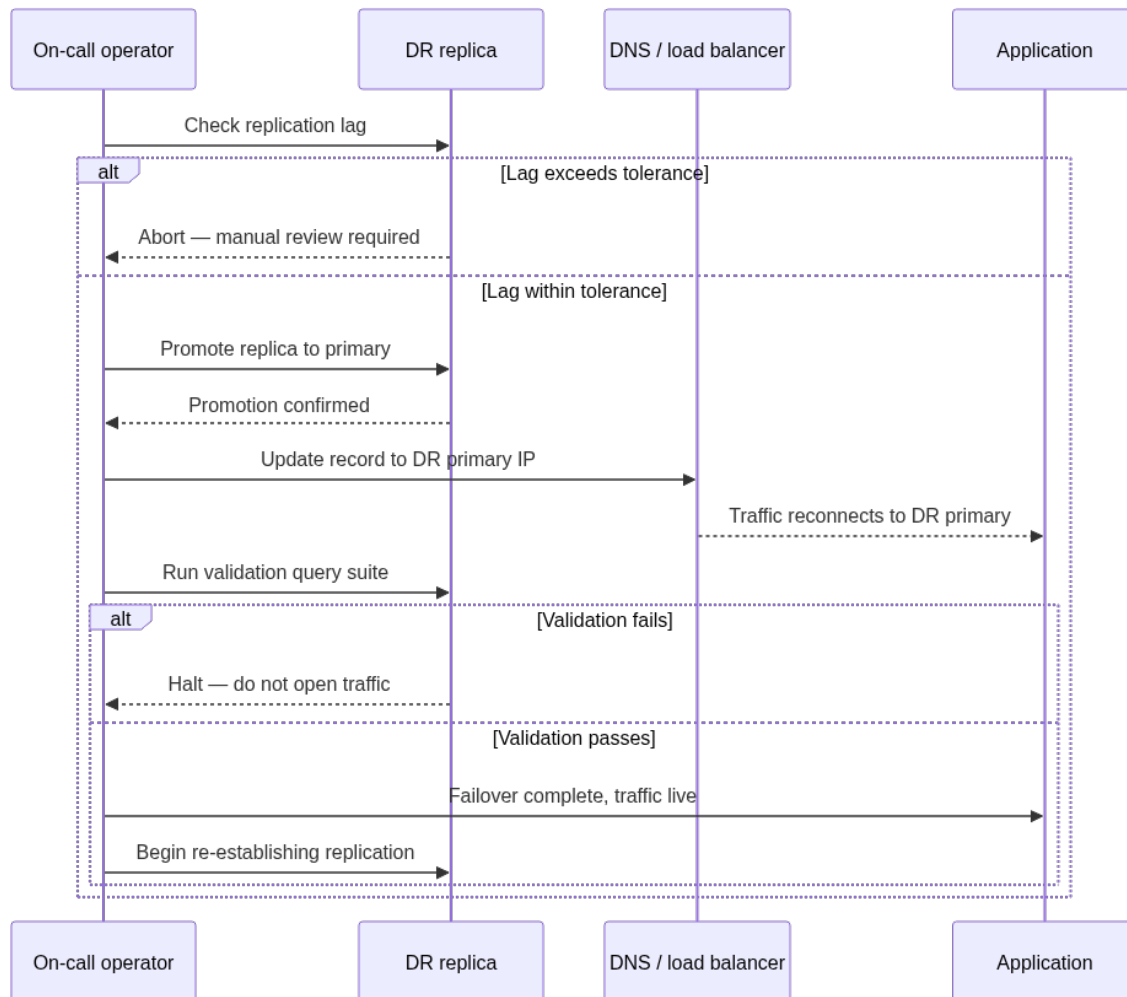


Figure 25: Every gate gives the operator a chance to abort rather than promote a lagging or unvalidated replica.

```

# Python
# Cross-region failover orchestration
# This script is designed to be run manually or by an on-call automation system
# after a human has confirmed the primary region is unreachable.

import psycpg2
import subprocess
import time
  
```

```

import logging
import json
from datetime import datetime
from typing import Optional

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)
log = logging.getLogger("vector_dr_failover")

# -----
# Step 1: Validate DR replica is caught up enough to promote
# -----

def check_replica_lag(
    replica_dsn: str,
    max_acceptable_lag_seconds: float = 30.0
) -> dict:
    """
    Connect to the DR replica and verify replication lag is within RTO tolerance.
    Refuse promotion if lag exceeds threshold - stale vectors cause silent
    search quality degradation, not obvious errors.
    """
    conn = psycopg2.connect(replica_dsn)
    conn.autocommit = True
    cur = conn.cursor()

    cur.execute("""
        SELECT
            now() - pg_last_xact_replay_timestamp()    AS lag_time,
            pg_last_wal_receive_lsn()                  AS last_received,
            pg_last_wal_replay_lsn()                    AS last_replayed,
            pg_is_in_recovery()                         AS is_standby
    """)
    row = cur.fetchone()
    cur.close()
    conn.close()

    lag_interval = row[0]
    lag_seconds = lag_interval.total_seconds() if lag_interval else float("inf")

    return {
        "lag_seconds": lag_seconds,
        "last_received_lsn": str(row[1]),
        "last_replayed_lsn": str(row[2]),
        "is_standby": row[3],
        "within_tolerance": lag_seconds <= max_acceptable_lag_seconds
    }

# -----
# Step 2: Promote PostgreSQL replica to primary
# -----

def promote_postgres_replica(
    replica_host: str,
    pg_ctl_path: str = "/usr/lib/postgresql/16/bin/pg_ctl",
    data_dir: str = "/var/lib/postgresql/16/main"
) -> bool:
    """
    Promote the standby to a writable primary.
    pg_promote() is available in PostgreSQL 12+; pg_ctl promote is the
    older equivalent for automation from outside the database.
    """
    log.info(f"Promoting PostgreSQL replica at {replica_host}")

    # Use pg_promote() from within the replica if we still have a connection
    try:

```

```

conn = psycopg2.connect(
    f"host={replica_host} dbname=postgres user=postgres"
)
conn.autocommit = True
cur = conn.cursor()
cur.execute("SELECT pg_promote(wait => true, wait_seconds => 60)")
result = cur.fetchone()[0]
cur.close()
conn.close()

if result:
    log.info("pg_promote() succeeded - replica is now primary")
    return True
else:
    log.warning("pg_promote() returned false - may have timed out")
    return False

except Exception as e:
    log.error(f"pg_promote() failed: {e}. Attempting pg_ctl fallback.")

# Fallback: call pg_ctl promote via SSH if pg_promote() is unavailable
result = subprocess.run(
    ["ssh", replica_host, f"sudo -u postgres {pg_ctl_path} promote -D {data_dir}"],
    capture_output=True,
    text=True,
    timeout=120
)

if result.returncode == 0:
    log.info("pg_ctl promote succeeded")
    return True
else:
    log.error(f"pg_ctl promote failed: {result.stderr}")
    return False

# -----
# Step 3: Update DNS / load balancer to point to DR primary
# -----

def update_dns_record(
    dns_name: str,
    new_ip: str,
    ttl: int = 30,
    hosted_zone_id: str = None
) -> None:
    """
    Update Route 53 DNS record to point vector-db.internal to the promoted replica.
    Applications reconnect automatically when their connection pool recycles.
    Pre-condition: DNS TTL should already be set low (30-60s) in normal operation.
    """
    import boto3
    r53 = boto3.client("route53")

    change_batch = {
        "Changes": [
            {
                "Action": "UPSERT",
                "ResourceRecordSet": {
                    "Name": dns_name,
                    "Type": "A",
                    "TTL": ttl,
                    "ResourceRecords": [{"Value": new_ip}]
                }
            }
        ]
    }

    response = r53.change_resource_record_sets(
        HostedZoneId=hosted_zone_id,

```

```

        ChangeBatch=change_batch
    )

    change_id = response["ChangeInfo"]["Id"]
    log.info(f"DNS update submitted: {change_id} - {dns_name} -> {new_ip}")

    # Wait for DNS change to propagate
    waiter = r53.get_waiter("resource_record_sets_changed")
    waiter.wait(Id=change_id)
    log.info("DNS change propagated")

# -----
# Step 4: Post-failover validation of vector search quality
# -----

def validate_vector_search_post_failover(
    new_primary_dsn: str,
    test_queries: list,
    min_results_per_query: int = 5
) -> dict:
    """
    Run a battery of known test queries against the promoted primary.
    For each query, we verify:
    1. Results are returned (index is queryable)
    2. Result count meets minimum
    3. Top result cosine distance is within expected range

    test_queries: list of dicts with keys 'query_vector' and 'expected_top_id'
    """
    conn = psycopg2.connect(new_primary_dsn)
    conn.autocommit = True
    cur = conn.cursor()

    results_summary = {
        "total_queries": len(test_queries),
        "passed": 0,
        "failed": 0,
        "failures": []
    }

    for i, test in enumerate(test_queries):
        query_vector = test["query_vector"]
        expected_top_id = test.get("expected_top_id")
        label = test.get("label", f"query_{i}")

        cur.execute("""
            SELECT
                id,
                document_id,
                1 - (embedding <=> %s::vector) AS cosine_similarity
            FROM document_embeddings
            ORDER BY embedding <=> %s::vector
            LIMIT %s
        """, (query_vector, query_vector, min_results_per_query))

        rows = cur.fetchall()
        result_count = len(rows)

        passed = result_count >= min_results_per_query
        top_id_match = True

        if expected_top_id and rows:
            top_id_match = str(rows[0][1]) == str(expected_top_id)
            if not top_id_match:
                log.warning(
                    f"Query '{label}': expected top doc {expected_top_id}, "
                    f"got {rows[0][1]} (similarity={rows[0][2]:.4f})"
                )

```



```

        if passed and top_id_match:
            results_summary["passed"] += 1
        else:
            results_summary["failed"] += 1
            results_summary["failures"].append({
                "label": label,
                "result_count": result_count,
                "top_id_match": top_id_match
            })

    cur.close()
    conn.close()

    results_summary["all_passed"] = results_summary["failed"] == 0
    return results_summary

# -----
# Main failover orchestration
# -----

def execute_failover(config: dict) -> None:
    """
    Execute full cross-region failover for vector store infrastructure.
    config keys:
        replica_dsn, replica_host, primary_dsn, dr_primary_ip,
        hosted_zone_id, max_lag_seconds, test_queries
    """
    log.info("=" * 60)
    log.info("VECTOR STORE FAILOVER INITIATED")
    log.info(f"Time: {datetime.utcnow().isoformat()}Z")
    log.info("=" * 60)

    # Step 1: Check lag
    lag_info = check_replica_lag(
        config["replica_dsn"],
        max_acceptable_lag_seconds=config.get("max_lag_seconds", 30)
    )
    log.info(f"Replica lag: {lag_info['lag_seconds']:.1f}s")

    if not lag_info["within_tolerance"]:
        log.error(
            f"Replica lag ({lag_info['lag_seconds']:.1f}s) exceeds tolerance. "
            "Manual review required before promotion."
        )
        raise RuntimeError("Replication lag too high for safe failover")

    # Step 2: Promote replica
    promoted = promote_postgres_replica(config["replica_host"])
    if not promoted:
        raise RuntimeError("Replica promotion failed - aborting failover")

    # Brief pause for promotion to fully settle
    time.sleep(5)

    # Step 3: Update DNS
    update_dns_record(
        dns_name=config["primary_dsn"],
        new_ip=config["dr_primary_ip"],
        hosted_zone_id=config["hosted_zone_id"]
    )

    # Step 4: Validate
    log.info("Running post-failover validation queries...")
    validation = validate_vector_search_post_failover(
        new_primary_dsn=config["dr_primary_dsn"],
        test_queries=config["test_queries"]
    )

    log.info(f"Validation: {validation['passed']}/{validation['total_queries']} passed")

```

```

if not validation["all_passed"]:
    log.error(f"Validation failures: {json.dumps(validation['failures'], indent=2)}")
    raise RuntimeError("Post-failover validation failed - manual investigation required")

log.info("Failover complete. Vector search is live on DR primary.")
log.info("Next step: begin re-establishing replication from DR primary to new standby.")

```

Recovery Point Objective and Recovery Time Objective Planning

RTO and RPO for vector stores have components that do not appear in relational DR planning:

RPO components for vector stores: 1. Source document RPO (how stale can your underlying content be?) 2. Embedding pipeline RPO (how many documents processed after the last backup are un-embedded?) 3. Vector index RPO (gap between last backup and failure)

All three must align. If your source PostgreSQL database recovers to T-minus-2-hours, your vector store recovers to T-minus-6-hours, and your embedding pipeline queue was processing documents at the time of failure, you could have semantic gaps that are never automatically identified.

RTO components for vector stores: 1. Infrastructure recovery time (restore volume or index from backup) 2. Index rebuild time (pgvector only — zero for managed services that persist the index) 3. Catch-up embedding time (documents written between backup and failure that need to be re-embedded) 4. Validation time (you cannot skip similarity query validation)

Table 35.2 gives representative RTO estimates by platform and database size:

Platform	Vectors	Infrastructure Restore	Index Rebuild	Catch-up Embedding	Total RTO
pgvector	5M × 1536-d	15–30 min	2–4 hours	Depends on gap	3–5 hours
pgvector	50M × 1536-d	45–90 min	8–16 hours	Depends on gap	10–18 hours
Weaviate (self-hosted)	10M	20–40 min	Included in restore	Depends on gap	30–90 min
Pinecone (managed)	10M	30–60 min	Not applicable	Depends on gap	45–90 min
Qdrant (self-hosted)	10M	15–30 min	Included in restore	Depends on gap	25–60 min

The pgvector index rebuild time is the most significant operational difference. For large deployments, this drives a strong architectural recommendation: maintain a hot standby rather than relying on backup-and-restore as your primary HA mechanism.

```

-- PostgreSQL / pgvector
-- Estimate index rebuild time before starting a restore
-- Run this on any machine with the same hardware profile as your DR target

-- First, check current index build parameters
SHOW max_parallel_maintenance_workers;
SHOW maintenance_work_mem;

```

```
-- Rough rebuild time estimation based on table size and vector dimensions
-- These ratios are empirical and will vary by hardware
WITH size_estimates AS (
    SELECT
        COUNT(*) AS row_count,
        AVG(vector_dims(embedding)) AS avg_dims,
        pg_total_relation_size('document_embeddings') AS total_bytes
    FROM document_embeddings
)
SELECT
    row_count,
    avg_dims,
    pg_size_pretty(total_bytes) AS table_size,
    -- HNSW build is roughly O(n log n) - use a linear approximation for planning
    -- Actual time depends heavily on ef_construction and m parameters
    ROUND(row_count::numeric / 1000000 * 12) AS estimated_hnsw_minutes_low,
    ROUND(row_count::numeric / 1000000 * 25) AS estimated_hnsw_minutes_high
FROM size_estimates;

-- After restore: rebuild HNSW index with maximum parallelism
-- Run this in a maintenance window; it will block on heavy I/O
SET maintenance_work_mem = '16GB'; -- Use as much RAM as you can spare
SET max_parallel_maintenance_workers = 7;

-- Drop old index if present (e.g., restored from pg_dump which included it)
DROP INDEX CONCURRENTLY IF EXISTS idx_document_embeddings_hnsw;

-- Rebuild with parameters tuned for recall vs. build time tradeoff
CREATE INDEX idx_document_embeddings_hnsw
    ON document_embeddings
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
-- ef_construction=64 builds faster; increase to 128 for higher recall
-- Monitor progress:
SELECT
    phase,
    blocks_done,
    blocks_total,
    tuples_done,
    tuples_total,
    ROUND(100.0 * tuples_done / NULLIF(tuples_total, 0), 1) AS pct_complete
FROM pg_stat_progress_create_index
WHERE relid = 'document_embeddings'::regclass;
```

The Re-Embedding Recovery Path

The most costly recovery scenario — one that has no equivalent in relational DR — is the case where you must regenerate all embeddings from source documents. This happens when:

- Both the vector store and the source embedding cache are lost
- You change embedding models (not strictly a recovery scenario, but operationally identical)
- The original embedding model is deprecated and results are no longer consistent with new queries

This section builds a production-grade re-embedding pipeline with checkpointing, so that a failure mid-run does not force you to start from zero.

```
# Python
# Re-embedding pipeline with checkpointing and rate-limit handling
# Designed for recovery scenarios where you must regenerate all vectors from source text

import asyncio
import psycpg2
```

```

import psycpg2.extras
import openai
import json
import time
import logging
from datetime import datetime
from dataclasses import dataclass
from typing import Optional

log = logging.getLogger("reembedding_pipeline")

@dataclass
class ReembeddingCheckpoint:
    """Persisted state for resuming a failed re-embedding job."""
    job_id: str
    started_at: str
    last_processed_id: int
    total_processed: int
    total_errors: int
    model: str
    status: str # "running", "completed", "failed"

def load_or_create_checkpoint(
    conn,
    job_id: str,
    model: str
) -> ReembeddingCheckpoint:
    """Load existing checkpoint or create a new one."""
    cur = conn.cursor(cursor_factory=psycpg2.extras.DictCursor)

    # Create checkpoint table if not exists
    cur.execute("""
        CREATE TABLE IF NOT EXISTS reembedding_jobs (
            job_id          TEXT PRIMARY KEY,
            started_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
            last_processed_id BIGINT NOT NULL DEFAULT 0,
            total_processed  BIGINT NOT NULL DEFAULT 0,
            total_errors     BIGINT NOT NULL DEFAULT 0,
            model            TEXT NOT NULL,
            status           TEXT NOT NULL DEFAULT 'running',
            updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
        )
    """)

    cur.execute(
        "SELECT * FROM reembedding_jobs WHERE job_id = %s",
        (job_id,)
    )
    row = cur.fetchone()

    if row:
        log.info(
            f"Resuming job {job_id} from ID {row['last_processed_id']} "
            f"({row['total_processed']} already processed)"
        )
        return ReembeddingCheckpoint(
            job_id=row["job_id"],
            started_at=str(row["started_at"]),
            last_processed_id=row["last_processed_id"],
            total_processed=row["total_processed"],
            total_errors=row["total_errors"],
            model=row["model"],
            status=row["status"]
        )
    else:
        cur.execute("""
            INSERT INTO reembedding_jobs (job_id, model, status)
            VALUES (%s, %s, 'running')
        """, (job_id, model))

```

```

        conn.commit()
        log.info(f"Created new re-embedding job: {job_id}")
        return ReembeddingCheckpoint(
            job_id=job_id,
            started_at=datetime.utcnow().isoformat(),
            last_processed_id=0,
            total_processed=0,
            total_errors=0,
            model=model,
            status="running"
        )

def save_checkpoint(conn, checkpoint: ReembeddingCheckpoint) -> None:
    cur = conn.cursor()
    cur.execute("""
        UPDATE reembedding_jobs SET
            last_processed_id = %s,
            total_processed   = %s,
            total_errors      = %s,
            status            = %s,
            updated_at        = NOW()
        WHERE job_id = %s
    """, (
        checkpoint.last_processed_id,
        checkpoint.total_processed,
        checkpoint.total_errors,
        checkpoint.status,
        checkpoint.job_id
    ))
    conn.commit()

def run_reembedding_pipeline(
    source_dsn: str,
    target_dsn: str,
    job_id: str,
    model: str = "text-embedding-3-large",
    batch_size: int = 100,
    checkpoint_interval: int = 1000,
    max_retries: int = 3
) -> ReembeddingCheckpoint:
    """
    Re-embed all documents from source table into target embeddings table.

    source_dsn: Connection to database with 'documents' table (source text)
    target_dsn: Connection to database with 'document_embeddings' table (destination)

    The pipeline:
    1. Loads documents in batches ordered by ID
    2. Generates embeddings via OpenAI API (with retry logic)
    3. Upserts into the embeddings table
    4. Checkpoints progress every checkpoint_interval records
    5. On resume, skips all records with ID <= last_processed_id
    """
    oai = openai.OpenAI()

    source_conn = psycopg2.connect(source_dsn)
    target_conn = psycopg2.connect(target_dsn)
    target_conn.autocommit = False

    checkpoint = load_or_create_checkpoint(target_conn, job_id, model)

    if checkpoint.status == "completed":
        log.info(f"Job {job_id} is already complete. Exiting.")
        return checkpoint

    log.info(f"Starting re-embedding from document ID > {checkpoint.last_processed_id}")

    source_cur = source_conn.cursor(cursor_factory=psycopg2.extras.DictCursor)

```

```

target_cur = target_conn.cursor()

# Fetch total count for progress reporting
source_cur.execute(
    "SELECT COUNT(*) FROM documents WHERE id > %s",
    (checkpoint.last_processed_id,)
)
remaining = source_cur.fetchone()[0]
log.info(f"Documents remaining to embed: {remaining:,}")

# Stream documents in ordered batches
source_cur.execute("""
    SELECT id, content_text, metadata
    FROM documents
    WHERE id > %s
    ORDER BY id ASC
""", (checkpoint.last_processed_id,))

batch = []
processed_this_run = 0

while True:
    rows = source_cur.fetchmany(batch_size)
    if not rows:
        break

    # Build embedding request
    texts = [row["content_text"] for row in rows]
    doc_ids = [row["id"] for row in rows]
    metadatas = [row["metadata"] for row in rows]

    # Call embedding API with retry
    embeddings = None
    for attempt in range(max_retries):
        try:
            response = oai.embeddings.create(
                model=model,
                input=texts
            )
            embeddings = [item.embedding for item in response.data]
            break
        except openai.RateLimitError:
            wait = 2 ** attempt * 5 # exponential backoff
            log.warning(f"Rate limited. Waiting {wait}s before retry {attempt+1}")
            time.sleep(wait)
        except openai.APIError as e:
            log.error(f"API error on attempt {attempt+1}: {e}")
            if attempt == max_retries - 1:
                checkpoint.total_errors += len(rows)
                log.error(f"Skipping batch after {max_retries} failures")

    if embeddings is None:
        continue

    # Upsert embeddings into target table
    records = [
        (
            doc_ids[i],
            0, # chunk_index - 0 for whole-document embeddings
            texts[i],
            embeddings[i],
            json.dumps(metadatas[i]) if metadatas[i] else "{}",
            model
        )
        for i in range(len(rows))
    ]

    psycopg2.extras.execute_values(
        target_cur,
        """

```

```
INSERT INTO document_embeddings
    (document_id, chunk_index, content_text, embedding, metadata, embedding_model)
VALUES %s
ON CONFLICT (document_id, chunk_index)
DO UPDATE SET
    embedding      = EXCLUDED.embedding,
    content_text   = EXCLUDED.content_text,
    metadata       = EXCLUDED.metadata,
    embedding_model = EXCLUDED.embedding_model,
    updated_at     = NOW()
""",
records,
template="%s, %s, %s, %s::vector, %s::jsonb, %s)"
)
target_conn.commit()

checkpoint.last_processed_id = doc_ids[-1]
checkpoint.total_processed += len(rows)
processed_this_run += len(rows)

# Periodic checkpoint save and progress log
if processed_this_run % checkpoint_interval == 0:
    save_checkpoint(target_conn, checkpoint)
    pct = 100 * checkpoint.total_processed / max(remaining, 1)
    log.info(
        f"Progress: {checkpoint.total_processed:,} processed "
        f"({pct:.1f}%) | last_id={checkpoint.last_processed_id} "
        f"| errors={checkpoint.total_errors}"
    )

# Mark complete
checkpoint.status = "completed"
save_checkpoint(target_conn, checkpoint)

log.info(
    f"Re-embedding complete. "
    f"Total processed: {checkpoint.total_processed:,} | "
    f"errors: {checkpoint.total_errors}"
)

source_cur.close()
target_cur.close()
source_conn.close()
target_conn.close()

return checkpoint
```

The checkpointing pattern here is the most important operational detail: every failure in a multi-hour re-embedding job that lacks checkpointing costs you the full restart. With `last_processed_id` persisted, a failure mid-run resumes from the last committed batch. At a batch size of 100 documents with checkpointing every 1,000 records, the maximum repeated work on resume is 1,000 documents — a few seconds of API calls, not hours.

Testing Your DR Plan

A DR plan that has never been tested is a hypothesis, not a safety net. Vector store DR introduces validation challenges that relational DR does not: after a successful restore, how do you know the similarity results are *correct*, not merely *present*?

Defining Validation Test Cases

Before any disaster occurs, record a set of known-good query pairs: a query vector and the expected top-k document IDs it should return. These become your DR validation corpus.

```
-- PostgreSQL / pgvector
-- Create a table of DR validation test cases
-- Populate this during normal operation, not during an incident

CREATE TABLE IF NOT EXISTS dr_validation_queries (
    test_id          SERIAL PRIMARY KEY,
    label            TEXT NOT NULL,
    description       TEXT,
    query_vector      VECTOR(1536) NOT NULL,
    expected_top_5    BIGINT[] NOT NULL, -- document_ids in expected order
    expected_max_distance FLOAT NOT NULL, -- cosine distance to top result under normal conditions
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_verified     TIMESTAMPTZ,
    model_version     TEXT NOT NULL DEFAULT 'text-embedding-3-large'
);

-- Insert a validation case by capturing the current top result for a known query
INSERT INTO dr_validation_queries
    (label, description, query_vector, expected_top_5, expected_max_distance, model_version)
SELECT
    'python_tutorial_query',
    'Search for Python programming tutorials - should always return the Python docs chunks',
    query_emb.embedding,
    ARRAY(
        SELECT document_id FROM document_embeddings
        ORDER BY embedding <=> query_emb.embedding
        LIMIT 5
    ),
    (
        SELECT MIN(embedding <=> query_emb.embedding)
        FROM document_embeddings
        ORDER BY embedding <=> query_emb.embedding
        LIMIT 1
    ),
    'text-embedding-3-large'
FROM (
    -- In practice, this would be a pre-computed embedding for your test phrase
    SELECT embedding FROM document_embeddings
    WHERE content_text ILIKE '%introduction to python%'
    LIMIT 1
) AS query_emb;

-- Run the full validation suite and return pass/fail per test
WITH validation_results AS (
    SELECT
        dq.test_id,
        dq.label,
        dq.expected_top_5,
        dq.expected_max_distance,
        ARRAY(
            SELECT document_id FROM document_embeddings
            ORDER BY embedding <=> dq.query_vector
            LIMIT 5
        ) AS actual_top_5,
        (
            SELECT MIN(embedding <=> dq.query_vector)
            FROM document_embeddings
            ORDER BY embedding <=> dq.query_vector
            LIMIT 1
        ) AS actual_top_distance
    FROM dr_validation_queries dq
)
SELECT
    test_id,
```



```

label,
expected_top_5[1]          AS expected_top_id,
actual_top_5[1]           AS actual_top_id,
expected_top_5[1] = actual_top_5[1] AS top_id_matches,
ROUND(expected_max_distance::numeric, 4) AS expected_dist,
ROUND(actual_top_distance::numeric, 4) AS actual_dist,
ABS(expected_max_distance - actual_top_distance) < 0.01 AS distance_within_tolerance,
CASE
    WHEN expected_top_5[1] = actual_top_5[1]
        AND ABS(expected_max_distance - actual_top_distance) < 0.01
    THEN 'PASS'
    ELSE 'FAIL'
END AS result
FROM validation_results
ORDER BY test_id;

```

The distance tolerance check (< 0.01) is important: a restored index may produce slightly different distance values due to HNSW's approximate nature and how graph construction is seeded. An exact distance match is neither expected nor required. What matters is that the top result is the correct document and the distance is in the same range — not shifted by a magnitude that indicates a corrupt or mismatched index.

Running Scheduled DR Tests

The most effective DR test schedule for vector stores is:

- **Weekly:** Restore a single collection or schema to an isolated test environment. Run validation queries. Tear down. This tests the restore path without touching production.
- **Monthly:** Full failover simulation — promote the replica, redirect a canary traffic percentage, run validation, failback. This tests the entire runbook including DNS switching.
- **Quarterly:** Re-embedding drill — take a random sample of 10,000 documents, re-generate embeddings, compare to production embeddings using cosine similarity to verify model consistency.

```

# Python
# Scheduled DR test: restore validation environment and run test suite

import subprocess
import psycpg2
import logging
import json
from datetime import datetime

log = logging.getLogger("dr_test")

def restore_test_environment(
    backup_path: str,
    test_db_name: str = "vector_dr_test",
    pg_host: str = "localhost"
) -> bool:
    """
    Create an isolated test database from the latest backup.
    Uses pg_restore for a custom-format dump, or psql for SQL dumps.
    """
    log.info(f"Restoring {backup_path} to {test_db_name}")

    # Drop and recreate the test database
    subprocess.run(
        ["psql", "-h", pg_host, "-U", "postgres",
         "-c", f"DROP DATABASE IF EXISTS {test_db_name}"],
        check=True
    )

```

```

subprocess.run(
    ["psql", "-h", pg_host, "-U", "postgres",
     "-c", f"CREATE DATABASE {test_db_name}"],
    check=True
)

# Restore from backup file
result = subprocess.run(
    ["pg_restore",
     "-h", pg_host,
     "-U", "postgres",
     "-d", test_db_name,
     "--no-owner",
     "--no-acl",
     "-j", "4", # 4 parallel restore workers
     backup_path],
    capture_output=True,
    text=True
)

if result.returncode != 0:
    log.error(f"pg_restore stderr: {result.stderr}")
    return False

log.info(f"Restore to {test_db_name} complete")
return True

def run_dr_validation_suite(
    test_dsn: str,
    validation_queries_path: str
) -> dict:
    """
    Load pre-recorded validation cases and run them against the test environment.
    validation_queries_path: JSON file with list of {label, vector, expected_top_id}
    """
    with open(validation_queries_path) as f:
        test_cases = json.load(f)

    conn = psycopg2.connect(test_dsn)
    conn.autocommit = True
    cur = conn.cursor()

    results = {
        "run_at": datetime.utcnow().isoformat(),
        "total": len(test_cases),
        "passed": 0,
        "failed": 0,
        "details": []
    }

    for tc in test_cases:
        label = tc["label"]
        vector = tc["query_vector"]
        expected_id = tc["expected_top_document_id"]

        cur.execute("""
            SELECT document_id, 1 - (embedding <=> %s::vector) AS similarity
            FROM document_embeddings
            ORDER BY embedding <=> %s::vector
            LIMIT 1
            """, (vector, vector))

        row = cur.fetchone()
        if row is None:
            passed = False
            detail = {"label": label, "pass": False, "reason": "no results returned"}
        else:
            actual_id, similarity = row
            passed = str(actual_id) == str(expected_id)

```

```

        detail = {
            "label": label,
            "pass": passed,
            "expected_id": expected_id,
            "actual_id": actual_id,
            "similarity": round(float(similarity), 4)
        }

        results["details"].append(detail)
        if passed:
            results["passed"] += 1
        else:
            results["failed"] += 1
            log.warning(f"FAIL: {label} - {detail}")

    cur.close()
    conn.close()

    results["pass_rate"] = round(results["passed"] / max(results["total"], 1) * 100, 1)
    log.info(
        f"DR validation: {results['passed']}/{results['total']} passed "
        f"({results['pass_rate']}%)"
    )
    return results

def teardown_test_environment(
    test_db_name: str = "vector_dr_test",
    pg_host: str = "localhost"
) -> None:
    """Remove the test database after validation."""
    subprocess.run(
        ["psql", "-h", pg_host, "-U", "postgres",
         "-c", f"DROP DATABASE IF EXISTS {test_db_name}"],
        check=True
    )
    log.info(f"Test environment {test_db_name} removed")

def weekly_dr_test(config: dict) -> dict:
    """
    Full DR test pipeline: restore, validate, teardown, report.
    Designed to run from a scheduled job (cron, Airflow, etc.).
    """
    report = {
        "test_date": datetime.utcnow().isoformat(),
        "restore_ok": False,
        "validation": None,
        "overall_pass": False
    }

    try:
        restored = restore_test_environment(
            backup_path=config["latest_backup_path"],
            test_db_name=config.get("test_db_name", "vector_dr_test"),
            pg_host=config.get("pg_host", "localhost")
        )
        report["restore_ok"] = restored

        if restored:
            validation = run_dr_validation_suite(
                test_dsn=config["test_dsn"],
                validation_queries_path=config["validation_queries_path"]
            )
            report["validation"] = validation
            report["overall_pass"] = (
                restored and validation["pass_rate"] >= config.get("min_pass_rate", 95.0)
            )

    finally:

```

```

teardown_test_environment(
    test_db_name=config.get("test_db_name", "vector_dr_test"),
    pg_host=config.get("pg_host", "localhost")
)

# Write report to file for audit trail
report_path = (
    f"/var/log/dr_tests/vector_dr_{datetime.utcnow().strftime('%Y%m%d')}.json"
)
with open(report_path, "w") as f:
    json.dump(report, f, indent=2, default=str)

log.info(f"DR test report written to {report_path}")
return report

```

The `min_pass_rate` threshold of 95% acknowledges that HNSW is an approximate index: a small percentage of queries may return a slightly different result set after restore due to non-deterministic graph construction during index rebuild. A 100% pass rate requirement would produce false failures. Define your acceptable approximate search tolerance in advance and encode it here.

Synchronizing Vector Stores with Source Systems After Recovery

One scenario that is easy to overlook in DR planning is the case where the vector store recovers successfully but the source system recovers to a different point in time — or vice versa. This creates semantic drift: the vector index contains embeddings for documents that no longer exist in the source, or the source has documents that have never been embedded.

```

-- PostgreSQL
-- Post-recovery drift detection: find documents in source that lack embeddings
-- Run this after any recovery event to identify re-embedding backlog

WITH source_docs AS (
    SELECT id, updated_at
    FROM documents
    WHERE deleted_at IS NULL
),
embedded_docs AS (
    SELECT DISTINCT document_id, MAX(updated_at) AS last_embedded_at
    FROM document_embeddings
    GROUP BY document_id
)
SELECT
    sd.id AS document_id,
    sd.updated_at AS source_updated_at,
    ed.last_embedded_at,
    CASE
        WHEN ed.document_id IS NULL THEN 'MISSING_EMBEDDING'
        WHEN sd.updated_at > ed.last_embedded_at + INTERVAL '5 minutes'
            THEN 'STALE_EMBEDDING'
        ELSE 'OK'
    END AS drift_status
FROM source_docs sd
LEFT JOIN embedded_docs ed ON sd.id = ed.document_id
WHERE
    ed.document_id IS NULL
    OR sd.updated_at > ed.last_embedded_at + INTERVAL '5 minutes'
ORDER BY sd.updated_at DESC;

-- Count by drift category for an at-a-glance recovery health check
WITH source_docs AS (
    SELECT id, updated_at FROM documents WHERE deleted_at IS NULL
),

```

```

embedded_docs AS (
    SELECT DISTINCT document_id, MAX(updated_at) AS last_embedded_at
    FROM document_embeddings GROUP BY document_id
)
SELECT
    CASE
        WHEN ed.document_id IS NULL THEN 'MISSING_EMBEDDING'
        WHEN sd.updated_at > ed.last_embedded_at + INTERVAL '5 minutes'
            THEN 'STALE_EMBEDDING'
        ELSE 'OK'
    END AS drift_status,
    COUNT(*) AS document_count
FROM source_docs sd
LEFT JOIN embedded_docs ed ON sd.id = ed.document_id
GROUP BY 1
ORDER BY 2 DESC;

-- SQL Server equivalent (using SQL Server with a vector extension or hybrid architecture
-- where SQL Server is the source system and pgvector/external store holds embeddings)

-- Post-recovery drift detection: documents in SQL Server source without embeddings
WITH SourceDocs AS (
    SELECT id, updated_at
    FROM dbo.documents
    WHERE deleted_at IS NULL
),
EmbeddedDocs AS (
    SELECT document_id, MAX(updated_at) AS last_embedded_at
    FROM dbo.embedding_sync_log -- sync log maintained by embedding pipeline
    GROUP BY document_id
)
SELECT
    sd.id AS document_id,
    sd.updated_at AS source_updated_at,
    ed.last_embedded_at,
    CASE
        WHEN ed.document_id IS NULL THEN 'MISSING_EMBEDDING'
        WHEN sd.updated_at > DATEADD(MINUTE, 5, ed.last_embedded_at)
            THEN 'STALE_EMBEDDING'
        ELSE 'OK'
    END AS drift_status
FROM SourceDocs sd
LEFT JOIN EmbeddedDocs ed ON sd.id = ed.document_id
WHERE
    ed.document_id IS NULL
    OR sd.updated_at > DATEADD(MINUTE, 5, ed.last_embedded_at)
ORDER BY sd.updated_at DESC;

-- Summary count
WITH SourceDocs AS (
    SELECT id, updated_at FROM dbo.documents WHERE deleted_at IS NULL
),
EmbeddedDocs AS (
    SELECT document_id, MAX(updated_at) AS last_embedded_at
    FROM dbo.embedding_sync_log
    GROUP BY document_id
)
SELECT
    CASE
        WHEN ed.document_id IS NULL THEN 'MISSING_EMBEDDING'
        WHEN sd.updated_at > DATEADD(MINUTE, 5, ed.last_embedded_at)
            THEN 'STALE_EMBEDDING'
        ELSE 'OK'
    END AS drift_status,
    COUNT(*) AS document_count
FROM SourceDocs sd
LEFT JOIN EmbeddedDocs ed ON sd.id = ed.document_id
GROUP BY
    CASE
        WHEN ed.document_id IS NULL THEN 'MISSING_EMBEDDING'

```

```
WHEN sd.updated_at > DATEADD(MINUTE, 5, ed.last_embedded_at)
THEN 'STALE_EMBEDDING'
ELSE 'OK'
END
ORDER BY document_count DESC;
```

After drift detection, feed the `MISSING_EMBEDDING` and `STALE_EMBEDDING` document IDs back into the re-embedding pipeline shown in the previous section. The pipeline's `ON CONFLICT DO UPDATE` upsert logic handles both new insertions and updates without special-casing.

DR Runbook Checklist

Operational runbooks should be concise enough to use under pressure. This is the condensed checklist version of everything in this chapter:

Before any failure (preparation): - ☐ Backup schedule established with RPO clearly documented - ☐ Backup restore tested within the last 30 days - ☐ DR validation query corpus populated and stored - ☐ Replication lag monitoring alerts configured (alert at > 60s, page at > 300s) - ☐ DNS TTL for vector store endpoint set to 60 seconds - ☐ Index rebuild time estimated and incorporated into RTO SLA - ☐ Re-embedding pipeline tested with checkpointing verified - ☐ Source-to-vector-store sync watermark table in place

During failover: - ☐ Confirm primary region is unreachable (do not fail over on transient issues) - ☐ Check DR replica replication lag — do not promote if lag exceeds RTO tolerance - ☐ Promote replica / restore from backup - ☐ Update DNS / load balancer target - ☐ Run validation query suite — do not open traffic until pass rate threshold - ☐ Run drift detection query — quantify the re-embedding backlog - ☐ Start re-embedding pipeline for drifted documents (background job) - ☐ Notify downstream teams of RPO impact (data as-of timestamp)

After failover: - ☐ Establish new replication from DR primary to a new standby - ☐ Schedule new backup immediately (failover cluster has no recent backup) - ☐ Document the incident timeline and actual RPO/RTO achieved - ☐ Update RTO estimates in Table 35.2 based on observed rebuild times

Key Takeaways

- **Embeddings are primary data, not cache.** The cost and time to regenerate a large vector corpus from source text makes vectors as recovery-critical as the transactional data they were derived from. Treat backup frequency and retention accordingly.
- **Index rebuild is a hidden RTO multiplier for pgvector.** Unlike managed vector databases that persist HNSW indexes through restore, pgvector rebuilds indexes from heap data after restore. For datasets in the tens of millions of rows, this can extend RTO by 8–16 hours beyond infrastructure recovery time, making hot standby replication the preferred HA strategy at scale.
- **Synchronization with source systems is a first-class DR concern.** Because vector stores are derived from upstream content, a recovery plan that restores the vector index to

a different point in time than the source database creates semantic drift — documents that exist in search results but not in the source, or source documents whose embeddings reflect outdated content. Post-recovery drift detection and targeted re-embedding are mandatory steps in any complete runbook.

- **Validation after restore must test similarity correctness, not just availability.** A restored vector index can return HTTP 200 and still produce incorrect results due to corrupt graph state, model mismatch, or partial restore. Pre-recording known-good query/result pairs and verifying them post-restore — with tolerance for HNSW’s approximate nature — is the only reliable way to confirm a restore was successful.
 - **Checkpointed re-embedding pipelines eliminate the catastrophic restart problem.** Multi-hour embedding regeneration jobs without checkpointing mean any failure requires starting from zero. Persisting a `last_processed_id` watermark after each committed batch caps the repeated work on resume to at most one checkpoint interval, turning a potentially day-long re-run into a recovery measured in minutes.
-

Chapter 36: Backup Strategies for Embedding Data

Embedding data presents a deceptively familiar backup challenge. The vectors themselves are just floating-point arrays — and any experienced DBA knows how to back up a table of floats. What makes embedding backups genuinely different is the relationship between three distinct artifacts that must remain in sync: the raw source data, the embeddings derived from that data, and the model that produced those embeddings. Back up only two of the three and you have a restoration problem that won't reveal itself until you're already in a production incident. This chapter covers the full backup and recovery strategy for vector workloads — from pgvector snapshots to managed Pinecone exports, from model versioning to the operational patterns that keep RAG pipelines recoverable at scale.

The Three-Artifact Problem

In traditional database backups, data is self-contained. A row in a customer table means the same thing whether you restore it today or in three years. Embeddings are not self-contained — they are a function of source text and model weights, and that function changes when either input changes.

Consider a document chunk stored in pgvector:

```
-- PostgreSQL / pgvector
CREATE TABLE document_chunks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id   UUID NOT NULL REFERENCES documents(id),
    chunk_text  TEXT NOT NULL,
    embedding   VECTOR(1536),
    model_id    VARCHAR(100) NOT NULL, -- e.g. 'text-embedding-3-small-v1'
    model_hash  VARCHAR(64),          -- SHA-256 of model release tag
    embedded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_source FOREIGN KEY (source_id)
        REFERENCES documents(id) ON DELETE CASCADE
);

-- Index for backup validation queries
CREATE INDEX idx_chunks_model ON document_chunks(model_id, embedded_at);

-- View: chunks without matching source data (backup integrity check)
CREATE VIEW orphaned_embeddings AS
SELECT dc.id, dc.source_id, dc.model_id, dc.embedded_at
FROM   document_chunks dc
LEFT JOIN documents d ON d.id = dc.source_id
WHERE  d.id IS NULL;
```

The `model_id` and `model_hash` columns are the starting point for a coherent backup strategy. If you restore embeddings generated by `text-embedding-3-small` but your application is now calling `text-embedding-3-large`, every similarity search will return nonsense — not an error, not a crash, just silently wrong results. Backing up the vector data without capturing what produced it means the restore is incomplete by definition.

The three artifacts to track together:

Artifact	What to Back Up	Where It Lives
Source data	Raw text, documents, metadata	Your primary RDBMS or object store
Embeddings	Vector arrays + model provenance	Vector database or pgvector table
Model reference	Model version, provider, API endpoint config	Model registry, config management

The goal of an embedding backup strategy is to ensure that after any restore event, you can either (a) serve existing embeddings from a consistent snapshot where all three artifacts match, or (b) regenerate embeddings from source data using the same model version. Both paths need explicit support in your backup design.

Backup Strategies for pgvector

PostgreSQL with pgvector behaves like any other PostgreSQL workload for the purposes of backup tooling. The difference is in how large the data gets. A table with 10 million document chunks at 1536 dimensions (float4, 4 bytes per dimension) holds approximately **58 GB** of vector data before indices. HNSW indices on that table add another 15–30 GB depending on `m` and `ef_construction` parameters. Plan storage accordingly.

Logical Backups with pg_dump

For moderate-scale deployments (under ~50 GB of vector data), `pg_dump` is still workable:

```
# shell
# Full logical backup - include custom format for parallel restore capability
pg_dump \
  --format=custom \
  --compress=9 \
  --jobs=4 \
  --table=document_chunks \
  --table=documents \
  --file=/backup/rag_vectors_$(date +%Y%m%d_%H%M%S).dump \
  rag_production_db

# Verify backup integrity before storing offsite
pg_restore --list /backup/rag_vectors_20240615_020000.dump | head -30

# Restore to a staging environment for validation
pg_restore \
  --format=custom \
  --jobs=4 \
  --dbname=rag_staging \
  /backup/rag_vectors_20240615_020000.dump
```

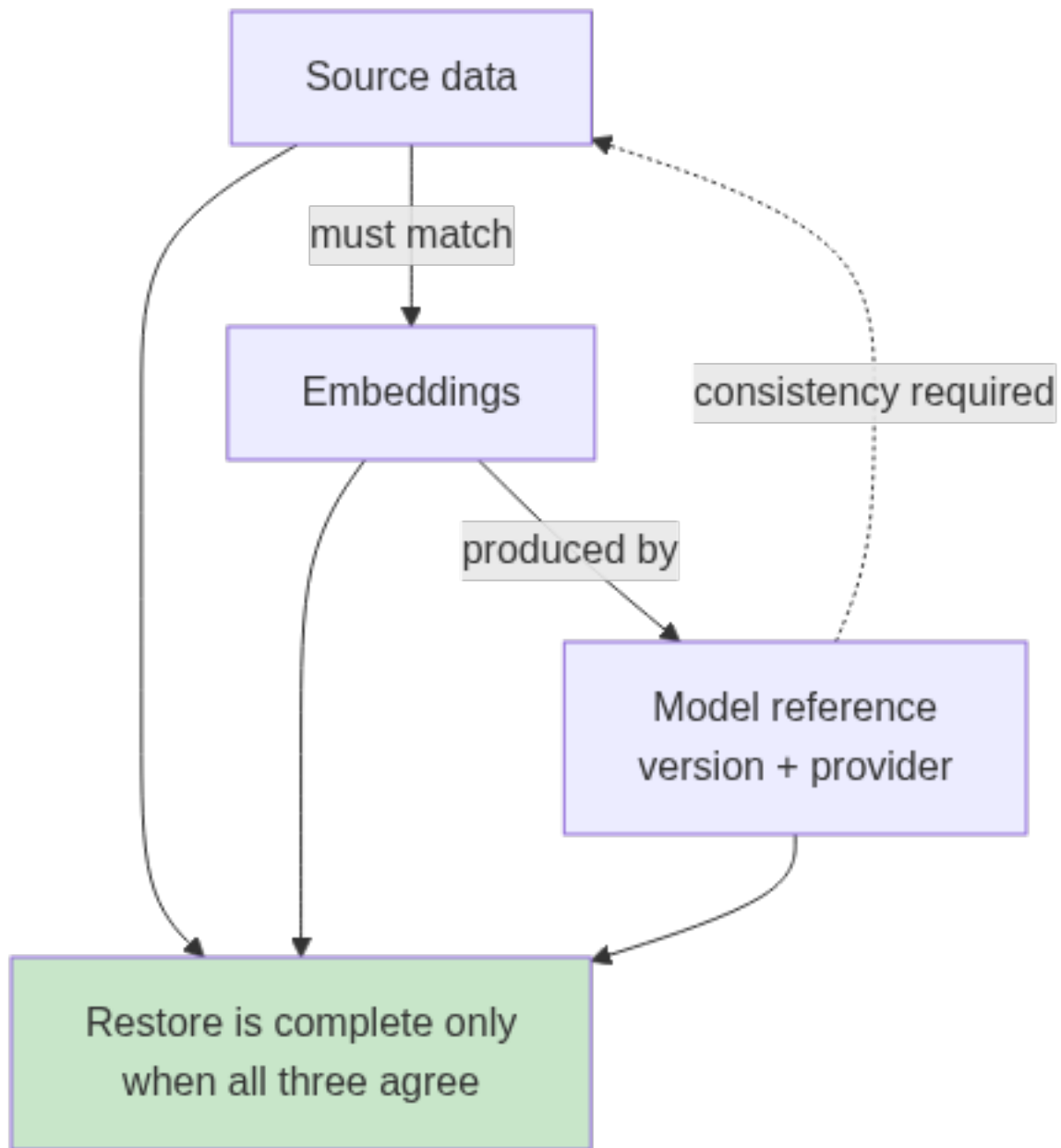


Figure 26: Back up any two of the three artifacts and the restore is incomplete by definition — a consistent snapshot requires all three to agree.

Note that `pg_dump` does not back up HNSW or IVFFlat indices — it captures the `CREATE INDEX` DDL, which means the index is rebuilt from scratch on restore. For a 10-million-row table, that rebuild takes 20–45 minutes depending on hardware. Budget this into your RTO. If rebuilding the index is unacceptable, use physical backup methods.

Physical Backups with pgBackRest

For large vector tables where index rebuild time breaks your RTO, pgBackRest with incremental backups is the right tool:

```
# shell
# pgbackrest.conf excerpt for vector workload
# /etc/pgbackrest/pgbackrest.conf

[global]
repo1-path=/mnt/backup/pgbackrest
repo1-retention-full=4          # 4 full backups (weekly cycle = 1 month)
repo1-retention-diff=14        # 14 differential backups
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<VAULT_SECRET>
compress-type=lz4              # lz4 is faster than gzip for large float arrays
compress-level=3
process-max=4

[rag-production]
pg1-path=/var/lib/postgresql/16/main
pg1-host=db-primary.internal

# Full backup - run weekly
pgbackrest --stanza=rag-production backup --type=full

# Differential backup - run daily
pgbackrest --stanza=rag-production backup --type=diff

# Restore to point-in-time (critical for embedding consistency)
pgbackrest --stanza=rag-production restore \
  --target="2024-06-15 03:00:00" \
  --target-action=promote
```

The PITR capability is particularly valuable for embedding data. If your embedding pipeline had a bug that corrupted embeddings between 14:00 and 16:00 yesterday, you can restore to 13:59 and have a clean state — HNSW index intact, no rebuild required.

Incremental Strategy for Append-Heavy Workloads

Most RAG pipelines are append-heavy: new documents get embedded and inserted, old embeddings rarely change. This makes partial exports viable as a complement to full backups:

```
-- PostgreSQL / pgvector
-- Export only embeddings created in the last 24 hours
COPY (
  SELECT
    dc.id,
    dc.source_id,
    dc.chunk_text,
    dc.embedding::TEXT,  -- cast vector to text for portable export
    dc.model_id,
    dc.model_hash,
    dc.embedded_at,
    d.title,
    d.source_url
  FROM   document_chunks dc
  JOIN   documents d ON d.id = dc.source_id
  WHERE  dc.embedded_at >= NOW() - INTERVAL '24 hours'
  ORDER BY dc.embedded_at
)
```

```
TO '/backup/incremental/chunks_20240615.csv'
WITH (FORMAT CSV, HEADER, DELIMITER ',');

-- Track export completion for audit trail
INSERT INTO backup_audit (export_type, row_count, export_start, export_end, file_path)
SELECT
    'incremental_daily',
    COUNT(*),
    MIN(embedded_at),
    MAX(embedded_at),
    '/backup/incremental/chunks_20240615.csv'
FROM document_chunks
WHERE embedded_at >= NOW() - INTERVAL '24 hours';
```

Exporting from Managed Vector Databases

Managed services like Pinecone, Weaviate Cloud, and Qdrant Cloud do not expose file-level backup interfaces. Backup means export — pulling vectors out via API and storing them yourself. This is the primary gap in managed vector database offerings as of 2024, and every production deployment needs an explicit strategy to address it.

Pinecone: Batch Fetch Export

Pinecone does not offer a native export API as of this writing. The operational pattern is to maintain a shadow copy in your source database and reconstruct from there, but if you need the actual vectors:

```
# Python
import os
import json
import time
from pinecone import Pinecone
from pathlib import Path

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index("rag-production")

def export_pinecone_namespace(
    index,
    namespace: str,
    output_dir: str,
    batch_size: int = 1000
) -> dict:
    """
    Export all vectors from a Pinecone namespace.
    Uses list() to paginate IDs, then fetch() in batches.
    Returns export statistics.
    """
    Path(output_dir).mkdir(parents=True, exist_ok=True)
    output_file = f"{output_dir}/pinecone_{namespace}_{int(time.time())}.jsonl"

    stats = index.describe_index_stats()
    namespace_stats = stats.namespaces.get(namespace, {})
    total_vectors = namespace_stats.get("vector_count", 0)

    print(f"Exporting {total_vectors} vectors from namespace '{namespace}'")

    exported = 0
    pages_written = 0

    with open(output_file, "w") as f:
        # list() paginates through all vector IDs
        for ids_page in index.list(namespace=namespace, limit=batch_size):
```

```

    if not ids_page:
        break

    # Fetch actual vectors + metadata
    result = index.fetch(ids=ids_page, namespace=namespace)

    for vector_id, vector_data in result.vectors.items():
        record = {
            "id": vector_id,
            "values": vector_data.values,
            "metadata": vector_data.metadata or {},
            "namespace": namespace
        }
        f.write(json.dumps(record) + "\n")
        exported += 1

    pages_written += 1
    if pages_written % 10 == 0:
        print(f" Progress: {exported}/{total_vectors} vectors exported")

    return {
        "namespace": namespace,
        "total_exported": exported,
        "output_file": output_file,
        "timestamp": time.time()
    }

# Export all namespaces
stats = index.describe_index_stats()
for namespace in stats.namespaces.keys():
    result = export_pinecone_namespace(
        index=index,
        namespace=namespace,
        output_dir="/backup/pinecone",
        batch_size=1000
    )
    print(f"Exported: {result}")

# Save index configuration alongside data
index_config = {
    "name": "rag-production",
    "dimension": stats.dimension,
    "metric": "cosine",
    "namespaces": {
        ns: data.vector_count
        for ns, data in stats.namespaces.items()
    },
    "export_timestamp": time.time()
}

with open("/backup/pinecone/index_config.json", "w") as f:
    json.dump(index_config, f, indent=2)

```

This export produces JSONL files that can be reimported to a new Pinecone index or cross-loaded into Weaviate or Qdrant if you need to migrate. The key requirement: store this alongside your model configuration so you know exactly which embedding model produced these vectors.

Weaviate: Native Backup API

Weaviate has a first-class backup API that supports S3, GCS, and Azure Blob Storage backends. This is significantly more operationally clean than the Pinecone fetch-loop approach:

```

# Python
import weaviate
import time
import os

client = weaviate.connect_to_wcs(

```

```

cluster_url=os.environ["WEAVIATE_URL"],
auth_credentials=weaviate.auth.AuthApiKey(os.environ["WEAVIATE_API_KEY"])
)

def create_weaviate_backup(
    client,
    backup_id: str,
    classes: list[str] | None = None, # None = all classes
    backend: str = "s3",
    bucket: str = "my-weaviate-backups",
    path: str = "backups"
) -> dict:
    """
    Trigger a Weaviate backup and wait for completion.
    Returns backup status details.
    """
    backup_config = {
        "id": backup_id,
        "include": classes # omit to back up all classes
    }

    print(f"Starting Weaviate backup: {backup_id}")

    result = client.backup.create(
        backup_id=backup_id,
        backend=backend,
        include_classes=classes,
        wait_for_completion=False # async - we'll poll
    )

    print(f"Backup initiated. Status: {result.status}")

    # Poll until completion
    while True:
        status = client.backup.get_create_status(
            backup_id=backup_id,
            backend=backend
        )

        if status.status == "SUCCESS":
            print(f"Backup completed: {backup_id}")
            return {
                "backup_id": backup_id,
                "status": "SUCCESS",
                "backend": backend,
                "path": f"s3://{bucket}/{path}/{backup_id}"
            }
        elif status.status == "FAILED":
            raise RuntimeError(f"Backup failed: {status.error}")
        else:
            print(f"Status: {status.status} - waiting 10s...")
            time.sleep(10)

    # Create a timestamped backup
    backup_id = f"rag-backup-{time.strftime('%Y%m%d-%H%M%S')}"

    backup_result = create_weaviate_backup(
        client=client,
        backup_id=backup_id,
        classes=["DocumentChunk", "Document"], # specific classes only
        backend="s3",
        bucket="my-weaviate-backups"
    )

    print(f"Backup stored: {backup_result}")

    # Restore from backup (e.g., disaster recovery scenario)
    def restore_weaviate_backup(client, backup_id: str, backend: str = "s3"):
        result = client.backup.restore(
            backup_id=backup_id,

```

```
        backend=backend,
        wait_for_completion=True
    )
    print(f"Restore status: {result.status}")
    return result

client.close()
```

Weaviate backups are consistent snapshots — they do not require downtime and they include schema definitions, object properties, and vector indices together. This is the model all vector databases should eventually follow, and it simplifies the three-artifact problem considerably because schema (which includes the model configuration) is part of the backup payload.

Model Versioning as a Backup Dependency

Vectors are only interpretable in the context of the model that created them. This makes model version management a first-class backup concern, not an afterthought. DBAs entering the AI infrastructure space often underestimate this dependency because traditional database backups have no equivalent.

The failure mode looks like this: you restore your vector database from a backup taken six months ago. The backup is clean. All vectors are present. You rebuild the HNSW index. Queries return results. But your application team upgraded from `text-embedding-ada-002` to `text-embedding-3-small` four months ago, and the two models produce vectors in incompatible embedding spaces. The cosine similarities you're computing between the restored old vectors and new query vectors are geometrically meaningless.

Model Registry Pattern

```
# Python
import json
import hashlib
import os
from datetime import datetime, timezone
from pathlib import Path

class EmbeddingModelRegistry:
    """
    Tracks model versions used to produce embeddings.
    Persisted as JSON alongside vector backups.
    In production, back this up to S3/GCS with versioning enabled.
    """

    def __init__(self, registry_path: str = "/etc/rag/model_registry.json"):
        self.registry_path = Path(registry_path)
        self.registry_path.parent.mkdir(parents=True, exist_ok=True)
        self._load()

    def _load(self):
        if self.registry_path.exists():
            with open(self.registry_path) as f:
                self.data = json.load(f)
        else:
            self.data = {"models": {}, "current": None}

    def _save(self):
        with open(self.registry_path, "w") as f:
            json.dump(self.data, f, indent=2)

    def register_model(
```



```

self,
model_id: str,
provider: str,
dimensions: int,
endpoint: str,
api_version: str | None = None,
notes: str = ""
) -> str:
    """
    Register a model version. Returns a stable model_hash
    that can be stored alongside each embedding row.
    """
    fingerprint = f"{provider}:{model_id}:{dimensions}:{api_version or ''}"
    model_hash = hashlib.sha256(fingerprint.encode()).hexdigest()[:16]

    self.data["models"][model_hash] = {
        "model_id": model_id,
        "provider": provider,
        "dimensions": dimensions,
        "endpoint": endpoint,
        "api_version": api_version,
        "registered_at": datetime.now(timezone.utc).isoformat(),
        "notes": notes,
        "hash": model_hash
    }

    self._save()
    return model_hash

def set_current(self, model_hash: str):
    assert model_hash in self.data["models"], f"Unknown model hash: {model_hash}"
    self.data["current"] = model_hash
    self._save()

def get_current(self) -> dict:
    assert self.data["current"], "No current model set"
    return self.data["models"][self.data["current"]]

def get_model(self, model_hash: str) -> dict:
    return self.data["models"].get(model_hash)

def compatibility_check(
    self,
    backup_model_hash: str,
    current_model_hash: str
) -> dict:
    """
    Check whether backup vectors are compatible with current model.
    Returns a dict with compatibility status and recommended action.
    """
    backup_model = self.get_model(backup_model_hash)
    current_model = self.get_model(current_model_hash)

    if backup_model_hash == current_model_hash:
        return {"compatible": True, "action": "restore_directly"}

    if backup_model["dimensions"] != current_model["dimensions"]:
        return {
            "compatible": False,
            "action": "re_embed_all",
            "reason": f"Dimension mismatch: {backup_model['dimensions']} vs
↪ {current_model['dimensions']}"
        }

    if backup_model["model_id"] != current_model["model_id"]:
        return {
            "compatible": False,
            "action": "re_embed_all",
            "reason": f"Different model: {backup_model['model_id']} vs {current_model['model_id']}"
        }

```

```

    return {
        "compatible": False,
        "action": "evaluate",
        "reason": "Same model ID but different versions - test similarity quality before deciding"
    }

# Usage
registry = EmbeddingModelRegistry()

# Register models in use
hash_ada = registry.register_model(
    model_id="text-embedding-ada-002",
    provider="openai",
    dimensions=1536,
    endpoint="https://api.openai.com/v1/embeddings",
    notes="Legacy model, used before 2024-03"
)

hash_v3 = registry.register_model(
    model_id="text-embedding-3-small",
    provider="openai",
    dimensions=1536,
    endpoint="https://api.openai.com/v1/embeddings",
    api_version="2024-02-01",
    notes="Current production model"
)

registry.set_current(hash_v3)

# Before restore: check compatibility
check = registry.compatibility_check(hash_ada, hash_v3)
print(check)
# {'compatible': False, 'action': 're_embed_all', 'reason': 'Different model: ...'}

```

Back up the model registry JSON to the same location as your vector exports and treat it as part of the backup artifact set. If the registry says a restore requires `re_embed_all`, your recovery procedure needs to include re-embedding time in the RTO calculation — which at 10 million chunks and API rate limits can be measured in hours, not minutes.

Recovery Procedures and RTO Planning

Recovery from a backup event in a vector workload has three distinct phases, each with its own time budget:

1. **Data restore** — Moving backup files to the target system and importing them
2. **Index rebuild** — Reconstructing the vector index (HNSW, IVFFlat, etc.)
3. **Re-embedding** — If model compatibility check fails, regenerating all vectors from source

Understanding the time cost of each phase is the foundation of a realistic RTO commitment.

Phase Timing Reference (approximate, hardware-dependent)

Operation	Scale	Typical Duration
pgBackRest physical restore	100 GB	15–30 min
pg_dump logical restore	100 GB	45–90 min

Operation	Scale	Typical Duration
HNSW index rebuild (10M vectors, 1536-dim)	—	20–45 min
Re-embedding via OpenAI API (10M chunks)	rate-limited	4–12 hours
Re-embedding via local sentence-transformers	GPU available	2–4 hours
Weaviate backup restore from S3	50 GB	10–20 min
Pinecone upsert re-import	10M vectors	3–6 hours

The re-embedding phase is the wildcard. If your recovery plan includes it, you need either pre-approved API rate limit increases with your embedding provider, or local model hosting capability. Planning for model-version-locked recovery avoids this phase entirely.

Automated Recovery Validation

After any restore, automated validation should confirm that the restored vectors actually work before the system accepts production traffic:

```
# Python
import numpy as np
from openai import OpenAI
import psycopg2
import os

def validate_embedding_restore(
    pg_conn_string: str,
    sample_size: int = 100,
    similarity_threshold: float = 0.85
) -> dict:
    """
    Post-restore validation: generate fresh embeddings for a sample
    of chunks and compare against stored embeddings.
    If avg cosine similarity < threshold, the restore is suspect.
    """
    openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
    conn = psycopg2.connect(pg_conn_string)
    cur = conn.cursor()

    # Sample chunks from the restored database
    cur.execute("""
        SELECT id, chunk_text, embedding::TEXT, model_id
        FROM document_chunks
        ORDER BY RANDOM()
        LIMIT %s
    """, (sample_size,))

    rows = cur.fetchall()
    chunk_ids = [r[0] for r in rows]
    chunk_texts = [r[1] for r in rows]
    stored_embeddings = [
        list(map(float, r[2].strip("[]").split(",")))
        for r in rows
    ]
    model_ids = [r[3] for r in rows]

    # Verify all samples use the same model
    unique_models = set(model_ids)
    if len(unique_models) > 1:
        return {
```

```
        "valid": False,
        "reason": f"Mixed model IDs in sample: {unique_models}",
        "action": "investigate_model_consistency"
    }

    model_id = unique_models.pop()

    # Re-embed the same texts using the current model
    response = openai_client.embeddings.create(
        input=chunk_texts,
        model=model_id
    )
    fresh_embeddings = [item.embedding for item in response.data]

    # Compute cosine similarities between stored and fresh
    similarities = []
    for stored, fresh in zip(stored_embeddings, fresh_embeddings):
        stored_arr = np.array(stored)
        fresh_arr = np.array(fresh)
        cosine_sim = np.dot(stored_arr, fresh_arr) / (
            np.linalg.norm(stored_arr) * np.linalg.norm(fresh_arr)
        )
        similarities.append(float(cosine_sim))

    avg_similarity = float(np.mean(similarities))
    min_similarity = float(np.min(similarities))
    below_threshold = [s for s in similarities if s < similarity_threshold]

    conn.close()

    valid = avg_similarity >= similarity_threshold and len(below_threshold) == 0

    return {
        "valid": valid,
        "model_id": model_id,
        "sample_size": sample_size,
        "avg_cosine_similarity": round(avg_similarity, 6),
        "min_cosine_similarity": round(min_similarity, 6),
        "below_threshold_count": len(below_threshold),
        "threshold": similarity_threshold,
        "action": "promote_to_production" if valid else "investigate_before_promote"
    }

# Run post-restore validation
result = validate_embedding_restore(
    pg_conn_string=os.environ["PG_CONN_STAGING"],
    sample_size=200,
    similarity_threshold=0.95
)
print(result)

# Block traffic promotion if validation fails
if not result["valid"]:
    raise RuntimeError(
        f"Restore validation failed: avg_similarity={result['avg_cosine_similarity']}, "
        f"{result['below_threshold_count']} vectors below threshold. "
        f"Action required: {result['action']}"
    )
```

A cosine similarity near 1.0 between stored and freshly computed embeddings confirms that the model producing embeddings during backup matches the model available at restore time. A lower value signals either a model version mismatch or data corruption. Set the threshold conservatively — 0.95 or higher — because even small embedding space shifts produce noticeably degraded retrieval quality in production RAG pipelines.

Backup Schedules and Retention Policies

The right backup schedule for an embedding workload depends on two factors that differ from traditional OLTP workloads: how expensive re-embedding is (which sets the floor on recovery point objective) and how frequently source data changes (which sets the cadence).

Recommended Backup Schedule Matrix

Workload Type	Full Backup	Incremental / WAL	Retention
Static corpus (rare updates)	Weekly	Daily incremental export	4 weeks full, 60 days incremental
Active RAG (daily ingestion)	Weekly	Continuous WAL (pgBackRest)	4 weeks full, 14 days WAL
High-churn (real-time ingestion)	Daily	Continuous WAL or CDC	7 days full, 48 hours WAL
Managed (Pinecone/Weaviate Cloud)	Weekly API export	N/A	4 exports retained

For managed services where you cannot enable WAL archiving, the weekly API export is the minimum viable posture. For Pinecone specifically, the operational recommendation is to keep your authoritative source data in a PostgreSQL or object storage backend that you control, so that a Pinecone export failure degrades into a re-embedding event rather than data loss.

SQL Server Mirror Pattern

Many enterprises run pgvector on PostgreSQL alongside SQL Server for their operational data. In those environments, the embedding backup schedule should be coordinated with the SQL Server backup schedule so that point-in-time recovery targets align:

```
-- SQL Server
-- Track embedding export events in SQL Server audit table
-- so DBA team has unified visibility across both systems
```

```
CREATE TABLE dbo.EmbeddingBackupAudit (
    AuditId          INT IDENTITY(1,1) PRIMARY KEY,
    BackupType       VARCHAR(50) NOT NULL, -- 'full', 'incremental', 'export'
    TargetSystem     VARCHAR(50) NOT NULL, -- 'pgvector', 'pinecone', 'weaviate'
    RowsExported     BIGINT,
    ModelHash        VARCHAR(64),
    BackupStartUtc   DATETIME2 NOT NULL,
    BackupEndUtc     DATETIME2,
    FilePath         NVARCHAR(500),
    Status           VARCHAR(20) NOT NULL DEFAULT 'STARTED',
    ErrorMessage     NVARCHAR(MAX),
    CONSTRAINT chk_status CHECK (Status IN ('STARTED','SUCCESS','FAILED'))
);
```

```
-- PostgreSQL (pgvector side)
-- Mirror audit table for unified cross-system backup reporting
CREATE TABLE embedding_backup_audit (
    audit_id         SERIAL PRIMARY KEY,
    backup_type      VARCHAR(50) NOT NULL,
    target_system    VARCHAR(50) NOT NULL,
    rows_exported    BIGINT,
    model_hash       VARCHAR(64),
    backup_start_utc TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    backup_end_utc   TIMESTAMPTZ,
```

```
file_path      TEXT,
status         VARCHAR(20) NOT NULL DEFAULT 'STARTED',
error_message  TEXT,
CONSTRAINT chk_status CHECK (status IN ('STARTED', 'SUCCESS', 'FAILED'))
);

-- Record backup event at start
INSERT INTO embedding_backup_audit (backup_type, target_system, model_hash)
VALUES ('full', 'pgvector', 'a3f9c2b1d7e4f012')
RETURNING audit_id;

-- Update on completion
UPDATE embedding_backup_audit
SET status = 'SUCCESS',
    backup_end_utc = NOW(),
    rows_exported = 9847231,
    file_path = '/backup/pgvector/full_20240615_020000.dump'
WHERE audit_id = 42;
```

Maintaining a unified audit trail across SQL Server and PostgreSQL lets operations teams answer the question “what was the consistent state of our entire data layer at 03:00 on June 15?” — which is exactly the question that comes up during a production incident.

Disaster Recovery Runbook

A backup strategy without a written runbook is an incomplete strategy. The runbook should be executable by a DBA who has never worked on the vector layer before, because incidents typically happen when the primary expert is unavailable.

DR Runbook: pgvector Full Restore

```
## pgvector Full Restore Runbook
## Estimated time: 45-90 minutes (data restore) + 20-45 minutes (index rebuild)

### Prerequisites
- pgBackRest installed on recovery target
- /etc/pgbackrest/pgbackrest.conf configured and tested
- VAULT access for cipher passphrase
- Recovery target PostgreSQL instance running (no data)

### Step 1: Identify recovery target
$ pgbackrest --stanza=rag-production info
# Note the backup labels and timestamps. Identify the backup to restore.

### Step 2: Stop PostgreSQL on target (if running)
$ systemctl stop postgresql@16-main

### Step 3: Restore from backup
$ pgbackrest --stanza=rag-production restore \
  --target="YYYY-MM-DD HH:MM:SS" \
  --target-action=promote \
  --recovery-option="recovery_target_inclusive=true"
```

```
### Step 4: Start PostgreSQL and verify
$ systemctl start postgresql@16-main
$ psql -c "SELECT count(*) FROM document_chunks;"
$ psql -c "SELECT max(embedded_at) FROM document_chunks;"

### Step 5: Check for orphaned embeddings
$ psql -c "SELECT count(*) FROM orphaned_embeddings;"
# Expected: 0. If nonzero, investigate before proceeding.

### Step 6: Validate embedding quality
$ python3 /ops/scripts/validate_embedding_restore.py \
    --sample-size=200 \
    --threshold=0.95
# Must return valid=True before proceeding.

### Step 7: Rebuild HNSW index if not using physical restore
$ psql -c "REINDEX INDEX CONCURRENTLY document_chunks_embedding_idx;"
# Monitor progress:
$ psql -c "SELECT phase, blocks_done, blocks_total FROM pg_stat_progress_create_index;"

### Step 8: Run smoke tests
$ python3 /ops/scripts/rag_smoke_test.py --env=recovery
# Performs 10 end-to-end RAG queries and validates result count and latency.

### Step 9: Promote to production
# Update DNS / load balancer target to point to recovery instance.
# Notify on-call team that recovery is complete.
```

The runbook format matters less than its completeness. The validation steps — orphaned embedding check, cosine similarity validation, smoke test — are the parts most often omitted in traditional DR runbooks and most important in embedding workloads.

Key Takeaways

- **Embedding backups are three-artifact problems.** Source data, vector arrays, and model version information must be backed up together and restored as a consistent set. A restore that produces vector data without model provenance is incomplete.
- **Physical backups preserve HNSW indices; logical backups do not.** For large vector tables where index rebuild time exceeds your RTO, pgBackRest with point-in-time recovery is the correct tool. For smaller deployments, pg_dump with documented index rebuild time is acceptable.
- **Managed vector databases require explicit export strategies.** Pinecone requires API-based batch export with no native backup capability. Weaviate provides a first-class backup API with cloud storage backends. Build your operational runbook around the specific capabilities of your chosen platform.

- **Model version compatibility must be checked before any restore is promoted to production.** A cosine similarity validation against fresh re-embeddings of a random sample will immediately surface model-space mismatches that would otherwise produce silently wrong retrieval results.
 - **Re-embedding is a valid recovery path, but its time cost must be planned.** At scale, regenerating embeddings through an external API can take hours. Factor API rate limits, local GPU availability, and chunk volume into your RTO commitment before declaring a recovery SLA to stakeholders.
-

Chapter 37: Cost Optimization: Dimensionality Reduction and Quantization

Storage and query costs in vector databases grow linearly with two variables: the number of vectors and the size of each vector. A production RAG system ingesting documents at scale can easily accumulate tens of millions of 1536-dimensional float32 vectors — roughly 90 GB of raw embedding data before any indexing overhead. This chapter covers the two primary techniques for controlling that growth without sacrificing retrieval quality: dimensionality reduction, which shrinks the number of dimensions per vector, and quantization, which shrinks the number of bits per dimension. Both approaches trade a small, measurable amount of recall for significant savings in storage, memory, and query latency. Understanding exactly where those tradeoffs land — and how to measure them — is the engineering work this chapter addresses.

The Cost Structure of Vector Storage

Before optimizing anything, understand what you're paying for. Vector databases carry costs along four axes that compound each other.

Raw storage is the floor. A float32 value occupies 4 bytes. A 1536-dimensional OpenAI `text-embedding-3-small` vector is 6,144 bytes (6 KB) per document. At 10 million documents, that's 58 GB before indexes. Switch to `text-embedding-3-large` at 3072 dimensions and you're at 117 GB. Pinecone, Weaviate, and Qdrant all charge for storage either directly or through node sizing, so this matters immediately.

Index memory multiplies the base storage. HNSW indexes — the most common structure across all major vector databases — maintain neighbor graphs that consume roughly $1.5\times$ to $4\times$ the raw vector memory depending on the `M` parameter (graph connectivity). A 58 GB float32 dataset can easily require 200+ GB of RAM to serve with HNSW at reasonable recall.

Query throughput cost ties to memory pressure. When working sets don't fit in RAM, queries hit disk, and latency spikes from single-digit milliseconds to hundreds. Cloud providers size instances to fit vector indexes in memory, so every unnecessary byte in your vectors translates directly to a larger, more expensive instance.

Embedding API costs sit upstream. OpenAI charges per token for embedding generation. If

you're re-embedding during pipeline changes or re-indexing after dimension changes, those API calls add up fast.

The table below shows approximate storage requirements for common embedding models at 10 million vectors:

Model	Dimensions	float32 Storage	int8 Storage	Binary Storage
text-embedding-3-small	1536	58 GB	14.6 GB	1.8 GB
text-embedding-3-large	3072	117 GB	29.3 GB	3.7 GB
text-embedding-ada-002	1536	58 GB	14.6 GB	1.8 GB
all-MiniLM-L6-v2	384	14.6 GB	3.7 GB	0.46 GB
Cohere embed-english-v3.0	1024	39 GB	9.8 GB	1.2 GB

The binary column is not a typo. That's the endgame of quantization: 1 bit per dimension instead of 32.

Dimensionality Reduction: PCA, Matryoshka, and Native Truncation

Dimensionality reduction eliminates dimensions from vectors, either by training a projection (PCA) or by using embedding models designed to work well at reduced sizes.

PCA-based reduction is the classical approach. You collect a representative sample of your vectors, fit a PCA transformation matrix, and project all vectors into a lower-dimensional space that preserves maximum variance. The projection matrix must be stored and applied to every new query vector at search time — this is the runtime cost of PCA.

```
# Python
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import normalize
import pickle

def fit_pca_reducer(sample_vectors: np.ndarray, target_dims: int) -> PCA:
    """
    Fit a PCA reducer on a sample of your corpus vectors.
    sample_vectors: shape (n_samples, original_dims)
    target_dims: number of dimensions to keep
    """
    pca = PCA(n_components=target_dims, random_state=42)
    pca.fit(sample_vectors)

    explained = pca.explained_variance_ratio_.sum()
    print(f"Explained variance with {target_dims} dims: {explained:.3f}")

    return pca

def reduce_and_normalize(vectors: np.ndarray, pca: PCA) -> np.ndarray:
    """
    Project and L2-normalize. Normalization is required after PCA
    if you use cosine similarity for search.
    """
    reduced = pca.transform(vectors)
    return normalize(reduced, norm='l2')
```

```

# Workflow: sample 100k vectors from your corpus
# (representative spread - don't use only recent documents)
sample = np.load("corpus_sample_100k.npy") # shape (100000, 1536)

pca = fit_pca_reducer(sample, target_dims=512)

# Save projection matrix for query-time use
with open("pca_reducer_1536_to_512.pkl", "wb") as f:
    pickle.dump(pca, f)

# Process your full corpus in batches
def process_batch(raw_batch: np.ndarray, pca: PCA) -> np.ndarray:
    return reduce_and_normalize(raw_batch, pca)

# Check how many dimensions you actually need
for dims in [128, 256, 512, 768, 1024]:
    test_pca = PCA(n_components=dims, random_state=42)
    test_pca.fit(sample)
    variance = test_pca.explained_variance_ratio_.sum()
    print(f" {dims} dims → {variance:.4f} explained variance ({variance*100:.1f}%)")

```

Running this on `text-embedding-3-small` vectors typically yields 95%+ explained variance at 512 dimensions. The problem is that explained variance doesn't directly map to recall — you need to measure recall separately, which means running benchmark queries before committing to a target dimension count.

Matryoshka Representation Learning (MRL) changes the game. OpenAI's newer models (`text-embedding-3-small`, `text-embedding-3-large`) and Cohere's v3 models are trained with MRL, meaning the first N dimensions of the full vector are themselves a valid, high-quality embedding. You can truncate rather than project. No projection matrix required, no transformation at query time, and the quality at reduced dimensions is better than PCA because the model was explicitly trained for it.

```

# Python
from openai import OpenAI
import numpy as np

client = OpenAI()

def embed_with_truncation(
    texts: list[str],
    model: str = "text-embedding-3-small",
    dimensions: int = 512 # native truncation - MRL models only
) -> np.ndarray:
    """
    OpenAI supports native dimension truncation via the `dimensions` parameter.
    Valid only for text-embedding-3-* models.
    """
    response = client.embeddings.create(
        model=model,
        input=texts,
        dimensions=dimensions # API truncates and re-normalizes for you
    )
    return np.array([item.embedding for item in response.data])

# Compare quality across dimension counts
test_queries = [
    "database indexing strategies",
    "vector similarity search performance",
    "cost optimization for embeddings"
]

for dims in [256, 512, 768, 1536]:
    embeddings = embed_with_truncation(test_queries, dimensions=dims)
    print(f"dims={dims}: shape={embeddings.shape}, "
          f"norm={np.linalg.norm(embeddings[0]):.4f}")

```

```
# For text-embedding-3-large: max 3072, usable at 256, 512, 1024, 3072
# OpenAI's benchmarks show 512-dim 3-small outperforms 1536-dim ada-002
# on MTEB - smaller is sometimes better when MRL-trained
```

The operational advantage here is significant: you can change your target dimension count by updating the API parameter and re-indexing, with no separate transformation pipeline to maintain.

Choosing a target dimension is an empirical question. The practical process:

1. Generate embeddings at your target dimension and at full dimension for a held-out query set (minimum 500 queries with known relevant documents).
2. Run nearest-neighbor retrieval at both sizes with identical HNSW parameters.
3. Compute Recall@10 at both sizes.
4. Accept the reduction if recall drop is under 3–5% for your use case.

For most RAG applications using MRL-trained models, 512 dimensions achieves recall within 2% of 1536 while cutting storage and index memory by two thirds.

Scalar Quantization: float32 to int8

Quantization reduces the number of bits per dimension. Scalar quantization maps each float32 value to an integer within a fixed range — typically int8 (256 levels) or even uint8. The loss in precision is real but bounded, and the memory savings are immediate: 4 bytes becomes 1 byte per dimension, a 4× reduction.

The math is straightforward. For a vector where values span $[-1.0, 1.0]$, each float32 is mapped to an int8 using:

```
quantized = round(float_value × 127)
dequantized = quantized / 127
```

Most vector databases handle this transparently. Qdrant’s scalar quantization is production-grade and simple to configure:

```
# Python
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance,
    ScalarQuantizationConfig, ScalarType,
    QuantizationSearchParams
)

client = QdrantClient(url="http://localhost:6333")

# Create collection with int8 scalar quantization
client.create_collection(
    collection_name="documents_sq",
    vectors_config=VectorParams(
        size=1536,
        distance=Distance.COSINE
    ),
    quantization_config=ScalarQuantizationConfig(
        type=ScalarType.INT8,
        quantile=0.99,      # clip outliers: 99th percentile maps to max int8
        always_ram=True    # keep quantized index in RAM, fetch originals from disk
    )
)
```

```

# always_ram=True is the key production setting:
# - Search uses quantized (1-byte) vectors for fast candidate selection
# - Top candidates are re-scored with original float32 vectors
# - Original vectors can live on disk (SSD), quantized in RAM
# This is "quantization with rescoring" - near full recall at fraction of memory

# Search with rescoring enabled (default when quantization is configured)
results = client.search(
    collection_name="documents_sq",
    query_vector=[0.1] * 1536, # your actual query embedding here
    limit=10,
    search_params=QuantizationSearchParams(
        ignore=False, # use quantization for initial search
        rescore=True, # rescore top candidates with originals
        oversampling=2.0 # fetch 20 candidates, rescore to return 10
    )
)

# oversampling controls the recall/speed tradeoff:
# 1.0 → fastest, lowest recall
# 3.0 → slower, highest recall (approaches float32 quality)
# 2.0 is the practical default for most production use cases

print(f"Retrieved {len(results)} results")
for r in results:
    print(f" id={r.id}, score={r.score:.4f}")

```

pgvector's quantization arrived in version 0.7.0 with halfvec (float16) support and binary quantization:

```

-- PostgreSQL / pgvector

-- Option 1: halfvec - stores float16, halves storage vs float32
-- Requires pgvector >= 0.7.0

CREATE TABLE documents_half (
    id          BIGSERIAL PRIMARY KEY,
    content     TEXT,
    embedding   HALFVEC(1536) -- 2 bytes per dim instead of 4
);

-- HNSW index on halfvec - same syntax as VECTOR
CREATE INDEX ON documents_half
USING hnsw (embedding halfvec_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Query: cast your float32 query to halfvec at search time
SELECT id, content, embedding <=> $1::HALFVEC AS distance
FROM documents_half
ORDER BY distance
LIMIT 10;

-- Option 2: Binary quantization with BIT type
-- Each dimension becomes 1 bit: positive float → 1, negative → 0
-- Requires pgvector >= 0.7.0

CREATE TABLE documents_binary (
    id          BIGSERIAL PRIMARY KEY,
    content     TEXT,
    embedding_full VECTOR(1536), -- keep original for rescoring
    embedding_bin BIT(1536)      -- binary quantized
);

-- Function to binarize a vector
CREATE OR REPLACE FUNCTION binarize(v VECTOR) RETURNS BIT AS $$
DECLARE
    result BIT;
    i      INT;

```

```

arr    FLOAT[];
bits   TEXT := '';
BEGIN
arr := ARRAY(SELECT unnest(v::FLOAT[]));
FOR i IN 1..array_length(arr, 1) LOOP
    IF arr[i] >= 0 THEN
        bits := bits || '1';
    ELSE
        bits := bits || '0';
    END IF;
END LOOP;
RETURN bits::BIT(1536);
END;
$$ LANGUAGE plpgsql IMMUTABLE;

-- Index on binary vectors using Hamming distance
CREATE INDEX ON documents_binary
USING hnsw (embedding_bin bit_hamming_ops)
WITH (m = 16, ef_construction = 64);

-- Two-phase search: binary for candidate selection, float32 for rescoring
WITH binary_candidates AS (
    SELECT id, embedding_full,
           bit_count(embedding_bin # $2::BIT(1536)) AS hamming_dist
    FROM documents_binary
    ORDER BY hamming_dist
    LIMIT 100 -- oversample: get 100 via binary search
)
SELECT id, embedding_full <=> $1::VECTOR AS cosine_dist
FROM binary_candidates
ORDER BY cosine_dist
LIMIT 10; -- rerank top 100 by exact cosine, return 10

-- $1 = float32 query vector, $2 = binary query vector (same query, both forms)

```

The two-phase pattern — coarse binary search, exact float32 reranking — is the same pattern Qdrant calls “rescoring.” It’s the standard production approach for binary quantization and delivers recall within 1–3% of full float32 search at a fraction of the memory cost.

Product Quantization and Vector Database Native Support

Product quantization (PQ) goes further than scalar quantization by dividing each high-dimensional vector into subvectors and quantizing each subvector against a learned codebook. The result is a compact code — often 8–32 bytes for a 1536-dimensional vector — that approximates the original with reasonable accuracy.

PQ is more complex than scalar quantization: it requires a training phase to build codebooks, and the codebooks must be stored and used at query time. The payoff is compression ratios of 32× to 96× with recall that holds up better than naive binary quantization for unoptimized models.

Faiss, the underlying library in many vector databases, exposes PQ directly. Weaviate and Milvus expose it through their configuration APIs:

```

# Python - Milvus PQ configuration via pymilvus
from pymilvus import (
    connections, FieldSchema, CollectionSchema,
    DataType, Collection, utility
)

connections.connect(host="localhost", port="19530")

```

```

fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
    FieldSchema(name="text", dtype=DataType.VARCHAR, max_length=4096),
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536)
]

schema = CollectionSchema(fields=fields, description="PQ quantized documents")
collection = Collection(name="documents_pq", schema=schema)

# IVF_PQ index: inverted file + product quantization
# This is the standard high-compression option in Milvus/Faiss
index_params = {
    "metric_type": "IP",          # inner product (use normalized vectors for cosine)
    "index_type": "IVF_PQ",
    "params": {
        "nlist": 1024,          # number of IVF clusters (scale with sqrt(n_vectors))
        "m": 16,               # number of subvectors (must divide dim evenly: 1536/16=96)
        "nbits": 8             # bits per subvector code (8 = 256 centroids per subquantizer)
    }
}

# Memory per vector: m * (nbits/8) bytes = 16 * 1 = 16 bytes vs 6144 bytes float32
# Compression ratio: 384x

collection.create_index(field_name="embedding", index_params=index_params)
collection.load()

# Search - nprobe controls how many IVF clusters to check
# More clusters → higher recall, higher latency
search_params = {
    "metric_type": "IP",
    "params": {"nprobe": 64}    # check 64 of 1024 clusters; 6.25% of index
}

results = collection.search(
    data=[[0.1] * 1536],      # query embedding (normalized)
    anns_field="embedding",
    param=search_params,
    limit=10,
    output_fields=["text"]
)

for hits in results:
    for hit in hits:
        print(f"id={hit.id}, score={hit.score:.4f}, text={hit.entity.get('text')[:80]}")

# PQ recall benchmarking guide:
# nprobe=10 → ~85% recall@10, fastest
# nprobe=64 → ~93% recall@10, good default
# nprobe=256 → ~97% recall@10, slowest (approaching brute force)
# Tune nprobe to hit your latency SLA, then evaluate recall

```

Weaviate exposes its own PQ interface through its schema configuration, which is worth knowing if you're running Weaviate in production:

```

# Python - Weaviate PQ configuration via weaviate-client v4
import weaviate
from weaviate.classes.config import (
    Configure, Property, DataType, VectorDistances
)

client = weaviate.connect_to_local()

# Create collection with PQ enabled
client.collections.create(
    name="DocumentsPQ",
    vectorizer_config=Configure.Vectorizer.none(),    # bring your own vectors
    vector_index_config=Configure.VectorIndex.hnsw(
        distance_metric=VectorDistances.COSINE,
        quantizer=Configure.VectorIndex.Quantizer.pq(

```

```

        training_limit=100_000,    # vectors needed before PQ kicks in
        segments=96,              # 1536 dims / 96 segments = 16 dims per segment
        centroids=256              # 2^8 centroids (8-bit codes)
    ),
    properties=[
        Property(name="content", data_type=DataType.TEXT),
        Property(name="source", data_type=DataType.TEXT)
    ]
)

# Weaviate trains PQ automatically once training_limit vectors are loaded
# Before training_limit: stores float32 flat (temporary, high memory)
# After training_limit: compresses existing vectors + new vectors

# For production: pre-populate with training_limit vectors before going live
# Or set a lower training_limit if your corpus starts small

client.close()

```

The practical PQ tradeoff: you’re trading recall and query-time CPU for memory. PQ requires decoding compact codes during distance computation, which is CPU-intensive. On modern hardware with SIMD support, Faiss makes this fast — but it’s not free. Benchmark both latency and recall before committing.

Measuring the Tradeoffs: A Recall Benchmarking Framework

Every compression decision requires recall validation. “Recall@K” measures what fraction of the true top-K nearest neighbors you retrieve when using an approximate or compressed index. This is your primary metric.

```

# Python
import numpy as np
from typing import Callable
import time

def compute_recall_at_k(
    queries: np.ndarray,
    corpus: np.ndarray,
    search_fn: Callable,    # function: query_vector -> list of retrieved IDs
    k: int = 10,
    batch_size: int = 100
) -> dict:
    """
    Computes Recall@K by comparing approximate search results
    against brute-force exact neighbors.

    queries:      shape (n_queries, dims)
    corpus:       shape (n_corpus, dims) - the indexed vectors
    search_fn:    your vector DB search call, returns list of IDs
    """

    # Ground truth: exact brute-force neighbors
    # For large corpora, compute this once and cache it
    print("Computing ground truth (brute force)...")
    ground_truth = []

    # Batch to avoid OOM on large corpora
    for q_start in range(0, len(queries), batch_size):
        q_batch = queries[q_start:q_start + batch_size]

        # Cosine similarity via dot product (assumes normalized vectors)
        similarities = q_batch @ corpus.T    # (batch, n_corpus)

```



```

top_k_indices = np.argsort(-similarities, axis=1)[: , :k]
ground_truth.extend(top_k_indices.tolist())

# Approximate search with the system under test
print("Running approximate search...")
retrieved = []
latencies = []

for i, query in enumerate(queries):
    start = time.perf_counter()
    result_ids = search_fn(query)
    elapsed = time.perf_counter() - start
    latencies.append(elapsed * 1000) # ms
    retrieved.append(result_ids[:k])

# Recall@K: what fraction of ground truth top-K did we find?
recall_scores = []
for true_ids, ret_ids in zip(ground_truth, retrieved):
    true_set = set(true_ids)
    ret_set = set(ret_ids)
    recall = len(true_set & ret_set) / len(true_set)
    recall_scores.append(recall)

return {
    "recall_at_k": np.mean(recall_scores),
    "recall_p50": np.percentile(recall_scores, 50),
    "recall_p10": np.percentile(recall_scores, 10), # worst queries
    "latency_p50_ms": np.percentile(latencies, 50),
    "latency_p95_ms": np.percentile(latencies, 95),
    "latency_p99_ms": np.percentile(latencies, 99),
    "k": k
}

# Example: compare float32 vs int8 quantized search in Qdrant
from qdrant_client import QdrantClient
from qdrant_client.models import QuantizationSearchParams

qdrant = QdrantClient(url="http://localhost:6333")

def search_float32(query: np.ndarray) -> list:
    """Search without quantization."""
    results = qdrant.search(
        collection_name="documents_float32",
        query_vector=query.tolist(),
        limit=10, search_params=QuantizationSearchParams(ignore=True) # disable quantization
    )
    return [r.id for r in results]

def search_int8_rescore(query: np.ndarray) -> list:
    """Search with int8 quantization + rescoring."""
    results = qdrant.search(
        collection_name="documents_sq",
        query_vector=query.tolist(),
        limit=10,
        search_params=QuantizationSearchParams(
            ignore=False,
            rescore=True,
            oversampling=2.0
        )
    )
    return [r.id for r in results]

def search_int8_no_rescore(query: np.ndarray) -> list:
    """Search with int8 quantization only - maximum speed."""
    results = qdrant.search(
        collection_name="documents_sq",
        query_vector=query.tolist(),
        limit=10,
        search_params=QuantizationSearchParams(
            ignore=False,

```

```

        rescore=False
    )
)
return [r.id for r in results]

# Load your benchmark queries
queries = np.load("benchmark_queries.npy")      # shape (1000, 1536)
corpus = np.load("corpus_vectors_sample.npy")    # shape (50000, 1536) - same IDs as DB

configs = {
    "float32_exact":      search_float32,
    "int8_with_rescore":  search_int8_rescore,
    "int8_no_rescore":    search_int8_no_rescore,
}

results = {}
for name, fn in configs.items():
    print(f"\nBenchmarking: {name}")
    results[name] = compute_recall_at_k(queries, corpus, fn, k=10)

# Print comparison table
print(f"\n{'Config':<25} {'Recall@10':>10} {'P10 Recall':>10} {'P50 Lat ms':>12} {'P95 Lat ms':>12}")
print("-" * 75)
for name, metrics in results.items():
    print(f"{'name':<25} {'metrics['recall_at_k']:>10.4f} "
          f"{'metrics['recall_p10']:>10.4f} "
          f"{'metrics['latency_p50_ms']:>12.2f} "
          f"{'metrics['latency_p95_ms']:>12.2f}")

```

The P10 recall column — the 10th percentile recall across your query set — is more informative than the mean. A mean recall of 0.95 can hide specific query categories that consistently miss. Those outliers often correspond to semantically sparse topics, short queries, or domain-specific vocabulary your embedding model handles poorly. Investigate them.

What to do when recall is too low:

- Increase `oversampling` (Qdrant) or `nprobe` (Milvus/Faiss) — the fastest fix
- Increase `ef` at query time on HNSW indexes — trades latency for recall
- Switch from binary to int8 quantization — better recall at more memory cost
- Use a higher-quality embedding model — more information preserved at same dimensions
- Increase target dimensions if using MRL truncation — recall usually improves monotonically

Combining Both Techniques: Dimension Reduction + Quantization

The highest compression ratios come from stacking both techniques: reduce dimensions first, then quantize. The order matters — quantize the reduced vectors, not the original ones.

```

# Python - Combined pipeline: MRL truncation + int8 in Qdrant

from openai import OpenAI
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance,
    ScalarQuantizationConfig, ScalarType,
    QuantizationSearchParams, PointStruct
)
import numpy as np
import uuid

```

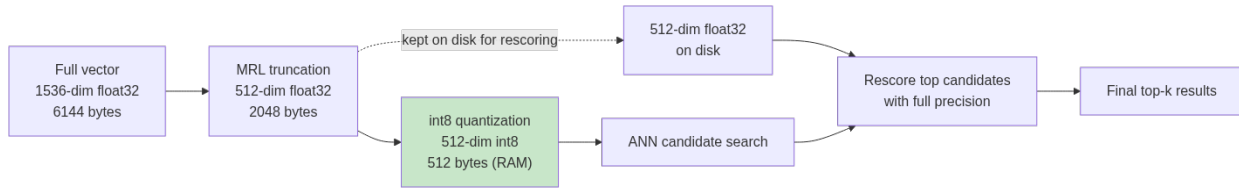


Figure 27: Dimension reduction happens first, then quantization compresses the already-reduced vector — reducing the original would discard information the truncation already preserved.

```

openai_client = OpenAI()
qdrant = QdrantClient(url="http://localhost:6333")

TARGET_DIMS = 512  # MRL truncation: 1536 → 512 (3.3× smaller)
# Combined with int8: 512 × 1 byte = 512 bytes vs 1536 × 4 = 6144 bytes → 12× smaller

# Create the collection: 512-dim vectors with int8 quantization
qdrant.recreate_collection(
    collection_name="documents_optimized",
    vectors_config=VectorParams(
        size=TARGET_DIMS,
        distance=Distance.COSINE
    ),
    quantization_config=ScalarQuantizationConfig(
        type=ScalarType.INT8,
        quantile=0.99,
        always_ram=True
    )
)

def embed_documents(texts: list[str]) -> np.ndarray:
    """Embed at target dimensions using MRL truncation."""
    response = openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=texts,
        dimensions=TARGET_DIMS
    )
    return np.array([item.embedding for item in response.data])

def ingest_batch(texts: list[str], ids: list[str] = None):
    """Embed and upsert a batch of documents."""
    embeddings = embed_documents(texts)

    if ids is None:
        ids = [str(uuid.uuid4()) for _ in texts]

    points = [
        PointStruct(
            id=id_,
            vector=embedding.tolist(),
            payload={"text": text}
        )
        for id_, text, embedding in zip(ids, texts, embeddings)
    ]

    qdrant.upsert(collection_name="documents_optimized", points=points)
    return ids

def search_optimized(query_text: str, top_k: int = 10) -> list[dict]:
    """
    Embed query at same truncated dimension, search with rescoring.
    """
    query_embedding = embed_documents([query_text])[0]

    results = qdrant.search(

```

```

collection_name="documents_optimized",
query_vector=query_embedding.tolist(),
limit=top_k,
search_params=QuantizationSearchParams(
    rescore=True,
    oversampling=2.0
),
with_payload=True
)

return [
    {"id": r.id, "score": r.score, "text": r.payload.get("text", "")}
    for r in results
]

# Storage math for this configuration:
# Original: 1536 dims × 4 bytes = 6,144 bytes per vector
# Optimized: 512 dims × 1 byte = 512 bytes per vector (quantized in RAM)
# + 512 dims × 4 bytes = 2,048 bytes on disk for rescoring originals
# RAM savings: 12× for the hot index; disk holds originals at 3×

```

A practical comparison across compression strategies for 10 million vectors:

Strategy	RAM (index only)	Disk	Recall@10	Query Latency
float32, 1536 dims	~200 GB	58 GB	0.99	baseline
float32, 512 dims (MRL)	~67 GB	19 GB	0.97	faster
int8, 1536 dims + rescore	~50 GB	58 GB	0.97	similar
int8, 512 dims + rescore	~17 GB	19 GB	0.95	faster
binary, 1536 dims + rescore	~6 GB	58 GB	0.93	fastest search
binary, 512 dims + rescore	~2 GB	19 GB	0.90	fastest overall

The int8 + 512 dims row represents the practical optimum for most production RAG systems: it cuts RAM by more than 10× versus full float32 HNSW, preserves 95% recall with rescoring, and keeps query latency low. Binary quantization pushes further but requires more careful recall validation — acceptable for some use cases, not for others.

pgvector Optimization Workflow

For teams running vector search inside PostgreSQL via pgvector, the optimization path looks different from managed vector databases but the principles are identical.

```

-- PostgreSQL / pgvector - full optimization workflow

-- Step 1: Baseline - full float32, HNSW
CREATE TABLE documents (
    id          BIGSERIAL PRIMARY KEY,
    content     TEXT,
    source      TEXT,
    embedding   VECTOR(1536)
);

CREATE INDEX idx_docs_embedding ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Step 2: Migrate to halfvec for 2× storage savings
-- (pgvector >= 0.7.0)

```

```

ALTER TABLE documents ADD COLUMN embedding_half HALFVEC(1536);

-- Backfill: cast existing vectors (batch this in production)
UPDATE documents SET embedding_half = embedding::HALFVEC;

-- Create HNSW on halfvec
CREATE INDEX idx_docs_half ON documents
USING hnsw (embedding_half halfvec_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Test: compare result sets between full and half precision
WITH full_results AS (
    SELECT id, embedding <=> '[0.1, ...]':VECTOR AS dist_full
    FROM documents ORDER BY dist_full LIMIT 20
),
half_results AS (
    SELECT id, embedding_half <=> '[0.1, ...]':HALFVEC AS dist_half
    FROM documents ORDER BY dist_half LIMIT 20
)
SELECT
    COALESCE(f.id, h.id) AS id,
    f.dist_full,
    h.dist_half,
    CASE WHEN f.id IS NULL THEN 'only in half'
         WHEN h.id IS NULL THEN 'only in full'
         ELSE 'in both' END AS status
FROM full_results f
FULL OUTER JOIN half_results h USING (id)
ORDER BY COALESCE(f.dist_full, h.dist_half);

-- Step 3: Dimension reduction at the application layer + halfvec
-- If using MRL truncation, store 512-dim halfvec directly
ALTER TABLE documents ADD COLUMN embedding_512h HALFVEC(512);

-- This column gets populated from the application with 512-dim embeddings
-- Storage: 512 * 2 bytes = 1024 bytes vs 1536 * 4 = 6144 bytes + 6x reduction

CREATE INDEX idx_docs_512h ON documents
USING hnsw (embedding_512h halfvec_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- HNSW query-time recall tuning via session parameter
SET hnsw.ef_search = 100; -- default is 40; increase for better recall
                        -- tune per query if needed

SELECT id, content,
       embedding_512h <=> $1::HALFVEC AS distance
FROM documents
ORDER BY distance
LIMIT 10;

-- Step 4: Measure index sizes
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    pg_size_pretty(pg_total_relation_size(indexrelid)) AS total_size
FROM pg_stat_user_indexes
WHERE relname = 'documents'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Step 5: IVFFlat as lower-memory alternative to HNSW
-- HNSW has better recall-latency tradeoff but uses more memory for the graph
-- IVFFlat uses less memory, needs lists tuning

CREATE INDEX idx_docs_ivfflat ON documents
USING ivfflat (embedding_512h halfvec_cosine_ops)
WITH (lists = 200); -- rule of thumb: sqrt(n_rows) for up to 1M rows
                  -- use 10*sqrt(n_rows) for larger datasets

-- IVFFlat requires probes at query time:

```

```
SET ivfflat.probes = 20;    -- check 20 of 200 lists (10%); tune for recall/speed

SELECT id, content,
       embedding_512h <=> $1::HALFVEC AS distance
FROM documents
ORDER BY distance
LIMIT 10;
```

The SQL Server pairing for pgvector doesn't have a native equivalent — SQL Server does not include a production-grade vector index type as of 2024. For teams standardized on SQL Server who need vector search, the practical options are Azure AI Search (a managed service with its own vector index support) or a sidecar vector database alongside SQL Server for the embedding workload.

Operational Checklist: Before You Compress

Compression decisions are difficult to reverse once data is in production. Run through this checklist before changing your vector storage strategy:

- 1. Benchmark with your actual queries, not synthetic ones.** Random query vectors don't represent the distribution of queries your users send. Use a sample of real production queries or, at minimum, queries from your domain.
- 2. Establish a recall floor, not just a recall target.** Decide in advance: what is the minimum acceptable Recall@10 for your application? If your RAG system fails to retrieve the right context, downstream LLM quality degrades. Set the floor before seeing the numbers, or you'll be tempted to rationalize whatever the benchmark shows.
- 3. Test re-indexing time.** Changing dimensions or quantization type requires re-embedding and re-indexing. For 10 million vectors, this can take hours. Know your re-indexing window before committing to a strategy that might need to change.
- 4. Keep originals if you use binary quantization.** Binary quantization without rescoring loses too much recall for most applications. Rescoring requires access to original float32 vectors. If you compress and discard originals, you lose the ability to rescore.
- 5. Stage changes through a shadow index.** Build the new compressed index in parallel, run your recall benchmark against both indexes simultaneously, and cut over only after validating recall. Never optimize directly on the production index.
- 6. Monitor recall in production, not just at launch.** Query distributions shift over time. A compression strategy that meets your recall floor at launch may degrade as users ask questions about topics not well-represented in your corpus sample. Build a production recall monitor using human-labeled query-document pairs sampled weekly.

Key Takeaways

- **Storage and memory costs scale with both vector count and vector size:** the two primary levers for controlling cost are dimensionality reduction (fewer dimensions) and

quantization (fewer bits per dimension), and they can be stacked for compounding savings.

- ****MRL-trained models (OpenAI text-embedding-3-*, Cohere v3) enable native truncation****: you can reduce from 1536 to 512 dimensions by setting a single API parameter, with better quality than PCA projection because the model was explicitly trained for it.
 - **Scalar quantization (int8) with rescoring is the lowest-risk compression step**: it delivers 4× memory reduction with recall loss typically under 3%, using the original float32 vectors stored on disk for rescoring top candidates identified by the compressed index.
 - **The practical production optimum for most RAG systems is 512-dim MRL + int8 quantization**: this combination achieves roughly 12× memory reduction versus full float32 HNSW while preserving approximately 95% Recall@10, and it is straightforward to implement in both Qdrant and pgvector.
 - **Recall benchmarking must use real queries and must track the P10 recall, not just the mean**: a high average recall can hide systematically poor retrieval on specific query categories, and those failures are the ones that degrade downstream LLM response quality in production.
-

Chapter 38: Edge Deployment of Vector Search

The push to run vector search closer to the data source—on mobile devices, embedded hardware, IoT gateways, retail point-of-sale systems, and CDN edge nodes—represents the next frontier for AI-enabled applications. Latency budgets that tolerate 200ms round trips to a central cloud region become unacceptable when you’re building real-time product recommendations in a store with spotty connectivity, or semantic search on a drone with no uplink at all. This chapter examines the architectural patterns, tooling choices, and operational tradeoffs of deploying vector search at the edge: what “edge” actually means in this context, which vector engines survive the constraints of limited RAM and CPU, how to synchronize edge indexes with a central store, and how to build applications that degrade gracefully when the edge goes offline.

What “Edge” Means for Vector Search

Edge is not a single topology. The term covers at least four distinct deployment tiers, each with different resource profiles and connectivity expectations:

Tier 1: Browser and mobile client. WebAssembly runtimes, React Native apps, and native iOS/Android applications. RAM is 256MB to 2GB usable, storage is flash, and the device may have no network connection for extended periods. The vector index must be embedded in the application bundle or downloaded on first run.

Tier 2: Edge server / CDN PoP. Cloudflare Workers, AWS Lambda@Edge, Fastly Compute@Edge. Memory is severely constrained (128MB to 512MB per worker isolate), and you cannot run a persistent process. Vector search here means either a very small in-memory index loaded at cold start, or a sidecar service running alongside the worker runtime.

Tier 3: On-premise gateway / industrial edge. A Raspberry Pi 4, NVIDIA Jetson, or Intel NUC sitting in a retail store, factory floor, or hospital wing. These have 4–16GB RAM, local NVMe storage, and intermittent WAN connectivity. This is the most capable edge tier and can run a full vector database instance.

Tier 4: Near-edge / regional. A small VM or bare-metal server in a regional co-location facility, 10–30ms from end users rather than 80–120ms to a central cloud region. Full database capabilities, but still subject to the deployment patterns that make edge useful.

The database analogy is familiar: this mirrors the read-replica topology DBAs have used for

decades. The central write master holds the source of truth; edge replicas serve reads locally. Vector databases add one wrinkle—the index is not just a data structure but an expensive artifact that takes time and compute to build. You cannot simply stream WAL records and reconstruct an HNSW graph incrementally the way you replay row changes into a B-tree.

Tier	RAM	Storage	Connectivity	Can run
Browser/mobile	256MB–2GB	Flash	Intermittent	SQLite-vec, Chroma (tiny)
CDN edge worker	128–512MB	None (ephemeral)	Always-on	Static index shard
On-premise gateway	4–16GB	NVMe	Intermittent WAN	Qdrant, SQLite-vec, pgvector (lite)
Regional near-edge	32–256GB	NVMe/SAN	Reliable	Full Qdrant, Weaviate, pgvector

Understanding which tier you’re targeting determines almost every subsequent decision: which engine to use, how to handle index updates, and how to write application code that degrades gracefully.

Choosing a Vector Engine for Constrained Environments

The dominant cloud vector databases—Pinecone, Weaviate Cloud, Qdrant Cloud—are designed for servers with abundant RAM and assume reliable network access to their control planes. Deploying them at Tier 1 or Tier 2 is not viable. Even Tier 3 requires careful selection.

SQLite-vec is the most portable option. It is a SQLite extension written in C that adds a `vec0` virtual table implementing flat KNN search and a quantized IVF-style index. The entire extension is a single shared library under 200KB. Because SQLite runs in-process, there is no network hop, no daemon to manage, and no port to expose. It is the right choice for mobile and browser-wasm deployments.

```
# Python
# SQLite-vec: embedded vector search for mobile/edge Python applications
# pip install sqlite-vec

import sqlite3
import sqlite_vec
import numpy as np
from sentence_transformers import SentenceTransformer

def build_local_index(documents: list[dict], db_path: str = "edge_search.db"):
    """
    Build a local vector index using SQLite-vec.
    Appropriate for Tier 3 edge gateways or bundled desktop applications.
    documents: list of {"id": str, "text": str, "metadata": dict}
    """
    model = SentenceTransformer("all-MiniLM-L6-v2") # 80MB, runs CPU-only
    embeddings = model.encode([d["text"] for d in documents], normalize_embeddings=True)
    dim = embeddings.shape[1] # 384 for MiniLM
```

```

conn = sqlite3.connect(db_path)
sqlite_vec.load(conn) # register the extension

conn.execute("DROP TABLE IF EXISTS vec_items")
conn.execute("DROP TABLE IF EXISTS item_metadata")

# Create the virtual vector table
conn.execute("""
    CREATE VIRTUAL TABLE vec_items USING vec0(
        item_id INTEGER PRIMARY KEY,
        embedding FLOAT[{dim}]
    )
""")

# Store metadata in a regular SQLite table (join at query time)
conn.execute("""
    CREATE TABLE item_metadata (
        item_id INTEGER PRIMARY KEY,
        doc_id TEXT,
        text TEXT,
        payload TEXT -- JSON blob for arbitrary metadata
    )
""")

import json
for i, (doc, emb) in enumerate(zip(documents, embeddings)):
    conn.execute(
        "INSERT INTO vec_items(item_id, embedding) VALUES (?, ?)",
        [i, emb.astype(np.float32).tobytes()]
    )
    conn.execute(
        "INSERT INTO item_metadata(item_id, doc_id, text, payload) VALUES (?, ?, ?, ?)",
        [i, doc["id"], doc["text"], json.dumps(doc.get("metadata", {}))]
    )

conn.commit()
print(f"Indexed {len(documents)} documents, dim={dim}, db_path={db_path}")
print(f"DB size: {os.path.getsize(db_path) / 1024 / 1024:.1f} MB")
conn.close()

def query_local_index(query_text: str, top_k: int = 5, db_path: str = "edge_search.db"):
    """
    Query the local SQLite-vec index. No network required.
    Typical latency: 2-15ms for 50K vectors on Raspberry Pi 4.
    """
    model = SentenceTransformer("all-MiniLM-L6-v2")
    query_vec = model.encode([query_text], normalize_embeddings=True)[0]

    conn = sqlite3.connect(db_path)
    sqlite_vec.load(conn)

    results = conn.execute("""
        SELECT
            m.doc_id,
            m.text,
            m.payload,
            v.distance
        FROM vec_items v
        JOIN item_metadata m ON m.item_id = v.item_id
        WHERE v.embedding MATCH ?
        AND k = ?
        ORDER BY v.distance
    """, [query_vec.astype(np.float32).tobytes(), top_k]).fetchall()

    conn.close()
    return [
        {"doc_id": r[0], "text": r[1], "metadata": r[2], "distance": r[3]}
        for r in results
    ]

```

```

import os

# Example usage
if __name__ == "__main__":
    sample_docs = [
        {"id": "p001", "text": "Wireless noise-canceling headphones with 30-hour battery", "metadata":
    ↪ {"category": "electronics", "price": 299}},
        {"id": "p002", "text": "Running shoes with carbon fiber plate for marathon training",
    ↪ "metadata": {"category": "footwear", "price": 180}},
        {"id": "p003", "text": "Stainless steel French press coffee maker 1 liter", "metadata":
    ↪ {"category": "kitchen", "price": 45}},
    ]
    build_local_index(sample_docs)
    hits = query_local_index("headphones for long flights")
    for h in hits:
        print(f"{h['doc_id']} | score={h['distance']:.4f} | {h['text'][:60]}")

```

Qdrant ships as a single statically linked binary (~50MB) and is the best choice for Tier 3 on-premise gateways. It supports HNSW indexing, scalar quantization (SQ8), and binary quantization—all of which reduce the RAM footprint dramatically. A collection of 500K 1536-dimensional vectors (OpenAI ada-002 size) uses approximately 3GB uncompressed but drops to under 800MB with binary quantization, making it viable on a device with 4GB total RAM.

```

# shell
# Deploy Qdrant on a Raspberry Pi 4 / Jetson / x86 mini-PC
# Using the static binary release (no Docker required on constrained hardware)

# Download the ARM64 binary (replace with x86_64 for Intel NUC)
QDRANT_VERSION="v1.9.2"
ARCH="aarch64-unknown-linux-musl"
wget -q "https://github.com/qdrant/qdrant/releases/download/${QDRANT_VERSION}/qdrant-${ARCH}.tar.gz"
tar -xzf "qdrant-${ARCH}.tar.gz"
chmod +x qdrant

# Minimal config for edge deployment - conservative memory usage
cat > config/config.yaml << 'EOF'
storage:
  storage_path: /data/qdrant/storage
  # Limit WAL to prevent filling flash storage
  wal:
    wal_capacity_mb: 32
    wal_segments_ahead: 0

service:
  host: 0.0.0.0
  http_port: 6333
  grpc_port: 6334
  # Disable telemetry on edge nodes
  enable_tls: false

# Critical for RAM-constrained devices
storage:
  mmap_threshold_kb: 1024      # Memory-map segments > 1MB (reads from flash, not RAM)
  quantization:
    always_ram: false          # Allow quantized vectors to be mmap'd too

telemetry_disabled: true

# Reduce background work that spikes CPU/IO on edge
optimizer:
  default_segment_number: 2
  max_segment_size_kb: 102400  # 100MB max segment size
  memmap_threshold_kb: 20000
  indexing_threshold_kb: 10000
  flush_interval_sec: 30
  max_optimization_threads: 1  # Single core optimization on edge
EOF

```

```
# Run as a systemd service for automatic restart
cat > /etc/systemd/system/qdrant-edge.service << 'EOF'
[Unit]
Description=Qdrant Vector Search Edge Node
After=network.target

[Service]
ExecStart=/usr/local/bin/qdrant --config-path /etc/qdrant/config.yaml
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
# Constrain RAM - OOM-kill before thrashing swap on flash storage
MemoryMax=2G
MemoryHigh=1800M

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now qdrant-edge
systemctl status qdrant-edge

# Create a collection with binary quantization for minimum RAM usage
curl -X PUT http://localhost:6333/collections/products \
  -H "Content-Type: application/json" \
  -d '{
    "vectors": {
      "size": 384,
      "distance": "Cosine"
    },
    "quantization_config": {
      "binary": {
        "always_ram": false
      }
    },
    "hnsw_config": {
      "m": 8,
      "ef_construct": 100,
      "full_scan_threshold": 5000,
      "on_disk": true
    },
    "optimizers_config": {
      "memmap_threshold": 10000
    }
  }'

echo "Qdrant edge node ready. Check RAM: $(free -h | awk '/^Mem/{print $3}')
```

pgvector on lightweight PostgreSQL is viable for Tier 3 when you already have PostgreSQL embedded in your application stack (common in retail and healthcare edge deployments). The tradeoff: PostgreSQL's baseline RAM footprint (~100MB `shared_buffers` minimum) plus `pgvector`'s HNSW index consumes more memory than Qdrant, but you get full SQL expressivity and can join vector search with transactional data in the same engine.

For Tier 1 (browser/mobile), the realistic options narrow further: **Transformers.js** (WebAssembly port of ONNX Runtime) handles embedding generation in-browser, and **SQLite-vec compiled to WASM** handles the search. Both fit in a browser tab with no server round-trip.

Index Synchronization: Keeping Edge in Sync with Central

The hardest operational problem in edge vector deployment is not the initial data load—it’s keeping edge indexes current as the central store evolves. This is the vector equivalent of replication lag, but with a fundamentally different character: you cannot apply vector index changes as a log of row mutations. An HNSW graph rebuild is not decomposable into atomic operations.

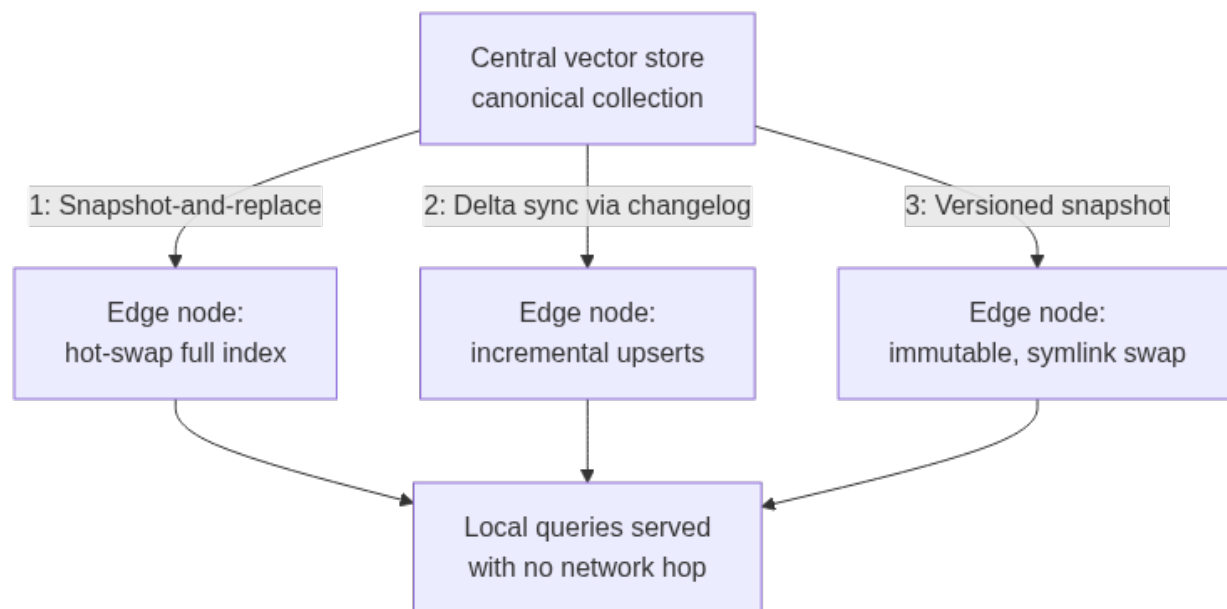


Figure 28: Three synchronization patterns for keeping an edge index current with the central store — the right choice depends on update frequency and acceptable staleness.

The practical patterns are:

1. Snapshot-and-replace. Periodically export the full collection from the central store as a binary snapshot, ship it to edge nodes, and hot-swap the index. Qdrant supports snapshots natively via its REST API. This is appropriate when the dataset is stable (changes fewer than 5% of vectors per day) and the snapshot fits within your transfer window.

2. Delta sync with a changelog. Maintain a changelog table (or Kafka topic) of vectors inserted, updated, or deleted since the last sync. The edge node consumes the changelog and applies individual upserts. This works well with Qdrant’s point upsert API and pgvector’s standard INSERT/UPDATE/DELETE. The index degrades in search quality as individual points are added outside the bulk-build path, but HNSW handles online insertions reasonably—quality remains acceptable until the index is 30–40% updated, at which point a full rebuild improves recall by 3–8%.

3. Versioned read-only snapshots. Treat the edge index as immutable. Build a new snapshot for every major data version, push it to edge nodes, and switch a symlink atomically. No online mutation. This is operationally simple and eliminates index drift, at the cost of requiring full rebuilds for every update cycle.

```

# Python
# Edge synchronization manager: delta sync from central Qdrant to edge Qdrant
# Run this on the edge node as a periodic job (cron / systemd timer)

```

```

import time
import json
import hashlib
import sqlite3
import requests
from datetime import datetime, timezone
from qdrant_client import QdrantClient
from qdrant_client.models import PointStruct, UpdateStatus

CENTRAL_URL = "https://central-qdrant.internal:6333"
CENTRAL_API_KEY = "central-api-key"
EDGE_URL = "http://localhost:6333"
COLLECTION = "products"
SYNC_STATE_DB = "/data/sync_state.db"
BATCH_SIZE = 256

def init_sync_state():
    """Track the last-synced offset per collection in local SQLite."""
    conn = sqlite3.connect(SYNC_STATE_DB)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS sync_state (
            collection TEXT PRIMARY KEY,
            last_offset TEXT, -- opaque cursor from central store
            last_sync TEXT,
            vectors_synced INTEGER DEFAULT 0
        )
    """)
    conn.commit()
    return conn

def get_last_offset(conn, collection: str) -> str | None:
    row = conn.execute(
        "SELECT last_offset FROM sync_state WHERE collection = ?",
        [collection]
    ).fetchone()
    return row[0] if row else None

def save_offset(conn, collection: str, offset: str, count: int):
    conn.execute("""
        INSERT INTO sync_state(collection, last_offset, last_sync, vectors_synced)
        VALUES (?, ?, ?, ?)
        ON CONFLICT(collection) DO UPDATE SET
            last_offset = excluded.last_offset,
            last_sync = excluded.last_sync,
            vectors_synced = sync_state.vectors_synced + excluded.vectors_synced
    """, [collection, offset, datetime.now(timezone.utc).isoformat(), count])
    conn.commit()

def fetch_delta_from_central(last_offset: str | None, batch_size: int = BATCH_SIZE):
    """
    Scroll through the central Qdrant collection from a given offset.
    Returns (points, next_offset). next_offset is None when fully caught up.
    """
    central = QdrantClient(url=CENTRAL_URL, api_key=CENTRAL_API_KEY)

    scroll_result = central.scroll(
        collection_name=COLLECTION,
        offset=last_offset, # None = start from beginning
        limit=batch_size,
        with_payload=True,
        with_vectors=True,
    )
    points, next_offset = scroll_result
    return points, next_offset

def apply_delta_to_edge(points):
    """Upsert a batch of points into the local edge Qdrant instance."""
    if not points:
        return 0

```

```

edge = QdrantClient(url=EDGE_URL)
upsert_points = [
    PointStruct(
        id=p.id,
        vector=p.vector,
        payload=p.payload,
    )
    for p in points
]

result = edge.upsert(
    collection_name=COLLECTION,
    points=upsert_points,
    wait=True,
)

if result.status != UpdateStatus.COMPLETED:
    raise RuntimeError(f"Edge upsert failed: {result.status}")

return len(points)

def run_delta_sync():
    conn = init_sync_state()
    last_offset = get_last_offset(conn, COLLECTION)
    total_synced = 0
    start_time = time.monotonic()

    print(f"[datetime.now().isoformat()] Starting delta sync for '{COLLECTION}',
    ↪ offset={last_offset}")

    while True:
        try:
            points, next_offset = fetch_delta_from_central(last_offset)
        except Exception as e:
            print(f"ERROR fetching from central: {e}")
            break

        if not points:
            print(f"Sync complete. Synced {total_synced} vectors in {time.monotonic()-start_time:.1f}s")
            break

        count = apply_delta_to_edge(points)
        total_synced += count
        last_offset = next_offset
        save_offset(conn, COLLECTION, str(next_offset) if next_offset else "COMPLETE", count)

        print(f" Batch: +{count} vectors | total={total_synced} | next_offset={next_offset}")

        if next_offset is None:
            print("Reached end of collection.")
            break

        time.sleep(0.05) # Brief pause - don't saturate edge IO

    conn.close()
    return total_synced

if __name__ == "__main__":
    synced = run_delta_sync()
    print(f"Delta sync finished. {synced} vectors applied to edge.")

```

A critical operational detail: on low-power flash storage (eMMC, SD card), the HNSW index build is IO-intensive enough to wear out the storage over months of frequent rebuilds. Profile the write amplification of your sync strategy. On a Raspberry Pi 4 with an SD card, a full rebuild of 100K vectors writes approximately 2–4GB to disk. At one rebuild per day, that is 60–120GB of writes per month—within the endurance budget of most industrial SD cards (10,000+ program/erase cycles

at 8–32GB), but worth tracking.

Offline-First Application Architecture

An edge deployment that assumes continuous connectivity is not truly an edge deployment—it’s a remote client. The application must be designed to function when the WAN link is down, when the central vector store is unreachable, and when the local index is days behind the central store. This is the same discipline as offline-first mobile databases, applied to vector search.

The pattern has three states:

- **Online with sync.** The edge index is current. Queries run locally. Writes are forwarded to the central store and reflected in the next delta sync.
- **Offline with stale index.** The edge index serves queries from its last-known-good state. Results may be incomplete (new vectors not yet synced) but are never wrong (no hallucinated data). The application should surface a freshness indicator to users: “Results reflect data as of 4 hours ago.”
- **Offline with missing index.** The edge node has just been provisioned and the initial sync has not completed, or the index has been deleted due to a storage failure. The application must degrade gracefully—either surfacing a text search fallback, a curated static list, or a “search unavailable” state.

```
# Python
# Offline-first vector search wrapper with graceful degradation
# Appropriate for a retail kiosk or industrial mobile app

import os
import time
import sqlite3
import sqlite_vec
import numpy as np
import json
import logging
from enum import Enum
from dataclasses import dataclass
from typing import Optional
from sentence_transformers import SentenceTransformer

logger = logging.getLogger("edge_search")

class SearchMode(Enum):
    VECTOR_LOCAL = "vector_local"      # Local vector index, fully operational
    TEXT_FALLBACK = "text_fallback"    # FTS5 fulltext search, no vectors
    STATIC_CATALOG = "static_catalog"  # Pre-baked result list, last resort
    UNAVAILABLE = "unavailable"        # All systems offline

@dataclass
class SearchResult:
    doc_id: str
    text: str
    score: float
    mode: SearchMode
    freshness_seconds: int  # How old is the index data?

class OfflineFirstVectorSearch:
    """
```

*Edge vector search with graceful degradation through multiple fallback modes.
Designed for environments where connectivity is intermittent.*

```

"""
def __init__(
    self,
    vector_db_path: str = "/data/edge_search.db",
    static_catalog_path: str = "/data/static_catalog.json",
    max_acceptable_staleness_hours: float = 48.0,
):
    self.vector_db_path = vector_db_path
    self.static_catalog_path = static_catalog_path
    self.max_staleness_seconds = max_acceptable_staleness_hours * 3600
    self._model: Optional[SentenceTransformer] = None

def _get_model(self) -> SentenceTransformer:
    if self._model is None:
        self._model = SentenceTransformer("all-MiniLM-L6-v2")
    return self._model

def _check_index_health(self) -> tuple[bool, int]:
    """
    Returns (is_healthy, staleness_seconds).
    Reads a metadata row written by the sync process.
    """
    if not os.path.exists(self.vector_db_path):
        return False, -1

    try:
        conn = sqlite3.connect(self.vector_db_path)
        sqlite_vec.load(conn)

        # Check that the vector table exists and has rows
        row = conn.execute(
            "SELECT COUNT(*) FROM vec_items"
        ).fetchone()
        if row[0] == 0:
            conn.close()
            return False, -1

        # Check sync metadata written by the sync process
        meta = conn.execute("""
            SELECT last_sync_ts FROM sync_metadata
            ORDER BY last_sync_ts DESC LIMIT 1
        """).fetchone()
        conn.close()

        if meta is None:
            return True, 999999 # Index exists but no sync record

        last_sync_ts = meta[0] # Unix timestamp written by sync job
        staleness = int(time.time() - last_sync_ts)
        return True, staleness

    except Exception as e:
        logger.warning(f"Index health check failed: {e}")
        return False, -1

def _vector_search(self, query_text: str, top_k: int) -> list[SearchResult]:
    model = self._get_model()
    query_vec = model.encode([query_text], normalize_embeddings=True)[0]

    conn = sqlite3.connect(self.vector_db_path)
    sqlite_vec.load(conn)

    rows = conn.execute("""
        SELECT
            m.doc_id,
            m.text,
            m.payload,
    """

```

```

        v.distance,
        s.last_sync_ts
    FROM vec_items v
    JOIN item_metadata m ON m.item_id = v.item_id
    CROSS JOIN (SELECT MAX(last_sync_ts) AS last_sync_ts FROM sync_metadata) s
    WHERE v.embedding MATCH ?
        AND k = ?
    ORDER BY v.distance
""" , [query_vec.astype(np.float32).tobytes(), top_k]).fetchall()
conn.close()

last_sync_ts = rows[0][4] if rows else time.time()
staleness = int(time.time() - last_sync_ts)

return [
    SearchResult(
        doc_id=r[0],
        text=r[1],
        score=float(r[3]),
        mode=SearchMode.VECTOR_LOCAL,
        freshness_seconds=staleness,
    )
    for r in rows
]

def _fts_fallback(self, query_text: str, top_k: int) -> list[SearchResult]:
    """
    Full-text search using SQLite FTS5. Available even when vector index is broken.
    Lower precision than vector search but always returns something for keyword queries.
    """
    if not os.path.exists(self.vector_db_path):
        return []

    try:
        conn = sqlite3.connect(self.vector_db_path)
        rows = conn.execute("""
            SELECT
                m.doc_id,
                m.text,
                bm25(fts_items) AS score
            FROM fts_items
            JOIN item_metadata m ON m.doc_id = fts_items.doc_id
            WHERE fts_items MATCH ?
            ORDER BY score
            LIMIT ?
        """, [query_text, top_k]).fetchall()
        conn.close()

        return [
            SearchResult(
                doc_id=r[0],
                text=r[1],
                score=float(r[2]),
                mode=SearchMode.TEXT_FALLBACK,
                freshness_seconds=999999,
            )
            for r in rows
        ]
    except Exception as e:
        logger.warning(f"FTS fallback failed: {e}")
        return []

def _static_catalog_fallback(self, top_k: int) -> list[SearchResult]:
    """
    Last-resort fallback: return top items from a pre-baked static catalog.
    The catalog is a JSON file bundled with the application, never stale.
    """
    if not os.path.exists(self.static_catalog_path):
        return []

```

```

try:
    with open(self.static_catalog_path) as f:
        catalog = json.load(f)

    return [
        SearchResult(
            doc_id=item["id"],
            text=item["text"],
            score=0.0,
            mode=SearchMode.STATIC_CATALOG,
            freshness_seconds=0, # catalog is versioned, not time-based
        )
        for item in catalog[:top_k]
    ]
except Exception as e:
    logger.warning(f"Static catalog fallback failed: {e}")
    return []

def search(self, query_text: str, top_k: int = 5) -> tuple[list[SearchResult], SearchMode]:
    """
    Main search entry point. Returns results and the mode actually used.
    Callers should surface the mode to users when it is not VECTOR_LOCAL.
    """
    healthy, staleness = self._check_index_health()

    # Attempt vector search if index is healthy and not excessively stale
    if healthy and (staleness < self.max_staleness_seconds or staleness == -1):
        try:
            results = self._vector_search(query_text, top_k)
            if results:
                return results, SearchMode.VECTOR_LOCAL
        except Exception as e:
            logger.error(f"Vector search failed: {e}")

    # Fall back to FTS
    logger.info("Falling back to FTS search")
    fts_results = self._fts_fallback(query_text, top_k)
    if fts_results:
        return fts_results, SearchMode.TEXT_FALLBACK

    # Fall back to static catalog
    logger.info("Falling back to static catalog")
    static_results = self._static_catalog_fallback(top_k)
    if static_results:
        return static_results, SearchMode.STATIC_CATALOG

    return [], SearchMode.UNAVAILABLE

# Example: retail kiosk application loop
def kiosk_search_demo():
    searcher = OfflineFirstVectorSearch(
        max_acceptable_staleness_hours=24.0
    )

    queries = [
        "wireless headphones under $300",
        "running shoes for flat feet",
        "gifts for coffee lovers",
    ]

    for query in queries:
        results, mode = searcher.search(query, top_k=3)

        freshness_msg = ""
        if results and mode == SearchMode.VECTOR_LOCAL:
            staleness_h = results[0].freshness_seconds / 3600
            if staleness_h > 1:
                freshness_msg = f" (index is {staleness_h:.1f}h old)"

```

```

mode_warning = {
    SearchMode.VECTOR_LOCAL: "",
    SearchMode.TEXT_FALLBACK: " [keyword search - vector index unavailable]",
    SearchMode.STATIC_CATALOG: " [showing popular items - search offline]",
    SearchMode.UNAVAILABLE: " [search unavailable]",
}[mode]

print(f"\nQuery: '{query}'{mode_warning}{freshness_msg}")
for r in results:
    print(f" {r.doc_id} | {r.text[:60]}")

if __name__ == "__main__":
    kiosk_search_demo()

```

The key insight in this pattern is that the application never throws an exception to the user. Every failure mode produces a degraded-but-functional result. This matches the principle that search on a retail floor must always show *something*—a blank screen is worse than a list of popular items that ignores the query.

WebAssembly: Bringing Vector Search to the Browser

Tier 1 edge—browser and mobile—requires a different execution model entirely. JavaScript runtimes cannot load native binaries; everything must run in WebAssembly. The two components you need are an embedding model and a vector search engine, both compiled to WASM.

Transformers.js provides ONNX-Runtime-compiled transformer models running in the browser via WebAssembly. Models are downloaded from Hugging Face and cached in the browser's Cache API. A quantized version of **all-MiniLM-L6-v2** is approximately 23MB after quantization—reasonable for a first-run download, cacheable thereafter.

SQLite-vec compiled to WASM is the search engine. The **sqlite-vec** project ships WASM builds. Combined with the **sql.js** or **wa-sqlite** SQLite WASM bindings, you get a fully in-browser vector index backed by the Origin Private File System (OPFS) for persistence.

```

// JavaScript / browser (ES modules)
// In-browser vector search using Transformers.js + SQLite-vec WASM
// No server round-trip for either embedding generation or search

import { pipeline } from "@xenova/transformers";
import initSqlJs from "sql.js";
// sqlite-vec WASM build; load as an ArrayBuffer from your CDN/assets
import sqliteVecWasm from "./sqlite_vec.wasm?url";

let db = null;
let embedder = null;
const DIM = 384; // all-MiniLM-L6-v2

/**
 * Initialize the embedding model and SQLite-vec database.
 * Call once at application startup.
 * Downloads ~23MB of model weights on first load (cached by browser thereafter).
 */
export async function initEdgeSearch() {
    // Load embedding model - runs entirely in WASM/WebWorker
    embedder = await pipeline(
        "feature-extraction",
        "Xenova/all-MiniLM-L6-v2",
        { quantized: true } // INT8 quantized: ~23MB vs ~90MB for fp32
    );
};

```

```

// Load SQLite with sqlite-vec extension
const SQL = await initSqlJs({
  locateFile: () => sqliteVecWasm,
});

// Attempt to restore from OPFS (Origin Private File System) for persistence
let dbData = null;
try {
  const root = await navigator.storage.getDirectory();
  const fileHandle = await root.getFileHandle("edge_search.db");
  const file = await fileHandle.getFile();
  dbData = new Uint8Array(await file.arrayBuffer());
  console.log(`Restored index from OPFS (${(dbData.length / 1024 / 1024).toFixed(1)} MB)`);
} catch {
  console.log("No existing OPFS index; starting fresh.");
}

db = dbData ? new SQL.Database(dbData) : new SQL.Database();

// Create tables if first run
db.run(`
  CREATE VIRTUAL TABLE IF NOT EXISTS vec_items USING vec0(
    item_id INTEGER PRIMARY KEY,
    embedding FLOAT[${DIM}]
  )
`);
db.run(`
  CREATE TABLE IF NOT EXISTS item_metadata (
    item_id INTEGER PRIMARY KEY,
    doc_id TEXT,
    text TEXT,
    payload TEXT
  )
`);

console.log("Edge search initialized.");
}

/**
 * Embed and index an array of documents into the in-browser SQLite-vec index.
 * @param {Array<{id: string, text: string, metadata?: object}>} documents
 */
export async function indexDocuments(documents) {
  if (!db || !embedder) throw new Error("Call initEdgeSearch() first");

  const texts = documents.map(d => d.text);

  // Generate embeddings in the browser via WASM
  const output = await embedder(texts, { pooling: "mean", normalize: true });
  const embeddings = output.tolist(); // float32[] []

  db.run("BEGIN TRANSACTION");
  try {
    const insertVec = db.prepare(
      "INSERT OR REPLACE INTO vec_items(item_id, embedding) VALUES (?, ?)"
    );
    const insertMeta = db.prepare(
      "INSERT OR REPLACE INTO item_metadata(item_id, doc_id, text, payload) VALUES (?, ?, ?, ?)"
    );

    for (let i = 0; i < documents.length; i++) {
      const emb = new Float32Array(embeddings[i]);
      insertVec.run([i, emb]);
      insertMeta.run([i, documents[i].id, documents[i].text,
        JSON.stringify(documents[i].metadata ?? {})]);
    }

    insertVec.free();
    insertMeta.free();
  }
}

```

```

    db.run("COMMIT");
  } catch (e) {
    db.run("ROLLBACK");
    throw e;
  }

  // Persist to OPFS so the index survives page reload
  await persistToOPFS();
  console.log(`Indexed ${documents.length} documents in-browser.`);
}

/**
 * Search the in-browser vector index.
 * @param {string} queryText
 * @param {number} topK
 * @returns {Array<{docId, text, metadata, distance}>}
 */
export async function searchLocal(queryText, topK = 5) {
  if (!db || !embedder) throw new Error("Call initEdgeSearch() first");

  const output = await embedder([queryText], { pooling: "mean", normalize: true });
  const queryVec = new Float32Array(output.tolist()[0]);

  const stmt = db.prepare(`
    SELECT
      m.doc_id,
      m.text,
      m.payload,
      v.distance
    FROM vec_items v
    JOIN item_metadata m ON m.item_id = v.item_id
    WHERE v.embedding MATCH ?
      AND k = ?
    ORDER BY v.distance
  `);

  const rows = [];
  stmt.bind([queryVec, topK]);
  while (stmt.step()) {
    const [docId, text, payload, distance] = stmt.getAsObject(["doc_id", "text", "payload", "distance"]);
    rows.push({ docId, text, metadata: JSON.parse(payload), distance });
  }
  stmt.free();

  return rows;
}

async function persistToOPFS() {
  try {
    const data = db.export(); // Uint8Array of the SQLite file
    const root = await navigator.storage.getDirectory();
    const fileHandle = await root.getFileHandle("edge_search.db", { create: true });
    const writable = await fileHandle.createWritable();
    await writable.write(data);
    await writable.close();
  } catch (e) {
    console.warn("OPFS persistence failed (may not be supported in this browser):", e);
  }
}

```

Browser-side vector search has concrete limits. A 50,000-vector index at 384 dimensions occupies roughly 75MB of memory as Float32 arrays, which is within the typical browser tab budget. At 768 dimensions (BERT-base), that doubles to 150MB and begins to compete with the rest of the page. For larger catalogs, apply scalar quantization to INT8 (4× reduction) before writing to the browser index, or shard the index across multiple OPFS files and use a routing layer to pick the relevant shard.

Security Considerations at the Edge

Edge deployments surface a threat model distinct from cloud deployments. A Qdrant instance running on a retail floor is physically accessible to an attacker in a way that a cloud database never is. The threat is not only network-based.

Physical access controls. The edge device should boot from an encrypted disk (LUKS on Linux, BitLocker on Windows IoT). The vector index itself may contain sensitive embeddings—for example, embeddings of customer images in a facial recognition system encode biometric information indirectly. If an attacker extracts the raw float vectors, inversion attacks (reconstructing approximate inputs from embeddings) are plausible for some model architectures.

Network segmentation. The Qdrant or SQLite-vec instance should not be reachable from the public network. Bind to 127.0.0.1 or a local VLAN interface. If the edge application is a web server that also serves customers, place the vector search on a separate internal interface and never proxy direct access to the vector DB port.

API key rotation. If the edge node connects to a central Qdrant or Pinecone instance to pull sync data, the API key on the edge device is a credential that could be extracted from the filesystem. Rotate these keys on a schedule and use short-lived tokens (OAuth 2.0 client credentials) where supported. Store keys in the OS keychain or TPM-backed secure enclave, not in plain configuration files.

Differential privacy for federated indexes. In cases where edge nodes generate embeddings from user behavior and those embeddings are sent back to the central store for model improvement, consider adding calibrated Laplace noise to embeddings before transmission. This prevents the central store from precisely reconstructing individual user interactions while preserving the aggregate signal for retraining.

Benchmarking Edge Hardware

Before committing to an edge hardware platform, profile the three operations that dominate vector search performance: embedding generation, index build, and query latency. The following shell-based benchmark harness uses Qdrant's built-in benchmark tool alongside a simple Python timing script.

```
# shell
# Edge hardware benchmarking: embedding + index build + query latency
# Run on the target edge device (Raspberry Pi 4 / Jetson Nano / NUC)

DEVICE_TAG=$(uname -m)-$(hostname)
RESULTS_DIR="/tmp/edge_bench_${DEVICE_TAG}_${date +%Y%m%d_%H%M%S}"
mkdir -p "$RESULTS_DIR"

echo "=== Edge Hardware Benchmark: $DEVICE_TAG ===" | tee "$RESULTS_DIR/report.txt"
echo "CPU: $(lscpu | grep 'Model name' | cut -d: -f2 | xargs)" | tee -a "$RESULTS_DIR/report.txt"
echo "RAM: $(free -h | awk '/^Mem/{print $2}')" | tee -a "$RESULTS_DIR/report.txt"
echo "Storage: $(df -h /data | awk 'NR==2{print $2, $4, "free"}')" | tee -a "$RESULTS_DIR/report.txt"
echo "" | tee -a "$RESULTS_DIR/report.txt"

# 1. Embedding generation throughput (sentences/sec)
```



```

echo "--- Embedding Generation Throughput ---" | tee -a "$RESULTS_DIR/report.txt"
python3 - << 'PYEOF' | tee -a "$RESULTS_DIR/report.txt"
import time
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
# Warm-up
model.encode(["warmup sentence"])

sentences = [f"This is test sentence number {i} for benchmarking purposes." for i in range(500)]

t0 = time.perf_counter()
_ = model.encode(sentences, batch_size=32, show_progress_bar=False)
elapsed = time.perf_counter() - t0

print(f"Embedded 500 sentences in {elapsed:.2f}s → {500/elapsed:.1f} sentences/sec")
print(f"Estimated time to embed 100K docs: {100000 / (500/elapsed) / 60:.1f} minutes")
PYEOF

# 2. SQLite-vec index build time
echo "" | tee -a "$RESULTS_DIR/report.txt"
echo "--- SQLite-vec Index Build (10K vectors, dim=384) ---" | tee -a "$RESULTS_DIR/report.txt"
python3 - << 'PYEOF' | tee -a "$RESULTS_DIR/report.txt"
import time, os, sqlite3, numpy as np
import sqlite_vec

BENCH_DB = "/tmp/bench_vec.db"
DIM = 384
N = 10000

conn = sqlite3.connect(BENCH_DB)
sqlite_vec.load(conn)
conn.execute(f"CREATE VIRTUAL TABLE bench_vecs USING vec0(id INTEGER PRIMARY KEY, v FLOAT[{DIM}])")

vecs = np.random.randn(N, DIM).astype(np.float32)
vecs /= np.linalg.norm(vecs, axis=1, keepdims=True)

t0 = time.perf_counter()
conn.execute("BEGIN")
for i, v in enumerate(vecs):
    conn.execute("INSERT INTO bench_vecs(id, v) VALUES (?, ?)", [i, v.tobytes()])
conn.execute("COMMIT")
elapsed = time.perf_counter() - t0

db_size_mb = os.path.getsize(BENCH_DB) / 1024 / 1024
print(f"Inserted {N} vectors in {elapsed:.2f}s → {N/elapsed:.0f} vecs/sec")
print(f"DB size: {db_size_mb:.1f} MB for {N} × dim{DIM}")

# Query latency
query = np.random.randn(DIM).astype(np.float32)
query /= np.linalg.norm(query)

latencies = []
for _ in range(100):
    t0 = time.perf_counter()
    conn.execute(
        "SELECT id, distance FROM bench_vecs WHERE v MATCH ? AND k = 10 ORDER BY distance",
        [query.tobytes()]
    ).fetchall()
    latencies.append((time.perf_counter() - t0) * 1000)

import statistics
print(f"Query latency (100 runs, k=10): "
      f"p50={statistics.median(latencies):.1f}ms "
      f"p95={sorted(latencies)[95]:.1f}ms "
      f"p99={sorted(latencies)[99]:.1f}ms")

conn.close()
os.unlink(BENCH_DB)
PYEOF

```

```
# 3. Qdrant query latency (if running)
echo "" | tee -a "$RESULTS_DIR/report.txt"
echo "--- Qdrant Edge Query Latency ---" | tee -a "$RESULTS_DIR/report.txt"
if curl -sf http://localhost:6333/healthz > /dev/null 2>&1; then
    python3 - << 'PYEOF' | tee -a "$RESULTS_DIR/report.txt"
import time, statistics, numpy as np
from qdrant_client import QdrantClient

client = QdrantClient(url="http://localhost:6333")
collections = client.get_collections().collections
if not collections:
    print("No collections found - skipping Qdrant benchmark")
else:
    col = collections[0].name
    info = client.get_collection(col)
    dim = info.config.params.vectors.size
    print(f"Benchmarking collection '{col}' (dim={dim})")

    latencies = []
    for _ in range(200):
        q = np.random.randn(dim).astype(np.float32)
        q /= np.linalg.norm(q)
        t0 = time.perf_counter()
        client.search(collection_name=col, query_vector=q.tolist(), limit=10)
        latencies.append((time.perf_counter() - t0) * 1000)

    print(f"Query latency (200 runs, k=10): "
          f"p50={statistics.median(latencies):.1f}ms "
          f"p95={sorted(latencies)[int(len(latencies)*0.95)]:.1f}ms "
          f"p99={sorted(latencies)[int(len(latencies)*0.99)]:.1f}ms")
PYEOF
else
    echo "Qdrant not running - skipping"
fi

echo "" | tee -a "$RESULTS_DIR/report.txt"
echo "Benchmark complete. Results in $RESULTS_DIR/report.txt"
cat "$RESULTS_DIR/report.txt"
```

When evaluating edge hardware, the metric that most surprises engineers is not raw query latency—HNSW on modern ARM chips is fast—but **embedding generation throughput**. A MiniLM model running on a Raspberry Pi 4 CPU produces roughly 40–80 sentences per second. If your application must embed user queries at search time, that is imperceptible (one query = one embedding 12–25ms). If it must re-embed a 100K document catalog on every full sync, that is 20–40 minutes of CPU time. The right mitigation is to embed at the central store and ship float vectors in the sync payload rather than re-embedding on edge.

Putting It Together: A Retail Edge Deployment

To make the patterns concrete, consider a mid-size retailer deploying semantic product search on 800 point-of-sale kiosks in stores across a country. Each kiosk runs on an Intel NUC (16GB RAM, 512GB NVMe, intermittent WAN via store Wi-Fi). The product catalog has 250,000 SKUs, each embedded at 384 dimensions using `all-MiniLM-L6-v2`.

Central infrastructure. A Qdrant Cloud instance holds the canonical collection. Product catalog updates—new SKUs, price changes, discontinuations—are pushed by the central ERP system via a Kafka topic consumed by an ingestion service that upserts into Qdrant.

Edge provisioning. New kiosks run a provisioning script on first boot that pulls a full Qdrant snapshot from an S3-compatible store (pre-built nightly from the central collection), hydrates the local Qdrant instance, and writes a sync state record. Snapshot size for $250K \times 384$ -dim vectors with binary quantization is approximately 125MB—a reasonable first-boot download even on slow store Wi-Fi.

Ongoing sync. A systemd timer runs the delta sync script every 15 minutes. If the WAN is down, the job exits cleanly and retries on the next cycle. The local index stays at whatever state it last successfully synced to.

Application layer. The kiosk application uses the `OfflineFirstVectorSearch` wrapper. If the local Qdrant instance is reachable and the index is less than 24 hours stale, queries go local (sub-10ms). If Qdrant is unhealthy (OOM-killed, disk full), the app falls back to an FTS5 index over product names maintained in SQLite. If the SQLite database is also missing, the app shows the store's top-50 bestsellers from a JSON file bundled with the application binary.

Observability. Each kiosk pushes a lightweight heartbeat every 5 minutes to a central monitoring endpoint when WAN is available: sync lag, index size, p95 query latency, and current search mode. The ops team can see at a glance how many kiosks are in `VECTOR_LOCAL` vs `TEXT_FALLBACK` vs `STATIC_CATALOG` mode and dispatch store IT when a device drifts into persistent fallback.

This architecture means a network outage or a Qdrant crash on a kiosk produces a degraded-but-functional experience—keyword search still works—rather than a broken one. A full store network outage still allows customers to find products. The catalog data, being relatively stable, makes stale vector results a minor quality concern rather than a correctness issue.

Key Takeaways

- **Edge vector search spans four distinct tiers**—browser/mobile, CDN worker, on-premise gateway, and near-edge regional—each with different RAM budgets, storage media, and connectivity assumptions. Match the engine to the tier: SQLite-vec for embedded/browser use, Qdrant binary with quantization for on-premise gateways, full-featured engines for near-edge.
- **Index synchronization is the defining operational challenge.** Unlike row-based replication, HNSW graphs cannot be updated as a log of atomic mutations indefinitely without quality degradation. Design a sync strategy—snapshot-and-replace, delta upsert, or versioned immutable snapshots—that matches your update frequency and acceptable recall budget.
- **Offline-first architecture requires multiple fallback layers.** A production edge search system should gracefully degrade from local vector search → full-text search → static curated results rather than failing hard. Surface index freshness to users when results may be stale; never present stale data as current without disclosure.
- **Embedding generation throughput, not query latency, is the edge bottleneck.** Generating embeddings on low-power ARM hardware is 10–20× slower than querying an HNSW index. Push embedding computation to the central store and ship float vectors in sync payloads to avoid re-embedding large catalogs on constrained hardware.
- **Edge deployments introduce physical security concerns.** Vector indexes may encode sensitive biometric or behavioral signals indirectly. Encrypt edge storage at rest, bind

database ports to localhost or internal interfaces only, use short-lived credentials for central sync, and add differential privacy noise to any embeddings transmitted back to central stores for model improvement.

Chapter 39: The Future: Hybrid Relational-Vector Databases

The database industry has seen consolidation cycles before — the document store era eventually gave way to multi-model databases, time-series engines absorbed into PostgreSQL extensions, and graph capabilities folded into relational systems. Vector search is following the same trajectory, but faster and with higher stakes. This chapter examines where the hybrid relational-vector database is going: what architectural patterns are winning, which vendors are placing serious bets, and how DBAs should position their systems and skills for the next five years. The question is no longer whether your relational database will support vector search — it already does or soon will. The question is whether that integration will be deep enough to matter, and what trade-offs you accept in exchange for keeping everything in one system.

The Convergence Is Already Happening

The distinction between “relational database” and “vector database” is eroding at the infrastructure layer faster than most practitioners realize. As of 2024, every major managed database platform has either shipped or publicly roadmapped native vector support:

- **PostgreSQL + pgvector:** Production-grade and widely adopted, supports HNSW and IVFFlat indexing, exact and approximate nearest neighbor search
- **Oracle AI Vector Search (Oracle 23ai):** Native **VECTOR** data type, HNSW and IVF indexes, hybrid SQL queries
- **SQL Server 2025:** Native vector type and **VECTOR_DISTANCE()** function in preview, targeting Azure SQL Database general availability
- **Neon:** Serverless PostgreSQL with pgvector baked in, designed specifically for AI application backends
- **Supabase:** Managed pgvector with **vecs** client library, AI-first feature positioning
- **SingleStore:** Dot-product and cosine similarity as first-class functions since version 8.0
- **AlloyDB (Google):** ScaNN-based approximate nearest neighbor, integrated with Vertex AI embedding generation
- **Aurora PostgreSQL (AWS):** pgvector extension available in all current versions, integrated with Bedrock embedding endpoints
- **Cosmos DB for MongoDB vCore:** Vector search indexes alongside document storage

This is not feature parity with dedicated vector databases. Pinecone still outperforms pgvector on billion-scale approximate nearest neighbor queries. Weaviate’s hybrid BM25+vector search is more

sophisticated than what most RDBMS extensions offer today. Qdrant’s filtering during HNSW graph traversal — what they call “payload-indexed filtering” — prevents index degradation in ways that pgvector’s current architecture does not. But these gaps are engineering problems, not fundamental limitations, and engineering problems get solved.

The architectural bet being placed by every major relational vendor is: *most applications do not need billion-scale pure vector search, and the operational simplicity of one system is worth some query performance trade-off*. The data, so far, supports this bet.

Hybrid Query Architectures: SQL Meets Semantic Search

The real innovation in hybrid databases is not simply storing vectors alongside relational rows — it is enabling queries that combine structured filters, full-text search, and vector similarity in a single execution plan. This is harder than it sounds.

The Filtering Problem

Consider a product catalog: 10 million SKUs, each with a 1536-dimension embedding from OpenAI’s `text-embedding-3-small`. A user searches for “comfortable running shoes for wide feet” (semantic), filtered to `brand = ‘Nike’`, `price < 120`, and `in_stock = true`. There are two execution strategies:

1. **Pre-filter then search:** Apply SQL predicates first, reducing the candidate set, then run approximate nearest neighbor (ANN) on the filtered subset. Fast but degrades ANN quality if the filtered set is small (HNSW graphs were built on the full index).
2. **Post-filter then search:** Run ANN on the full index, retrieve top-K candidates, then apply SQL filters. Quality is preserved but you waste cycles on results that will be filtered out.

pgvector 0.7+ supports iterative index scans for HNSW, which begins to address this. Qdrant’s approach — building payload indexes and pruning graph edges during traversal — is currently more sophisticated. This is the key architectural gap that relational systems need to close.

Here is how pgvector handles this today, with a hybrid query combining semantic search and structured filters:

```
-- PostgreSQL / pgvector
-- Schema: product catalog with embedding column
CREATE TABLE products (
  product_id  BIGINT PRIMARY KEY,
  name        TEXT NOT NULL,
  brand       TEXT NOT NULL,
  category    TEXT NOT NULL,
  price       NUMERIC(10, 2) NOT NULL,
  in_stock    BOOLEAN NOT NULL DEFAULT TRUE,
  description TEXT,
  embedding   VECTOR(1536)
);

-- HNSW index on embedding; partial index on in_stock products only
-- This pre-filters the index itself, reducing ANN search space
CREATE INDEX idx_products_embedding_hnsw
  ON products
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64)
  WHERE in_stock = TRUE;
```

```

-- GIN index for full-text search
CREATE INDEX idx_products_fts
  ON products
  USING gin (to_tsvector('english', name || ' ' || description));

-- B-tree indexes for structured filters
CREATE INDEX idx_products_brand  ON products (brand);
CREATE INDEX idx_products_price  ON products (price);

-- Hybrid query: semantic + structured + full-text combined
-- The planner will use the partial HNSW index (already filtered to in_stock)
-- and the B-tree indexes for brand/price
WITH semantic_results AS (
  SELECT
    product_id,
    name,
    brand,
    price,
    1 - (embedding <=> $1::VECTOR) AS semantic_score
  FROM products
  WHERE
    in_stock = TRUE
    AND brand  = 'Nike'
    AND price  < 120.00
    AND category = 'footwear'
  ORDER BY embedding <=> $1::VECTOR
  LIMIT 50 -- over-fetch before re-ranking
),
fts_results AS (
  SELECT
    product_id,
    ts_rank(
      to_tsvector('english', name || ' ' || description),
      plainto_tsquery('english', 'wide feet running comfortable')
    ) AS text_score
  FROM products
  WHERE
    to_tsvector('english', name || ' ' || description)
    @@ plainto_tsquery('english', 'wide feet running comfortable')
    AND brand = 'Nike'
    AND price < 120.00
    AND in_stock = TRUE
)
SELECT
  s.product_id,
  s.name,
  s.brand,
  s.price,
  -- Reciprocal rank fusion: combine semantic and text scores
  (0.7 * s.semantic_score) + (0.3 * COALESCE(f.text_score, 0)) AS combined_score
FROM semantic_results s
LEFT JOIN fts_results f USING (product_id)
ORDER BY combined_score DESC
LIMIT 10;

```

This pattern — over-fetching from the ANN index, joining with full-text results, and fusing scores — is the workhorse of production hybrid search in PostgreSQL today. The 0.7 / 0.3 weighting is a starting point; in practice you tune it against offline evaluation datasets.

Reciprocal Rank Fusion at the Database Layer

Weaviate and Elasticsearch both support reciprocal rank fusion (RRF) natively. PostgreSQL is approaching this capability through user-defined functions. Here is a portable RRF implementation that can live as a PostgreSQL function:

```

-- PostgreSQL / pgvector
-- Reciprocal rank fusion function

```

```

-- Standard RRF formula: 1 / (k + rank), where k=60 is the standard constant
CREATE OR REPLACE FUNCTION rrf_score(rank INT, k INT DEFAULT 60)
RETURNS FLOAT AS $$
    SELECT 1.0 / (k + rank)::FLOAT;
$$ LANGUAGE SQL IMMUTABLE STRICT;

-- View that materializes ranked semantic results
-- Use this pattern in application code for reusable hybrid search
CREATE OR REPLACE FUNCTION hybrid_product_search(
    query_embedding VECTOR(1536),
    query_text TEXT,
    filter_brand TEXT DEFAULT NULL,
    max_price NUMERIC DEFAULT NULL,
    result_limit INT DEFAULT 10
)
RETURNS TABLE (
    product_id BIGINT,
    name TEXT,
    combined_score FLOAT
) AS $$
WITH
semantic_ranked AS (
    SELECT
        p.product_id,
        ROW_NUMBER() OVER (ORDER BY p.embedding <=> query_embedding) AS sem_rank
    FROM products p
    WHERE
        (filter_brand IS NULL OR p.brand = filter_brand)
        AND (max_price IS NULL OR p.price <= max_price)
        AND p.in_stock = TRUE
    ORDER BY p.embedding <=> query_embedding
    LIMIT 100
),
text_ranked AS (
    SELECT
        p.product_id,
        ROW_NUMBER() OVER (
            ORDER BY ts_rank(
                to_tsvector('english', p.name || ' ' || p.description),
                plainto_tsquery('english', query_text)
            ) DESC
        ) AS txt_rank
    FROM products p
    WHERE
        to_tsvector('english', p.name || ' ' || p.description)
        @@ plainto_tsquery('english', query_text)
        AND (filter_brand IS NULL OR p.brand = filter_brand)
        AND (max_price IS NULL OR p.price <= max_price)
        AND p.in_stock = TRUE
    LIMIT 100
),
fused AS (
    SELECT
        COALESCE(sr.product_id, tr.product_id) AS product_id,
        COALESCE(rrf_score(sr.sem_rank), 0)
        + COALESCE(rrf_score(tr.txt_rank), 0) AS rrf
    FROM semantic_ranked sr
    FULL OUTER JOIN text_ranked tr USING (product_id)
)
SELECT
    f.product_id,
    p.name,
    f.rrf AS combined_score
FROM fused f
JOIN products p USING (product_id)
ORDER BY f.rrf DESC
LIMIT result_limit;
$$ LANGUAGE SQL STABLE;

-- Usage

```



```

SELECT * FROM hybrid_product_search(
  '[0.021, -0.034, ...]':VECTOR, -- pre-computed query embedding
  'wide feet running comfortable',
  filter_brand => 'Nike',
  max_price    => 120.00,
  result_limit => 10
);

```

Vector-Native Extensions to the SQL Standard

SQL itself is evolving. The ISO/IEC 9075 working group has been tracking vector operations since 2023, and multiple vendors have submitted proposals for standardized vector functions. Understanding where the SQL standard is heading helps you write portable code today.

Current De Facto SQL Vector API

There is no ratified standard yet, but a convergent API surface has emerged from vendor implementations:

Operation	pgvector	Oracle 23ai	SQL Server 2025
Cosine distance	<code><=></code>	<code>VECTOR_DISTANCE(v1, v2, COSINE)</code>	<code>VECTOR_DISTANCE(v1, v2, cosine)</code>
Euclidean (L2) distance	<code><-></code>	<code>VECTOR_DISTANCE(v1, v2, EUCLIDEAN)</code>	<code>VECTOR_DISTANCE(v1, v2, euclidean)</code>
Inner product	<code><#></code>	<code>VECTOR_DISTANCE(v1, v2, DOT)</code>	<code>VECTOR_DISTANCE(v1, v2, dot)</code>
Cast from text	<code>'[1,2,3]':vector</code>	<code>TO_VECTOR('[1,2,3]')</code>	<code>CAST('[1,2,3]' AS VECTOR(3))</code>
Dimension	<code>vector_dims(v)</code>	<code>VECTOR_DIMENSION(v)</code>	<code>VECTOR_DIMENSIONS(v)</code>

The Oracle and SQL Server function-based syntax is closer to what a future SQL standard will look like — it is easier to implement across different vendor parsers than operator overloading. pgvector's operator syntax (`<=>`, `<->`) is elegant for PostgreSQL but will not translate to a standard.

Neon + pgvector: Serverless-Native Pattern

Neon's serverless PostgreSQL is worth examining as a preview of how future hybrid databases will be deployed. It separates compute and storage, scales to zero, and branches databases the way Git branches code. For AI applications that embed pgvector, this means:

```

# Python
# Neon serverless PostgreSQL with pgvector
# Uses connection pooling via PgBouncer endpoint for stateless AI workloads

import os
import asyncio
import asyncpg
import numpy as np
from openai import AsyncOpenAI

NEON_CONNECTION_STRING = os.environ["NEON_DATABASE_URL"]
openai_client = AsyncOpenAI(api_key=os.environ["OPENAI_API_KEY"])

async def get_embedding(text: str) -> list[float]:

```

```

response = await openai_client.embeddings.create(
    input=text,
    model="text-embedding-3-small",
    dimensions=1536
)
return response.data[0].embedding

async def setup_schema(conn: asyncpg.Connection) -> None:
    await conn.execute("CREATE EXTENSION IF NOT EXISTS vector")
    await conn.execute("""
        CREATE TABLE IF NOT EXISTS knowledge_base (
            id          BIGSERIAL PRIMARY KEY,
            source       TEXT NOT NULL,
            chunk_index  INT  NOT NULL,
            content      TEXT NOT NULL,
            metadata     JSONB DEFAULT '{}',
            embedding     VECTOR(1536),
            created_at   TIMESTAMPTZ DEFAULT NOW()
        )
    """)
    # HNSW index with parameters tuned for RAG retrieval workloads
    # m=16 balances index size vs recall; ef_construction=128 for better recall
    await conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_kb_embedding
        ON knowledge_base
        USING hnsw (embedding vector_cosine_ops)
        WITH (m = 16, ef_construction = 128)
    """)
    await conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_kb_metadata
        ON knowledge_base USING GIN (metadata)
    """)

async def ingest_documents(
    conn: asyncpg.Connection,
    documents: list[dict]
) -> None:
    """Batch ingest with async embedding generation."""
    # Generate all embeddings concurrently
    embedding_tasks = [
        get_embedding(doc["content"]) for doc in documents
    ]
    embeddings = await asyncio.gather(*embedding_tasks)

    # Bulk insert using COPY protocol for efficiency
    records = [
        (
            doc["source"],
            doc["chunk_index"],
            doc["content"],
            doc.get("metadata", {}),
            embedding
        )
        for doc, embedding in zip(documents, embeddings)
    ]

    await conn.executemany("""
        INSERT INTO knowledge_base
        (source, chunk_index, content, metadata, embedding)
        VALUES ($1, $2, $3, $4::jsonb, $5::vector)
        ON CONFLICT DO NOTHING
    """, records)

async def hybrid_retrieve(
    conn: asyncpg.Connection,
    query: str,
    metadata_filter: dict | None = None,
    top_k: int = 5,
    ef_search: int = 100
) -> list[dict]:

```

```

"""
Hybrid retrieval: ANN vector search with JSONB metadata filtering.
ef_search controls recall vs speed tradeoff at query time.
Higher ef_search = better recall, slower query.
"""

query_embedding = await get_embedding(query)

# Set HNSW ef_search parameter for this session
# Default is 40; increasing to 100 improves recall at small latency cost
await conn.execute(f"SET hnsw.ef_search = {ef_search}")

# Build dynamic filter from metadata dict
filter_clause = ""
filter_params = [query_embedding]
param_idx = 2

if metadata_filter:
    conditions = []
    for key, value in metadata_filter.items():
        conditions.append(
            f"metadata->'${param_idx}' = '${param_idx + 1}'"
            .replace(f"${param_idx}", f"{key}")
            .replace(f"${param_idx + 1}", f"${param_idx}")
        )
        filter_params.append(str(value))
        param_idx += 1
    filter_clause = "AND " + " AND ".join(conditions)

rows = await conn.fetch(f"""
SELECT
    id,
    source,
    chunk_index,
    content,
    metadata,
    1 - (embedding <=> $1::vector) AS similarity
FROM knowledge_base
WHERE 1=1 {filter_clause}
ORDER BY embedding <=> $1::vector
LIMIT {top_k}
""", *filter_params)

return [
    {
        "id": row["id"],
        "source": row["source"],
        "content": row["content"],
        "metadata": row["metadata"],
        "similarity": float(row["similarity"])
    }
    for row in rows
]

async def main():
    conn = await asyncpg.connect(NEON_CONNECTION_STRING)
    try:
        await setup_schema(conn)

        # Ingest sample documents
        docs = [
            {
                "source": "docs/architecture.md",
                "chunk_index": 0,
                "content": "HNSW graphs provide logarithmic search complexity...",
                "metadata": {"category": "indexing", "version": "2024"}
            },
            {
                "source": "docs/performance.md",
                "chunk_index": 0,
                "content": "Approximate nearest neighbor search trades recall for speed..."
            }
        ]

```

```

        "metadata": {"category": "performance", "version": "2024"}
    }
]
await ingest_documents(conn, docs)

# Retrieve
results = await hybrid_retrieve(
    conn,
    query="how does approximate search work",
    metadata_filter={"category": "performance"},
    top_k=5,
    ef_search=100
)
for r in results:
    print(f"[{r['similarity']:.4f}] {r['source']}: {r['content'][:80]}")
finally:
    await conn.close()

asyncio.run(main())

```

Dedicated Vector Databases: Where They Remain Superior

Honest assessment matters here. Hybrid relational-vector databases are the right choice for a wide range of applications, but dedicated vector databases retain meaningful advantages in specific scenarios.

Where Pinecone, Weaviate, Qdrant, and Milvus Win

Scenario	Why Dedicated DB Wins
>500M vectors, single namespace	HNSW graph sharding is native; pgvector's graph lives in one PostgreSQL shard
Real-time embedding updates at >10K writes/sec	Dedicated systems use separate ingestion and serving paths
Multi-tenancy via namespaces (thousands of isolated indexes)	Pinecone pods and serverless namespaces have no equivalent in pgvector
Sparse+dense hybrid search (SPLADE + dense vectors)	Weaviate and Qdrant support this natively; PostgreSQL requires workarounds
Sub-10ms p99 at 100M+ vectors	Dedicated systems are purpose-built for this SLA

Pinecone's serverless architecture, for example, decouples the storage of vectors from the computation of similarity searches, using a proprietary compressed format that lets them serve from object storage with millisecond latency. PostgreSQL's HNSW index is stored on heap pages like any other index — the I/O model is fundamentally different.

Here is a Pinecone implementation showing the namespace-based multi-tenancy pattern that has no direct PostgreSQL equivalent:

```

# Python
# Pinecone serverless with namespace-based multi-tenancy
# Each tenant gets an isolated namespace - no cross-tenant index pollution

import os
from pinecone import Pinecone

```

```

from openai import OpenAI

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
openai_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

INDEX_NAME = "enterprise-knowledge"

def get_or_create_index() -> object:
    """Create serverless index if it doesn't exist."""
    existing = [idx.name for idx in pc.list_indexes()]
    if INDEX_NAME not in existing:
        pc.create_index(
            name=INDEX_NAME,
            dimension=1536,
            metric="cosine",
            spec={
                "serverless": {
                    "cloud": "aws",
                    "region": "us-east-1"
                }
            }
        )
    return pc.Index(INDEX_NAME)

def embed_texts(texts: list[str]) -> list[list[float]]:
    response = openai_client.embeddings.create(
        input=texts,
        model="text-embedding-3-small",
        dimensions=1536
    )
    return [item.embedding for item in response.data]

def ingest_for_tenant(
    index,
    tenant_id: str,
    documents: list[dict]
) -> None:
    """
    Each tenant gets its own namespace.
    Pinecone namespaces provide hard isolation:
    - Separate HNSW graphs per namespace
    - Query never crosses namespace boundaries
    - Independent scaling per namespace
    This would require separate PostgreSQL schemas or
    a discriminator column + partial indexes in pgvector,
    neither of which provides the same isolation guarantees
    at query time.
    """
    texts = [doc["content"] for doc in documents]
    embeddings = embed_texts(texts)

    vectors = [
        {
            "id": f"{tenant_id}_{doc['id']}",
            "values": embedding,
            "metadata": {
                "content": doc["content"],
                "source": doc.get("source", ""),
                "tenant_id": tenant_id,
                **doc.get("metadata", {})
            }
        }
        for doc, embedding in zip(documents, embeddings)
    ]

    # Namespace isolates this tenant's index completely
    index.upsert(
        vectors=vectors,
        namespace=f"tenant_{tenant_id}"
    )

```

```

print(f"Ingested {len(vectors)} vectors for tenant {tenant_id}")

def query_tenant(
    index,
    tenant_id: str,
    query_text: str,
    top_k: int = 10,
    metadata_filter: dict | None = None
) -> list[dict]:
    """Query is automatically scoped to tenant namespace."""
    query_embedding = embed_texts([query_text])[0]

    query_params = {
        "vector": query_embedding,
        "top_k": top_k,
        "namespace": f"tenant_{tenant_id}",
        "include_metadata": True
    }

    if metadata_filter:
        query_params["filter"] = metadata_filter

    results = index.query(**query_params)

    return [
        {
            "id": match.id,
            "score": match.score,
            "content": match.metadata.get("content", ""),
            "source": match.metadata.get("source", "")
        }
        for match in results.matches
    ]

def get_tenant_stats(index, tenant_id: str) -> dict:
    """Namespace-level stats - impossible to get per-tenant in pgvector."""
    stats = index.describe_index_stats()
    namespace_key = f"tenant_{tenant_id}"
    namespace_data = stats.namespaces.get(namespace_key, {})
    return {
        "tenant_id": tenant_id,
        "vector_count": int(namespace_data.get("vector_count", 0)),
        "total_index_size": stats.total_vector_count
    }

# Example usage
index = get_or_create_index()

# Tenant A ingestion
ingest_for_tenant(index, "acme_corp", [
    {"id": "doc1", "content": "ACME product return policy...", "source": "policy.pdf"},
    {"id": "doc2", "content": "ACME warranty terms...", "source": "warranty.pdf"}
])

# Tenant B ingestion - completely isolated index graph
ingest_for_tenant(index, "globex_inc", [
    {"id": "doc1", "content": "Globex data processing agreement...", "source": "dpa.pdf"},
])

# Query is automatically scoped - no risk of ACME seeing Globex data
acme_results = query_tenant(index, "acme_corp", "what is the return window")
globex_results = query_tenant(index, "globex_inc", "data retention requirements")

print(f"ACME results: {len(acme_results)}")
print(f"Globex results: {len(globex_results)}")

```

The equivalent in pgvector requires either row-level security policies with a tenant discriminator column and careful partial index design, or separate schemas per tenant — both of which impose query

planning overhead and operational complexity that Pinecone’s namespace model avoids entirely.

The Next Architectural Patterns

Several architectural shifts are underway that will define how hybrid databases work over the next three to five years.

1. Embedding Generation Inside the Database

Today, most workflows look like: application → embedding API → database. The next generation moves embedding generation into the database engine itself. This is already starting:

- **AlloyDB AI:** Calls Google Vertex AI embedding models directly from SQL using `google_ml.embedding()` function
- **Oracle 23ai:** `DBMS_VECTOR_CHAIN` PL/SQL package generates embeddings, chunks documents, and runs inference from within PL/SQL
- **Azure SQL + OpenAI Integration:** External REST calls to Azure OpenAI from T-SQL using `sp_invoke_external_rest_endpoint`

Here is what in-database embedding generation looks like in Oracle 23ai, alongside the Azure SQL Server equivalent using its REST integration:

```
-- Oracle 23ai
-- In-database embedding generation via ONNX model loaded into database
-- Oracle supports importing ONNX models directly into the Oracle Database
-- so no external API call is required; inference runs in-process

-- Step 1: Load ONNX embedding model (done once by DBA)
-- DBMS_VECTOR_LOAD_ONNX_MODEL loads the model into the ONNX_MODELS table
EXECUTE DBMS_VECTOR_LOAD_ONNX_MODEL(
  'dm_models',           -- directory object pointing to model file
  'all-MiniLM-L6-v2.onnx', -- ONNX model file
  'ALL_MINILM_L6_V2',    -- model name in catalog
  JSON('{"function":"embedding","embeddingOutput":"embedding","input":{"input":["DATA"]}}')
);

-- Step 2: Generate embedding inline in SQL
-- VECTOR_EMBEDDING() is Oracle's native function for in-database inference
SELECT
  product_id,
  name,
  VECTOR_EMBEDDING(ALL_MINILM_L6_V2 USING description AS DATA) AS embedding
FROM products
FETCH FIRST 5 ROWS ONLY;

-- Step 3: Similarity search without leaving the database
-- The entire RAG pipeline - embed query, search, return results - is one SQL statement
SELECT
  p.product_id,
  p.name,
  p.description,
  VECTOR_DISTANCE(
    p.embedding,
    VECTOR_EMBEDDING(ALL_MINILM_L6_V2 USING :query_text AS DATA),
    COSINE
  ) AS distance
FROM products p
WHERE p.in_stock = 1
ORDER BY distance
FETCH FIRST 10 ROWS ONLY;
```

```

-- SQL Server (Azure SQL Database)
-- In-database embedding generation via sp_invoke_external_rest_endpoint
-- Calls Azure OpenAI from T-SQL; no application code required

-- Stored procedure that generates an embedding for a given text
CREATE OR ALTER PROCEDURE dbo.GenerateEmbedding
    @input_text NVARCHAR(MAX),
    @embedding NVARCHAR(MAX) OUTPUT -- JSON array returned as string
AS
BEGIN
    DECLARE @url NVARCHAR(MAX);
    DECLARE @payload NVARCHAR(MAX);
    DECLARE @response NVARCHAR(MAX);
    DECLARE @retval INT;

    -- Azure OpenAI endpoint for your deployment
    SET @url = 'https://<your-resource>.openai.azure.com/openai/deployments/'
        + 'text-embedding-3-small/embeddings?api-version=2024-02-01';

    SET @payload = JSON_OBJECT(
        'input': @input_text,
        'dimensions': 1536
    );

    -- External REST call from within SQL Server
    EXEC @retval = sp_invoke_external_rest_endpoint
        @url = @url,
        @method = 'POST',
        @payload = @payload,
        @credential = [AzureOpenAICredential], -- DATABASE SCOPED CREDENTIAL
        @response = @response OUTPUT;

    -- Extract embedding vector from JSON response
    SET @embedding = JSON_QUERY(@response, '$.result.data[0].embedding');
END;
GO

-- Use the procedure to embed and search in one batch
DECLARE @query_text NVARCHAR(MAX) = 'comfortable running shoes wide feet';
DECLARE @query_vec_str NVARCHAR(MAX);

EXEC dbo.GenerateEmbedding
    @input_text = @query_text,
    @embedding = @query_vec_str OUTPUT;

-- Cast to VECTOR type and run similarity search
-- SQL Server 2025 native VECTOR type
SELECT TOP 10
    product_id,
    name,
    price,
    VECTOR_DISTANCE('cosine',
        CAST(@query_vec_str AS VECTOR(1536)),
        embedding
    ) AS distance
FROM dbo.products
WHERE
    brand = 'Nike'
    AND price < 120.00
    AND in_stock = 1
ORDER BY distance;

```

This pattern eliminates the embedding API round-trip from the application layer entirely. For high-throughput batch operations — reindexing a product catalog after a model upgrade, for example — the performance difference is material because you are not paying network serialization cost for millions of embedding requests.

2. Vector Quantization and Compressed Storage

At scale, storing full-precision float32 vectors is expensive. A 1536-dimension float32 vector is 6,144 bytes. Ten million such vectors is roughly 58 GB just for the embeddings. Several compression strategies are maturing:

- **Scalar quantization (SQ8)**: Compress each float32 to int8 — 4× storage reduction, minimal recall loss (~1–2% on typical datasets)
- **Binary quantization (BQ)**: Compress each dimension to a single bit — 32× storage reduction, meaningful recall loss (~5–15%), often used with re-ranking from full-precision vectors
- **Product quantization (PQ)**: Split vector into sub-vectors, quantize each sub-space — configurable tradeoff between compression ratio and recall

pgvector 0.7 introduced half-precision (**halfvec**) storage, which reduces float32 to float16 (2× reduction). Full scalar quantization is on the roadmap. Pinecone’s serverless uses proprietary quantization internally and abstracts it away entirely — you get the storage savings without configuring anything.

Here is how to use pgvector’s half-precision type for a storage-efficient schema:

```
-- PostgreSQL / pgvector 0.7+
-- halfvec type stores each dimension as float16 instead of float32
-- Cuts embedding storage from 6 KB to 3 KB per 1536-dim vector

-- Schema using halfvec for storage efficiency
CREATE TABLE documents_compressed (
  id          BIGSERIAL PRIMARY KEY,
  content     TEXT NOT NULL,
  source      TEXT,
  -- Full-precision embedding (used for re-ranking)
  embedding_full VECTOR(1536),
  -- Half-precision embedding (used for initial ANN retrieval)
  embedding_half  HALFVEC(1536)
);

-- HNSW index on half-precision vector for fast, compact index
CREATE INDEX idx_docs_hnsw_half
  ON documents_compressed
  USING hnsw (embedding_half halfvec_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- Two-phase retrieval: coarse search on halfvec, re-rank on full vector
-- Phase 1: ANN on half-precision (fast, compact)
-- Phase 2: Exact cosine on full-precision for top candidates (accurate)
WITH coarse_candidates AS (
  SELECT
    id,
    content,
    embedding_full,
    embedding_half <=> $1::HALFVEC AS approx_distance
  FROM documents_compressed
  ORDER BY embedding_half <=> $1::HALFVEC
  LIMIT 50
)
SELECT
  id,
  content,
  -- Exact re-rank using full-precision embedding
  1 - (embedding_full <=> $2::VECTOR) AS precise_similarity
FROM coarse_candidates
ORDER BY embedding_full <=> $2::VECTOR
LIMIT 10;
-- $1 and $2 are the same query embedding, cast to different types

-- SQL Server 2025
-- VECTOR type defaults to float32; no native half-precision support yet
-- Approximate storage compression via COMPRESS() on the raw JSON
```

```

-- representation (workaround until native quantization ships)

-- For now, SQL Server DBA can store a compressed binary blob alongside
-- the VECTOR column and use it for metadata/archival purposes
CREATE TABLE documents_compressed (
    id                BIGINT IDENTITY PRIMARY KEY,
    content            NVARCHAR(MAX) NOT NULL,
    source             NVARCHAR(500),
    embedding          VECTOR(1536),           -- native VECTOR for search
    embedding_compressed AS                             -- computed compressed copy
        COMPRESS(CAST(embedding AS NVARCHAR(MAX))) -- for cold storage / archival
    STORED,
    created_at         DATETIME2 DEFAULT SYSUTCDATETIME()
);

-- Index on VECTOR column
CREATE INDEX idx_docs_vector
ON documents_compressed (embedding)
WITH (VECTOR_SEARCH_OPTIONS = 'HNSW');
-- Note: exact DDL syntax may differ in GA release; preview syntax shown here

-- Standard similarity search
SELECT TOP 10
    id,
    content,
    VECTOR_DISTANCE('cosine', embedding, CAST(@query AS VECTOR(1536))) AS distance
FROM documents_compressed
ORDER BY distance;

```

3. Multi-Vector Representations and Late Interaction

The next frontier beyond single-vector-per-document is multi-vector representations. ColBERT-style late interaction models store one embedding per token in a document rather than one embedding for the whole document. This dramatically improves retrieval quality — especially for long documents — at the cost of storage (a 512-token document generates 512 vectors instead of 1).

Dedicated databases are handling this first. Qdrant supports multi-vector via its `NamedVectors` API. Weaviate has experimental multi-vector support. For PostgreSQL, the pattern requires storing vectors in an array column and using `unnesting`:

```

-- PostgreSQL / pgvector
-- Multi-vector (ColBERT-style) storage pattern
-- One row per document, but embedding column stores an ARRAY of vectors
-- This is a workaround; native multi-vector indexing is not yet available

CREATE TABLE documents_multivec (
    id                BIGSERIAL PRIMARY KEY,
    content            TEXT NOT NULL,
    source             TEXT,
    -- Array of per-token embeddings (ColBERT produces 128-dim per token)
    token_embeddings   VECTOR(128)[]
);

-- To query: unnest token embeddings, find max similarity per query token,
-- then aggregate across document (MaxSim scoring)
-- This is a full table scan - no ANN index support for multi-vector yet
WITH query_vectors AS (
    -- Assume $1 is an array of query token embeddings: VECTOR(128)[]
    SELECT UNNEST($1::VECTOR(128)[]) AS q_vec, generate_subscripts($1::VECTOR(128)[], 1) AS q_pos
),
token_scores AS (
    SELECT
        d.id,
        d.content,
        q.q_pos,
        MAX(1 - (tv <=> q.q_vec)) AS max_sim -- MaxSim per query token
    FROM documents_multivec d

```

```

CROSS JOIN LATERAL UNNEST(d.token_embeddings) AS tv
CROSS JOIN query_vectors q
GROUP BY d.id, d.content, q.q_pos
)
SELECT
  id,
  content,
  SUM(max_sim) AS colbert_score    -- Sum of per-token MaxSim scores
FROM token_scores
GROUP BY id, content
ORDER BY colbert_score DESC
LIMIT 10;

```

This SQL-based ColBERT implementation is instructive but not production-grade at scale — the cross join between all document tokens and all query tokens is expensive. Native multi-vector indexing in relational databases is a two-to-three year engineering project. Dedicated systems will hold this advantage for some time.

The Operational Convergence: One Platform, Many Workloads

Beyond query capabilities, the strategic advantage of hybrid databases is operational: one backup strategy, one monitoring stack, one set of RBAC policies, one disaster recovery plan. For organizations already running PostgreSQL or SQL Server in production, adding pgvector or SQL Server’s vector extension means:

- Vectors participate in existing PITR (point-in-time recovery) workflows
- Logical replication automatically replicates vector columns to read replicas
- `pg_dump` / `pg_restore` include vector data without special handling
- Existing connection poolers (PgBouncer, PgPool) work without modification
- Vector indexes appear in `pg_indexes` and are visible to all standard monitoring queries

Compare this to adding Pinecone or Weaviate to your stack: new backup/restore procedures, new monitoring dashboards, new alerting thresholds, new access control model, new cost center, new on-call runbook.

Here is a practical monitoring query that demonstrates the operational integration — this is standard PostgreSQL administration, but it works on vector indexes without modification:

```

-- PostgreSQL
-- Monitor HNSW index health and usage alongside all other indexes
-- Standard pg_stat_user_indexes view covers vector indexes transparently

SELECT
  schemaname,
  tablename,
  indexname,
  idx_scan      AS index_scans,
  idx_tup_read  AS tuples_read,
  idx_tup_fetch AS tuples_fetched,
  pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Check index bloat for HNSW indexes specifically
-- HNSW indexes do not support VACUUM the same way B-tree does;
-- heavy update workloads may require periodic REINDEX
SELECT

```

```

relname                                AS table_name,
indexrelname                           AS index_name,
pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
pg_size_pretty(pg_relation_size(indrelid))   AS table_size,
ROUND(
    100.0 * pg_relation_size(indexrelid)
    / NULLIF(pg_relation_size(indrelid), 0),
    2
)                                         AS index_to_table_pct
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE indexrelname LIKE '%hns%' OR indexrelname LIKE '%embedding%'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Cost estimation for vector queries
-- Use EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) to inspect ANN plan
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT id, content, embedding <=> '[0.1, 0.2, ...]':VECTOR AS dist
FROM documents
ORDER BY embedding <=> '[0.1, 0.2, ...]':VECTOR
LIMIT 10;

-- SQL Server
-- Monitor vector index usage via standard DMVs
-- sys.dm_db_index_usage_stats covers vector indexes

SELECT
    OBJECT_NAME(ius.object_id) AS table_name,
    i.name                     AS index_name,
    i.type_desc                AS index_type,
    ius.user_seeks,
    ius.user_scans,
    ius.user_lookups,
    ius.user_updates,
    8 * SUM(a.used_pages) / 1024 AS index_size_kb
FROM sys.dm_db_index_usage_stats ius
JOIN sys.indexes i
    ON ius.object_id = i.object_id
    AND ius.index_id = i.index_id
JOIN sys.partitions p
    ON i.object_id = p.object_id
    AND i.index_id = p.index_id
JOIN sys.allocation_units a
    ON p.partition_id = a.container_id
WHERE
    ius.database_id = DB_ID()
    AND OBJECT_NAME(ius.object_id) NOT LIKE 'sys%'
GROUP BY
    OBJECT_NAME(ius.object_id),
    i.name,
    i.type_desc,
    ius.user_seeks,
    ius.user_scans,
    ius.user_lookups,
    ius.user_updates
ORDER BY (ius.user_seeks + ius.user_scans) DESC;

```

The Five-Year Horizon: Predictions and Preparation

The following patterns are directionally reliable based on current vendor roadmaps and the historical pace of database feature adoption.

What will be standard by 2027

1. Every managed PostgreSQL service will ship HNSW and IVFFlat with scalar

quantization enabled by default. Renting vector search as a separate service for workloads under 100M vectors will be economically unjustifiable for most organizations.

2. **In-database embedding generation will be table-stakes for enterprise database vendors.** Oracle, SQL Server, and AlloyDB will all support calling embedding models from SQL without application-layer involvement. The architecture of the embedding pipeline will shift from application-centric to database-centric for many workloads.
3. **The SQL standard will include a VECTOR data type and at minimum VECTOR_DISTANCE() and KNN_SEARCH() functions.** ISO/IEC 9075 moves slowly, but commercial pressure from Oracle, Microsoft, and Google will accelerate ratification of the core subset. Portable SQL for vector search will become possible.
4. **Dedicated vector databases will consolidate around differentiated use cases.** Pinecone, Weaviate, Qdrant, and Milvus will not disappear, but the general-purpose market — “I need vector search for my RAG app” — will be largely captured by relational extensions. Dedicated databases will serve billion-scale, low-latency, multi-modal, or multi-vector workloads where relational systems have structural limitations.
5. **Multi-vector and late interaction retrieval will become a first-class database capability.** ColBERT-style indexing will move from research to production within major database engines, driven by the demonstrably superior retrieval quality for long-document RAG applications.

Skills That Will Compound in Value

For the DBA or data engineer positioning for this shift:

- **Query plan analysis for ANN operations:** Understanding when the planner chooses a sequential scan over an HNSW index, why `ef_search` affects recall, and how partial indexes interact with vector predicates will be as important as understanding B-tree selectivity today.
- **Embedding model selection and dimensionality tradeoffs:** Knowing that `text-embedding-3-small` at 512 dimensions often outperforms a 1536-dimension naive implementation for retrieval tasks (because of better normalization behavior) requires understanding the model, not just the database.
- **Hybrid scoring and ranking:** RRF, weighted combination, and re-ranking pipelines are becoming standard database patterns. DBAs who can implement and tune these in SQL without reaching for a separate service will be significantly more productive.
- **Quantization configuration:** The tradeoff between storage, recall, and latency for SQ8 vs BQ vs PQ compression is becoming a database tuning parameter, not a research topic.
- **Operational integration of AI workloads:** Backup, replication, schema migration (changing embedding dimensions requires dropping and rebuilding HNSW indexes), and connection pooling for embedding-heavy workloads all have non-obvious failure modes that experienced practitioners will navigate faster.

Choosing Your Architecture Today

Given all of the above, here is a decision framework for organizations deciding how to handle vector search in 2024–2025:

Vector dataset size?

< 10M vectors

pgvector / SQL Server vector extension

Reasoning: operational simplicity wins; ANN quality is sufficient

10M – 500M vectors

Latency SLA < 20ms p99?

YES → Dedicated vector DB (Pinecone serverless, Qdrant)

NO → pgvector with tuned HNSW (m=32, ef_search=200)
or AlloyDB AI (ScaNN-based, better at this scale)

Multi-tenant (1000+ isolated indexes)?

YES → Pinecone namespaces or Qdrant collections

NO → pgvector with RLS + partial indexes

> 500M vectors

Dedicated vector DB or distributed solution (Milvus on Kubernetes)

pgvector does not shard horizontally; single-node limits apply

Citus (distributed PostgreSQL) + pgvector is an option but complex

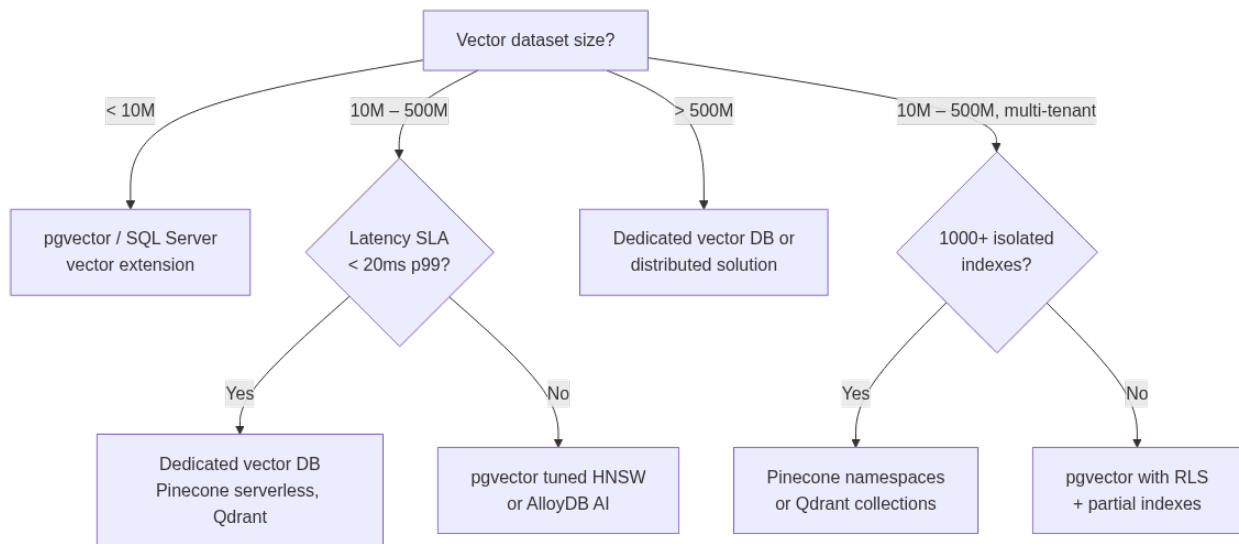


Figure 29: The same decision tree above, redrawn — the crossover points shift upward with each pgvector release, so revisit this periodically.

The crossover point — where pgvector’s operational simplicity stops compensating for its scale limitations — is moving upward with each release. pgvector 0.8 and beyond will push the inflection point closer to the billion-vector range for many workloads. Track the releases.

Key Takeaways

- **The relational-vector integration is not a temporary stopgap** — every major enterprise database vendor has made multi-year engineering commitments to native vector support, and the operational benefits of a single system are compelling for the majority of production workloads today.
 - **Hybrid query architecture — combining structured SQL filters, full-text search, and ANN similarity in one execution plan — is the defining capability of hybrid databases**, and the quality of that integration (especially how well the planner handles pre-filtering vs post-filtering) is the primary technical differentiator between implementations.
 - **Dedicated vector databases retain structural advantages at billion-scale, for multi-vector (ColBERT-style) retrieval, and for namespace-based multi-tenancy** — these are engineering limitations of the relational storage model, not just configuration gaps, and will take years to fully close.
 - **In-database embedding generation, vector quantization, and the ColBERT multi-vector pattern are the three capabilities to watch** on vendor roadmaps; whichever relational databases ship these first will capture workloads that currently require dedicated systems.
 - **The skills that compound are query plan analysis for ANN operations, hybrid scoring implementation in SQL, quantization configuration, and operational integration of AI workloads** — these are the database engineering fundamentals for the AI application layer, and practitioners who build them now will have a durable advantage as the architecture matures.
-

Chapter 40: Building a Production Vector DB Monitoring Dashboard

Monitoring a vector database in production is not the same as monitoring a relational database. The familiar metrics—query throughput, lock waits, buffer hit ratio, replication lag—still matter when the underlying engine is PostgreSQL or a similar system, but they tell you nothing about the health of your embedding pipeline, the quality of your search results, or whether your index structures are serving queries efficiently under load. This chapter brings together everything from earlier in the book and shows you how to build a dashboard that covers all three layers: infrastructure metrics, vector-specific operational metrics, and application-layer quality signals. By the end, you will have runnable code for a Prometheus/Grafana stack that monitors pgvector, Pinecone, and Weaviate simultaneously, a Python-based quality probe that runs continuously against your RAG pipeline, and a pattern for surfacing drift and recall degradation before your users notice.

The Three-Layer Monitoring Model

Most teams start with infrastructure monitoring because it is familiar: CPU, memory, disk I/O, and connection counts. That baseline is necessary but insufficient for vector workloads. You need to think in three distinct layers, each requiring different instrumentation.

Layer 1 — Infrastructure and Engine Metrics These are the metrics your existing tools already collect. For pgvector on PostgreSQL, that means `pg_stat_statements`, autovacuum activity, `shared_buffers` utilization, and WAL generation rate. For Pinecone, it means pod CPU and memory exposed through the Pinecone console API. For Weaviate, it means the built-in `/v1/nodes` endpoint and Prometheus scrape target.

Layer 2 — Vector Index Operational Metrics These metrics are specific to the vector engine and have no direct SQL equivalent. Index build status, HNSW graph construction rate, quantization compression ratio, segment count, and—most importantly—approximate nearest neighbor (ANN) query latency at the 50th, 95th, and 99th percentiles. Think of the HNSW index as an analogy to a B-tree: just as you would monitor B-tree bloat and split frequency, you monitor HNSW graph connectivity and segment merger rates.

Layer 3 — Embedding and Search Quality Metrics This layer has no equivalent in traditional database monitoring. It answers questions like: Are the embeddings being stored still aligned with the model that generated the query embeddings? Is the top-k recall degrading as new data is inserted without re-indexing? Are cosine similarity scores drifting toward lower values, indicating

that the vector space is becoming less discriminative? These signals require synthetic probes—queries you run against the database with known expected results—and they need to run on a schedule, not just when a user complains.

The architectural principle is that your dashboard should surface a failure at Layer 3 before it becomes visible at Layer 2, and a Layer 2 failure before it cascades to Layer 1. In practice, embedding drift shows up weeks before your HNSW index becomes so fragmented that query latency spikes.

Instrumenting pgvector with Prometheus and Custom Metrics

PostgreSQL does not expose vector-specific metrics natively. You need to instrument them yourself, either through a custom `postgres_exporter` configuration or by running a metrics collection script that queries system views and writes to a Prometheus pushgateway.

The following script collects the most important pgvector operational metrics and pushes them to a Prometheus pushgateway on a 60-second interval. It covers index status, approximate row counts per collection, and query latency histograms derived from `pg_stat_statements`.

```
# Python
# pgvector_metrics_collector.py
# Requires: pycpg2, prometheus_client, schedule

import time
import schedule
import pycpg2
import pycpg2.extras
from prometheus_client import (
    CollectorRegistry,
    Gauge,
    Histogram,
    push_to_gateway,
)

PUSHGATEWAY_URL = "http://localhost:9091"
PG_DSN = "host=localhost dbname=vectordb user=monitor password=monitor_pass"

registry = CollectorRegistry()

# Gauge: number of live rows per vector table
vector_table_row_count = Gauge(
    "pgvector_table_live_rows",
    "Estimated live row count for vector tables",
    ["table_name", "index_name"],
    registry=registry,
)

# Gauge: index size in bytes
vector_index_size_bytes = Gauge(
    "pgvector_index_size_bytes",
    "On-disk size of vector indexes",
    ["index_name", "table_name"],
    registry=registry,
)

# Gauge: HNSW index build progress (0=not building, 1=building)
vector_index_building = Gauge(
    "pgvector_index_building",
    "Whether an HNSW index is currently being built (1=yes)",
    ["index_name"],
    registry=registry,
```

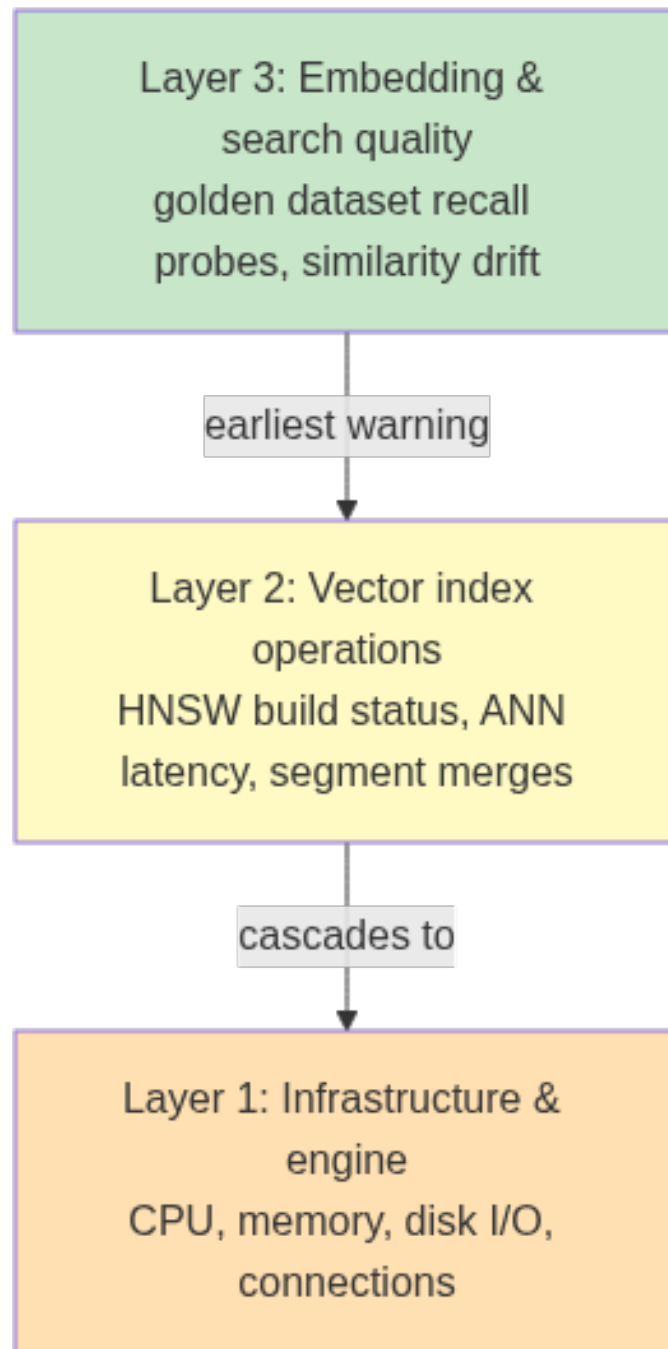


Figure 30: A Layer 3 failure — recall or similarity drift — typically shows up weeks before it cascades into a visible Layer 2 or Layer 1 problem, which is why quality probes matter as much as infrastructure dashboards.

```

)

# Histogram: ANN query latency from pg_stat_statements (p50, p95, p99)
vector_query_latency_ms = Gauge(
    "pgvector_ann_query_mean_latency_ms",
    "Mean latency in ms for ANN queries (from pg_stat_statements)",
    ["query_pattern"],
    registry=registry,
)

vector_query_calls_total = Gauge(
    "pgvector_ann_query_calls_total",
    "Total call count for ANN query patterns",
    ["query_pattern"],
    registry=registry,
)

def collect_metrics():
    conn = psycopg2.connect(PG_DSN)
    cur = conn.cursor(cursor_factory=psycopg2.extras.DictCursor)

    # Collect live row counts and index sizes for vector tables
    # Identify vector tables by looking for vector-typed columns
    cur.execute("""
        SELECT
            c.relname                AS table_name,
            i.relname                AS index_name,
            pg_relation_size(i.oid)  AS index_size_bytes,
            pg_stat_get_live_tuples(c.oid) AS live_rows,
            am.amname                AS index_method
        FROM
            pg_class c
            JOIN pg_index ix ON c.oid = ix.indrelid
            JOIN pg_class i  ON i.oid = ix.indexrelid
            JOIN pg_am am    ON i.relam = am.oid
            JOIN pg_attribute a ON a.attrelid = c.oid
                                AND a.attnum = ANY(ix.indkey)
            JOIN pg_type t   ON a.atttypid = t.oid
        WHERE
            t.typname = 'vector'
            AND c.relkind = 'r'
            AND am.amname IN ('hnsu', 'ivfflat')
        ORDER BY
            c.relname, i.relname;
    """)
    rows = cur.fetchall()
    for row in rows:
        vector_table_row_count.labels(
            table_name=row["table_name"],
            index_name=row["index_name"],
        ).set(row["live_rows"])
        vector_index_size_bytes.labels(
            index_name=row["index_name"],
            table_name=row["table_name"],
        ).set(row["index_size_bytes"])

    # Detect in-progress index builds
    cur.execute("""
        SELECT
            relname AS index_name,
            phase,
            blocks_done,
            blocks_total
        FROM
            pg_stat_progress_create_index
        WHERE
            command = 'CREATE INDEX CONCURRENTLY'
            OR command = 'CREATE INDEX';
    """)

```

```

building = cur.fetchall()
# Reset all to 0 first, then set active ones
for row in building:
    vector_index_building.labels(index_name=row["index_name"]).set(1)

# ANN query latency from pg_stat_statements
# Filter for queries that contain '<->' (L2), '<=>' (cosine), or '<#>' (inner product)
cur.execute("""
    SELECT
        LEFT(query, 80)                AS query_pattern,
        calls                          AS total_calls,
        mean_exec_time                 AS mean_ms
    FROM
        pg_stat_statements
    WHERE
        query LIKE '%<->%'
        OR query LIKE '%<=>%'
        OR query LIKE '%<#>%'
    ORDER BY
        total_exec_time DESC
    LIMIT 20;
""")
query_rows = cur.fetchall()
for qrow in query_rows:
    label = qrow["query_pattern"].strip().replace("\n", " "][:60]
    vector_query_latency_ms.labels(query_pattern=label).set(qrow["mean_ms"])
    vector_query_calls_total.labels(query_pattern=label).set(qrow["total_calls"])

cur.close()
conn.close()

push_to_gateway(
    PUSHGATEWAY_URL,
    job="pgvector_metrics",
    registry=registry,
)
print(f"[{time.strftime('%H:%M:%S')}] Metrics pushed to Prometheus gateway")

schedule.every(60).seconds.do(collect_metrics)

if __name__ == "__main__":
    collect_metrics() # Run immediately on start
    while True:
        schedule.run_pending()
        time.sleep(1)

```

This script intentionally uses `pg_stat_statements` rather than wrapping every query with timing code. That means it captures the full query load from all connections, not just the ones your application explicitly instruments. The `mean_exec_time` from `pg_stat_statements` is a rolling mean since the last statistics reset; pair it with the `calls` counter and you can compute a rate-weighted average in Grafana.

For the Grafana dashboard, create a panel with this PromQL expression to show the top-5 slowest ANN query patterns:

```

# shell
# Prometheus query for Grafana panel: Top 5 slowest ANN query patterns
# Panel type: Table
# PromQL:
topk(5, pgvector_ann_query_mean_latency_ms)

# Panel type: Time series for index size growth
# PromQL:
rate(pgvector_index_size_bytes[5m]) * 300
# Multiplied by 300 seconds (5-minute interval) to get bytes added per scrape window

```

```
# Alert rule: ANN query latency exceeds 200ms mean
# In Prometheus alerting rules YAML:
# alert: VectorQueryLatencyHigh
# expr: pgvector_ann_query_mean_latency_ms > 200
# for: 5m
# labels:
#   severity: warning
# annotations:
#   summary: "ANN query mean latency > 200ms for {{ $labels.query_pattern }}"
```

Monitoring Pinecone and Weaviate via Their Native APIs

Pinecone and Weaviate expose operational metrics through REST APIs, but they each have different granularity and different concepts of what “healthy” means.

Pinecone Monitoring

Pinecone’s Python client exposes `describe_index_stats()`, which gives you namespace-level vector counts and fullness. The `fullness` metric is the single most important operational signal for a pod-based Pinecone deployment: when it exceeds 0.90, query latency climbs steeply and inserts may fail.

```
# Python
# pinecone_metrics_collector.py
# Requires: pinecone-client>=3.0, prometheus_client

import os
import time
import schedule
from pinecone import Pinecone
from prometheus_client import (
    CollectorRegistry,
    Gauge,
    push_to_gateway,
)

PUSHGATEWAY_URL = "http://localhost:9091"
PINECONE_API_KEY = os.environ["PINECONE_API_KEY"]

pc = Pinecone(api_key=PINECONE_API_KEY)

registry = CollectorRegistry()

pinecone_vector_count = Gauge(
    "pinecone_namespace_vector_count",
    "Number of vectors in a Pinecone namespace",
    ["index_name", "namespace"],
    registry=registry,
)

pinecone_index_fullness = Gauge(
    "pinecone_index_fullness_ratio",
    "Index fullness ratio (0.0 to 1.0). Alert at > 0.85",
    ["index_name"],
    registry=registry,
)

pinecone_index_dimension = Gauge(
    "pinecone_index_dimension",
    "Vector dimension for a Pinecone index",
    ["index_name", "metric"],
    registry=registry,
)
```

```

pinecone_index_ready = Gauge(
    "pinecone_index_ready",
    "1 if the index state is Ready, 0 otherwise",
    ["index_name"],
    registry=registry,
)

def collect_pinecone_metrics():
    indexes = pc.list_indexes()

    for index_info in indexes:
        index_name = index_info.name
        idx = pc.Index(index_name)

        # Index status
        desc = pc.describe_index(index_name)
        is_ready = 1 if desc.status.ready else 0
        pinecone_index_ready.labels(index_name=index_name).set(is_ready)
        pinecone_index_dimension.labels(
            index_name=index_name,
            metric=desc.metric,
        ).set(desc.dimension)

        # Stats per namespace
        stats = idx.describe_index_stats()
        pinecone_index_fullness.labels(index_name=index_name).set(
            stats.index_fullness
        )

        for namespace, ns_stats in stats.namespaces.items():
            pinecone_vector_count.labels(
                index_name=index_name,
                namespace=namespace if namespace else "__default__",
            ).set(ns_stats.vector_count)

    push_to_gateway(
        PUSHGATEWAY_URL,
        job="pinecone_metrics",
        registry=registry,
    )
    print(f"[{time.strftime('%H:%M:%S')}] Pinecone metrics pushed")

schedule.every(30).seconds.do(collect_pinecone_metrics)

if __name__ == "__main__":
    collect_pinecone_metrics()
    while True:
        schedule.run_pending()
        time.sleep(1)

```

Weaviate Monitoring

Weaviate ships with a built-in Prometheus endpoint at `http://<host>:2112/metrics` when you set `PROMETHEUS_MONITORING_ENABLED=true` in the environment. The native metrics cover object count, index segment count, vector dimension, and batch import throughput. Beyond those, you want to query the `/v1/nodes` REST endpoint for cluster health.

```

# Python
# weaviate_health_probe.py
# Requires: weaviate-client>=4.0, requests, prometheus_client

import os
import time
import requests
import schedule

```

```

import weaviate
from weaviate.classes.init import Auth
from prometheus_client import (
    CollectorRegistry,
    Gauge,
    push_to_gateway,
)

PUSHGATEWAY_URL = "http://localhost:9091"
WEAVIATE_URL = os.environ.get("WEAVIATE_URL", "http://localhost:8080")
WEAVIATE_API_KEY = os.environ.get("WEAVIATE_API_KEY", "")

registry = CollectorRegistry()

weaviate_node_status = Gauge(
    "weaviate_node_healthy",
    "1 if the Weaviate node is healthy",
    ["node_name"],
    registry=registry,
)

weaviate_object_count = Gauge(
    "weaviate_collection_object_count",
    "Number of objects in a Weaviate collection",
    ["collection_name", "node_name"],
    registry=registry,
)

weaviate_shard_count = Gauge(
    "weaviate_collection_shard_count",
    "Number of shards for a Weaviate collection",
    ["collection_name"],
    registry=registry,
)

weaviate_vector_index_queue_length = Gauge(
    "weaviate_async_index_queue_length",
    "Number of objects waiting to be indexed asynchronously",
    ["collection_name", "shard_name"],
    registry=registry,
)

def collect_weaviate_metrics():
    # Use REST directly for /v1/nodes - gives richer cluster info than client
    headers = {}
    if WEAVIATE_API_KEY:
        headers["Authorization"] = f"Bearer {WEAVIATE_API_KEY}"

    try:
        resp = requests.get(
            f"{WEAVIATE_URL}/v1/nodes",
            headers=headers,
            params={"output": "verbose"},
            timeout=10,
        )
        resp.raise_for_status()
        nodes_data = resp.json()
    except Exception as e:
        print(f"Failed to reach Weaviate nodes endpoint: {e}")
        return

    collection_shards: dict[str, int] = {}
    collection_objects: dict[tuple, int] = {}

    for node in nodes_data.get("nodes", []):
        node_name = node.get("name", "unknown")
        node_status = node.get("status", "UNHEALTHY")
        weaviate_node_status.labels(node_name=node_name).set(
            1 if node_status == "HEALTHY" else 0

```



```
)

for shard in node.get("shards", []) or []:
    coll_name = shard.get("class", "unknown")
    shard_name = shard.get("name", "unknown")
    obj_count = shard.get("objectCount", 0)
    queue_len = shard.get("vectorIndexingQueue", 0)

    collection_shards[coll_name] = (
        collection_shards.get(coll_name, 0) + 1
    )
    key = (coll_name, node_name)
    collection_objects[key] = collection_objects.get(key, 0) + obj_count

    weaviate_vector_index_queue_length.labels(
        collection_name=coll_name,
        shard_name=shard_name,
    ).set(queue_len)

for (coll_name, node_name), count in collection_objects.items():
    weaviate_object_count.labels(
        collection_name=coll_name,
        node_name=node_name,
    ).set(count)

for coll_name, shard_count in collection_shards.items():
    weaviate_shard_count.labels(collection_name=coll_name).set(shard_count)

push_to_gateway(
    PUSHGATEWAY_URL,
    job="weaviate_metrics",
    registry=registry,
)

print(f"[{time.strftime('%H:%M:%S')}] Weaviate metrics pushed")

schedule.every(30).seconds.do(collect_weaviate_metrics)

if __name__ == "__main__":
    collect_weaviate_metrics()
    while True:
        schedule.run_pending()
        time.sleep(1)
```

The `weaviate_async_index_queue_length` metric is particularly important when using Weaviate's async indexing mode. A queue that grows monotonically without draining indicates that your import rate exceeds the indexing throughput—the vector equivalent of a replication lag that never catches up. Set an alert at queue depth $> 50,000$ for most workloads, and $> 10,000$ for latency-sensitive applications.

Embedding Quality and Recall Probes

Infrastructure metrics tell you whether your database is running. Quality probes tell you whether it is working. This distinction matters enormously for RAG applications because a system can show green on every infrastructure dashboard while silently returning wrong results.

The right way to implement quality probes is to maintain a **golden dataset**: a small set of queries (typically 50–200) for which you know the correct top-k results. These are generated once, stored in a separate table or file, and then replayed continuously. The probe computes **recall@k**—the fraction of expected results that appear in the actual results—and reports it as a metric.

```

# Python
# quality_probe.py
# Runs recall@10 probes against pgvector, Pinecone, and Weaviate
# Requires: psycpg2, pinecone-client>=3.0, weaviate-client>=4.0,
#           sentence-transformers, prometheus_client, numpy

import os
import json
import time
import numpy as np
import psycpg2
import psycpg2.extras
import schedule
import weaviate
import requests
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer
from prometheus_client import (
    CollectorRegistry,
    Gauge,
    Histogram,
    push_to_gateway,
)

PUSHGATEWAY_URL = "http://localhost:9091"
GOLDEN_DATASET_PATH = "golden_queries.json"
EMBED_MODEL = "all-MiniLM-L6-v2" # 384-dim, fast, good baseline
TOP_K = 10

# Load golden dataset once
# Format: [{"query": "...", "expected_ids": ["id1", "id2", ...], "collection": "docs"}]
with open(GOLDEN_DATASET_PATH) as f:
    golden_queries = json.load(f)

embedder = SentenceTransformer(EMBED_MODEL)

registry = CollectorRegistry()

recall_at_k = Gauge(
    "vector_search_recall_at_k",
    "Recall@K for a named golden query set",
    ["backend", "collection", "k"],
    registry=registry,
)

mean_similarity_score = Gauge(
    "vector_search_mean_top1_similarity",
    "Mean cosine similarity of the top-1 result across golden queries",
    ["backend", "collection"],
    registry=registry,
)

probe_latency_ms = Gauge(
    "vector_search_probe_latency_ms",
    "Mean query latency in ms during quality probe run",
    ["backend", "collection"],
    registry=registry,
)

# pgvector probe

def probe_pgvector():
    conn = psycpg2.connect(os.environ["PG_DSN"])
    cur = conn.cursor(cursor_factory=psycpg2.extras.DictCursor)

    # Group golden queries by collection (maps to table name in pgvector)
    from itertools import groupby
    from operator import itemgetter

```

```

grouped = {}
for q in golden_queries:
    if q.get("backend", "pgvector") != "pgvector":
        continue
    coll = q["collection"]
    grouped.setdefault(coll, []).append(q)

for collection, queries in grouped.items():
    recalls = []
    similarities = []
    latencies = []

    for gq in queries:
        query_vec = embedder.encode(gq["query"]).tolist()
        expected = set(gq["expected_ids"])

        t0 = time.perf_counter()
        cur.execute(
            f"""
            SELECT id, 1 - (embedding <=> %s::vector) AS cosine_sim
            FROM {collection}
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """,
            (query_vec, query_vec, TOP_K),
        )
        rows = cur.fetchall()
        elapsed_ms = (time.perf_counter() - t0) * 1000

        returned_ids = {str(r["id"]) for r in rows}
        hit_count = len(expected & returned_ids)
        recall = hit_count / len(expected) if expected else 0.0
        recalls.append(recall)

        if rows:
            similarities.append(rows[0]["cosine_sim"])
            latencies.append(elapsed_ms)

    recall_at_k.labels(
        backend="pgvector",
        collection=collection,
        k=str(TOP_K),
    ).set(np.mean(recalls))

    mean_similarity_score.labels(
        backend="pgvector",
        collection=collection,
    ).set(np.mean(similarities) if similarities else 0.0)

    probe_latency_ms.labels(
        backend="pgvector",
        collection=collection,
    ).set(np.mean(latencies))

cur.close()
conn.close()

# Pinecone probe

def probe_pinecone():
    pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])

    grouped = {}
    for q in golden_queries:
        if q.get("backend", "pinecone") != "pinecone":
            continue
        coll = q["collection"]
        grouped.setdefault(coll, []).append(q)

```

```
for index_name, queries in grouped.items():
    idx = pc.Index(index_name)
    recalls = []
    similarities = []
    latencies = []

    for gq in queries:
        query_vec = embedder.encode(gq["query"]).tolist()
        expected = set(gq["expected_ids"])
        namespace = gq.get("namespace", "")

        t0 = time.perf_counter()
        result = idx.query(
            vector=query_vec,
            top_k=TOP_K,
            namespace=namespace,
            include_metadata=False,
        )
        elapsed_ms = (time.perf_counter() - t0) * 1000

        returned_ids = {m.id for m in result.matches}
        hit_count = len(expected & returned_ids)
        recall = hit_count / len(expected) if expected else 0.0
        recalls.append(recall)

        if result.matches:
            similarities.append(result.matches[0].score)
            latencies.append(elapsed_ms)

    recall_at_k.labels(
        backend="pinecone",
        collection=index_name,
        k=str(TOP_K),
    ).set(np.mean(recalls))

    mean_similarity_score.labels(
        backend="pinecone",
        collection=index_name,
    ).set(np.mean(similarities) if similarities else 0.0)

    probe_latency_ms.labels(
        backend="pinecone",
        collection=index_name,
    ).set(np.mean(latencies))

# Weaviate probe
def probe_weaviate():
    weaviate_url = os.environ.get("WEAVIATE_URL", "http://localhost:8080")
    weaviate_api_key = os.environ.get("WEAVIATE_API_KEY", "")
    headers = {}
    if weaviate_api_key:
        headers["Authorization"] = f"Bearer {weaviate_api_key}"

    grouped = {}
    for q in golden_queries:
        if q.get("backend", "weaviate") != "weaviate":
            continue
        coll = q["collection"]
        grouped.setdefault(coll, []).append(q)

    for collection_name, queries in grouped.items():
        recalls = []
        similarities = []
        latencies = []

        for gq in queries:
            query_vec = embedder.encode(gq["query"]).tolist()
            expected = set(gq["expected_ids"])
```

```

# Use Weaviate's GraphQL nearVector API
graphql_query = {
    "query": f"""
    {{
        Get {{
            {{collection_name}}(
                nearVector: {{
                    vector: {json.dumps(query_vec)}
                    certainty: 0.0
                }}
            limit: {TOP_K}
        }} {{
            _additional {{
                id
                certainty
            }}
        }}
    }}
    """
}

t0 = time.perf_counter()
resp = requests.post(
    f"{weaviate_url}/v1/graphql",
    json=graphql_query,
    headers=headers,
    timeout=15,
)
elapsed_ms = (time.perf_counter() - t0) * 1000
resp.raise_for_status()
data = resp.json()

objects = (
    data.get("data", {})
    .get("Get", {})
    .get(collection_name, []) or []
)
returned_ids = {
    obj["_additional"]["id"]
    for obj in objects
    if obj.get("_additional", {}).get("id")
}
hit_count = len(expected & returned_ids)
recall = hit_count / len(expected) if expected else 0.0
recalls.append(recall)

if objects:
    top_certainty = objects[0].get("_additional", {}).get("certainty", 0.0)
    similarities.append(top_certainty)
    latencies.append(elapsed_ms)

recall_at_k.labels(
    backend="weaviate",
    collection=collection_name,
    k=str(TOP_K),
).set(np.mean(recalls))

mean_similarity_score.labels(
    backend="weaviate",
    collection=collection_name,
).set(np.mean(similarities) if similarities else 0.0)

probe_latency_ms.labels(
    backend="weaviate",
    collection=collection_name,
).set(np.mean(latencies))

```

```
# Orchestration

def run_all_probes():
    try:
        probe_pgvector()
    except Exception as e:
        print(f"pgvector probe failed: {e}")

    try:
        probe_pinecone()
    except Exception as e:
        print(f"Pinecone probe failed: {e}")

    try:
        probe_weaviate()
    except Exception as e:
        print(f"Weaviate probe failed: {e}")

    push_to_gateway(
        PUSHGATEWAY_URL,
        job="vector_quality_probes",
        registry=registry,
    )
    print(f"[{time.strftime('%H:%M:%S')}] Quality probe metrics pushed")

schedule.every(5).minutes.do(run_all_probes)

if __name__ == "__main__":
    run_all_probes()
    while True:
        schedule.run_pending()
        time.sleep(1)
```

Building the Golden Dataset

The quality probe is only as good as the dataset it runs against. Building that dataset correctly is the step most teams skip, and skipping it is why they discover recall regressions from user complaints rather than from dashboards.

The golden dataset should satisfy three properties:

Coverage: Queries should represent the real distribution of production queries. If your RAG application handles product support questions, customer onboarding topics, and billing inquiries in roughly equal measure, your golden set should reflect those proportions. Capture 30 days of production query logs, cluster them by topic using k-means on their embeddings, then sample representative queries from each cluster.

Stability: The expected results must be verified by a human or by a higher-quality reference retrieval system, not by your production system at a point in time. If you generate ground truth by querying your own system and storing what it returns, you are measuring consistency, not correctness. Run your golden queries against a re-ranked or BM25+vector hybrid result set, review the top-10 by hand for a sample, and store verified IDs.

Versioning: When you upgrade your embedding model, your golden dataset expected IDs become invalid because the same document will have a different ID if it was re-embedded and re-inserted, or a different vector if the document ID is stable but the model changed. Tag each golden query with the embedding model version that was used to generate both the query embedding and the

document embeddings. When you rotate models, regenerate the golden set for the new model in parallel before cutting over.

The following SQL creates the golden dataset table in PostgreSQL:

```
-- PostgreSQL
-- Schema for the golden query dataset

CREATE TABLE monitoring.golden_queries (
  id SERIAL PRIMARY KEY,
  query_text TEXT NOT NULL,
  backend TEXT NOT NULL CHECK (backend IN ('pgvector', 'pinecone', 'weaviate')),
  collection_name TEXT NOT NULL,
  namespace TEXT,
  embedding_model TEXT NOT NULL,
  expected_ids TEXT[] NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  retired_at TIMESTAMPTZ,
  notes TEXT
);

CREATE INDEX ON monitoring.golden_queries (backend, collection_name)
WHERE retired_at IS NULL;

COMMENT ON TABLE monitoring.golden_queries IS
'Ground-truth query/result pairs for continuous recall monitoring. '
'Retire rows (set retired_at) when embedding model changes; do not delete.';

-- SQL Server
-- Schema for the golden query dataset

CREATE TABLE monitoring.golden_queries (
  id INT IDENTITY(1,1) PRIMARY KEY,
  query_text NVARCHAR(MAX) NOT NULL,
  backend NVARCHAR(20) NOT NULL
    CHECK (backend IN ('pgvector', 'pinecone', 'weaviate')),
  collection_name NVARCHAR(256) NOT NULL,
  namespace NVARCHAR(256) NULL,
  embedding_model NVARCHAR(128) NOT NULL,
  -- SQL Server has no native array type; store as JSON
  expected_ids NVARCHAR(MAX) NOT NULL
    CHECK (ISJSON(expected_ids) = 1),
  created_at DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
  retired_at DATETIMEOFFSET NULL,
  notes NVARCHAR(MAX) NULL
);

CREATE INDEX IX_golden_queries_backend_collection
ON monitoring.golden_queries (backend, collection_name)
WHERE retired_at IS NULL;
```

Note that SQL Server does not have a native array column type, so `expected_ids` is stored as a JSON string (e.g., `'["id1","id2"]'`) and the `ISJSON` check constraint validates the format. Your probe code reads from this table and deserializes accordingly.

Alerting Rules and Escalation Thresholds

Collecting metrics is only half the problem. The other half is deciding when to page someone versus when to log a warning and move on. The thresholds below are starting points calibrated for a production RAG application with moderate query volume; you will tune them based on your baseline.

Recall degradation alert: Fire a warning when `vector_search_recall_at_k` (for any backend or collection) drops below 0.80 for two consecutive probe cycles. Fire a critical alert when it drops below 0.65. A drop from your established baseline of, say, 0.92 to 0.85 warrants investigation; a drop to below 0.65 typically indicates a model mismatch or index corruption that affects users now.

Index fullness alert (Pinecone): Warning at 0.80 fullness; critical at 0.90. Above 0.90, Pinecone pod-based indexes begin rejecting upserts silently in some configurations. Scale your pods before hitting this threshold.

Async indexing queue alert (Weaviate): Warning when `weaviate_async_index_queue_length` exceeds 25,000 for more than 10 minutes. Critical when it exceeds 100,000. At high queue depths, newly inserted objects are searchable via object lookup but not yet via vector similarity—users may see recently-uploaded documents missing from search results.

ANN query latency alert (pgvector): Warning when `pgvector_ann_query_mean_latency_ms` exceeds 150ms; critical at 500ms. The primary causes are: (1) HNSW `ef_search` set too high for the current load, (2) `shared_buffers` too small to keep the HNSW graph in memory, or (3) concurrent index builds competing for I/O.

Similarity score drift: Track the 7-day rolling mean of `vector_search_mean_top1_similarity`. If it drops more than 0.08 below the 30-day baseline, emit a warning. Gradual similarity score decline with no change in recall usually means your corpus is becoming more heterogeneous—new document categories whose embeddings populate different regions of the vector space, pulling average top-1 similarity down without reducing accuracy. Sudden drops in both recall and similarity together almost always indicate an embedding model change was applied to queries but not to stored documents.

The Prometheus alerting rules YAML for the recall and latency alerts:

```
# YAML
# prometheus_vector_alerts.yaml
# Place in your Prometheus rules directory and reference in prometheus.yml

groups:
- name: vector_database_alerts
  interval: 60s
  rules:

  - alert: VectorRecallDegradedWarning
    expr: vector_search_recall_at_k < 0.80
    for: 10m
    labels:
      severity: warning
    annotations:
      summary: "Recall@{{ $labels.k }} below 0.80 on {{ $labels.backend }}/{{ $labels.collection }}"
      description: >
        The quality probe reports recall@{{ $labels.k }} = {{ $value | printf "%.3f" }}
        for backend={{ $labels.backend }}, collection={{ $labels.collection }}.
        This may indicate index fragmentation, model drift, or data pipeline issues.

  - alert: VectorRecallDegradedCritical
    expr: vector_search_recall_at_k < 0.65
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "CRITICAL: Recall@{{ $labels.k }} below 0.65 on {{ $labels.backend }}/{{
↵ $labels.collection }}"
      description: >
        Search quality has degraded severely. Immediate investigation required.
        Current recall = {{ $value | printf "%.3f" }}.
```



```
- alert: PineconeIndexFullnessWarning
  expr: pinecone_index_fullness_ratio > 0.80
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "Pinecone index {{ $labels.index_name }} fullness > 80%"
    description: >
      Index fullness = {{ $value | printf "%.2f" }}. Plan pod scaling before reaching 90%.

- alert: PineconeIndexFullnessCritical
  expr: pinecone_index_fullness_ratio > 0.90
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "CRITICAL: Pinecone index {{ $labels.index_name }} fullness > 90%"
    description: >
      Upserts may fail. Scale pods immediately.

- alert: WeaviateIndexQueueHigh
  expr: weaviate_async_index_queue_length > 25000
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Weaviate async index queue depth > 25k on {{ $labels.collection_name }}"
    description: >
      Queue = {{ $value }} objects. Recently inserted documents may not be searchable.

- alert: WeaviateIndexQueueCritical
  expr: weaviate_async_index_queue_length > 100000
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "CRITICAL: Weaviate async index queue depth > 100k"
    description: >
      Import rate is outpacing indexing throughput. Reduce import concurrency
      or add Weaviate replica nodes.

- alert: PgvectorANNLatencyWarning
  expr: pgvector_ann_query_mean_latency_ms > 150
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "pgvector ANN query mean latency > 150ms"
    description: >
      Query pattern: {{ $labels.query_pattern }}.
      Mean latency = {{ $value | printf "%.1f" }}ms.
      Check ef_search setting, shared_buffers, and concurrent index builds.

- alert: PgvectorANNLatencyCritical
  expr: pgvector_ann_query_mean_latency_ms > 500
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "CRITICAL: pgvector ANN query latency > 500ms"
    description: >
      Vector search is severely degraded. Check for concurrent REINDEX operations
      or shared_buffers exhaustion.
```

Grafana Dashboard Layout

With all metrics flowing into Prometheus, the Grafana dashboard organizes into four rows. Here is the recommended panel layout with the PromQL for each panel:

Row 1 — Health Overview (Status panels)

Panel	PromQL	Visualization
pgvector ANN Latency (p-mean)	<code>pgvector_ann_query_mean_latency_ms</code>	Stat (threshold: green < 100ms, yellow < 200ms, red > 200ms)
Pinecone Index Fullness	<code>pinecone_index_fullness_ratio</code>	Gauge (0–1, threshold at 0.80 and 0.90)
Weaviate Node Health	<code>weaviate_node_healthy</code>	Stat (1=green, 0=red)
Overall Recall@10	<code>min(vector_search_recall_at_k{at=10})</code>	Stat (threshold: green > 0.85, yellow > 0.70, red < 0.70)

Row 2 — Vector Index Operational Metrics (Time series)

Panel	PromQL	Notes
pgvector Index Size Growth	<code>pgvector_index_size_bytes</code>	One series per index; watch for sudden jumps indicating failed vacuums
pgvector Live Row Counts	<code>pgvector_table_live_rows</code>	Divergence from expected growth rate signals pipeline stalls
Pinecone Vector Count by Namespace	<code>pinecone_namespace_vector_count</code>	Flat lines indicate ingestion stopped
Weaviate Async Queue Depth	<code>weaviate_async_index_queue_length</code>	Should drain between import batches

Row 3 — Query Latency Breakdown (Time series + heatmap)

Panel	PromQL	Notes
pgvector Top-5 Slowest Patterns	<code>topk(5, pgvector_ann_query_mean_latency_ms)</code>	Table panel, sorted descending
Pinecone Query Probe Latency	<code>probe_latency_ms{backend="pinecone"}</code>	Reliability probe; measures round-trip including network
Weaviate Query Probe Latency	<code>probe_latency_ms{backend="weaviate"}</code>	Compare against Weaviate's native /metrics histogram

Row 4 — Recall and Similarity Quality (Time series)

Panel	PromQL	Notes
Recall@10 by Backend	<code>vector_search_recall_at_k{k="10"} -> 0</code>	One day all backends; regression shows immediately
Top-1 Similarity Score	<code>vector_search_mean_top1_similarity</code>	Downward trend without recall drop = corpus drift
Recall@10 7-Day Delta	<code>vector_search_recall_at_k{k="10"} - vector_search_recall_at_k{k="10"} offset 7d</code>	Negative values indicate week-over-week degradation

The JSON export for this Grafana dashboard would be several hundred lines; rather than reproducing the full JSON here, the pattern above is sufficient to construct it using Grafana’s dashboard builder, or you can use the `grafana_dashboard` Terraform resource if your infrastructure is managed with Terraform.

Handling Embedding Model Rotation

One operational scenario that defeats most monitoring setups is an embedding model rotation: the moment when your team switches from one model (say, `text-embedding-ada-002` at 1536 dimensions) to another (say, a fine-tuned 768-dimension model). The transition period creates a mixed-state problem where some documents use the old embeddings and some use the new, and queries always use one or the other, never both.

Your monitoring dashboard needs to detect this state explicitly. The most reliable signal is a sudden change in `vector_search_mean_top1_similarity` combined with a drop in `vector_search_recall_at_k`. Similarity alone is ambiguous—a new model in a new vector space has different absolute similarity scales. Recall dropping while the system appears operationally healthy is the diagnostic signature of a model mismatch.

The operational procedure for a safe model rotation:

1. **Create a new collection/index** alongside the existing one. Never overwrite in-place.
2. **Run dual-write**: new documents are embedded with both models and inserted into both collections.
3. **Re-embed the corpus** in the new collection using a background job. Track progress with a dedicated Prometheus gauge (`embedding_backfill_progress_ratio`, `embedding_backfill_rows_complete`, `embedding_backfill_rows_total`).
4. **Promote the new collection** as the read target only when `embedding_backfill_progress_ratio` ≥ 1.0 AND `vector_search_recall_at_k` (measured against a golden set generated with the new model) exceeds your baseline threshold.
5. **Retire the old collection** after a soak period of at least 72 hours with no recall alerts.

```
# Python
# backfill_progress_tracker.py
# Reports embedding backfill progress as a Prometheus metric

import os
import time
import psycpg2
```

```

import schedule
from prometheus_client import CollectorRegistry, Gauge, push_to_gateway

PUSHGATEWAY_URL = "http://localhost:9091"
registry = CollectorRegistry()

backfill_progress = Gauge(
    "embedding_backfill_progress_ratio",
    "Fraction of rows re-embedded for a collection (0.0 to 1.0)",
    ["collection_name", "new_model"],
    registry=registry,
)

backfill_rows_complete = Gauge(
    "embedding_backfill_rows_complete",
    "Count of rows that have been re-embedded",
    ["collection_name", "new_model"],
    registry=registry,
)

backfill_rows_total = Gauge(
    "embedding_backfill_rows_total",
    "Total rows to re-embed for the collection",
    ["collection_name", "new_model"],
    registry=registry,
)

def collect_backfill_metrics():
    conn = psycopg2.connect(os.environ["PG_DSN"])
    cur = conn.cursor()

    # This query assumes a backfill_status table that your backfill job maintains
    cur.execute("""
        SELECT
            collection_name,
            new_model,
            rows_complete,
            rows_total,
            CASE WHEN rows_total > 0
                THEN rows_complete::float / rows_total
                ELSE 0
            END AS progress_ratio
        FROM
            monitoring.embedding_backfill_status
        WHERE
            completed_at IS NULL;
    """)
    for row in cur.fetchall():
        coll, model, done, total, ratio = row
        backfill_progress.labels(collection_name=coll, new_model=model).set(ratio)
        backfill_rows_complete.labels(collection_name=coll, new_model=model).set(done)
        backfill_rows_total.labels(collection_name=coll, new_model=model).set(total)

    cur.close()
    conn.close()

    push_to_gateway(PUSHGATEWAY_URL, job="backfill_progress", registry=registry)

schedule.every(30).seconds.do(collect_backfill_metrics)

if __name__ == "__main__":
    collect_backfill_metrics()
    while True:
        schedule.run_pending()
        time.sleep(1)

```

The supporting table:

```
-- PostgreSQL
CREATE TABLE monitoring.embedding_backfill_status (
  id SERIAL PRIMARY KEY,
  collection_name TEXT NOT NULL,
  new_model TEXT NOT NULL,
  rows_total BIGINT NOT NULL DEFAULT 0,
  rows_complete BIGINT NOT NULL DEFAULT 0,
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ,
  notes TEXT
);

-- SQL Server
CREATE TABLE monitoring.embedding_backfill_status (
  id INT IDENTITY(1,1) PRIMARY KEY,
  collection_name NVARCHAR(256) NOT NULL,
  new_model NVARCHAR(128) NOT NULL,
  rows_total BIGINT NOT NULL DEFAULT 0,
  rows_complete BIGINT NOT NULL DEFAULT 0,
  started_at DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
  completed_at DATETIMEOFFSET NULL,
  notes NVARCHAR(MAX) NULL
);
```

Putting It All Together: The Monitoring Stack Deployment

Running three separate metrics collector scripts alongside your application is manageable, but they need to be deployed reliably. The cleanest approach is to containerize each collector and deploy them as sidecar containers or separate services in the same Kubernetes namespace as your application.

A minimal `docker-compose.yml` that wires together Prometheus, the pushgateway, Grafana, and all three collectors:

```
# YAML
# docker-compose.yml
# Vector database monitoring stack

version: "3.9"

services:

  prometheus:
    image: prom/prometheus:v2.51.0
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus_vector_alerts.yml:/etc/prometheus/rules/vector_alerts.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.retention.time=30d"
      - "--web.enable-lifecycle"
    ports:
      - "9090:9090"
    restart: unless-stopped

  pushgateway:
    image: prom/pushgateway:v1.8.0
    ports:
      - "9091:9091"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:10.4.0
```

```
environment:
  GF_SECURITY_ADMIN_PASSWORD: "${GRAFANA_ADMIN_PASSWORD}"
  GF_USERS_ALLOW_SIGN_UP: "false"
volumes:
  - grafana_data:/var/lib/grafana
ports:
  - "3000:3000"
depends_on:
  - prometheus
restart: unless-stopped

pgvector_collector:
  build:
    context: .
    dockerfile: Dockerfile.collector
  command: python pgvector_metrics_collector.py
  environment:
    PG_DSN: "${PG_DSN}"
  depends_on:
    - pushgateway
  restart: unless-stopped

pinecone_collector:
  build:
    context: .
    dockerfile: Dockerfile.collector
  command: python pinecone_metrics_collector.py
  environment:
    PINECONE_API_KEY: "${PINECONE_API_KEY}"
  depends_on:
    - pushgateway
  restart: unless-stopped

weaviate_health_probe:
  build:
    context: .
    dockerfile: Dockerfile.collector
  command: python weaviate_health_probe.py
  environment:
    WEAVIATE_URL: "${WEAVIATE_URL}"
    WEAVIATE_API_KEY: "${WEAVIATE_API_KEY}"
  depends_on:
    - pushgateway
  restart: unless-stopped

quality_probe:
  build:
    context: .
    dockerfile: Dockerfile.collector
  command: python quality_probe.py
  environment:
    PG_DSN: "${PG_DSN}"
    PINECONE_API_KEY: "${PINECONE_API_KEY}"
    WEAVIATE_URL: "${WEAVIATE_URL}"
    WEAVIATE_API_KEY: "${WEAVIATE_API_KEY}"
  volumes:
    - ./golden_queries.json:/app/golden_queries.json:ro
  depends_on:
    - pushgateway
  restart: unless-stopped

backfill_tracker:
  build:
    context: .
    dockerfile: Dockerfile.collector
  command: python backfill_progress_tracker.py
  environment:
    PG_DSN: "${PG_DSN}"
  depends_on:
    - pushgateway
```

```
restart: unless-stopped

volumes:
  prometheus_data:
  grafana_data:
```

The `prometheus.yml` to scrape both the pushgateway (for pushed metrics from all collectors) and Weaviate’s native endpoint:

```
# YAML
# prometheus.yml

global:
  scrape_interval: 30s
  evaluation_interval: 60s

rule_files:
  - /etc/prometheus/rules/*.yaml

scrape_configs:
  - job_name: pushgateway
    honor_labels: true
    static_configs:
      - targets: ["pushgateway:9091"]

  - job_name: weaviate_native
    # Weaviate's built-in Prometheus endpoint
    # Requires PROMETHEUS_MONITORING_ENABLED=true in Weaviate config
    static_configs:
      - targets: ["weaviate:2112"]
    metrics_path: /metrics

  - job_name: prometheus_self
    static_configs:
      - targets: ["localhost:9090"]
```

Key Takeaways

- **Three-layer monitoring is non-negotiable for vector workloads.** Infrastructure metrics alone will not show you recall degradation or embedding drift. You need Layer 1 (infrastructure), Layer 2 (vector index operations), and Layer 3 (embedding quality probes) running simultaneously before problems become user-visible.
- **The golden dataset is your most valuable monitoring asset.** A set of 50–200 human-verified query/result pairs, tagged by embedding model version, lets you compute recall@k continuously. Build it early, version it carefully, and regenerate it whenever you rotate embedding models.
- **Pinecone fullness, Weaviate async queue depth, and pgvector HNSW latency are the three backend-specific metrics that most directly predict user-facing degradation.** Alert on them with two-tier thresholds (warning and critical) so on-call engineers can act before users are affected.
- **Embedding model rotation is the highest-risk operational event in a vector database’s lifecycle.** A mixed-state corpus—some documents on the old model, some on the new—will degrade recall quietly and systematically. The safe pattern is: create a parallel collection, dual-write, backfill, measure recall against a new-model golden set, then promote.

Never overwrite in-place.

- **Continuous quality probes make the invisible visible.** Cosine similarity score drift and recall degradation are silent failures. A probe that runs every five minutes against your golden dataset and pushes recall and similarity metrics to Prometheus gives you a continuous signal that your search system is doing what it was built to do—something no infrastructure metric can tell you.